

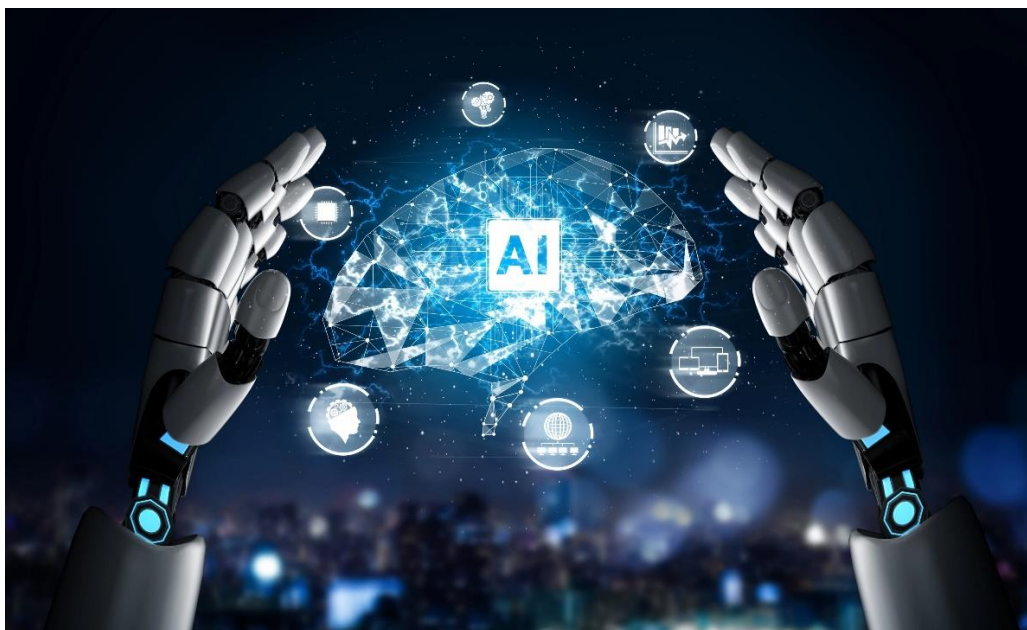
**O‘ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKASIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN’IY INTELLEKT”
KAFEDRASI**



“SUN’IY INTELLEKT ASOSLARI” FANIDAN

O‘QUV-USLUBIY MAJMUUA



Toshkent 2025

“TASDIQLAYMAN”

O‘R HX va MU AXBOROT-KOMMUNIKATSIYA
TEXNOLOGIYALARI VA HARBIY ALOQA INSTITUTI
BOSHLIG‘INING BIRINCHI O‘RINBOSARI
(O‘QUV VA ILMIY ISHLAR BO‘YICHA)

kapitan

B. To‘rayev

2025-yil “___” _____

“SUN'YIY INTELLEKT ASOSLARI” FANIDAN

O‘QUV-USLUBIY MAJMUA

“KELISHILGAN”

AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI “AXBOROT TEXNOLOGIYALARI VA SUN'YIY
INTELLEKT” KAFEDRASI BOSHLIG‘I

kapitan

B. Yusupov

2025-yil “___” _____

AKT VA HAI “AXBOROT TEXNOLOGIYALARI VA SUN'YIY
INTELLEKT” KAFEDRASI “SUN'YIY INTELLEKT” SIKLI BOSHLIG‘I
kapitan

M. Isakov

2025-yil “___” _____

Tuzuvchilar:

- | | |
|----------------------------------|---|
| kapitan
B.K. Yusupov | – PhD, professor, Axborot-kommunikatsiya texnologiyalari va harbiy aloqa instituti, axborot texnologiyalari va sun'iy intellekt kafedrası boshlig'i, PhD, professori; |
| kapitan
M. Isakov | – Axborot-kommunikatsiya texnologiyalari va harbiy aloqa instituti, axborot texnologiyalari va sun'iy intellekt kafedrası sun'iy intellekt sikli boshlig'i. |
| katta leytenant
R. Qodomboyev | – Axborot-kommunikatsiya texnologiyalari va harbiy aloqa instituti, axborot texnologiyalari va sun'iy intellekt kafedrası sun'iy intellekt sikli katta o'qituvchisi. |

Taqrizchilar:

- | | |
|--------------------------------|--|
| podpolkovnik
A. Mulladjanov | – O'R QK Bosh shtabi AAT va AH BB Axborot-kommunikatsiya texnologiyalarini rivojlantirish boshqarmasi boshlig'i; |
| podpolkovnik
O. Abdiroziqov | – O'R HX va MU "Istiqbolli harbiy texnologiyalar" kafedrası boshlig'i, PhD, dotsent. |

O'quv-uslubiy majmua Axborot-kommunikatsiya texnologiyalari va harbiy aloqa instituti "Axborot texnologiyalari va sun'iy intellekt" kafedrasining 2025-yil 15-avgust kunidagi majlisida ko'rib chiqilingan va 2-sonli bayonnomasi bilan o'quv jarayoniga joriy etishga ruxsat berilgan.

№	M U N D A R I J A	
1.	Kirish.....	6
2.	Fan bo'yicha o'qitiladigan materiallar to'plami.....	9
3.	Fan bo'yicha mashg'ulotlar o'tish rejalari.....	309
4.	Glossariy.....	373
5.	Tarqatma materiallar to'plami (1-ilova).....	381
6.	Fan bo'yicha yakuniy nazorat qabul qilish uchun uslubiy ishlanma (2-ilova).....	399
8.	Fanni o'qitishda foydalaniladigan interfaol ta'lim metodlari (3-ilova).	405
9.	Fanning o'quv dasturi.....	415
10.	Fanning ishchi o'quv dasturi.....	421
11.	Asosiy va qo'shimcha adabiyotlar va axborot manbalari (4-ilova).....	443

KIRISH

Ushbu o'quv-uslubiy majmuada "Sun'iy intellekt asoslari" faniga tegishli bo'lgan dasturlash qismiga tegishli bo'lgan barcha mavzular bo'yicha kursantlarga Davlat ta'lim standartlari asosida yetkazilishi shart bo'lgan minimum bilimlar va ko'nikmalarni to'la qamrab olingan.

Fanning asosiy maqsadi - yuqori malakali aloqa qo'shinlari ofitserlarini tayyorlashda axborot-kommunikatsiya texnologiyalari vositalarining dasturiy ta'minoti tuzilishi va ishlash jarayonini hamda yaratilish bosqichlarini nazariy va amaliy jihatdan o'rgatishdan iborat.

Fanni o'zlashtirish kursantlarning "Informatika", "Kompyuter tizimlarini ekspluatatsiya qilish", "Dasturlash texnologiyalari" fanlaridan olgan bilimlariga tayanadi. Fanni o'zlashtirish quyidagi mashg'ulot turlarini o'z ichiga oladi: ma'ruza, seminar, guruhli va amaliy mashg'ulotlar, shuningdek kursantlarga mustaqil tayyorgarlik vaqtida maslahatlar berish. Ma'ruza materiallari bayoni mustaqil va tugallangan hususiyatga ega bo'lib, avval bayon qilingan materiallarga mantiqiy bog'langan hamda boshqa fanlarda, hamda amaliyotda qo'llanishga yo'naltirilgan bo'lishi kerak. Amaliy mashg'ulotlarda kursantlar olgan nazariy bilimlarini qo'llay olishni o'rganishlari kerak.

Fanning maqsad va vazifalari

Fanni o'qitishdan maqsad – harbiy dasturiy mahsulotlarni ishlab chiqish usullarini, loyihalash usullarini, dasturiy ta'minotni ishlab chiqishni, dasturiy mahsulotning effektivligini, yo'nalishning profiliga mos bilim, ko'nikma va malakani shakllantirishdan iborat.

Fanning vazifasi – harbiy dasturiy mahsulotlarni yaratish bosqichlari, tamoyillari, testlash usullari, arxitekturasini va loyihalash usullaridan foydalanishni o'rgatishdan iborat.

Fan bo'yicha kursantlarning bilimiga, ko'nikma va malakasiga qo'yiladigan talablar:

"Harbiy maqsadlarga yo'naltirilgan dasturlash" o'quv fanini uzlashtirish jarayonida amalga oshiriladigan masalalar doirasida kursantlar:

- shaxsiy kompyuterlardan foydalanishni; dasturiy mahsulotlarni yaratishning eng zamonaviy dasturlash texnologiyalari to'g'risida tasavvurga ega bo'lishi;
- masalalarni algoritmlash va modellash usullarini, optimal dasturlash tilini tanlashni; tayyor dasturiy mahsulotlardan foydalanish, testdan o'tkazish hamda ularga xizmat ko'rsatish usullarini bilishi va ishlata olishni, dasturlarni funksional xarakteristikalariga talablar qo'yishni;
- programmani ishlab chiqish bosqichlarini aniqlashni; algoritmlarni yozish usullarini bilishi kerak.
- dastur algoritmini, blok-sxemasini ishlab chiqish, tanlab olingan algoritmgacha test misollarini ishlab chiqish, Dasturdagi xatoliklar topish va tuzatish ko'nikmalariga ega bo'lishi kerak.

Fan bo'yicha kursantlarning tasavvur, bilim, ko'nikma va malakalariga qo'yiladigan talablar

Fanni o'rganish natijasida kursantlar quyidagi tushunchalarga ega bo'lishi lozim:

O'zbekiston Respublikasi Qurolli Kuchlari aloqa tizimida dasturlash asoslarining o'rni va ro'li haqida;

- dasturlash asoslarida ishlanayotgan harbiy dasturlarni hamda ularni qo'llay olish tadbirlarini;

- dasturlash asoslari algoritmlarini.

quyidagilarni bilishi va qo'llay olishi lozim:

- Python C, C++, C# dasturlash tillarini;

- kompyuter tizimlarini muhofazasini ta'minlashda qo'llaniladigan dasturiy vositalarni;

- O'zbekiston Respublikasi Mudofaa Vazirligi qo'shinlari axborot tizimlarida dasturiy ta'minotni ta'minlash bo'yicha me'yoriy hujjatlarni.

quyidagi ko'nikmalarga ega bo'lishi lozim:

- dasturlash tillarida ishlanayotgan dasturlarga kirish huquqlarini chegaralash tizimini yaratish bo'yicha;

- ma'lumotlarni muhofazalashning zamonaviy dasturiy vositalarini sozlash bo'yicha bilim va ko'nikmalarga ega bo'lishi kerak.

Shuning uchun harbiy sohada dasturiy ta'minotni ta'minlash uchun dasturlash asoslarini yaxshi bilish, loyihalash va takomillashtirish talab qilinadi.

Fanni o'qitishda zamonaviy axborot va pedagogik texnologiyalar

O'quv jarayoni bilan bog'liq ta'lim sifatini belgilovchi holatlar quyidagilar: yuqori ilmiy-pedagogik darajada dars berish, muammoli ma'ruzalar o'qish, darslarni savol-javob tarzida qiziqarli tashkil qilish, ilg'or pedagogik texnologiyalardan va mul'timedia vositalaridan foydalanish, kursantlarni undaydigan, o'ylantiradigan muammolarni, ular oldiga qo'yish, talabchanlik, kursantlar bilan individual ishlash, erkin muloqot yuritishga, ilmiy izlanishga jalb qilish. Kursantlarning "Sun'iy intellekt asoslari" fanini o'zlashtirishlari uchun o'qitishning zamonaviy va ilg'or usullaridan foydalanish, yangi axborot-pedagogik texnologiyalarini tadbiq qilish muhim ahamiyatga egadir. Fanni o'zlashtirishda o'quv qo'llanmalar, ma'ruza matnlari, tarqatma materiallar, elektron materiallar, virtual stendlardan foydalaniladi. Ma'ruza, guruhli va amaliy mashg'ulotlarida mos ravishdagi zamonaviy pedagogik va axborot texnologiyalaridan foydalaniladi.

Mashg'ulotlarni asosiy qismini guruh va amaliy mashg'ulotlardan iborat bo'lib, ularda o'quv elementlarni mazmunlari va mavzularni bir-birlariga mantiqiy bog'liqligi o'rganiladi. Mashg'ulotlarda kursantlar har bir element va unga qo'yilgan o'quv savollari bo'yicha mustaqil o'rganib, o'qituvchi yordamida bilim va ko'nikmalarini takomillashtirishlari lozim. Mashg'ulotlarni asosiy turlari zamonaviy pedagogik va innovatsion texnologiya usullari va interaktiv shakllarini qo'llagan holda olib borilishi va unda kursantlarni faol qatnashuvi fanni o'rganishning asosiy omillaridan biri hisoblanadi.

**O'ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN'IY INTELLEKT”
KAFEDRASI**



“SUN'IY INTELLEKT ASOSLARI”

**FANI BO'YICHA O'QITILADIGAN
MATERIALLAR TO'PLAMI**

MA'RUZA MASHG'ULOTI UCHUN O'QUV MATERIALLARI

1-mavzu: "Sun'iy intellekt asoslari" faniga kirish va asosiy tushunchalari.

1-mashg'ulot. Sun'iy intellekt asoslarining klassifikatsiyasi va rivojlanish tarixi. Sun'iy intellekt asoslarining asosiy tushunchalari.

O'quv savollari:

1. Fanning mazmuni, maqsadi, vazifalari;
2. Pythonni o'rnatish. PyCharm dasturini o'rnatish;
3. Pythonda "Hello world!" dasturini tuzish.
4. Python tilining asosiy operatorlari bilan tanishish

1. O'quv fanining maqsad va vazifalari .

"Sun'iy intellekt asoslari" fanining maqsadi kursantlarga dasturlash texnologiyalari haqida tushuncha berish va O'zbekiston Respublikasi Qurolli Kuchlari tizimida qo'llaniladigan dasturlash tillarini o'rgatish, ularga operatsion tizim tomonidan qo'llaniladigan tayyor dasturiy ta'minotni yaratish uchun zarur bo'lgan dasturiy vositalardan foydalanishni o'rgatish.

Kredit-moduli tizimi asosida fan bo'yicha o'quv kursi ma'ruza, amaliy mashg'ulotlar, shuningdek, mavzu bo'yicha topshiriq va mustaqil topshiriqlarni o'z ichiga oladi. Ma'ruza o'quv material, amaliy ishlar ko'rsatilgan mavzular bo'yicha nazariy va amaliy ma'lumotlar beradi, amaliy ish, mustaqil ish va ishlarni bajarish tartibini tushuntiradi. Kursda belgilangan o'quv material kursantlar tomonidan mustaqil ravishda o'rganiladi, testlar, amaliy ishlar individual ravishda amalga oshiriladi.

Kurs 1 semestrda mo'ljallangan bo'lib, umumiy auditoriya 90 soatni tashkil qiladi. 7-Semestrda 16 soat ma'ruza, 50 soat guruhli, 24 soat amaliyot mavjud.

Sinfdagi yuklamadan tashqari mustaqil ishlarga ham vaqt ajratiladi. Shunga ko'ra, 7-semestrda 60 soat mustaqil ish.

Mustaqil tayyorgarlik mashg'ulotlarida kursantdan mustaqil ravishda turli xil dasturlarni yaratishni va yaratgan dasturi haqida gapirib berishi talab qilinadi.

Kursantlarni baholash

Kursantlarning bilimini baholash semestr davomida va yakuniy nazorat jarayonida o'quv materialini o'zlashtirish ko'rsatkichi asosida amalga oshiriladi.

7-semestr davomida "Sun'iy intellekt asoslari" fanini o'rganishda kursantlar 100 ballik tizim bo'yicha baholanadi. Shundan joriy nazoratga 40 ball va oraliq nazoratga 20 ball, yakuniy nazoratga esa 40 ball beriladi. Joriy va oraliq nazorat ballarining umumiy natijasi 30 balldan past bo'lgan kursantlar yakuniy nazorat imtihoniga qo'yilmaydi. Yakuniy nazoratga kiritilgan, unda 30 va undan ortiq ball to'plagan kursantlar o'tgan semestrda fanni o'zlashtirgan deb hisoblanadi.

Joriy, oraliq va yakuniy nazorat ballari quyidagicha taqsimlanadi:

Joriy nazorat	40 ball
Oraliq nazorat	20 ball
Yakuniy nazorat	40 ball
Fan bo'yicha jami:	100 ball

Sun'iy intellekt asoslari va uning rivojlanish tarixi.

Sun'iy intellekt asoslari. Python yuqori darajadagi dasturlash tili bo'lib, u AT ning turli sohalarida, masalan, mashinani o'rganish, turli ilovalarni ishlab chiqish, web illovalari ishlab chiqish, tahlil qilish va boshqalarda keng qo'llaniladi.

2019-yilda Python Java tilini 10 foizga ortda qoldirib, eng ommabop dasturlash tiliga aylandi. Bu juda ko'p sabablarga bog'liq bo'lib, ulardan biri malakali mutaxassislarga to'lanadigan ish haqining yuqoriligidir (yiliga taxminan 100 ming dollar).

Hozirgi kunda Turli xil dasturlash tillari mavjud bo'lib, odatda ular ma'lum bir sohada (yoki bir nechta) liderlik qilishadi. Sun'iy intellekt asoslari bo'lsa, turli xil sohalarda ilovalar yaratish imkomiyatiga egaligi, dasturchilarni o'ziga jalb qilmasdan qolmaydi.

Web ilovalar ishlab chiqish bozorining kichik qismini Python tili yaratilgan web illovalari egallagan. Python tilini desktop ilovalarini yozish uchun ham ishlatilsa, mashinani o'rganish (machine learning) sohasida to'liq yetakchilik qiladi.

Python nomining kelib chiqish tarixi.

Gvido van Rossum Sun'iy intellekt asoslarini yaratganidan keyin, uni nomlamasdan turib tarqatishni hohlamadi. Ammo mukammal nom tanlash uchun vaqt sarflash Gvido uchun vaqtni behuda o'tkazish hisoblanardi. Shunda uning xayoliga kelgan birinchi fikr "Python" bo'ldi va dasturlash tilining nomini "Python" deb nomlab qo'yAQoldi. Chunki 1970-yillarning boshida mashhur bo'lgan, "Monty Python's Flying Circus" komediya seryali uning sevimli shoularidan biri edi.

Yangi til nomi bilan bir qatorda logotip o'ylab topish kerak edi va Guido tasodifiy shrift tanladi va "Python" so'zini shu shriftda yozdi. Gvido bu nomning ilonlarga aloqasi yo'qligini qayta-qayta ta'kidlagan bo'lsada, ommaning fikriga ta'sir qilishning ilojisi yo'q edi.

Ko'pchilik dasturchilar Python tilini ilonlar bilan bog'lab, uning nomini emas turli turdagi ilonlar rasmlari bilan kitoblar chop qilishar edi. Bu holat 2006 yilgacha davom etdi.

Payton (python) yoki Piton (ПИТОН) – bu qanday talaffuz qilinadi?

Bu ingliz teleko'rsatuvining nomi bo'ladimi yoki "ilon" so'zining inglizcha tarjimasi bo'ladimi, "Python" nomi Python kabi to'g'ri talaffuz qilinadi. Biroq, rus dasturchilarining taxminan 80% "Piton" (ПИТОН) deb talaffuz qilishga o'rganib qolishgan.

Ushbu variantlardan birini aniq to'g'ri ikkinchisini xato deb aytish qiyin. Biroq, "Piton" deb talaffuz qilish varianti faqat rus tilida so'zlashuvchi suhbatdoshlar bilan suhbatda foydalanish uchun mos keladi, chunki har qanday xalqaro konferentsiyada "Piton" so'zi tushunarsiz bo'ladi. Chunki ingliz tilida bunday so'z mavjud emas.

Bunday konferensiyalarda faqat "Python (Python)" deb talaffuz qilish kerak bo'ladi.

Logotip. Logotipda ikkita ilon kvadrat shaklda tasvirlangan, bu esa ko'pincha foydalanuvchilarni til nomini sudralib yuruvchi bilan bog'lashga undaydi.



1-rasm. Python logotipi

Bu logotip muallifning ukasi, dasturchi va tip dizayneri Joost van Rossum tomonidan yaratilgan. U logotip dizaynini ishlab chiqishda kvadrat shaklini egallagan ikkita ilon va Flux Regular matn shriftida yozilgan Python so'zidan foydalandi.

Yaratilish tarixi. Til 1980-yillarning oxirida dasturchi Guido van Rossum tomonidan ishlab chiqilgan. O'sha paytda u Niderlandiyadagi matematika va informatika markazida ishlayotgan edi.

Guido van Rossum maktab yillaridanoq apparat vositalari bilan ishlashni yaxshi ko'rardi va u tengdoshlari tomonidan qo'llab-quvvatlanmasa ham, ma'qullamasa ham, bu uning mustaqil ravishda dasturlash tilini rivojlantirishiga to'sqinlik qilmadi.

Rossum bo'sh vaqtlarida Pythonda ishlagan va u bir paytlar rivojlanishiga yordam bergan ABC dasturlash tiliga asoslanib ishlagan.

Sun'iy intellekt asoslari tarixidagi bosqichlar :

- **1991-yilning fevralda** python tilining kodlari *alt.sources* saytida chop etildi. Til obyektga yo'naltirilgan yondashuvga amal qildi va obyekt, class, metodlar, merosxorlik, funksiyalar, istisnolarni qayta ishlash va barcha asosiy ma'lumotlar tuzilmalari bilan ishlash imkoniyatiga ega edi.

- **2000 yilda Pythonning ikkinchi versiyasi chiqdi.** Unga ko'plab muhim vositalar, jumladan Unicode yordami va qoldiqlarni yig'uvchi funksiyasi qo'shildi.

- **2008-yil 3-dekabrda Pythonning uchinchi versiyasi ishlab chiqildi.** Bu versiyasi hozircha oxirgi versiya hisoblanadi. Bu versiyada tilning ko'pgina xususiyatlari qayta ishlangan va oldingi versiyalar bilan mos kelmaydigan holga keltirildi. Uchinchi versiyaning funksionalligi ikkinchisidan kam bo'lmasada, tilning rivojlanishi ikki tarmoqqa bo'lingan. Kimdir eski loyihalarni qo'llab-quvvatlash uchun Pythonning 2-versiyasida foydalanishni davom ettirsa, kimdir butunlay uchinchi versiyada ishlashni tanladi.

Ikkinchi versiyaning tugatilish vaqti 2015-yilga belgilangan edi. Biroq barcha mavjud kodlarni Python 3 ga o'tkazishga ulgurmaslikdan qo'rqib, Python 2 ning ishlash muddati 2020-yilgacha uzaytirilgan.

Pythonning sintaksisi uni har doim boshqa dasturlash tillaridan ajratib turadi. U ortiqcha operatorlardan xalos qilingan. Oddiy ingliz tili bilan sintaksisning o'xshashligi hatto oddiy foydalanuvchiga ham kodni tushunishga imkon beradi. Bundan tashqari, dasturchi kamroq kod yozadi, chunki ortiqcha belgilarni ishlatishning hojati bo'lmaydi. Masalan ; { }....

Pythonning oddiyligi qisman tilning ABC tiliga asoslanganligi bilan bog'liq bo'lib, u dasturlashni va dasturchi bo'lmaganlarning kundalik ishlarini o'rgatish uchun ishlatilgan.

Python kod yozishni soddalashtiradi va rivojlanishni tezlashtiradi, chunki u quyidagi xususiyatlarga ega:

- **Dinamik tip.** Dasturchi o'zgaruvchilar tipini ko'rsatishi shart emas, tilning o'zi o'zgaruvchiga qanday ma'lumot yuklanganiga qarab tipini aniqlab oladi.

- **Funksiya orqali bir nechta qiymatlarni qaytarish.** Ushbu qiymatlarni vergul bilan ajratilgan holda to'rtburchak qavslar ichiga yoziladi va avtomatik ravishda ro'yxatga aylantiriladi. Funksiyadan massivni qaytarish uchun "return ro'yxat_nomi" yozish kifoya.

- **Avtomatik xotiradan joy ajratish.** Dasturchi o'zgaruvchilarga xotiradan joy ajratish uchun ortiqcha kod yozishi talab qilinmaydi. Bu, bir tomondan, dasturchining dastur ustidan nazoratini kamaytiradi, ikkinchi tomondan, rivojlanish sezilarli darajada tezlashadi.

- **Chiqindilarni yig'uvchi.** Agar obyekt keraksiz bo'lib qolsa, u chiqindi yig'uvchi tomonidan avtomatik ravishda o'chiriladi. Chiqindi yig'uvchi sizga xotiradan foydalanishni optimallashtirish va keraksiz narsalarni qo'lda o'chirmaslik imkonini beradi.

- **a, b = b, a ifodasi.** Bu ifoda o'zgaruvchilarning qiymatlarini o'zaro almashtiradi. Endi a o'zgaruvchining qiymati b o'zgaruvchisiga yuklandi va aksincha. Shunday qilib, siz nafaqat ikkita o'zgaruvchining, balki uch va undan ortiq obyektlarning qiymatlarini mos ravishda bir-biriga almashtirishingiz mumkin. Faqat taraflardagi o'zgaruvchilar soni teng bo'lishi talab qilinadi.

- **O'zgaruvchi tipining qiymatga bog'liqligi.** O'zgaruvchining qiymati - bu uning turini va boshqa xususiyatlarini belgilaydigan atributlarga ega bo'lgan qandaydir obyekt. O'zgaruvchi esa bu obyektning nomidir. Ushbu yondashuv qat'iy tipdagi ta'riflarga bo'lgan ehtiyojni yo'qqa chiqaradi va o'zgaruvchilarga boshqa turdagi qiymatni qayta tayinlashni sezilarli darajada soddalashtirdi.

•**For sikli** . Pythonda massivlar, ro'yxatlar va boshqa konteynerlar bilan ishlash oson va qulay. Uning barcha elementlarini takrorlash zarur bo'lganda, konstruktsiya quyidagicha ko'rinadi: **for x in konteyner:** (*iteratsiya 0 dan oxirgi -1 elementgacha elementlarni x ga qaytaradi*). Agar siz siklni aniq bir miqdorda takrorlanishini amalga oshirish uchun **range()** funksiyasidan foydalanish kerak bo'ladi: **for x in range(1,9):** (*sikl 1 dan 8 gacha bo'lgan qiymatlarini x ga uzatadi*).

•**Machine learning sohasidagi yetakchiligi.** Sun'iy intellekt asoslari "Mashina o'rganish" (Machine learning) sohasida yaqqol yetakchilik qilib kelmoqda. Ilm-fan bilan u yoki bu tarzda shug'ullanadigan odamlar kod yozish kabi narsalarga ko'p vaqt sarflamaslikni afzal ko'radilar, shuning uchun Python ularga yuklangan vazifalarni amalga oshirish uchun juda mos keladi.

Python ham soddalik, ham kuchli vositalarni o'zaro birlashtiradi. U deyarli har qanday dasturning prototipini yaratish uchun ishlatilishi mumkin.

Kod misoli :

```
def kattasi(a, b):
    if a > b:
        print(a, "katta", b, "dan")
    else:
        print(b, "kichik", a, "dan")
def max_mass (massiv):
    max = 0
    for x in massiv:
        if massiv[x-1] > max:
            max = massiv[x]
    return max

def 2mass(massiv):
    massiv = massiv * 2
    return massiv

print("Oddiy Python dasturi")

a = [1,2,3,6,1,6]
kattasi (1,5)
r1 = max_mass(a)
r2 = 2mass (a)

print("Massiv max elementi -", r1)
print("Ikkilangan massiv - ", r2)
```

Dastur natijasi:

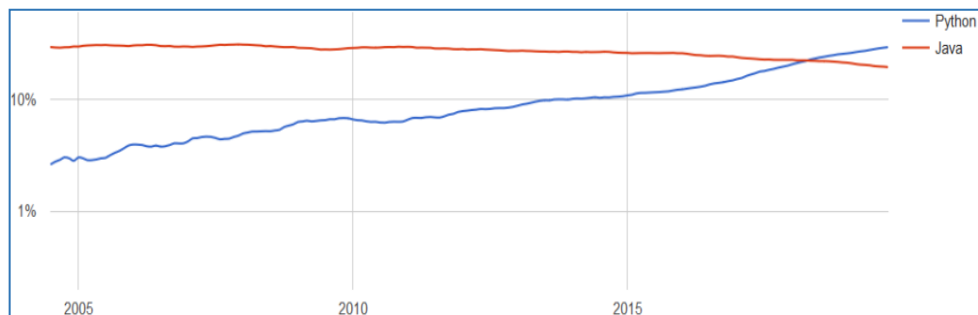
```
Oddiy Python dasturi
5 katta 1 dan
Massiv max elementi - 6
Ikkilangan massiv - [1, 2, 3, 6, 1, 6, 1, 2, 3, 6, 1, 6]
```

Ommalashganligi. Til 30 yoshdan oshgan bo'lsada, butun dunyodagi dasturchilar orasida juda keng tarqalgan. Python deyarli har bir o'rta yoki katta loyihada, asosiy ishlab chiqish vositasi sifatida bo'lmasada, prototiplash yoki uning bir qismini yozish uchun vosita sifatida ishlatiladi.

U o'z atrofida juda katta dasturchilar guruhini to'pladi.

Stackoverflow saytida o'tkazilgan so'rov natijalariga ko'ra, Python deyarli 39% ovoz bilan 7-o'rinni egalladi.

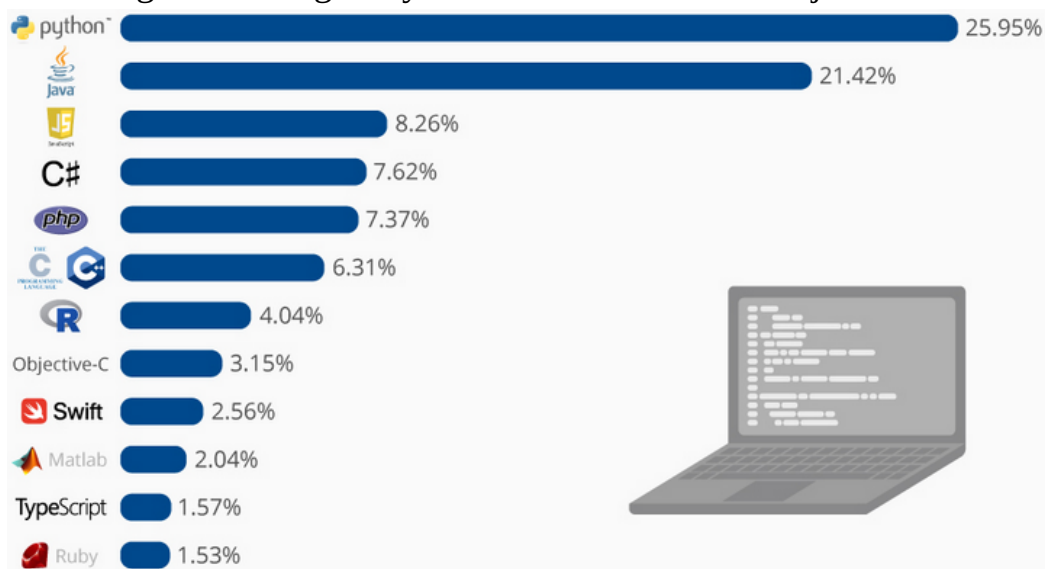
PYPL. Ushbu indeks tilni o'rganish materiallari bilan bog'liq qidiruv so'rovlari soniga asoslanadi.



2-rasm. PYPL indeksi bo'yicha solishtirma sxemasi

PYPL ma'lumotlariga ko'ra, Python mashhurligi bo'yicha Javadan 10% ga oldinda.

statista.com. Xizmat turli xil statistik ma'lumotlarni taqdim etadi, ular orasida dasturlash tillarining mashhurligi bo'yicha statistikalar ham mavjud.



3-rasm. Statista.com saytidagi solishtirish diagrammasi

Ish tezligi. Dasturchilar ko'pincha o'zlariga savol berishadi: "Python-dan foydalanish ishlashning pasayishiga olib keladimi?". Batafsil tekshiruvsiz hech qanday xulosa chiqarmang.

Agar biz faqat kodni bajarish tezligini hisobga olsak, Python C kabi boshqa dasturlash tillaridan past ekanligi ayon bo'ladi. Darhaqiqat, dinamik terish, izohlash va

dasturchi ishini osonlashtiradigan boshqa xususiyatlar ishlashning pasayishiga olib keladi. Biroq, zamonaviy IT-da nafaqat dasturlarning tezligi, balki ularni ishlab chiqish tezligi ham muhimdir. Ishlab chiqish, sinovdan o'tkazish, disk raskadrovka va qo'llab-quvvatlash - bularning barchasi juda ko'p pul talab qiladi. Agar Python dasturlarining tezligi bo'yicha past bo'lishi mumkin, ammo rivojlanish tezligida bo'yicha unga teng keladigani yo'q.

Dasturchilar Pythonda yaratilgan dasturlarni bajarilish tezligini oshirish uchun turli yo'llardan foydalanadilar:

- **C kodini o'rnatish.** Ushbu texnikadan foydalanib, siz ishlashni sezilarli darajada yaxshilashingiz mumkin, odatda vaqt birligida ko'p so'rovlarni qayta ishlaydigan kod bo'limlari C tilida yoziladi. Masalan, bitta ma'lumotlar bazasidan ma'lumotlarni qabul qiladigan, uni qayta ishlaydigan va boshqasiga yuboradigan funksiyalar, agar o'tadigan ma'lumotlarning miqdori yetarlicha katta bo'lsa, C tilida yozilgani afzal xisoblanasi.

- **ifodalarni optimallashtirish.** Python dasturlarining tezligi ifodalarning ishlash tezligiga juda bog'liq. Quyidagi jadvalda tezroq va sekinroq ishlovshi ifodalar keltirilgan.

1-jadval

Tezroq	Sekinroq
$a, b = c, d$	$a = c; b = d$
$a < b < c$	$a < b \text{ and } b < c$
not not a	bool(a)
$a = 5$	$a = 2 + 3$

- **Dasturni testlash uchun tayyor modullar.** Kodning qaysi bo'limlari umumiy ish faoliyatini sezilarli darajada kamaytirishini aniqlash uchun dasturchi testlash uchun maxsus modullardan foydalanishi mumkin. Shunday qilib, bu modullar yordamida qaysi kodni optimallashtirish yoki C kodi bilan almashtirish kerakligini bilib olishi mumkin.

- **Tayyor asboblari.** Aksariyat vazifalar uchun samarali echimlar allaqachon ishlab chiqilgan. O'z yechimingizni noldan yozishdan ko'ra, ba'zi kutubxonaning tayyor, tuzatilgan kodini ishlatish yaxshiroqdir, bu 100% samarali bo'lmaydi.

Pythonda qanday dasturlar yozish mumkin?

Python tilida turli sferalarda dasturlar tuzish mumkin.

Back-end – saytning dasturiy qismi. Saytning server tomonini rivojlantirish uchun turli frameworklar qo'llaniladi: **Django** va **Flask**. Bu frameworklar Pythonni boshqa mashhur vositalar bilan raqobatlashadigan imkoniyatlarga ega server tomonidagi dasturlash tiliga aylantiradilar.

PHP tili server tomonidagi web ishlab chiqish bozorining ko'p qismini nazorat qilsada, tobora ko'proq dasturchilar Pythonda ishlab chiqishni afzal ko'rishmoqda.

Blokcheyn. Blokcheyn - bu ketma-ket bloklar zanjiri bo'lib, unda har bir blok ma'lumotni o'z ichiga oladi va har doim oldingisiga ulanadi. Texnologiya har qanday sohada qo'llanilishi mumkin. Ayniqsa moliya sektorida va bitcoin kriptovalyutasi sohasida mashhurdir.

Blokcheyn axborot xavfsizligi va ochiqqligini o'zida mujassamlashtiradi. U foydalanuvchiga dunyoning istalgan nuqtasidan ma'lumotlarga kirish imkonini beradi.

Bot. Bu ma'lum bir vaqtda yoki berilgan signalga javoban ba'zi harakatlarni avtomatik ravishda bajaradigan dasturdir. Botlar inson xatti harakatlarini bevosita taqlid qilishi mumkin. Shuning uchun ular ko'pincha texnik yordam ko'rsatishda (chat botlarida), Internetda ma'lumot qidirishda (qidiruv botlarida), virtual dunyoda odam yoki boshqa mavjudotning harakatlarini taqlid qilishda (kompyuter o'yinlari) zarur bo'ladi.

Python sizga tezkor xususiyatga ega va nisbatan aqlli botlarni yaratish imkonini beradi. Shuni tushunish kerakki, botlar 500 qatorli kodlardan iborat oddiy dastur emas. Biznes uchun bot yaratish buyurtmasi bir necha millionga tushishi mumkin. Narx insondan farqlash qiyin bo'ladigan botni loyihalash juda qiyin ekanligi bilan bog'liq. Muloqotning ko'plab variantlarini ta'minlash, inson xatti-harakatlarining omillarini tahlil qilish va ularni dasturda amalga oshirish kerak. Oddiy qilib aytganda, faqat nol va birlarni tushunadigan mashinadan siz ibtidoiy "miya" qilishingiz kerak bo'ladi.

Malumotlar bazasi. Ma'lumotlar bazasi umumiy xususiyatlar va maxsus qoidalarga muvofiq tizimlashtirilgan ma'lumotlardir. Har qanday yirik loyihada ma'lumotlar bazalaridan foydalaniladi. Ular foydalanuvchilar haqidagi ma'lumotlarni, dasturdagi o'zgarishlarni va hokazolarni saqlaydi.

Ma'lumotlar bazasini boshqarish tizimi Pythonda yozilishi mumkin.

Kengaytirilgan haqiqat (Дополненная реальность). "To'ldirilgan reallik" virtual texnologiyalar yordamida jismoniy dunyoni to'ldiradi. Ya'ni, virtual obyektlar real muhitga proyeksiya qilinadi va oddiy jismoniy obyektlarning xususiyatlari va xatti harakatlariga taqlid qiladi.

"Kengaytirilgan haqiqat"ni turli 3D filmlarda guvohi bo'lish mumkin. Haqiqiy dunyoda u, masalan, jangovar jangchilarda (maqsad tizimi) ishlatiladi.

"Kengaytirilgan haqiqat" ishi teglar bilan o'zaro ta'sirga asoslangan. Elektron qurilma ma'lumot oladi va atrofdagi makonni tahlil qiladi, kompyuter ko'rish yordamida u odamning oldida nimani ko'rayotganini "Tushunadi". Keyin qurilma real dunyoda "Virtual qatlam"ni qoplaydi.

"Kengaytirilgan haqiqat" sferasida yaratilgan mukammal dasturlar taxminan 7-10 ming AQSH dollarini tashkil qiladi. Ularni loyihalash va yozish oson emas, 3D-

dizaynerlardan dasturchilargacha turli mutaxassislar ishlab chiqish jarayonida tinmasdan mehnat qilishadi.

Python – bunday sferadagi loyihalarni yaratish uchun ajoyib tanlov.

BitTorrent mijozi. BitTorrent - bu Internet orqali katta hajmdagi ma'lumotlarni tezlikda uzatish, qabul qilish imkonini beruvchi noyob texnologiya.

BitTorrentning 6-versiyasidan keyingi versiyalari butunlay Pythonda yozilgan. Keyinchalik u C++ da butunlay qayta yozilgan bo'lsa-da, bu Python tilidan xuddi shunga o'xshash vazifalarni bajarish uchun foydalanish mumkinligini anglatadi.

Neyron tarmoq. “Neyron tarmoq” tushunchasi dasturlashga biologiyadan kirib kelgan. Biologiyada neyron tarmoq bir-biriga bog'langan neyronlar ketma-ketligidir. Dasturiy ravishda yaratilgan neyron tarmoqlar nafaqat ma'lumotni tahlil qilish va saqlash, balki uni xotira qayta ishlab chiqishga qodir.

Ular inson miyasi tomonidan bajariladigan hisob-kitoblarni talab qiladigan murakkab masalalarni hal qilish uchun ishlatiladi. Odatda, neyron tarmoqlar biror narsani xususiyatlari bo'yicha tasniflash, bashorat qilish, masalan, fotosurat yoki videodan odamni tanib olish kabi funksiyalarni amalga oshirish uchun foydalaniladi.

Python neyron tarmoqlarni rivojlantirishda yaqqol yetakchi hisoblanadi. Standart vositalardan tashqari, u “Mashinani o'rganish” sohasida juda ko'p kutubxonalar mavjud. Buning yordamida hatto katta va murakkab loyihalar nisbatan tez yozilishi mumkin.

Parser - bu ma'lumotlarni yig'ish va qayta ishlash uchun mo'ljallangan dastur. Foydalanuvchi valyuta kursi kabi ma'lumotlarni tahlil qilishi yoki turli kompaniyalar aksiyalaridagi o'zgarishlarni kuzatishi va tahlil qilishi mumkin.

Parser turli xil tillarda yozilishi mumkin. Ya'ni parser dasturlar boshqa dasturlash tillari yordamida ham yaratish mumkin. Lekin parser (ma'lumot yig'uvchi) dasturlarni python tilida boshqa dasturlash tillariga qaraganda tez va samarali yozish mumkin.

O'yin yaratish. Pythonda katta o'yinlar yaratilmaydi. U prototipni (demo versiyasini) ishlab chiqish yoki ba'zi bir qismini amalga oshirish uchun ishlatiladi (masalan, o'yinning server qismidagi logik qismi).

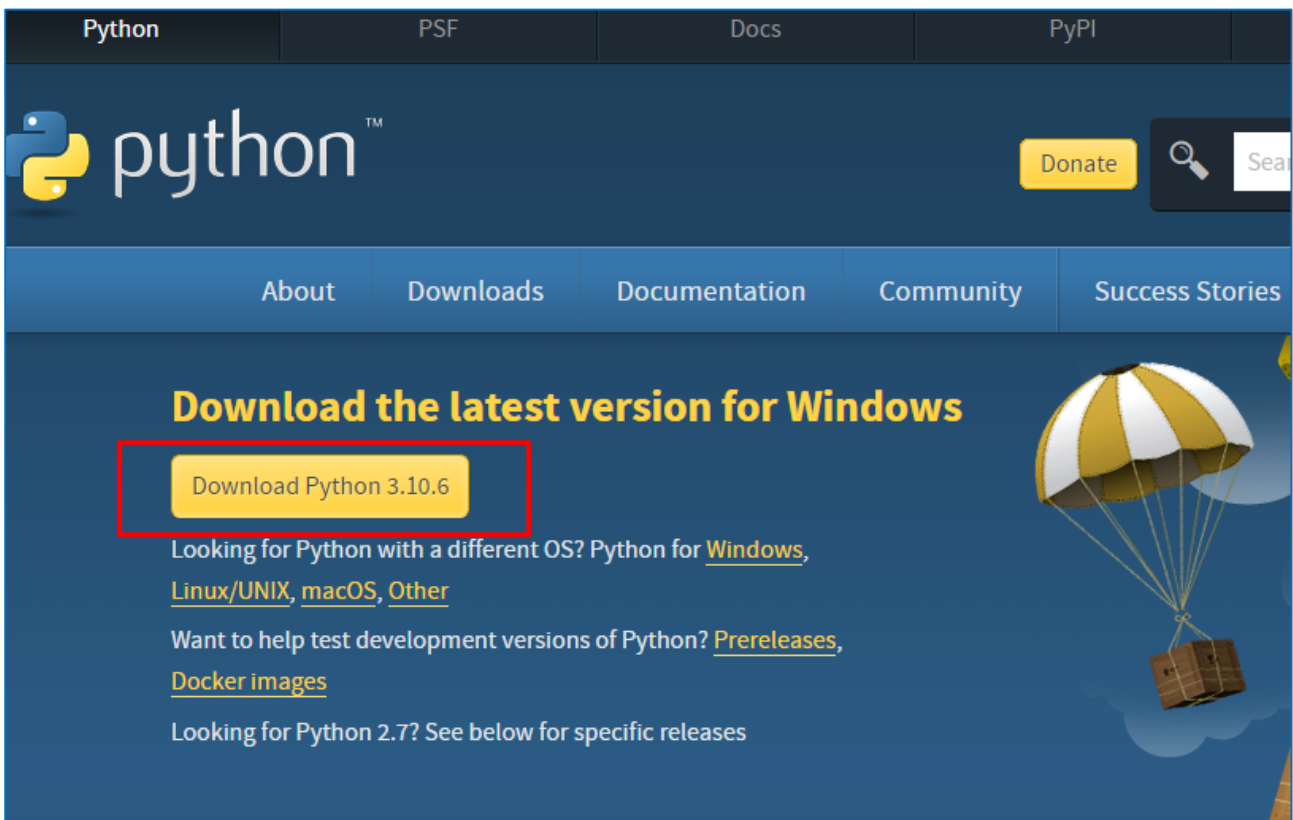
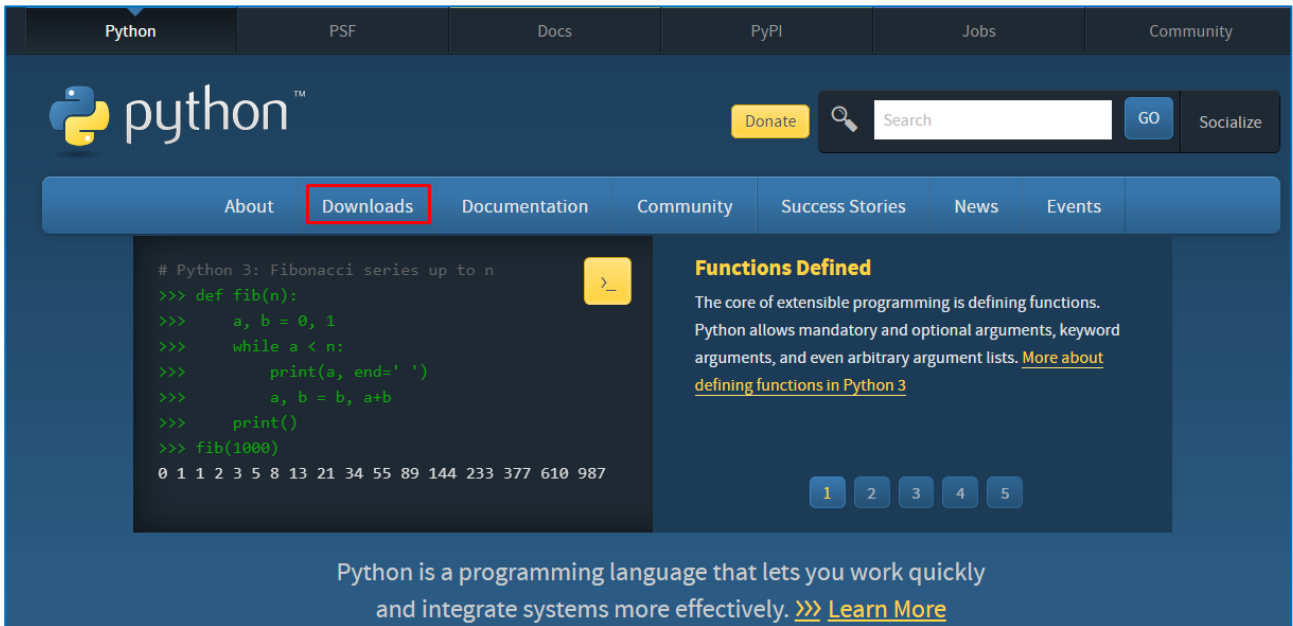
Kichik loyihani yozish uchun siz kichik 2D o'yinni yaratish uchun barcha kerakli vositalarni taqdim etadigan Pygame kutubxonasidan foydalanishingiz mumkin.

2. Pythonni o'rnatish. PyCharm dasturini o'rnatish

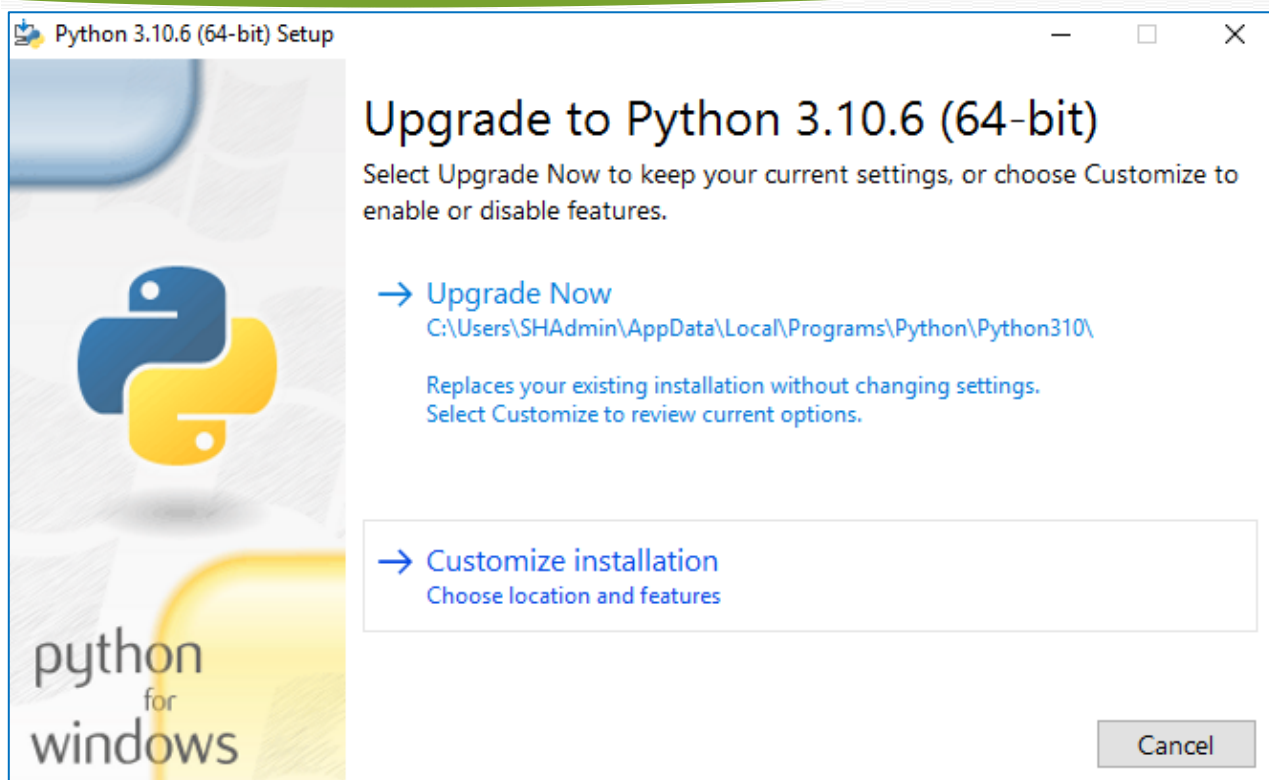
PyCharm - JetBrains tomonidan ishlab chiqilgan kross-platforma muharriri. Pycharm samarali Python rivojlanishi uchun zarur bo'lgan barcha vositalarni taqdim etadi. Quyida Windows-da Python va PyCharm-ni o'rnatish bo'yicha batafsil ko'rsatmalar mavjud.

Pythonni o'rnatish

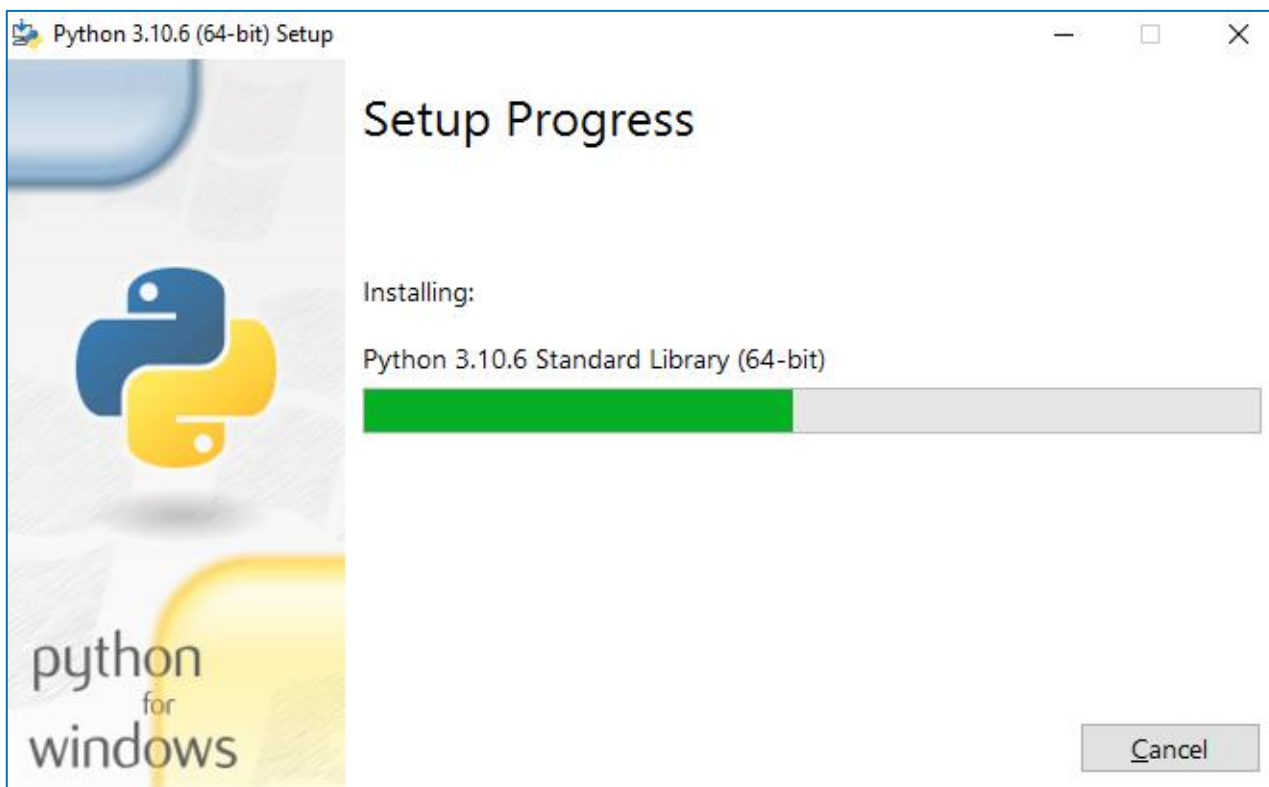
1-qadam. Python-ni yuklab olish va o'rnatish uchun rasmiy Python veb-saytiga tashrif buyuring [//www.python.org/downloads/](https://www.python.org/downloads/) va tegishli versiyani tanlang. Biz Python 3.10.6 versiyasini tanladik



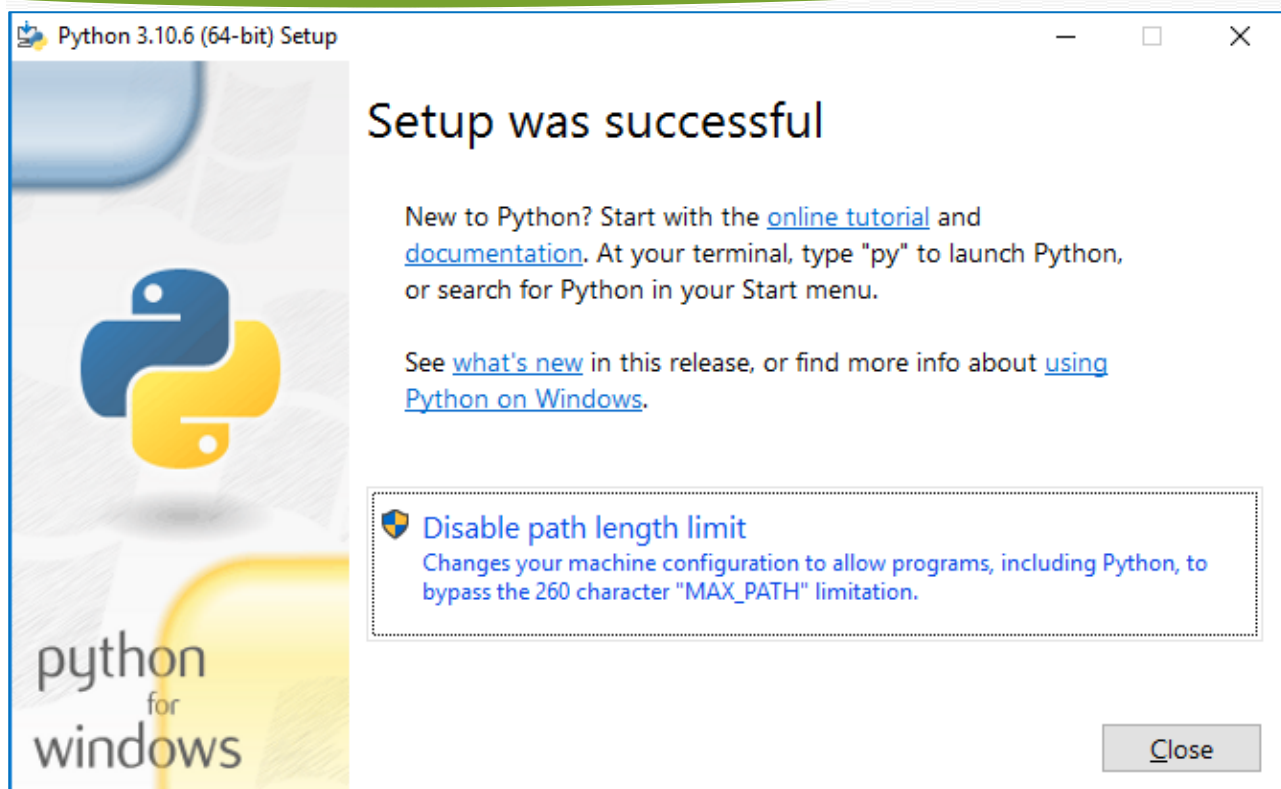
2-qadam. Yuklab olish tugallangach, Python-ni o'rnatish uchun .exe faylini ishga tushiring. Keyin "Hozir o'rnatish" tugmasini bosing.



3-qadam. Keyingi bosqichda Python o'rnatilishi jarayonini ko'rishingiz mumkin.



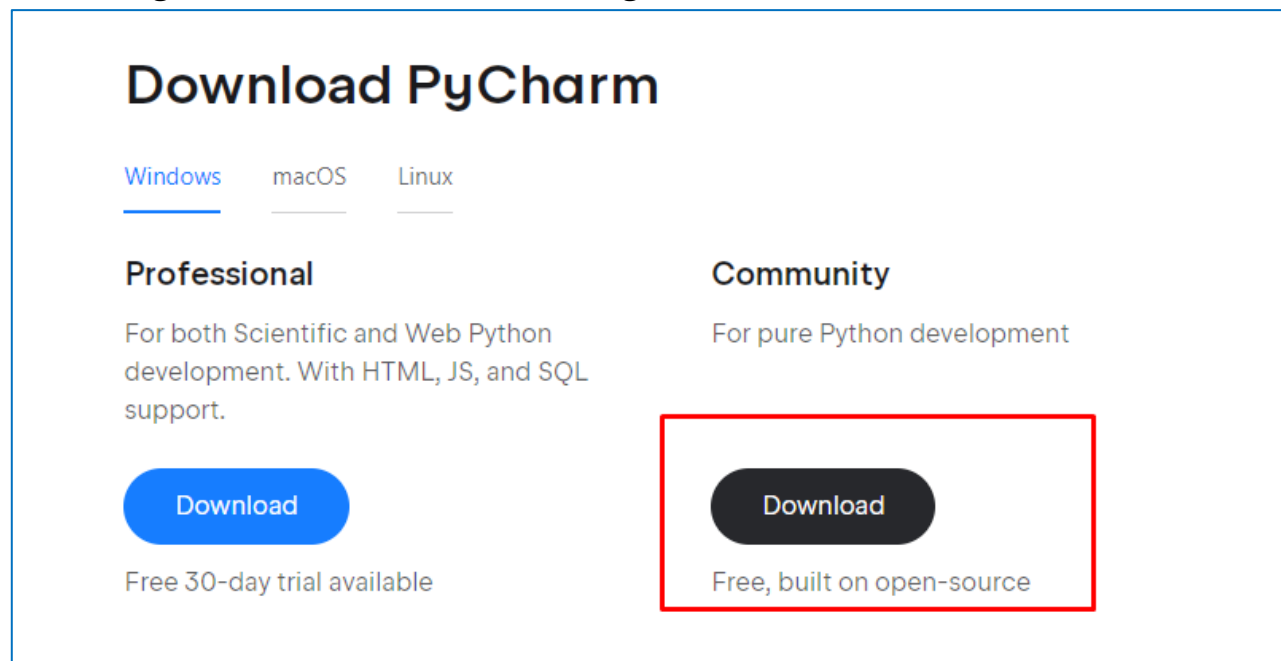
4-qadam. O'rnatish tugallangach, o'rnatish muvaffaqiyatli bo'lganligi haqida panelni ko'rasiz. Endi "Yopish" tugmasini bosing.



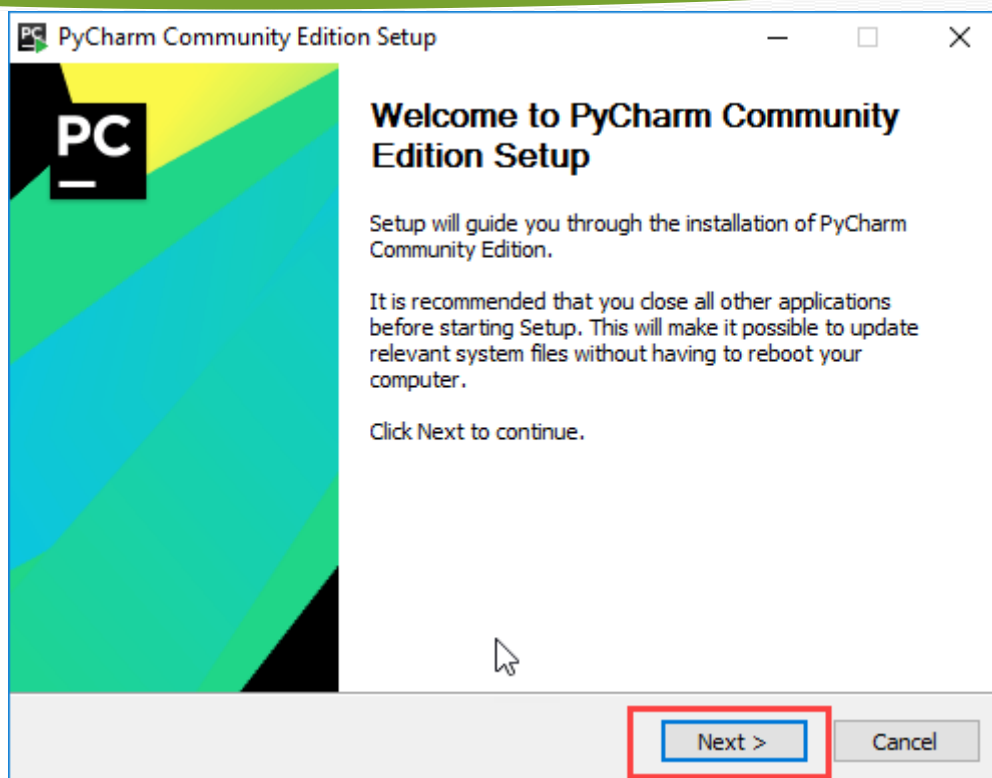
Pycharm dasturini o'rnatish

1-qadam.

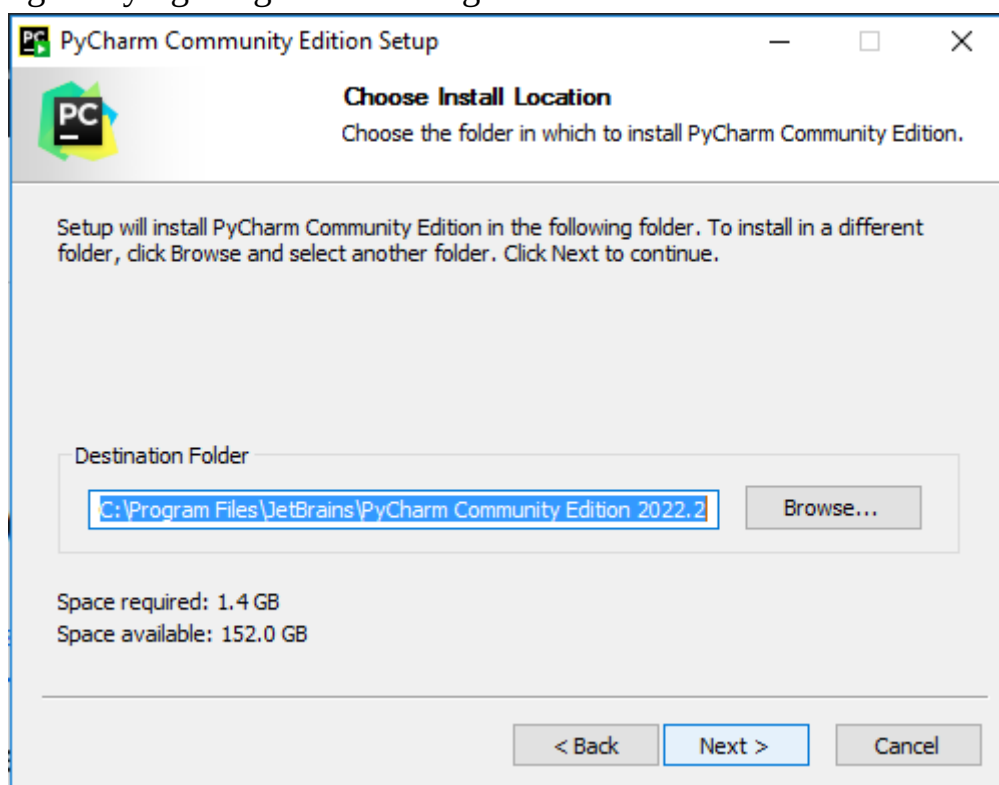
PyCharm-ni yuklab olish uchun [//www.jetbrains.com/pycharm/download/](https://www.jetbrains.com/pycharm/download/) veb-saytiga tashrif buyuring va jamoat bo'limidagi YUKLASH havolasini bosing.



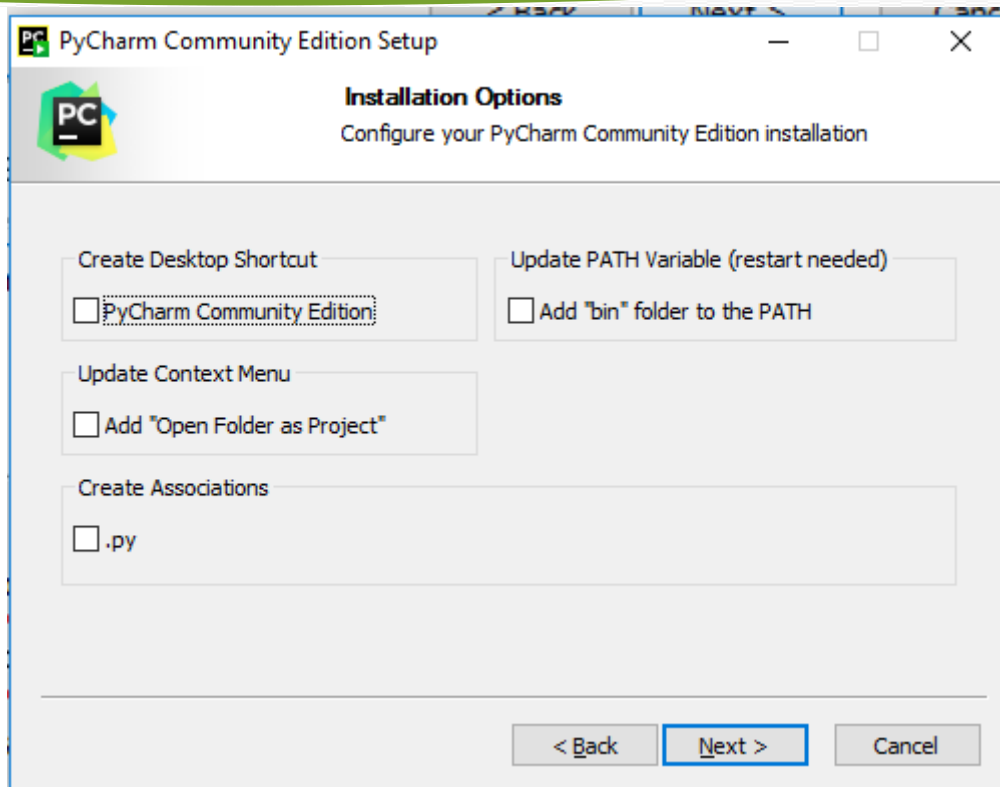
2-qadam. Yuklab olish tugallangach, PyCharm-ni o'rnatish uchun .exe faylini ishga tushiring. O'rnatish ustasi ishga tushishi kerak. "Keyingi" tugmasini bosing.



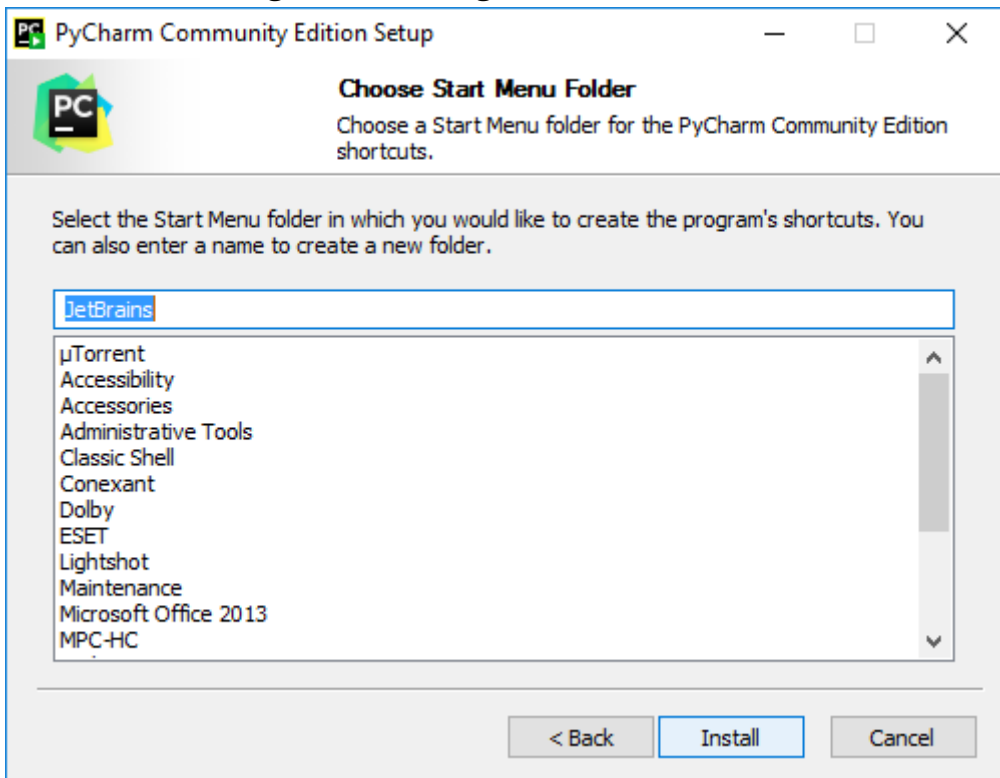
3-qadam. Keyingi bosqichda, agar kerak bo'lsa, o'rnatish yo'lini o'zgartiring. "Keyingi" tugmasini bosing.



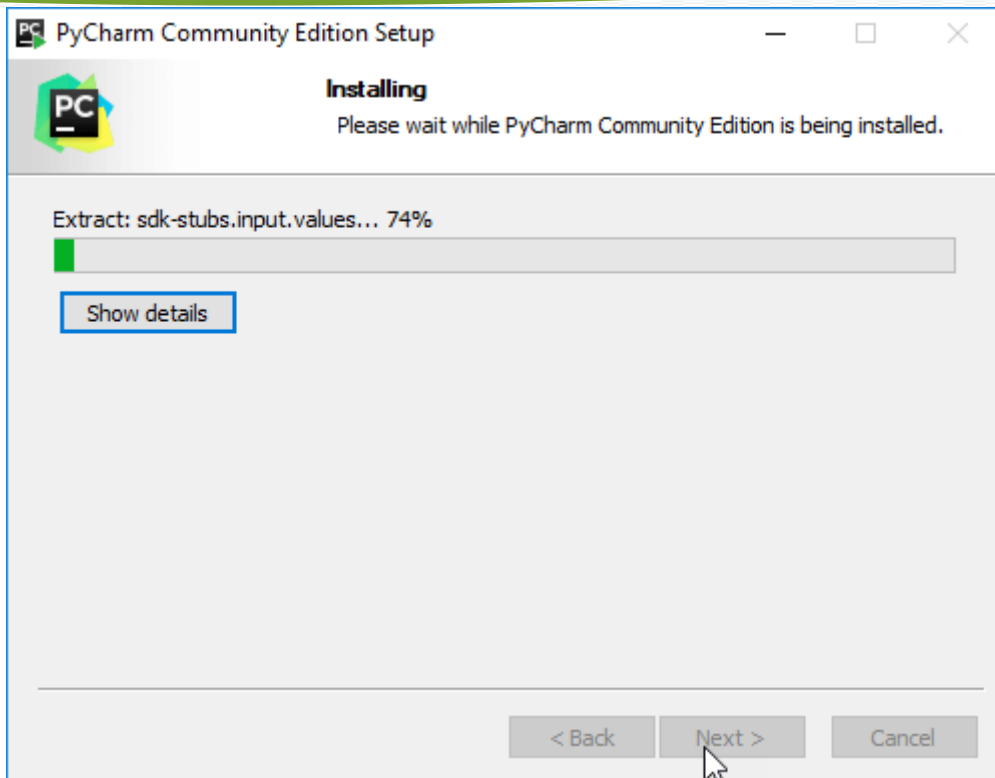
4-qadam. Keyingi bosqichda, agar xohlasangiz, ish stolida yorliq yaratishingiz mumkin, so'ngra "Next" tugmasini bosing..



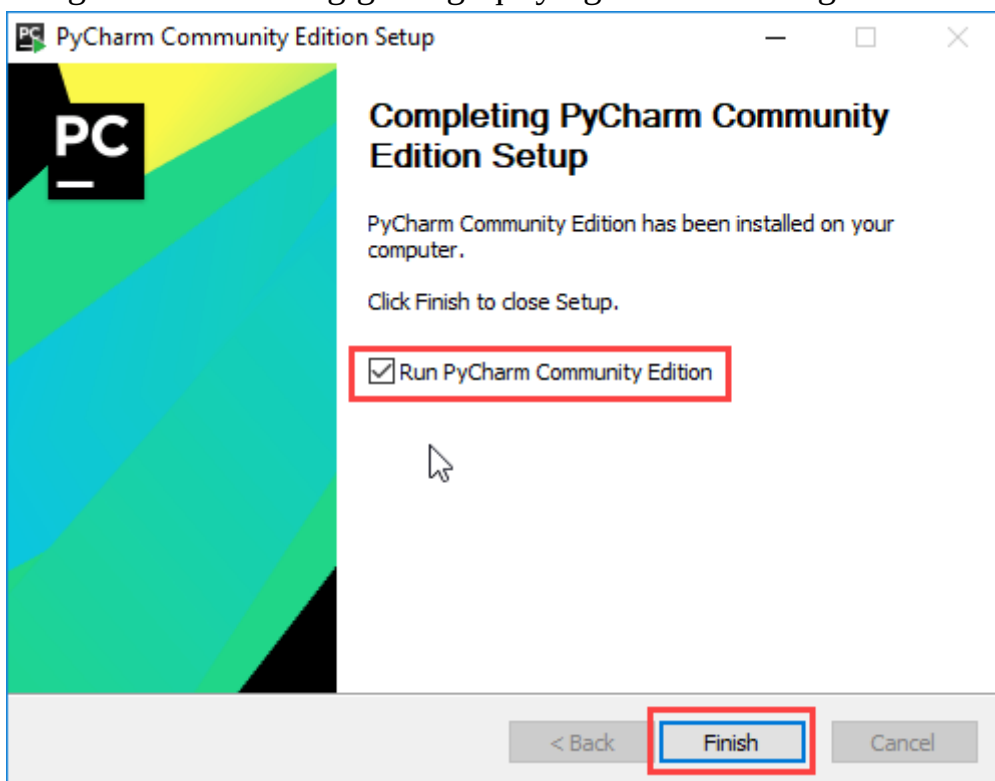
5-qadam. Boshlash menyusi papkasini tanlang. JetBrains-ni sukut bo'yicha qoldiring va "O'rnatish" tugmasini bosing.



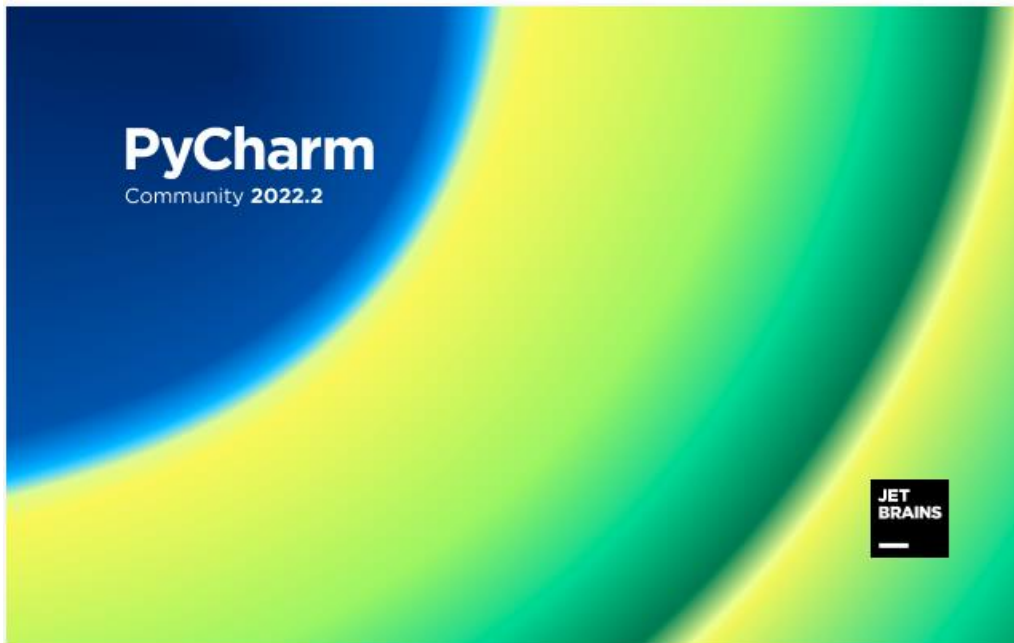
6-qadam. O'rnatish tugashini kuting.



7-qadam. O'rnatish tugallangandan so'ng siz PyCharm o'rnatilganligi haqida xabar olasiz. Agar siz uni ishga tushirishni xohlasangiz, avval "PyCharm Community Edition-ni ishga tushirish" katagiga belgi qo'ying va "Finish" tugmasini bosib.

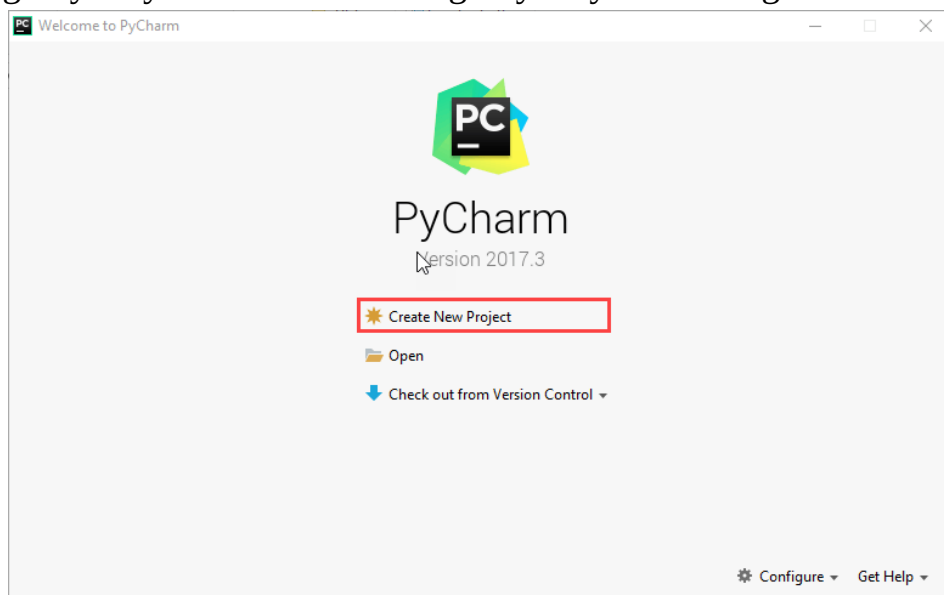


8-qadam. Finish tugmasini bosganingizdan so'ng quyidagi ekran paydo bo'ladi.



3. Pythonda “Salom dunyo !” dasturini yaratish.

1-qadam. PyCharm muharririni oching. PyCharm xush kelibsiz panelini ko'rasiz. Yangi loyiha yaratish uchun "Yangi loyiha yaratish" tugmasini bosib.

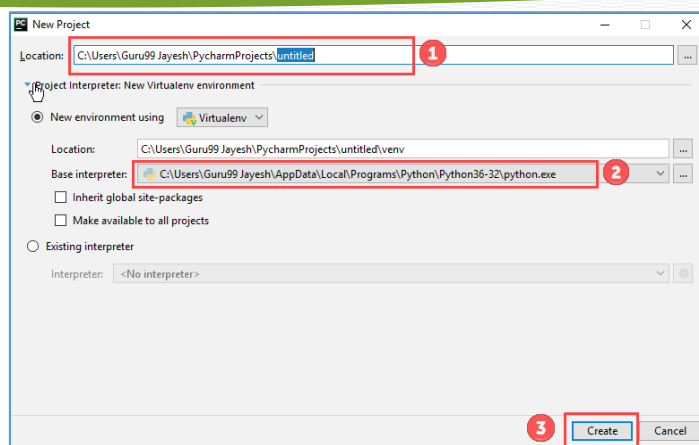


2-qadam.

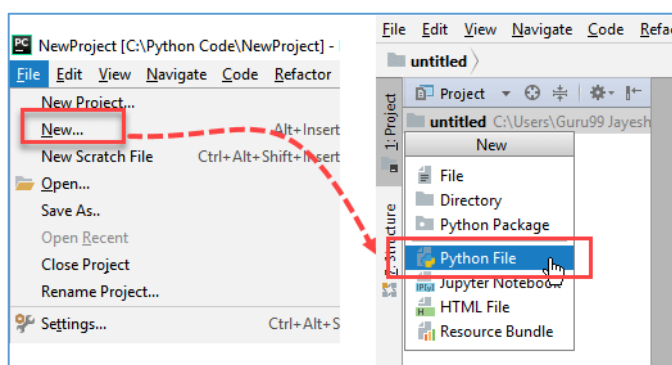
Siz o'rindiqni tanlashingiz kerak.

Siz loyihani yaratmoqchi bo'lgan joyni tanlashingiz mumkin. Agar siz joylashuvni o'zgartirishni xohlamasangiz, uni avvalgidek qoldiring, lekin hech bo'lmaganda "nomsiz" nomini "Birinchi loyiha" kabi mazmunliroq narsaga o'zgartiring.

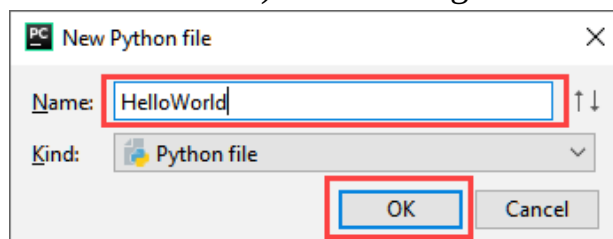
PyCharm siz avval o'rnatgan Python tarjimonini topgan bo'lishi kerak edi. Keyin, "Yaratish" tugmasini bosib.



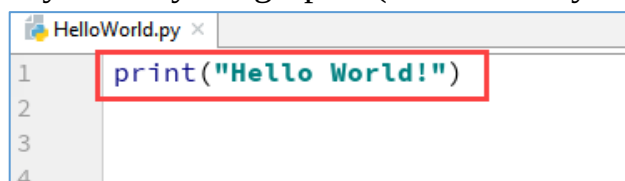
3-qadam. Endi "Fayl" menyusiga o'ting va "Yangi" ni tanlang. Keyin, "Python fayli" ni tanlang.



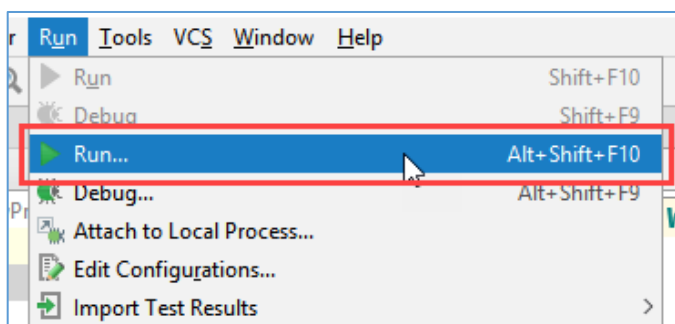
4-qadam. Yangi qalqib chiquvchi oyna paydo bo'ladi. Endi fayl nomini kiriting (bu yerda biz "HelloWorld" ni o'rnatamiz) va "OK" tugmasini bosing.



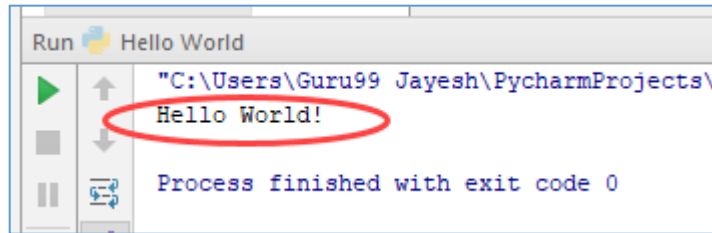
5-qadam. Endi oddiy dastur yozing - `print("Salom Dunyo!")`.



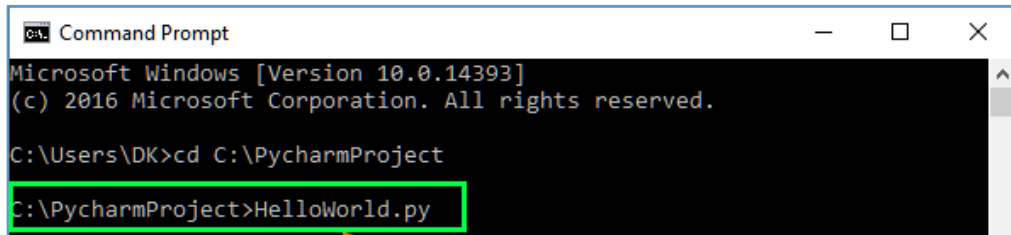
6-qadam. Endi Run menyusiga o'ting va dasturingizni ishga tushirish uchun Run-ni tanlang.



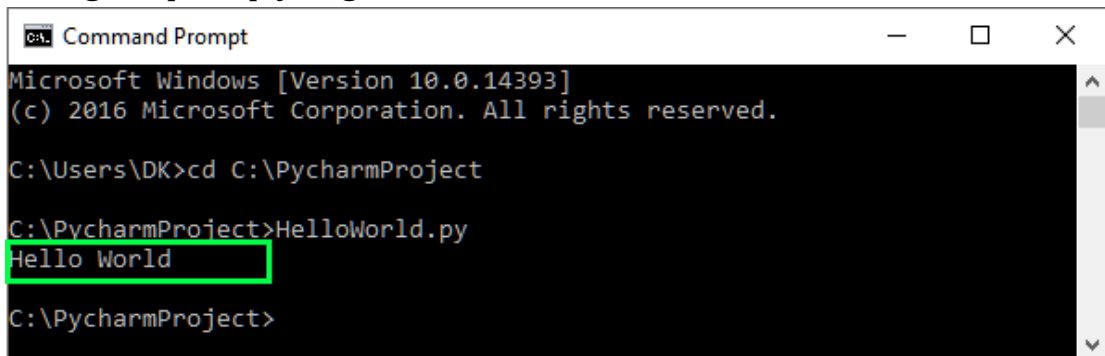
7-qadam. Dasturning chiqishini ekranning pastki qismida ko'rishingiz mumkin.



8-qadam. Agar sizda Pycharm Editor o'rnatilmagan bo'lsa, tashvishlanmang, kodni buyruq satridan ishga tushirishingiz mumkin. Dasturni ishga tushirish uchun buyruq satriga to'g'ri fayl yo'lini kiriting.



Kodning chiqishi quyidagicha bo'ladi:



9-qadam. Agar siz hali ham dasturni ishga tushira olmasangiz, bizda siz uchun Python muharriri mavjud. Iltimos, ushbu kodni ishga tushiring.

4. Python tilining asosiy operatorlari bilan tanishish

Sun'iy intellekt asoslarida yozilgan dastur ifodalar jamlanmasidan iborat. Har bir ifoda yangi qatorga joylashtiriladi. Masalan:

```
print(2 + 3)
print("Hello")
```

Sun'iy intellekt asoslari kodida ifodalarning oldidan qoldirilgan oraliq masofalar katta ro'l o'ynaydi. Agar dasturchi tomonidan bilmasdan yoki chiroq uchun qoldirilgan har qanday oraliq masofa xatolik kelib chiqishiga sabab bo'lishi mumkin. Masalan, garchi kod yuqoridagi bilan deyarli bir xil bo'lsada, quyidagi holatda dasturda oraliq masofalar bilan bog'liq xatolik sodir bo'ladi:

```
print(2 + 3)
print("Hello")
```

Shuning uchun, Sun'iy intellekt asoslarida dastur kodining boshi satrning boshidan boshlanishi kerak. Bu Python bilan C# yoki Java kabi boshqa dasturlash tillari o'rtasidagi muhim farqlardan biridir.

Ammo shuni yodda tutish kerakki, tilning ba'zi operatorlarining sintaksisi bir necha qatordan iborat bo'ladi. Masalan, shart operatori if:

```
if 1 < 2:
    print("salom")
```

Bu dasturda, agar 1 soni 2 dan kichik bo'lsa, u holda “salom” so'zi ekranga chiqariladi. Bu dasturda print(“Salom”) ifodasining oldida bo'sh joy bo'lishi kerak, chunki bu ifoda if shartli konstruksiyasining bir qismi sifatida ishlatiladi.

Bundan tashqari, kodni yozish mobaynida qoldirib ketiladigan bo'shliqlar 4 ga karrali probellar soniga (ya'ni 4, 8, 16 va boshqalar) teng bo'lishi maqsadga muvofiqdir. Ammo 4 ga karrali bo'lmay qolishi xatolik keltirib chiqarmaydi. Ifoda oldida qoldiriladigan masofalarning 4 ta probelga yoki 4ga karrali probellar soniga teng bo'lishining sababi shundaki, 1 ta tab 4 ta probelga tengligida.

Harflar katta-kichikligini farqlanishi (Регистрозависимость)

Python katta-kichik harflarni turli belgilan sifatida qabul qiladigan tildir. Shuning uchun **print** va **Print** yoki **PRINT** ifodalari turli ifodalarni anglatadi. Shunday ekan agar konsolga “Salom dunyo” xabarini Print funksiyasi orqali chiqarmoqchi bo'lsak, hech qanday narsa chiqara olmaymiz, aksincha xatolik sodir bo'ladi:

```
Print("Hello World")
```

Izohlar

Izohlar – kodning ma'lum bir qismida nima amalga oshirilayotganligini ifodalab foydalanuvchi uchun tushunchalar berib ketish uchun foydalaniladi. Dastur interpretatsiya qilinganda (ishga tushirilganda) interpretator izohlarni e'tiborsiz qoldiradi, shuning uchun ular dasturning ishlashiga ta'sir qilmaydi. Pythondagi izohlar blok va inline izohlarda keladi.

Bitta satrda joylashuvchi izohlardan oldin panjara belgisi- # qo'yiladi:

```
# Konsolga chiqish
# Salom Dunyo xabarlari
print("Salom Dunyo ")
```

belgisidan keyingi har qanday belgilar to'plami izohni bildiradi. Ya'ni, yuqoridagi misolda kodning birinchi ikki qatori izohlar hisoblanadi.

Shuningdek izohlar ifodalardan so'ng, til ifodalari bilan bir qatorda joylashgan bo'lishi ham mumkin:

```
print("Salom dunyo") # Xabarni konsolga chop etish
```

Blok izohlarida sharh matnidan oldin va keyin uchta bitta tirnoq qo'yiladi: “‘sharh matni’”. Masalan:

```
'''
    Konsol chiqishi
    Salom dunyo xabarlari
'''
```



```
print ("Salom dunyo")
```

Asosiy funksiyalari

Python ko'plab o'rnatilgan, standart funksiyalarni taqdim etadi. Ulardan ba'zilari, ayniqsa, til o'rganishning dastlabki bosqichlarida juda tez-tez qo'llaniladi, shuning uchun ularni ko'rib chiqaylik.

Konsolga ma'lumot chiqarishning asosiy funksiyasi - bu `print()` funksiyasi hisoblanadi. Biz chiqarmoqchi bo'lgan satr ushbu funksiyaga argument sifatida uzatiladi:

```
print ("Salom Python")
```

Agar konsolga bir nechta qiymatlarni chop etish kerak bo'lsa, ularni vergul bilan ajratib, `print()` funksiyasiga beriladi:

```
print("To'liq ismi:", "Tom", "Smit")
```

Natijada, vergul bilan ajratilgan joylarga probel bilan ajratilgan holatda, barcha berilgan qiymatlar, bir qatorga joylashtirilib konsolga chiqariladi:

```
To'liq ismi: Tom Smit
```

Agar **`print()`** funksiyasi chiqarish uchun javobgar bo'lsa, u holda **`input()`** funksiyasi axborotni kiritish uchun xizmat qiladi. **`input()`** funksiyasi kiritilgan ma'lumotni satr tipida qaytaradi va bu satrni o'zgaruvchiga saqlanadi va shu qiymatga o'zgaruvchi orqali murojaat qilinadi:

```
name = input("Ismni kiriting:")
print ("Salom", ism)
```

Konsol chiqishi:

```
Ismni kiriting: Eugene
Salom Eugene
```

Nazorat savollari:

1. Python paketini qanday o'rnatish mumkin ?
2. Python kodini oddiy matnli faylga saqlash orqali ishga tushirish uchun qanday qadamlarni bajarishim kerak ?
3. Sun'iy intellekt asoslari bilan ishlash uchun qanday tarjimonlardan foydalanish mumkin ?
4. Sun'iy intellekt asoslarining nomi qayerdan kelib chiqqan?
5. Sun'iy intellekt asoslarining tarixi haqida gapirib bering.
6. Sun'iy intellekt asoslari logotipini kim ixtiro qilgan.
7. Pythonni oddiy va tushunarli dasturlash tili nima qiladi?
8. Pythonning boshqa dasturlash tillaridan qanday afzalliklari bor?
9. Sun'iy intellekt asoslari yordamida qanday dasturlar yaratish mumkin?
10. Pythonda dastur strukturasi qanday tuzilgan?
11. Satrning oldida qoldirilgan oraliq masofalar (indents) nimani anglatadi?
12. Pythonning sintaksisi boshqa dasturlash tillarining sintaksisidan eng katta farqli jihati nimada?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

2-mashg'ulot. Pythonda arifmetik operatorlar.

O'quv savollari:

1. Arifmetik operatorlar.
2. Sonlar bilan ishlash. O'zgaruvchilar.
3. Bool tipli ma'lumotlar bilan ishlash.

1. Arifmetik operatorlar.

Python barcha turdagi arifmetik operatsiyalarni qo'llab-quvvatlaydi :

Qo'shish (+):	<code>print(6 + 2)</code>	<code># 8</code>
Ayirish (-):	<code>print(6 - 2)</code>	<code># 4</code>
Ko'paytirish (*):	<code>print(6 * 2)</code>	<code># 12</code>
Bo'linish (/):	<code>print(6/2)</code>	<code># 3.0</code>
Bo'lib butunni olish (//):	<code>print(7 // 2)</code>	<code># 3</code>
Darajaga oshirish (**):	<code>print(6 ** 2)</code>	<code># 6² natija 36</code>
Bo'lib qoldig'ini olish (%):	<code>print(7 % 2)</code>	<code># 1</code>

Agar bitta arifmetik ifodada bir nechta arifmetik amallar ketma-ket foydalanilsa, amallarning prioritetiga muvofiq ketma-ketlikda bajariladi. Birinchi navbatda yuqori ustuvorlikdagi operatsiyalar amalga oshiriladi. Arifmetik amallarning prioritetlarining kamayish tartibida quyidagi 2-jadvalda ko'rsatilgan.

Operatsiya yo'nalishi:

2-jadval

1	<code>**</code>	o'ngdan chapga
2	<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>	Chapdan o'ngga
3	<code>+</code> , <code>-</code>	Chapdan o'ngga

Keling, quyidagi ifodani ko'rib chiqamiz:

```
raqam = 3 + 4 * 5 ** 2 + 7
print(raqam) # 110
```

Bu yerda birinchi navbatda darajaga oshirish amali (`5 ** 2`) bajariladi, keyin natija 4 ga (`25 * 4`) ko'paytiriladi, keyin qo'shish (`3 + 100`) amali bajariladi va keyin yana qo'shish (`103 + 7`) amalga oshiriladi. Shunday qilib natija 110 ga teng bo'ladi.

Qavslar amallar tartibini qayta aniqlash uchun ishlatilishi mumkin:

```
raqam = (3 + 4) * (5 ** 2 + 7)
print(raqam) # 224
```

Shuni ta'kidlash kerakki, arifmetik amallarda ham butun, ham kasr sonlar ishtirok etishi mumkin. Agar butun son (int) va vergulli (kasr) son (float) bir xil amalda ishtirok etsa, u holda butun son float turiga o'tkaziladi.

Arifmetik amallarning tenglik belgisi bilan ishlatilishi

Bu maxsus operatsiyalar amal bajarilgandan keyin natijani birinchi operandga yuklash imkonini beradi:

3-jadval

+=	Qo'shishdan hosil bo'lgan natijani o'zlashtirish
-=	Ayirishdan hosil bo'lgan natijani o'zlashtirish
*=	Ko'paytirishdan hosil bo'lgan natijani o'zlashtirish
/=	Bo'linishdan hosil bo'lgan natijani o'zlashtirish
//=	Bo'lib butunni olishdan hosil bo'lgan natijani o'zlashtirish
**=	Raqam darajaga oshirishdan hosil bo'lgan natijani o'zlashtirish
%=	Bo'lib qoldig'ini oshirishdan hosil bo'lgan natijani o'zlashtirish

Operatsiyalarni dasturda qo'llanilishi:

```

raqam = 10
raqam += 5
print(raqam)    # 10 + 5 = 15
raqam -= 3
print(raqam)    # 15 - 3 = 12
raqam *= 4
print(raqam)    # 12 * 4 = 48
raqam /= 2
print(raqam)    # 48 / 2 = 24.0
raqam //= 5
print(raqam)    # 24 // 5 = 4
raqam **= 3
print(raqam)    # 4 ** 3 = 64
raqam %= 6
print(raqam)    # 64 % 6 = 4

```

Yaxlitlash va "round" funksiyasi

Dasturlashda float tipli raqamlar bilan operatsiyalar bajarayotganda, natijalar aniq biz o'ylaganimiz kabi bo'lmasligi mumkin. Masalan :

```

bir_son = 2.0001
ikki_son = 5
uch_son = bir_son / ikki_son
print(uch_son) # 0.40002000000000004

```

Bunday holda, dasturchi 0,40002 kabi natijani olishni kutdi, lekin oxirida bir qator nollar orqali, qandaydir to'rt paydo bo'lmoqda. Yoki boshqa ifoda:

```
print(2.0001 + 0.2)    # 2.1002000000000003
```

Yuqoridagi holatda, natijani yaxlitlash uchun o'rnatilgan round() funksiyasidan foydalanishimiz mumkin:

```

bir_son = 2.0001
ikki_son = 0.1

```

```
uch_son = bir_son + ikki_son
print(round(uch_son)) # 2
```

Yaxlitlanadigan raqam `round()` funksiyasiga o'tkaziladi. Agar yuqoridagi misoldagi kabi funksiyaga bitta raqam uzatilsa, u butun songa yaxlitlanadi.

`round()` funksiyasi ikkinchi raqamni ham olishi mumkin, natijada olingan sonda nechta o'nli kasr bo'lishi kerakligini ko'rsatadi:

```
bir_son = 2.0001
ikki_son = 0.1
uch_son = bir_son + ikki_son
print(round(uch_son, 4)) # 2.1001
```

Bu holda `uch_son` verguldan keying 4 ta raqamgacha yaxlitlanadi.

Agar funksiyaga faqat bitta qiymat o'tkazilsa - faqat yaxlitlanadigan raqam, u eng yaqin butun songa yaxlitlanadi.

Yaxlitlash misollari:

```
# butun songacha yaxlitlash
print(round(2.49))
print(round(2.51))
```

Biroq, agar yaxlitlanadigan qism ikkita butun sondan bir xil masofada bo'lsa, yaxlitlash eng yaqin juft songa o'tadi:

```
print(round(2.5)) # 2
print(round(3.5)) # 4
```

`round()` funksiyasining ikkinchi parametrini beradigan bo'lsak, verguldan keyin shuncha xonagacha yaxlitlaydi. Yaxlitlagandayam tushirib qoldiriladigan raqam 5 dan katta yoki teng bo'lsa, unda oldingisiga birni qo'shib yaxlitlaydi:

```
# verguldan keyingi 2 ta raqamgacha yaxlitlash
print(round(2.554, 2)) # 2.55
print(round(2.5551, 2)) # 2.56
print(round(2.554999, 2)) # 2.55
print(round(2.499, 2)) # 2.5
```

Biroq, `round()` funksiyasi ideal vosita emasligini yodda tuting. Masalan, yuqorida, butun sonlarga yaxlitlashda, agar yaxlitlanadigan qism ikki qiymatdan bir xil masofada bo'lsa, yaxlitlash eng yaqin juft qiymatgacha amalga oshiriladi, degan qoida qo'llaniladi. Pythonda, sonning kasr qismini suzuvchi raqam sifatida to'g'ri ko'rsatib bo'lmisligi sababli, bu butunlay kutilmagan natijalarga olib kelishi mumkin. Masalan:

```
# verguldan keyingi 2 ta raqamgacha yaxlitlash
print(dumaloq(2.545, 2)) # 2.54
print(dumaloq(2.555, 2)) # 2.56
print(dumaloq(2.565, 2)) # 2.56
print(dumaloq(2.575, 2)) # 2.58
print(dumaloq(2.655, 2)) # 2.65
print(dumaloq(2.665, 2)) # 2.67
print(dumaloq(2.675, 2)) # 2.67
```

2. Sonlar bilan ishlash. O'zgaruvchilar

O'zgaruvchilar

O'zgaruvchilar qiymatlarni saqlash uchun mo'ljallangan dastur elementi. Python tilidagi o'zgaruvchi nomi alifbo belgisi yoki pastki chiziq bilan boshlanishi kerak va alfanumerik belgilar va pastki chiziqni o'z ichiga olishi mumkin. Bundan tashqari, o'zgaruvchining nomi Python tilidagi kalit so'zlarning nomi bilan bir xil bo'lmasligi kerak. Kalit so'zlar ko'p emas, ularni eslab qolish mumkin:

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

O'zgaruvchilarni e'lon qilish quyidagicha amalga oshiriladi:

```
foydalanuvchi = "Tomson"
```

Bu yerda "foydalanuvchi" nomli o'zgaruvchiga "Tom" satri o'zlashtirilmoqda.

Dasturlash tillarida o'zgaruvchilarni nomlashning ikki xil usuli mavjud: **camel case** va **underscore notation**.

Camel case o'zgaruvchi nomidagi har bir keying so'z bosh harf bilan boshlanadi va oldingi so'zga biriktirib yoziladi. Masalan:

```
userName = "Tomson"
```

Underscore notation – bu usulda o'zgaruvchi nomidagi so'zlar pastki chiziq bilan ajratiladi. Masalan:

```
user_name = "Tomson"
```

Shuningdek, siz katta-kichik registr sezgirligini hisobga olishingiz kerak, shuning uchun **ism** va **Ism** o'zgaruvchilari turli obyektlarni ifodalaydi.

```
# ikki xil o'zgaruvchi
ism = "Jenifer"
Ism = "Loktfud"
```

O'zgaruvchilarning o'ziga xos xususiyati shundagi dastur davomida uning qiymati va tipi o'zgartirishi mumkin.

Quyida o'zgaruvchini aniqlab, uni dastur davomida qiymatidan foydalanishni ko'rib chiqamiz:

```
name = "Tomson" # o'zgaruvchi nomi "Tomson" ga teng
print(name)    # print: Tomson
name = "Bobbi" # qiymatni "Bobbi" ga o'zgartiring
print(name)    # chiqish: Bobbi
```

Ma'lumotlar turlari

O'zgaruvchi ma'lumotlar turlaridan birining ma'lumotlarini saqlaydi. Pythonda juda ko'p turli xil ma'lumotlar turlari mavjud. Shulardan eng asosiy turlarni ko'rib chiqamiz: **bool**, **int**, **float** va **str**.

bool (Mantiqiy qiymatlar). bool turli o'zgaruvchilar ikkita mantiqiy qiymatni qabul qiladi: True (rost yoki 1) yoki False (yolg'on yoki 0). True qiymati biror narsaning haqiqat ekanligini ko'rsatish uchun ishlatiladi. False qiymati esa, aksincha biror fikrning yoki narsa noto'g'ri ekanligini ko'rsatadi. Ushbu turdagi o'zgaruvchilardan foydalanishga misol:

```
isMarried = False
print(isMarried)      # False
isAlive = True
print(isAlive)        # True
```

int (Butun sonlar). int turi 1, 4, 8, 50 kabi butun sonlar to'plamidagi sonlarni ifodalaydi. Ya'ni minus cheksizlikdan plus cheksizlikkacha bo'lgan barcha butun sonlar to'plamiga tushuvchi biror bir sonni qabul qiladi. dasturda foydalanishga misol:

```
age = 21
print("Yosh:", age)    # Yosh: 21
count = 15
print("Hisob:", count) # Hisob: 15
```

Odatiy bo'lib, standart raqamlar o'nlik sonlar sifatida qabul qilinadi. Ammo Python ikkilik, sakkizlik va o'n oltilik tizimlardagi raqamlarni ham qo'llab-quvvatlaydi.

Raqam **ikkilik sanoq sistemasida** ekanligini ko'rsatish uchun raqam oldiga **0b** prefiksi qo'yiladi:

```
a = 0b11
b = 0b1011
c = 0b100001
print(a)      # 3 o'nlik sanoq sistemasida
print(b)      # 11 o'nlik sanoq sistemasida
print(c)      # 33 o'nlik sanoq sistemasida
```

Raqam **sakkizlik sanoq sistemasini** ifodalashini ko'rsatish uchun raqamga **00** prefiksi qo'yiladi:

```
a = 007
b = 0011
c = 0017
print(a)      # 7 o'nlik sanoq sistemasida
print(b)      # 9 o'nlik sanoq sistemasida
print(c)      # 15 o'nlik sanoq sistemasida
```

Raqam **o'n oltilik sanoq sistemasini** ifodalashini ko'rsatish uchun raqamga **0x** prefiksi qo'yiladi:

```
a = 0x0A
b = 0xFF
c = 0xA1
print(a)      # 10 o'nlik sanoq sistemasida
print(b)      # 255 o'nlik sanoq sistemasida
print(c)      # 161 o'nlik sanoq sistemasida
```

Shuni ta'kidlash kerakki, qaysi tizimda biz konsolga chiqarish uchun bosib chiqarish funksiyasiga raqam o'tkazsak, u sukut bo'yicha o'nli kasrda chiqariladi.

float (Kasr sonlar). float turi vergulli sonini ifodalaydi va haqiqiy sonlar to'pamiga kiruvchi biror-bir sonni qabul qiladi. masalan: 1,2 yoki 34,76 va hokazo. Butun va kasr qismlar orasidagi ajratuvchi sifatida nuqta ishlatiladi.

```
uzunligi = 1.68
p = 3.14
vazn = 68.
print (uzunligi)    # 1.68
print(p)            # 3.14
print(vazn)         # 68.0
```

kasr sonini eksponensial belgida aniqlash mumkin:

```
x = 3.9e3
print(x)    # 3900.0

x = 3.9e-3
print(x)    # 0.0039
```

Bu yerda $x = 3.9e3$ ifodadagi $e3 = 10^3$ ni anglatadi va $3.9 \cdot 10^3$ ko'payma x o'zgaruvchisiga yuklanmoqda. O'z navbatida $e-3 = 10^{-3}$

str (satrlar tip). str turi satrlarni ifodalaydi. Satr "salom" va 'dunyo' kabi bitta yoki qo'sh tirnoq ichiga olingan belgilar ketma-ketligini ifodalaydi. Python 3.x da satrlar Unicode belgilar to'plamini ifodalaydi

```
hi = "Salom dunyo!"
print (hi)    # Salom dunyo!
ism = "Tomson"
print (ism)   # Tomson
```

O'zgaruvchan tip. Sun'iy intellekt asoslarida o'zgaruvchilarning tipi dinamik tip hisoblanadi. Bu o'zgaruvchining tiplari dastur davomida qiymatning tipiga bog'liq tarzda o'zgarishi mumkinligini anglatadi.

Shunday qilib, ikki yoki bitta tirnoq ichida satr tayinlanganda, o'zgaruvchi str turida bo'ladi. Butun sonni o'zlashtirganda Python avtomatik ravishda o'zgaruvchining turini int sifatida aniqlaydi. O'zgaruvchini float obyekt sifatida aniqlash uchun unga kasr son beriladi, bu yerda kasr va butun qismlarni ajratuvchi sifatida nuqtadan foydalaniladi. Quyida bunga misol keltirilgan:

```
f_Id = "abc"    # satr turiga mansub
print (f_Id)

f_Id = 234    # butun sonlar turiga mansub
print (f_Id)
```

Standart **type()** funksiyasidan foydalanib, o'zgaruvchining joriy tipini bilib olishingiz mumkin:

```
f_Id = "abc"    # tip str
print(type(f_Id)) # <class 'str'>
f_Id = 234    # tip int
print(type(f_Id)) # <class 'int'>
```

3. Bool tipli ma'lumotlar bilan ishlash

Sun'iy intellekt asoslarida ham bir qancha mantiqiy ifodalar mavjud. Ushbu mantiqiy ifodalarning barchasi ikkita o'zgaruvchi bilan ishlatiladi va umumiy natija sifatida bool turidagi mantiqiy qiymat qaytaradi. Mantiqiy qiymatlar ikkita bo'lib, quyidagilar: **True** (rost) va **False** (yolg'on).

Solishtirish operatorlari. Eng oddiy shartli ifodalar ikki qiymatni taqqoslaydigan taqqoslash amallaridir. Python quyidagi taqqoslash operatsiyalarini qo'llab-quvvatlaydi:

4-jadval

==	Ikkala operand teng bo'lsa, True qiymatini qaytaradi. Aks holda False qaytaradi.
!=	Ikkala operand teng bo'lmasa, True qiymatini qaytaradi. Aks holda False qaytaradi.
> (katta)	Agar birinchi operand ikkinchisidan katta bo'lsa, True qiymatini qaytaradi.
< (kichik)	Agar birinchi operand ikkinchidan kichik bo'lsa, True qiymatini qaytaradi.
>= (katta yoki teng)	Birinchi operand ikkinchidan katta yoki teng bo'lsa, True qiymatini qaytaradi.
<= (kichik yoki teng)	Birinchi operand ikkinchidan kichik yoki teng bo'lsa, True qiymatini qaytaradi.

Taqqoslash operatsiyalariga misollar:

```
a = 6
b = 9
res = 6 == 9 # natijani o'zgaruvchiga yuklash
print(res) # False
print(a > b) # False
print(a != b) # True
print(a < b) # True

boolean1 = True
boolean2 = False
print(boolean1 == boolean2) # False
```

Taqqoslash operatsiyalari turli obyektlarni - satrlarni, raqamlarni, mantiqiy amallarni solishtirishi mumkin, ammo operatsiyaning ikkala operandlari ham bir xil turni ifodalashi kerak.

Mantiqiy amallar. Turli murakkablikka ega bo'lgan mantiqiy ifodalarni yaratish uchun mantiqiy operatorlar ishlatiladi. Python quyidagi mantiqiy operatorlarni qo'llab quvvatlaydi:

and (mantiqiy ko'paytirish) operatori ikkita operandga qo'llaniladi:

```
x and y
```


Birinchidan, **and** operatori x ifodasini baholaydi va agar u False bo'lsa, uning qiymati qaytariladi. Agar u True bo'lsa, ikkinchi operand y baholanadi va y ning qiymati qaytariladi.

```
yosh = 20
vazn = 65
res = yosh > 19 and vazn == 65
print(res) # True
```

Bu holda **and** operatori ikkita ifoda natijalarini taqqoslaydi: $yosh > 19$ hamda $vazn == 65$. Agar bu ifodalarning ikkalasi ham True qiymatini qaytarsa, unda **and** operatori ham res o'zgaruvchisiga True qiymatini qaytaradi (rasmiy ravishda oxirgi operandning qiymati qaytariladi).

Lekin **and** operatorining operandlari True va False bo'lishi shart emas. Bu har qanday qiymat bo'lishi mumkin. Masalan:

```
natija = 14 va "s"
print(natija) # s, chunki 14 soni True ga ekvivalent,
#shuning uchun oxirgi operandning qiymati qaytariladi

natija = 0 va "s"
print(natija) # 0, chunki 0 False ga ekvivalent
```

Bunday holda, 0 raqami va bo'sh "" qatori noto'g'ri deb hisoblanadi, qolgan barcha raqamlar va bo'sh bo'lmagan satrlar "True" ga ekvivalentdir.

or (mantiqiy qo'shish) ikkita operandga ham tegishli:

```
x or y
```

Or operatori avval x ifodasini baholaydi va agar u True bo'lsa, uning qiymati qaytariladi. Agar u False bo'lsa, ikkinchi operand y baholanadi va y ning qiymati qaytariladi. Masalan:

```
yosh = 22
natija = yosh > 21 or False
print(natija) # True
```

Shuningdek, **OR** operatori har qanday qiymatlarga qo'llanilishi mumkin. Masalan:

```
res = 4 or "f"
print(res) # 4, chunki 4 rost ga teng, shuning uchun birinchi #operandning
qiymati qaytariladi
natija = 0 or "f"
print(natija) # f, chunki 0 False ga ekvivalent, shuning uchun #oxirgi
operandning qiymati qaytariladi
```

not (mantiqiy inkor - !)

Ifoda False bo'lsa, True qiymatini qaytaradi

```
yosh = 25
isSingle = False
print(not yosh > 21) # False
print(not isSingle) # True
print(not 45) # False
```

```
print(not 0) # True
```

in operatori

Agar ba'zi qiymatlar to'plamida ma'lum qiymat mavjud bo'lsa, **in** operatori "True" ni qaytaradi. U quyidagi shaklga ega:

```
qiymat in [qiymat_to'plamidagi]
```

Masalan, satr belgilar to'plamini ifodalaydi. Va in operatori bilan biz unda biron bir qiymat mos kelishini va mavjudligini tekshirishimiz mumkin:

```
xabar = "hello world!"
hi = "hello"
print(hi in xabar)
zol = "golden"
print(zol in xabar)
```

Agar qiymatlar to'plamida qiymat mavjud masligini tekshirishimiz kerak bo'lsa, unda biz **not in** modifikatsiyasidan foydalanishimiz kerak bo'ladi. Agar qiymat [qiymatlar to'plami] da mavjud bo'lmasa, u True qiymatini qaytaradi:

```
xabar = "hello world!"
hi = "hello"
print(hi not in xabar) # False

golden = " golden "
print(golden not in xabar) # True
```

Nazorat savollari:

1. pythonda qanday arifmetik amallar mavjud?
2. % - qanday amalni bajarish uchun ishlatiladi?
3. // - qanday amalni bajarish uchun ishlatiladi?
4. ** - qanday amalni bajarish uchun ishlatiladi?
5. Kvadrat ildiz qanday ishlatiladi?
6. Oddiy bo'lish amalini bajarilganda qanday tipli qiymat qaytaradi?
7. += , -=, *=, /=, **= va %= - operatorlarining vazifasi qanday?
8. Round() funksiyasining ishlash prinsipini tushuntirib bering.

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

3-mashg'ulot. Satrlar bilan ishlovchi operatorlar va metodlar.

O'quv savollari:

1. Satrlar bilan ishlovchi operatorlar va metodlar.
2. str.format() metodi yordamida satrlarni formatlash.

1. Satrlar bilan ishlovchi operatorlar va metodlar.

Satr deb mavjud belgilar ketma-ketligiga aytiladi. Satrlarni pythonda aniqlash uchun ham bittalik, ham qo'sh tirnoqlardan foydalaniladi. Masalan:

```
msg = 'Salom Dunyo!'
print(msg)    # Salom Dunyo!

nomi="Jeyson"
print(nomi)    # Jeyson
```

Agar satrdagi simvollar soni ko'p bo'lsa, uni qismlarga bo'lish va ularni turli xil kod satrlariga joylashtirish mumkin. Bunday holda, butun satr qavs ichiga olinadi va uning alohida qismlari tirnoq ichiga joylashtiriladi:

```
text = ("Salom hurmatli kursantlar"
        "Bugun biz sizlar bilan"
        "Satrlar ustida amallar bajarib ko'ramiz"
        )
print(text)
```

Agar ko'p qatorli matnni hosil qilish kerak bo'lsa, unda bunday matn ikki tomondan uch qo'sh (""") yoki bittalik (‘‘) tirnoq ichiga olinib yoziladi:

```
'''
Bu izoh
'''

text = '''Salom hurmatli kursantlar
Bugun biz sizlar bilan
Satrlar ustida amallar
bajarib ko'ramiz
'''
print(text)
```

Faqat shuni esda tutish kerakki uchta qo'shtirnoqdan izohlar uchun ham foydalaniladi. Agar uchta qo'shtirnoq ichidagi matn o'zgaruvchiga tayinlangan bo'lsa, unda bu izoh emas, satr hisoblanadi. Izoh qoldirish uchun esa izoh bo'lishi kerak bo'lgan matnni uchta qo'shtirnoqqa o'rab olinadi holos.

Satrdagi qo'llaniladigan maxsus simvollar. Satr bir qator maxsus belgilarni o'z ichiga olishi mumkin. Ulardan ba'zilar quyida keltirilgan:

- ❖ \: satr tarkibiga slesh simbolini qo'shish imkonini beradi
- ❖ \': satr ichiga bitta tirnoq qo'shish imkonini beradi
- ❖ \": qatorga qo'sh tirnoq qo'shish imkonini beradi

- ❖ **\n:** Yangi qatorga o'tadi
- ❖ **\t:** Tabulyatsiya qo'shadi (4 ta probelga teng bo'sh joy)

Quyida ba'zi maxsus belgilardan satr ichida foydalanish bo'yicha misollar keltirilgan:

```
text = "Xabar:\n\"Salom Dunyo\""  
print(text)
```

Dasturning konsoldagi ko'rinishi:

```
Xabar:  
"Salom Dunyo"
```

Garchi bunday maxsus belgilar satrga bittalik yoki qo'sh tirnoq qo'yish, jadval tuzish, boshqa qatorga o'tkazish kabi ko'plab amallarni bajarishga imkon bersada, ba'zi hollarda ular dasturda xatolik keltirib chiqaradi. Masalan:

```
URL = "D:\python_course\name.txt"  
print(URL)
```

Bu yerda URL o'zgaruvchisi faylga yo'lni o'zlashtirib oladi. Biroq, satr ichida "\n" maxsus belgisi mavjud bo'lib, garchi dasturchi bu belgini faylga yo'lni ko'rsatish uchun foydalanayotgan bo'lsada, python interpretatori uni maxsus operator sifatida tarjima qiladi. Shunday qilib satrni ekranga chiqarishda natija quyidagicha bo'ladi:

```
c:\python_course  
ame.txt
```

Bunday vaziyatni oldini olish uchun satr oldiga r belgisi qo'yiladi:

```
URL = r"C:\python_course\name.txt"  
print(URL)
```

```
c:\python_course\name.txt
```

Qatorga qiymatlarni kiritish

Python boshqa o'zgaruvchilarning qiymatlarini satrga joylashtirish imkonini beradi. Buning uchun satr ichida o'zgaruvchilar jingalak qavslarga {} joylashtiriladi va satr oldidagi qo'shtirnoq belgisi oldiga f harfi qo'yiladi:

```
user_name = "Jeyson"  
user_age = 22  
user = f"Ismi: { user_name } Yoshi: { user_age }"  
  
print(user_age) # Ismi: Jeyson Yoshi: 22
```

Bunday holda, user_name o'zgaruvchisining qiymati {user_name} o'rniga kiritiladi. Xuddi shunday, {user_age} o'rniga user_age o'zgaruvchisining qiymati kiritiladi.

Satr tarkibidagi belgilarga murojaat qilish. Sun'iy intellekt asoslarida ham boshqa dasturlash tillaridagi kabi satrning tarkibidagi simvollarga murojaat qilib ular ustida turli amallar bajarish mumkin. Satr tarkibidagi belgilarga murojaat qilish uchun satr_nomi va kvadrat qavslar ichida simvolning indeksi kiritiladi. Indeks bu yerda simvolning satrdagi joylashgan o'zni bo'lib, simvollar satrda 0 dan boshlab indexlangan bo'ladi. Demak satrdagi simvollar soni oxirgi simvolning indeksidan

bittaga ortiq bo'ladi. Quyida satr tarkibidagi belgilarga murojaat qanday amalga oshirilishiga misol keltirilgan:

```
str = "hello world!"
belgi = str[0] # h
print(belgi)

belgi = str[6] # w
print(belgi)

belgi = str[11] # error: IndexError: string index out of range
print(belgi)
```

Agar satrda mavjud bo'lmagan indeksdagi elementga murojaat qilinsa, `IndexError` xatoligi kelib chiqadi. Misol uchun, yuqoridagi holatda, `str` nomli satr 11 ta belgidan iborat va belgilari 0 dan 10 gacha indeksdarga ega. Shuning uchun `str[11]` ifodasi xatolik qaytardi. Chunki satrda 11 ta element va oxirgi elementning indeksi 10 ga teng.

Satr oxiridan boshlab elementlarga murojlat qilish uchun manfiy indekslardan foydalanish mumkin. Shunday qilib, indeks -1 oxirgi belgini va indeks -2 oxirgi belgidan bitta oldingi ya'ni oxiridan 2- belgini ifodalaydi va hokazo. Masalan:

```
str = "salom dunyo!"
belgi = str[-1]
print(belgi) #!
belgi2 = str[-5]
print(belgi2) #u
```

Belgilar bilan ishlashda satr o'zgarmas (immutable) tip ekanligini hisobga olish kerak, shuning uchun satrning har qanday individual belgisini o'zgartirishga harakat qilsak, quyidagi holatda bo'lgani kabi xatoga duch kelamiz:

```
str = "Salom DASTURCHILAR!"
str[1] = "R"
```

`TypeError: 'str' object does not support item assignment`

Faqat satr qiymatini unga boshqa qiymat belgilash orqali butunlay qayta o'rnatishimiz mumkin.

Satrning barcha elementlarini sikl yordamida olish. Satrdagi barcha belgilarni for siklini qo'llash orqali olish mumkin:

```
str = "salom dunyo"
for belgi in str:
    print(belgi)
```

s
a
l
o
m

d

u
n
y
o

Matndan belgilar ketma-ketligini olish. Satr tarkibidan nafaqat bitta belgini, balki belgilar ketma-ketligini ham olish mumkin. Buning uchun quyidagi sintaksis qo'llaniladi:

str[:end] – ushbu ifoda 0 - indeksdan to end - indeksgacha belgilar ketma-ketligini chiqaradi. end – elementni o'zini olmaydi. End o'rnida son beriladi. str[:10]

str[start:end] – belgilar ketma-ketligi **start** dan to **end** - indeksigacha bo'lgan belgilar ketma-ketligini qaytaradi. Ammo ikkinchi index – end xisobga olinmaydi.

string[start:end:step] – belgilar ketma-ketligini **start** dan to **end** - indeksigacha bo'lgan belgilar ketma-ketligini **step** – qadami bilan qaytaradi.

Quyida **str** satrining tarkibidan belgilar ketma-ketligini olish uchun barcha variantlardan foydalanilgan dastur keltirilgan:

```
str = "salom dunyo"

# 0-indeksdan 5 - indeksigacha
sub_str1 = str[:5]
print(sub_str1)      # salom

# 2-indeksdan 5-indeksigacha
sub_str2 = str[2:5]
print(sub_str2)      # llo

# 2 - indeksdan 9-indeksigacha oraliqda 2 qadam bilan
sub_str3 = str[2:9:2]
print(sub_str3)      # Lmdn
```

Satrlarni birlashtirish. Satrlar ustida eng keng tarqalgan amallardan biri ularni birlashtirish yoki qo'shish (konkatinatsiya) qilish. Starlar ustida arifmetik qo'shish amalini bajaradigan bo'lsak, satrlarni birlashtirish amalini bajargan bo'lamiz:

```
name = "Jeyson"
surname = "Stethom"

full_name = name + " " + surname

print(full_name)  # Jeyson Stethom
```

Ikki qatorni birlashtirish oson, lekin agar satr va raqam qo'shishi kerak bo'lsa-chi? Bunday holda **str()** funksiyasidan foydalanib raqamni satrga o'tkazish talab qilinadi:

```
name=" Jeyson "
age = 38

info_per = "Ism: " + name + " Yoshi: " + str(age)
print(info_per)  # Ism: Jeyson Yoshi: 38
```

Satrlarni takrorlash. Satrlarga arifmetik ko'paytirish amalini qo'llash, ularning shuncha marta takrorlaydi:

```
print("char" * 3) # charcharchar
print("u" * 4) # uuuu
```

Satrlarni taqqoslash. Satrlarni taqqoslashda asosiy diqqatni belgiga va ularning katta yoki kichik harfligiga qaratiladi. Demak, raqamli belgi shartli ravishda har qanday alifbo belgisidan kichik hisoblanadi. Katta harfdagi alifbo belgilari shartli ravishda kichik alifbo belgilaridan kichik hisoblanadi. Masalan:

```
str1 = "1a"
str2 = "aa"
str3 = "aa"
print(str1 > str2) #False, chunki str1 ni 1- belgisi raqam
print(str2 > str3) #Rost, chunki str2 ni 1- belgisi kichik harf
```

Shuning uchun "1a" qatori shartli ravishda "aa" qatoridan kichikdir. Avval birinchi belgi solishtiriladi. Agar ikkala satrning bosh belgilari raqamlarni ifodalasa, u holda kichikroq raqam kichikroq hisoblanadi, masalan, "1a" satri "2a" dan kichik.

Agar boshlang'ich belgilar bir xil holatda alifbo belgilarini ifodalasa, unda alifbo bo'yicha tartibiga qaratiladi. Shunday qilib, "aa" "da" dan kichik va "da" "fa" dan kichikdir.

Agar birinchi belgilar bir xil bo'lsa, ikkinchi belgilar solishtiriladi va hokazo.

Satrlarni katta kichik harflarini bir biri bilan solishtirishdan qochish maqsadida ularning harflarini bir xilga keltirib olish mumkin. Ya'ni katta yoki kichik registrga o'tkazib olish mumkin.

lower() funksiyasi satrdagi barcha harflarni kichik harfga, **upper()** funksiyasi esa satrdagi barcha harflarni bosh harfga o'zgartiradi. Bu funkmsiyalar satrning o'ziga o'zgartirish kiritadi.

```
str1 = "Tomas"
str2 = "TOMAS"
print(str1 == str2) # False - satrlar teng emas

print(str1.lower() == str2.lower()) # Rost
```

ord va **len** funksiyalari. Satr Unicode belgilarni o'z ichiga olganligi sababli, Unicode belgisining raqamli qiymatini olish uchun **ord()** funksiyasidan foydalanish mumkin:

```
print(ord("B")) # 66
```

String uzunligini olish uchun **len()** funksiyasidan foydalaniladi:

```
str = "salom dunyo!"
uzunlik = len(str)
print(uzunlik) # 12
```

Satrdagi qidirish. **text in str** ifodasidan foydalanib, **str** qatoridan **text** satrini qidirib topish mumkin. Agar **str** satrida **text** topilsa, u holda ifoda "True" ni qaytaradi, aks holda "False" ni qaytaradi:

```

str = "salom dunyo"
is_true = "salom" in str
print(is_true)      # rost

is_true = "sdunyo" in string
print(is_true)      # False

```

Satr bilan ishlovchi operatorlar va metodlardan foydalanish

Satrlar bilan ishlash uchun ko'p qo'llanladigan asosiy metodlarni ko'rib chiqaylik:

isalpha (): agar satr faqat alfavit belgilaridan iborat bo'lsa, True qaytaradi;

islower (): Agar satr faqat kichik harflardan iborat bo'lsa, True qiymatini qaytaradi;

isupper (): Agar satrdagi barcha belgilar katta harflardan iborat bo'lsa, True qiymatini qaytaradi;

isdigit (): agar satrning barcha belgilari raqam bo'lsa, True qiymatini qaytaradi;

isnumeric(): agar satr raqam bo'lsa, True qiymatini qaytaradi;

startswith(str): agar satr pastki qator str bilan boshlansa, True qiymatini qaytaradi;

endswith(str): agar satr oxiri **str** bilan tugasa, True qiymatini qaytaradi;

low(): satrni kichik harfga o'zgartiradi;

upper(): satrni katta harfga aylantirish;

title(): Satrdagi barcha so'zlarning birinchi harflarini bosh harfga o'zgartiradi;

capitalize(): satrning faqat birinchi so'zining birinchi harfini bosh harf bilan yozadi;

lstrip(): satr boshidagi probellarni o'chirib, satrni tozalaydi;

rstrip(): satr oxiridagi probellarni o'chirib, satrni tozalaydi;

strip(): satrdan oldingi va keyingi bo'shliqlarni olib tashlaydi;

ljust(width): agar satr uzunligi width parametridan kichik bo'lsa, kenglik qiymatini to'ldirish uchun satrning o'ng tomoniga bo'shliqlar qo'shiladi va satrning o'zi oqlanadi;

rjust(width): agar satr uzunligi width parametridan kichik bo'lsa, kenglik qiymatini to'ldirish uchun satrning chap tomoniga bo'shliqlar qo'shiladi va satrning o'zi o'ng tomonga asoslanadi;

center(width): agar satrning uzunligi kenglik parametridan kichik bo'lsa, kenglik qiymatini to'ldirish uchun satrning chap va o'ng tomoniga bo'shliqlar teng ravishda qo'shiladi va satrning o'zi markazlashtiriladi;

find(str[, start [, end]]): satrning ko'rsatilgan intervalidan qism qatorini qidirish. Agar satrdan qism qator topilsa indeksini qaytaradi. Agar qator topilmasa, -1 raqami qaytariladi;

replace(old, new[, num]): satrdagi bir qism qatorni boshqasiga almashtiradi;

split([delimiter[, num]]): ajratuvchiga qarab qatorni qism qatorlarga ajratadi;

join(strs): bir nechta satrni ular orasiga ma'lum ajratuvchi qo'yib, bir qatorga birlashtiradi.

Yuqorida keltirilgan metodlarni dasturda misollar bilan ko'rib chiqaylik. Masalan, agar raqam klaviaturadan kiritilishi kerak bo'lsa, kiritilgan satrni raqamga aylantirishdan oldin isnumeric() usuli yordamida haqiqatda raqam kiritilganligini tekshirishimiz mumkin va agar shunday bo'lsa, raqamga o'girish amalini bajariladi (**isnumeric()** metodi):

```
str = input(" Raqamni kiriting :")
if str.isnumeric():
    raqam = int(str)
    print(raqam)
```

Satr ma'lum bir qism qator bilan boshlanishi yoki tugashini tekshirish (**startswith()** metodi):

```
f = "hello.py"

startswithHello = f.startswith("hello") #True
endswithExe = f.endswith("exe") # False
```

Satrnin boshida va oxiridagi bo'shliqlarni olib tashlash (**strip()** metodi):

```
str = "    Salom dunyo!    "
str = str.strip()
print(str)           # salom  dunyo!
```

Bo'shliqlar bilan satrni to'ldirish (**rjust()** metodi):

```
print("Redmi 7:", "500".rjust(10))
print("Nokia P10:", "360".rjust(10))
```

Konsol chiqishi:

```
Redmi 7:          500
Nokia N10:        360
```

Satrdagi qidirish

Satrdagi qism qatorni topish uchun Pythonda **find()** metodi mavjud, u satrda qism qatorning birinchi paydo bo'lish indeksini qaytaradi. **find()** metodining uchta ko'rinishi mavjud:

find(str): str qism satrni satr boshidan oxirigacha qidiriladi;

find(str, start): start parametri qidiriladigan boshlang'ich indeksni belgilaydi va start -indeksdan oxirigacha bo'lgan simvollar ketma-ketligidan qidiradi;

find(str, start, end): bu holda str qism satri satrning start va end indeksleri orasidan qidiradi.

Agar qism qator topilmasa, find metodi -1 qiymatni qaytaradi:

```
Hi_bye = "Salom dunyo! Xayr dunyo!"
indeks = Hi_bye.find("dun")
print(indeks)           #6

# 10 indeksdan qidirish
```

```

indeks = Hi_bye.find("dun",10)
print(indeks)          #21

# 10 dan 15 gacha indekslarni qidirish
indeks = Hi_bye.find("dun",10,15)
print(indeks)          # -1 - topilmaganini anglatadi

```

Satrd almashtirish (*replace*). Satrdagi bitta str satrini boshqasiga almashtirish uchun **replace()** metodidan foydalaning. Replace metodini bir nechta turda qo'llash mumkin:

replace (old, new): satrdagi **old** satrlarni **new** satr bilan almashtiradi;

replace(old, new, num): **num** parametri **old** satrlarning nechtasini **new** satri bilan almashtirishni belgilab beradi. Odatda **num** parametridan foydalanilmasa, u -1 qiymatni qabul qiladi, bu esa barcha **old** satrlarni **new** satri bilan almashtirilishini anglatadi.

```

tel_num = "+99-899-667-89-10"

# defislarni bo'sh joylar bilan almashtirish
tahrirlangan_telefon = tel_num.replace("-", " ")
print(tahrirlangan_telefon)      # +99 899 667 89 10

# defislarni olib tashlash
tahrirlangan_telefon = tel_num.replace("-", "")
print(tahrirlangan_telefon)      # +998996678910

# faqat birinchi defisni almashtirish
tahrirlangan_telefon = tel_num.replace("-", "", 1)
print(tahrirlangan_telefon)      # +99899-667-89-10

```

Qism qatorlarga ajratish. *Split()* metodi chegaralovchiga qarab qatorni qism qatorlar ro'yxatiga ajratadi. Ajratuvchi har qanday belgi yoki belgilar ketma-ketligi bo'lishi mumkin. Ushbu metod quyidagi ko'rinishlarda qo'llanilishi mumkin:

split(): ajratuvchi sifatida bo'sh joy qilinadi. Va probeldan probelgacha bo'lgan joylarni element qilib oladi.

split(delimeter): cheklovchi sifatida delimiter dan foydalaniladi.

split(delimeter, num): num parametri bo'linish uchun delimiter (ajratuvchi) ning necha marta ishlatilishini belgilaydi. Qatorning qolgan qismi qismlarga ajratilmasdan ro'yxatga qo'shiladi.

```

text = "Bu katta, ikki bo'yli eman, shoxlari singan va po'stlog'i singan"
# probellardan ajratiladi

splitted_text = text.split()
print(splitted_text)
print(splitted_text[6]) #singan

# vergul ustida tanaffus
splitted_text = text.split(",")
print(splitted_text)

```

```
print(splitted_text[1]) # ikki bo'yli eman

# birinchi besh bo'shliqqa bo'lingan
splitted_text = text.split(" ", 5)
print(bo'lingan_matn)
print(bo'lingan_matn[5])
# shoxlari singan va po'stlog'i singan
```

Satrlarni birlashtirish. Satrlar ustidagi eng oddiy amallarni ko'rib chiqq turib, qo'shish amali yordamida qatorlarni birlashtirishni ko'rsatdik. Satrlarni birlashtirishning yana bir imkoniyati - **join() usuli**: u bir nechta satrlarni birlashtirish uchun ishlatiladi.

Bundan tashqari, ushbu metod chaqirilgan joriy satr ajratuvchi sifatida ishlatiladi:

```
strings = ["Let's", "go", "speaking", "from", "my", "heart", "into", " My box"]

# ajratuvchi - bo'sh joy
sentence = " ".join(strings)
print(sentence)
# Let's go speaking from my heart into My box

# ajratuvchi - vertikal chiziq
sentence = " | ".join(strings)
print(sentence)
# Let's | go | speaking | from | my | heart | into | My box
```

Ro'yxat o'rniga oddiy satr qo'shilish usuliga o'tkazilishi mumkin, keyin ajratuvchi ushbu satr belgilari orasiga kiritiladi:

```
str = "hello"
joined_str = "\".join(str)

print(joined_str) # h\l\l\l\o
```

2. str.format() metodi yordamida satrlarni formatlash

Oldingi mavzularda satr oldiga f belgisini qo'yish orqali ba'zi qiymatlarni qanday kiritish mumkinligi ko'rib chiqilgan edi. Masalan:

```
first_name="Jeyson"
text = f"Hello, {first_name}."
print(text) # Hello, Jeyson.

name="Bobo"
age=73
info = f"Ismi: {name}\t Yoshi: {age}"
print(info) # Ismi: Bobo Yoshi: 73
```

Satrga o'zgaruvchining qiymatini kiritish uchun maxsus parametrlardan foydalaniladi, ular jingalak qavslar ({}) bilan o'rab olinadi.

Nomlangan parametrlar. Formatlanayotgan satrga chaqirilayotgan o'zgaruvchilarni **format()** metodiga parametr sifatida beriladi. Format metodi esa

ushbu parametrlarning qiymatlarini satr tarkibida jingalak qavslarda ko'rsatilgan mos nomlarga uzatadi:

```
txt = "Salom, {first_name}.".format(first_name="Jeyson")
print(txt)      # Salom, Jeyson.

info = "Ismi: {name}\t Yoshi: {age}.".format(name="Bobo", age=73)
print(info)     # Ismi: Bobo Yoshi: 73
```

Demak, formatlash metodida parametrlar qatordagi parametrlar bilan satr tarkibidagi jingalak qavslar ichidagi nomlar ham bir xil bo'llishi talab qilinadi. Shunday qilib, agar parametr yuqorida ko'rsatilga dasturda bo'lgani kabi `first_name` deb ataladigan bo'lsa, u holda qiymat tayinlangan argument ham `first_name` deb ataladi.

Joylashgan o'rni bo'yicha parametrlar. Shuningdek, ketma-ket argumentlar to'plamini formatlash usuliga o'tkazish va bu argumentlarni format satrining o'ziga, ularning indeksini jingalak qavslar ichida ko'rsatish mumkin (indekslash noldan boshlanadi):

```
info = "Ismi: {0}\t Age: {1}.".format("Bobo", 73)
print(info)      # Ismi: Bobo Age: 73
```

Bunday holda, argumentlar satrga bir necha marta chaqirilishi ham mumkin:

```
txt = "Salom, {0} {0} {0}.".format("Jeyson")
Salom, Jeyson Jeyson Jeyson
```

Almashtirishlar. Formatlangan qiymatlarni satrga o'tkazishning yana bir usuli - o'rniga ma'lum qiymatlar qo'shiladigan almashtirishlar yoki maxsus to'ldiruvchilardan foydalanish. Formatlash uchun quyidagi to'ldiruvchilardan foydalanish mumkin:

s: String tipli ma'lumot kiritish uchun;

d: Double tipli sonlarni kiritish uchun;

f: Float tipli sonlarni kiritish uchun. Bu tur uchun nuqta orqali kasr sonini aniqlash ham mumkin;

%: qiymat 100 ga ko'paytiriladi va foiz belgisini qo'shib qo'yiladi.

To'ldiruvchining umumiy sintaksisi quyidagicha:

```
{:[maxsus to'ldiruvchi] }
```

To'ldiruvchiga qarab qo'shimcha parametrlar qo'shilishi mumkin. Masalan, float raqamlarini formatlash uchun siz quyidagi variantlardan foydalanishingiz mumkin

```
{:[belgilar_soni][vergul][.sonning_verguldan_keyingi_qismi] to'ldiruvchi}
```

Format metodini chaqirganda, qiymatlar unga argumentlar sifatida uzatiladi, ular to'ldiruvchilar o'rniga kiritiladi. String almashtirish (`{:s}`) ga misol:

```
hi = "Salom sizga {:s}"
name = "Jeyson"
formatted_txt = hi.format(name)
print(formatted_txt)      # Salom sizga Jeyson
```

Format() usuli natijada yangi formatlangan qatorni qaytaradi.

{:d} - butun son formatlashga misol:

```
txt = "{:d} ta simvol"
number = 12
formatted_txt = txt.format(number)
print(formatted_txt) # 12 ta simvol
```

Agar formatlash kerak bo'lgan raqam 999 dan katta bo'lsa, vergulni minglik ajratuvchi sifatida ishlatmoqchi ekanligimizni to'ldiruvchi ta'rifida belgilash ({:,d}) mumkin:

```
txt = "{:,d} belgi"
print(txt.format(300)) # 300 belgi
```

Bundan tashqari, to'ldiruvchilar f-satrlarida ham ishlatilishi mumkin:

```
n = 300
txt = f"{n:,d} belgi"
print(txt) # 300 belgi
```

Kasr sonlar uchun, sonning kasr qismida qancha belgi ko'rinishini aniqlash mumkin. Buning uchun verguldan keyin ko'rinishi kerak bo'lgan sonlar soni, nuqtadan keyin to'ldiruvchidan oldin kiritiladi. Masalan:

```
num = 115.2548695
print("{:.2f}".format(num)) # 115.25
print("{:.3f}".format(num)) # 115.254
print("{:.4f}".format(num)) # 115.2548
print("{:,.2f}".format(12582.23664)) # 12,582.23
```

Yana bir parametr formatlangan qiymatning minimal kengligini belgilar bilan belgilash imkonini beradi:

```
print("{:10.2f}".format(115.2548695)) # 115.25
print("{:8d}".format(25)) # 25
```

format metodini o'rnini bosuvchi **f** usuli:

```
n1 = 115.2548695
print(f"{n1:10.2f}") # 115.25
n2 = 26
print(f"{n2:8d}") # 26
```

Foizlarni ekranga chiqarish uchun "%" kodini ishlatish yaxshiroqdir :

```
num = .123456
print("{:%}".format(num)) # 12.34560000%
print("{:.0%}".format(num)) # 12%
print("{:.1%}".format(num)) # 12.3%

print(f"{num:%}") # 12.34560000%
print(f"{num:.0%}") # 12%
print(f"{num:.1%}") # 12.3%
```

Format metodini ishlatmasdan formatlash. Quyidagi sintaksis bilan formatlashning yana bir usuli ham mavjud:

```
string%(param1, param2,..paramN)
```

Ya'ni, boshida yuqorida ko'rib chiqilgan bir xil to'ldiruvchilarni o'z ichiga olgan qator mavjud (% to'ldiruvchidan tashqari) satrdan keyin foiz - % belgisi qo'yiladi va keyin qatorga kiritiladigan qiymatlar ro'yxati keltiriladi. Aslida, foiz belgisi yangi qatorga olib keladigan operatsiyani anglatadi:

```
info = " Ism : %s \t Yoshi : %d" % ("Jeck", 32)
print(info)      # Ism : Jeck      Yoshi : 32
```

To'ldiruvchidan keyin foiz belgisi qo'yiladi va format funksiyasidan farqli o'laroq, bu yerda jingalak qavslar kerak emas.

Bundan tashqari, bu yerda raqamlarni formatlash usullari ham qo'llaniladi:

```
raqam = 23.8689578
print("%0.2f - %e" % (raqam, raqam))
# 23.87 - 2.386896 e +01
```

Nazorat savollari:

1. Format metodining qo'llanilishi qanday?
2. Format metodining "Nomlangan parametrlar" usulini tushuntirib bering.
3. Format metodining "Pozitsiya bo'yicha parametrlar" usulini tushuntirib bering
4. Almashtirishlar – bu qanday vazifani bajaradi?
5. Format metodisiz formatlash qanday amalga oshiriladi?
6. isalpha () - metodining vazifasi qanday?
7. islower () - metodining vazifasi qanday?
8. isupper () - metodining vazifasi qanday?
9. isdigit () - metodining vazifasi qanday?
10. startswith () - metodining vazifasi qanday?
11. endswith () - metodining vazifasi qanday?
12. replace () - metodining vazifasi qanday?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

4-mashg'ulot. Pythonda shart operatori uning qo'llanilishi.

O'quv savollari:

1. IF, IF-ELSE va IF-ELIF-ELSE operatorlari.
2. Shart operatorlarida mantiqiy ifodalar (and, or, not) ning qo'llanilishi.
3. Ichma-ich (nested) IF operatorlaridan foydalanish, ularning afzallik va kamchiliklari.

1. IF, IF-ELSE va IF-ELIF-ELSE operatorlari.

Shartli operatorlari shartli ifodalardan foydalanadi va ularning qiymatiga qarab, dasturning bajarilishini yo'llardan biri bo'ylab yo'naltiradi. Shunday operatorlardan biri **if** operatoridir. U quyidagi quyidagicha sintaksisga ega:

```
if mantiqiy_amal:
    [if bloki]
elif mantiqiy_amal:
    [elif bloki]
else:
    [else bloki]
```

Eng oddiy shaklda if kalit so'zidan keyin mantiqiy ifoda keladi. Va agar bu mantiqiy ifoda "True" ni qaytarsa, u holda ko'rsatmalarning keyingi bloki bajariladi, ularning har biri blokga tegishlilikini oldiga 4 ta probel yoki 1 ta tab tashlanishi bilan aniqlanadi (4 bo'sh joy yoki bo'shliqlar sonini cheklash maqsadga muvofiqdir) bu 4 ga karrali):

```
til = "uzbekcha"
if til == "uzbekcha":
    print("Assalomu alaykum")
    print("Xayr")
```

Bu holda **til** o'zgaruvchisining qiymati "uzbekcha" bo'lganligi sababli, if bloki bajariladi, unda faqat bitta operator mavjud - print("Assalomu alaykum"). Natijada, konsol quyidagi qatorlarni ko'rsatadi:

```
Assalomu alaykum
Xayr
```

Kodning oxirgi qatoriga e'tibor bering, unda "Xayr" xabari ko'rsatiladi. Chunki print("Xayr") oldidan tab tashlanmagan, shuning uchun u if blokiga tegishli emas va if konstruktsiyasidagi ifoda False qiymatini qaytarsa ham bari bir print("Xayr") amali bajariladi.

Ammo agar biz uni cheklasak, u if konstruktsiyasiga ham tegishli bo'ladi:

```
til = "uzbekcha"
if til == "uzbekcha":
    print("Assalomu alaykum")
    print("Xayr")
else:
```

Mabodo if ifodasi False qiymatini qaytarsa, unda else bloki ishga tushadi:

```
til = "uzbekcha"
if til == "uzbekcha":
    print("Assalomu alaykum")
else:
    print("Hi")
    print("Xayr")
```

Agar `til == "uzbekcha"` iborasi True qiymatini qaytarsa, u holda if bloki bajariladi, aks holda else bloki bajariladi. Va bu holda shart `til == "uzbekcha"` False qiymatini qaytarganligi sababli, else blokidagi operator bajariladi.

Bundan tashqari, else bloki tarkibiga kiruvchi operatorlarning oldidan ham satr boshidan tab (4ta probelga teng bo'sh joy) tashlanishi kerak. Misol uchun, yuqoridagi misolda `print("Xayr")` ning oldidan tab tashlanmagan, shuning uchun u else bloki tarkibiga kiritilmagan va `til` o'zgaruvchisining qiymati qaysi tilda bo'lishidan qat'iy nazar bajariladi. Ya'ni, konsol quyidagi qatorlarni ko'rsatadi:

```
Assalomu alaykum
Xayr
```

else bloki tarkibi bir nechta qatorlardan tashkil topgan bo'lishi ham mumkin:

```
til = "uzbekcha"
if til == "uzbekcha":
    print("Salom")
    print("Dunyo!")
else:
    print("Hello")
    print("World!")
```

Agar siz bir nechta aniq shartlarni kiritishingiz kerak bo'lsa, unda siz qo'shimcha elif bloklaridan foydalanishingiz kerak bo'ladi:

```
til = "uzbekcha"
if til == "uzbekcha":
    print("Assalomu")
    print("Alaykum")
elif til == "german":
    print("Hallo")
    print("Welton")
else:
    print("What's")
    print("up")
```

Python bu yerda birinchi, if ning shartini tekshiradi. Agar rost bo'lsa, if blokidagi operatorlar bajariladi. Agar bu shart False qiymatini qaytarsa, Python elif shartini tekshiradi.

Agar elifning sharli ifodasi True bo'lsa, u holda elif blokidagi opertorlar bajariladi. Ammo agar u ham False qaytarsa, else blokidagi bajariladi.

Shartlarni elif operatoridan foydalangan holda ixtiyoriy davom ettirishingiz mumkin. Masalan :


```

til = "Uz"
if til == "english":
    print("Hello")
elif til == "german":
    print("Hallo")
elif til == "french":
    print("Salut")
else:
    print("Salom")

```

Ichma-ich joylashgan if operatori. if operatorining sharti bajarilganidan keyin yanayam aniqlashtirish uchun ichida yana bir necha shartga tekshiriladi:

```

til = "uzbekcha"
payt == "ertalabki vaqt"
if til == "uzbekcha":
    print("Uzbek")
if payt == "ertalabki vaqt":
    print("Unda hayrli tong!")
elif payt == "kechgi vaqt":
    print("Unda sizga hayrli kech!")

```

Bu yerda if konstruktsiyasi ichki o'rnatilgan if/elif konstruktsiyalarini o'z ichiga oladi. Ya'ni, agar til o'zgaruvchisi "uzbekcha" ga teng bo'lsa, u holda ichki o'rnatilgan if/elif konstruktsiyalarisi kunduzgi o'zgaruvchining qiymatini qo'shimcha ravishda tekshirishadi - payt o'zgaruvchisining qiymati "ertalabki vaqt" ga tengmi yoki "kechgi vaqt" ga tengligini tekshiradi va shunga qarab javob qaytariladi. Va bu holda, biz quyidagi konsol chiqishini olamiz:

```

Uzbek
Unda hayrli tong

```

Esda tutingki, ichki o'rnatilgan if ifodalarining oldidan tab qoldirish kerak, ichki o'rnatilgan konstruktsiyalardagi operatorlarning oldidan ham tab (bo'sh joy) qoldirilishi kerak. To'g'ri joylashtirilmagan tabulyatsiya dastur mantig'ini o'zgartirishi mumkin.

Xuddi shunday, ichki o'rnatilgan if/elif/else konstruktsiyalari elif va else bloklariga joylashtirilishi mumkin:

```

til = "uzbek"
payt = "ertalab"

if til == "uzbek":
    if payt == "ertalab":
        print("Hayrli tong")
    else:
        print("Hayrli kech")
    else:
        if payt == "ertalab":
            print("Good morning")
        else:
            print("Good evening")

```

2. Shart operatorlarida mantiqiy ifodalar (**and**, **or**, **not**) ning qo'llanilishi

Python'da shart operatorlari faqat bitta shartni emas, balki **bir nechta shartlarni birlashtirib** tekshirishga ham imkon beradi. Bunda **mantiqiy operatorlar** — **and**, **or**, **not** ishlatiladi. Ular shartlar orasida bog'lovchi vazifasini bajaradi.

1. and operatori

- **Ma'nosi:** ikkala shart ham **rost** bo'lsa, natija True.
- Aks holda — False.

Misol:

```
yosh = 20
tajriba = 2
if yosh >= 18 and tajriba >= 1:
    print("Ishga qabul qilinadi")
else:
    print("Talabga javob bermaydi")
```

Natija: "Ishga qabul qilinadi" (ikkala shart ham rost bo'lgani uchun).

2. or operatori

- **Ma'nosi:** shartlardan **kamida bittasi rost** bo'lsa, natija True.
- Faqat hammasi yolg'on bo'lsa — False.

Misol:

```
bal = 55
if bal >= 60 or bal == 55:
    print("Imtihondan o'tdi")
else:
    print("O'ta olmadi")
```

Natija: "Imtihondan o'tdi" (chunki ikkinchi shart rost).

3. not operatori

- **Ma'nosi:** shartni **teskari qiymatga** o'zgartiradi.
- not True → False, not False → True.

Misol:

```
parol = ""
if not parol:
    print("Parol kiritilmadi")
else:
    print("Parol qabul qilindi")
```

Natija: "Parol kiritilmadi" (bo'sh satr False, not False → True).

4. Bir nechta operatorlarni birlashtirish

Mantiqiy operatorlar yordamida murakkabroq shartlarni yozish mumkin.

Misol:

```
yosh = 19
bal = 85
if yosh >= 18 and (bal >= 80 or bal == 75):
    print("Grant asosida o'qishga qabul qilindi")
else:
    print("Qabul qilinmadi")
```

Natija: "Grant asosida o'qishga qabul qilindi"

Muhim eslatmalar

1. and operatori **ikkala** shartni tekshiradi, ikkinchisi faqat birinchisi rost bo'lsa qaraladi.
2. or operatori **bittasi rost** bo'lsa yetarli, qolganlari tekshirilmaydi.
3. not operatori shartni **inkor** qiladi, ya'ni natijani teskari qiladi.
4. Murakkab shartlarda qavslar (()) orqali ifodani aniq ko'rsatish tavsiya etiladi.

Amaliy mashqlar

1-savol: Berilgan sonning **musbat va juft** ekanligini aniqlang. Shart bajarilsa "*Musbat juft son*", bajarilmasa "*Shart bajarilmadi*" deb chiqaring.

Javob:

```
x = 8
if x > 0 and x % 2 == 0:
    print("Musbat juft son")
else:
    print("Shart bajarilmadi")
```

2-savol: Foydalanuvchining yoshi 18 dan katta **yoki** u talaba bo'lsa, "*Kirishga ruxsat*" chiqaring, aks holda "*Kirish taqiqlanadi*".

Javob:

```
yosh = 16
talaba = True
if yosh > 18 or talaba:
    print("Kirishga ruxsat")
else:
    print("Kirish taqiqlanadi")
```

3-savol: Foydalanuvchi imtihondan o'tishi uchun shart: ball 60 dan katta **va** hujjati topshirilgan bo'lishi kerak. Shart bajarilsa "*Imtihondan o'tdi*", bajarilmasa "*O'ta olmadi*" chiqaring.

Javob:

```
ball = 75
hujjat = True
if ball >= 60 and hujjat:
    print("Imtihondan o'tdi")
else:
    print("O'ta olmadi")
```

4-Savol: Onlayn do'kondan buyurtma qilish sharti:

- Xarid summasi 200 000 so'mdan yuqori **va** foydalanuvchi "Premium" obunachisi bo'lsa — bepul yetkazib beriladi.

- **Yoki** xarid summasi 500 000 so'mdan yuqori bo'lsa ham — bepul yetkazib beriladi.
- Boshqa hollarda — yetkazib berish pullik.

Natija sifatida "*Bepul yetkazib beriladi*" yoki "*Pullik yetkazib berish*" chiqaring.

Javob:

```
summ = 180000
premium = True

if (summ > 200000 and premium) or summ > 500000:
    print("Bepul yetkazib beriladi")
else:
```

```
print("Pullik yetkazib berish")
```

3. Ichma-ich (nested) IF operatorlari

Ichma-ich IF nima?

- **Ichma-ich IF** — bu bir if bloki ichida yana boshqa if ishlatilishi.
- Ya'ni bir shart bajarilgandan keyin ichkarida yana boshqa shart tekshiriladi.

Oddiy sintaksis:

```
if shart1:
    if shart2:
        # kod bajariladi
```

Misol

```
yosh = 20
tajriba = 2

if yosh >= 18:
    if tajriba >= 1:
        print("Ishga qabul qilinadi")
    else:
        print("Tajribasi yetarli emas")
else:
    print("Yoshi kichik")
```

Avval yosh tekshiriladi. Agar shart rost bo'lsa, ichkarida tajriba ham tekshiriladi.

Afzalliklari

1. **Murakkab shartlarni bosqichma-bosqich tekshirish** mumkin.
2. **Mantiqiy jarayonni** aniq ko'rsatadi: avval bir shart, keyin uning ichida boshqasi.
3. **Qisqa kod** yozishdan ko'ra ba'zi hollarda tushunarliroq bo'lishi mumkin.

Kamchiliklari

1. **Ko'p qavatli** bo'lsa, kodni o'qish qiyinlashadi.
2. **elif bilan yozish mumkin bo'lgan joylarda** ichma-ich if kodni ortiqcha murakkablashtiradi.
3. Katta loyihalarda **xatolik ehtimoli oshadi**.

Qachon ishlatish kerak?

- Shartlar **bir-biriga bog'liq bo'lsa**, masalan: avval foydalanuvchi kirganini tekshirish, keyin parolini tekshirish.
- Agar shartlar mustaqil bo'lsa, ichma-ich emas, balki elif yoki mantiqiy operatorlar (and, or)dan foydalanish qulayroq.

1-savol

Talaba stipendiya olish shartlari:

- Agar talaba kontrakt to'lovini to'lagan bo'lsa, keyin uning o'qish reytingi tekshiriladi.
 - Agar reyting 86 va undan yuqori bo'lsa "Stipendiya beriladi".
 - Aks holda "Stipendiya berilmaydi".
- Agar kontrakt to'lanmagan bo'lsa "Stipendiya olish huquqi yo'q".

Javob:

```
kontrakt_tolandi = True
reyting = 90

if kontrakt_tolandi:
```

```

if reyting >= 86:
    print("Stipendiya beriladi")
else:
    print("Stipendiya berilmaydi")
else:
    print("Stipendiya olish huquqi yo'q")

```

2-savol

Onlayn buyurtma shartlari:

- Agar buyurtma summasi 100 000 soʻmdan katta boʻlsa, yetkazib berish shartlari tekshiriladi:
 - Agar mijoz premium obunachisi boʻlsa — "Bepul yetkazib beriladi".
 - Aks holda "Yetkazib berish pullik".
- Agar buyurtma summasi 100 000 soʻmdan kichik boʻlsa "Minimal summa toʻplanmadi"

chiqaring.

Javob:

```

summ = 120000
premium = False

if summ >= 100000:
    if premium:
        print("Bepul yetkazib beriladi")
    else:
        print("Yetkazib berish pullik")
else:
    print("Minimal summa to'planmadi")

```

3-savol

Kredit olish shartlari:

- Agar mijozning yoshi 21 dan katta boʻlsa, keyin daromadi tekshiriladi:
 - Agar daromadi 3 000 000 soʻmdan koʻp boʻlsa "Kredit tasdiqlandi".
 - Aks holda "Daromad yetarli emas".
- Agar yoshi 21 dan kichik boʻlsa "Yoshi yetarli emas".

Javob:

```

yosh = 25
daromad = 2500000

if yosh > 21:
    if daromad >= 3000000:
        print("Kredit tasdiqlandi")
    else:
        print("Daromad yetarli emas")
else:
    print("Yoshi yetarli emas")

```

4-savol

Sport musobaqasiga qoʻyish shartlari:

- Agar ishtirokchi roʻyxatdan oʻtgan boʻlsa, keyin shifokor ruxsatnomasi tekshiriladi:
 - Agar ruxsatnoma bor boʻlsa "Musobaqaga qoʻyildi".
 - Aks holda "Tibbiy ruxsatnoma yoʻq".
- Agar roʻyxatdan oʻtmagan boʻlsa "Roʻyxatga olinmagan".

Javob:

```
royxatdan_otgan = True
tibbiy_ruxsat = False

if royxatdan_otgan:
    if tibbiy_ruxsat:
        print("Musobaqaga qo'yildi")
    else:
        print("Tibbiy ruxsatnoma yo'q")
else:
    print("Ro'yxatga olinmagan")
```

Nazorat savollari:

1. If operatori qanday vazifasi bajaradi?
2. If operatorining strukturasi qanday?
3. Elif operatorining vazifasi qanday?
4. Ichma – ich foydalaniladigan shart operatoriga misollar keltiring.
5. Bloklarning if ifodaga tegishliligini qanday bilish mumkin?

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

5-mashg'ulot. Pythonda shart operatori satrlarga doir masalalar yechish. Shart operatoriga doir dasturlari tuzish.

O'quv savollari:

1. IF, IF-ELSE va IF-ELIF-ELSE operatorlariga doir dasturlar tuzish.
2. Pythonda satrlarga doir masalalar yechish

IF, IF-ELSE va IF-ELIF-ELSE operatorlariga doir dasturlar tuzish.

Dasturlashda ko'p

incha turli shartlarga qarab qaror qabul qilish talab qilinadi. Masalan:

Foydalanuvchidan kiritilgan son musbatmi yoki manfiymi?

Talaba olgan bali qaysi bahoga teng keladi?

Parol to'g'ri kiritildimi yoki yo'qmi?

Bunday vaziyatlarda Python dasturlash tilida **shart operatorlari** (if, if-else, if-elif-else) qo'llaniladi.

1. IF operatori

if operatori eng sodda shart tekshiruvchi operator hisoblanadi. Shart to'g'ri bo'lsa (True), dastur bajariladi. Agar noto'g'ri (False) bo'lsa, kod bajarilmaydi.

Umumiy ko'rinishi:

```
if shart:
    bajariladigan_kod
```

Misol 1: Musbat sonni aniqlash

```
son = int(input("Son kiriting: "))
if son > 0:
    print("Bu musbat son.")
```

Agar foydalanuvchi 5 kiritisa – ekranga “*Bu musbat son*” chiqadi. Agar -3 kiritisa – hech narsa chiqmaydi.

2. IF-ELSE operatori

if-else operatorida ikkita yo'l mavjud: shart bajarilsa birinchi qism ishlaydi, shart bajarilmasa ikkinchi qism ishlaydi.

Umumiy ko'rinishi:

```
if shart:
    bajariladigan_kod_1
else:
    bajariladigan_kod_2
```

Misol 2: Son juft yoki toq ekanligini aniqlash

```
son = int(input("Son kiriting: "))
if son % 2 == 0:
    print("Bu juft son.")
else:
    print("Bu toq son.")
```

Agar foydalanuvchi 6 kiritsa – “*Bu juft son*”, 7 kiritsa – “*Bu toq son*”.

3. IF-ELIF-ELSE operatori

Agar bir nechta shartlarni ketma-ket tekshirish kerak bo‘lsa, if-elif-else operatori qo‘llaniladi.

Umumiy ko‘rinishi:

```
if shart1:
    bajariladigan_kod_1
elif shart2:
    bajariladigan_kod_2
elif shart3:
    bajariladigan_kod_3
else:
    bajariladigan_kod_n
```

Misol 3: Baho aniqlash dasturi

```
baho = int(input("Bahoni kiriting: "))
if baho >= 90:
    print("A'lo")
elif baho >= 70:
    print("Yaxshi")
elif baho >= 50:
    print("Qoniqarli")
else:
    print("Qoniqarsiz")
```

Agar foydalanuvchi 95 kiritsa – “*A’lo*”, 75 kiritsa – “*Yaxshi*”, 40 kiritsa – “*Qoniqarsiz*”.

4. Qo‘shimcha amaliy misollar

Misol 4: Foydalanuvchi yoshi bo‘yicha ruxsat berish

```
yosh = int(input("Yoshingizni kiriting: "))

if yosh >= 18:
    print("Sizga saytga kirishga ruxsat beriladi.")
else:
    print("Kechirasiz, siz hali voyaga yetmagansiz.")
```

Misol 5: Maksimal sonni topish

```
a = int(input("a sonini kiriting: "))
b = int(input("b sonini kiriting: "))

if a > b:
    print("a katta")
elif b > a:
    print("b katta")
else:
    print("Ikkalasi teng")
```

Misol 6: Haroratni tahlil qilish

```
t = int(input("Haroratni kiriting: "))

if t < 0:
```



```

    print("Havo juda sovuq")
elif t < 15:
    print("Havo salqin")
elif t < 30:
    print("Havo iliq")
else:
    print("Havo issiq")

```

Python dasturlash tilida **satrlar (string)** juda muhim ma'lumot turi hisoblanadi. Satrlar ustida turli amallar bajarish orqali matnlarni qayta ishlash, tahlil qilish va tekshirish mumkin.

Python'da satrlar bitta ' ' yoki qo'shtirnoq " " orasida yoziladi:

matn = "Salom, Dunyo!"

Asosiy amallar

len(satr) → satr uzunligini qaytaradi.

satr[i] → satrdagi i-indeksli belgini qaytaradi.

satr.lower() → hamma harflarni kichik qiladi.

satr.upper() → hamma harflarni katta qiladi.

satr[::-1] → satrni teskari qiladi.

"a" in satr → satrda a belgisi mavjudligini tekshiradi.

Amaliy masalalar

Masala 1: Satr uzunligini hisoblash

```

matn = input("Matn kiriting: ")
print("Matn uzunligi:", len(matn))
Foydalanuvchi "Python" kiritrsa → chiqadi: Matn uzunligi: 6

```

Masala 2: So'zda ma'lum harf bor-yo'qligini tekshirish

```

soz = input("So'z kiriting: ")
if "a" in soz:
    print("So'zda 'a' harfi mavjud.")
else:
    print("So'zda 'a' harfi yo'q.")
"salom" kiritilrsa → So'zda 'a' harfi mavjud

```

Masala 3: Palindrom so'z tekshirish

(Palindrom – teskari o'qiganda ham bir xil so'z, masalan: *kok*, *level*, *radar*).

```

soz = input("So'z kiriting: ")
if soz == soz[::-1]:
    print("Bu palindrom so'z.")
else:
    print("Bu palindrom emas.")
"level" → Bu palindrom so'z.

```

Masala 4: Foydalanuvchi ismining bosh harfini tekshirish

```

ism = input("Ismingizni kiriting: ")
if ism[0].isupper():
    print("Ism katta harf bilan yozilgan.")
else:
    print("Ism kichik harf bilan yozilgan.")

```

"Ali" → *Ism katta harf bilan yozilgan.*

Masala 5: Satrni faqat kichik yoki katta harfga o'tkazish

```
matn = input("Matn kiriting: ")
print("Katta harflarda:", matn.upper())
print("Kichik harflarda:", matn.lower())
"Python" → PYTHON va python
Masala 6: Satrdagi unli harflar sonini topish
matn = input("Matn kiriting: ")
unlilar = "aeiouAEIOU"
soni = 0

for belgi in matn:
    if belgi in unlilar:
        soni += 1

print("Unli harflar soni:", soni)
"Salom" → Unli harflar soni: 2
```

Masala 7: Satr ichida so'z qidirish

```
matn = "Bugun havo juda yaxshi"
soz = input("Qaysi so'zni qidiramiz? ")
if soz in matn:
    print("So'z matnda mavjud.")
else:
    print("So'z topilmadi.")
havo kiritilsa → So'z matnda mavjud.
```

Masala 8: So'zlar sonini hisoblash

```
matn = input("Matn kiriting: ")
sozlar = matn.split()
print("So'zlar soni:", len(sozlar))
```

"Men Python o'rganaman" → *So'zlar soni: 3*

Masala 9: Eng uzun so'zni topish

```
matn = input("Matn kiriting: ")
sozlar = matn.split()
uzun = max(sozlar, key=len)
print("Eng uzun so'z:", uzun)
```

"Python dasturlash tilidir" → *Eng uzun so'z: dasturlash*

Masala 10: Har bir so'zning birinchi harfini katta qilish

```
matn = input("Matn kiriting: ")
print(matn.title())
```

"men python o'rganaman" → *Men Python O'rganaman*

Nazorat savollari :

1. Python'da satr (string) nima va u qanday yoziladi?
2. len() funksiyasining vazifasi nima?
3. Satrlarni indekslash va kesish (slicing) nima uchun ishlatiladi? Misol keltiring.
4. Quyidagi metodlarning vazifasini tushuntiring: .upper() .lower() .title() .split()
5. "in" operatori satrlarda qanday ishlaydi? Misol keltiring.S

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

6-mashg'ulot. Pythonda takrorlanuvchi jarayonlarni dasturlash.

O'quv savollari:

1. Sikl operatorlari – For va while bilan ishlash.
2. Break, continue va else operatorlarining qo'llanilishi.

1. Sikl operatorlari – For va while bilan ishlash.

Sikllar biror bir shartning bajarilishiga qarab ba'zi harakatlarni bajarishga imkon beradi. Pythonda quyidagi turdagi sikllar mavjud: **while** va **for**.

WHILE sikli. while sikli qandaydir bir shartning to'g'riligini tekshiradi va agar shart True bo'lsa, u holda sikl tanasidagi blokni bajaradi. U quyidagicha sintaksisga ega:

```
while <shartli_ifoda>:
    [sikl tanasi]
```

while kalit so'zidan keyin shartli ifoda keladi va bu ifoda True qiymat qaytarsa, sikl tanasiga kirib u yerdagi operatorlar ketma-ket bajariladi.

while sikliga tegishli barcha bloklar oldidan bo'sh joy qoldirilib ketishi va boshlanish qismlari bir ustunga joylashgan bo'lishi kerak va while kalit so'zidan bir xilda chekingan bo'lishi kerak.

```
son = 1
while son < 5:
    print(f"son = {son}")
    son += 1
    print("Dastur tugatildi")
```

Bunday holda, **son** o'zgaruvchisi 5 dan kichik bo'lsa, while sikli ishlaydi. Aks holda sikldan keyingi amal bajariladi.

Bu sikl tanasi ikkita ifodadan iborat:

```
print(f"son = {son}")
son += 1
```

Shuni ham yodda tutingki, oxirgi print(“Dastur tugallandi”) satr boshidan chekinmaydi, shuning uchun u while siklining bir qismi emas.

Siklning butun jarayonini quyidagicha ifodalash mumkin: Birinchi, **son** o'zgaruvchisining qiymati 5 dan kichik yoki kichik emasligini tekshiriladi. Va o'zgaruvchi dastlab 1 ga teng bo'lganligi sababli, bu shart True ni qaytaradi va shuning uchun sikl operatorlari bajariladi.

Sikl ko'rsatmalari satr **son=1** ni konsolga chiqaradi. Va keyin raqam o'zgaruvchisining qiymati bittaga oshiriladi - endi u 2 ga teng bo'ladi. sikl ko'rsatmalari blokini bir marta bajarish iteratsiya deb ataladi. Ya'ni, shu tarzda, birinchi takrorlash siklda amalga oshiriladi.

son < 5 sharti yana tekshiriladi. Yana True qaytariladi, chunki son 2 ga teng, shuning uchun sikl operatorlari yana bir marta bajariladi.

Sikl ko'rsatmalari konsolga satr **son** = 2 ni chiqaradi. Va keyin **son** o'zgaruvchisining qiymati yana bittaga oshiriladi - endi u 3 ga teng bo'ldi. Shunday qilib, ikkinchi takrorlash amalga oshiriladi.

son < 5 shart yana tekshiriladi. Yana True qiymat qaytariladi, chunki son = 3, shuning uchun sikl operatorlari bajariladi.

Sikl ko'rsatmalari konsolga satr **son** = 3 ni chiqaradi. Va keyin **son** o'zgaruvchisining qiymati yana bittaga oshiriladi - endi u 4 ga teng. Ya'ni uchinchi iteratsiya amalga oshiriladi.

son < 5 sharti yana tekshiriladi. Yana True qiymat qaytariladi, chunki son = 4, shuning uchun sikl ko'rsatmalari bajariladi.

Sikl ko'rsatmalari konsolga son= 4 qator raqamini chiqaradi. Va keyin son o'zgaruvchisining qiymati yana bittaga oshiriladi - endi u 5 ga teng. Ya'ni to'rtinchi takrorlash amalga oshiriladi.

Va shart son < 5 yana tekshiriladi. Lekin endi u False qiymat qaytaradi, chunki son o'zgaruvchisining qiymati endi 5 ga teng, shuning uchun sikl tugatiladi. Shundan so'ng sikldan keyin operatorlar bajariladi. Shunday qilib, bu sikl to'rtta o'tish yoki to'rtta takrorlashni amalga oshiradi

Natijada, kodni bajarishda biz quyidagi konsol chiqishini olamiz:

```
son = 1
son = 2
son = 3
son = 4
Dastur tugatildi
```

While sikli bilan yana bir oprator – else ham ishlatiladi. U while siklidagi shart False qiymat qaytarganida ishga tushadi va tarkibidagi blok bajariladi:

```
raqam = 1
while raqam < 5:
    print(f"raqam = {raqam}")
    raqam += 1
else:
    print(f"raqam = {raqam}. sikl tugallandi")
    print("Dastur tugatildi")
```

Ya'ni, bu holda birinchi navbatda shart tekshiriladi va while operatorlari bajariladi. Keyin shart False ga aylanganda else blokidagi gaplar bajariladi. E'tibor bering, else blokidagi iboralar ham sikl konstruksiyasining boshidan chekinadi. Natijada, bu holda biz quyidagi konsol chiqishini olamiz:

```
raqam = 1
raqam = 2
```

```

raqam = 3
raqam = 4
raqam = 5. sikl tugallandi
Dastur tugadi

```

For sikli. Sikl ning yana bir turi for konstruktsiyasidir. Ushbu sikl qiymatlar to‘plamini takrorlaydi, har bir qiymatni o‘zgaruvchiga qo‘yadi va keyin siklda biz ushbu o‘zgaruvchi bilan turli harakatlarni bajarishimiz mumkin. For siklining strukturasi quyidagicha:

```

for [o‘zgaruvchi] in [qiymatlar_to‘plami]:
[sikl tanasi]

```

For kalit so‘zidan keyin qiymatlar joylashtiriladigan o‘zgaruvchining nomi keladi. Keyin in operatoridan keyin qiymatlar to‘plami va ikki nuqta qo‘yiladi.

Va keyingi qatordan sikl tanasidagi operatorlar boshlanadi. Bu operatorlar siklning bir qismi ekanligi uchun ular for kalit so‘zi boshidan ichkariga kiritilishi kerak.

Siklni bajarishda Python ketma-ket barcha qiymatlarni to‘plamdan oladi va ularni o‘zgaruvchiga uzatadi. To‘plamdagi barcha qiymatlar takrorlangandan keyin sikl tugaydi.

Masalan, qiymatlar to‘plami sifatida siz aslida belgilar to‘plamini ifodalovchi satrni ko‘rib chiqishingiz mumkin. Keling, bir misolni ko‘rib chiqaylik :

```

string = "Hello"
for char in string:
    print(char)

```

char o‘zgaruvchisi siklda aniqlanadi, **in** operatoridan keyin **string** o‘zgaruvchisi "Hello" satrini saqlaydigan takrorlangan to‘plam sifatida ko‘rsatiladi. Natijada, for sikli **string** tarkibidagi barcha belgilarni ketma-ketlikda takrorlaydi va ularni **char** o‘zgaruvchisiga o‘zlashtiradi. Sikl blokining o‘zi **char** o‘zgaruvchining qiymatini konsolga chop etadigan bitta funksiyadan iborat. Dasturning konsol chiqishi:

```

H
e
l
l
o

```

For siklida sikl tugagandan keyin bajariladigan qo‘shimcha else bloki ham bo‘lishi mumkin:

```

string = "Hello"
for char in string:
    print(char)
else:
    print(f"oxirgi simvol: {char}")
    print("Dastur yakunlandi")

```

Bunday holda, biz quyidagi konsol chiqishini olamiz:

```

H

```

```
e
l
l
o
oxirgi simvol: o
Dastur yakunlandi
```

Shuni ta'kidlash kerakki, `else` bloki `for` siklida aniqlangan barcha o'zgaruvchilarga kirish huquqiga ega.

Ichma-ich sikllar. Ba'zi sikllar o'zlarida boshqa sikllarni o'z ichiga olishi mumkin. Ko'paytirish jadvalining chiqishi misolini ko'rib chiqing:

```
a = 1
b = 1
while a < 10:
    while b < 10:
        print(a * b, end="\t")
        b += 1
    print("\n")
    b = 1
    a += 1
```

Tashqi sikl `while a < 10`: `a` o'zgaruvchisi 10 ga teng bo'lgunga qadar 9 marta ishlaydi. Bu sikl ichida ichki sikl `while b < 10`: ishlaydi. `b` o'zgaruvchisi 10 ga teng bo'lgunga qadar ichki sikl ham 9 marta ishlaydi. Bundan tashqari, ichki siklning barcha 9 ta takrorlanishi tashqi siklning bir iteratsiyasi ichida ishga tushiriladi.

Ichki halqaning har bir iteratsiyasida konsolda `a` va `b` raqamlarining mahsuloti ko'rsatiladi. Keyin `b` o'zgaruvchining qiymati bittaga oshiriladi. Ichki sikl o'z ishini tugatgandan so'ng, `b` o'zgaruvchining qiymati 1 ga o'rnatiladi va `a` o'zgaruvchining qiymati bittaga oshiriladi va tashqi siklning keyingi takrorlanishiga o'tish sodir bo'ladi. Va `a` o'zgaruvchisi 10 ga teng bo'lgunga qadar hamma narsa takrorlanadi. Shunga ko'ra, ichki sikl tashqi siklning barcha iteratsiyasi uchun faqat 81 marta ishlaydi. Natijada, biz quyidagi konsol chiqishini olamiz:

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63

8	16	24	32	40	48	56	64	72
---	----	----	----	----	----	----	----	----

9	18	27	36	45	54	63	72	81
---	----	----	----	----	----	----	----	----

for siklini xuddi shunday aniqlanishi mumkin:

```
for c1 in "ab":
    for c2 in "ba":
        print(f"{c1}{c2}")
```

Bunday holda, tashqi sikl "ab" qatoridan o'tadi va har bir belgini c1 o'zgaruvchisiga qo'yadi. Ichki sikl "ba" satrini takrorlaydi, satrdagi har bir belgini c2 o'zgaruvchisiga qo'yadi va ikkala belgi kombinatsiyasini konsolga chiqaradi. Ya'ni, oxirida biz a va b belgilarning barcha mumkin bo'lgan kombinatsiyalarini olamiz:

```
ab
aa
bb
ba
```

2. Break, continue va else operatorlarining qo'llanilishi

Biz siklni boshqarish uchun maxsus break va continue operatorlaridan foydalanishimiz mumkin. Break operatori siklni to'xtatadi. continue operatori esa siklning keyingi iteratsiyasiga o'tkazadi.

Agar siklda uning keyingi bajarilishiga mos kelmaydigan shartlar hosil bo'lsa, break operatoridan foydalanish mumkin. Quyidagi misolni ko'rib chiqing :

```
raqam = 0
while raqam < 5:
    raqam += 1
    if raqam == 3: # agar raqam = 3 bo'lsa, sikldan chiqamiz
        break
    print(f"raqam = {raqam}")
```

Bu yerda while sikli raqam<5 shartini rostlikka tekshiradi. Va agar raqam o'zgaruvchisining qiymati hozircha 5 ga teng bo'lmasa, raqamning qiymati konsolda chop etiladi. Ammo ungacha sikl ichida yana bir shart ham tekshiriladi: if raqam == 3. Ya'ni **raqam**ning qiymati 3 ga teng bo'lsa, u holda break operatori yordamida sikldan chiqamiz. Va oxirida biz quyidagi konsol chiqishini olamiz:

```
raqam = 1
raqam = 2
```

Break operatoridan farqli o'laroq, **continue** operatori siklning keyingi iteratsiyasiga uni tugatmasdan o'tadi. Misol uchun, oldingi misolda break ni continue bilan almashtiramiz:

```
raqam = 0
while raqam < 5:
    raqam += 1
    if raqam == 3: # agar raqam = 3 bo'lsa, siklning yangi iteratsiyasiga o'ting
        continue
    print(f"raqam = {raqam}")
```

Va bu holda, agar raqam o'zgaruvchisining qiymati 3 ga teng bo'lsa, continue operatori siklni keyingi aylanishga o'tkazib yuboradi va navbatdagi qadam amalga oshiriladi:

```
raqam = 1  
raqam = 2  
raqam = 4  
raqam = 5
```

Nazorat savollari :

1. Dasturlashda takrorlanuvchi jarayonlarni dasturlash qanday ro'l egallaydi?
2. Takrorlanuvchi jarayonlarni qanday operatorlar yordamida dasturini tuziladi?
3. **for** operatorining ishlash prinsipi qanday?
4. **while** operatorining sintaksisi va ishlash prinsipi qanday?
5. **for** va **while** operatorlarining bir biridan asosiy farqlari nimada?
6. **else** operatorini siklik jarayonlarda nima maqsadda foydalaniladi?
7. **range** operatorining vazifasi va ishlash prinsipi qanday?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

7-mashg'ulot. Pythonda for,while, break va continue operatorlariga doir dasturlar tuzish.

O'quv savollari:

1. Sikl operatorlari – For va while ga doir dasturlar tuzish.
2. Break, continue va else operatorlariga doir dasturlartuzish.

SIKL OPERATORLARI – FOR VA WHILE GA DOIR DASTURLAR TUZISH

Dasturlashda ko‘p hollarda bir xil amalni qayta-qayta bajarishga to‘g‘ri keladi.

Masalan:

1 dan 100 gacha sonlarni chiqarish,

Ro‘yxat elementlarini ketma-ket ekranga chiqarish,

Foydalanuvchi kiritgan qiymatlarni qayta-qayta tekshirish.

Agar bunday vazifalarni odatiy usulda yozsak, kod juda uzun va chalkash bo‘ladi. Shu sababli **sikl operatorlaridan** foydalanamiz.

Sikl operatorlari dasturda biror amalni takror-takror bajarish imkonini beradi.

Python dasturlash tilida ikkita asosiy sikl operatori mavjud:

1. **for sikli** – ketma-ketlik elementlari ustida ishlaydi.

2. **while sikli** – berilgan shart rost (True) bo‘lganda ishlaydi.

for sikli ketma-ketlik (masalan: ro‘yxat, qator, range()) elementlarini bittalab olib, ular ustida amal bajaradi.

```
for o'zgaruvchi in ketma_ketlik:
    bajariladigan_amallar
```

3.3. Misollar

Misol 1. 1 dan 5 gacha sonlarni chiqarish:

```
for i in range(1, 6):
    print(i)
```

Misol 2. So‘zdagi harflarni chiqarish:

```
matn = "Python"
for harf in matn:
    print(harf)
```

Misol 3. Ro‘yxat elementlari yig‘indisini hisoblash:

```
sonlar = [2, 4, 6, 8, 10]
yigindi = 0
for son in sonlar:
    yigindi += son
print("Yig'indi:", yigindi)
```

While sikli

while sikli shart rost bo‘lgan holatda ishlaydi. Agar shart noto‘g‘ri (False) bo‘lsa, sikl tugaydi.

Sintaksis

```
while shart:
    bajariladigan_amallar
```

Misollar

Misol 1. 1 dan 5 gacha sonlarni chiqarish:

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

Misol 2. Foydalanuvchi 0 kiritmaguncha sonlarni qabul qilish:

```
son = int(input("Son kiriting (0 tugatadi): "))
while son != 0:
    print("Siz kiritgan son:", son)
    son = int(input("Yana son kiriting (0 tugatadi): "))
print("Dastur tugadi.")
```

Misol 3. 1 dan 10 gacha sonlar yig'indisini hisoblash:

```
i = 1
yigindi = 0
while i <= 10:
    yigindi += i
    i += 1
print("Yig'indi:", yigindi)
```

For va While sikllarini taqqoslash

Xususiyat	For sikli
Takrorlanish soni oldindan ma'lum bo'lsa ishlatiladi	Takrorlanish sharti aniq bo'lganda ishlatiladi
Odatda range(), ro'yxat yoki matn bilan ishlatiladi	Odatda foydalanuvchi kiritishi yoki cheksiz shartlar bilan ishlatiladi
Qisqa va sodda sintaksisga ega	Ba'zan uzunroq, ammo moslashuvchanroq
Cheksiz siklga tushishi qiyin	Agar shart yangilanmasa cheksiz siklga tushib qoladi

Amaliy topshiriqlar (Mashqlar)

- 1 dan 20 gacha bo'lgan juft sonlarni for yordamida chiqarish.
- 1 dan 50 gacha bo'lgan sonlar yig'indisini while yordamida topish.
- Foydalanuvchidan 5 ta son olib, ularning eng kattasini aniqlash.
- Matn ichidagi unli harflar sonini for yordamida hisoblash.
- 1 dan 100 gacha bo'lgan sonlardan faqat 3 ga bo'linadiganlarini chiqarish.
- For sikli yordamida ko'paytirish jadvalini (1×1 dan 10×10 gacha) chiqarish.

Break, continue va else operatorlariga doir dasturlar tuzish

Sikl operatorlari (for va while) ko'p hollarda boshqa yordamchi operatorlar bilan ishlatiladi. Python tilida sikllarni boshqarish uchun quyidagi qo'shimcha operatorlar mavjud:

- break** – siklni butunlay to'xtatadi.
- continue** – joriy iteratsiyani tashlab o'tadi va keyingi takrorlanishga o'tadi.
- else** – sikl tugagach (agar break ishlatilmagan bo'lsa) bajariladi.

Bu operatorlar sikllarni yanada moslashuvchan va qulay ishlatishga imkon beradi.

Break operatori

break operatori ishlatilsa, sikl darhol to'xtaydi va sikldan tashqariga chiqadi.

Misollar

Misol 1. 1 dan boshlab sonlarni chiqarish, lekin 10 ga yetganda siklni to'xtatish:

```
for i in range(1, 20):
    if i == 10:
        break
    print(i)
Natija: 1, 2, 3, 4, 5, 6, 7, 8, 9
```

Misol 2. While siklida foydalanuvchi stop so'zini kiritrsa sikl to'xtasin:

```
while True:
    matn = input("So'z kiriting (stop tugatadi): ")
    if matn == "stop":
        break
    print("Siz kiritdingiz:", matn)
```

Continue operatori

continue operatori ishlatilsa, siklning joriy bosqichi tashlab yuboriladi va sikl keyingi iteratsiyadan davom etadi.

Misollar

Misol 1. 1 dan 10 gacha bo'lgan sonlardan faqat toqlarini chiqarish:

```
for i in range(1, 11):
    if i % 2 == 0:
        continue
    print(i)
Natija: 1, 3, 5, 7, 9
```

Misol 2. Foydalanuvchi bo'sh satr kiritrsa, uni tashlab yuborish:

```
for i in range(5):
    matn = input("So'z kiriting: ")
    if matn == "":
        continue
    print("Siz kiritdingiz:", matn)
```

Else operatori

else operatori sikl tugagach (ya'ni shart bajarilmay qolsa yoki elementlar tugasa) bajariladi. Ammo agar **break** ishlatilsa, else bajarilmaydi.

Misollar

Misol 1. 1 dan 5 gacha sonlarni chiqarib bo'lgach "Sikl tugadi" degan xabar chiqsin:

```
for i in range(1, 6):
    print(i)
else:
    print("Sikl tugadi")
```

Misol 2. Foydalanuvchidan son qabul qilib, agar u manfiy bo'lsa siklni to'xtatish, aks holda oxirida xabar chiqarish:

```
for i in range(5):  
    son = int(input("Son kiriting: "))  
    if son < 0:  
        print("Manfiy son kiritildi!")  
        break  
else:  
    print("Barcha sonlar musbat kiritildi.")
```

Nazorat savollari:

1. Sikl operatorlari nima va ular dasturlashda qanday maqsadda qo'llaniladi?
2. Python dasturlash tilida nechta asosiy sikl operatori mavjud va ular qaysilar?
3. for sikli qanday ishlaydi? Uning umumiy sintaksisini yozing.
4. while sikli qanday ishlaydi? Uning umumiy sintaksisini yozing.
5. for va while sikllarining asosiy farqlari nimalardan iborat?
6. break operatori nima vazifa bajaradi? Misol keltiring.
7. continue operatori nima vazifa bajaradi? Misol keltiring.
8. else operatori sikllarda qanday ishlaydi? U qaysi holatlarda bajarilmaydi?
9. Siklda cheksiz aylanib qolish sabablari nimalar bo'lishi mumkin?
10. For va while sikllari dastur samaradorligiga qanday ta'sir qiladi?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

8-mashg'ulot. Pythonda ro'yxatlar va ular bilan ishlovchi metoddlar

O'quv savollari:

1. Ro'yxatlar va ularning qo'llanilishi.
2. Ro'yxatlarni yaratish usullari.
3. Ro'yxatlar bilan ishlovchi metodlar.

1. Ro'yxatlar va ularning qo'llanilishi.

Ma'lumotlar to'plamlari bilan ishlash uchun Python ro'yxat, kortej va lug'atlar kabi o'rnatilgan turlarni taqdim etadi.

Ro'yxat (list) - elementlar to'plami yoki ketma-ketligini saqlaydigan ma'lumotlar turi. Ko'pgina dasturlash tillarida massiv deb ataladigan o'xshash ma'lumotlar tuzilmasi mavjud.

2. Ro'yxatlarni yaratish usullari

Ro'yxatni yaratish uchun kvadrat qavslar `[]` ishlatiladi. Ularning ichida ro'yxat elementlari vergul bilan ajratilgan holda saqlanadi. Masalan, raqamlar ro'yxatini yarataylik:

```
raqamlar = [0, 2, 12, 44, 51]
```

Xuddi shunday, boshqa turdagi ma'lumotlar bilan ro'yxatlarni belgilash mumkin, masalan, satrlar (ya'ni string turdagi ma'lumotlar) ro'yxatini yaratish quyidagicha amalga oshiriladi:

```
odamlar = ["Jasur", "Akmal", "Bobur"]
```

Ro'yxat yaratish uchun **list()** konstruktor funksiyasidan ham foydalanish mumkin :

```
raqamlar1 = []
raqamlar2 = list()
```

Ushbu ikkala ro'yxat ta'riflari o'xshash - ular bo'sh ro'yxat yaratadi.

Ro'yxatda faqat bir xil turdagi obyektlar bo'lishi shart emas. Bir vaqtning o'zida satrlarni, raqamlarni va boshqa ma'lumotlar turlarining obyektlarini bir xil ro'yxatga joylashtirish mumkin:

```
obyektlar = [1, 2.26, "Salom dunyo", True, False]
```

Ro'yxat elementlarini ekranga chiqarish uchun standart print funksiyasidan foydalanish mumkin. Bu ro'yxat tarkibini to'liq shaklda konsolga chop etadi:

```
raqamlar = [1, 2, 3, 4, 5]
odamlar = ["Jasur", "Akmal", "Bobur"]

print(raqamlar)    # [1, 2, 3, 4, 5]
print(odamlar)     # ["Jasur", "Akmal", "Bobur"]
```

list() konstruktori qiymatlar to'plamini qabul qilishi mumkin, ular asosida ro'yxat tuziladi:

```

raqamlar1 = [1, 2, 3, 4, 5]
raqamlar2 = list(raqamlar1)
print(raqamlar2)    # [1, 2, 3, 4, 5]

harflar = list("Salom")
print(harflar)      # ['S', 'a', 'l', 'o', 'm']

```

Agar bir xil qiymat bir necha marta takrorlanadigan ro'yxatni yaratish kerak bo'lsa, unda yulduzcha - * belgisidan foydalanish mumkin, ya'ni aslida mavjud ro'yxatga ko'paytirish amalini qo'llaniladi:

```

raqamlar = [15]*6      # 15 marta 6 takrorlash
print(raqamlar)        # [15, 15, 15, 15, 15, 15]

odamlar = ["Jasur"]*3  # "Tom" ni 3 marta takrorlash
print(odamlar)         # ["Jasur", "Jasur", "Jasur"]

talabalar = ["Bobur", "Saman"] * 2
# "Bobur", "Saman" ni 2 marta takrorlash

print(talabalar)
# ["Bobur", "Saman", "Bobur", "Saman"]

```

Ro'yxat elementlariga murojaat qilish. Ro'yxat elementlariga murojaat qilish uchun ro'yxatdagi element sonini ifodalovchi indekslardan foydalanish kerak. Indeksler noldan boshlanadi. Ya'ni, birinchi element indeksi 0, ikkinchi element indeksi 1 ga teng bo'ladi va hokazo. Elementlarga teskari tartibda ya'ni oxiridan boshiga qarab murojaat qilish uchun manfiy indeks tushunchasi kiritilgan. Bu manfiy indekslar -1 dan boshlanadi. Ya'ni, oxirgi element -1 indeksiga ega bo'ladi, oxirigidan bitta oldingisi esa -2 indeksiga ega bo'ladi va hokazo.

```

odamlar = ["Jasur", "Samat", "Bobur"]
# ro'yxat boshidan elementlarni olish

print(odamlar[0])    # Jasur
print(odamlar[1])    # Samat
print(odamlar[2])    # Bobur

# ro'yxat oxiridagi elementlarni olish
print(odamlar[-2])   # Samat
print(odamlar[-1])   # Bobur
print(odamlar[-3])   # Jasur

```

Ro'yxat elementini o'zgartirish uchun unga yangi qiymat yuklash kifoya:

```

odamlar = ["Jasur", "Samat", "Bobur"]

odamlar[1] = "Akmal"  # ikkinchi elementni o'zgartirish
print(odamlar[1])    # Akmal
print(odamlar)        # ["Jasur", "Akmal", "Bobur"]

```

Ro'yxat elementlarini ajratish. Python sizga ro'yxatni alohida elementlarga ajratish imkonini beradi:

```
odamlar = ["Jasur", "Bobur", "Samat"]
jasur, bobur, samat = odamlar
print(jasur)      # Jasur
print(bobur)      # Bobur
print(samat)      # Samat
```

Bunday holda, `jasur`, `bobur` va `samat` o'zgaruvchilari odamlar ro'yxatidan ketma-ket tayinlangan elementlardir. Biroq, o'zgaruvchilar soni tayinlangan ro'yxat elementlari soniga teng bo'lishi talab qilinadi. Aks xolda xatolik kelib chiqadi.

Ro'yxat elementlarini olish uchun `for` yoki `while` siklidan foydalanish mumkin.

`For` sikli bilan takrorlash:

```
odamlar = ["Tomson", "Sems", "Bob"]
for odam in odamlar:
    print(odam)
```

Bu yerda odamlar ro'yxati takrorlanadi va har bir element shaxs o'zgaruvchisiga joylashtiriladi.

Iteratsiyani `while` sikli bilan ham bajarish mumkin:

```
odamlar = ["Tomson", "Sems", "Bob"]
i = 0
while i < len(odamlar):
    print(odamlar[i])
    # elementni olish uchun indeksdan foydalanish
    i += 1
```

`len()` funksiyasidan foydalanib takrorlash uchun ro'yxat uzunligini olinadi. Bu dasturda `i` - hisoblagich qiymati ro'yxat uzunligiga teng bo'lguncha chop etadi. `i` ning qiymati esa har safar birga oshirilib boraveradi.

Ro'yxatni taqqoslash. Ikki ro'yxat, agar ular bir xil elementlar to'plamini o'z ichiga olsa, teng hisoblanadi:

```
raqamlar1 = [1, 2, 3, 4, 5]
raqamlar2 = list([1, 2, 3, 4, 5])

if raqamlar1 == raqamlar2:
    print("1 - raqamlar 2 - raqamlarga teng")
else:
    print("1-raqamlar 2-raqamlarga teng emas")
```

Bunday holda, ikkala ro'yxat ham teng bo'ladi.

3. Ro'yxatlar bilan ishlovchi metodlar

Ro'yxat elementlarini boshqarish uchun bir qancha metodlar mavjud. Ulardan asosan ko'p marta murojaat qilinadiganlarini quyida keltirilgan:

`append(element)`: `element` - ni ro'yxat oxiriga qo'shish;

`insert (indeks, element)`: ro'yxatga *elementni indeks* – element qilib qo'shish;

`extend (items)`: ro'yxat oxiriga *items* to'plamini qo'shish;

remove(item): ro'yxatdan **item** elementini olib tashlaydi. Ro'yxatdan birinchi uchragan **item** elementi o'chiriladi. Agar element topilmasa, ValueError xatoligi qaytariladi;

clear(): ro'yxatdagi barcha elementlarni o'chirish;

index(item): **item** elementining indeksini qaytaradi. Agar element topilmasa, ValueError xatoligi qaytariladi;

pop([indeks]): **indeks** indeksidagi elementni olib tashlaydi va qaytaradi. Agar indeks ko'rsatilmay faqat pop() holida foydalanilsa, oxirgi elementni olib tashlanadi.

count(item): ro'yxatdagi **item** elementining takrorlanishlar sonini qaytaradi. Agar mavjud bo'lmas, 0 qaytaradi;

sort([key]): ro'yxat elementlarini tartiblaydi. Standart bo'yicha o'sish tartibida saralanadi. Lekin **key** parametr yordamida saralashni qanday tartibda bo'lishini boshqarish mumkin.

reverse(): Ro'yxatdagi barcha elementlarni teskari tartibda qayta joylashtiradi.

copy(): ro'yxatni nusxalaydi.

Bundan tashqari, Python ro'yxatlar bilan ishlash uchun bir qator standart funksiyalarni taqdim etadi:

len(list): ro'yxat uzunligini aniqlash;

sorted(list, [key]): tartiblangan ro'yxatni qaytaradi;

min(list): ro'yxatning eng kichik elementini topish;

max(ro'yxat): ro'yxatdagi eng katta elementni topish.

Ro'yxatga element qo'shish uchun append() , extension va insert metodlari, ro'yxatdan elementni olib tashlash uchun **remove()**, **pop()** va **clear()** usullari qo'llaniladi.

Foydalanish usullari:

```
list1= ["Timur", "Bobur"]

# ro'yxat oxiriga qo'shish
list1.append("Elmurod") #["Timur", "Bobur", "Elmurod"]

# ikkinchi pozitsiyaga qo'shish
list1.insert(1, "Bilol") #["Timur", " Bilol", "Bobur", "Elmurod"]

# elementlar to'plamini qo'shish ["Malik", "Samat"]
list1.extend(["Malik", "Samat"])
# ["Timur", " Bilol", "Bobur", "Elmurod", "Malik", "Samat"]

# element indeksini olish
index_of_tom = list1.index("Bobur") # 2

# ushbu indeksda o'chirish
removed_item = list1.pop(index_of_tom)
# ["Timur", " Bilol", "Elmurod", "Malik", "Samat"]

# oxirgi elementni olib tashlang
```



```
oxirgi_element = list1.pop() # "Samat"
print(list1) # ["Timur", " Bilol", "Elmurod", "Malik"]

# "Elmurod" elementini olib tashlash
list1.remove("Elmurod")
print(list1) # ["Timur", " Bilol", "Malik"]

# barcha elementlarni o'chirish
list1.clear()
print(list1) # []
```

IN kalit so'zining ishlatirishi

Agar biron bir element topilmasa, olib tashlash va indekslash usullari xatolik qaytaradi. Bunday vaziyatni oldini olish uchun elementni ishlatishdan oldin **in** kalit so'zidan foydalanib uning mavjudligini tekshirish lozim:

```
list_odamlar = ["Timur", "Bobur", "Samat"]
if "Ali" in list_odamlar:
    list_odamlar.remove("Ali")

print(list_odamlar) # ["Timur", "Bobur", "Samat"]
```

Odamlar ro'yxatida "Ali" elementi bo'lsa, **if "Ali" in list_odamlar** ifodasi "True" qiymati qaytaradi. Shunday ekan bu vaziyatda False qiymati qaytadi va **list_odamlar.remove("Ali")** funksiyasi bajarilmaydi va **list_odamlar** nomli ro'yxat tarkibi o'zgarishsiz qoladi.

DEL metodi

Pythonda ro'yxatdan elementni o'chirish uchun yana bir **del** nomli operatoridan ham foydalaniladi. O'chiriladigan element yoki elementlar to'plami ushbu operatorga parametr sifatida uzatiladi:

```
list_odamlar = ["Timur", " Bilol", "Bobur", "Elmurod", "Malik", "Samat"]

# ikkinchi elementni olib tashlash
del list_odamlar [1]
print(list_odamlar) # ["Timur", "Bobur", "Elmurod", "Malik", "Samat"]

# to'rtinchi elementgacha o'chirish
list_odamlar [:3]
print(list_odamlar) # ["Elmurod", "Malik", "Samat"]

del list_odamlar[1:]#ikkinchi elementdan oxirigacha
print(list_odamlar) # ["Timur"]
```

COUNT() metodi

Agar element ro'yxatda necha marta borligini bilish kerak bo'lsa, **count()** metodidan foydalanish maqsadga muvofiq bo'ladi:

```
odamlar = ["Bobur", " Bilol", "Bobur", "Elmurod", "Malik", "Bobur"]

odamlar_soni = odamlar.count("Bobur")
print(odamlar_soni) # 3
```

SORT() metodi

Sort() metodi standart holatda o'sish tartibida saralash uchun ishlatiladi:

```
odamlar = ["Timur", "Bilol", "Bobur", "Elmurod", "Malik", "Samat"]
odamlar.sort()
print(odamlar)
# ['Bilol', 'Bobur', 'Elmurod', 'Malik', 'Samat', 'Timur']
```

Agar ma'lumotlarni teskari tartibda saralash zarur bo'lsa, sort() metodidan keyin reverse() usulini qo'llash mumkin:

```
odamlar = ["Timur", "Bilol", "Bobur", "Elmurod", "Malik", "Samat"]

odamlar.sort()
odamlar.reverse()
print(odamlar)
# ['Timur', 'Samat', 'Malik', 'Elmurod', 'Bobur', 'Bilol']
```

Saralash aslida ikkita obyektни taqqoslaydi va qaysi biri "kichik" bo'lsa, "katta" obyektдан oldin joylashtiriladi. "katta" va "kichik" tushunchalari albatta nisbiydir. Va agar ro'yxatning barcha elementlari raqamlardan tarkib topgan bo'lsa, unda barchasi oddiy - raqamlar o'sish tartibida joylashtiriladi. Agar ro'yxatda satrlar va boshqa obyektlar mavjud bo'lsa va saralash kerak bo'lsa, unda vaziyat yanada murakkabroq bo'ladi. Bunday holda harflar sonlardan oldin joylashtiriladi. Albatta sonli satrlar haqida gap ketmoqda. Chunki butun tipli ma'lumot bilan string tipli ma'lumot solishtirish xatolik keltirib chiqaradi. Demak, string tiplarni solishtirishda, agar ularning birinchi belgilar teng bo'lsa, ikkinchi belgilar solishtiriladi va hokazo. Bunda raqamli belgi alifbodagi bosh harfdan "kichik" deb hisoblanadi va o'z navbatida bosh harf ham kichik harfdan kichik hisoblanadi.

Shunday qilib, agar ro'yxat tarkibida katta va kichik harflar mavjud bo'lsa va saralash amali bajarilsa, unda unchalik to'g'ri bo'lmagan natijalarni olinishi mumkin, chunki asosan "bob" matni "Tom" matnidan oldin kelishi kerak. Bunday holda sort metodining standart tartiblashini o'zgartirish uchun unga parametr sifatida funksiyani beriladi:

```
odamlar = ["Timur", "bilol", "Bobur", "elmurod", "Malik", "Samat"]

odamlar.sort() # standart tartiblash
print(odamlar) # ['Bobur', 'Malik', 'Samat', 'Timur', 'bilol', 'elmurod']

odamlar.sort(key=str.lower)
# kichik katta-kichik tartiblash

print ( odamlar )
# ['bilol', 'Bobur', 'elmurod', 'Malik', 'Samat', 'Timur']
```

SORTED metodi

Saralash usuliga qo'shimcha ravishda ikkita shaklga ega bo'lgan tartiblangan standart funksiyadan foydalanish mumkin:

sorted(list): list ro'yxatini tartiblaydi

sorted(list, key): list ro'yxati elementlariga key funksiyasini qo'llash orqali ro'yxatni tartiblaydi

```
list_per = ["Tom", "bob", "alis", "Sem", "Bill"]

sorted_per = sorted(list_per, key=str.lower)
print (sorted_per)
# ["alisa", "Bill", "bob", "Sem", "Tom"]
```

Ushbu funksiyadan foydalanganda shuni yodda tutish kerakki, bu funksiya tartiblangan ro'yxatni o'zgartirmaydi, ammo u barcha tartiblangan elementlarni yangi ro'yxatga joylashtiradi va natijani qaytaradi.

MIN va MAX funksiyalari

Pythonning **min()** va **max()** funksiyalari mos ravishda minimum va maksimum qiymatlarni topishga imkon beradi:

```
raqamlar = [9, 221, 12, 10, 3, 125, 18]

print (min(raqamlar))      # 3
print (max(raqamlar))      # 221
```

COPY() funksiyasi

Ro'yxatlarni nusxalashda, ro'yxatlar o'zgaruvchan tur ekanligini yodda tutish kerak. Shuning uchun ikkala o'zgaruvchi ham bir xil ro'yxatga ishora qilsa, bitta o'zgaruvchini o'zgartirish boshqa o'zgaruvchiga ta'sir qiladi:

```
list_per = ["Timur", "Bobur", "Alisey"]
list_per2 = list_per
list_per2.append("Samat") # elementni ikkinchi ro'yxatga qo'shish

# kishi1 va odamlar2 bir xil ro'yxatga ishora qiladi
print(list_per)      # ["Timur", "Bobur", "Alisey", " Samat"]
print(list_per2)     # ["Timur", "Bobur", "Alisey", " Samat"]
```

Bu yuqorida keltirilgan dasturdagi bir o'zgaruvchining qiymatini ikkinchisiga yuklab qo'yish holati nusxalash deb atalmaydi. Nusxalash maqsadida bunday hatti harakatni qilmagan ma'qul. Elementlarni nusxalash uchun chuqurroq nusxalash talab qilinadi. Shundagina bir vaqtning nusxalangan va aslnusxadagi ro'yxatlar qiymatlari bir xil bo'lsada, aslida turli xil ro'yxatlarni ifodalaydi. Buning uchun oddiygina **copy()** metodidan foydalanib nusxalash kerak bo'ladi:

```
list_per = ["Tom", "Bob", "Alis"]
list_per2 = list_per.copy() # elementlarni list_per dan list_per2 ga
nusxalash

# elementni FAQAT ikkinchi ro'yxatga qo'shish
list_per2.append("Sem")

# list_per va list_per2 turli ro'yxatlarni bildiradi
print(list_per)      # ["Tom", "Bob", "Alis"]
print(list_per2)     # ["Tom", "Bob", "Alis", "Sem"]
```

Ro'yxatning bir qismini nusxalash

Agar butun ro'yxatni emas, balki uning ma'lum bir qismini nusxalash kerak bo'lsa, unda quyidagi shakllarni qabul qilishi mumkin bo'lgan maxsus sintaksisdan foydalanish kerak bo'ladi:

`list[:end]`: ro'yxat boshidan end elementigacha bo'lgan oraliqni tanlash.
`list[start:end]`: ro'yxatning start va end elementlari oraliq'ini belgilaydi
`list[start:end:step]`: ro'yxatning start va end elementlari oraliq'ini step qadam bilan belgilaydi. Odatiy bo'lib, bu step parametr **1** ga teng bo'ladi.

```
list_per = ["Tomson", "Bobi", "Elison", "Semmi", "Timati", "Billi"]

slice_people1 = list_per [:3] # 0 dan 3 gacha
print(slice_people1)      # ['Tomson', 'Bobi', 'Elison']

slice_people2 = list_per [1:3] #1 dan 3 gacha
print(slice_people2)      # ["Bobi", "Elison"]

slice_people3 = list_per[1:6:2] #1 dan 6 gacha 2 ga oshib
print(slice_people3)      # ["Bobi", "Semmi", "Billi"]
```

Manfiy indeks

Ro'yxatning oxirgi elementidan boshlab murojaatni boshlash kerak bo'lganda manfiy indeksdan foydalanish kerak bo'ladi. Masalan, -1 – element oxirgi element, -2 – element oxiridan bitta oldingi va hokazo.

```
list_per = ["Tomson", "Bobi", "Elison", "Semmi", "Timati", "Billi"]

# nolinchidan oxirgisigacha, oxirgi xisobga kirmaydi
slice_people1 = list_per[:-1]
print(slice_people1)
# ['Tomson', 'Bobi', 'Elison', 'Semmi', 'Timati']

# oxirgi uchinchi elementdan oxirgisigacha, oxirgisini olmaydi
slice_people2 = list_per [-3:-1]
print(slice_people2)      # ["Semmi", "Timati"]
```

Ro'yxatlarni ulash

Qo'shish (+) operatori ro'yxatlarni birlashtirish uchun ishlatiladi:

```
people1 = ["Tomson", "Bobi", "Alison"]
people2 = ["Tomson", "Semmi", "Timati", "Billi"]
people3 = people1 + people2
print (people3)
# ["Tomson", "Bobi", "Elison", "Tomson", "Semmi", "Timati", "Billi"]
```

Ichma-ich Ro'yxatlar

Ro'yxatlar satrlar, raqamlar kabi standart ma'lumotlardan tashqari, boshqa ro'yxatlarni ham o'z ichiga olishi mumkin. Bunday ro'yxatlarni jadvallar bilan bog'lash mumkin, bu yerda ichki ro'yxatlar qatorlar vazifasini bajaradi. Masalan:

```
odamlar = [
    ["Tomson", 29],
    ["Elison", 33],
    ["Bobi", 27]
```

```

]

print(odamlar[0])          # ["Tomson", 29]
print(odamlar[0][0])       # "Tomson"
print(odamlar[0][1])       # 29

```

Ichki ro'yxatning elementiga murojaat qilish uchun bir juft indeksdan foydalanish kerak: `odamlar[0][1]` - birinchi joylashtirilgan ro'yxatning ikkinchi elementiga murojaat.

Umumiy ro'yxatni, shuningdek, ichki ro'yxatlarni qo'shish, o'chirish va o'zgartirish oddiy (bir o'lchovli) ro'yxatlar kabi amalga oshiriladi:

```

odamlar = [
    ["Tomson", 29],
    ["Elison", 33],
    ["Bobi", 27]
]

# ichki ro'yxat yaratish
odam = list()
odam.append("Billi")
odam.append(41)

# ichki ro'yxat qo'shish
odamlar.append(odam)
print(odamlar[-1])    # ["Billi", 41]

# ichki ro'yxatga qo'shish
odamlar[-1].append("+79996543210")

print(odamlar[-1])    # ["Billi", 41, "+79996543210"]

# oxirgi elementni o'rnatilgan ro'yxatdan olib tashlash
odamlar[-1].pop()
print(odamlar[-1])    # ["Billi", 41]

# oxirgi kiritilgan ro'yxatni butunlay o'chirish
odamlar.pop(-1)

# birinchi elementni o'zgartirish
odamlar[0] = ["Semmi", 18]
print(odamlar)
# [ ["Semmi", 18], ["Alison", 33], ["Bobi", 27]]

```

Ichki ro'yxatlar ustida takrorlash:

```

odamlar = [
    ["Tomson", 29],
    ["Elison", 33],
    ["Bobi", 27]
]

for odam in odamlar:
    for item in odam:

```

```
print(item, end=" | ")
```

Konsol chiqishi :

Tomson | 29 | Elison | 33 | Bobi | 27 |

Nazorat savollari:

1. Ro'yxat deb nimaga aytiladi?
2. Ro'yxatlarni qanday yaratiladi?
3. Ro'yxatlarning elementlari bir turga mansub bo'lishi shartmi?
4. ro'yxat elementlariga murojaatlar qanday amalga oshiriladi?
5. Ro'yxat uzunligini qanday aniqlanadi?
6. Ro'yxat elementini biro songa ko'paytirish nimani beradi?
7. Manfiy index tushunchasi nimani anglatadi?
8. tomson, bobi, semmi = odamlar – ushbu ifodaning ma'nosi nima?
9. Ro'yxatga ma'lumotni sikl orqali kiritish qanday amalga oshiriladi?
10. Ro'yxat elementlarini saralash uchun qaysi metodni ishlatiladi?

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

9-mashg'ulot. Pythonda ro'yxatlarga doir masalalar yechish.

O'quv savollari:

1. Pythonda ro'yxatlarga doir oddiy masalalar yechish.
2. Ro'yxatlarni ishlovchi metodlarga doir masalalar yechish.

1. Pythonda ro'yxatlarga doir oddiy masalalar yechish.

Pythondagi lug'at (dictionary) elementlar to'plamini saqlaydi. Lug'at tarkibidagi har bir element o'ziga xos, yagona kalitga va u bilan bog'liq bo'lgan qiymatga ega bo'ladi.

Lug'at ta'rifi quyidagi sintaksisga ega:

```
dictionary={kalit1: qiymat1, kalit2: qiymat2, ....}
```

Lug'atning barcha elementlari jingalak qavslar ichiga joylanadi. Har bir element bir-biri bilan vergul orqali ajratildi. Har bir elementning yagona kaliti va unga bog'langan qiymat mavjud bo'ladi. Elementning kaliti birinchi keladi va undan keyin qiymat keladi. Kalit va qiymatini ikki nuqta ajratib turadi.

2. Lug'atlarni yaratish usullari.

Lug'atni aniqlash:

```
users = {1: "Tomson", 2: "Bobi", 3: "Billi"}
```

`users` lug'ati raqamlarni kalit sifatida va satrlarni qiymat sifatida ishlatadi. Ya'ni, 1-kalitga ega bo'lgan element "Tomson" qiymatiga teng, 2-kalitga ega element "Bobi" qiymatiga ega va hokazo.

Lug'atdagi elementlar tarkibi bir xil tipli ma'lumotlar bo'lishi ham mumkin. Masalan:

```
e_emails = {"tomson@gmail.com": "Tomson", "bobi@gmail.com": "Bobi", "semmi@gmail.com": "Semmi"}
```

`e_emails` lug'atida kalit sifatida satrlardan foydalanildi - foydalanuvchi elektron pochta manzillari va qiymat sifatida ham satrlardan - foydalanuvchi nomlaridan foydalanildi.

Lekin kalitlar va satrlar bir xil turdagi bo'lishi shart emas. Ular turli xil turlarni ko'rsatishi mumkin:

```
objects = {1: "Tomson", "2": False, 3: 150.64}
```

Bundan tashqari, umuman elementsiz, bo'sh lug'atni aniqlash ham mumkin:

```
dict_object = {}
```

yoki shunga o'xshash:

```
dict_object = dict()
```

Ro'yxatlar va kortejlarni lug'atga aylantirish

Lug'at va ro'yxat tuzilmaviy jihatdan bir-biriga o'xshamaydigan turlar bo'lsa-da, standart **dict()** funksiyasi yordamida ma'lum turdagi ro'yxatlarni lug'atga

aylantirish mumkin. Buning uchun ro'yxat ichki ro'yxatlar to'plamini saqlashi kerak. Har bir ichki ro'yxat ikkita elementdan iborat bo'lishi kerak - lug'atga aylantirilganda birinchi element kalitga, ikkinchisi esa qiymatga aylanadi:

```
user_list=[
    ["+554513455", "Tomson"],
    ["+002549557", "Bobi"],
    ["+445454512", "Elison"]
]
user_dict = dict(user_list)
print(user_dict) # {"+554513455", "Tomson:", "+002549557", "#Bobi",
"+445454512": "Elison"}
```

Xuddi shunday, ikkita kortej lug'atga aylantirilishi mumkin, o'z navbatida lug'at ikkita kortejni o'z ichiga oladi:

```
user_tuple = (
    ("+554513455", "Tomson"),
    ("+002549557", "Bobi"),
    ("+445454512", "Elison")
)
user_dict = dict (user_tuple)
print (user_dict)
```

Lug'at elementlariga murojaat qilish va o'zgartirish

Lug'at elementlariga murojaat qilish uchun uning nomidan keyin element kaliti kvadrat qavs ichida ko'rsatiladi:

```
Dictionary_name[key]
```

Lug'atdagi elementlarni olish va o'zgartirish uchun:

```
dict_users = {
    "+11002255": "Tomson",
    "+15245999": "Bobi",
    "+15415154": "Elison"
}

# "+11002255" kaliti bilan elementni olish
print (dict_users ["+11002255"]) # Tomson

# element qiymatini "+15245999" tugmasi bilan belgilash
dict_users["+15245999"] = "Bob Smit"
print (dict_users["+15245999"]) # "Bob Smit"
```

Agar bunday kalit bilan elementning qiymatini belgilashda lug'atda hech qanday element bo'lmasa, u qo'shiladi:

```
dict_users ["+5555555"] = "Semmi"
```

Ammo agar lug'atda mavjud bo'lmagan kalit bilan qiymat olishga harakat qilinsa, Python `KeyError` xatoligini chiqaradi:

```
foydalanuvchi=foydalanuvchilar["+4444444"] # KeyError
```


Bunday holatni oldini olish maqsadida elementga murojaat qilishdan oldin "key in dict" ifodasi yordamida lugʻatda element mavjudligini tekshirish mumkin. Agar kalit lugʻatda boʻlsa, u holda bu ifoda "True" ni qaytaradi:

```
key = "+00000044"
if key in dict_users:
    dict_user = dict_users[key]
    print(dict_user)
else:
    print ("Element topilmadi")
```

Shuningdek lugʻat elementlariga murojaat qilish uchun **get** metodidan ham foydalaniladi.

Roʻyxatlarni ishlovchi metodlarga doir masalalar yechish.

get(key): lugʻatdan kalit tugmasi boʻlgan elementni qaytaradi. Agar ushbu kalitga ega boʻlgan element mavjud boʻlmasa, u holda "False" qiymati qaytariladi;

get(key, default): lugʻatdan key kaliti boʻlgan elementni qaytaradi. Agar bunday kalitga ega element boʻlmasa, u **default** qiymatni qaytaradi.

```
foydalanuvchilar = {
    "+11002255": "Tomson",
    "+15245999": "Bobi",
    "+15415154": "Elison"
}

user1 = users.get("+15415154")
print (user1)          # Alison
user2=users.get("+15245999","Noma'lum foydalanuvchi")
print (user2 )         #Bobi
user3=users.get("+0000000", "Noma'lum foydalanuvchi")
print(user3)          # Noma'lum foydalanuvchi
```

DEL funksiyasi

Del funksiyasi elementni kalit bilan olib tashlash uchun ishlatiladi :

```
foydalanuvchilar = {
    "+11002255": "Tomson",
    "+15245999": "Bobi",
    "+15415154": "Elison"
}

del foydalanuvchilar["+15415154"]
print (foydalanuvchilar)
# { "+11002255": "Tomson", "+15245999": "Bobi"}
```

Ammo shuni yodda tutish kerakki, agar bunday kalit lugʻatda boʻlmasa, **KeyError** xatoligi qaytariladi. Shuning uchun, oʻchirilishi kerak boʻlgan elementni oldin, berilgan kalitli element mavjudligini tekshirish tavsiya etiladi.

```
dict_user = {
    "+11002255": "Tomson",
    "+15245999": "Bobi",
    "+15415154": "Elison"
```

```

}

key = "+99990000"
if key in dict_user:
    del dict_user[key]
    print(f"{key} kaliti element olib tashlandi")
else:
    print(f"{key} kalitli element topilmadi")

```

POP() metodi

O'chirishning yana bir usuli - **pop()** usuli. Pop metodidan ikki xil ko'rinishda foydalanish mumkin:

1) pop(key): *key* kalit berilgan elementni olib tashlaydi va olib tashlangan elementni qaytaradi. Agar berilgan kalit bilan element topilmasa, **KeyError** xatoligi qaytariladi.

2) pop(key, default): *key* kalitli elementni o'chiradi va olib tashlangan elementni qaytaradi. Agar berilgan kalit bilan element bo'lmasa, **default** qiymati qaytariladi.

```

foydalanuvchilar = {
    "+11002255": "Tomson",
    "+15245999": "Bobi",
    "+15415154": "Elison"
}
key = "+11002255"
user1 = users.pop (key)
print (user1)          # Tomson
user1 = users.pop("+00000000", "Noma'lum foydalanuvchi")
print (user1)          # Noma'lum foydalanuvchi

```

Agar barcha elementlarni olib tashlash kerak bo'lsa, ya'ni bir so'z bilan aytganda tozalash kerak bo'lsa, unda **clear()** metodidan foydalanish kerak bo'ladi:

```
per.clear()
```

Lug'atlarni nusxalash va birlashtirish

copy() usuli lug'at tarkibini nusxalab, yangi lug'atni qaytaradi:

```

foydalanuvchilar={"998888":"Tomson","998771": "Bobi"}
talabalar = users.copy()
print (talabalar) # {"998888": "Tomson", "998771": "Bobi"}

```

update() metodi ikkita lug'atni birlashtiradi:

```

foydalanuvchilar={"998888":"Tomson","998771": "Bobi"}

users2 = {"222222": "Semmi", "666000": "Kail"}
users.update(users2)

print(users)      # {"998888": "Tomson", "998771": "Bobi", "222222": "Semmi",
"666000": "Kail"}

print(users2)     # {"222222": "Semmi", "666000": "Kail"}

```

Bunday holda, **users2** lug'ati o'zgarishsiz qoladi. Faqat foydalanuvchilarning lug'ati o'zgartiriladi, unga boshqa lug'at elementlari qo'shiladi. Ammo agar ikkala

manba lug'ati ham o'zgarishsiz qolishi kerak bo'lsa va birlashmaning natijasi uchinchi lug'at bo'lsa, avval bitta lug'atni boshqasiga nusxalanishi kerak:

```
users3 = users.copy()
users3.update(users2)
```

Lug'atni takrorlash

Lug'at tarkibini sikldan foydalanib olish uchun for siklidan foydalanish mumkin:

```
users = {"998888": "Tomson", "998771": "Bobi"}
for a in users:
    print(f"id: {a}   Foydalanuvchi: {users[a]} ")
```

Takrorlash orqali elementlarga murojaat qilishda joriy elementning kalitini elementning o'zini olish uchun ishlatiladi.

Elementlarni olishning yana bir usuli - **items()** metodidan foydalanish:

```
users = {"998888": "Tomson", "998771": "Bobi"}
for key, val in users.items():
    print(f"id: {key}   Foydalanuvchi: {val} ")
```

items() usuli kortejlar to'plamini qaytaradi. Har bir kortej elementning kaliti va qiymatini o'z ichiga oladi, ularga murojaat qilishda **key** va **val** o'zgaruvchilarini chaqirilsa yetarli.

Shuningdek, kalitlar ustida takrorlash va qiymatlarni takrorlash uchun alohida usul ham mavjud. Lug'atdagi kalitlarni olish uchun **keys()** metodidan foydalaniladi:

```
for key in dict_users.keys():
    print(key)
```

Faqat qiymatlarni takrorlash uchun lug'atning **values()** metodidan foydalaniladi:

```
for val in dict_users.values():
    print(val)
```

Murakkab lug'atlar

Raqamlar va satrlar kabi eng oddiy obyektlardan tashqari, lug'atlar murakkabroq obyektlarni ham saqlashi mumkin. Masalan bir xil ro'yxatlar, kortejlar yoki boshqa lug'atlarni o'z ichiga element sifatida joylashtirishi mumkin:

```
dict_users = {
    Tomson: {
        "id": "745",
        "e_main": "tomson12@gmail.com"
    },
    Bobi: {
        "id": "444",
        "e_mail": "bobi@gmail.com",
        "skype": "bob123"
    }
}
```

Bunda lug'atning har bir elementining qiymati o'z navbatida alohida lug'atni ifodalashi mumkin.

Ichki lugʻat elementlariga kirish uchun mos ravishda ikkita kalitdan foydalanish kerak:

```
old_e_mail = dict_users["Tomson"]["e_mail"]
dict_users["Tomson"]["e_mail"] = "supertomson@gmail.com"
print(dict_users["Tomson"])
# { "id": "745", "e_main": "supertomson@gmail.com" }
```

Ammo agar lugʻatda boʻlmagan kalit orqali qiymat olishga harakat qilinsa, Python **KeyError** xatoligini chiqaradi:

```
tom_skype = dict_users["Tom"]["skype"] # KeyError
```

Xatolikka yoʻl qoʻymaslik uchun lugʻatda kalit mavjudligini tekshirish kerak:

```
key = "skype"
if key in dict_users["Tomson"]:
    print(dict_users["Tomson"]["skype"])
else:
    print("skype lugʻatdan topilmadi!")
```

Boshqa barcha jihatlarida murakkab va ichki lugʻatlar bilan ishlash oddiy lugʻatlar bilan ishlash kabi bajariladi.

Nazorat savollari:

1. Pythonda tugʻat tushunchasiga tarif bering.
2. Pythonda lugʻatlar qanday usullar bilan yaratilishi mumkin?
3. Roʻyxatlar va kortejlarni lugʻatga aylantirish qanday amalga oshiriladi?
4. Lugʻat Elementlariga murojaat qilish va oʻzgartirish qanday amalga oshiriladi?
5. **get()** metodining vazifasi qanday va uning qabul qiluvchi parametrlari qanday?
6. **del** metodining vazifasi va ishlash prinsipi haqida gapirib bering.
7. **pop()** metodining vazifasi qanday va uning qabul qiluvchi parametrlari qanday?
8. **clear()** metodining vazifasi va ishlash prinsipi haqida gapirib bering.
9. **copy()** metodining vazifasi qanday va uning qabul qiluvchi parametrlari qanday?
10. **upgrate()** metodining vazifasi va ishlash prinsipi haqida gapirib bering
11. **items()** metodining vazifasi va ishlash prinsipi haqida gapirib bering

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

10-mashg’ulot. Pythonda Funksiya tushunchasi. Foydalanuvchi funksiyasi.

O‘quv savollari:

1. Funksiyalarni aniqlash va uni chaqirish.
2. Parametrli va parametrsiz funksiya.

1. Funksiyalarni aniqlash va uni chaqirish.

Funksiyalar ma’lum bir vazifani bajaradigan va dasturning boshqa qismlarida qayta ishlatilishi mumkin bo‘lgan kod blokini ifodalaydi. Funksiyalardan oldingi mavzularni tushuntirish davomida allaqachon foydalaib o‘tildi. Masalan, konsolga qiymatlarni chop etadigan **print()** funksiyasidan deyarli har bir mavzuda foydalanildi. Python ko‘plab standart o‘rnatilgan funksiyalarga ega bo‘lgan dasturlash tilidir. Bunday standart funksiyalardan tashqari foydalanuvchi o‘ziga kerakli funksiyalarni yaratish imkoni mavjud. Funksiya yaratish uchun quyidagi sintaksidan foydalaniladi:

```
def function_name([parametrlar]):  
    [funksiya tanasi]
```

Pythonda funksiya boshqa dasturlash tillaridan farqli ravishda def kalit so‘zi bilan boshlanadi. Def kalit so‘zidan keyin funksiya nomi va agar qabul qiluvchi parametrlari mavjud bo‘lsa, ularni qavs ichida, vergul bilan ajratgan holda nomlari keltiriladi va qavslardan keyin osti-usti ikki nuqta qo‘yiladi. Va keyingi qatordan funksiya bajaradigan ko‘rsatmalar bloki boshlanadi. Barcha funksiya tanasidagi operatorlar satr boshidan ichkariga chekingan holda bir ustundan boshlanishi kerak.

Masalan, eng oddiy funksiyaning ko‘rinishi:

```
def say_hello():  
    print ("Salom")
```

Funksiya nomi: **say_hello** deb ataladi. U hech qanday parametrlarga ega emas va konsolga "Salom" satrini chop etadigan bitta operatorni o‘z ichiga oladi.

E’tibor bering, funksiya iboralari funksiya boshidan chekinishi kerak. Masalan:

```
def say_hello():  
    print("Salom")  
print("Xayr")
```

Bu yerda **print("Xayr")** buyrug‘i **say_hello** funksiyasi bilan bir ustunda va shuning uchun bu funksiyaga tegishli bo‘lmagan operator hisoblanadi. Funksiya tarkibiga kiruvchi blok va funksiyaga tegishli bo‘lmagan operatorlar bir-biri bilan oldidagi qoldirilgan tab (bo‘sh joy) bilan farq qiladi.

Funksiyaga murojaat. Funksiyani chaqirish uchun funksiya nomi va uning qabul qiluvchi parametrlariga mos qiymatlar qavs ichida ko‘rsatiladi:

```
funksiya_nomi ([parametrlar])
```

Masalan, salom tekstini konsolga chiqaruvchi bitta oddiy funksiya yaratamiz va uni chaqiramiz:

```
def hello():
    print ("Salom")
hello()
hello()
hello()
```

Bu yerda hello funksiyasi ketma-ket uch marta chaqirilmoqda. Natijada, konsolda quyidagi natijalar paydo bo'ladi:

```
Salom
Salom
Salom
```

Shuni esda tutish kerakki, funksiya yuqorida yaratiladi va keyin chaqiriladi.

Agar funksiya tarkibida faqat bitta operator mavjud bo'lsa, bu operatorni funksiya ta'rifining davomidan ikki nuqtadan keyin joylashtirilishi mumkin:

```
def hello(): print("Salom")
hello ()
```

Boshqa funksiyalarni ham xuddi shunday aniqlash va chaqirish mumkin. Masalan, bir nechta funksiyalarni aniqlaymiz va bajaramiz:

```
def say_hello():
    print ("Salom")
def say_goodbye():
    print ("Xayr")
say_hello()
say_goodbye()
```

Natijaning konsolda ko'rinishi :

```
Salom
Xayr
```

Lokal funksiya. Ba'zi funksiyalarni boshqa funksiyalar ichida aniqlash mumkin. Bunday funksiyalarni ichki funksiyalar ya'ni **lokal funksiyalar** deb ataladi. Lokal funksiyalar faqat o'zi tegishli bo'lgan funksiya doirasida ishlatilishi mumkin. Masalan :

```
def print_msg():
    def say_hello(): print("Salom")
    def say_bye(): print("Xayr")
say_hello()
say_bye()
print_msg()
```

Bu yerda say_hello() va say_bye() funksiyalari print_msg() funksiyasi ichida aniqlanadi va shuning uchun u uchun lokal funksiya hisoblanadi. Shunga ko'ra, ular faqat print_msg() funksiyasi ichidagina ishlatilishi mumkin.

main funksiyasi va undan qo'llanilishi. Dasturda ko'plab funksiyalar ishtirok etishi mumkin. Va ularning barchasini birlashtirish uchun asosiy bitta funksiya

ishlatiladi. Odatda bu funksiya `main` deb ataladi va unda boshqa funksiyalar chaqiriladi:

```
def main_f():
    say_hello()
    say_bye()
def say_hello():
    print("Salom")
def say_bye():
    print("Xayr")
# Asosiy funksiyani chaqirish
main_f()
```

2. Parametrli va parametrsiz funksiya

Funksiyalarni parametrli va parametrsiz funksiyalarga ajratishimiz mumkin. Parametrsiz funksiyalar tashqaridan hech qanday qiymat qabul qilmaydi. Parametrli funksiyalar esa funksiya chaqirilayotganda biror bir qiymatni berilishini kutadigan funksiyalardir. Ushbu parametrlar orqali funksiya turli ma'lumotlarni uzatish mumkin. Buni **print()** funksiyasi misolida ko'rib chiqamiz. **print()** parametr sifatida konsolga chop etilishi kerak bo'lgan xabarli qabul qiladi.

Endi bitta oddiy, ism parametrli funksiya yaratamiz va uni chaqiramiz:

```
def say_hello(ism):
    print(f"Salom, { ism }")

say_hello("Jamshid")
say_hello("Boboy")
say_hello("Asilbek")
```

`Say_hello` funksiyasi `ism` parametriga ega va funksiya chaqirilganda ushbu parametrba ba'zi qiymatlarni berish mumkin. Funksiya ichida parametrni oddiy o'zgaruvchi sifatida ishlatish mumkin. Masalan, ushbu parametrning qiymatini `print` funksiyasi bilan konsolga chop etish. Shunday qilib, ifodada:

```
say_hello("Jamshid ")
```

"Jamshid" qatori `ism` parametriga uzatiladi. Natijada, dasturni bajarishda quyidagi natijani konsol chiqaradi:

```
Salom, Jamshid
Salom, Boboy
Salom, Asilbek
```

Funksiya chaqirilganda, qiymatlar pozitsiya bo'yicha parametrlarga o'tkaziladi. Masalan, bir nechta parametrli funksiyani quyidagicha yaratish va chaqirish mumkin:

```
def print_per (ism, yosh):
    print(f"Ism: {ism}")
    print(f"Yosh: {yosh}")
print_per ("Jamshid", 25)
```

Bu yerda `print_per` funksiyasi ikkita parametrni qabul qiladi: `ism` va `yosh`. Funksiyani chaqirganda :

```
print_per("Jamshid", 25)
```

Funksiya chaqirilish vaqtidagi unga parametr sifatida beriladigan qiymatlar mos parametrlarga qabul qilinadi. Ya'ni birinchi qiymat – "Jamshid" birinchi parametrga, ya'ni `ism` parametriga qabul qilinsa, ikkinchi qiymat – 25 bo'lsa, `yosh` parametriga qabul qilinadi. Va funksiya ichida parametr qiymatlari konsolga chop etiladi:

```
Ism: Tom
Yosh: 37
```

Standart qiymatlar

Funksiyani yaratish mobaynida, funksiyaning mavjud parametrlari uchun boshlang'ich qiymatlarni belgilash orqali funksiyaning ba'zi parametrlariga qiymat berishni ixtiyoriy qilinishi mumkin. Masalan :

```
def say_hello(ism="Jamshid"):
    print(f"Salom, {ism}")
say_hello()          # bu yerda ism = "Jamshid"
say_hello("Bobo")    # bu yerda ism = "Bobo"
```

Bu yerda `ism` parametriga qiymat berish majburiy emas. Ya'ni, agar funksiyaning chaqirishda uning `ism` parametriga qiymat berilmasa standart qiymat – "Jamshid" qo'llaniladi. Ushbu dasturning konsolda ko'rinishi quyidagicha bo'ladi:

```
Salom, Jamshid
Salom, Bobo
```

Agar funksiya bir nechta parametrlarga ega bo'lsa, qiymat berish ixtiyoriy bo'lgan parametrlar, qiymat berish majburiy bo'lgan qilinganlardan keyin kelishi kerak. Masalan :

```
def print_per(ism, yosh = 54):
    print(f"Ism: {ism}   Yosh: {yosh}")
print_per("Bobo")
print_per("Jamshid", 25)
```

Bu yerda `yosh` parametri ixtiyoriy va standart bo'yicha 54 ga teng. Undan oldin qiymat berish majburiy bo'lgan `ism` parametri joylashgan. Shuning uchun funksiyaning chaqirishda `yosh` parametriga qiymat o'tkaza olmaymiz, lekin `ism` parametriga qiymat uzatish kerak.

Agar kerak bo'lsa, biz barcha parametrlarni ixtiyoriy qilishimiz mumkin:

```
def print_per(ism = "Jamshid", yosh = 18):
    print(f"Ism: {ism}   Yosh: {yosh}")
print_per()          # Ism: Jamshid Yosh: 18
print_per("Bobo")    # Ism: Bobo   Yosh: 18
print_per("Samat", 54) # Ism: Samat  Yosh: 54
```

Nomlangan parametrlar. Yuqoridagi misollarda funksiya chaqirilganda parametrlarga qiymatlar joylashish o'rnini bo'yicha uzatildi. Ammo qiymatlarni parametrlarga nomi ko'ra uzatish ham mumkin. Buning uchun funksiyaning chaqirishda parametr nomi ko'rsatiladi va unga qiymat beriladi:

```
def print_per(ism, yosh):
    print(f"Ism: {ism}   Yosh: {yosh}")
```



```
print_per(yosh = 22, ism = "Jamshid")
```

Bunday holda, yosh va ism parametrlari nomiga ko'ra qiymatlarni topib olishadi. Funksiya yaratilayotganda ism parametri birinchi o'rinda turibdi. Shunga qaramay, funksiyani chaqirishda `print_per(yosh= 22, ism = "Jamshid")` kabi chaqirilsa ham bo'ladi va shu tariqa yosh parametriga 22 ni va ism parametriga "Jamshid" qiymatini berish mumkin.

Funksiya parametriga faqat nomi bo'yicha qiymat berish. * belgisi qaysi parametrlarga nom berilishini belgilash imkonini beradi. Ya'ni qiymatlarni faqat nomi bo'yicha uzatish mumkin bo'lgan parametrlar. * belgisining o'ng tomonida joylashgan barcha parametrlar qiymatlarni faqat nomi bilan qabul qiladi:

```
def print_per(ism, *, yosh, tashkilot):
    print(f"Ism: {ism} Yosh: {yosh} Tashkilot: {tashkilot}")
    print_per("Bobo", yosh = 41, tashkilot = "Microsoft")
```

Bunday holda, yosh va tashkilot parametrlari nomi bilan qiymatlar qabul qilishadi.

Parametrlar ro'yxatiga * bilan prefiks qo'yish orqali barcha parametrlarni nomlash mumkin:

```
def print_person(*, ism, yosh, tashkilot):
    print(f"Ism: {ism} Yosh: {yosh} Tashkilot: {tashkilot}")
```

Aksincha, siz qiymatlarni faqat pozitsiya bo'yicha, ya'ni pozitsion parametrlar bo'yicha qabul qilish mumkin bo'lgan parametrlarni belgilashingiz kerak bo'lsa, unda / belgisidan foydalanish mumkin.

Bunda / belgisidan olda joylashgan barcha parametrlar faqat joylashish o'rniga ko'ra qiymatlar va qabul qilishi mumkin.

```
def print_per(ism, /, yosh, tashkilot="Micros"):
    print(f"Ism:{ism} Yosh: {yosh} Tashkilot: {tashkilot}")
    print_per("Jamshid", tashkilot="ShamsWeb", yosh = 11)
    print_per("Bobo", 41)
```

Bunday holda, ism parametri pozitsiondir.

Bitta funksiyada bir vaqtning o'zida pozitsion va nomli parametrlar mavjud bo'lishi mumkin.

```
def print_per(ism, /, yosh = 18, *, tashkilot):
    print(f"Ism: {ism} Yosh:{yosh} Tashkilot: {tashkilot}")
    print_per("Samat", tashkilot = "Google")
    print_per("Tomson", 37, tashkilot = "JetBrains")
    print_per("Bobo", tashkilot = "Micros", yosh= 42)
```

Bunday holda, ism parametri / belgisining chap tomonida joylashgan, shuning uchun u pozitsion, majburiydir hamda faqat pozitsiya bo'yicha qiymat berilishi mumkin.

Tashkilot parametri * belgisining o'ng tomonida joylashganligi sababli nomlangan parametr hisoblanadi. Yosh parametri nomi bo'yicha ham, joylashgan o'ni bo'yicha ham qiymat qabul qilishi mumkin.

Aniqlanmagan sondagi parametrlar. Funksiyaga noma'lum miqdordagi qiymatlarni uzatish uchun yulduzcha belgisidan foydalaniladi. Bu funksiya nomalumi miqdordagi, bir nechta qiymatlarni qabul qilishda qo'llaniladi. Masalan, sonlar yig'indisini hisoblash funksiyasini ko'rib chiqamiz:

```
def sum(*raqamlar):
    natija = 0
    for j in raqamlar:
        natija += j
    print(f"sum = {natija}")
sum(1, 2, 3, 4, 5)      # sum = 15
sum(3, 4, 5, 6)        # sum = 18
```

Bunday holda, `sum` funksiyasi bitta parametrni oladi - `* raqamlar`, lekin parametr nomi oldidagi yulduzcha, aslida ushbu parametr o'rniga cheksiz miqdordagi qiymatlarni yoki qiymatlar to'plamini qabul qilish mumkinligini ko'rsatadi. Funksiyaning o'zida `for` siklidan foydalanib, ushbu to'plamdan birma-bir elementlarni olish mumkin, ushbu to'plamdan har bir qiymatni `j` o'zgaruvchiga olish va u bilan qandaydir amallarni bajarish mumkin. Misol uchun, bu dasturda uzatilgan raqamlarning yig'indisi hisoblandi.

3. Dekoratorlar.

Dekoratorlarni tushunish har qanday Python dasturchisi uchun muhim bosqichdir. Ushbu maqola dekorativlar sizga qanday qilib samaraliroq va samarali Python dasturchisi bo'lishingizga yordam berishi haqida bosqichma-bosqich qo'llanmadir.

Python-dagi dekorativlar chaqiriladigan obyektning o'zini doimiy ravishda o'zgartirmasdan, *chaqiriladigan* obyektlarning (funksiyalar, usullar va sinflar) xatti-harakatlarini kengaytirish va o'zgartirish imkonini beradi.

Mavjud sinf yoki funksiyaning xatti-harakatlariga "biriktirilishi" mumkin bo'lgan har qanday etarlicha umumiy funktsionallik dekorativlardan foydalanishning ajoyib namunasidir. Bunga quyidagilar kiradi:

- ro'yxatga olish,
- kirish nazorati va autentifikatsiyani ta'minlash,
- vaqtni boshqarish vositalari va funksiyalari,
- Tezlik chegarasi,
- keshlash va boshqalar.

Nima uchun Pythonda dekorativlarni o'rganishingiz kerak?

Bu adolatli savol. Men hozir gapirgan narsa mavhum bo'lib tuyuladi va dekorativlar Python dasturchisiga kundalik ishida qanday yordam berishini tushunish qiyin bo'lishi mumkin. Mana bir misol:

Tasavvur qiling, sizning hisobot dasturingiz biznes mantig'iga ega 30 ta funksiya ega. Yomg'irli dushanba kuni ertalab menejeringiz sizga aytadi:

“TPS hisobotlarini eslaysizmi? Hisobot ishlab chiqaruvchining har bir bosqichida kirish va chiqish ma'lumotlarini jurnalga qo'shishimiz kerak. XYZ kompaniyasiga bu audit uchun kerak. Men ularga buni chorshanbagacha hal qilishimiz mumkinligini aytdim.

Python dekorativlari bilan qanchalik tanish ekanligingizga qarab, bu so'rov sizning qon bosimingizni oshiradi yoki ushbu yangi talablarni asta-sekin qabul qiladi.

Dekorator haqida ma'lumotga ega bo'lmasangiz, ehtimol siz keyingi uch kun davomida ushbu 30 ta funksiyaning har birini o'zgartirishga va ularni qo'lda ro'yxatdan o'tish qo'ng'iroqlari bilan aralashtirib yuborishga harakat qilasiz. Umuman olganda, vaqtingizni qiziqarli o'tkazing.

Agar siz dekoratorlarni bilsangiz, xotirjam jilmayib, "Yaxshi, bugun soat 14:00 da tayyor bo'ladi" kabi bir narsa deysiz.

Shundan so'ng, siz umumiy `@audit_log` dekoratorining kodini chop etasiz (faqat 10 qator uzunlikda) va uni har bir funksiya ta'rifidan oldin qo'shasiz. Keyin siz va'da qilasiz va bir chashka qahva ichasiz.

Bu yerda men bo'rttirib aytyapman, albatta. Ammo ozgina. Dekoratorlar haqiqatan ham kuchli bo'lishi mumkin.

Men dekorativlarni tushunish har qanday tajribali Python dasturchisi uchun muhim bosqich deb aytardim. Ular bir nechta ilg'or til tushunchalarini, shu jumladan birinchi darajali funksiyalarning xususiyatlarini yaxshi tushunishni talab qiladi.

Lekin: **Dekoratorlarni tushunish bunga arziydi**

Python-da dekorativlarning qanday ishlashini tushunish juda foydali.

Albatta, dekorativlarni birinchi marta taqdim etilganda tushunish juda qiyin bo'lib tuyuladi, lekin bu juda foydali xususiyat bo'lib, uni uchinchi tomon ramkalari va Python standart kutubxonasida tez-tez ko'rishingiz mumkin.

Dekoratorlarni tushuntirish har qanday yaxshi Python o'quv qo'llanmasining muhim bo'limidir. Ushbu maqolada men sizni ular bilan bosqichma-bosqich tanishtirishga harakat qilaman.

Mavzuga kirishdan oldin, keling, Python-da birinchi darajali funksiyalarning xususiyatlarini ko'rib chiqaylik. Men dbader.org saytida ularga qo'llanma yozdim, uni o'qish uchun bir necha daqiqa vaqt ajratishingizni tavsiya qilaman. Bu erda dekorativlarni tushunish uchun "birinchi darajali funksiyalar" dan eng muhim xulosalar:

- Funksiyalar obyektlardir - ular o'zgaruvchilarga tayinlanishi, boshqa funksiyalarga o'tkazilishi va qaytarilishi mumkin.

- Funksiyalar boshqa funksiyalar ichida aniqlanishi mumkin va bola funksiyasi ota-ona funksiyasining mahalliy holatini (leksik yopilishlar) olishi mumkin.

Xo'sh, ketishga tayyormisiz? Keling, boshlaymiz.

Python dekorator asoslari

Xo'sh, aslida dekorativlar nima? Ular boshqa funksiyani "bezatadi" yoki "o'radi" va kodni o'ralgan funksiya bajarilishidan oldin va keyin bajarishga imkon beradi.

Dekoratorlar boshqa funksiyalarning harakatlarini o'zgartirishi yoki kengaytirishi mumkin bo'lgan qayta ishlatiladigan modullarni aniqlash imkonini beradi. Bundan tashqari, ular sizga o'ralgan funksiyani doimiy ravishda o'zgartirmasdan buni amalga oshirishga imkon beradi. Funksiyaning harakati faqat bezatilganida *o'zgaradi*.

Xo'sh, oddiy dekoratorni amalga oshirish nimaga o'xshaydi? Umuman olganda, dekorator chaqiriladigan obyekt bo'lib, u qo'ng'iroq qilinadigan obyektни kiritadi va boshqa chaqiriladigan obyektни qaytaradi.

Quyidagi funksiya bu xususiyatga ega va siz yozishingiz mumkin bo'lgan eng oddiy dekorativ deb hisoblanishi mumkin:

```
def null_decorator(func):
    return func
```

Ko'rib turganingizdek, null_decorator chaqiriladigan obyekt bo'lib, u boshqa chaqiriladigan obyektни kiritish sifatida qabul qiladi va uni o'zgartirmasdan bir xil kirish obyektini qaytaradi.

Keling, uni boshqa funksiyani bezash (yoki o'rash) uchun ishlatamiz:

```
def greet():
    return 'Hello!'

greet = null_decorator(greet)

>>> greet()
'Hello!'
```

Ushbu misolda greetmen funksiyani belgilab qo'ydim va keyin uni funksiya orqali ishga tushirish orqali darhol bezadim null_decorator. Bilaman, bu hozircha unchalik foydali ko'rinmaydi (biz maxsus null dekoratorni foydasiz qilib ishlab chiqdik, to'g'rimi?), lekin bir muncha vaqt o'tgach, bu Pythonda dekorator sintaksisi qanday ishlashini aniqroq qiladi.

O'zgaruvchini aniq chaqirish va keyin uni qayta tayinlash null_decorator o'rniga, funksiyani bir bosqichda bezash uchun Python sintaksisidan foydalanishingiz mumkin :greetgreet@

@null_decorator

```
def greet():
    return 'Hello!'

>>> greet()
'Hello!'
```

Funksiya ta'rifidan oldin @null_decorator satrlarini qo'yish avval funksiyani belgilash va keyin unga dekoratorni qo'llash bilan bir xil. Sintaksisidan

foydalanish @oddiygina sintaktik shakar va bu tez-tez ishlatiladigan naqsh uchun stenografiyadir.

E'tibor bering, sintaksisdan foydalanish @funksiyani to'g'ridan-to'g'ri belgilash vaqtida bezatadi. Bu mo'rt xakerlarsiz bezaksiz asl nusxaga kirishni qiyinlashtiradi. Shuning uchun, bezaksiz funksiyani chaqirish imkoniyatini saqlab qolish uchun siz ba'zi funksiyalarni qo'lda bezashingiz mumkin.

Hozirgacha juda yaxshi. Keling, bu qanday amalga oshirilganini ko'rib chiqaylik.

Dekoratorlar xatti-harakatni o'zgartirishi mumkin

Endi siz dekorator sintaksisi bilan bir oz tanish bo'lganingizdan so'ng, keling, *aslida* biror narsani bajaradigan va bezatilgan funksiyaning harakatini o'zgartiradigan boshqa dekorator yozamiz.

Mana, bezatilgan funksiya natijasini katta harflarga o'zgartiradigan biroz murakkabroq dekorator:

```
def uppercase(func):
    def wrapper():
        original_result = func()
        modified_result = original_result.upper()
        return modified_result
    return wrapper
```

Kirish funksiyasini null dekorator qilganidek oddiygina qaytarish o'rniga, dekorator uppercaseyangi funksiyani (yopish) aniqlaydi va undan kirish funksiyasini chaqirish vaqtida uning harakatini o'zgartirish uchun o'rash uchun ishlatadi.

Yopish wrapperbezaksiz kirish funksiyasiga kirish huquqiga ega va kirish funksiyasi chaqirilishidan oldin va keyin qo'shimcha kodni bajarishi mumkin. (Texnik jihatdan kiritish funksiyasini umuman chaqirish shart emas).

E'tibor bering, bezatilgan funksiya ilgari hech qachon bajarilmagan. Aslida, bu nuqtada kirish funksiyasini chaqirish hech qanday ma'noga ega emas - dekorator chaqirilganda o'zining kiritish funksiyasining harakatini o'zgartirishi kerak.

Uppercase dekoratorning harakatini ko'rish vaqti keldi . Agar asl funksiyani u bilan bezasangiz nima bo'ladi

```
@uppercase
def greet():
    return 'Hello!'
>>> greet()
'HELLO!'
```

Umid qilamanki, bu siz kutgan natija bo'ldi. Keling, bu erda nima sodir bo'lganini batafsil ko'rib chiqaylik. dan farqli o'laroq null_decorator, dekorator funksiyani bezashda *boshqa funksiya obyektini* uppercase qaytaradi :

```
>>> greet
<function greet at 0x10e9f0950>
>>> null_decorator(greet)
```

```
<function greet at 0x10e9f0950>
>>> uppercase(greet)
<function uppercase.<locals>.wrapper at 0x10da02f28>
```

Avval ko'rganingizdek, bu bezatilgan funksiya chaqirilganda uning harakatini o'zgartirish uchun kerak. Katta harf dekoratorining o'zi funksiyadir. Va u bezatgan kirish funksiyasining "kelajakdagi xatti-harakatiga" ta'sir qilishning yagona yo'li - kirish funksiyasini yopish bilan almashtirish (yoki o'rash).

Shuning uchun uppercaseu keyinroq chaqirilishi mumkin bo'lgan boshqa funksiyani (yopish) belgilaydi va qaytaradi, dastlabki kiritish funksiyasini ishga tushiradi va uning chiqishini o'zgartiradi.

Dekoratorlar qo'ng'iroq qilinadigan obyektning harakatini o'ram yordamida o'zgartiradilar, shuning uchun siz doimo asl nusxani o'zgartirishingiz shart emas. Chaqiriladigan obyekt doimiy o'zgarishlarga duch kelmaydi - uning xatti-harakati faqat bezatilganida o'zgaradi.

Bu sizga mavjud funksiyalar va sinflarni jurnallar va boshqa vositalar kabi qayta foydalanish mumkin bo'lgan modullar bilan "biriktirish" imkonini beradi. Bu dekorativlarni Python-da ko'pincha standart kutubxona va uchinchi tomon paketlarida ishlatiladigan kuchli xususiyatga aylantiradi.

Qisqa tanaffus

Aytgancha, agar siz hozirgi vaqtda qisqa kofe tanaffusga muhtoj ekanligingizni his qilsangiz, bu mutlaqo normaldir. Menimcha, yopilishlar va dekorativlar Pythonda tushunish eng qiyin tushunchalardan biridir. Shoshilmang va birdaniga hamma narsani tushunishga urinmang. Tarjimon sessiyasida birin-ketin misollar keltirish ko'pincha mazmunni tushunishga yordam beradi.

Bilaman, hammasi siz uchun yaxshi bo'ladi :)

Bitta funksiya bir nechta dekorativlarni qo'llash

Funksiyaga bir nechta dekorator qo'llanilishi ajablanarli emas. Bu ularning effektlarini to'playdi va dekorativlarni qayta foydalanish mumkin bo'lgan modullar kabi foydali qiladi.

Mana bir misol. Keyingi ikkita dekorator bezatilgan funksiyaning chiqish satrini HTML teglari bilan o'rab oladi. Teglar qanday joylashtirilganiga qarab, Python bir nechta dekorativlarni qo'llash tartibini ko'rishingiz mumkin:

```
def strong(func):
    def wrapper():
        return '<strong>' + func() + '</strong>'
    return wrapper

def emphasis(func):
    def wrapper():
        return '<em>' + func() + '</em>'
    return wrapper
```

Keling, ushbu ikkita dekoratorni olib, ularni greetbir vaqtning o'zida bizning funksiyamizga qo'llaymiz. Buni amalga oshirish uchun siz odatiy sintaksisdan foydalanishingiz @va bitta funksiya ustiga bir nechta dekorativlarni joylashtirishingiz mumkin:

```
@strong
@emphasis
def greet():
    return 'Hello!'
```

Bezatilgan funksiyani ishga tushirsangiz, qanday natija ko'rishni kutasiz? @emphasis dekoratori avval o'z tegini qo'shadimi yoki @strong ustunlik qiladimi? Bezatilgan funksiyani chaqirganingizda nima sodir bo'ladi:

```
>>> greet()
'<strong><em>Hello!</em></strong>'
```

Bu yerda siz dekorativlar qanday tartibda ishlatilganligini aniq ko'rishingiz mumkin: *pastdan yuqoriga* . Dastlab kiritish funksiyasi @emphasis decorator bilan o'ralgan , so'ngra hosil bo'lgan (bezatilgan) funksiya yana @strong decorator bilan o'ralgan .

Ushbu pastdan yuqoriga tartibni eslash uchun men bu xatti-harakatni "dekorator to'plami" deb atashni yaxshi ko'raman. Siz stackni pastki qismdan qurishni boshlaysiz va keyin yuqoriga ko'tarilish uchun yangi bloklarni qo'shishda davom etasiz.

Agar biz yuqoridagi misolni buzsak va dekorativlarni qo'llash uchun sintaksisdan foydalanmasak @, dekorativ funksiyalarga qo'ng'iroqlar zanjiri quyidagicha ko'rinadi:

```
decorated_greet = strong(emphasis(greet))
```

Bu yerda yana dekoratorning birinchi qo'llanilishini ko'rishingiz mumkin emphasis, keyin esa natijada o'ralgan funksiya yana dekoratorga o'ralgan strong.

Bu shuningdek, dekorativ qoplamaning chuqur darajalari oxir-oqibat ishlashga ta'sir qiladi, chunki ular ichki funksiya chaqiruvlarini qo'shishda davom etadilar. Amalda bu odatda muammo emas, lekin agar siz unumdorlikni talab qiluvchi kod ustida ishlayotgan bo'lsangiz, buni yodda tutish kerak.

Argumentlarni qabul qiladigan dekoratsiya funksiyalari

Greet hozirgacha berilgan barcha misollar hech qanday dalil talab qilmaydigan oddiy nullary funksiyani bezatadi . Shunday qilib, siz bu erda ko'rgan dekoratorlar kiritish funksiyasiga argumentlarni uzatish bilan shug'ullanmagan.

Agar siz ushbu dekoratorlardan birini argumentlarni qabul qiluvchi funksiyaga qo'llamoqchi bo'lsangiz, u to'g'ri ishlamaydi. O'zboshimchalik bilan argumentlar oladigan funksiyani qanday bezash kerak?

Bu yerda Python funksiyasi o'zgaruvchan sonli argumentlar bilan ishlashda **args* yordam beradi. ***kwargs* Dekorator proxyushbu xususiyatdan foydalanadi:

```
def proxy(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```

Dekoratorlarning ishlash tamoyili

- Dekorator — bu boshqa funksiyaga “o‘ralib” ishlaydigan funksiya.
- Dekorator ichidagi **args* va ***kwargs* yozilishi shuni anglatadiki, **funksiya chaqirilganda kelgan barcha pozitsion va kalit argumentlarni yig‘ib olish** mumkin.

- Keyin ular asl funksiyaga qaytadan uzatiladi (*func(*args, **kwargs)* orqali).

Shu orqali dekorator asl funksiyaga qo‘shimcha imkoniyatlar qo‘shib beradi, masalan, log yozish, vaqtni o‘lchash yoki natijani o‘zgartirish.

Misol: trace dekoratori

```
def trace(func):
    def wrapper(*args, **kwargs):
        print(f'TRACE: calling {func.__name__}() '
              f'with {args}, {kwargs}')

        original_result = func(*args, **kwargs)

        print(f'TRACE: {func.__name__}() '
              f'returned {original_result!r}')

        return original_result
    return wrapper
```

Bu dekorator har bir funksiya chaqirilganda:

1. Uning nomi va argumentlarini chiqaradi.
2. Natijani ham ekranga yozadi.

Foydasi: **disk raskadrovka (debug)** jarayonida nima chaqirilayotganini va nima qaytarilayotganini aniq ko‘rish mumkin.

Dekoratorlardan keladigan muammo

Dekorator asl funksiyani boshqa funksiya (*wrapper*) bilan almashtiradi. Shu sababli **asl funksiyaning metama’lumotlari yo‘qoladi**:

- funksiya nomi (*__name__*),
- docstring (*__doc__*),
- parametrlar haqida ma’lumot.

Masalan:

```
def greet():
    """Return a friendly greeting."""
    return 'Hello!'
```



```
decorated_greet = uppercase(greet)

print(decorated_greet.__name__) # 'wrapper'
print(decorated_greet.__doc__)  # None
```

Ko'rib turganingizdek, greet funksiyasining nomi va izohi yo'qolib ketadi.

Yechim: functools.wraps

Buni tuzatish uchun Python functools.wraps dekoratorini beradi.

U asl funksiya metama'lumotlarini o'ram (wrapper) ichiga **nusxalaydi**.

```
import functools

def uppercase(func):
    @functools.wraps(func)
    def wrapper():
        return func().upper()
    return wrapper
```

Endi:

```
@uppercase
def greet():
    """Return a friendly greeting."""
    return 'Hello!'

print(greet.__name__) # 'greet'
print(greet.__doc__)  # 'Return a friendly greeting.'
```

4. Lambda funksiyasi

Sun'iy intellekt asoslaridagi lambda ifodalari - bu lambda operatori yordamida aniqlangan kichik anonim funksiyalar. Lambda ifodasining sintaksisi quyidagicha:

```
lambda [parametrlar]: operatorlar
```

Eng oddiy lambda ifodasini aniqlaymiz:

```
msg = lambda: print("salom")
msg() # salom
```

Bu yerda lambda ifodasi msg o'zgaruvchisiga yuklatilgan. Ushbu misolda lambda ifodasi hech qanday parametrga ega emas, hech narsa qaytarmaydi va shunchaki konsolga "salom" xabarini chop etadi. Va msg o'zgaruvchisi orqali bu lambda ifodasini oddiy funksiya kabi chaqirishimiz mumkin. Aslida, u quyidagi funksiyaga o'xshaydi:

```
def msg():
    print("salom")
```

Agar lambda ifodasi parametrlarga ega bo'lsa, ular lambda kalit so'zidan keyin aniqlanadi. Agar lambda ifodasi natijani qaytarsa, u holda u ikki nuqtadan keyin ko'rsatiladi. Masalan, sonning kvadratini qaytaruvchi lambda ifodasini yozaylik:

```
kvadrat = lambda a : a * a
print(kvadrat(4)) # 16
print(kvadrat(5)) # 25
```

Bunday holda, lambda ifodasi bitta a nomli parametrni qabul qiladi va undan keyingi Ikki nuqtaning o'ng tomonida qaytariladigan qiymat - $a * a$. Ushbu lambda ifodasi quyidagi funksiyaning alternativi hisoblanadi:

```
def kvadrat2(a): return a * a
```

Xuddi shu usulda, bir nechta parametrlarni qabul qiluvchi lambda ifodalarini yaratish mumkin:

```
sum = lambda a, b: a + b
print(sum(4, 5))      # 9
print(sum(5, 6))      # 11
```

Lambda ifodalari funksiya ta'riflarini biroz qisqartirishga imkon bersada, ular faqat bitta operatsiyani bajarishi mumkinligi bilan cheklangan. Biroq, parametr sifatida o'tish yoki boshqa funksiyaga qaytish uchun funksiyadan foydalanish kerak bo'lgan hollarda ular juda qulay bo'lishi mumkin. Masalan, lambda ifodasini parametr sifatida o'tkazish:

```
def do_operatsiya(a, b, operatsiya):
    result = operatsiya(a, b)
    print(f"result = {result}")
do_operatsiya(5, 4, lambda a, b: a + b)  #result = 9
do_operatsiya(5, 4, lambda a, b: a * b)  #result = 20
```

Bunday holda, oxirgi mavzuda bo'lgani kabi, ularni parametr sifatida jo'natish uchun funksiyalarni belgilashimiz shart emas.

Xuddi shu narsa funksiyalardan lambda ifodalarini qaytarish uchun ham amal qiladi:

```
def tanlash_funk(tanlangan):
    if tanlangan == 1:
        return lambda a, b: a + b
    elif tanlangan == 2:
        return lambda a, b: a - b
    else:
        return lambda a, b: a * b

operation = tanlash_funk (1) # operation = a+b
print(operation(10, 6))    # 16

operation = tanlash_funk (2) # operation = a-b
print(operation(10, 6))    # 4

operation = tanlash_funk (3) # operation = a*b
print(operation(10, 6))    # 60
```

Nazorat savollari:

1. Lambda funksiyasining vazifasi nima?
2. Lambda funksiyasining sintaksisi va ishlash prinsipi qanday?
3. Oddiy funksiyadan qanday afzallik va kamchiliklarga ega?
4. Lokal funksiya deb qanday funksiyalarga aytiladi?
5. Funksiyalar dasturlashda qanday ro'l o'ynaydi?

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

11-mashg’ulot. Pythonda Funksiyaga doir dasturlar tuzish.

O‘quv savollari:

1. Funksiyalarni aniqlash va uni chaqirishni o‘rganish.
2. Parametrli va parametrsiz funksiyalarga doir dastur tuzish.

Funksiyalarni aniqlash va uni chaqirish haqida ma’lumot

Python dasturlash tilida **funksiya** – bu biror vazifani bajarish uchun yozilgan kodlar to‘plamidir. Funksiyalar yordamida dastur qismlarga bo‘linadi, kod takrorlanishini kamaytiradi va uni tushunarli qiladi.

Funksiyani aniqlash

Python’da funksiya **def** kalit so‘zi yordamida aniqlanadi.

Sintaksis:

```
def funksiya_nomi(parametrlar):
    # Funksiya tanasi
    # bajariladigan amallar
    return natija
```

def – funksiyaning aniqlash uchun ishlatiladi.

funksiya_nomi – funksiya berilgan nom (masalan, hisobla, salom_ber).

parametrlar – funksiya uzatiladigan qiymatlar (ixtiyoriy).

return – funksiya natijasini qaytaradi (bo‘lishi shart emas).

Funksiyani chaqirish

Funksiya aniqlangandan keyin uni **chaqirish** orqali ishlatamiz.

Sintaksis:

```
funksiya_nomi(argumentlar)
```

Oddiy misollar

Parametrsiz funksiya

```
def salom_ber():
    print("Assalomu alaykum!")
```

Funksiyani chaqirish

```
salom_ber()
```

Natija:

Assalomu alaykum!

Parametrli funksiya

```
def salom_ber(ism):
    print("Assalomu alaykum,", ism)
# Funksiyani chaqirish
salom_ber("Ali")
salom_ber("Madina")
```

Natija:

Assalomu alaykum, Ali

Assalomu alaykum, Madina

return operatoridan foydalanish

```
def kvadrat(x):
    return x * x
# Funksiyani chaqirish
natija = kvadrat(5)
print("Natija:", natija)
```

Natija:

Natija: 25

Bir nechta parametrlı funksiya

```
def yigindi(a, b):
    return a + b
print("3 + 7 =", yigindi(3, 7))
print("10 + 20 =", yigindi(10, 20))
```

Natija:

3 + 7 = 10

10 + 20 = 30

Standart qiymatli parametrlar

Agar foydalanuvchi parametr bermasa, funksiyada **standart qiymat** ishlatiladi.

```
def salom_ber(ism="Do'st"):
    print("Salom,", ism)
salom_ber() # standart qiymat ishlatiladi
salom_ber("Rustam") # argument berildi
```

Natija:

Salom, Do'st

Salom, Rustam

- Funksiya kodni tartibli va qayta foydalanish mumkin bo'lgan shaklga keltiradi.
- def orqali aniqlanadi va **chaqirilganda** bajariladi.
- Parametrlar orqali funksiya turli qiymatlar bilan ishlashi mumkin.
- return natijani qaytarish uchun ishlatiladi.

Parametrlı va parametrsiz funksiyalar

Python dasturlash tilida **funksiya** – bu biror vazifani bajaruvchi, qayta-qayta ishlatilishi mumkin bo'lgan kod bo'lagi. Funksiyalar dastur tuzilishini soddalashtiradi va kodni tartibli qiladi.

Funksiya ikki xil bo'ladi:

Parametrsiz funksiya – hech qanday kirish qiymatisiz ishlaydi.

Parametrlı funksiya – unga tashqaridan ma'lumot (argument) yuboriladi.

Parametrsiz funksiya

Parametrsiz funksiya faqat o'zidagi kodni bajaradi, tashqaridan qiymat olmaydi.

```
def salom_ber():
    print("Assalomu alaykum, xush kelibsiz!")
# Funksiyani chaqirish
salom_ber()
```

Bu yerda `salom_ber()` funksiyasi hech qanday parametr olmaydi. Uni chaqirganimizda ekranga matn chiqaradi.

Parametrli funksiya

Parametrli funksiya tashqaridan qiymat qabul qiladi va shu qiymatlar asosida natija qaytaradi.

```
def salom_ber_ism(ism):
    print(f"Assalomu alaykum, {ism}!")
# Funksiyani chaqirish
salom_ber_ism("Rasulbek")
salom_ber_ism("Dilnoza")
```

Bu yerda `ism` parametri orqali foydalanuvchi ismi funksiyaga uzatiladi va natija shunga mos chiqadi.

Bir nechta parametrli funksiya

Funksiya birdan ortiq parametr ham olishi mumkin.

```
def kvadrat_hisobla(a, b):
    natija = a ** 2 + b ** 2
    print(f"{a} va {b} sonlarining kvadratlar yig'indisi = {natija}")
# Chaqirish
kvadrat_hisobla(3, 4)
```

Bu misolda funksiya ikkita parametr (`a` va `b`) qabul qiladi va ularning kvadratlari yig'indisini hisoblaydi.

Natija qaytaruvchi funksiya (return)

Funksiya bajarilganidan so'ng natija qaytarishi ham mumkin. Buning uchun `return` operatori ishlatiladi.

```
def kvadrat(son):
    return son ** 2
```

Funksiyani ishlatish

```
natija = kvadrat(6)
print("6 ning kvadrati =", natija)
```

Bu misolda `kvadrat()` funksiyasi sonning kvadratini qaytaradi. Uni keyin boshqa hisoblarda ham ishlatish mumkin.

Parametrsiz funksiya – tashqaridan qiymat olmaydi, faqat ichidagi kodni bajaradi.

Parametrli funksiya – tashqaridan qiymat olib, uni qayta ishlaydi.

Funksiyalar kodni ixcham va qayta foydalanishga yaroqli qiladi.

`return` operatori yordamida funksiyadan natija qaytarish mumkin.

Nazorat savollari:

1. Funksiya nima va u dastur tuzishda qanday qulaylik beradi?
2. Parametrsiz funksiyaga misol keltiring.
3. Parametrli funksiyani ishlatish jarayonini tushuntiring.
4. `return` operatori nima vazifa bajaradi?
5. Bir nechta parametrli funksiyaga misol yozing.

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

12-mashg’ulot. Fayllar va kataloglar bilan ishlash.

O‘quv savollari:

1. Faylni ochish. Fayllar bilan ishlovchi metodlar.
2. os modulining imkoniyatlari.
3. Fayl va katalog yo‘lini o‘zgartirish.
4. Katalog va fayl bilan ishlovchi funksiya va metodlar.

1. Faylni ochish. Fayllar bilan ishlovchi metodlar.

Sun’iy intellekt asoslaridagi fayllar bilan o‘zaro aloqada bo‘lish foydalanuvchiga ilova tomonidan qayta ishlangan ma’lumotlarni saqlash imkoniyatini beradi va foydalanuvchi keyinchalik istalgan qulay vaqtda faylga murojaat qilishi mumkin bo‘ladi. Ushbu Sun’iy intellekt asoslarining asosiy funksiyalari fayllarni yaratish, faylga ma’lumotlarni yozish va fayldan ma’lumotlarni o‘qishni ancha osonlashtiradi.

Faylni yaratish va ochish

Fayl bilan ishlash uchun albatta avval uni yaratish kerak. Buni standart operatsion tizim vositalari yordamida kerakli katalogga o‘tish va ***.txt** formati bilan yangi hujjat yaratish orqali amalga oshirish ham mumkin. Biroq, shunga o‘xshash harakat Sun’iy intellekt asoslarida **open()** metodi yordamida amalga oshiriladi. Bu metodga **fayl nomi** va uni **qayta ishlash rejimi** parametrlar sifatida berilishi kerak.

Quyidagi kod fayl o‘zgaruvchisini yangi hujjatga ishora qilishini ko‘rsatadi. Agar ushbu dasturni ishga tushirilsa, u manba kodi saqlanadigan papkada **test.txt** matn faylini yaratadi.

```
file_txt = open("test.txt", "w")
file_txt.close()
```

Agar **test.txt** nomli fayl kod katalogida allaqachon mavjud bo‘lsa, dastur yangi hujjat yaratmasdan u bilan ishlashni davom ettiradi. Ko‘rib turganingizdek, fayl nomi **open()** metodining birinchi parametridir. Undan so‘ng ma’lumotlarni **qayta ishlash usulini** ko‘rsatadigan maxsus belgi parametr sifatida keladi. Bu yerda ikkinchi parametr sifatida kelgan “w” - fayl faqat yozish uchun ochilayotganini bildiradi.

Endi ma’lumot yo‘qolmasligini ta’minlash uchun fayl ustida turli amallarni bajarilgandan so‘ng, uni **close()** funksiyasidan foydalanib yopish kerak bo‘ladi.

Oldingi misolda faylga kirish uchun nisbiy yo‘l ishlatilgan bo‘lib, u qattiq diskdagi obyektning joylashuvi haqida to‘liq ma’lumotni o‘z ichiga olmaydi. Ularni o‘rnatish uchun **open()** funksiyaga birinchi argument sifatida mutlaq yo‘lni ko‘rsatilishi kerak. Bunday holda, test.txt hujjati dastur papkasida emas, balki D diskidagi qaysidir bir katalogga joylashgan bo‘ladi.

```
file_txt = open(r"D:\test\test.txt", "w")
file_txt.close()
```

Bu yerda faylga yo'lni ko'rsatishda satrdan oldin **r** belgisi qo'yiladi. Aks holda, kompilyator "\" ketma-ketligini maxsus operator sifatida ko'rib chiqadi va xatolik kelib chiqadi.

Faylni ochilish rejimlari

Barcha dasturlash tillaridagi kabi Sun'iy intellekt asoslarida ham faylni ochishda ishlatiladigan maxsus belgilardan foydalaniladi. Ular faylni ochish rejimi deb ataladi va faylni qanday maqsadda ochishni aniq beradi. Ularning barchasi quyidagi jadvalda keltirilgan, unda ularning belgisi va qisqacha mazmuni keltirilgan:

5-jadval

Belgi	Mazmuni
«r»	O'qish uchun ochiq (standart)
«w»	Yozish uchun ochiladi. Agar fayl ko'rsatilgan kotalogda mavjud bo'lmasa, yangisi yaratiladi. Yozilgan ma'lumot hujjatdagi barcha ma'lumotlarni o'rniga yoziladi
«a»	Yozish uchun ochiladi. Yozilgan ma'lumot hujjatning oxiriga qo'shiladi
«b»	Faylni ikkilik rejimda ochish
«t»	Faylni matn rejimida ochish (standart)
«+»	O'qish va yozish uchun bir vaqtning o'zida ochish. Ushbu amal alohida o'zi ochish rejimi sifatida ishlatilmaydi. a+, r+, w+ kabi ishlatiladi.

Open() metodining ikkinchi argumentidan foydalanib, turli xil fayl rejimlarini birlashtirish mumkin. Masalan, yozma ma'lumotlarni ikkilik rejimda o'qish uchun "rb" ni belgilash kerak.

Yana bir misol: "r+" va "w+" o'rtasidagi farq shundaki, "w+" - agar fayl mavjud bo'lmasa, yangi fayl yaratiladi. Birinchi holda, xatolik kelib chiqadi. "r+" va "w+" dan foydalanish faylni o'qish va yozish uchun ochadi.

Usullari

open() funksiyasi tomonidan qaytarilgan obyekt mavjud faylga tavsifni o'z ichiga olgan to'rtta parametрни qaytaradi. Quyidagi jadvalda ularning barchasi nomlari va ma'nolari keltirilgan:

6-jadval

Nomi	Ma'nosi
name	fayl nomini qaytaradi
mode	ochilgan rejimni qaytaradi
closed	fayl yopiq bo'lsa true ni, ochiq bo'lsa false qiymatini qaytaradi
softspace	agar faylning chiqishi alohida bo'sh joy belgisini o'z ichiga olmasa, true qiymatini qaytaradi

Faylning xususiyatlarini ko'rsatish uchun kirish operatoridan, ya'ni nuqtadan foydalanish va keyin buni **print()** funksiyasiga parametr sifatida berish kifoya bo'ladi.

Masalan :

```
f = open(r"D:\test\test.txt", "w")
print(f.name)
f.close()
```

Natijasi:

```
D:\test\test.txt
```

Faylga ma'lumot yozish

Faylga ma'lumot yozish **write()** usuli yordamida amalga oshiriladi. Usul mavjud faylga ishora qiluvchi obyektga chaqiriladi. Shuni esda tutish kerakki, buni amalga oshirish uchun avval hujjatni ochish funksiyasidan foydalanib ochish va "w" belgisi bilan yozish rejimini belgilash kerak bo'ladi. **Write()** usuli argument sifatida matn fayliga yoziladigan ma'lumotlarni qabul qiladi. Quyidagi kod misolida "Salom dunyo!" satrini kiritilishi ko'rsatilgan:

```
file_txt = open("test.txt", "w")
file_txt.write("Salom dunyo!")
file_txt.close()
```

Agar ilgari yozilgan ma'lumotlarga yangi ma'lumotlarni qo'shish kerak bo'lsa, u holda ochish rejimi sifatida "a" belgisini ko'rsatib, **open** funksiyasini qayta chaqirish kerak. Aks holda, **test.txt** faylidagi barcha ma'lumotlar butunlay o'chirib tashlanadi. Quyidagi kod misolida faylga ma'lumot qo'shish uchun matnli hujjatni ochadi. Shundan so'ng unda " Salom dunyo!" satri hujjat oxiriga joylashtiriladi. Shunday qilib, **test.txt** faylida "Salom dunyo! Salom dunyo!" satri paydo bo'ladi. Bularning barchasidan so'ng, faylni majburiy yopish haqida unutmang.

```
file_txt = open("test.txt", "a")
file_txt.write(" Salom dunyo!")
file_txt.close()
```

Matn faylga ma'lumotlarni yozishning eng oddiy protsedurasi shunday amalga oshiriladi. Shuni ta'kidlash kerakki, Sun'iy intellekt asoslarida hujjatlar bilan yanada ilg'or ishlash uchun ko'plab qo'shimcha vositalar mavjud bo'lib, ular yaxshilangan yozishni ham o'z ichiga oladi.

Quyidagi matn mazmuni tozalandi, mantiqiy xatolar tuzatildi va sintaksis ham tartibga keltirildi. Kod namunalari Python'ning to'g'ri amaliyotlariga moslab yangilandi.

1) IKKILIK (BINARY) MA'LUMOTLARNI YOZISH

Ikkilik rejimda yozish uchun `open(..., "wb")` dan foydalaning. Masalan, UTF-8 kodlashda satrni baytlarga aylantirib yozish:

```
# test.dat fayliga UTF-8 kodlashda yozish
with open('test.dat', 'wb') as f:
    f.write('наша строка'.encode('utf-8')) # bytes(..., 'utf-8') ham bo'ladi
```

with blokidan foydalanish faylni avtomatik yopadi, `close()` chaqirish shart emas.

2) MATNLI FAYLDAN O'QISH

Matn faylidan o'qish uchun "r" rejimini tanlang va `read()` metodidan foydalaning:

```
try:
    with open("test.txt", "r", encoding="utf-8") as f:
        print(f.read()) # butun faylni o'qiydi
except FileNotFoundError:
    print('Fayl topilmadi!')
except OSError:
    print('Qandaydir xatolik!')
```

`read(n)` varianti n ta belgi/bytni o'qiydi. Masalan, faqat "Salom" so'zini o'qish uchun:

```
with open("test.txt", "r", encoding="utf-8") as f:
    print(f.read(5)) # "SaLoM"
```

Eslatma: with blokidan foydalanish **faylni yopishni** avtomatlashtiradi, ammo **istisnolarni o'zi bostirmaydi** — xatoni ushlamoqchi bo'lsangiz, baribir try/except yozishingiz kerak.

3) IKKILIK (BINARY) MA'LUMOTLARNI O'QISH

Ikkilik rejimda o'qish uchun "rb" dan foydalaning. Quyidagi misol faylni baytma-bayt o'qib, ularni o'n oltilik ko'rinishda yig'adi:

```
try:
    hex_str = ""
    with open("test.dat", "rb") as f:
        while True:
            b = f.read(1) # 1 bayt o'qish
            if b == b"": # EOF
                break
            hex_str += b.hex()
        print(hex_str)
except OSError:
    print('Xatolik yuz berdi')
```

Namunaviy natija:

```
81d182d180d0bed0bad0b081d182d180d0bed0bad0b0
```

Bu yerda har bir o'qilgan bayt `hex()` orqali o'n oltilik (hex) ko'rinishga aylantirilib, satrga qo'shiladi.

4) WITH ... AS KONSTRUKSIYASI

`with` bloklari fayl (yoki boshqa resurs) bilan ishlashni soddalashtiradi:

- `close()` ni qo'lda chaqirish shart emas — blok tugashi bilan avtomatik yopiladi.
- Xatolik bo'lsa ham resurs to'g'ri yopiladi, lekin **istisno baribir ko'tariladi** (ya'ni xatoni ushlash uchun `try/except` kerak bo'lsa, alohida yozing).

```
with open('test.txt', 'r', encoding='utf-8') as f:
    print(f.read())
```

Agar fayl mavjud bo'lmasa, blok ichidagi kod ishlamaydi va `FileNotFoundError` ko'tariladi.

5) OS MODULINING IMKONIYATLARI

`os` — operatsion tizim bilan ishlashga yordam beradigan standart kutubxona. Fayl/kataloglarni boshqarish, muhit o'zgaruvchilariga kirish, yo'llar bilan ishlash kabi ko'plab funksiyalarni taklif etadi.

5.1. OT haqida ma'lumot

```
import os
print(os.name)      # 'nt' (Windows), 'posix' (Linux/macOS) va h.k.
```

Muhit o'zgaruvchilari:

```
import os

print(os.environ)    # muhit o'zgaruvchilari xaritasi
print(os.getenv("TMP")) # aniq o'zgaruvchini olish
```

5.2. Ishchi katalog

```
import os

print(os.getcwd())    # joriy ishchi katalog
os.chdir(r"D:\folder") # ishchi katalogni almashtirish
print(os.getcwd())
```

6) FAYL VA KATALOG YO'LLARI

Yo'llar bilan ishlashda `os.path` yoki zamonaviyroq `pathlib` dan foydalanish tavsiya etiladi.

6.1. Faylga mutlaq yo'l

```
import os
print(os.path.abspath('file.txt'))

from pathlib import Path
print(Path('file.txt').resolve())
```

6.2. To'liq yo'ldan fayl nomi

```
import os
file_name = os.path.basename(r'C:\python\file.txt')
print(file_name) # file.txt
```

6.3. Kengaytmasiz fayl nomi

```
from os import path

full_name = path.basename(r'C:\python\file.tar.gz')
name_wo_last_ext = path.splitext(full_name)[0]
print(name_wo_last_ext) # file.tar

# Birinchi nuqtagacha faqat bazaviy nom kerak bo'lsa:
base = name_wo_last_ext.split('.', 1)[0]
print(base) # file
```

6.4. Fayl kengaytmasi (bir yoki ko'p)

```
from os import path

full_name = path.basename(r'C:\python\file.tar.gz')
ext = path.splitext(full_name)[1]
print(ext) # .gz

from pathlib import Path
p = Path(r'C:\python\file.tar.gz')
print(p.suffix) # .gz
print(p.suffixes) # ['.tar', '.gz']
```

7) FAYLLAR BILAN ISHLOVCHI METODLAR

7.1. Mavjudligini tekshirish

```
import os

print(os.path.exists(r"D:\test\test.txt")) # True/False
print(os.path.isfile(r"D:\test\test.txt")) # True agar fayl bo'lsa
print(os.path.isdir(r"D:\test\folder")) # True agar papka bo'lsa
```

7.2. Nusxa ko'chirish (shutil)

```
import shutil

# Faqat kontentni ko'chirish (metama'lumotlarsiz)
shutil.copyfile(r"D:\test\test.txt", r"D:\test\test2.txt")

# Nomni saqlagan holda papkaga nusxa
shutil.copy(r"D:\test\test.txt", r"D:\folder")

# Kontent + metama'lumotlar (vaqt tamg'alari va h.k.)
shutil.copy2(r"D:\test\test.txt", r"D:\folder\test2.txt")
```

7.3. O'chirish

```
import os
os.remove(r"D:\test\test.txt")

# yoki pathlib bilan:
from pathlib import Path
Path(r"D:\test\test.txt").unlink(missing_ok=True)
```

7.4. Fayl hajmi

```
import os
print(os.path.getsize(r"D:\test\test.txt")) # baytlarda
```

```
# pathlib orqali:
from pathlib import Path
print(Path(r"D:\test\test.txt").stat().st_size)
```

seek/tell orqali (o'qishsiz) o'lcham olish:

```
import os
with open(r"D:\test\test.txt", "rb") as f:
    f.seek(0, os.SEEK_END)
    print(f.tell())
```

7.5. Nomini o'zgartirish / ko'chirish

```
import os
os.rename(r"D:\test\test.txt", r"D:\test\test1.txt")

# Ko'chirish yoki nomini o'zgartirish uchun yana bir usul:
import shutil
shutil.move(r"D:\test\test1.txt", r"D:\test\renamed.txt")
```

Yakuniy eslatmalar

- Ikkilik rejimda ("wb", "rb") **baytlar** bilan ishlaysiz; matn rejimida ("w", "r") **satrlar** bilan ishlaysiz.
- with blokidan foydalanish — **eng yaxshi amaliyot**: resursni to'g'ri yopadi.
- Xatolarni ushlash kerak bo'lsa, try/except dan foydalaning — with istisnolarni avtomatik yutmaydi.
- Yo'llar bilan ko'p platformali ishlash uchun pathlib qulayroq.
- Nusxa ko'chirishda metama'lumotlar ham kerak bo'lsa, shutil.copy2 ni tanlang.

Nazorat savollari:

1. Mutlaq fayl yo'li qanday olinadi?
2. Fayl nomini qanday olish mumkin?
3. Fayl kengaytmasini qanday olish mumkin?
4. Fayl kengaytmasini o'zgartirish qanday amalga oshiriladi?
5. Qanday turdagi fayllar mavjud?
6. Fayl yaratishning qanday usullari mavjud?

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

13-mashg’ulot. Fayllar va kataloglar bilan ishlashga doir masalalar yechish.

O‘quv savollari:

1. Fayllar bilan ishlash metodlaridan foydalanish.
2. Kataloglar ustida amallar bajarish.
3. Fayl va kataloglarda qidirish va nusxa olish.

Fayllar bilan ishlovchi metodlardan foydalanib dasturlar tuzish.

Dasturlash jarayonida ko‘plab hollarda ma’lumotlarni faqatgina vaqtinchalik o‘zgaruvchilar orqali saqlash yetarli bo‘lmaydi. Chunki dastur yopilgandan so‘ng o‘zgaruvchilarda saqlangan barcha ma’lumotlar xotiradan o‘chib ketadi. Shu sababli dasturchilar doimiy ravishda **fayllar** bilan ishlashga muhtoj bo‘ladi.

Fayllar yordamida biz:

Matnli ma’lumotlarni saqlash,

Foydalanuvchilardan kiritilgan qiymatlarni yozib borish,

Hisobotlar tuzish,

Foydalanuvchi dasturdan chiqqanidan keyin ham saqlanib qoladigan bazaviy ma’lumotlarni yig‘ishimiz mumkin.

Bundan tashqari, **kataloglar (papkalar)** yordamida fayllar tartibli saqlanadi. Dasturchi kataloglar ichida fayl yaratishi, uni boshqa joyga ko‘chirishi yoki o‘chirishi mumkin.

1. Python’da fayllar bilan ishlash

Python’da fayl bilan ishlashning asosiy jarayoni uch bosqichdan iborat:

1. **Faylni ochish**
2. **Fayldan o‘qish yoki faylga yozish**
3. **Faylni yopish**

Faylni ochishda biz `open()` funksiyasidan foydalanamiz:

```
# Faylni ochish
fayl = open("matn.txt", "w")
fayl.write("Assalomu alaykum, bu faylga yozilayotgan matn!")
fayl.close()
```

Fayl ochish rejimlari:

- "r" – faqat o‘qish uchun (default)
- "w" – yozish uchun (avvalgi ma’lumotlar o‘chadi)
- "a" – yozishni davom ettirish (eski ma’lumot o‘chmaydi)
- "x" – yangi fayl yaratish (agar fayl bo‘lsa, xatolik beradi)
- "rb", "wb" – faylni ikkilik (binary) rejimida ochish

2. Fayldan o‘qish

Fayldan o'qish uchun turli metodlardan foydalaniladi:

```
# Bitta fayldan o'qish usullari
fayl = open("matn.txt", "r")
print(fayl.read())          # Butun faylni o'qish
fayl.seek(0)                # Imlochini boshiga qaytarish
print(fayl.readline())      # Faqat bitta satr o'qish
fayl.seek(0)
print(fayl.readlines())     # Fayldagi barcha satrlarni ro'yxat sifatida
                             # qaytaradi
fayl.close()
```

3. Faylga yozish

Faylga yozishda ikkita asosiy rejim ishlatiladi: "w" va "a".

```
# Faylga yozish ("w" rejimida)
fayl = open("data.txt", "w")
fayl.write("Birinch qatorda matn bor.\n")
fayl.write("Ikkinchi qatorda matn bor.\n")
fayl.close()

# Faylga yozish ("a" rejimida - qo'shib yozish)
fayl = open("data.txt", "a")
fayl.write("Yangi qator ham qo'shildi!\n")
fayl.close()
```

4. Fayl bilan ishlashda with operatori

Ko'pincha faylni yopishni unutib qo'yish mumkin. Shuning oldini olish uchun Python'da with konstruktsiyasi ishlatiladi:

```
with open("demo.txt", "w") as f:
    f.write("Salom, dunyo!")
# Fayl avtomatik ravishda yopiladi
```

5. Kataloglar bilan ishlash

Fayllardan tashqari kataloglar bilan ishlash ham muhimdir. Buning uchun Python'da os va shutil modullari ishlatiladi.

Asosiy funksiyalar (os kutubxonasi):

```
import os
# Joriy ishchi katalogni ko'rish
print(os.getcwd())
# Yangi katalog yaratish
os.mkdir("yangi_papka")
# Bir nechta katalog yaratish
os.makedirs("asosiy/sub1/sub2")
# Fayl yoki katalog bor-yo'qligini tekshirish
print(os.path.exists("yangi_papka"))
# Katalog ichidagi fayllarni ko'rish
print(os.listdir(".")) # Joriy katalogdagi fayllar
```

Faylni ko'chirish va o'chirish (shutil kutubxonasi):

```
import shutil
# Faylni ko'chirish
shutil.move("data.txt", "yangi_papka/data.txt")
# Faylni nusxalash
```

```
shutil.copy("yangi_papka/data.txt", "data_copy.txt")
# Faylni o'chirish
os.remove("data_copy.txt")
# Butun katalogni o'chirish
shutil.rmtree("asosiy")
```

Amaliy masalalar

1. Foydalanuvchi kiritgan ism va familiyani faylga yozing.
 2. Foydalanuvchi 5 ta son kiritadi, ular faylga yozilsin. Keyin fayldan o'qib, sonlarning yig'indisini hisoblang.
 3. Fayl ichidagi satrlarni alifbo bo'yicha tartiblang va ekranga chiqaring.
 4. Faylga yozilgan matndagi so'zlar sonini hisoblang.
 5. Berilgan papkada nechta .txt kengaytmali fayl borligini aniqlang.
- Fayllar va kataloglar bilan ishlash dasturlashda juda muhim hisoblanadi. Fayl orqali biz dasturdan mustaqil ma'lumotlarni saqlab qolamiz, kataloglar orqali esa ularni tartiblaymiz. Amaliy dasturlarda loglar, foydalanuvchi ma'lumotlari, hisobotlar va ko'plab boshqa axborotlar aynan fayllarda saqlanadi. Shu sababli bu mavzu har bir dasturchi uchun asosiy bilimlardan biridir.

Nazorat savollari:

1. Faylni ochish uchun open() funksiyasining qaysi rejimlari mavjud?
2. Faylni o'qishning read(), readline(), readlines() metodlari orasidagi farq nimada?
3. "w" va "a" rejimlari qanday vazifani bajaradi?
4. Fayl bilan ishlashda with operatorining afzalligi nimada?
5. os kutubxonasi yordamida katalog yaratish va o'chirish qanday amalga oshiriladi?
6. Faylni ko'chirish va nusxalash uchun qaysi kutubxona ishlatiladi?
7. Fayl ichidagi so'zlar sonini qanday hisoblash mumkin?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

14-mashg'ulot. Pythonda OOP asoslari.

O'quv savollari:

1. OOP asoslari. Klasslarni e'lon qilish va nusxasini yaratish.
2. Sinf va obyekt. Sinf konstruktori.
3. `__init__()` va `__del__()` metodlari.

1. OOP asoslari. Klasslarni e'lon qilish va nusxasini yaratish.

Pythonda dasturda foydalanish mumkin bo'lgan `int`, `str` va shunga o'xshash ko'plab o'rnatilgan turlar mavjud. Ammo Python, sinflar yordamida o'z tiplarimizni aniqlashga imkon beradi.

Quyidagi o'xshashlikni ham o'tkazish mumkin. Har birimiz ismi-sharifi, yoshi, va shu kabi boshqa xususiyatlarga ega bo'lgan inson haqida qandaydir tasavvurga egamiz. Inson muayyan harakatlarni bajarishi mumkin: yurish, yugurish, o'ylash va hokazo. O'ziga xos xususiyatlar va harakatlar majmuasini o'z ichiga olgan bu ko'rinishni **class** deb atash mumkin. Aniq bir xususiyatlarni o'zida jamlagan bu class obyektlar uchun shablon xisoblanadi va turli o'yektlar bir-biri bilan farq qiladi. masalan: bir odamning ism bunaqa bo'lsa, boshqalari boshqa ismga ega.

Sinf class kalit so'zi yordamida aniqlanadi:

```
class sinf_nomi:
    sinf_atributlari
    class_methods
```

Sinf ichida uning atributlari aniqlanadi, ular sinfnig turli xususiyatlarini saqlaydi va metodlar sinfnig funksiyalari vazifasini bajarishadi.

Keling, oddiy sinf yarataylik :

```
class Person:
    pass
```

Bunda shartli ravishda shaxsni ifodalovchi `Person` sinfi aniqlanadi. Bunday holda, sinfda hech qanday metodlar yoki atributlar aniqlanmaydi. Biroq, unda biror narsa aniqlanishi kerakligi sababli, `pass` iborasi sinf funkcionalligi o'rnini bosuvchi sifatida ishlatiladi. Ushbu bayonot sintaktik jihatdan ba'zi kodlarni aniqlash uchun zarur bo'lganda ishlatiladi, lekin biz buni xohlamaymiz va ma'lum bir kod o'rniga biz `pass` bayonotini kiritamiz.

Sinf yaratilgandan so'ng, ushbu sinf obyektlarini aniqlash mumkin. Masalan :

```
class Person:
    pass

tomson = Person()      # tomson obyektini aninqlash
bobbi = Person()       # bobbi obyektini aninqlash
```


Person sinfi aniqlangandan so'ng, Person sinfining ikkita - tomson va bobbi obyektlari yaratildi. Klass obyektini yaratish uchun maxsus funksiyadan - konstruktordan foydalaniladi. U class nomi bilan chaqiriladi va class obyektini qaytaradi. Ya'ni, bu holda, Person() chaqiruvi konstruktor chaqiruvini ifodalaydi. Har bir yaratilgan classda parametrlarsiz standart konstruktor mavjud bo'ladi:

```
tomson = Person() # Person() - bu Person sinfining obyektini qaytaruvchi konstruktor chaqiruvi
```

2. Sinf va obyekt. Sinf konstruktori

Class metodlari aslida class ichida belgilangan va uning xususiyatlarini belgilaydigan funksiyalarni ifodalaydi. Masalan, bitta metod bilan Person sinfini aniqlaylik:

```
class Person:          # Person klasining aniqlanishi
    def say_hello(self):
        print("Salom")

tom = Person()
tom.say_hello()        # Salom
```

Bu yerda **say_hello()** metodi aniqlandi, u shartli ravishda Salom xabarini konsolga chop etadi. Classga tegishli har qanday metodlarini aniqlashda shuni yodda tutish kerakki, ularning barchasi birinchi parametr sifatida **self** kalit so'zini qabul qilishadi. Self bu o'zi degan ma'noni anglatadi va ushbu klasga tegishli ekanligini bildiradi. Ammo metod chaqirilganda, bu marametr hisobga olinmaydi hamda qiymat berilmaydi.

obyekt nomidan foydalanib, classning metodlariga murojaat qilish mumkin. Mavjud metodlarni chaqirish uchun nuqta belgisi qo'llaniladi - obyekt nomidan keyin nuqta qo'yiladi va undan keyin metod chaqiruvi keladi:

```
Obyekt.metod([metod qabul qiladigan parametrlar])
```

Masalan, konsolga salomlashishni chop etish uchun say_hello() metodini chaqirish:

```
tomson.say_hello()    # Salom
```

Natijada, ushbu dastur konsolga "Salom" qatorini chop etadi.

Agar metod self dan tashqari boshqa parametrlarni qabul qiladigan bo'lsa, ular self parametridan keyin belgilanadi va bunday metodni chaqirishda, ushbu parametrlariga qiymatlar berish talab qilinadi:

```
class Person:          # Person klasining aniqlanishi
    def say(self, msg):  # method
        print(msg)

tom = Person()
tom.say("Hello , HOW ARE YOU?") # Hello , HOW ARE YOU?
```

Say() metodi bu yerda aniqlanadi. Bu metod ikkita parametr oladi: **self** va **msg**. Va ikkinchi parametr uchun - **msg**, metodni chaqirganda ushbu parametrqa qiymat berilishi kerak.

Self kalit so'zi orqali classning atributi va funksiyalariga murojaat qilish mumkin:

```
self.attribute    # methodning atributiga murojaat qilish
self.method       # method chaqiruvi
```

Masalan, Person sinfida ikkita metodni aniqlaymiz :

```
class Person:
    def say(self, msg):
        print(msg)

    def say_hello(self):
        self.say("Hello JOB")

tom = Person()
tom.say_hello()    # Hello work
```

Bu yerda **say_hello()** metodi ichida turib **say()** metodiga murojaat amalga oshirilmoqda

```
self.say("Hello JOB")
```

Say() metodi o'ziga qo'shimcha ravishda parametrlarni (msg parametrini) qabul qilganligi sababli, metod chaqirilganda ushbu parametr uchun qiymat berish kerak.

Bundan tashqari, metod ichida turib, obyekt metodini chaqirganda, self kalit so'zini ishlatishimiz kerak. Aks holda xatolik kelib chiqadi:

```
def say_hello(self):
    say("Hello job")    # Xatolik
```

3. **__init__()** va **__del__()** metodlari

Konstruktorlar. Konstruktor sinf obyektini yaratish uchun ishlatiladi. Shunday qilib, yuqorida, Person sinfining obyektlarini yaratganda, hech qanday parametrlarni qabul qilmaydigan va barcha sinflar bilvosita ega bo'lgan standart konstruktordan foydalanildi:

```
tom = Person()
```

Biroq, **__init__()** deb nomlangan maxsus metod yordamida sinflarda konstruktorni aniq belgilash mumkin (har bir tomonda ikkita past chiziqcha). Masalan, keling, Person sinfini unga konstruktor qo'shish orqali o'zgartiramiz:

```
class Person:
    # konstruktor
    def __init__(self):
        print("Person obyektining yaratildi")

    def say_hello(self):
        print("Hello")

tom = Person()    # Person obyekti yaratildi
```

```
tom.say_hello()    # Hello
```

Demak, bu yerda Person sinfining kodida konstruktor va say_hello () metodi aniqlangan. Birinchi parametr sifatida konstruktor ham metodlar kabi – **self** ni qabul qiladi. Odatda konstruktorlar obyekt yaratilganda bajariladigan amallarni aniqlash uchun ishlatiladi:

```
tom = Person()
```

Person sinfidagi **__init__()** konstruktori chaqiriladi, u konsolga " Person obyekti yaratildi " matnini chop etadi.

Obyekt atributlari. Atributlar obyekt holatini saqlaydi. Class ichidagi atributlarni aniqlash va o'rnatish uchun **self** so'zidan foydalanish mumkin. Masalan , quyidagi Person sinfini aniqlaymiz :

```
class Person:

    def __init__(self, nom):
        self.nom = nom    # shaxning nomi
        self.yosh = 1     # shaxsning yoshi

tomson = Person("Tomson")

# Atributga murojaat
# qiymatni qabul qilish
print(tomson.nom)      # Tom
print(tomson.yosh)     # 1

# Qiymatni o'zgartirish
tomson.yosh = 37
print(tomson.yosh)     # 37
```

Endi Person klassi konstruktori yana bitta parametr - **nom** ni qabul qiladi. Ushbu parametr orqali yaratilgan person obyektining nomi konstruktorga uzatiladi.

Konstruktor ichida ikkita atribut o'rnatilgan – **nom** va **yosh** (shartli ravishda, shaxsning ismi va yoshi):

```
def __init__(self, nom):
    self.nom = nom
    self.yosh = 1
```

self.nom atributi **nom** o'zgaruvchisiga tenglashtirib qo'yildi. **yosh** atributining boshlang'ich qiymati 1 ga teng deb qabul qilindi.

Agar sinfda **__init__** konstruktoridan foydalanilgan bo'lsa, endi standart konstruktorni chaqira olmaymiz. Endi biz aniq belgilangan **__init__** konstruktorimizni chaqirishimiz kerak, unga **nom** parametri uchun qiymat berilishi kerak:

```
tomson = Person("Tomson")
```

Keyinchalik ushbu nom orqali obyektning atributlariga murojaat qilib, uni olish va o'zgartirish mumkin:

```
print(tomson.nom) # nom atributining qiymatini olish
```

```
tomson.yosh = 37 # yosh atributining qiymatini o'zgartiring
```

Asosan, class ichida atributlarni aniqlamasdan uni obyektning o'ziga biriktirib qo'yish mumkin. Python bu atributni dinamik ravishda obyektga qo'shishga imkon beradi:

```
class Person:
    def __init__(self, nom):
        self.nom = nom      # shaxsning nomi
        self.yosh = 1      # shaxsning yoshi

tomson = Person("Tomson")
tomson.company = "Microsoft"
print(tomson.company) # Microsoft
```

Bu yerda kompaniya atributi dinamik ravishda o'rnatildi. Bu atribut person obyektining ish joyini o'zida saqlaydi. Va o'rnatgandan so'ng, uning qiymatiga murojaat qilib, qiymatini olish mumkin. Shuni esda turish kerakki, agar atributni aniqlashdan oldin unga murojlat qilishga harakat qilinsa, dastur xatolik qaytaradi:

```
tomson = Person("Tomson")
print(tomson.company) # ! Xatolik - AttributeError:
# Person object has no attribute company
```

Sinf ichidagi obyektning atributlariga murojaat qilish uchun uning metodlarida self so'zi ham qo'llaniladi:

```
class Person:
    def __init__(self, nom):
        self.nom = nom      # shaxsning nomi
        self.yosh = 1      # shaxsning yoshi

    def display_info(self):
        print(f"Nom: {self.nom}  Yoshi: {self.yosh}")

tomson = Person("Tomson")
tomson.display_info()      # Nom: Tomson  Yoshi: 1
```

Bu konsolga ma'lumotlarni chop etadigan **display_info()** metodini belgilaydi. Metoddagi obyekt atributlariga murojaat qilish uchun esa **self** so'zi ishlatiladi: **self.nom** va **self.yosh**

Obyektlarni yaratish. Yuqorida bitta obyekt yaratilgan. Ammo shunga o'xshash tarzda class ning boshqa obyektlarni yaratish mumkin:

```
class Person:
    def __init__(self, nom):
        self.nom = nom      # shaxsning nomi
        self.yosh = 1      # shaxsning yoshi
    def display_info(self):
        print(f"Nom: {self.nom}  Yoshi: {self.yosh}")

tomson = Person("Tomson")
tomson.yosh = 37
tomson.display_info()      # Nom: Tomson  yoshi: 37
```

```
bobbi = Person("Bobbi")
bobbi.yosh = 41
bobbi.display_info()      # Nomi: Bobbi  Yosh: 41
```

Ushbu dasturda Person sinfining ikkita obykti: tomson va bobbi yaratilgan. Ushbu obyektlar Person sinfga tegishli bo'lib, ular bir xil atributlar va metodlar to'plamiga egalar. Biroq ular bir-biridan xususiyatlari bilan farq qilishadi.

Dastur bajarilganda, Python self parametri dinamik ravishda aniqlanadi - bu chaqiriladigan obyektни ifodalaydi. Masalan, satrda:

```
tomson.display_info()    # Nomi: Tomson  yoshi: 37
bobbi.display_info()     # Nomi: Bobbi  Yosh: 41
```

Nazorat savollari:

1. Pythonda obyekt tushunchasiga ta'rif bering.
2. Pythonda class tushunchasiga ta'rif bering.
3. Classning methodlari qanday yaratiladi?
4. Classga murojaat qanday amalga oshiriladi?
5. Class obykti qanday hosil qilinadi?
6. Metodga qanday murojaat qilinadi?

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

15-mashg’ulot. Pythonda Vorislikdan foydalanish.

O‘quv savollari:

1. Vorislik.
2. Maxsus metodlar.
3. Klass xususiyatlari.

1. Python tilidagi “Merosxo‘rlik” tushunchasi, atributlari va xossalari

OOP da merosxo‘rlik konsepsiyasi mavjud class asosida yangi class yaratish imkonini beradi. Inkapsulyatsiya bilan bir qatorda merosxorlik obyektga yo‘naltirilgan dasturlashning asoslaridan hisoblanadi.

Merosxorlik konsepsiyasining asosiy tushunchalari: **subclass** va **superclass** lardir. **Subklass** barcha umumiy atributlar va metodlarni yuqori classdan meros qilib oladi. Shuningdek superclass, asosiy (**base class**) yoki ota (**parent class**) deb atalsa, subclass - voris class (**derived class**) yoki bola class (**child class**) deb ataladi.

Merosxorlik classini yaratish sintaksisi quyidagicha ko‘rinishda bo‘ladi:

```
class subclass (superclass):
    subclass_metodlari
```

Masalan, shaxsni ifodalovchi Person sinfi mavjud bo‘lsin:

```
class Person:
    def __init__(self, nom):
        self.__nom = nom

    @property
    def nom(self):
        return self.__nom

    def display_info(self):
        print(f"Nom: {self.__nom} ")
```

Aytaylik, bizga qandaydir korxonada ishlaydigan ishchilar classi kerak. Biz noldan yangi sinf yaratishimiz mumkin, masalan, *Employee* sinfi:

```
class Employee:
    def __init__(self, nom):
        self.__nom = nom

    @property
    def nom(self):
        return self.__nom

    def display_info(self):
        print(f"Nom: {self.__nom} ")

    def work(self):
```

```
print(f"{self.nom} works")
```

Biroq, *Employee* classi *Person* classi bilan bir xil atribut va metodlarga ega bo'lishi mumkin, chunki xodimlar ham inson classiga mansub ob'yeklar hisoblanadi. Shunday qilib, yuqoridagi ***Employee*** sinfida faqat ***work*** metodi qo'shiladi va qolgan kod *Person* sinfining funksionalligidan nusxalab olinadi. Agar merosxorlikda foydalanilmasa, bir xil funktsionallik ikkala sinfda ham takrorlanaveradi. Bunday holda merosxorlik konsepsiyasidan foydalanish ancha kodni tejalishiga olib keladi.

Shunday qilib, ***Person*** sinfidan ***Employee*** sinfini meros qilib olaylik :

```
class Person:
    def __init__(self, nom):
        self.__nom = nom

    @property
    def nom(self):
        return self.__nom

    def display_info(self):
        print(f"Nomi: {self.__nom} ")

class Employee(Person):
    def work(self):
        print(f"{self.nom} works")

tomson = Employee("Tomson")
print(tomson.nom)          # Tomson
tomson.display_info()     # Nomi: Tomson
tomson.work()             # Tomson works
```

Employee sinfi ***Person*** sinfining funksiyalarini to'liq o'ziga nusxalab oladi, faqat ***work()*** metodi *Employee* sinfiga qo'shimcha tarzda qo'shiladi. Shunga ko'ra, *Employee* sinfi obyektini yaratishda ***Person*** dan meros qilib olingan konstruktordan foydalanish mumkin :

```
tomson = Employee ("Tomson")
```

Shuningdek, meros qilib olingan atributlar/ xususiyatlar va metodlarga murojaat qilish mumkin:

```
print(tomson.nom)          # Tomson
tomson.display_info()     # Nomi: Tomson
```

Shuni yodda tutish kerakki, *Employee* sinfi uchun *Person* sinfining yopiq atributi bo'lgan ***__nom*** atributiga kirishga ruxsat yo'q. Masalan, ***work()*** metodi tarkibida ***self.__nom*** xususiy atributiga murojaat qila olmaymiz :

```
def work(self):
    print(f"{self.__nom} works")    # Xatolik!
```

Ko'p merosxorlik

Python tilining ajralib turadigan xususiyatlaridan biri - bu bir ko'p merosxorlik konsepsiyasini qo'llab-quvvatlashligidir. Ya'ni bitta sinf bir nechta sinflardan meros olishi mumkin:

```
class Employee:
    def work(self):
        print("Employee jobs")

class Students:
    def study(self):
        print("Students studying")

class WorkingStudents(Employee, Students):
    pass

tomson = WorkingStudent()
tomson.job()      # Employee jobs
tomson.study()    # Students studying
```

U firma xodimini ifodalovchi `Employee` sinfini va talabalarini ifodalovchi `Students` sinfini belgilaydi. Ishlayotgan talabani ifodalovchi `WorkingStudents` sinfi tarkibida hech qanday metod va atributlar aniqlanmaganligi `pass` operatori bilan to'ldirib qo'yilgan holos.

`WorkingStudents` sinfi ikkita sinfdan `Employee` va `Students` funktsionallikni meros qilib oladi. Shunday ekan ushbu sinf obyektida ikkala sinfning metodlarini chaqirish imkoniyati mavjud.

Shu bilan birga, meros qilib olingan sinflar funktsional jihatdan murakkabroq bo'lishi mumkin, masalan:

```
class Employee:
    def __init__(self, nom):
        self.__nom = nom

    @property
    def nom(self):
        return self.__nom

    def work(self):
        print(f"{self.nom} works")

class Students:
    def __init__(self, nom):
        self.__nom = nom

    @property
    def nom(self):
        return self.__nom

    def study(self):
        print(f"{self.nom} studying")
```



```
class WorkingStudents(Employee, Students):
    pass
```

```
tomson = WorkingStudents("Tomson")
tomson.work()
tomson.study()
```

```
C:\Users\PC10\AppData\Loc
Tomson works
Tomson studying
```

2. Maxsus metodlar

Pythonda obyektlar bilan ishlashni yanada qulay qilish uchun bir nechta maxsus metodlar bor. Bu metodlarning nomi ikki pastki chiziq bilan yozilgani uchun, double underscore yoki qisqa qilib dunder metodlar deb ataladi. Dunder metodlar yordamida obyektlarga qo'shimcha qulayliklar va vazifalar qo'shishimiz mumkin. Klass yoki obyektga oid dunder metodlar ro'yxatini ko'rish uchun `dir()` funksiyasidan foydalanamiz:

```
>>> dir(Avto)
['_Avto__num_avto', '__class__', '__delattr__',
 '__dict__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__', '__getattribute__', '__gt__',
 '__hash__', '__init__', '__init_subclass__',
 '__le__', '__lt__', '__module__', '__ne__',
 '__new__', '__reduce__', '__reduce_ex__', '__repr__',
 '__setattr__', '__sizeof__', '__str__', '__subclasshook__',
 '__weakref__', 'make', 'model', 'narh', 'rang', 'yil']
```

Dunder metodlardan biz `__init__` metodi bilan tanishdik. Bu metod klassdan obyekt yaratishda chaqiriladi va obyektning xususiyatlarini belgilaydi. Ushbu darsimizda esa maxsus metodlarning ba'zilarini bilan tanishamiz.

OBJEKT HAQIDA MA'LUMOT

Obyektga `print()` yoki `str()` orqali murojat qilinganda obyekt haqida tushunarli ma'lumot qaytarish uchun `__repr__` va `__str__` metodlaridan foydalanamiz. Tushunarli bo'lishi uchun avvalgi darsimizdagi `Avto` klassiga qaytamiz:

```
class Avto:
    __num_avto = 0
    """Avtomobil klassi"""
    def __init__(self, make, model, rang, yil, narh):
        """Avtomobilning xususiyatlari"""
        self.make = make
        self.model = model
        self.rang = rang
        self.yil = yil
        self.narh = narh
    Avto.__num_avto += 1
```

Yuqoridagi klassdan yangi obyekt yaratamiz va obyekt haqida ma'lumot olish uchun `print()` funksiyasini chaqiramiz:

```
avto1 = Avto("GM", "Malibu", "Qora", 2020, 40000)
print(avto1) # obyekt haqida ma'lumot
```

Natija: <__main__.Avto object at 0x00000238A6DAE0C8>

Qandaydur tushunarsiz ma'lumot. Ekrandagi natijadan biz faqat `avto1` obyektimiz `Avto` klassiga tegishli ekanini ko'ramiz. Qanday qilib shuning o'rniga obyekt haqida tushunarliroq ma'lumot olishimiz mumkin?

Gap shundaki biz har gal obyektga `print()` (yoki `str()` yoki `repr()`) orqali murojat qilganimizda, Python obyekt ichida `__str__` yoki `__repr__` metodlariga murojat qiladi. Agar biz bu metodlarni yozmagan bo'lsak, yuqoridagi kabi umumiy ma'lumot qayataradi.

Biz ushbu metodlarni yangidan yozib, biz istagan ma'lumotni qayataradigan qilishimiz mumkin. Odatda, yuqoridagi ikki metoddan birini yozish kifoya. Odatda, `__repr__` umumiyorq, `__str__` esa batafsilroq ma'lumot olish uchun ishlatiladi.

Ikkalasidan birini tanlaganda, `__repr__` metodiga yon bosiladi, sababi bu metod `print()`, `str()` va `repr()` funksiyalarining hammasi bilan ishlaydi. Keling biz ham yuqirdagi klassimizga `__repr__` metodini qo'shamiz:

```
class Avto:
    __num_avto = 0
    """Avtomobil klassi"""
    def __init__(self, make, model, rang, yil, narh):
        """Avtomobilning xususiyatlari"""
        self.make = make
        self.model = model
        self.rang = rang
        self.yil = yil
        self.narh = narh
        Avto.__num_avto += 1

    def __repr__(self):
        """Obyekt haqida ma'lumot"""
        return f"Avto: {self.rang} {self.make} {self.model}"
```

Qaytadan `print()` funksiyasini chaqiramiz:

```
avto1 = Avto("GM", "Malibu", "Qora", 2020, 40000)
print(avto1)
```

Natija: Avto: Qora GM Malibu

Mana endi natijamiz ancha tushunarli ko'rinishda chiqdi.

SUPER KLASS VA VORIS KLASS

Obyektga yo'naltirilgan dasturlashning qulayliklaridan biri bu klasslardan boshqa klass yaratish imkoniyati. Bizga kerak bo'lgan yangi klass, avval yaratilgan boshqa klass bilan o'xshashlik joylari bo'lsa, biz bu klassdan voris klass yaratishimiz mumkin. Bunda asl klass - ota, yoki super klass deb ataladi.

Super klassdan yaratilgan voris klass otaning barcha yoki tanlangan xususiyatlari va metodlarini meros olish bilan birga, o'ziga xos xususiyat va metodlariga ega bo'ladi.

Keling boshlanishiga Shaxs klassini yaratamiz, bu klassimiz keyinchalik boshqa klasslar uchun super klass vazifasini bajaradi:

```
class Shaxs:
    """Shaxslar haqida ma'lumot"""
    def __init__(self, ism, familiya, passport, tyil):
        self.ism = ism
        self.familiya = familiya
        self.passport = passport
        self.tyil = tyil

    def get_info(self):
        """Shaxs haqida ma'lumot"""
        info = f"{self.ism} {self.familiya}. "
        info += f"Passport:{self.passport}, {self.tyil}-yilda tug`ilgan"
        return info

    def get_age(self, yil):
        """Shaxsning yoshini qaytaruvchi metod"""
        return yil - self.tyil
```

Klassimizni tekshirib ko'ramiz:

```
inson = Shaxs("Hasan", "Alimov", "FB001122", 1995)
print(f"{inson.get_info()}. {inson.get_age(2021)} yoshda.")
```

Natija: Hasan Alimov. Passport:FB001122, 1995-yilda tug`ilgan. 26 yoshda.

VORIS KLASS YARATISH

Endi avvalgi darsimizda yaratgan Talaba klassimizni qaytadan yaratamiz. Bu safar, avvalgidan farqli ravishda, Talaba ni yaratishda, Shaxs dan super klass sifatida foydalanamiz:

```
class Talaba(Shaxs):
    """Talaba klassi"""
    def __init__(self, ism, familiya, passport, tyil):
        """Talabaning xususiyatlari"""
        super().__init__(ism, familiya, passport, tyil)
```

Kodimizni tahlil qilaylik:

1-qatorda klass nomidan so'ng, qavs ichida super klass nomini berdik

5-qatorda (def __init__ ichida) klassimiz super klassning xususiyatlarini meros olishini ko'rsatdik

Yangi yaratgan Talaba klassimiz Shaxsning barcha xususiyatlari va metodlariga ega bo'ladi.

```
talaba = Talaba("Valijon", "Aliyev", "FA112299", 2000)
print(talaba.get_info())
```

Natija: Valijon Aliyev. Passport:FA112299, 2000-yilda tug`ilgan

Talaba klassi uchun alohida `get_info()` metodini yozmagan bo'lsakda, bu metod Talabaga Shaxsdan meros o'tdi.

Xuddi shu kabi `get_age()` metodiga ham murojat qilishimiz mumkin:

```
>>>print(talaba.get_age(2021))
```

Dastur davomida super klass voris klasslardan avval yozilgan (chaqirilgan) bo'lishi kerak.

VORIS KLASSGA XOS XUSUSIYATLAR VA METODLAR

Hozirgi ko'rinishda Talaba va Shaxs klasslari o'rtasida hech qanday farq yo'q. Keling Talaba klassimizga o'ziga xos xususiyatlar va metodlar yarataylik. Avvalosiga, talabaning bosqichi va ID raqamini xususiyat sifatida qo'shamiz. Bunda ID raqami obyekt yaratilishida parameter sifatida uzatiladi, bosqich esa standart qiymatga ega.

```
class Talaba(Shaxs):
    """Talaba klassi"""
    def __init__(self, ism, familiya, passport, tyil, idraqam):
        """Talabaning xususiyatlari"""
        super().__init__(ism, familiya, passport, tyil)
        self.idraqam = idraqam
        self.bosqich = 1
```

Endi yangi, Talaba obyektini yaratishda qo'shimcha idraqam parametrini ham kiritish talab qilinadi:

```
talaba = Talaba("Valijon", "Aliyev", "FA112299", 2000, "0000012")
```

So'ngra, bu qiymatlarni qaytaruvchi alohida metodlar yozamiz:

```
class Talaba(Shaxs):
    """Talaba klassi"""
    def __init__(self, ism, familiya, passport, tyil, idraqam):
        """Talabaning xususiyatlari"""
        super().__init__(ism, familiya, passport, tyil)
        self.idraqam = idraqam
        self.bosqich = 1

    def get_id(self):
        """Talabaning ID raqami"""
        return self.idraqam

    def get_bosqich(self):
        """Talabaning o'qish bosqichi"""
        return self.bosqich
```

Metodlarni tekshirib ko'ramiz:

```
>>>print(f"{talaba.get_info()}. ID raqami:{talaba.get_id()}")
```

Valijon Aliyev. Passport:FA112299, 2000-yilda tug'ilgan. ID raqami:0000012

```
>>>print(f"{talaba.get_bosqich()}-bosqich talabasi")
```

1-bosqich talabasi

Shu zayilda yangi klassimizga istalgancha yangi xususiyatlar va metodlar qo'shishimiz mumkin. Bunda, agar yangi xususiyat yoki metod super klassga ham aloqador bo'lsa uni birdan super klassga qo'shish tavsiya qilinadi.

Voris klass boshqa klass uchun super klass bo'lishi mumkin.

POLIMORFIZM — SUPER KLASS METODLARINI QAYTA YOZISH

Voris klassga super klassdan meros qolgan istalgan metodni qayta talqin qilish mumkin. Avvalgi misolimizdagi `get_info()` super metodini ko'raylik, bu metod talabaning ismi, familiyasi, passport raqami va tug'ilgan yilini qaytaradi:

```
>>> print(talaba.get_info())
```

Valijon Aliyev. Passport:FA112299, 2000-yilda tug'ilgan

Endigeta `get_info()` metodi talabaga mos ma'lumotlar qaytarishi uchun, Talaba klassi ichida huddi shu nomli metodni qayta yozamiz:

```
class Talaba(Shaxs):
    """Talaba klassi"""
    def __init__(self, ism, familiya, passport, tyil, idraqam):
        """Talabaning xususiyatlari"""
        super().__init__(ism, familiya, passport, tyil)
        self.idraqam = idraqam
        self.bosqich = 1

    def get_id(self):
        """Talabaning ID raqami"""
        return self.idraqam

    def get_bosqich(self):
        """Talabaning o'qish bosqichi"""
        return self.bosqich

    def get_info(self):
        """Talaba haqida ma'lumot"""
        info = f"{self.ism} {self.familiya}. "
        info += f"{self.get_bosqich()}-bosqich. ID raqami: {self.idraqam}"
        return info
```

Metodni tekshirib ko'ramiz:

```
>>> print(talaba.get_info())
```

Valijon Aliyev. 1-bosqich. ID raqami: 0000012

OBJEKT ICHIDA OBJEKT

Ba'zida klassimiz xususiyatlar va ular bilan ishlaydigan metodlarga to'lib ketishi, bu esa o'z navbatida obyektga murojat qilishni qiyinlashtirishi mumkin. Shunday holatlarda ba'zi xususiyatlarni alohida klass ko'rinishida yozish va keyinchalik bu klassdan yaratilgan obyektning boshqa obyektning xususiyati sifatida foydalanish mumkin.

Misol uchun, yuqoridagi Shaxs klassimizga yana bir manzil degan xususiyat qo'shaylik. Odatda manzil bir nechta qismlardan iborat bo'ladi (xonadon, ko'cha,

mahalla, tuman/shahar, viloyat, indeks va hokazo) va ularning har biri uchun Shaxs ichida alohida xususiyat yaratmasdan, alohida manzil degan klassga yuklash maqsadga muvofiq bo'ladi.

```
class Manzil:
    """Manzil saqlash uchun klass"""
    def __init__(self, uy, kocha, tuman, viloyat):
        """Manzil xususiyatlari"""
        self.uy = uy
        self.kocha = kocha
        self.tuman = tuman
        self.viloyat = viloyat

    def get_manzil(self):
        """Manzilni ko'rish"""
        manzil = f"{self.viloyat} viloyati, {self.tuman} tumani, "
        manzil += f"{self.kocha} ko'chasi, {self.uy}-uy"
        return manzil
```

Talaba klassimizga ham qo'shimcha manzil xususiyatini qo'shamiz:

```
class Talaba(Shaxs):
    """Talaba klassi"""
    def __init__(self, ism, familiya, passport, tyil, idraqam, manzil):
        """Talabaning xususiyatlari"""
        super().__init__(ism, familiya, passport, tyil)
        self.idraqam = idraqam
        self.bosqich = 1
        self.manzil = manzil

    def get_id(self):
        """Talabaning ID raqami"""
        return self.idraqam

    def get_bosqich(self):
        """Talabaning o'qish bosqichi"""
        return self.bosqich

    def get_info(self):
        """Talaba haqida ma'lumot"""
        info = f"{self.ism} {self.familiya}. "
        info += f"{self.get_bosqich()}-bosqich. ID raqami: {self.idraqam}"
        return info
```

Keling endi talaba obyektini qayta yaratamiz. Bu safar talabaning manzili ham alohida obyekt sifatida talaba ga uzatiladi:

```
talaba_manzil = Manzil(12, 'Olmazor', "Bog'bon", "Samarqand")
talaba = Talaba("Valijon", "Aliyev", "FA112299", 2000, "0000012", talaba_manzil)
```

Obyekt ichidagi obyektning xususiyatlari va metodlariga ham avvalgidek nuqta orqali murojat qilishimiz mumkin:

```
>>> print(talaba.manzil.get_manzil())
```

Samarqand viloyati, Bog'bon tumani, Olmazor ko'chasi, 12-uy

```
>>> print(talaba.manzil.tuman)
```

Amaliyot uchun masalalar

1. Talaba klassiga yana bir, fanlar degan xususiyat qo'shing. Bu xususiyat parametr sifatida uzatilmasin va obyekt yaratilganida bo'sh ro'yxatdan iborat bo'lsin (self.fanlar=[])
2. Fan degan yangi klass yarating va bu klassdan turli fanlar uchun alohida obyektlar yarating.
3. Talaba klassiga fanga_yozil() degan yangi metod yozing. Bu metod parametr sifatida Fan klassiga tegishli obyektlarni qabul qilib olsin va talabaning fanlar ro'yxatiga qo'shib qo'ysin.
4. Talabaning fanlari ro'yxatidan biror fanni o'chirib tashlash uchun remove_fan() metodini yozing. Agar bu metodga ro'yxatda yo'q fan uzatilsa "Siz bu fanga yozilmagansiz" xabarini qaytarsin.
5. Yuqoridagi Shaxs klassidan boshqa turli voris klasslar yaratib ko'ring (masalan Professor, Foydalanuvchi, Sotuvchi, Mijoz va hokazo)
6. Har bir klassga o'ziga hoz xususiyatlar va metodlar yuklang.
7. Barcha yangi klasslar uchun get_info() metodini qayta yozib chiqing.
8. Voris klasslardan yana boshqa voris klass yaratib ko'ring. Misol uchun Foydalanuvchi klassidan Admin klassini yarating.
9. Admin klassiga foydalanuvchida yo'q yangi metodlar yozing, masalan, ban_user() metodi konsolga "Foydalanuvchi bloklandi" degan matn chiqarsin.

Nazorat savollari:

1. Merosxorlik tushunchasiga tarif bering;
2. Pythonda merosxorlik qanday amalga oshiriladi?
3. Inkopsulyatsiya nima maqsadda ishlatiladi?
4. Merosxor classning qanday afzalliklari mavjud?
5. Subclass va Superclass tushunchasiga tarif bering.

MA'RUZA MASHG'ULOT UCHUN O'QUV MATERIALLARI

2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

1-mashg'ulot. PyQt5 paketi va uning imkoniyatlari bilan tanishish.

O'quv savollari:

1. PyQt5 paketining imkoniyatlari. PyQt5 paketini o'rnatish.
2. PyQt5 paketini pip yordamida o'rnatish.
3. QtDesigner dasturini o'rnatish hamda uning imkoniyatlari bilan tanishish.

1. PyQt5 paketining imkoniyatlari. PyQt5 paketini o'rnatish

Python, boshqa dasturlash tillari kabi, grafikalar bilan ishlash qobiliyatiga ega. Quyida Pythonning eng yaxshi 5 ta grafik kutubxonasi haqida foydali ma'lumotlar keltirilgan:

1. PyQt5
2. Python Tkinter
3. PySide 2
4. Kivi
5. wxPython

Dasturlash tillaridan foydalanib turli dasturlar tuzish davomida GUI dasturlar tuzish degan tushunchaga duch kelinadi. Xo'sh GUI o'zi nima?

GUI (Graphics User Interface)- bu shakllar, hujjatlar, testlar va boshqalar kabi turli vaziyatlarda foydalanuvchilardan javob olish uchun dasturchi tomonidan ishlab chiqilgan interaktiv muhit. Bu foydalanuvchiga an'anaviy Buyruqlar qatori interfeysi (CLI) ga qaraganda chiroyli interaktiv ekranni taqdim etadi.

PyQt5 - bu Python uchun grafik foydalanuvchi interfeysi (GUI). Bu dasturchilar orasida juda mashhur va GUI kodlash yoki QT dizayner dasturi tomonidan ishlab chiqilishi mumkin. QT freymworki foydalanuvchi interfeyslarini yaratish uchun vidjetlarni bevosita sichqoncha yordamida tanlab kerakli joyga joylashtirish imkonini beruvchi vizual tizimdir.

Bu bepul va ochiq kodli bog'lovchi dasturiy ta'minot bo'lib, crossplatformali ilovalarni ishlab chiqish uchun xizmat qiladi. U Windows, Mac, Android, Linux, Arduino va Raspberry PI da qo'llanilishi mumkin.

PyQt5 ni o'rnatish uchun quyidagi buyruqdan foydalaniladi:

```
pip install pyqt5
```

Bu yerda oddiy koddan foydalanib, PyQt5 kutubxonasidan qanday foydalanishni ko'rsatib o'tilgan:

```
from PyQt5.QtWidgets import QApplication, QMainWindow
import sys
class Window(QMainWindow):
    def __init__(self):
```

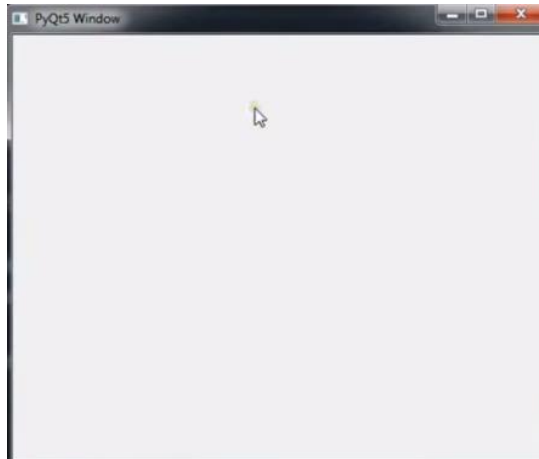


```

super().__init__()
self.setGeometry(300, 300, 600, 400)
self.setWindowTitle("PyQt5 window")
self.show()
app = QApplication(sys.argv)
window = Window()
sys.exit(app.exec_())

```

Dastur natijasi:



PyQt 5 kutubxonasi interfeysi ko'rinishi

PyQt – bu Sun'iy intellekt asoslari va Qt platformasini bir-biri bilan hamjihatlikda ishlashiga imkon beruvchi mukammal bir kutubxona. U murakkab grafik interfeyslar uchun maxsus vidjetlar yaratish uchun o'rnatilgan vidjetlar va asboblarning boy tanlovini, shuningdek, ma'lumotlar bazalariga ulanish va ular bilan o'zaro ishlash uchun mustahkam SQL ma'lumotlar bazasini qo'llab-quvvatlaydi.

2. PyQt5 paketini pip yordamida o'rnatish

PyQt - Sun'iy intellekt asoslari uchun Qt grafik freymvorki uchun Python kengaytmasi sifatida amalga oshirilgan kengaytmalar (bog'lashlar) to'plami.

PyQt Britaniyaning Riverbank Computing kompaniyasi tomonidan ishlab chiqilgan python kutubxonasi. PyQt Qt tomonidan qo'llab-quvvatlanadigan barcha platformalarda ishlaydi: Linux, UNIX, macOS va Windows va shu kabi operatsion tizimlar. Hozirgi kungacha PyQt ning 3 ta versiyasi ishlab chiqilgan: PyQt4, PyQt5 va PyQt6. PyQt kutubxonasi GPL (pythonning 2 va 3 versiyalari) va tijorat litsenziyalari ostida tarqatiladi.

PyQt deyarli to'liq Qt imkoniyatlarini qamrab olgan. Bu esa 600 dan ortiq sinflar, 6000 dan ortiq funksiyalar va metodlar degani.

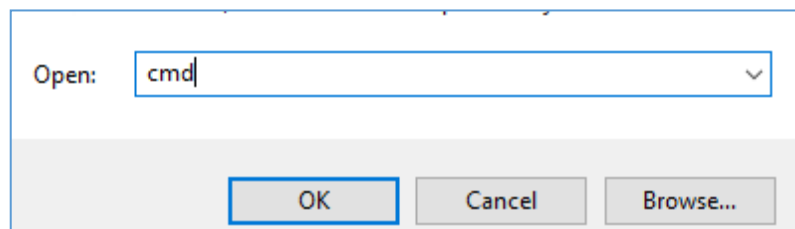
PyQt shuningdek, QtDesigner nomli GUI dizaynerini ham o'z ichiga oladi. QtDesigner dasturida yaratilgan *.ui formatli dizayn faylini Python faylga o'girib, qayta ishlash yoki to'g'ridan-to'g'ri *.ui fayli bilan *.py fayllari o'rtasida bog'lanish hosil qilish uchun **pyuic** kutubxonasidan foydalaniladi. QtDesigner dasturi PyQtni tez prototiplash uchun juda foydali vositaga aylantiradi. Bundan tashqari, Qt

Designer dasturiga Pythonda yozilgan yangi grafik boshqaruv elementlarini qo'shish mumkin.

PyQt5 paketini o'rnatish. Pythonning PyQt5 paketini Windows operatsion tizimiga o'rnatish uchun quyidagi amallarni bajarish kerak:

1-qadam: Python operatsion tizimga o'rnatilganligiga ishonch hosil qilish kerak. Buning uchun klaviaturadan **WIN+R** klaviaturalarini bir vaqtda bosiladi va ekranda **Выполнить** oynasi paydo bo'ladi. Bu oynaga **CMD** buyrug'i yoziladi va ok tugmachasini bosiladi:

Выполнить oynasi ko'rinishi



Shundan so'ng consol oynasi ochiladi va u yerga operatsion tizimda python o'rnatilganligini bilish maqsadida oddiy pythonning versiyasini bilib beruvchi buyruqni kiritamiz: **python -V**

Shundab so'ng, agar tizimda python allaqachon o'rnatilgan bo'lsa, buyruqning natijasi pythonning versiyasi bo'ladi :

```
C:\Users\PC10>python -V
Python 3.11.2
```

Python versiyasini aniqlash

Aks holda, xatolik yuzaga keladi va python o'rnatilishi kerak bo'ladi.

2-qadam. Kompyuterga PyQt5 o'rnatilmaganligiga ishonch hosil qilish.

Bunda foydalanuvchi PyQt avvaldan o'rnatilmaganligiga ishonch hosil qilish uchun quyidagi buyruqni kiritishi kerak.

Consolni ochiladi va u yerga pythonning o'rnatilgan barcha paketlari ro'yxatini chiqaruvchi buyruqni yoziladi: **pip list**.

Shunday qilib PyQt5 tizimga o'rnatilmagan yoki o'rnatilganligini bilib olinadi.

3-qadam. PyQt-ni o'rnatish. Demak operatsion tizimga python dasturi o'rnatilganligini va PyQt5 paketi o'rnatilmaganligiga ishonch hosil qilindi.

Endi PyQt5 paketini tizimga o'rnatish uchun foydalanuvchi quyida keltirilgan buyruqni konsolga kiritishi kerak: **pip install PyQt5**

```
PS C:\Users\ksahn> pip list
Package                                Version
-----
charset-normalizer                     2.0.9
cycler                                 0.11.0
fonttools                             4.28.3
idna                                   3.3
imutils                               0.5.4
kiwisolver                            1.3.2
matplotlib                             3.5.1
mysql-connector-python                 8.0.27
numpy                                  1.21.4
opencv-contrib-python                 4.5.4.6
packaging                             21.3
Pillow                                8.4.0
```

O'rnatilgan paketlar ro'yxatini aniqlash.

Shundan so'ng hech qanday xatoliklar sodir bo'lmasa, PyQt5 tizimga muvaffaqiyatli o'rnatiladi va 2-qadamni takrorlash orqali foydalanuvchi PyQt o'rnatilganligini tekshirishi mumkin.

```
PS C:\Users\ksahn> pip install PyQt5
Collecting PyQt5
  Downloading https://files.pythonhosted.org/packages/3b/d3/76/93500af5d323160/PyQt5-5.13.0-5.13.0-cp35.cp36.cp37.cp38-none-win32.whl (49.7MB)
    100% |#####| 49.7MB 97kB/s
Requirement already satisfied: PyQt5_sip<13,>=4.19.14 in c:\python38\lib\site-packages (from PyQt5) (4.19.14)
Installing collected packages: PyQt5
Successfully installed PyQt5-5.13.0
You are using pip version 19.0.3, however version 19.1.1 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.
```

PyQt 5 paketini o'rnatish

Windows muhitida Python uchun PyQtni muvaffaqiyatli o'rnatildi.

3. QtDesigner dasturini o'rnatish hamda uning imkoniyatlari bilan tanishish.

QtDesigner dasturi **pyqt5-tools** to'plami bilan birga o'rnatiladi. Va uni o'rnatish juda oddiy. Kompyuterning konsol oynasiga quyidagi buyruqni yoziladi va ishga tushiriladi:

```
pip install pyqt5-tools
```

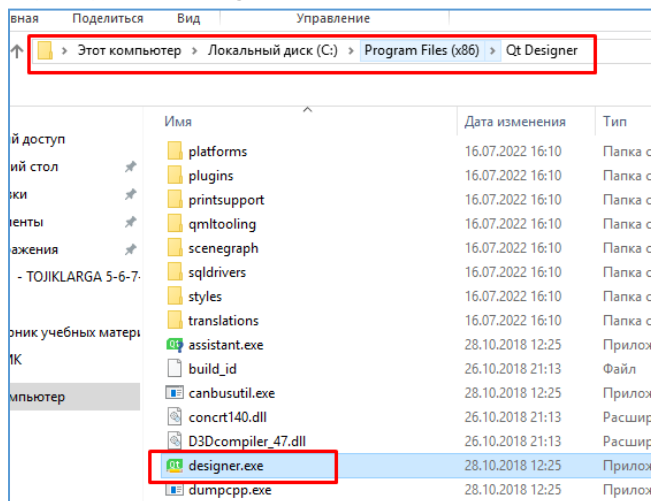
Agar yuqoridagi buyruq ishlamasa, unda quyidagi- pip buyrug'i pip3 buyrug'i bilan almashtirilgan buyruqni sinab ko'rish tavsiya etiladi:

```
pip3 install pyqt5-tools
```

QtDesignerni ishga tushiring.

Agar hammasi normada o'rnatilgan bo'lsa, unda bemaol dasturni ishga tushirib undan foydalanish mumkin bo'ladi. Buning uchun oldin QtDesigner dasturi o'rnatilgan joyni topish kerak. U odatda python o'rnatilgan kotalogda bo'ladi:

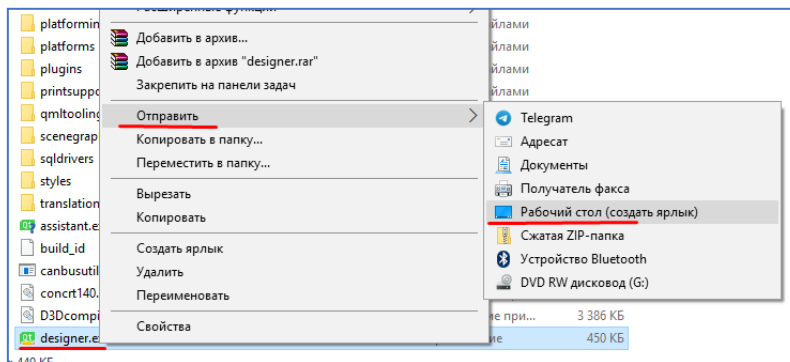
C:\Program Files (x86)\Qt Designer



Qt Designer dasturini ishga tushirish .

Odatda yorliq yaratish va uni ish stoliga joylashtirish tavsiya etiladi (38-rasm):

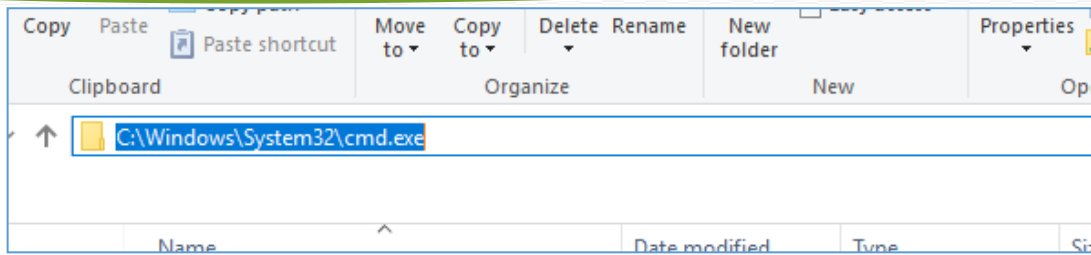
Endi dizayn dasturini ishga tushirish va python tilidagi dasturning grafik interaktiv qismini yaratishni boshlash mumkin.



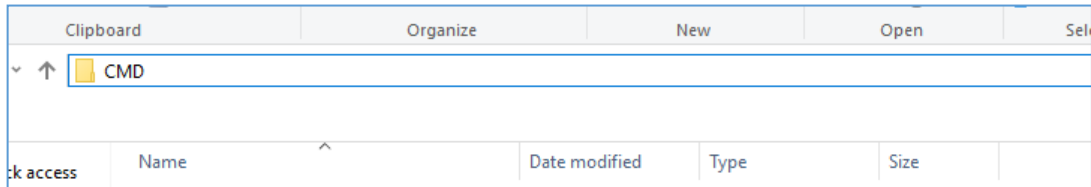
Qt Designer yorliq dasturini yaratish

QtDesigner dasturidan yaratilgan, ilova ko'rinishini o'zida saqlovchi faylni ichida python kodlarini joylashtirish uchun ushbu faylni python faylga o'girib olish talab qilinadi. Buning uchun quyidagi amallarni bajarish kerak bo'ladi:

1. QtDesigner dasturida yaratilgan **.ui** formatli faylni topiladi;
2. Ushbu fayl saqlangan kotalogdan turib konsolni ochiladi. Buning uchun kotlaogning **URL** manzili yoziladigan joyga **CMD** deb yoziladi va **enter** tugmasini bosish orqali konsolni kerakli kotologda ochish amali bajariladi (39-, 40-rasmlar):



Papkaning URL manzili yoziladigan joyi



CMD buyrug'i yozilishi kerak bo'lgan joy.

3. Ochilgan konsol oynasiga quyidagi buyruqni yoziladi. Faqat `ui_file_name` yozuvi o'rniga QtDesigner dasturining fayli nomi yozilsa, `python_file_name` o'rniga hosil bo'ladigan python fayl nomi yozilishi kerak bo'ladi:

```
pyuic5 -x "ui_file_name".ui -o "python_file_name".py
```

Shundan so'ng ushbu katalogda ko'rsatilgan nomdagi .py formatdagi fayl ham paydo bo'lganini ko'rishingiz mumkin. Va ushbu python faylning tarkibiga turli python kodlarini qo'shish, vidjetlarining qiymatlarini olish va ularni ishga tushirish uchun turli funksiyalar class, metodlar qo'shish imkoniyati mavjud.

Nazorat savollari :

1. Python PyQt 5 paketining xususiyatlarini tavsiflab bering .
2. Windows muhiti uchun PyQt5 paketi qanday o'rnatiladi?
3. QtDesigner dasturini o'rnatish qanday amalga oshiriladi?
4. QtDesigner dasturining .ui faylini .py formatli faylga o'girish uchun qanday amallarni bajarish kerak bo'ladi?
5. Nima uchun .ui formatli faylni python faylga og'irish kerak?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

2-mashg'ulot. PyQt5 kutubxonasi. QLabel vidjeti.

O'quv savollari:

1. QLabel vidjeti;
2. QLabel shrift, o'lcham va text xususiyatlari;

1. QLabel vidjeti

QLabel matn **satrlarini** ko'rsatish qobiliyatiga ega PyQt-dagi eng oddiy vidjetlardan biridir. Ushbu vidjet xohlagan vaqtda ushbu matnni olish va yangilash imkonini beruvchi ko'plab yordamchi funksiyalar va usullar bilan birga keladi.

QLabel bilan bog'liq barcha usullarning to'liq ro'yxatini ushbu sahifaning pastki qismida topish mumkin.

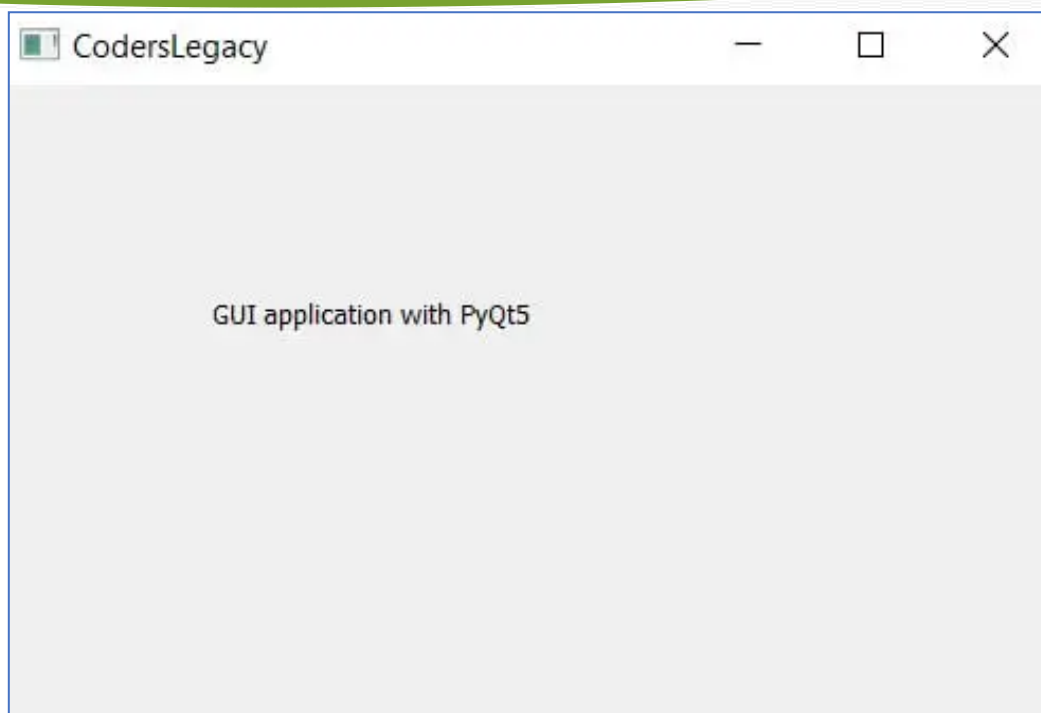
QLabel yaratish. Quyidagi misolda PyQt5 paketi yordamida oddiy QLabel yaratilishini ko'rsatib o'tilgan:

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import QApplication, QMainWindow
import sys

app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,500,300)
win.setWindowTitle("CodersLegacy")
label = QtWidgets.QLabel(win)
label.setText("GUI application with PyQt5")
label.adjustSize()
label.move(100,100)

win.show()
sys.exit(app.exec_())
```

Ushbu kodning chiqishi quyida ko'rsatilgan:



42-rasm. PyQt5 QLabel

QLabel bilan bog'liq kodni satr bo'yicha muhokama qilib chiqaylik.

```
label = QtWidgets.QLabel(win)
```

QtWidgets.QLabel() ga murojaat qilish GUI uchun foydalanish mumkin bo'lgan yorliq obyektini yaratadi. Biz yaratgan (win) oyna obyektini uning parametrlari sifatida berilishi yorliq ushbu oynaning bir qismi ekanligini anglatadi.

```
label.setText("GUI application with PyQt5")
```

Yuqoridagi oddiy kod labelga matn o'rnatadi.

```
label.adjustSize()
```

adjustSize() QLabel o'z o'lchamini avtomatik ravishda sozlashiga imkon beradi.

```
label.move(100,100)
```

Ushbu **move (100, 100)** labening dastur ekranining yuqori chap burchagidagi (100, 100) koordinatalarida paydo bo'lishini ta'minlaydi. Yodda tutingki, **ekranning** yuqori chap burchagida (0, 0) pozitsiya mavjud.

2. QLabel shrift, o'lcham va text xususiyatlari

Ushbu bo'limda hozirda QLabel da ko'rsatilgan matnni qanday yangilashni, shuningdek QLabelda ko'rsatilayotgan joriy matnni qanday qilib olish mumkinligini ko'rib chiqamiz. Bunday ishlarni qilish uchun QPushButton vidjetidan foydalanish kerak, u kerakli buyruqlarni o'z ichiga olgan funksiyalarni chaqiradi.

QLabelni yangilash uchun avvalgi **setText()** usulidan foydalaniladi va matn qiymatini olish uchun ushbu **text()** usulidan foydalaniladi:

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import QApplication, QMainWindow
import sys
```

```

def update():
    matn.setText("Yangilandi!")

def retrieve():
    print(matn.text())

app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,500,300)
win.setWindowTitle("QLabel widgetining ishlashi")

matn = QtWidgets.QLabel(win)
matn.setText("PyQt5 paketida GUI dasturlari")
matn.adjustSize()
matn.move(100,100)

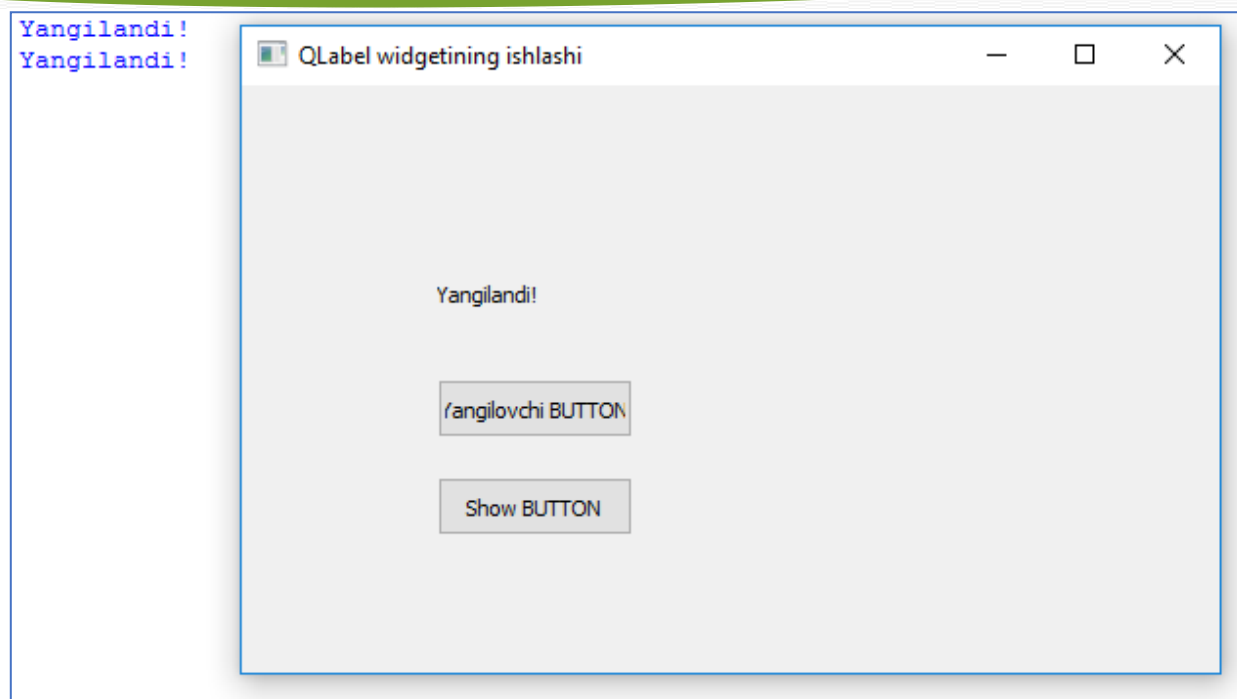
upd_btn = QtWidgets.QPushButton(win)
upd_btn.clicked.connect(update)
upd_btn.setText("Yangilovchi BUTTON")
upd_btn.move(100,150)

show_btn = QtWidgets.QPushButton(win)
show_btn.clicked.connect(retrieve)
show_btn.setText("Show BUTTON")
show_btn.move(100,200)

win.show()
sys.exit(app.exec_())

```

Yuqoridagi kodni ishga tushirgandan so‘ng ikkita QPushButton va QLabel vidgetlari orqali ishlovchi ilova paydo bo‘ladi. Unda QLabelga **“PyQt5 paketida GUI dasturlari”** yozuvi, buttonlarning biriga **“Yangilovchi BUTTON”**, ikkinchisiga esa **“Show BUTTON”** yozuvlari yozilgan bo‘ladi. Ilovaning ishlashi quyidagicha: **“Yangilovchi BUTTON”** tugmachasi bosilganda labelning matnini **“Yangilandi!”** yozuviga o‘zgartiruvchi update funksiyasini ishga tushiradi. **Show BUTTON** tugmachasi bosish orqali esa **“retrieve”** funksiyasini ishga tushirish orqali QLabel ning text xususiyatidagi yozuvni konsolga chiqariladi.



43-rasm. QLabel uchun yozilgan dasturning natijasi.

Label vidjeti uchun mavjud bo'lgan eng muhim metodlar:

Metod nomi	Tavsif
.setAlignment()	Label ichidagi matnni tekislash. U 4 xil qiymatlardan birini olishi mumkin: <i>Qt.AlignLeft</i> , <i>Qt.AlignRight</i> , <i>Qt.AlignCenter</i> va <i>Qt.AlignJustify</i> .
.setIndent()	Label matni uchun "tab"larni belgilaydi.
.setPixmap()	Labelga qo'yilgan rasmni ko'rsatish uchun ishlatiladi.
.text()	Label uchun (ko'rsatilgan) matn qiymatini qaytaradi.
.setText()	Labelda ko'rsatiladigan matnni o'rnatadi.
.selectedText()	Foydalanuvchi tomonidan tanlangan matnni qaytaradi. (Buning ishlashi uchun <i>textInteractionFlag</i> <i>TextSelectableByMouse</i> xususiyatlariga o'rnatilishi kerak).
.setWordWrap()	Label matnini avtomatik ravishda keying qatorga tushadigan xususiyatini yoqish.
.adjustSize()	Label o'lchamini ko'rsatilayotgan matnga avtomatik moslashtiradi.

Nazorat savollari:

1. QLabel vidjetidan qanday maqsadda foydalaniladi?
2. QLabel vidjetining `adjustSize()` metodining vazifasi qanday?
3. QLabel vidjetining `move()` metodining vazifasi qanday?
4. Label ichidagi matnni tekislash uchun qaysi metod ishlatiladi?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

3-mashg'ulot. PyQt5 kutubxonasi. QLineEdit vidjeti.

O'quv savollari:

1. QLineEdit vidjeti;
2. setStyleSheet() metodi.

1. QLineEdit vidjeti

QLineEdit - bu matnning bir qatorini kiritish va tahrirlash imkonini beruvchi vidjet. Ushbu vidjetda Cencel va Repeat, Cut va Past hamda “Sudrab olib tashlash” funksiyalari mavjud.

```
import sys
from PyQt5.QtWidgets import (QWidget, QLabel,
                             QLineEdit, QApplication)

class Examples(QWidget):
    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):
        self.lbl = QLabel(self)
        qlineE = QLineEdit(self)

        qlineE.move(100, 100)
        self.lbl.move(100, 40)

        qlineE.textChanged[str].connect(self.changed)
        self.setGeometry(300, 300, 280, 170)
        self.setWindowTitle('QlineEdit Example')
        self.show()

    def changed(self, text):
        self.lbl.setText(Label text)
        self.lbl.adjustSize()

if __name__ == '__main__':

    app = QApplication(sys.argv)
    ex = Examples()
    sys.exit(app.exec_())
```

Ushbu misolda QLineEdit vidjeti va QLabel ko'rsatilgan. Tahrirlash paneliga kiritgan matnimiz darhol teg vidjetida ko'rsatiladi.

```
qlineE = QLineEdit(self)
```

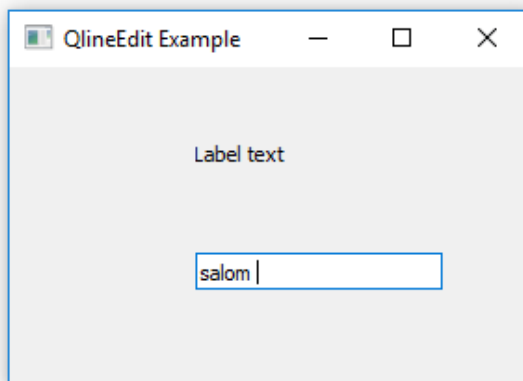
QLineEdit vidjetining `qline` obykti yaratildi.

```
qlineE.textChanged[str].connect(self.changed)
```

Agar QLineEdit vidjetidagi matn o'zgarsa, `changed()` funksiyasi ishga tushadi.

```
def changed(self, text):
    self.lbl.setText(text)
    self.lbl.adjustSize()
```

`changed` funksiyasining ichida kiritilgan matnni teg vidjetiga o'rnatish buyrug'i joylashtirilgan. Yorliq o'lchamini matn uzunligiga mos ravishda o'zgartirish uchun `adjustSize()` metodidan foydalanilgan .



Yuqoridagi dastur ko'rinishi

PyQt QLineEdit

PyQt QLineEdit bir qatorli matnli vidjet yaratish imkonini beradi. Odatda, siz QLineEditma'lumotlarni kiritish shaklida foydalanasiz .

Amalda siz ko'pincha QLineEditvidjetni vidjet bilan ishlatasiz QLabel.

Vidjet yaratish uchun QLineEditquyidagi amallarni bajaring.

Birinchidan, moduldan import QLineEditqiling PyQt6.QtWidgets:

from PyQt6.QtWidgets import QLineEditKod tili: Python (python)

Ikkinchidan, QLineEditfoydalanadigan yangi obyekt yarating:

- Hech qanday dalil yo'q.
- Faqat ota-ona vidjeti bilan.
- Yoki birinchi argument sifatida standart satr qiymati bilan.

Masalan:

line_edit = QLineEdit('Default Value', self)Kod tili: Python (python)

Bundan tashqari, siz quyidagi qo'shimcha xususiyatlardan foydalanishingiz mumkin:

Metod	Turi	Ta'ri
matn	ip	Satrn tahrirlash mazmuni
readOnly	Mantiqiy	To'g'ri yoki noto'g'ri. Agar rost bo'lsa, qatorni tahrir qilib bo'lmaydi
clearButtonEnabled	Mantiqiy	Aniq tugmani qo'shish to'g'ri

Metod	Turi	Ta'rifi
placeholderText	ip	Satrni tahrirlash bo'sh bo'lganda paydo bo'ladigan matn
maxLength	butun son	Maksimal kiritilishi mumkin bo'lgan belgilar sonini belgilang
echoMode	QLineEdit.EchoMode	Matnni ko'rsatish usulini o'zgartiring, masalan, parol

PyQt QLineEdit vidjetiga misollar

Keling, vidjetdan foydalanishga misollar keltiraylik QLineEdit.

1) Oddiy PyQt QLineEdit misoli

Quyidagi dastur vidjetni qanday yaratishni ko'rsatadi QLineEdit:

```
import sys
from PyQt6.QtWidgets import (
    QApplication,
    QWidget,
    QLineEdit,
    QVBoxLayout
)

class MainWindow(QWidget):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

        self.setWindowTitle('PyQt QLineEdit Widget')
        self.setGeometry(100, 100, 320, 210)

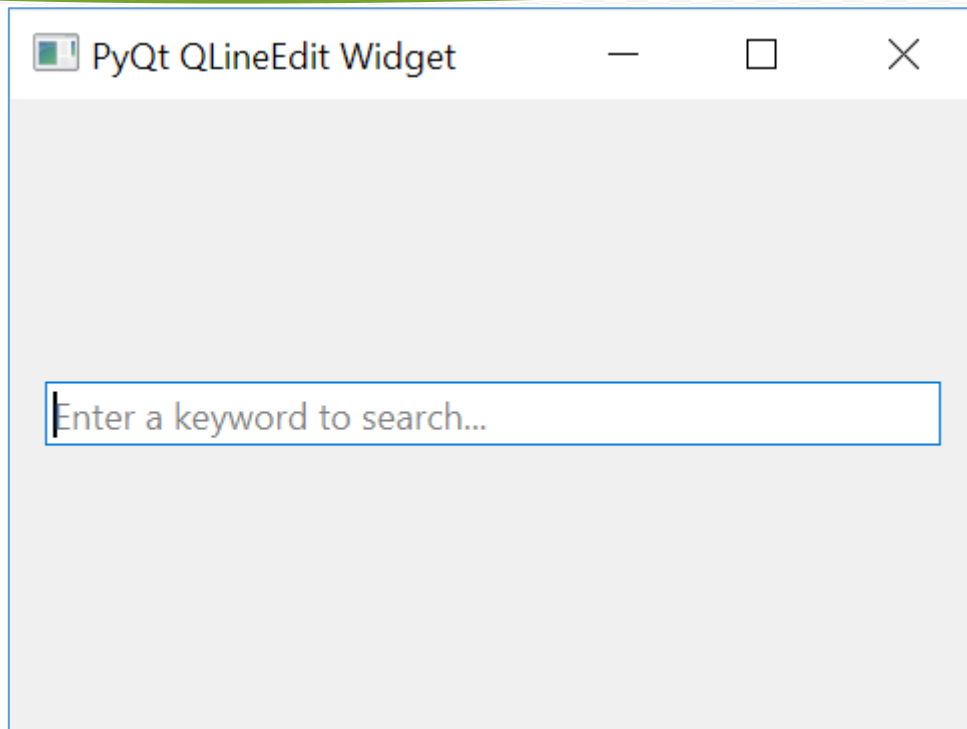
        search_box = QLineEdit(
            self,
            placeholderText='Enter a keyword to search...',
            clearButtonEnabled=True
        )

        # place the widget on the window
        layout = QVBoxLayout()
        layout.addWidget(search_box)
        self.setLayout(layout)

        # show the window
        self.show()

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = MainWindow()
    sys.exit(app.exec())
```

Chiqish:



2) Parol yozuvini yaratish uchun PyQt QLineEdit dan foydalanish
Quyidagi dastur QLineEditparol kiritish sifatida yangi vidjet yaratadi:

```
import sys
from PyQt6.QtWidgets import (
    QApplication,
    QWidget,
    QLineEdit,
    QVBoxLayout
)

class MainWindow(QWidget):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

        self.setWindowTitle('PyQt QLineEdit Widget')
        self.setGeometry(100, 100, 320, 210)

        password = QLineEdit(self, echoMode=QLineEdit.EchoMode.Password)

        # place the widget on the window
        layout = QVBoxLayout()
        layout.addWidget(password)
        self.setLayout(layout)

        # show the window
        self.show()

if __name__ == '__main__':
    app = QApplication(sys.argv)
```

```

window = MainWindow()
sys.exit(app.exec())

```

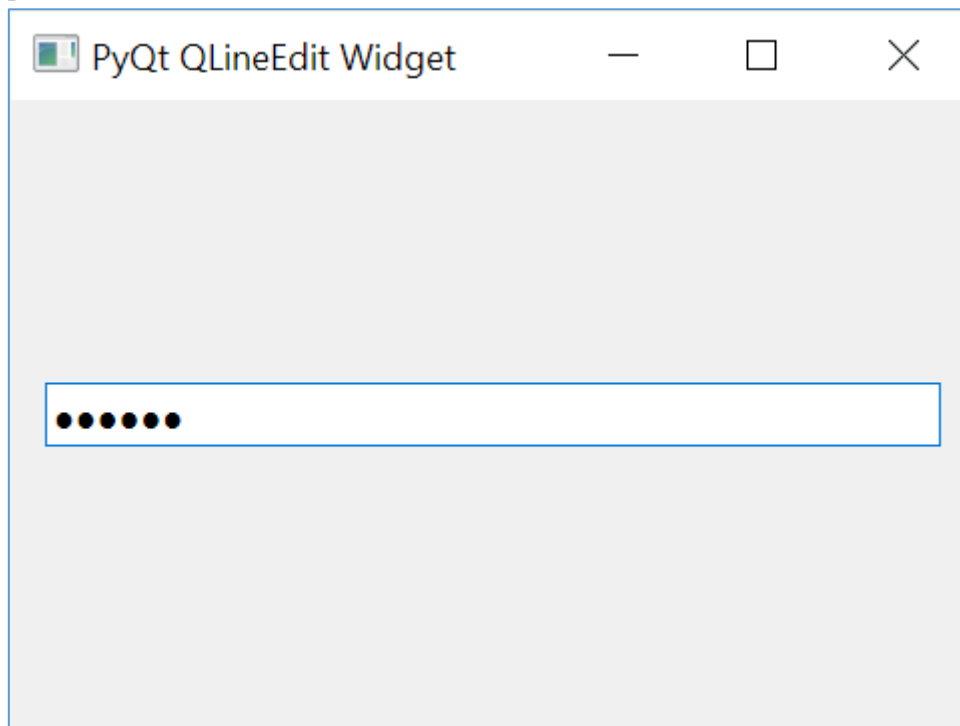
QLineEditVidjetni parol kiritishga aylantirish uchun siz echoModeni o'rnatasiz QLineEdit. EchoMode.Password:

```

password = QLineEdit(self, echoMode=QLineEdit.EchoMode.Password)

```

Chiqish:



3) PyQt QLineEdit-dan avtomatik to'ldirish xususiyati bilan foydalanish

Avtomatik to'ldirish xususiyati bilan yozuv yaratish uchun siz quyidagi amallarni bajaring:

Birinchidan, modulni import QCompleterqiling PyQt6.QtWidgets.

Ikkinchidan, QCompleteravtomatik to'ldirish funksiyasi uchun ishlatiladigan satrlar ro'yxati bilan vidjet yarating:

```

completer = QCompleter(word_list)

```

Uchinchidan, a yarating va to'ldiruvchi obyekt bilan QLineEdituning usulini chaqiring :setCompleter()

```

line_edit = QLineEdit(self)
line_edit.setCompleter(completer)

```

Masalan, quyidagi dastur QLineEditavtomatik to'ldirish xususiyatiga ega vidjetni ko'rsatadi:

```

import sys
from PyQt6.QtWidgets import (
    QApplication,
    QWidget,
    QLineEdit,
    QVBoxLayout,

```

```

        QCompleter
    )

    class MainWindow(QWidget):
        def __init__(self, *args, **kwargs):
            super().__init__(*args, **kwargs)

            self.setWindowTitle('PyQt QLineEdit Widget')
            self.setGeometry(100, 100, 320, 210)

            common_fruits = QCompleter([
                'Apple',
                'Apricot',
                'Banana',
                'Carambola',
                'Olive',
                'Oranges',
                'Papaya',
                'Peach',
                'Pineapple',
                'Pomegranate',
                'Rambutan',
                'Ramphal',
                'Raspberries',
                'Rose apple',
                'Starfruit',
                'Strawberries',
                'Water apple',
            ])
            fruit = QLineEdit(self)
            fruit.setCompleter(common_fruits)

            # place the widget on the window
            layout = QVBoxLayout()
            layout.addWidget(fruit)
            self.setLayout(layout)

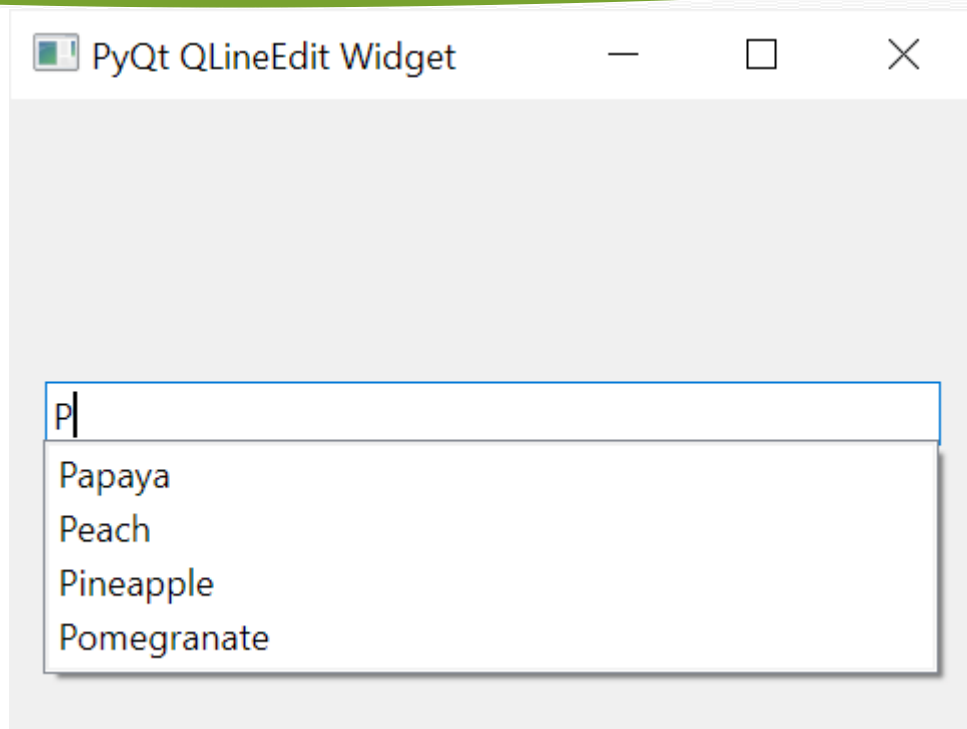
            # show the window
            self.show()

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = MainWindow()
    sys.exit(app.exec())

```

Kod tili: Python (python)

Chiqish:



- QLineEditBir qatorli kirish vidjetini yaratish uchun foydalaning .
- echoModeMatnni ko'rsatish usulini o'zgartirish uchun xususiyatdan foydalaning .
- QLineEditAvtomatik to'ldirish funksiyasini qo'llab-quvvatlash uchun vidjetdan QCompleter vidjeti bilan foydalaning .

Nazorat savollari:

1. PyQt da yorliq yaratish uchun qaysi vidgetdan foydalaniladi?
2. QLineEdit vidjetining vazifasi qanday?
3. QLineEdit vidjetining xususiyatlari qanday?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

4-mashg'ulot. PyQt5 kutubxonasidan foydalanib turli qiyinchilikdagi masalalarning dasturlarini tuzish.

O'quv savollari:

1. Turli vidgetlardan foydalanib oddiy arifmetik dasturlar tuzish;
2. Turli vidgetlardan foydalanib ro'yxatlarga doir dasturlar tuzish.

Turli vidgetlardan foydalanib oddiy arifmetik dasturlar tuzish;

Har bir obyektida shuningdek, bosilgan, ochiladigan va kengaytiriladigan Pos 3 ta maxsus signal mavjud bo'lib, biz ularni maxsus slotlarga ulaymiz. Nihoyat, biz oyna hajmini yangi tarkibga moslashtirish uchun o'lchamini o'zgartirishni chaqiramiz. Bu, aslida, faqat deraza qisqarganida kerak bo'ladi - Qt uni avtomatik ravishda o'stiradi.

```
def init_map(self):
    # Add positions to the map
    for x in range(0, self.b_size):
        for y in range(0, self.b_size):
            w = Pos(x,y)
            self.grid.addWidget(w, y, x)
            # Connect signal to handle expansion.
            w.clicked.connect(self.trigger_start)
            w.revealed.connect(self.on_reveal)
            w.expandable.connect(self.expand_reveal)

    QTimer.singleShot(0, lambda: self.resize(1,1)) # <1>
```

Taymer `singleShot` biz voqea sikliga qaytganimizdan so'ng va Qt yangi tarkibdan xabardor bo'lganimizdan keyin o'lchamni o'zgartirishni ta'minlash uchun talab qilinadi.

Endi bizda pozitsion plitka obyektlari to'plami mavjud, biz o'yin taxtasining dastlabki sharoitlarini yaratishni boshlashimiz mumkin. Bu bir qator funksiyalarga bo'lingan. Biz ularni nomlaymiz `_reset`(etakchi pastki chiziq tashqi foydalanish uchun mo'ljallanmagan shaxsiy funksiyani ko'rsatadigan konventsiyadir). Asosiy funksiya `reset_map`uni sozlash uchun navbatma-navbat bu funksiyalarni chaqiradi.

Jarayon quyidagicha -

1. Maydondan barcha minalarni olib tashlang (va ma'lumotlarni qayta o'rnatish).
2. Maydonga yangi minalar qo'shing.
3. Har bir pozitsiyaga ulashgan minalar sonini hisoblang.
4. Boshlang'ich belgisini (raketa) qo'shing va dastlabki tadqiqotni ishga tushiring.
5. Taymerni qayta o'rnatish.

Buni amalga oshirish uchun kod:

```
def reset_map(self):
```

```

self._reset_position_data()
self._reset_add_mines()
self._reset_calculate_adjacency()
self._reset_add_starting_marker()
self.update_timer()

```

1 dan 5 gacha bo'lgan alohida qadamlar quyida har bir qadam uchun kod bilan batafsil tavsiflanadi.

Birinchi qadam xaritada har bir pozitsiya uchun ma'lumotlarni qayta o'rnatishdir. Biz doskadagi har bir pozitsiyani takrorlaymiz, `.reset()` har bir nuqtada vidjetni chaqiramiz. Funksiya kodi `.reset()` bizning maxsus sinfimizda belgilangan `Pos`, biz keyinroq batafsil ko'rib chiqamiz. Hozircha u minalarni, bayroqlarni tozalab, o'z o'rnini oshkor etilmagan holatga keltirishini bilish kifoya.

```

def _reset_position_data(self):
    # Clear all mine positions
    for x in range(0, self.b_size):
        for y in range(0, self.b_size):
            w = self.grid.itemAtPosition(y, x).widget()
            w.reset()

```

Endi barcha pozitsiyalar bo'sh, biz xaritaga minalar qo'shish jarayonini boshlashimiz mumkin. Minalar maksimal soni `n_mines` yuqorida tavsiflangan daraja sozlamalari bilan belgilanadi.

```

def _reset_add_mines(self):
    # Add mine positions
    positions = []
    while len(positions) < self.n_mines:
        x, y = random.randint(0, self.b_size-1), random.randint(0,
self.b_size-1)
        if (x, y) not in positions:
            w = self.grid.itemAtPosition(y, x).widget()
            w.is_mine = True
            positions.append((x, y))

    # Calculate end-game condition
    self.end_game_n = (self.b_size * self.b_size) - (self.n_mines + 1)
    return positions

```

Minalar joyida bo'lsa, endi biz har bir pozitsiya uchun "qo'shni" raqamini hisoblashimiz mumkin - berilgan nuqta atrofida 3x3 o'lchamdagi to'rdan foydalanib, yaqin atrofdagi minalar sonini. Maxsus funksiya shunchaki ma'lum va joylashuv `get_surrounding` atrofidagi pozitsiyalarni qaytaradi. Biz kon va do'kon bo'lgan bularning sonini hisoblaymiz. `xyis_mine == True`

Bu yerda qo'shni hisoblarni oldindan hisoblash keyinchalik ochish mantiqini soddalashtirishga yordam beradi. Ammo bu shuni anglatadiki, biz foydalanuvchiga

o'zining dastlabki harakatini tanlashiga ruxsat bera olmaymiz - biz buni "raketa atrofidagi dastlabki tadqiqot" deb tushuntirib, uni butunlay oqilona qilishimiz mumkin.

Hisobni kechiktirish orqali buni o'zingiz hal qilishga harakat qiling!

```
def _reset_calculate_adjacency(self):

    def get_adjacency_n(x, y):
        positions = self.get_surrounding(x, y)
        return sum(1 for w in positions if w.is_mine)

    # Add adjacencies to the positions
    for x in range(0, self.b_size):
        for y in range(0, self.b_size):
            w = self.grid.itemAtPosition(y, x).widget()
            w.adjacent_n = get_adjacency_n(x, y)
```

Birinchi harakat har doim haqiqiy bo'lishini ta'minlash uchun boshlang'ich belgisi ishlatiladi. Bu biz mina bo'lmagan pozitsiyani topmagunimizcha, tasodifiy pozitsiyalarni samarali sinab ko'rish orqali tarmoq bo'ylab *qo'pol kuch* qidirish sifatida amalga oshiriladi. Buning uchun qancha urinishlar ketishini bilmasligimiz sababli, uni uzluksiz halqaga o'rashimiz kerak.

Bu joy topilgach, biz uni boshlang'ich joy sifatida belgilaymiz va keyin atrofda barcha pozitsiyalarni o'rganishni ishga tushiramiz. Biz sikldan chiqib, tayyor holatni tiklaymiz.

```
def _reset_add_starting_marker(self):
    # Place starting marker.

    # Set initial status (needed for .click to function)
    self.update_status(STATUS_READY)

    while True:
        x, y = random.randint(0, self.b_size - 1), random.randint(0, self.b_size
- 1)

        w = self.grid.itemAtPosition(y, x).widget()
        # We don't want to start on a mine.
        if not w.is_mine:
            w.is_start = True
            w.is_revealed = True
            w.update()

        # Reveal all positions around this, if they are not mines either.
        for w in self.get_surrounding(x, y):
            if not w.is_mine:
                w.click()
            break

    # Reset status to ready following initial clicks.
    self.update_status(STATUS_READY)
```

Turli vidgetlardan foydalanib ro'yxatlarga doir dasturlar tuzish

O'yin strukturaviy o'yin bo'lib, individual plitka pozitsiyalari o'z holati haqida ma'lumotga ega bo'ladi. Bu shuni anglatadiki, **Pos**plitkalar o'zlarining o'yin mantiqlarini boshqarishga qodir.

Sinf nisbatan murakkab bo'lganligi sababli **Pos**, u bu erda bir necha qismlarga bo'linadi va ular o'z navbatida muhokama qilinadi. Dastlabki o'rnatish `__init__` bloki oddiy bo'lib, xva ypozitsiyasini qabul qiladi va uni obyektida saqlaydi. **Pos**pozitsiyalar yaratilgandan keyin hech qachon o'zgarmaydi.

O'rnatishni yakunlash uchun `.reset()` barcha obyekt atributlarini sukut bo'yicha, nol qiymatlariga qaytaradigan funksiya chaqiriladi. Bu konni *boshlang'ich pozitsiyasi emas , mina emas , oshkor qilinmagan va belgilanmagan* deb belgilaydi . Biz qo'shni hisobni ham tiklaymiz.

```
class Pos(QWidget):

    expandable = pyqtSignal(int,int)
    revealed = pyqtSignal(object)
    clicked = pyqtSignal()

    def __init__(self, x, y, *args, **kwargs):
        super(Pos, self).__init__(*args, **kwargs)

        self.setFixedSize(QSize(20, 20))
        self.x = x
        self.y = y
        self.reset()

    def reset(self):
        self.is_start = False
        self.is_mine = False
        self.adjacent_n = 0
        self.is_revealed = False
        self.is_flagged = False

        self.update()
```

O'yin o'yin maydonidagi plitkalar bilan sichqonchanning o'zaro ta'siriga qaratilgan, shuning uchun sichqonchani bosishlarini aniqlash va ularga munosabat bildirish asosiy hisoblanadi. Qt da biz sichqonchani bosishlarini aniqlash orqali ushlaymiz `mouseReleaseEvent`. Buni bizning shaxsiy vidjetimiz uchun qilish uchun **Pos**biz sinfda ishlov beruvchini aniqlaymiz. Bu `QMouseEvent`sodir bo'lgan voqealarni o'z ichiga olgan ma'lumot bilan qabul qilinadi. Bunday holda, biz faqat sichqonchanning chap tugmasi yoki o'ng tugmasi bilan bo'shatilganligi bilan qiziqamiz.

Sichqonchanning chap tugmachasini bosish uchun biz kafelning bayroqlanganligini yoki allaqachon ochilganligini tekshiramiz. Agar shunday bo'lsa, biz chertishni e'tiborsiz qoldiramiz - bayroqchali plitalarni "xavfsiz" qilib, tasodifan

bosish mumkin emas. Agar kafel belgilanmagan bo'lsa, biz shunchaki `.click()` usulni ishga tushiramiz (keyinroq ko'ring).

Sichqonchanning o'ng tugmasi bilan ko'rsatilmagan plitkalarni bosish uchun biz `.toggle_flag()` bayroqni yoqish va o'chirish usulini chaqiramiz.

```
def mousePressEvent(self, e):

    if(e.button()==Qt.RightButton and not self.is_revealed):
        self.toggle_flag()

    elif (e.button() == Qt.LeftButton):
        # Block clicking on flagged mines.
        if not self.is_flagged and not self.is_revealed:
            self.click()
```

Ishlovchi tomonidan chaqiriladigan usullar `mousePressEvent` quyida tavsiflanadi.

Ishlovchi `.toggle_flag` shunchaki `.is_flagged` o'ziga teskarisini o'rnatadi (`True` bo'ladi `False`, `False` bo'ladi `True`) uni yoqish va o'chirish effektiga ega. `.update()` E'tibor bering, biz holatni o'zgartirib, qayta chizishga majburlash uchun qo'ng'iroq qilishimiz kerak. `.clicked` Shuningdek, biz taymerni ishga tushirish uchun ishlatiladigan maxsus signalimizni chiqaramiz - chunki bayroqni qo'yish kvadratni ochish emas, balki boshlang'ich sifatida ham hisoblanishi kerak.

Usul `.click()` sichqonchanning chap tugmachasini bosadi va o'z navbatida kvadratni ochishni boshlaydi. Agar bunga qo'shni minalar soni `Pos` nolga teng bo'lsa, biz `.expandable` o'rganilayotgan hududni avtomatik ravishda kengaytirish jarayonini boshlash uchun signalni ishga tushiramiz (keyinroq ko'ring). Nihoyat, biz yana `.clicked` o'yin boshlanishini bildirish uchun chiqaramiz.

Nihoyat, `.reveal()` usul kafel allaqachon aniqlanganligini tekshiradi va agar `.is_revealed` bo'lmasa `True`. `.update()` Vidjetni qayta bo'yashni boshlash uchun yana qo'ng'iroq qilamiz.

Signalning ixtiyoriy emissiyasi `.revealed` faqat o'yin oxiri to'liq xaritasini ochish uchun ishlatiladi. Har bir ochilish qaysi plitkalar aniqlanishi mumkinligini aniqlash uchun qo'shimcha qidiruvni ishga tushirganligi sababli, butun xaritani ochish juda ko'p ortiqcha qo'ng'iroqlarni keltirib chiqaradi. Bu erda signalni bostirish orqali biz bundan qochamiz.

```
def toggle_flag(self):
    self.is_flagged = not self.is_flagged
    self.update()

    self.clicked.emit()

def click(self):
    self.reveal()
    if self.adjacent_n == 0:
```

```

        self.expandable.emit(self.x, self.y)

    self.clicked.emit()

    def reveal(self, emit=True):
        if not self.is_revealed:
            self.is_revealed = True
            self.update()

            if emit:
                self.revealed.emit(self)

```

Va nihoyat, biz vidjetimiz uchun joriy joylashuv holatini ko'rsatishni boshqarish uchun maxsus `paintEvent` usulni aniqlaymiz. `Pos[bobda]` tasvirlanganidek, vidjet tuvalini maxsus bo'yashni amalga oshirish uchun biz chizishimiz kerak bo'lgan chegaralarni - bu holda vidjetning tashqi chegarasini ko'rsatadigan belgini `QPainter` olamiz `.event.rect()`

Ochilgan plitkalar, kafelning *boshlang'ich pozitsiyasi*, *bomba* yoki *bo'sh joy* bo'lishiga qarab turlicha chiziladi. Birinchi ikkitasi mos ravishda raketa va bomba piktogrammalari bilan ifodalanadi. `QRect` Ular yordamida plitka ichiga tortiladi `.drawPixmap`. `QImage` E'tibor bering, biz o'tish orqali o'tish orqali doimiylarni piksel xaritalariga aylantirishimiz kerak `QPixmap`.

Siz shunday deb o'ylashingiz mumkin: "Nima uchun ularni faqat obyektlar sifatida saqlamaslik kerak, chunki biz aynan shu narsadan foydalanamiz? Afsuski, siz o'zingiz ishga tushmasdan oldin obyektlar `QPixmap` yarata olmaysiz. `QPixmapQApplication`

Bo'sh pozitsiyalar uchun (raketalar emas, bombalar emas) biz ixtiyoriy ravishda qo'shni raqamni ko'rsatamiz, agar u noldan katta bo'lsa. Matnni matnga chizish uchun biz, hizalama bayroqlari va qator sifatida chizish uchun raqamni o'tkazishdan `QPainter` foydalanamiz. Foydalanish qulayligi uchun biz har bir raqam uchun standart rangni belgilab oldik (da saqlangan) `.drawText()` `QRectNUM_COLORS`

Ochilmagan plitkalar uchun biz to'rtburchakni ochiq kul rang bilan to'ldirib, plitka chizamiz va quyuqroq kul rangning 1 pikselli chegarasini chizamiz. Agar o'rnatilgan bo'lsa, biz va plitkadan `.is_flagged` foydalanib, kafelning yuqori qismiga bayroq belgisini ham chizamiz `.drawPixmapQRect`

```

def paintEvent(self, event):
    p = QPainter(self)
    p.setRenderHint(QPainter.Antialiasing)

    r = event.rect()

    if self.is_revealed:
        if self.is_start:

```

```

        p.drawPixmap(r, QPixmap(IMG_START))

    elif self.is_mine:
        p.drawPixmap(r, QPixmap(IMG_BOMB))

    elif self.adjacent_n > 0:
        pen = QPen(NUM_COLORS[self.adjacent_n])
        p.setPen(pen)
        f = p.font()
        f.setBold(True)
        p.setFont(f)
        p.drawText(r, Qt.AlignHCenter | Qt.AlignVCenter,
str(self.adjacent_n))

    else:
        p.fillRect(r, QBrush(Qt.lightGray))
        pen = QPen(Qt.gray)
        pen.setWidth(1)
        p.setPen(pen)
        p.drawRect(r)

    if self.is_flagged:
        p.drawPixmap(r, QPixmap(IMG_FLAG))

```

Mexanika

Biz odatda ma'lum bir nuqta atrofidagi barcha plitkalarni olishimiz kerak, shuning uchun bizda bu maqsad uchun maxsus funksiya mavjud. U nuqta atrofida 3x3 o'lchamdagi to'r bo'ylab oddiy takrorlanadi va biz to'r chetlarida chegaradan chiqmasligimizni tekshirib ko'ring ($0 \leq x \leq \text{self.b_size}$). Qaytarilgan ro'yxatda **Poshar** bir atrofdagi joydan vidjet mavjud.

```

def get_surrounding(self, x, y):
    positions = []

    for xi in range(max(0, x - 1), min(x + 2, self.b_size)):
        for yi in range(max(0, y - 1), min(y + 2, self.b_size)):
            if not (xi == x and yi == y):
                positions.append(self.grid.itemAtPosition(yi, xi).widget())

    return positions

```

Ushbu **expand_reveal** nolgga qo'shni minalar bo'lgan kafelni bosishga javoban ishga tushiriladi. Bunday holda, biz bosish atrofidagi maydonni nolgga qo'shni minalar bo'lgan har qanday bo'shliqqa kengaytirishni xohlaymiz, shuningdek, bu kengaytirilgan maydonning chegarasi atrofidagi har qanday kvadratlarni (bu minalar emas) aniqlamoqchimiz.

Biz keyingi iteratsiyada tekshiriladigan pozitsiyalarni o'z ichiga olgan ro'yxat, ko'rsatiladigan plitka vidjetlarini o'z ichiga olgan **to_expand** ro'yxat

va sikldan qachon chiqishni aniqlash uchun bayroqdan boshlaymiz. Birinchi marta yangi vidjetlar qo'shilmasa, sikl to'xtaydi `.to_revealany_addedto_reveal`

Loop ichida biz `any_added` ni `False` ga qayta o'rnatamiz, ro'yxatni bo'shatamiz va takrorlash uchun `to_expand` ni vaqtincha `1` o'zgaruvchisida saqlab qolamiz. Har bir joy uchun biz 8 ta qo'shni vidjetni olamiz. Agar ushbu vidjetlardan biri **mina bo'lmasa** va `to_reveal` ro'yxatida bo'lmasa, uni `to_reveal` ga qo'shamiz. Bu kengaytirilgan maydonning barcha qirralari oshkor bo'lishini ta'minlaydi. Agar pozitsiyada qo'shni minalar soni `0` bo'lsa, keyingi iteratsiyada tekshiriladigan koordinatalarni `to_expand` ga qo'shamiz.

`to_reveal` ga faqat oshkor qilinmagan plitkalarni qo'shish va `to_expand` orqali faqat hali ko'rilmagan plitkalarni kengaytirish natijasida, biz bir xil kafelga bir necha marta tashrif buyurmasligimizga ishonch hosil qilamiz.

```
def expand_reveal(self, x, y):
    """
    Iteratsiyani boshlang'ich nuqtadan tashqariga kengaytirib,
    yangi joylarni navbatga qo'shamiz. Bu bir necha callback'lara
    tayanmasdan, barchasini bir martada kengaytirish imkonini beradi.
    """
    to_expand = [(x, y)]
    to_reveal = []
    any_added = True

    while any_added:
        any_added = False
        # avvalgi to_expand'ni l ga saqlab, to_expand'ni bo'shatamiz
        to_expand, l = [], to_expand

        for x, y in l:
            positions = self.get_surrounding(x, y)
            for w in positions:
                if not w.is_mine and w not in to_reveal:
                    to_reveal.append(w)
                    if w.adjacent_n == 0:
                        to_expand.append((w.x, w.y))
                        any_added = True

        # Topilgan barcha pozitsiyalarni oshkor qilamiz.
        for w in to_reveal:
            w.reveal()
```

2. Dasturni testlash

O'yin yakun holati plitka ustiga bosilgandan so'ng oshkor qilish jarayonida aniqlanadi. Ikkita mumkin bo'lgan natija bor:

1. Plitka mina bo'lsa — o'yin tugaydi.
2. Plitka mina bo'lmasa — `self.end_game_n` (bo'sh plitkalar soni) kamaytiriladi.

Bu qiymat 0 ga yetguncha davom etadi; 0 bo'lganda g'alaba jarayoni (game_won) boshlanadi. Muvaffaqiyat ham, muvaffaqiyatsizlik ham xaritani oshkor qilish (reveal_map) va tegishli holatni o'rnatish orqali yakunlanadi.

```
def on_reveal(self, w):
    if w.is_mine:
        self.game_over()
    else:
        # qolgan bo'sh joylarni kamaytiramiz
        self.end_game_n -= 1
        if self.end_game_n == 0:
            self.game_won()

def game_over(self):
    self.reveal_map()
    self.update_status(STATUS_FAILED)

def game_won(self):
    self.reveal_map()
    self.update_status(STATUS_SUCCESS)
```

Nazorat savollari:

1. PyQt5 kutubxonasi nima va u qaysi maqsadlarda qo'llaniladi?
2. PyQt5 kutubxonasining asosiy komponentlari (QWidget, QMainWindow, QApplication, va boshqalar) haqida gapirib bering.
3. PyQt5 dasturida asosiy oynani (Main Window) yaratish jarayonini tushuntiring.
4. QPushButton vidjeti qanday yaratiladi va unga bosilganda bajariladigan amal qanday ulanadi?
5. PyQt5 da QLineEdit va QTextEdit vidjetlarining farqi nimada?
6. QLabel yordamida dastur oynasiga matn yoki rasm qanday joylashtiriladi?
7. Signal va Slot tushunchasi PyQt5 da qanday ishlaydi? Misol keltiring.
8. PyQt5 da layoutlar (QHBoxLayout, QVBoxLayout, QGridLayout) nima vazifa bajaradi?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

5-mashg'ulot. PyQt5 modal dialog. QMessageBox vidjeti bilan ishlash.

O'quv savollari:

1. QMessageBox vidjetining vazifasi.
2. QMessageBox vidjetining asosiy xususiyatlari.
3. Statik funksiyalari.
4. Piktogramma va QPixmap xususiyatlari.

QMessageBox vidjetining vazifasi

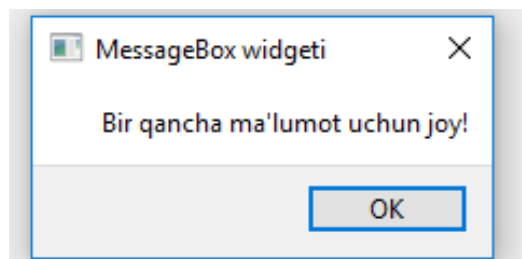
QMessageBox - bu dialoglar yaratish uchun ishlatiladigan foydali PyQt5 vidjeti. Ushbu dialog oynalari turli maqsadlarda ishlatilishi va turli xil xabarlar va tugmalarni ko'rsatish uchun ko'p jihatdan moslashtirilgan bo'lishi mumkin.

Quyida oddiy QMessageBox vidjetini yaratmoqda. Ammo bu hech qanday qo'shimcha funksiyalar yoki sozlashsiz faqat MessageBox vidjetining o'zining tinchlik xolatidagi ko'rinishi bo'ladi.

```
def display():
    msg = QMessageBox(wind)
    msg.setWindowTitle("MessageBox widgeti")
    msg.setText("Bir qancha ma'lumot uchun joy!")

    x = msg.exec_()
```

Va quyida chiqish. Agar siz ekranda oddiy xabarni ko'rsatmoqchi bo'lsangiz, buni qilishning yo'li.



61-rasm. QMessageBox vidjetining o'zining ko'rinishi

QMessageBox vidjetining asosiy xususiyatlari

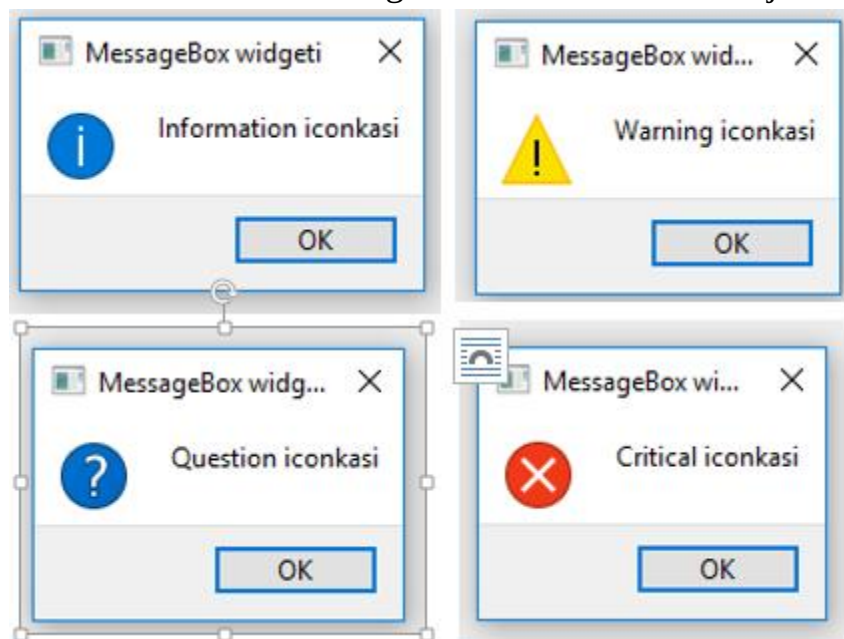
QMessageBox – da **.setIcon()** metodini chaqirish orqali foydalanish mumkin bo'lgan 4 xil turdagi piktogramma mavjud bo'lib, ko'rsatmoqchi bo'lgan xabar turiga qarab qaysi birini ishlatishni hal qilinadi.

- QMessageBox.Question
- QMessageBox.Information
- QMessageBox.Warning
- QMessageBox.Critical

Yuqoridagilardan birini quyida ko'rsatilganidek **setIcon()** funksiyasiga o'tkazish kifoya.

```
msg.setIcon(QMessageBox.Question)
```

Quyida ko'rsatilishi mumkin bo'lgan to'rt xil xabarlar mavjud.



62-rasm. QMessageBox pictogrammalarining turlari

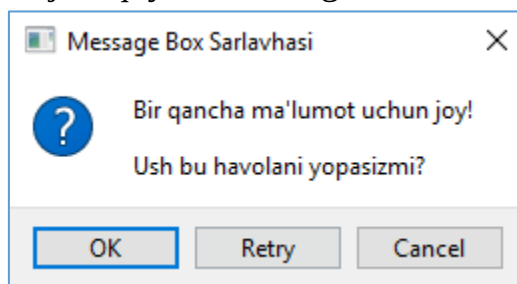
QMessageBox tugmalari

Hozirgacha biz QMessageBox taqdim etgan standart "OK" tugmasidan foydalanib kelmoqdamiz. Biroq, QMessageBox foydalanish uchun o'ndan ortiq turli xil tugmalar taklif qiladi. Quyida ba'zi tugmalardan foydalanish misoli keltirilgan.

setStandardButtons() funksiyadan foydalanib xohlagan tugma turini o'ratish mumkin.

```
def display():
    msg = QMessageBox(wind)
    msg.setWindowTitle("Message Box Sarlavhasi")
    msg.setText("Bir qancha ma'lumot uchun joy!")
    msg.setIcon(QMessageBox.Question)
    msg.setStandardButtons(QMessageBox.Cancel
        | QMessageBox.Ok
        | QMessageBox.Retry)
    msg.setInformativeText("Ush bu havolani yopasizmi?")
    x = msg.exec_()
```

Ushbu dasturning natijasi quyida keltirilgan:



63-rasm. QMessageBox ga doir oddiy dastur natijasi

Buni allaqachon payqagan bo'lishingiz mumkin, lekin tugmalar tartibi ularning xabarlar oynasida qanday ko'rinishiga ta'sir qilmaydi.

Default Button

Yuqoridagi rasimga qarab "OK" tugmasi atrofida ko'k kontur borligini payqash mumkin. Bu ushbu tugma **Default Button** ekanligini bildiradi.

setDefaultButton() funksiyasidan foydalanib, standart tugmani o'zgartirish mumkin. Kodga quyidagi qatorni qo'shish ko'k rangni "Cancel" tugmasi atrofida bo'lishiga olib keladi.

```
msg.setDefaultButton(QMessageBox.Cancel)
```

Bu yerda **QMessageBox** vidjetida foydalanish mumkin bo'lgan turli xil tugma turlarining to'liq ro'yxati keltirilgan:

- QMessageBox.Ok
- QMessageBox.No
- QMessageBox.Yes
- QMessageBox.Cancel
- QMessageBox.Close
- QMessageBox.Abort
- QMessageBox.open
- QMessageBox.Retry
- QMessageBox.Ignore
- QMessageBox.Save
- QMessageBox.Retry
- QMessageBox.Apply
- QMessageBox.Help
- QMessageBox.Reset
- QMessageBox.SaveAll
- QMessageBox.YesToAll
- QMessageBox.NoToAll

3. Statik funksiyalari

Odatda QMessageBox-da Matnni ko'rsatadigan faqat bitta maydon mavjud. Aslida esa, qo'shimcha matn qo'shish uchun yana ikkita qo'shimcha maydon mavjud.

Birinchisi, QMessageBox oynasining o'zida qo'shimcha matn bo'limi. Bu qo'shish mumkin bo'lgan qo'shimcha matn qatoriga o'xshaydi. Ushbu yangi matn maydonini qo'shish uchun faqat setInformativeText() funksiyasiga murojaat qilish kerak.

Ikkinchisi QMessageBox-dan kengaytirilgan maydonda ko'rsatiladi. Bu **Detailed Text** – ya'ni "Batafsil matn" deb ataladi. Ushbu bo'limni o'rnatish avtomatik ravishda ushbu maydonni ko'rsatish uchun ishlatiladigan tugmani qo'shadi. Ushbu

batafsil matn yoziladigan maydonni qo'shish uchun faqat **setDetailedText()** funksiya kerak bo'ladi.

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import *
import sys

def show_popup():
    msg = QMessageBox(win)
    msg.setWindowTitle("Message Box sarlavhasi")
    msg.setText("Bu qandaydir matn")
    msg.setIcon(QMessageBox.Question)

    msg.setStandardButtons(QMessageBox.Cancel|QMessageBox.Ok)
    msg.setDefaultButton(QMessageBox.Ok)

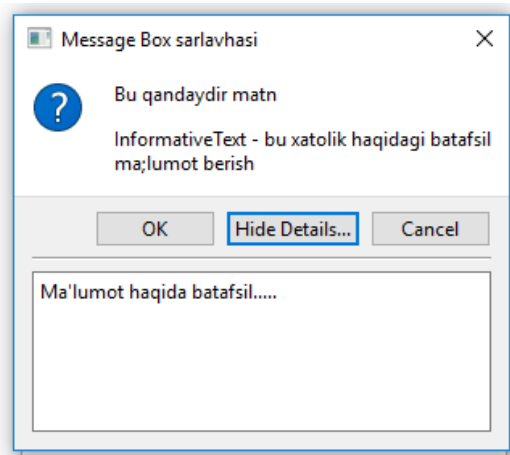
    msg.setDetailedText("Ma'lumot haqida batafsil.....")
    msg.setInformativeText("InformativeText - bu xatolik haqidagi batafsil ma'lumot berish")
    x = msg.exec_()

app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,300,300)
win.setWindowTitle("QMessageBox widgeti haqida")

button = QtWidgets.QPushButton(win)
button.setText("A Button")
button.clicked.connect(show_popup)
button.move(100,100)

win.show()
sys.exit(app.exec_())
```

Yuqoridagi dasturning ishga tushirilganidan keyin quyidagi messagebox oynasi paydo bo'ladi:



QMessageBox widgetining to'liq ko'rinishi

Hide Details/Show Details tugmasini bosish mos ravishda xabarlar qutisi ostidagi qo'shimcha maydonni yashirish/ko'rsatish mumkin.

QMessageBox qiymatlarini olish

Yuqorida juda ko'p turli xil sozlashlar va xususiyatlarni muhokama qilindi, ammo aslida bu turli xususiyat va tugmalarning hech birini kodga bog'lanmadi.

Misol uchun, agar MessageBoxda 3 xil tugma bo'lsa, qaysi biri bosilganligini qanday bilish mumkin? Buning uchun quyida ko'rsatilgan kodyaqqol javob bo'la oladi:

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import *
import sys

def show_popup():
    msg = QMessageBox(win)
    msg.setWindowTitle("Message Box sarlavhasi")
    msg.setText("Bu qandaydir matn")
    msg.setIcon(QMessageBox.Question)
    msg.setStandardButtons(QMessageBox.Cancel|QMessageBox.Ok)
    msg.setDefaultButton(QMessageBox.Ok)

    msg.setDetailedText("Ma'lumot haqida batafsil.....")
    msg.setInformativeText("InformativeText - bu xatolik haqidagi batafsil ma'lumot berish")
    msg.buttonClicked.connect(popup)
    x = msg.exec_()

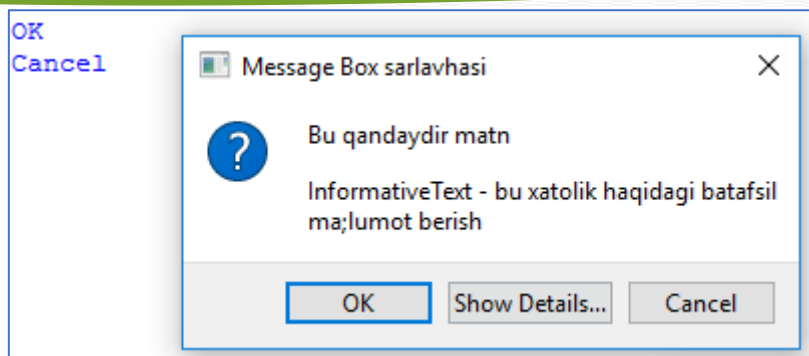
def popup(i):
    print(i.text())

app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,300,300)
win.setWindowTitle("QMessageBox widgeti haqida")

button = QtWidgets.QPushButton(win)
button.setText("A Button")
button.clicked.connect(show_popup)
button.move(100,100)

win.show()
sys.exit(app.exec_())
```

buttonClicked.connect() usulidan foydalanib MessageBoxdagi birorta tugma bosilganda funksiyani ishga tushirish mumkin. Bunday holda, messageboxni **popup** deb nomlangan funksiyaga bog'landi. popup() funksiyas bitta parametr qabul qiladi.



QMessageBox vidjetining to'liq imkoniyatlari bilan ko'rinishi

MessageBoxning tugmasi bosilganda, u avtomatik ravishda bosilgan tugmani **buttonClicked.connect()** da e'lon qilingan funksiyaga uzatadi. Ushbu tugmadagi **text()** funksiyasini chaqirish esa tugmaning matnini qaytaradi.

Nazorat savollari:

1. QMessageBox vidjeti qanday vazifani bajaradi?
2. QMessageBox qanday ishga tushiriladi?
3. QMessageBox ning qanday xususiyatlari mavjud?
4. QMessageBox vidjetida qanday pictogrammalar mavjud?
5. QMessageBoxga o'rnatish mumkin bo'lgan standart buttonlarni keltiring?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

6-mashg'ulot. PyQt da rasmlar va menyular.

O'quv savollari:

1. PyQt paketi yordamida rasm joylashtirish usullari.
2. Menu yaratish. Menu vidjetining xususiyatlari.

1. PyQt paketi yordamida rasm joylashtirish usullari

QPixmap vidjeti

QPixmap – bu tasvirlarni ekranga chiqarish uchun ishlatiladigan vidjetlardan biri. Quyidagi misolda tasvirni ekranda ko'rsatish uchun **QPixmap** metodidan foydalanilgan.

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import QPixmap

class Example(QWidget):
    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):

        h_box = QHBoxLayout(self)
        pixmap = QPixmap("blue_black.png")

        lbl = QLabel(self)
        lbl.setPixmap(pixmap)

        h_box.addWidget(lbl)
        self.setLayout(h_box)

        self.move(300, 200)
        self.setWindowTitle('Blue and Black')
        self.show()

if __name__ == '__main__':

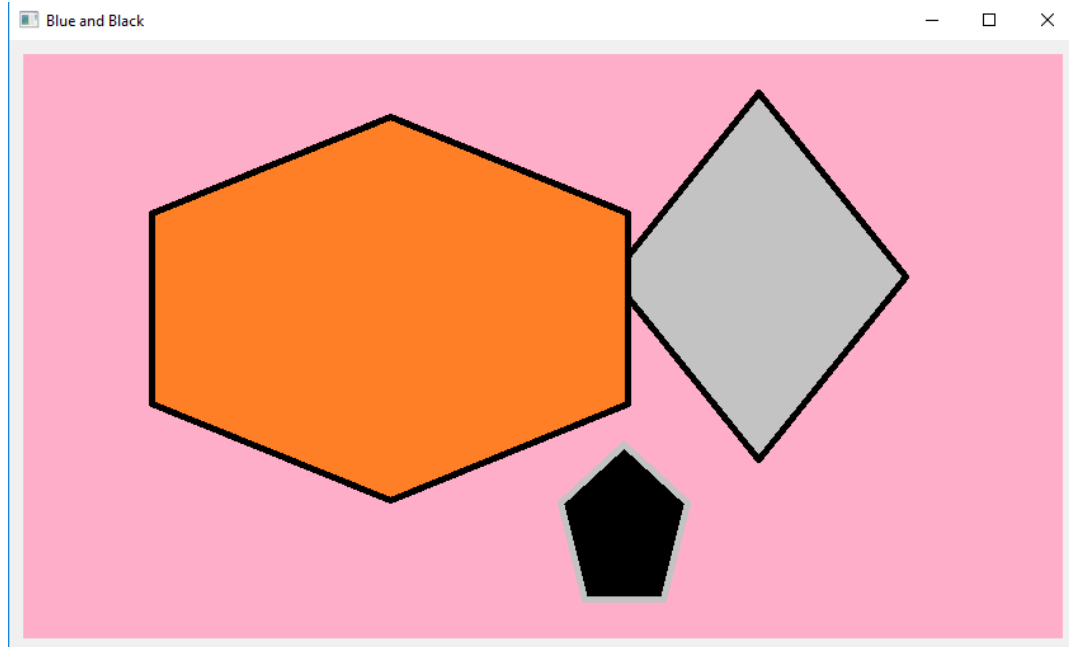
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())
```

Ushbu dasturda `blue_black.png` nomli rasmni oynada ko'rsatish uchun quyidagi amallarni bajarildi:

1. `pixmap = QPixmap("blue_black.png")` - `QPixmap` sinfi obyektini yaratildi.

2. `lbl = QLabel(self)`

`lbl.setPixmap(pixmap)` - rasmni `QLabel` vidjetidan foydalanib ekranga joylashtirildi.



71-rasm. `QPixmap` vidjetiga rasm joylashtirilgandagi ko'rinishi

2. Menu yaratish. Menu vidjetining xususiyatlari.

PyQt5 ga bag'ishlangan bobning ushbu qismida menyu va asboblari paneli (asboblari paneli) vidjetlaridan foydalanishni ko'rib chiqiladi.

Menyu - bu menyu satrida joylashgan buyruqlar guruhi. Asboblari panelida ilovadagi ba'zi umumiy buyruqlar uchun tugmalar mavjud.

Menyu paneli

Menyu paneli GUI ilovasining umumiy qismidir. Bu turli xil menyularda joylashgan buyruqlar guruhi.

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import QIcon

class Example(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):

        exitAction = QAction(QIcon('exit.png'), '&Exit', self)
        exitAction.setShortcut('Ctrl+Q')
        exitAction.setStatusTip('Exit application')
        exitAction.triggered.connect(qApp.quit)
```

```

self.statusBar()

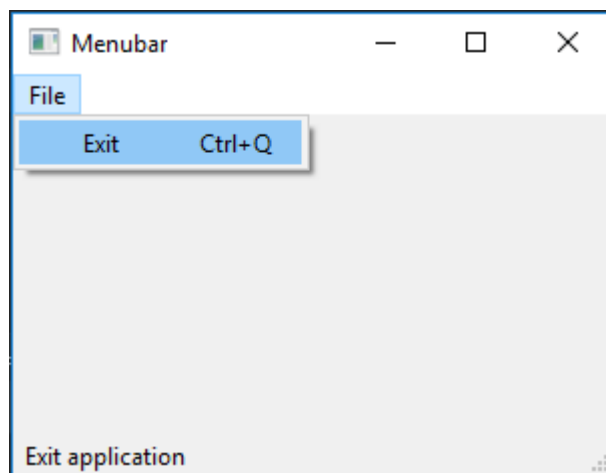
menubar = self.menuBar()
fileMenu = menubar.addMenu('&File')
fileMenu.addAction(exitAction)

self.setGeometry(300, 300, 300, 200)
self.setWindowTitle('Menubar')
self.show()

if __name__ == '__main__':

    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

```



Menubar widgetining ilovada ko'rinishi.

Yuqoridagi misolda bitta menubar vidjeti yordamida menyu panelini yaratildi. Ushbu menyu tarkibida dasturni to'xtatadigan bitta buyruq kiritilgan. Ushbu buyruqni menular panelidan sichqoncha bilan tanlash yoki klaviaturadan Ctrl + q tugmalari yordamida ishga tushirish ham mumkin.

Datur tahlili:

```

exitAction = QAction(QIcon('exit.png'), '&Exit', self)
exitAction.setShortcut('Ctrl+Q')
exitAction.setStatusTip('Exit application')

```

Ushbu uchta qatorda tegishli belgi bilan hodisa yaratilgan. Bundan tashqari, ushbu hodisa uchun tugmalar birikmasi aniqlanadi. Uchinchi qator kursorni menyu bandi ustiga olib borilganda satr yonida maslahatchiga o'xshagan yozuv yaratadi.

```
exitAction.triggered.connect(qApp.quit)
```

Ushbu maxsus harakatni tanlaganimizda, signal ishga tushadi. Signal **QApplication** vidjetining **quit()** metodini ishga tushiradi va ilovani yakunlaydi.

```

menubar = self.menuBar()
fileMenu = menubar.addMenu('&File')
fileMenu.addAction(exitAction)

```

Bu yerga **menyuBar()** vidjeti yordamida bitta menular satri yaratilyapdi. fileMenu o'zgaruvchisiga menular satri File nomli menu qo'shish buyrug'i yuklanmoqda. Va uchinchi satrda Fayl menyusining tarkibida dasturdan chiqish amalini qo'shilmoqda.

Asboblari paneli (toolbar)

Menyular ilovada foydalanishimiz mumkin bo'lgan barcha buyruqlarni guruhlaydi.

Asboblari paneli eng ko'p ishlatiladigan buyruqlarga tezkor kirishni ta'minlaydi.

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import QIcon

class Example(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initUI()

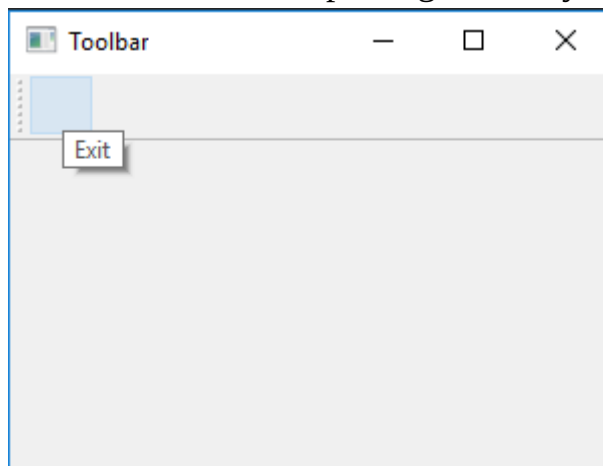
    def initUI(self):
        exitAction = QAction(QIcon('exit24.png'), 'Exit', self)
        exitAction.setShortcut('Ctrl+Q')
        exitAction.triggered.connect(qApp.quit)

        self.toolbar = self.addToolBar('Exit')
        self.toolbar.addAction(exitAction)

        self.setGeometry(300, 300, 300, 200)
        self.setWindowTitle('Toolbar')
        self.show()

if __name__ == '__main__':
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())
```

Bu yerdagi deyarli hamma narsa status paneliga o'xshaydi.



Asboblari panelining ilovada ko'rinishi.

Hammasini birlashtirish

Ushbu bo‘limning oxirgi misolida menyu satri, asboblari paneli va holat panelini yaratildi. Shuningdek, markaziy vidjetni ham ular bilan birlashtirilgan.

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import QIcon

class Example(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):

        textEdit = QTextEdit()
        self.setCentralWidget(textEdit)

        exitAction = QAction(QIcon('exit24.png'), 'Exit', self)
        exitAction.setShortcut('Ctrl+Q')
        exitAction.setStatusTip('Exit application')
        exitAction.triggered.connect(self.close)

        self.statusBar()

        menubar = self.menuBar()
        fileMenu = menubar.addMenu('&File')
        fileMenu.addAction(exitAction)

        toolbar = self.addToolBar('Exit')
        toolbar.addAction(exitAction)

        self.setGeometry(300, 300, 350, 250)
        self.setWindowTitle('Main window')
        self.show()

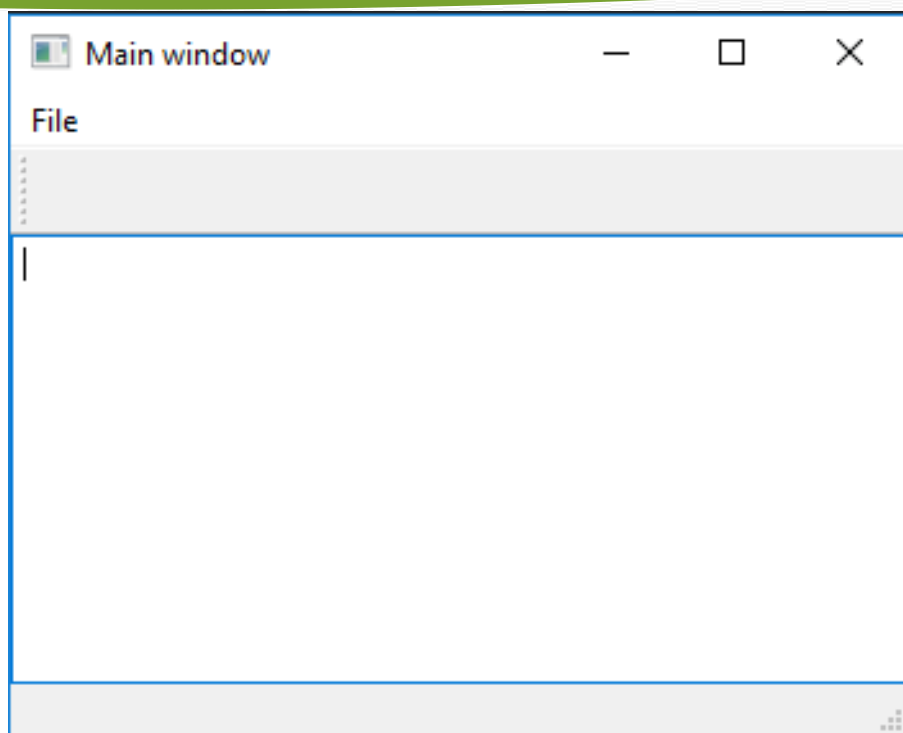
if __name__ == '__main__':

    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())
```

Ushbu kod misoli menyu, asboblari paneli va holat paneli bilan klassik GUI ilovasining skeletini yaratadi.

```
textEdit = QTextEdit()
self.setCentralWidget(textEdit)
```

Bu yerda matnni tahrirlash vidjetini yaratilmoqda. Uni **QMainWindow** ning markaziy vidjetiga aylantiriladi. Markaziy vidjet qolgan barcha bo‘sh joyni egallaydi.



Menu va uning asboblar paneli ko'rinishi.

Nazorat savollari:

1. Menuni hosil qilishi uchun qanday widgetdan foydalaniladi?
2. Statusbar widgeti nima vazifani bajaradi?
3. Menular panelini qaysi widgetdan foydalanib o'rnatiladi?
4. QPixmap vidjetining imkoniyatlaridan qanday foydalaniladi?
5. QPixmap vidjetining qanday xususiyatlari mavjud?
6. QPixmap vidjeti yordamida rasm joylashtirish qanday amalga oshiriladi?

O'quv savollari:

- ## 1. Yaratiladigan matn muharriri uchun kerakli uskunalarni tanlash

Bizga nimalar kerak bo‘ladi?

Birinchi navbatda Windowsda PyQt paketini va Qt Designer dasturini yuklab olish hamda oʻrnatish talab qilinadi. QtDesigner dasturi PyQt paketi bilan birgalikda oʻrnatiladi.

Linux : Sizga kerak bo'lgan hamma narsa, ehtimol, tarqatish omborlarida. Qt Designer ilova markazidan o'rnatilishi mumkin, lekin PyQt terminal orqali o'rnatilishi kerak. Bizga kerak bo'lgan hamma narsani bitta buyruq bilan o'rnatishingiz mumkin, masalan:

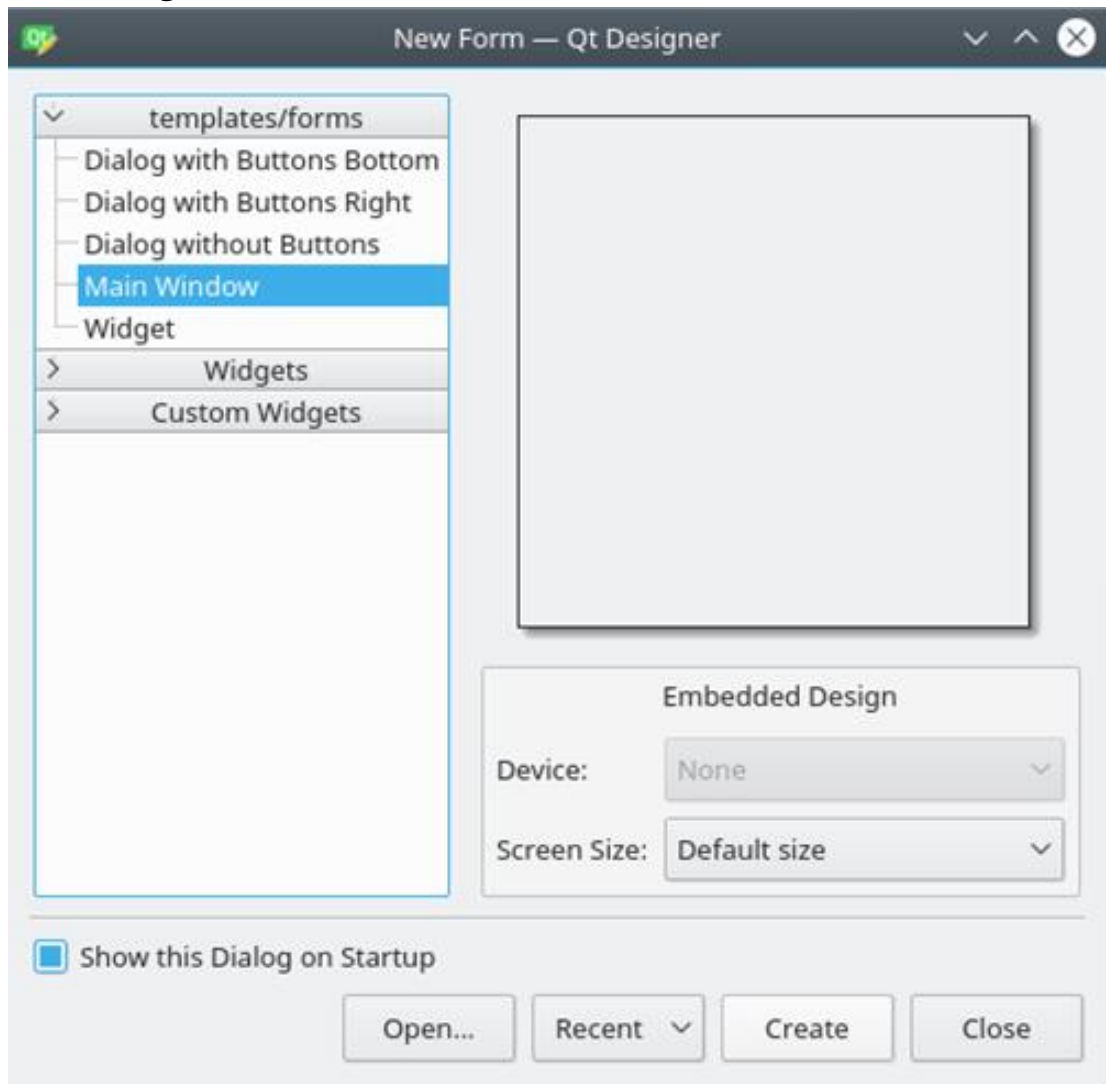
```
$ sudo apt install python3-gt5 pyqt5-dev-tools qtccreator
```

Error: one **input** ui-file must be specified

Dastur Dizaynini yaratish

170

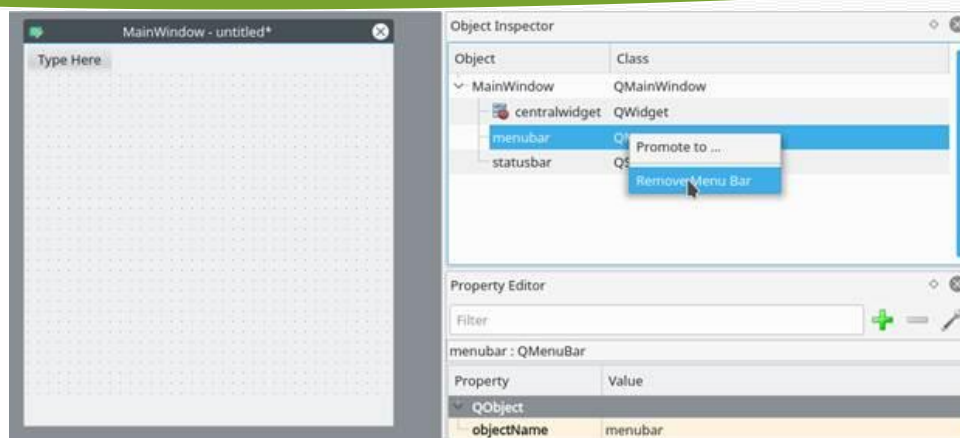
Qt Designer dasturini ishga tushuriladi, yangi ochilgan oynada kichik dialog oynasi paydo bo'ladi. U yerdan "**Main Window**" bo'limi tanlab, **Create** tugmasini bosing.



Shundan so'ng sizda **forma** - oyna dizayn uchun shablon paydo bo'lishi kerak, uning o'lchamini o'zgartirish mumkin va vidjet oynasidan obyektlarni qayerga hohlasangiz qo'shishingiz mumkin bo'ladi. QtDesigner dasturi interfeysi bilan tanishib chiqing, bu juda ham oddiy tuzulishga ega.

Endi asosiy oynamiz hajmini biroz o'zgartiramiz. Bizga bunchalik katta bo'lishi shart emas. Va keling, avtomatik qo'shilgan menyu va holat panelini ham olib tashlaymiz, chunki ular bizning ilovamizda foydali bo'lmaydi.

Barcha shakl elementlari va ularning ierarxiyasi sukut bo'yicha Qt Designer oynasining o'ng tomonidagi "*Object Inspector*" deb nomlangan oynada ko'rsatiladi. Ushbu oynada obyektlarni sichqonchani o'ng tugmasi bilan osongina o'chirishingiz mumkin. Yoki ularni asosiy shaklda tanlashingiz va klaviaturangizdagi **DEL** tugmasini bosishingiz mumkin.



Natijada, bizda deyarli bo'sh shakl mavjud. Qolgan yagona obyekt - **centralwidget**, lekin bizga kerak bo'ladi, shuning uchun biz u bilan hech narsa qilmaymiz.

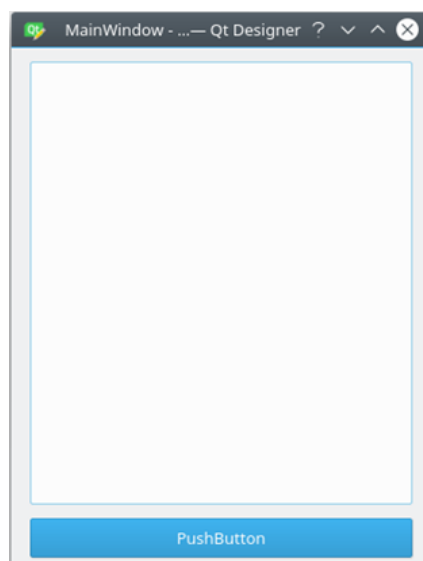
2. Tanlangan uskunalarni matn muharriri ekraniga joylashtirish

Endi *WidgetBox* dan *ListWidget* ni (*ListView* emas) va *PushButton* larni asosiy shaklning biror joyiga joylashtiring.

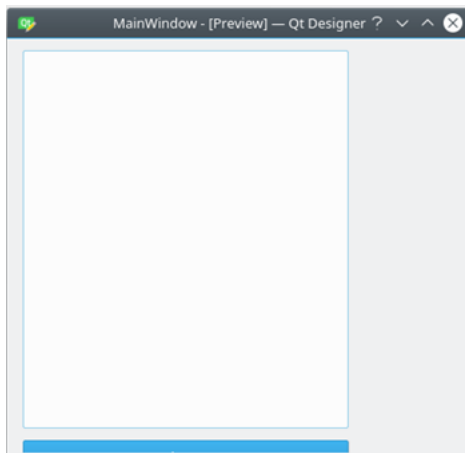
Maketlar. Ilovangizdagi elementlar uchun qat'iy pozitsiyalar va o'lchamlardan foydalanish o'rniga, maketlardan foydalangan ma'qul. Ruxsat etilgan pozitsiyalar va o'lchamlar sizga yaxshi ko'rinadi (oyna o'lchamini o'zgartirmaguningizcha), lekin boshqa mashinalar va/yoki operatsion tizimlarda aynan bir xil bo'lishiga hech qachon ishonch hosil qila olmaysiz.

Maketlar - bu vidjetlar uchun konteynerlar bo'lib, ularni boshqa elementlarga nisbatan o'z o'rnida ushlab turadi. Shuning uchun, oyna hajmi o'zgartirilganda, vidjetlarning o'lchami ham o'zgaradi.

Keling, maketlardan foydalanmasdan birinchi shaklimizni yarataylik. Shakldagi ro'yxat va tugmani torting va ularning o'lchamlarini quyidagicha ko'rinishda o'zgartiring:

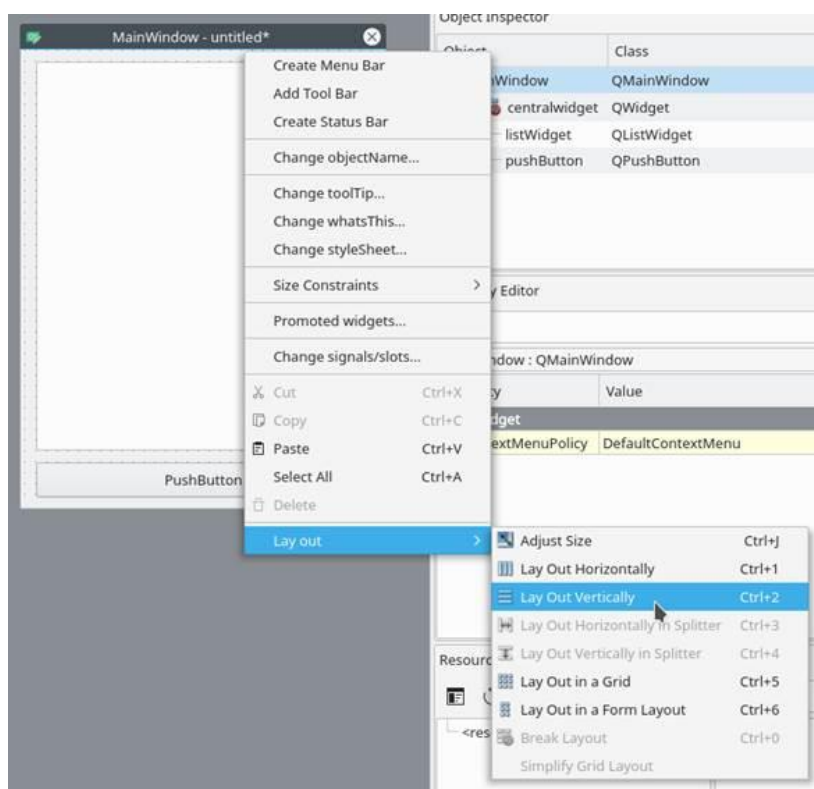


Endi Qt Designer menyusida *Formni* bosing, so'ng *Preview-ni* tanlang va yuqoridagi skrinshotga o'xshash narsani ko'rasiz. Yaxshi ko'rinadi, shunday emasmi? Ammo oyna o'lchamini o'zgartirganda nima sodir bo'ladi:



Asosiy oynaning o'lchami o'zgargan va tugma deyarli ko'rinmas bo'lsada, bizning obyektlarimiz bir xil joylarda qoldi va o'z o'lchamlarini saqlab qoldi. Shuning uchun ko'p hollarda maketlardan foydalanishga arziydi. Albatta, masalan, obyekt uchun qattiq yoki minimal/maksimal kenglik kerak bo'lgan paytlar mavjud. Ammo umuman olganda, dasturni ishlab chiqishda maketlardan foydalanish yaxshiroqdir.

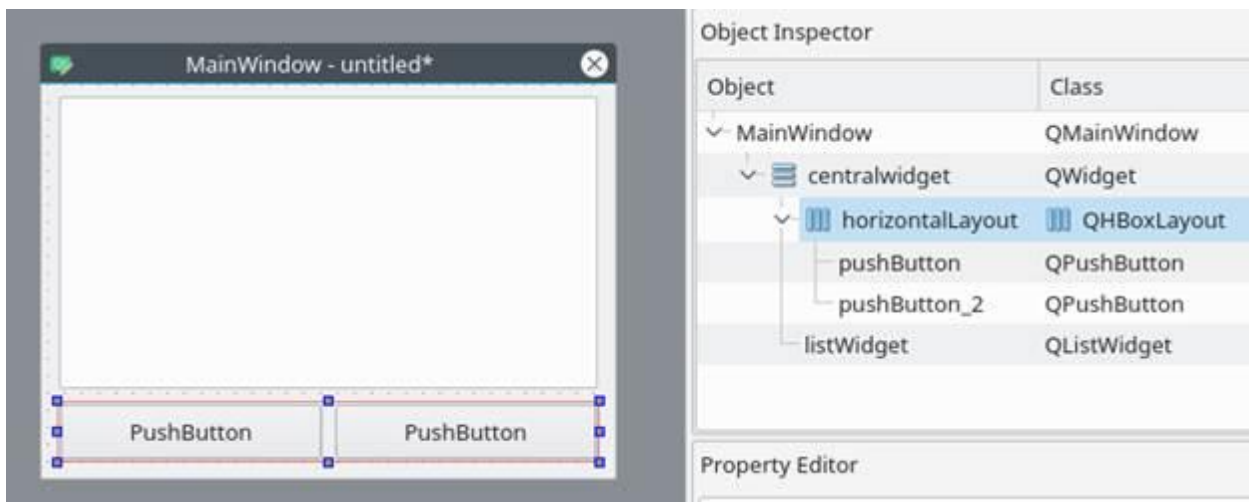
Asosiy oyna allaqachon maketlarni qo'llab-quvvatlaydi, shuning uchun formamizga hech narsa qo'shishimiz shart emas. *Obyekt* inspektoridagi **Main Window** ning o'ng tugmasini bosing va **Lay Out** → **Lay out vertically** ni tanlang. Shuningdek, siz formadagi bo'sh joyni sichqonchaning o'ng tugmasi bilan bosishingiz va bir xil variantlarni tanlashingiz mumkin:



3. Matn muharririr ekrani dizaynini shakllantirish

Sizning elementlaringiz o'zgarishlar kiritilishidan oldingi tartibda bo'lishi kerak, ammo agar ular bo'lmasa, ularni kerakli joyga sudrab olib boring.

Biz vertikal joylashtirishdan foydalanganimiz sababli, biz qo'shgan barcha elementlar vertikal ravishda joylashtiriladi. Istalgan natijani olish uchun joylashtirishlarni birlashtira olasiz. Masalan, ikkita tugmani gorizontal ravishda vertikalga qo'yish quyidagicha ko'rinadi:



Agar siz asosiy oynada elementni ko'chira olmasangiz, buni *Object Inspector* oynasida qilishingiz mumkin .

Yakuniy urinishlar

Endi vertikal joylashtirish tufayli bizning elementlarimiz to'g'ri hizalanadi. Qolgan yagona narsa (lekin shart emas) elementlarning nomini va ularning matnini o'zgartirishdir.

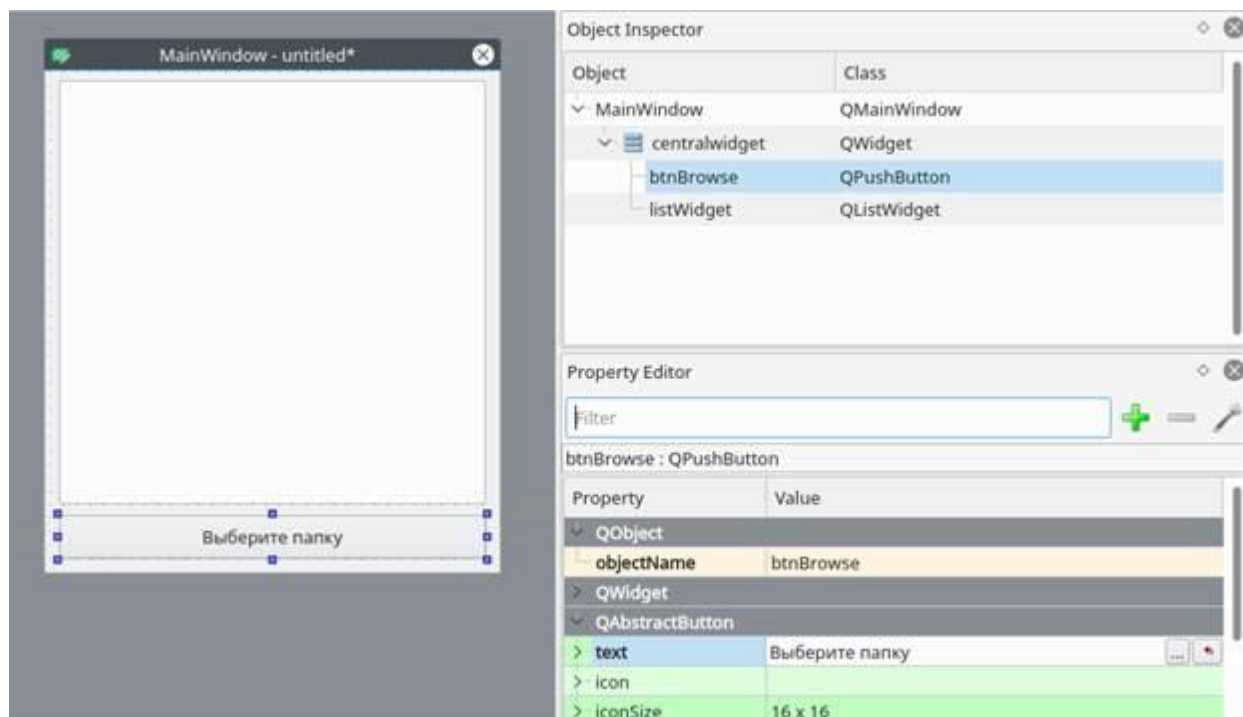
Ro'yxat va tugma bo'lgan bu kabi oddiy dasturda nomlarni o'zgartirish shart emas, chunki baribir undan foydalanish oson. Biroq, elementlarning to'g'ri nomlanishi siz boshidanoq o'rganishingiz kerak bo'lgan narsadir.

Element xususiyatlari **Property Editor** bo'limida o'zgartirilishi mumkin.

Ko'rsatma: Ish jarayonini tezlashtirish uchun siz Qt Designer interfeysiga tez-tez ishlatiladigan elementlarning o'lchamini o'zgartirishingiz, ko'chirishingiz yoki qo'shishingiz mumkin. View menyusi elementi orqali interfeysning yashirin/yopiq (скрытые/закрытые) qismlarini qo'shishingiz mumkin.

Shaklga qo'shgan tugmani bosing. Endi *Property Editor* da siz ushbu elementning barcha xususiyatlarini ko'rishingiz kerak. Biz hozirda QAbstractButton bo'limidagi objectName va matnga qiziqamiz. Bo'lim nomini bosish orqali *Property Editor* da bo'limlarni yopishingiz mumkin.

ObjectName qiymatini *btnBrowse* ga o'zgartiring va matnni *Jildni* tanlang.
Bu shunday bo'lishi kerak:



Nazorat savollari:

1. Pythonda gui bilan ishlash uchun qanday kutubxonalari mavjud?
2. PyQt5 paketini qanday o'rnatiladi va chaqirib ishlatish qanday amalga oshiriladi?
3. pyuic5 buyrug'i nima uchun foydalaniladi?
4. Papkalarni ochish uchun qanday kutubxonadan foydalaniladi?
5. Kataloglar ustida qanday amallar bajarish mumkin?
6. Faylni saqlash uchun qanday amal bajariladi?

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

8-mashg'ulot. Matn muharriri dasturini yozish.

O'quv savollari:

1. PyQt5 da matn muharriri dizaynidagi elementlarning funksiyalari uchun dastur yozish.
2. Dasturni testlash.

PyQt5 da matn muharriri dizaynidagi elementlar funksiyalari uchun dastur yozish

Ro'yxat obyekt nomi **listWidget** bo'lishi kerak — bu to'g'ri. Dizaynni loyiha papkasida **design.ui** nomi bilan saqlang.

Dizaynni kodga aylantirish

.ui fayllarini Python kodida to'g'ridan-to'g'ri ishlatish mumkin, ammo yanada qulay usul ham mavjud — ya'ni .ui faylini Python fayliga aylantirish. Buning uchun terminal yoki buyruqlar satrida **pyuic5** buyrug'idan foydalanamiz.

Masalan, design.ui faylini design.py nomli Python fayliga o'tkazish uchun quyidagi buyruqni kiriting:

```
$ pyuic5 path/to/design.ui -o output/path/to/design.py
```

Kod yozishni boshlash

Endi bizda **QtDesigner** dasturida yaratilgan dizaynning Python faylga aylantirilgan versiyasi — design.py mavjud. Endi biz uning mantiqiy qismini, ya'ni asosiy dastur qismini yozishni boshlaymiz.

Birinchi navbatda, design.py joylashgan papkada **main.py** faylini yarating.

QtDesigner dizaynidan foydalanish

Python GUI ilovasi uchun quyidagi modullar kerak bo'ladi:

```
import sys
from PyQt5 import QtWidgets
```

Shuningdek, avval yaratilgan dizayn faylini ham import qilamiz:

```
import design # Bu bizning o'zgartirilgan dizayn faylimiz
```

Dizayn fayli har safar o'zgartirilganda qayta yaratiladi, shuning uchun uni tahrirlamaymiz. Buning o'rniga yangi sinf yaratamiz — **ExampleApp**, u design.py dagi barcha funksiyalar bilan birlashtiriladi:

```
class ExampleApp(QtWidgets.QMainWindow, design.Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self)
```

Bu sinfda biz interfeys elementlari bilan o‘zaro ishlaymiz, signal-slot (ulanish) funksiyalarini qo‘shamiz va dastur mantiqini yozamiz. Endi esa bu sinfni ishga tushirish uchun **main()** funksiyasini yozamiz:

```
def main():
    app = QtWidgets.QApplication(sys.argv)
    window = ExampleApp()
    window.show()
    app.exec_()
```

Va uni quyidagicha chaqiramiz:

```
if __name__ == '__main__':
    main()
```

Natijada main.py quyidagicha ko‘rinadi:

```
import sys
from PyQt5 import QtWidgets
import design

class ExampleApp(QtWidgets.QMainWindow, design.Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self)

    def main():
        app = QtWidgets.QApplication(sys.argv)
        window = ExampleApp()
        window.show()
        app.exec_()

if __name__ == '__main__':
    main()
```

Agar ushbu kodni python3 main.py orqali ishga tushirsangiz — ilova oynasi ochiladi. Ammo hozircha tugmalar hech qanday amal bajarmaydi, shuning uchun funkSIONallik qo‘shamiz.

Ilovaga funkSIONallik qo‘shish

Eslatma: Quyidagi barcha kodlar ExampleApp sinfi ichida yoziladi.

Avvalo “papkani tanlash” tugmasidan boshlaymiz. Tugma bosilganda funkSIYANI ishga tushirish uchun **clicked.connect()** metodidan foydalanamiz:

```
self.btnBrowse.clicked.connect(self.browse_folder)
```

Bu qatorni __init__ metodiga joylashtiramiz.

Tushuntirish:

- self.btnBrowse — Qt Designer’da aniqlangan tugma nomi.
- clicked — tugma bosilganda sodir bo‘ladigan hodisa (signal).
- connect() — hodisani funkSIYAGA bog‘laydigan metod.
- self.browse_folder — biz yozadigan metod nomi.

Papka tanlash oynasi

Papka tanlash oynasini chaqirish uchun quyidagidan foydalanamiz:

```
directory = QtWidgets.QFileDialog.getExistingDirectory(self, "Papkani tanlash")
```

Agar foydalanuvchi katalogni tanlasa, directory o'zgaruvchisiga tanlangan papkaning manzili yoziladi; agar oynani yopsa — qiymat None bo'ladi. Shuning uchun biz if directory: bilan shart qo'yamiz.

Papka tarkibini olish uchun os moduli kerak bo'ladi:

```
import os
```

Papka ichidagi fayllarni olish uchun:

```
os.listdir(path)
```

listWidget ga element qo'shish uchun addItem() metodidan, hammasini tozalash uchun clear() metodidan foydalanamiz.

Natijada browse_folder funksiyasi quyidagicha bo'ladi:

```
def browse_folder(self):
    self.listWidget.clear()
    directory = QtWidgets.QFileDialog.getExistingDirectory(self, "Papkani tanlash")

    if directory:
        for file_name in os.listdir(directory):
            self.listWidget.addItem(file_name)
```

Yakuniy kod

Quyida Python GUI ilovasining to'liq kodi keltirilgan:

```
import sys
import os
from PyQt5 import QtWidgets
import design

class ExampleApp(QtWidgets.QMainWindow, design.Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self)
        self.btnBrowse.clicked.connect(self.browse_folder)

    def browse_folder(self):
        self.listWidget.clear()
        directory = QtWidgets.QFileDialog.getExistingDirectory(self, "Papkani tanlash")

        if directory:
            for file_name in os.listdir(directory):
                self.listWidget.addItem(file_name)

def main():
    app = QtWidgets.QApplication(sys.argv)
    window = ExampleApp()
```

```
window.show()
app.exec_()

if __name__ == '__main__':
    main()
```

Bu PyQt5 va Qt Designer yordamida Python GUI ilovasini ishlab chiqishning asosiy bosqichlari edi. Endi siz dizaynni o'z g'oyalaringizga moslashtirishingiz va pyuic5 yordamida .ui fayllarini Python kodiga oson aylantirishni bilasiz.

Nazorat savollari:

1. Pythonda gui bilan ishlash uchun qanday kutubxonalari mavjud?
2. PyQt5 paketini qanday o'rnatiladi va chaqirib ishlatish qanday amalga oshiriladi?
3. Pyuic5 buyrug'i nima uchun foydalaniladi?
4. Papkalarni ochish uchun qanday kutubxonadan foydalaniladi?
5. Kataloglar ustida qanday amallar bajarish mumkin?
6. Faylni saqlash uchun qanday amal bajariladi?

MA'RUZA MASHG'ULOT UCHUN O'QUV MATERIALLARI

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

1-mashg'ulot. Tarmoqda ma'lumot almashuvchi klient-server dasturini tuzish.

O'quv savollari:

1. Socket modulining asosiy metodlari bilan tanishish.
2. TCP protokoli asosida klient-server ulanishini tashkil qilish.
3. Ma'lumotlarni uzatish va qabul qilish jarayonini amalga bajarish.
4. Klient va server dasturlarida xatoliklarni aniqlash va bartaraf etish.

1. Socket modulining asosiy metodlari bilan tanishish

Python tarmoq xizmatlariga kirishning ikki darajasini ta'minlaydi. Past darajada, siz asosiy operatsion tizimdagi asosiy rozetka yordamiga kirishingiz mumkin, bu sizga ulanishga yo'naltirilgan va ulanishsiz protokollar uchun mijozlar va serverlarni amalga oshirish imkonini beradi.

Python, shuningdek, FTP, HTTP va boshqalar kabi ma'lum amaliy qatlam tarmoq protokollariga yuqori darajadagi kirishni ta'minlaydigan kutubxonalarga ega.

Ushbu bo'lim sizga Internetdagi eng mashhur kontsepsiya - Socket Programming haqida tushuncha beradi.

Soketlar nima?

Soketlar ikki tomonlama aloqa kanalining so'nggi nuqtalari hisoblanadi. Soketlar jarayon ichida, bitta mashinadagi jarayonlar o'rtasida yoki turli qit'alardagi jarayonlar o'rtasida aloqa qilishi mumkin.

Soketlar bir nechta turli turdagi quvurlar uchun amalga oshirilishi mumkin: Unix domen soketlari, TCP, UDP va boshqalar. *Soket* kutubxonasi umumiy transportlarni boshqarish uchun maxsus sinflarni, shuningdek, qolganlarini boshqarish uchun umumiy interfeysni taqdim etadi.

Soketlarning o'z lug'ati bor.

t/r	Muddati va tavsifi
1	Domen Transport mexanizmi sifatida foydalaniladigan protokollar oilasi. Bu qiymatlar AF_INET, PF_INET, PF_UNIX, PF_X25 va boshqalar kabi konstantalardir.
2	turi Ikki so'nggi nuqta o'rtasidagi aloqa turi, odatda ulanishga yo'naltirilgan protokollar uchun SOCK_STREAM va ulanishsiz protokollar uchun SOCK_DGRAM.
3	protokol

	Odatda nolga teng, bu domen va turdagi protokol variantini aniqlash uchun ishlatilishi mumkin.
4	xost nomi Tarmoq interfeysi identifikatori - • Xost nomi, to'rt nuqtali manzil yoki ikki nuqta (va ehtimol nuqta) belgisidagi IPV6 manzili bo'lishi mumkin bo'lgan qator. • INADDR_BROADCAST manzilini bildiruvchi "<broadcast>" qatori. • INADDR_ANY ni belgilaydigan nol uzunlikdagi qator yoki • Xost bayt tartibida ikkilik manzil sifatida talqin qilingan butun son.
5	port Har bir server bir yoki bir nechta portga qo'ng'iroq qilayotgan mijozlarni tinglaydi. Port Fixnum port raqami, port raqami qatori yoki xizmat nomi bo'lishi mumkin.

Domen

Transport mexanizmi sifatida foydalaniladigan protokollar oilasi. Bu qiymatlar AF_INET, PF_INET, PF_UNIX, PF_X25 va boshqalar kabi konstantalardir.

type

Ikki so'nggi nuqta o'rtasidagi aloqa turi, odatda ulanishga yo'naltirilgan protokollar uchun SOCK_STREAM va ulanishsiz protokollar uchun SOCK_DGRAM.

protokol

Odatda nolga teng, bu domen va turdagi protokol variantini aniqlash uchun ishlatilishi mumkin.

xost nomi

Tarmoq interfeysi identifikatori -

Xost nomi, to'rt nuqtali manzil yoki ikki nuqta (va ehtimol nuqta) belgisidagi IPV6 manzili bo'lishi mumkin bo'lgan qator.

INADDR_BROADCAST manzilini bildiruvchi "<broadcast>" qatori.

INADDR_ANY ni belgilaydigan nol uzunlikdagi qator yoki

Xost bayt tartibida ikkilik manzil sifatida talqin qilingan butun son.

port

Har bir server bir yoki bir nechta portga qo'ng'iroq qilayotgan mijozlarni tinglaydi. Port Fixnum port raqami, port raqami qatori yoki xizmat nomi bo'lishi mumkin.

socket moduli

Soket yaratish uchun umumiy sintaksisga ega *socket* modulida mavjud *socket.socket()* funksiyasidan foydalanishingiz kerak.

```
s = socket.socket(socket_family, socket_type, protocol=0)
```

Bu erda variantlarning tavsifi:

- **socket_family** AF_UNIX yoki AF_INET, avvalroq tushuntirilganidek.
- **socket_type** - SOCK_STREAM yoki SOCK_DGRAM.
- **protokol** - odatda ko'rsatilmaydi, standart 0.

socket_family AF_UNIX yoki AF_INET, avvalroq tushuntirilganidek.

socket_type - SOCK_STREAM yoki SOCK_DGRAM.

protokol - odatda ko'rsatilmaydi, standart 0.

Soket obyektiga ega bo'lganingizdan so'ng, mijoz yoki server dasturini yaratish uchun kerakli funksiyalardan foydalanishingiz mumkin. Quyida talab qilinadigan xususiyatlar ro'yxati -

Server soket usullari

t/r	Usul va tavsif
1	s.bind() Ushbu usul manzilni (xost nomi, port raqamlari juftligi) rozetkaga bog'laydi.
2	s.tinglash() Ushbu usul TCP tinglovchisini o'rnatadi va ishga tushiradi.
3	s.accept() Bu passiv ravishda TCP mijoz ulanishini qabul qiladi, ulanish o'rnatilguncha kutadi (blokirovka).

s.bind()

Ushbu usul manzilni (xost nomi, port raqamlari juftligi) rozetkaga bog'laydi.

s.tinglash()

Ushbu usul TCP tinglovchisini o'rnatadi va ishga tushiradi.

s.accept()

Bu passiv ravishda TCP mijoz ulanishini qabul qiladi, ulanish o'rnatilguncha kutadi (blokirovka).

Mijoz soket usullari

t/r	Usul va tavsif
1	s.connect() Ushbu usul TCP serveriga ulanishni faol ravishda boshlaydi.

s.connect()

Ushbu usul TCP serveriga ulanishni faol ravishda boshlaydi.

Umumiy soket usullari

t/r	Usul va tavsif
1	s.recv() Ushbu usul TCP xabarini oladi
2	s.send() Ushbu usul TCP xabarini yuboradi
3	s.recvfrom() Bu usul UDP xabarini oladi
4	s.sendto() Bu usul UDP xabarini yuboradi
5	s.close() Ushbu usul rozetkani yopadi
6	socket.getostname() Xost nomini qaytaradi.

s.recv()

Ushbu usul TCP xabarini oladi

s.send()

Ushbu usul TCP xabarini yuboradi

s.recvfrom()

Bu usul UDP xabarini oladi

s.sendto()

Bu usul UDP xabarini yuboradi

s.close()

Ushbu usul rozetkani yopadi

socket.getostname()

Xost nomini qaytaradi.

oddiy server. Internet-serverlarni yozish uchun biz rozetka obyektini yaratish uchun rozetka modulida mavjud **soket funksiyasidan foydalanamiz**. Soket obyekti soket serverini sozlash uchun boshqa funksiyalarni chaqirish uchun ishlatiladi.

Endi berilgan hostda xizmatigiz uchun *portni* belgilash uchun **bog'lash (hostname, port) funksiyasini chaqiring**.

Keyin qaytarilgan obyekt *qabul qilish usulini chaqiring*. Ushbu usul mijoz siz ko'rsatgan portga ulanishini kutadi va keyin ushbu mijozga ulanishni ifodalovchi *Connection obyekti*ni qaytaradi.

```
#!/usr/bin/python          # This is server.py file

import socket              # Import socket module

s = socket.socket()        # Create a socket object
host = socket.gethostname() # Get local machine name
port = 12345               # Reserve a port for your service.
s.bind((host, port))       # Bind to the port

s.listen(5)                # Now wait for client connection.
while True:
    c, addr = s.accept()    # Establish connection with client.
    print 'Got connection from', addr
    c.send('Thank you for connecting')
    c.close()               # Close the connection
```

Oddiy mijoz. Berilgan port 12345 va berilgan xostga ulanishni ochadigan juda oddiy mijoz dasturini yozamiz. *Python soket* moduli funksiyasidan foydalangan holda soket mijozini yaratish juda oddiy .

Socket.connect(hostname, port) portdagi xost *nomi bilan* TCP ulanishini ochadi . Ochiq rozetkaga ega bo'lganingizdan so'ng, siz undan istalgan kiritish-chiqarish obyekti kabi o'qishingiz mumkin. Ishingiz tugagach, faylni yopganingizdek, uni yopishni unutmang.

Quyidagi kod juda oddiy mijoz bo'lib, u berilgan xost va portga ulanadi, rozetkadan barcha mavjud ma'lumotlarni o'qiydi va keyin chiqadi.

```
#!/usr/bin/python          # This is client.py file

import socket              # Import socket module

s = socket.socket()        # Create a socket object
host = socket.gethostname() # Get local machine name
port = 12345               # Reserve a port for your service.

s.connect((host, port))
print s.recv(1024)
s.close()                  # Close the socket when done
```

Endi bu server.py ni fonda ishga tushiring va natijani ko'rish uchun yuqoridagi client.py ni ishga tushiring.

```
# Following would start a server in background.
$ python server.py &

# Once server is started run client as follows:
$ python client.py
```

Bu quyidagi natijani beradi:

```
Got connection from ('127.0.0.1', 48437)
Thank you for connecting
```

Python Internet modullari

Python Network/Internet dasturlashdagi ba'zi muhim modullar ro'yxati.

protokol	Umumiy funktsiya	Port raqami.	Python moduli
HTTP	veb-sahifalar	80	httplib, urllib, xmlrpclib
NNTP	Usenet yangiliklari	119	nntplib
FTP	Fayl uzatish	20	ftplib, urllib
SMTP	Email yuborish	25	smtplib
POP3	Email qabul qilinmoqda	110	poplib
IMAP4	Email qabul qilinmoqda	143	imaplib
telnet	Buyruqlar qatorlari	23	telnetlib
gofer	Hujjatlarni topshirish	70	gopher, urllib

Iltimos, FTP, SMTP, POP va IMAP protokollari bilan ishlash uchun yuqorida ko'rsatilgan barcha kutubxonalarni tekshiring.

2) TCP protokoli asosida klient–server ulanishini tashkil qilish

TCP nima? U ishonchli, oqimli (stream) transport protokoli. Paketlar yo'qolsa qayta jo'natadi, tartibni kafolatlaydi, lekin xabar chegaralarini saqlamaydi (stream — pastda bu muhim bo'ladi).

Asosiy rollar:

- **Server:** socket() → bind() → listen() → accept() ketma-ketligi bilan portga “quloq soladi”.
- **Klient:** socket() → connect() bilan serverga ulanadi.

• **3-bosqichli qo'l siqish** (SYN → SYN-ACK → ACK) natijasida TCP sessiya ochiladi.

Tavsiyalar:

- Test uchun HOST='127.0.0.1', PORT=5000 kabi lokal portdan foydalaning.
- Qayta ishga tushirishda "Address already in use" bo'lmasligi uchun SO_REUSEADDR ni yoqing.
- Uzun sessiyalar uchun SO_KEEPALIVE ni yoqish foydali.
- Kechikishni kamaytirish kerak bo'lsa TCP_NODELAY (Nagle off) ni sozlash mumkin.

Minimal server (multi-client, thread bilan):

```
# server.py
import socket, threading, struct

HOST, PORT = '127.0.0.1', 5000

def recv_exact(sock, n):
    buf = b''
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionResetError("Peer closed")
        buf += chunk
    return buf

def recv_msg(sock):
    hdr = recv_exact(sock, 4) # 4 baytli uzunlik (network
byte order)
    (ln,) = struct.unpack('!I', hdr)
    return recv_exact(sock, ln)

def send_msg(sock, data: bytes):
    sock.sendall(struct.pack('!I', len(data)) + data)

def handle_client(conn, addr):
    conn.settimeout(20) # himoya: muzlab qolmasin
    try:
        while True:
            data = recv_msg(conn) # xabarni o'qiyviz
            if not data:
                break
            # Bu yerda biznes-mantiq: masalan, ECHO (katta harfga o'girish)
            reply = data.upper()
            send_msg(conn, reply)
    except (socket.timeout, ConnectionResetError):
        pass
    finally:
        conn.shutdown(socket.SHUT_RDWR)
```

```

        conn.close()

def main():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
        s.bind((HOST, PORT))
        s.listen(100)
        print(f"Listening on {HOST}:{PORT}")
        while True:
            conn, addr = s.accept()
            threading.Thread(target=handle_client, args=(conn, addr),
daemon=True).start()

if __name__ == "__main__":
    main()

```

Minimal klient:

```

# client.py
import socket, struct

HOST, PORT = '127.0.0.1', 5000

def send_msg(sock, data: bytes):
    sock.sendall(struct.pack('!I', len(data)) + data)

def recv_exact(sock, n):
    buf = b''
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionResetError("Peer closed")
        buf += chunk
    return buf

def recv_msg(sock):
    ln, = struct.unpack('!I', recv_exact(sock, 4))
    return recv_exact(sock, ln)

with socket.create_connection((HOST, PORT), timeout=10) as s:
    for text in ["salom", "tcp sinovi", "xayr"]:
        send_msg(s, text.encode('utf-8'))
        print("Server:", recv_msg(s).decode('utf-8'))

```

Ishga tushirish:

1. python server.py
2. boshqa terminalda python client.py

3) Ma'lumotlarni uzatish va qabul qilishni amalda bajarish

TCP **stream** bo'lgani uchun "bir send = bir recv" **emas**. Xabar chegaralarini o'zingiz belgilashingiz kerak. Amaliy usullar:

1. **Uzunlik-prefiks** (yuqoridagi kod): xabar boshiga 4 bayt uzunlik (!l – network byte order) qo'shiladi. Eng ko'p qo'llanadigan, sodda va ishonchli usul.

2. **Delimiter** (masalan, ¥n bilan tugaydigan satrlar): loglar, buyruqlar uchun qulay, lekin kontent ichida delimiter bo'lmashligini kafolatlash kerak.

3. **Protobuf/JSON**: JSON jo'natsangiz ham yuqoridagidek uzunlik bilan jo'natish yaxshi amaliyot (aks holda recv bo'linib ketishi mumkin).

4. **Katta fayllar**: fayl hajmini oldindan jo'nating, so'ngra recv_exact bilan bo'lak-bo'lak qabul qiling; yoki sendfile() / memoryview dan foydalanib samaradorlikni oshiring.

Misol – fayl yuborish (klientdan serverga):

- **Klient:** `header = struct.pack('!l', len(filename_bytes)) + filename_bytes + struct.pack('!Q', file_size)`
- **Server:** avval sarlavhani o'qiydi, so'ng file_size baytni siklda qabul qilib faylga yozadi.

Koddagi muhim punktlar:

- **sendall** ishlating — u butun bufer jo'natilmaguncha qaytmaydi.
- **recv** ni aynan kerakli uzunlikka yetguncha takrorlang (recv_exact).
- **Kodlash:** matnlar uchun utf-8 (yuborishda .encode(), qabulda .decode()).
- **Timeout** qo'ying (settimeout) — "osilib qolish"dan saqlaydi.
- **Shutdown/close** tartibi: avval shutdown(SHUT_RDWR), so'ng close().

4) Klient va server dasturlarida xatoliklarni aniqlash va bartaraf etish

Keng tarqalgan xatoliklar va yechimlar

Muammo / xato	Sabab	Aniqlash	Yechim
ConnectionRefusedError	Server ishlamayapti yoki port bloklangan	Klient ulana olmaydi	Serverni ishga tushiring; IP/port to'g'riligini tekshiring; firewall'ni ruxsat qiling
Address already in use	Port band (oldingi instansiya to'liq yopilgan)	bind() da xato	SO_REUSEADDR; eski jarayonni yoping; portni o'zgartiring
ConnectionResetError / BrokenPipeError	Ikkinchi tomon ulanishni yopdi	O'qish/yozishda istisno	Protokol bo'yicha to'g'ri yopish; qayta urinish (retry) strategiyasi
socket.timeout	Tarmoq sust yoki peer javob bermadi	recv/connect kutishi tugadi	settimeout qiymatini oshiring; qayta urinish; vaqtinchalik xatoni loglab, foydalanuvchiga xabar bering

“Truncated/garbled message”	Framing yo‘q (stream bo‘linib ketdi)	Noto‘liq xabar	Uzunlik-prefiks yoki delimiter bilan framingni joriy qiling
Kodlash xatolari	utf-8/cp1251 aralash	.decode() da xato	Standartni bir xil qiling; binar bilan ishlaganda hech qachon avtomatik .decode() qilmang
Sekin yoki notekis ishlash	Nagle, kichik bo‘laklar, locklar	Kechikish katta	Paketlashni birlashtiring; kerak bo‘lsa TCP_NODELAY; oqimni buferlang

Diagnostika va kuzatuv

- **Loglar:** har xabar uchun `client_id`, uzunlik, vaqt, xatolik kodlarini yozing.
- **Net vositalar:**
 - `netstat -an | find "5000"` (Windows) yoki `ss -tlnp/lsof -i :5000` (Linux) — port holati.
 - `telnet 127.0.0.1 5000` yoki `nc 127.0.0.1 5000` — ochiqligini tekshirish.
 - **Wireshark/tcpdump** — paket darajasida tahlil (SYN/SYN-ACK/ACK, RST, retransmit).
- **Sog‘lomlik signallari:** serverga **heartbeat/ping** RPC qo‘shing.
- **Backpressure:** server band bo‘lsa, mijozga “busy” javobi qaytaring yoki navbatga qo‘ying.

Himoya va barqarorlik

- **Kirishni cheklash:** IP-filtrlash, autentifikatsiya (token).
- **Resurslar:** maksimal ulanishlar, bufer o‘lchamlari, fayl hajmi limitlari.
- **TLS/SSL:** agar internet orqali ishlasa, ssl moduli bilan socketni o‘rab shifrlang.
- **Arxitektura:** ko‘p mijozli yukda `asyncio` (event loop) yoki `worker-pool` tanlang.

Istisnolarni to‘liq qo‘lga olish (shablon):

```
try:
    # tarmoq amali
except (ConnectionResetError, BrokenPipeError) as e:
    logger.warning("Peer disconnected: %s", e)
    # retry yoki sessiyani yumshoq yopish
except socket.timeout:
    logger.warning("Timeout; retry...")
except OSError as e:
    logger.exception("OS error: %s", e)
finally:
    try:
        sock.shutdown(socket.SHUT_RDWR)
    except OSError:
        pass
    sock.close()
```

Qisqa chek-ro‘yxat (cheat-sheet)

- ☐ **Server:** bind → listen → accept; **Klient:** connect.
- ☐ SO_REUSEADDR, SO_KEEPALIVE, kerak bo'lsa TCP_NODELAY.
- ☐ **Framing:** 4 baytli uzunlik prefiksi (struct.pack('!l', len)), **sendall/recv_exact**.
- ☐ utf-8 kodlash; fayllar uchun binar oqim.
- ☐ settimeout; istisnolarni tutish; loglash.
- ☐ **Yopish:** shutdown → close.
- ☐ **Test:** telnet/nc, netstat/ss, Wireshark.

Nazorat savollari:

1. Tarmoq dasturlash deganda nimani tushunasiz?
2. Tarmoqda ishlash uchun python tilining qaysi paketi bilan ishlanadi?
3. Qanday metodlarni bilasiz?
4. Bind() metodining vazifasi qanday?
5. connect metodining vazifasi qanday?
6. Protokol nima?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

2-mashg'ulot. Socket moduli bilan ishlash.

O'quv savollari:

1.Socket modulining asosiy metodlari bilan tanishish.

2.socket(), .bind, .listen, .accept(), .connect(), .send(), recv(), .close() metodlari bilan ishlash.

1. Socket modulining asosiy metodlari bilan tanishish

Soket - bu jarayonlar o'rtasida ma'lumot almashinuvini ta'minlash uchun dasturlash interfeysi.

Socket turlari:

- Server - xabarlarni qabul qiluvchi socket.
- Mijoz - xabarlarni yuboradigan socket.

Socketlar protokollarning transport qatlamida ishlaydi va shunga mos ravishda 2 tur mavjud:

- **Oqim** (TCP asosida, kodda ko'rsatilgan **SOCK_STREAM**) - TCP protokoli asosida o'rnatilgan ulanishga ega socketlar, ikki tomonlama bo'lishi mumkin bo'lgan baytlar oqimini uzatadi - ya'ni Ilova ma'lumotlarni qabul qilishi va yuborishi mumkin.

- **Datagram** (UDP asosida, kodda ko'rsatilgan **SOCK_DGRAM**) - ular o'rtasida aniq ulanishni talab qilmaydigan socketlar. Xabar belgilangan socketga yuboriladi va shunga mos ravishda belgilangan socketdan qabul qilinishi mumkin.

Soket tarkibiasllida IP manzil va portdan iborat bo'ladi.

IP manzil - IP protokoli yordamida qurilgan kompyuter tarmog'idagi tugunning yagona tarmoq manzili. IPv4 protokoli versiyasida IP manzili uzunligi 4 bayt (masalan, 192.168.0.3), IPv6 protokoli versiyasida esa IP manzili 16 bayt (masalan, 2001:0db8:85a3:0000:0000: 8a2e: 0370: 7334). IP-manzil noyob bo'lishi kerak.

Port - transport qatlami protokollari (TCP, UDP va boshqalar) sarlavhalarida yozilgan natural son. Port bir xil xost ichidagi paketni qabul qilish jarayonini aniqlash uchun ishlatiladi.

Python socketlar bilan ishlash uchun o'rnatilgan `socket` kutubxonasidan foydalanadi. Modulning asosiy funksiyalaridan biri bu `socket()` ulanish bilan ishlash uchun tegishli funksiyalarga ega bo'lgan socket tipidagi obyektini qaytaradigan funksiyadir.:

```
class socket.socket
sock = socket.socket()
```

2 .socket(), .bind(), .listen(), .accept(), .connect(), .send(), .recv(), .close() metodlari bilan ishlash

- **socket.bind(address)** - Socketsni manzilga bog'laydi (IP manzil va portni ishga tushiradi). Bundan oldin socket ulanmagan bo'lishi kerak.
- **socket.listen([backlog])** - Ulanishlarni qabul qilish uchun serverni almashtiradi. "backlog (int)" parametri server qabul qiladigan ulanishlar sonidir.
- **socket.accept()** - Ulanishni qabul qiladi va mijozdan xabar kutayotganda dasturni bloklaydi. Natijada kortejni qaytaradi:
 - **conn**: ma'lumotlarni yuborish/qabul qilish uchun ishlatilishi mumkin bo'lgan ulanish obyekti (socket);
 - **address**: mijozning manzili.
- **socket.recv(bufsize[, flags])** - socketdan ma'lumotlarni ikkilik formatda (baytlar to'plami) o'qiydi va qaytaradi. Parametr - bitta xabardagi baytlarning maksimal soni. **bufsize (int)**
- **socket.send(bytes[, flags])** - Mijozga ma'lumotlarni yuboradi va yuborilgan baytlar sonini qaytaradi. Parametr ikkilik ma'lumotlardir. **bytes (bytes)**
- **socket.close()** - Socketsni yopadi.

Sockets bilan ishlash ko'p jihatdan fayl obyekti bilan ishlashga o'xshaydi. Printsip - ochiq ulanish - ma'lumotlar - yopiq ulanish deb hisoblanadi.

Nazorat savollari:

1. port deb nimaga aytiladi?
2. Socket nima ?
3. Socketning qanday turlari mavjud?
4. Socket obyektining qanday funksiyalari mavjud?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

3-mashg'ulot. Pythonda TCP klient-server dasturini tuzish.

O'quv savollari:

1. Socket modulining asosiy metodlari yordamida client–server dasturini tuzish.
2. TCP ulanishini o'rnatish va xabar almashishni sinash (connect/send/recv).
3. Xatoliklarni qayta ishlash va ulanish holatini boshqarish (timeout/reconnect/close).

1. Socket modulining asosiy metodlari yordamida client-server dasturini tuzish.

Python-da o'rnatilgan socket moduli tarmoq orqali muloqot qilish uchun funktsionallikni ta'minlaydi. Ushbu modul socket sinfi shaklida so'rovlarni yuborish va qabul qilish uchun past darajadagi interfeysda aniqlanadi. Va yaratayotgan narsadan qat'iy nazar - server yoki mijoz dasturi, so'rovlarni yuborish uchun socket obyektini yaratish kerak bo'ladi:

```
import socket
sock = socket.socket()
```

socket bilan ishlashni tugatgandan so'ng, uni `close()` metodi yordamida yopish kerak.

```
import socket
client = socket.socket()
client.close()
```

Socket bilan keyingi harakatlar qanday rozetka yaratilayotganiga bog'liq bo'lib, server yoki mijoz uchun yaratiladi. Bunday holda, Python socketlarida eng oddiy mijozni yaratishni ko'rib chiqiladi.

Serverga bog'lanish. `connect()` metodi serverga ulanish uchun ishlatiladi.

```
socket.connect(address)
```

Ushbu **connect** metodi parametr sifatida server manzilini qabul qiladi. Manzil odatda kortej sifatida ifodalanadi

```
(host, port)
```

Birinchi element string sifatida xost hisoblanadi. Bu, masalan, "127.0.0.1" shaklidagi IP-manzil yoki lokol xost nomi bo'lishi mumkin. Ikkinchi parametr raqamli port raqamidir. Port 0 dan 65535 gacha bo'lgan 2 baytlik qiymatdir. Masalan:

```
import socket
client = socket.socket()
client.connect(("www.kun.uz", 80))
print("Connected...")
client.close()
```

Bu yerda birinchi parametr sifatida: "www.kun.uz" manziliga va ikkinchi parametr sifatida 80-portga ulanishga harakat qilmoqda. Ya'ni, `www.kun.uz80-port`da ishlaydigan oddiy veb-saytga ulanishga harakat qilinmoqda (odatda veb-server 80-

portda ishlaydi). Ulangandan so'ng, shunchaki konsolga satr chiqariladi va socketni yopiladi.

Server bilan o'zaro ulanish.

Ulanish o'rnatilgandan so'ng serverga ma'lumotlarni yuborish va undan qabul qilish mumkin. Ma'lumotlarni yuborish uchun **socket.send()** usuli qo'llaniladi, u parametr sifatida yuboriladigan ma'lumotlar to'plamini oladi.

```
socket.send(bytes)
```

Soketdan ma'lumotlarni qabul qilish uchun **socket.recv()** usuli qo'llaniladi.

```
bytes = socket.recv(bufsize[, flags])
```

Bu zarur parametr sifatida bir vaqtning o'zida boshqa socketdan olinishi mumkin bo'lgan baytlardagi maksimal bufer hajmini oladi. Bufer hajmini 2 ga karrali qilish yaxshidir, masalan, 512, 1024 va boshqalar. Qaytish qiymati boshqa socketdan olingan baytlar to'plamidir.

Masalan, "www.kun.uz" serveriga ma'lumotlarni yuboriladi va undan qaytgan javobni qabul qilinadi:

```
import socket
client = socket.socket()
client.connect(("www.kun.uz", 80))
message = "GET / HTTP/1.1\r\nHost:www.kun.uz\r\n Connection: close\r\n\r\n"
print("Connecting...")
client.send(message.encode())
data = client.recv(1024)
print("Server sent: ", data.decode())
client.close()
```

Bunday holda, satr veb-sayt tushunadigan standart HTTP protokoli sarlavhalari bilan yuboriladi. "GET/HTTP/1.1\r\nHost: www.kun.uz HTTP so'rov formati birinchi navbatda so'rov turini, so'ralgan manbaga yo'lni va maxsus protokol versiyasidan iborat so'rov qatorini () o'z ichiga oladi . Ya'ni, bu yerda xabarda GET so'rovi "/" yo'li bo'ylab yuborilganligini bildiriladi (ya'ni www.kun.uz saytining ildiziga). Bu holda HTTP / 1.1 protokoli qo'llaniladi. So'rov qatori ikki karek to'plami va qaytariladigan belgilar qatori \r\n bilan tugashi kerak.

Bundan tashqari, HTTP so'rovida sarlavhalar bo'lishi mumkin. Shunday qilib, bu holda "Xost" sarlavhasini yuboriladi, bu esa xost manziliga ishora qiladi. Bu holda bu "www.kun.uz:80". Va shuningdek, bu holatda, "close" qiymatiga ega bo'lgan "connection" sarlavhasini yuboriladi - bu qiymat serverga ulanishni yopishni buyuradi.

Bu yerda satrlarni emas, faqat baytlarni yuborish mumkinligi sababli, uni yuborilganida satrni baytlar to'plamiga aylantiriladi. Buning uchun **bytes** sinf obyektini qaytaradigan **encode()** usuli qo'llaniladi.

```
client.send(message.encode())
```

Serverga xabar yuborilganidan so'ng, recv() funksiyasidan foydalanib ma'lumotlarni olishga harakat qilinadi. Bunday holda, olingan ma'lumotlar uchun bufer 1024 bayt hajmiga ega bo'ladi. Usulning natijasi olingan ma'lumotlardir. Biroq,

socket ma'lumotlarni baytlar to'plami shaklida oladi (aniqrog'i, obyekt bytes). Ularni satrga aylantirish uchun decode() usulidan foydalaning:

```
data = client.recv(1024) # serverdan ma'lumotlarni olish
print("Server sent: ", data.decode())
```

Va agar ushbu dasturni Python-da ishga tushirilsa, quyidagi kabi javob olish mumkin:

```
Connected...
Server sent: HTTP/1.1 301 Moved Permanently
Connection: close
Content-Length: 0
Server: Varnish
Retry-After: 0
Location: https://www.python.org/
Accept-Ranges: bytes
Date: Tue, 02 May 2023 17:52:32 GMT
Via: 1.1 varnish
X-Served-By: cache-bma1653-BMA
X-Cache: HIT
X-Cache-Hits: 0
X-Timer: S1683049953.637569,VS0,VE0
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

Shuningdek, server bizga http sarlavhalarida veb-sayt <https://www.kun.uz/> manziliga ko'chirilganligini ko'rsatadigan qatorni yuboradi.

Server dasturini yaratish

Python-da serverni, shuningdek mijozni yaratish uchun socket sinfi ham ishlatiladi, ammo bu mijoz socketi bilan ishlashdan biroz farq qiladi.

Serverni ulanish

Server kiruvchi ulanishlarni kutib turadi (listening), ularni qandaydir tarzda qayta ishlaydi va javob yuboradi. Server o'z ishini boshlashi uchun, avvalo, u ulanishlarni tinglaydigan manzilni aniqlab olishi kerak. Buning uchun **bind()** metodi qo'llaniladi.

```
socket.bind(address)
```

Bu usul server ishga tushadigan manzilni qabul qiladi. Odatiy bo'lib, manzil ikki elementdan iborat to'plamdir:

```
(host, port)
```

Birinchi element string sifatida xost hisoblanadi. Bu, masalan, "127.0.0.1" shaklidagi IP-manzil yoki mahalliy xost nomi bo'lishi mumkin. Ikkinchi parametr raqamli port raqamidir. Port 0 dan 65535 gacha bo'lgan 2 baytlik qiymatni ifodalaydi. Bir xil manzilda (bir xil mashinada) bir nechta turli tarmoq ilovalari ishga tushirilishi mumkinligi sababli, port ushbu ilovalarni farqlash imkonini beradi. Masalan:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
```

```
server.bind((hostname, port))
```

Bu yerda server 12345-portda ishlaydi. E'tibor bering, barcha portlar ham bepul bo'lmashligi mumkin. Ammo, qoida tariqasida, band bo'lgan portlar unchalik ko'p emas.

Manzil sifatida joriy xost nomidan foydalanamiz. Uni olish uchun funksiyadan foydalaniladi **socket.gethostname()** (odatda bu joriy kompyuterning nomi).

Ulanishlarni tinglash

Serverni bog'lagandan so'ng ulanishlarni tinglash uchun uni ishga tushirish kerak. Buning uchun **listen()** metodi qo'llaniladi.

```
socket.listen([backlog])
```

Bu backlog parametrini qabul qiladi. backlog – bu socket uchun ruxsat etilgan navbatdagi kiruvchi ulanishlarning maksimal soni. Ya'ni, mijozlar ulanganda, ular navbatda turishadi va server joriy mijozni qayta ishlaguncha kutishadi. Agar navbatda server tomonidan qayta ishlanishini kutayotgan mijozlar soni allaqachon belgilangan bo'lsa, barcha yangi mijozlar rad etiladi. Masalan:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)
```

Mijozni qabul qilish va qayta ishlash

accept() metodi kiruvchi ulanishlarni qabul qilish uchun ishlatiladi. Bu usul 2 ta tuple tipli qiymatni qaytaradi

```
(conn, address)
```

Birinchi element conn - server mijoz bilan aloqa qiladigan boshqa socket obyektini ifodalaydi. Ikkinchi element address - ulangan mijozning manzili hisoblanadi. Shuni ta'kidlash kerakki, mijoz bilan o'zaro aloqalar tugagandan so'ng, conn socketini close() usuli bilan yopish kerak bo'ladi.

Tuplening birinchi elementi - conn yordamida mijozga xabar yuborish yoki aksincha ma'lumotlarni qabul qilish mumkin. Soketdan ma'lumotlarni qabul qilish uchun **socket.recv()** usuli qo'llaniladi.

```
bytes = socket.recv(bufsize)
```

Bu zarur parametr sifatida bir vaqtning o'zida boshqa socketdan olinishi mumkin bo'lgan baytlardagi maksimal bufer hajmini oladi. Qaytish qiymati boshqa socketdan olingan baytlar to'plamidir.

Ma'lumotlarni yuborish uchun **socket.send()** metodi qo'llaniladi, u parametr sifatida yuboriladigan ma'lumotlar to'plamini oladi.

```
socket.send(bytes)
```

Endi bir misolni ko'rib chiqaylik. Server.py faylida serverni quyidagi kod bilan aniqlaylik:


```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)

print("Server starts")

con, addr = server.accept()
print("connection: ", conn)
print("client address: ", addr)

message = "Hello Client!"
con.send(message.encode())
con.close()
print("Server ends")
server.close()
```

Bu yerda server mijozni qabul qiladi, uning ulanishi haqidagi ma'lumotlarni chiqaradi va mijozga "Hello Client!" qatorini qaytarib yuboradi.

Va client.py faylida quyidagi mijoz kodini aniqlanadi:

```
import socket
client = socket.socket()
hostname = socket.gethostname()
port = 12345
client.connect((hostname, port))
data = client.recv(1024)
print("Server sent: ", data.decode())
client.close()
```

Bizning serverimiz va mijozimiz bitta kompyuterda ishlaganligi sababli, **socket.gethostname()** server kodidagi kabi ulanish uchun server manzilini aniqlash uchun funksiya va port 12345 ishlatiladi. Serverga ulangandan so'ng undan ma'lumotlarni olinadi va uni konsolda ko'rsatadi.

Avval server kodini ishga tushiraylik. Va keyin mijoz kodini ishga tushiriladi. Natijada, server ulanishni qabul qiladi, u haqidagi ma'lumotlarni konsolda ko'rsatadi va mijozga xabar yuboradi:

```
c:\python>python server.py
Server starts
connection: <socket.socket fd=432, family=2, type=1, proto=0,
laddr=('192.168.0.102', 12345), raddr=('192.168.0.102', 61824)>
client address: ('192.168.0.102', 61824)
Server ends
```

Xususan, bu yerda server va mijoz 192.168.0.102 da ishlayotganini va mijoz 61824 portidan foydalanayotganini ko'ramiz.

Va mijoz serverdan xabar oladi va uni konsolga chop etadi:

```
c:\python>python client.py
Server sent: Hello Client!
```

Bir nechta mijozlar bilan ishlash

Bunday holda, server bitta mijozga xizmat qiladi va ishlashni to'xtatadi. Agar biz server ko'p mijozlar bilan ishlashni istasak, unda cheksiz sikldan foydalanishga to'g'ri keladi:

```
import socket
from datetime import datetime

server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)

print("Server running")
while True:
    con, addr = server.accept()
    print("client address: ", addr)
    message = datetime.now().strftime("%H:%M:%S")
    con.send(message.encode())
    con.close()
```

Bunday holda, masalan, o'rnatilgan modul `datetime` va `datetime.now().strftime()` funksiyasidan foydalanib, joriy vaqtni satr sifatida olish, keyinchalik uni mijozga yuborish mumkin. Natijada, mijoz so'rov bilan joriy vaqthaqidagi ma'lumotlarni ham oladi.

Ikki tomonlama aloqa

Yuqoridagi misollarda aloqa bir tomonlama edi - server ma'lumotlarni yuboradi va mijoz uni qabul qiladi. Keling, mijoz ham, server ham ma'lumotlarni jo'natgan va qabul qilganda eng oddiy ikki tomonlama aloqani ko'rib chiqaylik. Serverga mijozdan bir nechta satr olishiga ruxsat beraylik, uni o'zgartirib so'ngra mijozga qaytarib yuboraylik:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)

print("Server running")
while True:
    con, _ = server.accept()
    data = con.recv(1024)
    message = data.decode()
    print(f"Client sent: {message}")
    message = message[::-1]
```

```
con.send(message.encode())
con.close()
```

Mijoz konsoldan satr kiritish va uni serverga yuborish uchun kodni aniqlasin:

```
import socket

client = socket.socket()
hostname = socket.gethostname()
port = 12345
client.connect((hostname, port))
message = input("Input a text: ")
client.send(message.encode())
data = client.recv(1024)
print("Server sent: ", data.decode())
client.close()
```

Yuqoridagi ishning natijasi:

Client:

```
c:\python>python client.py
Input a text: hello
Server sent: olleh
c:\python>
```

Server:

```
c:\python>python server.py
Server running
Client sent: hello
```

2) TCP ulanishini o'rnatish va xabar almashishni sinash (connect/send/recv)

Minimal server (echo)

```
# server.py
import socket

HOST, PORT = '127.0.0.1', 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind((HOST, PORT))
    s.listen(1)
    print(f"Listening on {HOST}:{PORT}")
    conn, addr = s.accept()
    with conn:
        print("Client:", addr)
        while True:
            data = conn.recv(4096)          # 0 => peer yopdi
            if not data:
                break
            conn.sendall(data)              # echo
```

Minimal klient

```
# client.py
```

```
import socket

HOST, PORT = '127.0.0.1', 5000
msg = b"salom, tcp!"

with socket.create_connection((HOST, PORT), timeout=5) as s:
    s.sendall(msg)                # barcha baytlar jo'natiladi
    reply = s.recv(4096)          # javobni o'qish
    print("Server javobi:", reply.decode('utf-8'))
```

Test: birinchi terminalda python server.py, ikkinchisida python client.py.

Eslatma: TCP **stream**; bitta send = bitta recv bo'lishi **shart emas**. Barqaror format uchun **framing** kiriting (quyida).

Satrli protokol uchun framing (uzunlik-prefiks)

```
import struct

def send_msg(sock, payload: bytes):
    sock.sendall(struct.pack('!I', len(payload)) + payload)

def recv_exact(sock, n):
    buf = b''
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed")
        buf += chunk
    return buf

def recv_msg(sock) -> bytes:
    (ln,) = struct.unpack('!I', recv_exact(sock, 4))
    return recv_exact(sock, ln)
```

Bu yordamchilarni server/klientga qo'shsangiz, xabarlar bo'linib ketmaydi.

3) Xatoliklarni qayta ishlash va ulanish holatini boshqarish (timeout/reconnect/close)

Mustahkam klient shabloni

```
import socket, time, struct, random

HOST, PORT = '127.0.0.1', 5000
BASE, CAP = 0.5, 8.0 # backoff (sekund)

def send_msg(sock, payload: bytes):
    sock.sendall(struct.pack('!I', len(payload)) + payload)

def recv_exact(sock, n):
    buf = b''
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed")
```

```

        buf += chunk
    return buf

def recv_msg(sock):
    ln, = struct.unpack('!I', recv_exact(sock, 4))
    return recv_exact(sock, ln)

def connect_with_retry():
    delay = BASE
    while True:
        try:
            sock = socket.create_connection((HOST, PORT), timeout=5)
            sock.settimeout(10) # I/O timeout
            return sock
        except (ConnectionRefusedError, TimeoutError, OSError) as e:
            print("Ulanmadi:", e)
            time.sleep(delay + random.random()*0.2) # jitter
            delay = min(delay*2, CAP) # eksponent backoff

def run_client():
    s = connect_with_retry()
    try:
        for text in ["ping", "hello", "bye"]:
            try:
                send_msg(s, text.encode())
                reply = recv_msg(s).decode()
                print("reply:", reply)
            except (socket.timeout, ConnectionError, OSError) as e:
                print("I/O xato:", e, "→ qayta ulanish")
            try:
                s.shutdown(socket.SHUT_RDWR)
            except OSError:
                pass
            s.close()
            s = connect_with_retry() # avtomatik reconnect
    finally:
        try:
            s.shutdown(socket.SHUT_RDWR) # muloyim yopish
        except OSError:
            pass
        s.close()

if __name__ == "__main__":
    run_client()

```

Amaliy qoidalar (muxtasar)

- **Timeoutlar:**

- `create_connection(..., timeout=5)` — ulanish limiti.
- `sock.settimeout(10)` — har bir send/recv uchun I/O limiti.

- **Reconnect:** xatoda **eksponent backoff + jitter** qo'llang; cheksiz sikl yoki maksimal urinish soni.
- **Holatni tekshirish:**
 - `recv()` **0 bayt** qaytarsa — peer socketni yopgan.
 - `socket.timeout` — tarmoq sust; qayta urinish yoki foydalanuvchiga xabar.
 - `ConnectionResetError/BrokenPipeError` — qarshi tomon to'satdan uzdi.
- **Yopish tartibi:** `shutdown(SHUT_RDWR)` → `close()` (har doim `try/except` bilan).
- **SO_REUSEADDR/KEEPALIVE:** serverda `SO_REUSEADDR`; uzoq sessiyalar uchun `SO_KEEPALIVE`.
- **Framing:** uzunlik-prefiks yoki delimiter (¥n) — **majburiy**; aks holda xabarlar bo'linadi/qo'shiladi.
- **Loglash:** ulanish holati, xatolik turlari, qayta urinishlar sonini yozib boring.

Tezkor diagnostika

- Port tinglayaptimi?
 - **Linux:** `ss -tlnp | grep 5000` yoki `lsof -i :5000`
 - **Win:** `netstat -ano | find "5000"`
- Soddashtirilgan test: `nc -l 5000` (server o'rnida) yoki `nc 127.0.0.1 5000` (klient o'rnida).
- Paket darajasi: Wireshark — SYN/SYN-ACK/ACK, RST, retransmitlarni ko'ring.

Nazorat savollari:

1. Server bilan aloqani qanday amalga oshiriladi?
2. `socket.bind()` metodi nima amalni bajaradi?
3. `socket.gethostname()` metodi qanday vazifani bajaradi?
4. `listen()` metodining ishlash tamoyili qanday?
5. `socket.recv()` metodidan nima maqsadda foydalaniladi?
6. Socket nima?
7. `decode()` funksiyasi nima maqsadda foydalaniladi?
8. Serverga ma'lumot jo'natish qanday amalga oshiriladi?
9. Serverdan ma'lumotni qanday qabul qilin olinadi?
10. `Connect()` metodi qanday parametrlarni qabul qiladi?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

4-mashg'ulot. TCP klient-server dasturini testlash.

O'quv savollari:

1. TCP client-server dasturi yordamida ma'lumot almashish.
2. TCP ulanishini o'rnatish va testlash.
3. TCP client va serverdagi xatoliklarni aniqlash va bartaraf etish.

1. TCP client-server dasturi yordamida ma'lumot almashish.

Serverni ulanish

Server kiruvchi ulanishlarni kutib turadi (listening), ularni qandaydir tarzda qayta ishlaydi va javob yuboradi. Server o'z ishini boshlashi uchun, avvalo, u ulanishlarni tinglaydigan manzilni aniqlab olishi kerak. Buning uchun **bind()** metodi qo'llaniladi.

```
socket.bind(address)
```

Bu usul server ishga tushadigan manzilni qabul qiladi. Odatiy bo'lib, manzil ikki elementdan iborat to'plamdir:

```
(host, port)
```

Birinchi element string sifatida xost hisoblanadi. Bu, masalan, "127.0.0.1" shaklidagi IP-manzil yoki mahalliy xost nomi bo'lishi mumkin. Ikkinchi parametr raqamli port raqamidir. Port 0 dan 65535 gacha bo'lgan 2 baytlik qiymatni ifodalaydi. Bir xil manzilda (bir xil mashinada) bir nechta turli tarmoq ilovalari ishga tushirilishi mumkinligi sababli, port ushbu ilovalarni farqlash imkonini beradi. Masalan:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
```

Bu yerda server 12345-portda ishlaydi. E'tibor bering, barcha portlar ham bepul bo'lmasligi mumkin. Ammo, qoida tariqasida, band bo'lgan portlar unchalik ko'p emas.

Manzil sifatida joriy xost nomidan foydalanamiz. Uni olish uchun funksiyadan foydalaniladi **socket.gethostname()** (odatda bu joriy kompyuterning nomi).

Ulanishlarni tinglash

Serverni bog'lagandan so'ng ulanishlarni tinglash uchun uni ishga tushirish kerak. Buning uchun **listen()** metodi qo'llaniladi.

```
socket.listen([backlog])
```

Bu backlog parametrini qabul qiladi. backlog – bu socket uchun ruxsat etilgan navbatdagi kiruvchi ulanishlarning maksimal soni. Ya'ni, mijozlar ulanganda, ular navbatda turishadi va server joriy mijozni qayta ishlaguncha kutishadi. Agar navbatda

server tomonidan qayta ishlanishini kutayotgan mijozlar soni allaqachon belgilangan bo'lsa, barcha yangi mijozlar rad etiladi. Masalan:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)
```

Mijozni qabul qilish va qayta ishlash

accept() metodi kiruvchi ulanishlarni qabul qilish uchun ishlatiladi. Bu usul 2 ta tuple tipli qiymatni qaytaradi

```
(conn, address)
```

Birinchi element conn - server mijoz bilan aloqa qiladigan boshqa socket obyektini ifodalaydi. Ikkinchi element address - ulangan mijozning manzili hisoblanadi. Shuni ta'kidlash kerakki, mijoz bilan o'zaro aloqalar tugagandan so'ng, conn socketini close() usuli bilan yopish kerak bo'ladi.

Tuplening birinchi elementi - conn yordamida mijozga xabar yuborish yoki aksincha ma'lumotlarni qabul qilish mumkin. Socketdan ma'lumotlarni qabul qilish uchun **socket.recv()** usuli qo'llaniladi.

```
bytes = socket.recv(bufsize)
```

Bu zarur parametr sifatida bir vaqtning o'zida boshqa socketdan olinishi mumkin bo'lgan baytlardagi maksimal bufer hajmini oladi. Qaytish qiymati boshqa socketdan olingan baytlar to'plamidir.

Ma'lumotlarni yuborish uchun **socket.send()** metodi qo'llaniladi, u parametr sifatida yuboriladigan ma'lumotlar to'plamini oladi.

```
socket.send(bytes)
```

Endi bir misolni ko'rib chiqaylik. Server.py faylida serverni quyidagi kod bilan aniqlaylik:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)

print("Server starts")

con, addr = server.accept()
print("connection: ", con)
print("client address: ", addr)

message = "Hello Client!"
con.send(message.encode())
con.close()
print("Server ends")
```



```
server.close()
```

Bu yerda server mijozni qabul qiladi, uning ulanishi haqidagi ma'lumotlarni chiqaradi va mijozga "Hello Client!" qatorini qaytarib yuboradi.

Va client.py faylida quyidagi mijoz kodini aniqlanadi:

```
import socket
client = socket.socket()
hostname = socket.gethostname()
port = 12345
client.connect((hostname, port))
data = client.recv(1024)
print("Server sent: ", data.decode())
client.close()
```

Bizning serverimiz va mijozimiz bitta kompyuterda ishlaganligi sababli, **socket.gethostname()** server kodidagi kabi ulanish uchun server manzilini aniqlash uchun funksiya va port 12345 ishlatiladi. Serverga ulangandan so'ng undan ma'lumotlarni olinadi va uni konsolda ko'rsatadi.

Avval server kodini ishga tushiraylik. Va keyin mijoz kodini ishga tushiriladi. Natijada, server ulanishni qabul qiladi, u haqidagi ma'lumotlarni konsolda ko'rsatadi va mijozga xabar yuboradi:

```
c:\python>python server.py
Server starts
connection: <socket.socket fd=432, family=2, type=1, proto=0,
laddr=('192.168.0.102', 12345), raddr=('192.168.0.102', 61824)>
client address: ('192.168.0.102', 61824)
Server ends
```

Xususan, bu yerda server va mijoz 192.168.0.102 da ishlayotganini va mijoz 61824 portidan foydalanayotganini ko'ramiz.

Va mijoz serverdan xabar oladi va uni konsolga chop etadi:

```
c:\python>python client.py
Server sent: Hello Client!
```

Bir nechta mijozlar bilan ishlash

Bunday holda, server bitta mijozga xizmat qiladi va ishlashni to'xtatadi. Agar biz server ko'p mijozlar bilan ishlashni istasak, unda cheksiz sikldan foydalanishga to'g'ri keladi:

```
import socket
from datetime import datetime

server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)

print("Server running")
```

```

while True:
    con, addr = server.accept()
    print("client address: ", addr)
    message = datetime.now().strftime("%H:%M:%S")
    con.send(message.encode())
    con.close()

```

Bunday holda, masalan, o'rnatilgan modul `datetime` va `datetime.now().strftime()` funksiyasidan foydalanib, joriy vaqtni satr sifatida olish, keyinchalik uni mijozga yuborish mumkin. Natijada, mijoz so'rov bilan joriy vaqthaqidagi ma'lumotlarni ham oladi.

Ikki tomonlama aloqa

Yuqoridagi misollarda aloqa bir tomonlama edi - server ma'lumotlarni yuboradi va mijoz uni qabul qiladi. Keling, mijoz ham, server ham ma'lumotlarni jo'natgan va qabul qilganda eng oddiy ikki tomonlama aloqani ko'rib chiqaylik. Serverga mijozdan bir nechta satr olishiga ruxsat beraylik, uni o'zgartirib so'ngra mijozga qaytarib yuboraylik:

```

import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)

print("Server running")
while True:
    con, _ = server.accept()
    data = con.recv(1024)
    message = data.decode()
    print(f"Client sent: {message}")
    message = message[::-1]
    con.send(message.encode())
    con.close()

```

Mijoz konsoldan satr kiritish va uni serverga yuborish uchun kodni aniqlasin:

```

import socket

client = socket.socket()
hostname = socket.gethostname()
port = 12345
client.connect((hostname, port))
message = input("Input a text: ")
client.send(message.encode())
data = client.recv(1024)
print("Server sent: ", data.decode())
client.close()

```

Yuqoridagi ishning natijasi:

Client:

```
c:\python>python client.py
```

```
Input a text: hello
Server sent: olleh
c:\python>
```

Server:

```
c:\python>python server.py
Server running
Client sent: hello
```

2) TCP ulanishini o'rnatish va testlash**Minimal ECHO server**

```
# server.py
import socket

HOST, PORT = '127.0.0.1', 5000
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind((HOST, PORT))
    s.listen(1)
    print(f"Listening on {HOST}:{PORT}")
    conn, addr = s.accept()
    with conn:
        print("Client:", addr)
        while True:
            data = conn.recv(4096)          # 0 bayt → mijoz yopdi
            if not data:
                break
            conn.sendall(data)              # echo
```

Minimal klient

```
# client.py
import socket

HOST, PORT = '127.0.0.1', 5000
with socket.create_connection((HOST, PORT), timeout=5) as s:
    s.sendall(b"salom, tcp!")
    print("Server javobi:", s.recv(4096).decode('utf-8'))
```

Testlash:

1. python server.py (server terminalida)
2. python client.py (boshqa terminalda)
3. Agar “salom, tcp!” qaytsa — ulanish OK.

TCP **stream**: bitta send = bitta recv bo'lishi **shart emas**. Barqaror xabar almashuv uchun framing qo'llang (quyida).

Framing: uzunlik-prefiks (tavsiya etiladi)

```
import struct

def send_msg(sock, payload: bytes):
    sock.sendall(struct.pack('!I', len(payload)) + payload)
```

```
def recv_exact(sock, n):
    buf = b''
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed")
        buf += chunk
    return buf

def recv_msg(sock):
    ln, = struct.unpack('!I', recv_exact(sock, 4))
    return recv_exact(sock, ln)
```

Shu yordamchilarni server/klientga qo'shsangiz, xabarlar bo'linmaydi va qo'shilib ketmaydi.

3) TCP client va serverdagi xatoliklarni aniqlash va bartaraf etish Mustahkam klient shabloni (timeout + reconnect)

```
import socket, time, random, struct

HOST, PORT = '127.0.0.1', 5000
BASE, CAP = 0.5, 8.0 # reconnect uchun eksponent backoff

def send_msg(sock, payload: bytes):
    sock.sendall(struct.pack('!I', len(payload)) + payload)

def recv_exact(sock, n):
    buf = b''
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed")
        buf += chunk
    return buf

def recv_msg(sock):
    ln, = struct.unpack('!I', recv_exact(sock, 4))
    return recv_exact(sock, ln)

def connect_with_retry():
    delay = BASE
    while True:
        try:
            s = socket.create_connection((HOST, PORT), timeout=5)
            s.settimeout(10) # har bir I/O uchun timeout
            return s
        except (ConnectionRefusedError, TimeoutError, OSError) as e:
            print("Ulanmadi:", e)
            time.sleep(delay + random.random()*0.2) # jitter
            delay = min(delay*2, CAP)
```

```

def run_client():
    s = connect_with_retry()
    try:
        for text in ["ping", "hello", "bye"]:
            try:
                send_msg(s, text.encode('utf-8'))
                print("reply:", recv_msg(s).decode('utf-8'))
            except (socket.timeout, ConnectionError, OSError) as e:
                print("I/O xato:", e, "→ reconnect")
                try: s.shutdown(socket.SHUT_RDWR)
                except OSError: pass
                s.close()
                s = connect_with_retry()
    finally:
        try: s.shutdown(socket.SHUT_RDWR)
        except OSError: pass
        s.close()

if __name__ == "__main__":
    run_client()

```

Serverda ko'p mijozni boshqarish (thread bilan)

```

# qisqa shablon
import socket, threading, struct
HOST, PORT = '127.0.0.1', 5000

def recv_exact(s, n):
    buf=b''
    while len(buf)<n:
        c=s.recv(n-len(buf))
        if not c: raise ConnectionError("peer closed")
        buf+=c
    return buf

def recv_msg(s):
    import struct
    ln,=struct.unpack('!I', recv_exact(s,4))
    return recv_exact(s, ln)

def send_msg(s, b):
    s.sendall(struct.pack('!I', len(b)) + b)

def handle(conn, addr):
    conn.settimeout(20)
    try:
        while True:
            data = recv_msg(conn)
            send_msg(conn, data.upper()) # misol: ECHO
    except (socket.timeout, ConnectionError, OSError):
        pass
    finally:
        try: conn.shutdown(socket.SHUT_RDWR)

```

```

except OSError: pass
conn.close()

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind((HOST, PORT)); srv.listen(100)
    print("Listening...")
    while True:
        c, a = srv.accept()
        threading.Thread(target=handle, args=(c, a), daemon=True).start()

```

Tez-tez uchraydigan xatolar va yechimlar

Xato/holat	Sabab	Bartaraf etish
ConnectionRefusedError	Server ishlamaydi, IP/port xato, firewall	Serverni ishga tushiring; IP/portni tekshiring; ruxsat bering
Address already in use	Port band (oldingi jarayon yopilmagan)	SO_REUSEADDR yoqing; portni almashtiring; eski jarayonni yoping
socket.timeout	Ulanish/I/O sust	settimeoutni moslang; qayta urinish strategiyasi
ConnectionResetError / BrokenPipeError	Peer to'satdan uzdi	Protokol bo'yicha yopish (shutdown→close); reconnect
Noto'liq yoki qo'shilib ketgan xabarlar	Framing yo'q	Uzunlik-prefiks yoki delimiter joriy qiling
Kodlash muammolari	UTF-8/CP1251 aralash	Har ikki tomonda bir xil kodlash; binar oqimda .decode() qilmaslik

Diagnostika va monitoring

- Port tinglayaptimi?
 - Linux: `ss -tlnp | grep 5000 / lsof -i :5000`
 - Windows: `netstat -ano | find "5000"`
- Soddalashtirilgan test: `nc 127.0.0.1 5000` yoki `telnet`.
- Paket darajasi: **Wireshark** — SYN/SYN-ACK/ACK, RST, retransmitlarni ko'ring.
- Loglar: ulanish ID, xabar uzunligi, vaqt, xatolik kodlari.

Nazorat savollari:

- Server bilan aloqani qanday amalga oshiriladi?
- `socket.bind()` metodi nima amalni bajaradi?
- `socket.gethostname()` metodi qanday vazifani bajaradi?
- `listen()` metodining ishlash tamoyili qanday?
- `socket.recv()` metodidan nima maqsadda foydalaniladi?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

5-mashg'ulot. PyQt5 paketidan foydalanib zamonaviy chat dasturini tuzish.

O'quv savollari:

1. PyQt5 orqali TCP client uchun GUI yaratish.
2. Socket ulanishini boshqarish (connect/send/receive) va xabar almashish.
3. Signal–slot mexanizmi va xatoliklarni GUI'da ko'rsatish.

TCP dasturini client qismini GUI ko'rinishda ishlab chiqish.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Tue Jul 24 13:05:46 2018

@author: JC
"""

import socket
import sys
import threading
import time
import functools
from PyQt5 import QtCore, QtGui
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import QMainWindow, QApplication, QWidget, QPushButton
from PyQt5.QtWidgets import QVBoxLayout, QHBoxLayout, QMessageBox,
QTabWidget
from PyQt5.QtWidgets import QGridLayout, QScrollArea, QLabel, QListView
from PyQt5.QtWidgets import QLineEdit, QComboBox, QGroupBox, QAction
from PyQt5.QtGui import QStandardItemModel, QStandardItem, QFont

class MyTableWidget(QWidget):
    def __init__(self, parent):
        super(QWidget, self).__init__(parent)
        #conecion
        self.conn = socket.socket()
        self.connected = False
        #tab UI
        self.layout = QVBoxLayout(self)
        self.tabs = QTabWidget()
        self.tabs.resize(300,200)
        self.tab1 = QWidget()
        self.tab2 = QWidget()
        self.tabs.addTab(self.tab1, "Home")
        self.tabs.addTab(self.tab2, "Chat Room")
        self.tabs.setTabEnabled(1,False)
        #<Home>
        gridHome = QGridLayout()
        self.tab1.setLayout(gridHome)
```

```

self.IPBox = QGroupBox("IP")
self.IPLineEdit = QLineEdit()
self.IPLineEdit.setText("127.0.0.1")
IPBoxLayout = QVBoxLayout()
IPBoxLayout.addWidget(self.IPLineEdit)
self.IPBox.setLayout(IPBoxLayout)
self.portBox = QGroupBox("port")
self.portLineEdit = QLineEdit()
self.portLineEdit.setText("33002")
portBoxLayout = QVBoxLayout()
portBoxLayout.addWidget(self.portLineEdit)
self.portBox.setLayout(portBoxLayout)
self.nameBox = QGroupBox("Name")
self.nameLineEdit = QtWidgets.QLineEdit()
nameBoxLayout = QVBoxLayout()
nameBoxLayout.addWidget(self.nameLineEdit)
self.nameBox.setLayout(nameBoxLayout)
self.connStatus = QLabel("Status", self)
font = QFont()
font.setPointSize(16)
self.connStatus.setFont(font)
self.connBtn = QPushButton("Connect")
self.connBtn.clicked.connect(self.connect_server)
self.disconnBtn = QPushButton("Disconnect")
self.disconnBtn.clicked.connect(self.disconnect_server)
gridHome.addWidget(self.IPBox,0,0,1,1)
gridHome.addWidget(self.portBox,0,1,1,1)
gridHome.addWidget(self.nameBox,1,0,1,1)
gridHome.addWidget(self.connStatus,1,1,1,1)
gridHome.addWidget(self.connBtn,2,0,1,1)
gridHome.addWidget(self.disconnBtn,2,1,1,1)
gridHome.setColumnStretch(0, 1)
gridHome.setColumnStretch(1, 1)
gridHome.setRowStretch(0, 0)
gridHome.setRowStretch(1, 0)
gridHome.setRowStretch(2, 9)
#</Home>
#<Chat Room>
gridChatRoom = QGridLayout()
self.tab2.setLayout(gridChatRoom)
    self.messageRecords = QLabel("<font color='#000000'>Welcome to
chat room</font>", self)
    self.messageRecords.setStyleSheet("background-color: white;");
    self.messageRecords.setAlignment(QtCore.Qt.AlignTop)
    self.messageRecords.setAutoFillBackground(True);
    self.scrollRecords = QScrollArea()
    self.scrollRecords.setWidget(self.messageRecords)
    self.scrollRecords.setWidgetResizable(True)
    self.sendTo = "ALL"
    self.sendChoice = QLabel("Send to :ALL", self)
    self.sendComboBox = QComboBox(self)

```



```

self.sendComboBox.addItem("ALL")
self.sendComboBox.activated[str].connect(self.send_choice)
self.lineEdit = QLineEdit()
self.lineEnterBtn = QPushButton("Enter")
self.lineEnterBtn.clicked.connect(self.enter_line)
self.lineEdit.returnPressed.connect(self.enter_line)
self.friendList = QListView()
self.friendList.setWindowTitle('Room List')
self.model = QStandardItemModel(self.friendList)
self.friendList.setModel(self.model)
self.emojiBox = QGroupBox("Emoji")
self.emojiBtn1 = QPushButton("😊👉👈")
    self.emojiBtn1.clicked.connect(funcutils.partial(self.send_emoji,
"😊👉👈"))
    self.emojiBtn2 = QPushButton("(👉👈)")
self.emojiBtn2.clicked.connect(funcutils.partial(self.send_emoji, "(
👉👈)"))
    self.emojiBtn3 = QPushButton("( ~·3·)")
self.emojiBtn3.clicked.connect(funcutils.partial(self.send_emoji, "(
~·3·)"))
    self.emojiBtn4 = QPushButton("👉(ò_ó~)👈")
    self.emojiBtn4.clicked.connect(funcutils.partial(self.send_emoji,
"👉(ò_ó~)👈"))
    emojiLayout = QHBoxLayout()
    emojiLayout.addWidget(self.emojiBtn1)
    emojiLayout.addWidget(self.emojiBtn2)
    emojiLayout.addWidget(self.emojiBtn3)
    emojiLayout.addWidget(self.emojiBtn4)
    self.emojiBox.setLayout(emojiLayout)
    gridChatRoom.addWidget(self.scrollRecords,0,0,1,3)
    gridChatRoom.addWidget(self.friendList,0,3,1,1)
    gridChatRoom.addWidget(self.sendComboBox,1,0,1,1)
    gridChatRoom.addWidget(self.sendChoice,1,2,1,1)
    gridChatRoom.addWidget(self.lineEdit,2,0,1,3)
    gridChatRoom.addWidget(self.lineEnterBtn,2,3,1,1)
    gridChatRoom.addWidget(self.emojiBox,3,0,1,4)
    gridChatRoom.setColumnStretch(0, 9)
    gridChatRoom.setColumnStretch(1, 9)
    gridChatRoom.setColumnStretch(2, 9)
    gridChatRoom.setColumnStretch(3, 1)
    gridChatRoom.setRowStretch(0, 9)
    #</Chat Room>
    #Initialization
    self.layout.addWidget(self.tabs)
    self.setLayout(self.layout)

def enter_line(self):
    #assure the person still in room before send out
    if self.sendTo != self.sendComboBox.currentText():

```

```

        self.message_display_append("The person left. Private message
not delivered")
        self.lineEdit.clear()
        return
    line = self.lineEdit.text()
    if line == "":#prevent empty message
        return
    if self.sendTo != "ALL":#private message, send to myself first
        #this is a trick leverage the server sending back a copy to
myself

        send_msg = bytes("{}+self.userName+"}"+line, "utf-8")
        self.conn.send(send_msg)
        time.sleep(0.1) #this is important for not overlapping two
sending

        send_msg = bytes("{}+self.sendTo+"}"+line, "utf-8")
        self.conn.send(send_msg)
        self.lineEdit.clear()
        self.scrollRecords.verticalScrollBar().setValue(self.scrollRecords.
verticalScrollBar().maximum())

    def send_emoji(self, emoji):
        #assure the person still in room before send out
        if self.sendTo != self.sendComboBox.currentText():
            self.message_display_append("The person left. Private message
not delivered")
            return
        if self.sendTo != "ALL":#private message, send to myself first
            #this is a trick leverage the server sending back a copy to
myself

            send_msg = bytes("{}+self.userName+"}"+emoji, "utf-8")
            self.conn.send(send_msg)
            time.sleep(0.1) #this is important for not overlapping two
sending

            send_msg = bytes("{}+self.sendTo+"}"+emoji, "utf-8")
            self.conn.send(send_msg)

    def message_display_append(self, newMessage, textColor = "#000000"):
        oldText = self.messageRecords.text()
        appendText = oldText+"<br /><font
color='\"'+textColor+'\">"+newMessage+"</font><font color='\"#000000\"></font>"
        self.messageRecords.setText(appendText)
        time.sleep(0.2) #this helps the bar set to bottom, after all message
already appended
        self.scrollRecords.verticalScrollBar().setValue(self.scrollRecords.
verticalScrollBar().maximum())

    def updateRoom(self):
        while self.connected:
            data = self.conn.recv(1024)
            data = data.decode("utf-8")

```

```

        print(data)
        if data != "":
            if "{CLIENTS}" in data:
                welcome = data.split("{CLIENTS}")
                self.update_send_to_list(welcome[1])
                self.update_room_list(welcome[1])
                if not welcome[0][5:] == "":
                    self.message_display_append(welcome[0][5:])
                    self.scrollRecords.verticalScrollBar().setValue(self
f.scrollRecords.verticalScrollBar().maximum())
                elif data[:5] == "{MSG}": #{MSG} includes broadcast and
server msg
                    self.message_display_append(data[5:], "#006600")
                    self.scrollRecords.verticalScrollBar().setValue(self.sc
rollRecords.verticalScrollBar().maximum())
                else: #private message is NONE format
                    self.message_display_append("{private}"+data, "#cc33cc")
                    self.scrollRecords.verticalScrollBar().setValue(self.sc
rollRecords.verticalScrollBar().maximum())
                time.sleep(0.1) #this is for saving thread cycle time

    def connect_server(self):
        if self.connected == True:
            return
        name = self.nameLineEdit.text()
        if name == "":
            self.connStatus.setText("Status :"+"Please enter your name")
            return
        self.userName = name
        IP = self.IPLineEdit.text()
        if IP == "":
            IP = "127.0.0.1"
        port = self.portLineEdit.text()
        if port == "" or not port.isnumeric():
            self.portLineEdit.setText("33002")
            self.connStatus.setText("Status :"+"Port format invalid")
            return
        else:
            port = int(port)
        try:
            self.conn.connect((IP, port))
        except:
            self.connStatus.setText("Status :"+" Refused")
            self.conn = socket.socket()
            return
        send_msg = bytes("{REGISTER}"+name, "utf-8")
        self.conn.send(send_msg)
        self.connected = True
        self.connStatus.setText("Status :"+" Connected")
        self.nameLineEdit.setReadOnly(True) #This setting is not functional

```

well

```

self.tabs.setTabEnabled(1,True)
self.rT = threading.Thread(target= self.updateRoom)
self.rT.start()

def disconnect_server(self):
    if self.connected == False:
        return
    send_msg = bytes("{QUIT}", "utf-8")
    self.conn.send(send_msg)
    self.connStatus.setText("Status : "+" Disconnected")
    self.nameLineEdit.setReadOnly(False)
    self.nameLineEdit.clear()
    self.tabs.setTabEnabled(1,False)
    self.connected = False
    self.rT.join()
    self.conn.close()
    self.conn = socket.socket()

def update_room_list(self, strList):
    L = strList.split("|")
    self.model.clear()
    for person in L:
        item = QStandardItem(person)
        item.setCheckable(False)
        self.model.appendRow(item)

def update_send_to_list(self, strList):
    L = strList.split("|")
    self.sendComboBox.clear()
    self.sendComboBox.addItem("ALL")
    for person in L:
        if person != self.userName:
            self.sendComboBox.addItem(person)
    previous = self.sendTo
    index = self.sendComboBox.findText(previous)
    print("previous choice:",index)
    if index != -1:
        self.sendComboBox.setCurrentIndex(index) #updating, maintain
receiver
    else:
        self.sendComboBox.setCurrentIndex(0) #updating, the receiver
left, deafault to "ALL"

def send_choice(self,text):
    self.sendTo = text
    print(self.sendTo)
    self.sendChoice.setText("Send to: "+text)

class Window(QMainWindow):
    def __init__(self):
        super(Window, self).__init__()

```

```

        self.setGeometry(50, 50, 500, 300)
        self.setWindowTitle("Chat-Client")
        self.table_widget = MyTableWidget(self)
        self.setCentralWidget(self.table_widget)
        self.show()

    def closeEvent(self, event):
        close = QMessageBox()
        close.setText("You sure?")
        close.setStandardButtons(QMessageBox.Yes | QMessageBox.Cancel)
        close = close.exec()
        if close == QMessageBox.Yes:
            self.table_widget.disconnect_server() #disconnect to server
before exit
            event.accept()
        else:
            event.ignore()

    def run():
        app = QApplication(sys.argv)
        GUI = Window()
        sys.exit(app.exec_())

if __name__ == "__main__":
    run()

```

2. Socket ulanishini boshqarish (connect / send / receive) va xabar almashish

Nazariy qism

Socket — bu tarmoq orqali ikkita dastur (klient va server) oʻrtasida maʼlumot almashish uchun moʻljallangan interfeys. TCP/IP protokoli asosida ishlaganda, socket quyidagi bosqichlarda ishlaydi:

1. Server tomonida:

- socket() — yangi tarmoq socketini yaratadi.
- bind() — IP va portni socketga bogʻlaydi.
- listen() — ulanishlar uchun “tinglash” holatiga oʻtkazadi.
- accept() — klient ulanayotganda ulanishni qabul qiladi.

2. Klient tomonida:

- socket() — socket yaratadi.
- connect() — server bilan ulanadi.
- send() — serverga xabar yuboradi.
- recv() — serverdan xabar oladi.

3. Maʼlumot almashish:

- Ikkala tomon send() va recv() yordamida almashadi.

- TCP holatida oqimli (stream-based) uzatish bo‘ladi — xabar bo‘linmasligi uchun framing ishlatiladi.

Amaliy misol: TCP klient va server

server.py

```
import socket

HOST, PORT = '127.0.0.1', 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen(1)
    print(f"Server {HOST}:{PORT} da tinglamoqda...")

    conn, addr = server.accept()
    with conn:
        print("Ulandi:", addr)
        while True:
            data = conn.recv(1024)
            if not data:
                break
            print("Kelgan xabar:", data.decode())
            conn.sendall(data.upper())
```

client.py

```
import socket

HOST, PORT = '127.0.0.1', 5000

with socket.create_connection((HOST, PORT)) as client:
    xabar = input("Serverga xabar yozing: ")
    client.sendall(xabar.encode())
    javob = client.recv(1024)
    print("Serverdan javob:", javob.decode())
```

Natija: Klient “salom” deb yuborsa, server uni “SALOM” qilib qaytaradi.

3. Signal–slot mexanizmi va xatoliklarni GUI’da ko‘rsatish

Nazariy qism

Signal–slot mexanizmi — bu **PyQt5** (yoki **PySide**) da grafik interfeys (GUI) elementlari o‘rtasida aloqa o‘rnatish tizimi.

- **Signal** — biror hodisa sodir bo‘lishi (masalan, tugma bosilishi).
- **Slot** — bu hodisaga javoban bajariladigan funksiya yoki metod.

Misol:

```
self.pushButton.clicked.connect(self.send_message)
```

Bu kodda clicked signali send_message slotiga bog‘langan — tugma bosilganda funksiya ishga tushadi.

Amaliy GUI misol: TCP klient PyQt5’da

client_gui.py

```

import sys
import socket
from PyQt5 import QtWidgets, QtCore

class TcpClient(QtWidgets.QWidget):
    def __init__(self):
        super().__init__()
        self.init_ui()
        self.sock = None

    def init_ui(self):
        self.setWindowTitle("TCP Klient GUI")
        self.resize(400, 250)

        self.ip_label = QtWidgets.QLabel("Server IP:")
        self.ip_input = QtWidgets.QLineEdit("127.0.0.1")

        self.port_label = QtWidgets.QLabel("Port:")
        self.port_input = QtWidgets.QLineEdit("5000")

        self.msg_label = QtWidgets.QLabel("Xabar:")
        self.msg_input = QtWidgets.QLineEdit()

        self.connect_btn = QtWidgets.QPushButton("Ulanish")
        self.send_btn = QtWidgets.QPushButton("Yuborish")
        self.output = QtWidgets.QTextEdit()
        self.output.setReadOnly(True)

        grid = QtWidgets.QGridLayout()
        grid.addWidget(self.ip_label, 0, 0)
        grid.addWidget(self.ip_input, 0, 1)
        grid.addWidget(self.port_label, 1, 0)
        grid.addWidget(self.port_input, 1, 1)
        grid.addWidget(self.msg_label, 2, 0)
        grid.addWidget(self.msg_input, 2, 1)
        grid.addWidget(self.connect_btn, 3, 0)
        grid.addWidget(self.send_btn, 3, 1)
        grid.addWidget(self.output, 4, 0, 1, 2)
        self.setLayout(grid)

        # Signal-slot bog'lanishi
        self.connect_btn.clicked.connect(self.connect_server)
        self.send_btn.clicked.connect(self.send_message)

    def connect_server(self):
        try:
            host = self.ip_input.text()
            port = int(self.port_input.text())
            self.sock = socket.create_connection((host, port), timeout=5)
            self.output.append(f"[OK] Ulandi: {host}:{port}")

```

```

except Exception as e:
    self.output.append(f"[XATO] Ulanishda xato: {e}")

def send_message(self):
    if not self.sock:
        self.output.append("[XATO] Avval serverga ulang!")
        return
    try:
        msg = self.msg_input.text().encode()
        self.sock.sendall(msg)
        reply = self.sock.recv(1024).decode()
        self.output.append(f"Server: {reply}")
    except Exception as e:
        self.output.append(f"[XATO] Jo'natishda yoki qabulda xato: {e}")
        self.sock = None # ulanish uzilgan

```

ishga tushirish:

```

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    window = TcpClient()
    window.show()
    sys.exit(app.exec_())

```

GUI'da xatoliklarni ko'rsatish mexanizmlari**1. TextEdit / QLabel orqali:**

```
self.output.append("[XATO] Ulanishda muammo: Serverga bog'lanmadi.")
```

2. Xabar oynasi orqali:

```
QtWidgets.QMessageBox.warning(self, "Xato", "Serverga ulanib bo'lmadi!")
```

3. Signal orqali xatoni uzatish:

```

self.error_signal = QtCore.pyqtSignal(str)
self.error_signal.connect(self.show_error)
def show_error(self, msg):
    self.output.append(f"[XATO]: {msg}")

```

Xulosa

- **Socket** — tarmoqda ma'lumot almashish interfeysi.
- **connect/send/recv** — TCP orqali xabar yuborish va olish uchun asosiy funksiyalar.
- **Signal-slot** — PyQt5'da hodisalar va funksiyalar o'rtasidagi aloqa.
- GUI'da xatoliklarni foydalanuvchiga ko'rsatish uchun QTextEdit, QMessageBox yoki statusBar ishlatiladi.

Nazorat savollari

1. TCP protokolining asosiy bosqichlari (connect, send, recv) qanday ketma-ketlikda bajariladi?
2. Socket obyektining bind(), listen(), va accept() funksiyalari nimani bajaradi?
3. PyQt5'dagi **signal-slot** mexanizmi nima va u qanday ishlaydi?
4. GUI'da xatoliklarni foydalanuvchiga ko'rsatishning uchta usulini yozing.

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

6-mashg'ulot. Pythonda GUI paketidan foydalanib chat dasturini tuzishni yakunlash

O'quv savollari:

1. TCP client–server dasturini GUI ko'rinishda ishlab chiqish.
2. Xabar almashish oqimini (connect/send/receive) tekshirish va sinovdan o'tkazish.
3. Xatoliklarni qayta ishlash, holatlarni (status/reconnect) GUI'da boshqarish.

TCP client-server dasturini GUI ko'rinishda ishlab chiqish.

```
import argparse
from socket import AF_INET, socket, SOCK_STREAM
from threading import Thread

def accept_incoming_connections():
    """Sets up handling for incoming clients."""
    while True:
        client, client_address = SERVER.accept()
        print("%s:%s has connected." % client_address)
        addresses[client] = client_address
        Thread(target=handle_client, args=(client,)).start()

def handle_client(client): # Takes client socket as argument.
    """Handles a single client connection."""
    name = ""
    prefix = ""

    while True:
        msg = client.recv(BUFSIZ)

        if not msg is None:
            msg = msg.decode("utf-8")

            if msg == "":
                msg = "{QUIT}"

            # Avoid messages before registering
            if msg.startswith("{ALL}") and name:
                new_msg = msg.replace("{ALL}", "{MSG}" + prefix)
                send_message(new_msg, broadcast=True)
                continue

            if msg.startswith("{REGISTER}"):
                name = msg.split(" ")[1]
                welcome = '{MSG}Welcome %s!' % name
                send_message(welcome, destination=client)
```

```

        msg = "{MSG}%s has joined the chat!" % name
        send_message(msg, broadcast=True)
        clients[client] = name
        prefix = name + ": "
        send_clients()
        continue

    if msg == "{QUIT}":
        client.close()
        try:
            del clients[client]
        except KeyError:
            pass
        if name:
            send_message("{MSG}%s has left the chat." % name, broadcast=True)
            send_clients()
        break

    # Avoid messages before registering
    if not name:
        continue

    # We got until this point, it is either an unknown message or for an
    # specific client...
    try:
        msg_params = msg.split("{}")
        dest_name = msg_params[0][1:] # Remove the {
        dest_sock = find_client_socket(dest_name)
        if dest_sock:
            send_message(msg_params[1], prefix=prefix, destination=dest_sock)
        else:
            print("Invalid Destination. %s" % dest_name)
    except:
        print("Error parsing the message: %s" % msg)

def send_clients():
    send_message("{CLIENTS}" + get_clients_names(), broadcast=True)

def get_clients_names(separator="|"):
    names = []
    for _, name in clients.items():
        names.append(name)
    return separator.join(names)

def find_client_socket(name):
    for cli_sock, cli_name in clients.items():
        if cli_name == name:
            return cli_sock
    return None

```

```

def send_message(msg, prefix="", destination=None, broadcast=False):
    send_msg = bytes(prefix + msg, "utf-8")
    if broadcast:
        """Broadcasts a message to all the clients."""
        for sock in clients:
            sock.send(send_msg)
    else:
        if destination is not None:
            destination.send(send_msg)

clients = {}
addresses = {}

parser = argparse.ArgumentParser(description="Chat Server")
parser.add_argument(
    '--host',
    help='Host IP',
    default="127.0.0.1"
)
parser.add_argument(
    '--port',
    help='Port Number',
    default=33002
)

server_args = parser.parse_args()

HOST = server_args.host
PORT = int(server_args.port)
BUFSIZ = 2048
ADDR = (HOST, PORT)

stop_server = False

SERVER = socket(AF_INET, SOCK_STREAM)
SERVER.bind(ADDR)

if __name__ == "__main__":
    try:
        SERVER.listen(5)
        print("Server Started at {}:{}".format(HOST, PORT))
        print("Waiting for connection...")
        ACCEPT_THREAD = Thread(target=accept_incoming_connections)
        ACCEPT_THREAD.start()
        ACCEPT_THREAD.join()
        SERVER.close()
    except KeyboardInterrupt:

```

```
print("Closing...")
ACCEPT_THREAD.interrupt()
```

2) Xabar almashish oqimini (connect / send / receive) tekshirish va sinovdan o'tkazish

Nazariya (muxtasar)

- **TCP oqim (stream):** send() va recv() paket chegaralarini saqlamaydi. Shuning uchun **framing** kerak (masalan, 4 baytli uzunlik-prefiks).

- **sendall():** butun bufer jo'natilmaguncha qaytmaydi — qisqa jo'natmalarga yechim.

- **recv():** talab qilingan bayt sonidan kam qaytishi mumkin; kerakli uzunlikka yetguncha yig'ish lozim.

- **Testlash:**

1. Lokal echo-server (127.0.0.1, masalan 5000-port).
2. Klientdan bir nechta xabar yuborib, javoblarni tekshirish.
3. Uzun/past tezlikli sharoitlar (simulyatsiya) — framing sinfi buzilmasligi kerak.

Oddiy echo-server (sinov uchun)

```
# server.py
import socket, struct

HOST, PORT = '127.0.0.1', 5000

def recv_exact(s, n):
    buf=b''
    while len(buf)<n:
        c=s.recv(n-len(buf))
        if not c: raise ConnectionError("peer closed")
        buf+=c
    return buf

def recv_msg(s):
    ln, = struct.unpack('!I', recv_exact(s, 4))
    return recv_exact(s, ln)

def send_msg(s, payload: bytes):
    s.sendall(struct.pack('!I', len(payload)) + payload)

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind((HOST, PORT)); srv.listen(1)
    print(f"Echo server: {HOST}:{PORT}")
    conn, addr = srv.accept()
    with conn:
        print("Client:", addr)
        try:
```

```

while True:
    data = recv_msg(conn)
    send_msg(conn, data.upper())    # echo + transform
except ConnectionError:
    pass

```

3) Xatoliklarni qayta ishlash, holatlarni (status/reconnect) gui'da boshqarish

Nazariya (muxtasar)

- **GUI bloklanmasin:** tarmoqli ishlar **asosiy GUI oqimidan alohida** (QThread yoki QRunnable) bajarilishi kerak.
- **Signal–slot:** fon ishchi (socket) → GUI'ga **signallar** yuboradi: connected, messageReceived, error, disconnected, statusChanged.
- **Timeout & reconnect:** setTimeout() bilan I/O cheklash, xatoda **eksponent backoff + jitter** bilan qayta ulanish.
- **Holat boshqaruvi:** status-indikator (QLabel/QStatusBar), “Auto reconnect” belgilash, “Connect/Disconnect” tugmalarining faollashuvi.
- **Xatolarni ko'rsatish:** QTextEdit log, kerak bo'lsa QMessageBox (bloklovchi dialogni me'yorida qo'llash).

PyQt5 GUI: klient + fon ishchi (QThread)

Ushbu kod server.py bilan birga ishlaydi. GUI bloklanmaydi, statuslar real-vaqtda yangilanadi, framing ishlatiladi, xatolarda reconnect ixtiyoriy.

```

# client_gui.py
import sys, time, socket, struct, random
from PyQt5 import QtWidgets, QtCore

# ----- Framing yordamchilari -----
def _send_msg(sock, payload: bytes):
    sock.sendall(struct.pack('!I', len(payload)) + payload)

def _recv_exact(sock, n):
    buf=b''
    while len(buf)<n:
        c=sock.recv(n-len(buf))
        if not c:
            raise ConnectionError("peer closed")
        buf+=c
    return buf

def _recv_msg(sock) -> bytes:
    ln, = struct.unpack('!I', _recv_exact(sock, 4))
    return _recv_exact(sock, ln)

# ----- Worker: tarmoq QThread'da -----
class SocketWorker(QtCore.QThread):

```

```

connected = QtCore.pyqtSignal()
disconnected = QtCore.pyqtSignal()
messageReceived = QtCore.pyqtSignal(str)
error = QtCore.pyqtSignal(str)
status = QtCore.pyqtSignal(str)

def __init__(self, host, port, auto_reconnect=False, parent=None):
    super().__init__(parent)
    self.host = host
    self.port = port
    self.auto_reconnect = auto_reconnect
    self._sock = None
    self._running = True
    self._outbox = QtCore.QQueue() # xabarlar navbati
    self._mtx = QtCore.QMutex() # navbat uchun
    self._hasData = QtCore.QWaitCondition() # yuborish uyg'otgichi

# Qt'da QQueue yo'q - kichik wrapper: list + mtx + condition
# Qulaylik uchun o'zimiz imitatsiya qilamiz
def queue_put(self, item: bytes):
    self._mtx.lock()
    try:
        if not hasattr(self, "_queue"):
            self._queue = []
        self._queue.append(item)
        self._hasData.wakeAll()
    finally:
        self._mtx.unlock()

def queue_get(self):
    self._mtx.lock()
    try:
        while self._running and (not hasattr(self, "_queue") or not
self._queue):
            self._hasData.wait(self._mtx, 50) # 50 ms
        if not self._running:
            return None
        return self._queue.pop(0)
    finally:
        self._mtx.unlock()

def stop(self):
    self._running = False
    try:
        if self._sock:
            self._sock.shutdown(socket.SHUT_RDWR)
    except OSError:
        pass
    try:
        if self._sock:

```

```

        self._sock.close()
    except OSError:
        pass

def _connect_with_retry(self):
    delay, cap = 0.5, 8.0
    while self._running:
        try:
            s = socket.create_connection((self.host, self.port), timeout=5)
            s.settimeout(10)
            self.status.emit(f"Ulandi: {self.host}:{self.port}")
            self.connected.emit()
            return s
        except (ConnectionRefusedError, TimeoutError, OSError) as e:
            self.error.emit(f"Ulanmadi: {e}")
            if not self.auto_reconnect:
                return None
            self.status.emit(f"Qayta urinish... {delay:.1f}s")
            self.msleep(int((delay + random.random()*0.2) * 1000))
            delay = min(delay*2, cap)
    return None

def run(self):
    while self._running:
        self._sock = self._connect_with_retry()
        if not self._sock:
            self.disconnected.emit()
            break

        try:
            # Asosiy sikl: navbatdan yuborish va soketdan qabul qilish
            while self._running:
                # Yuborish
                item = self.queue_get()
                if item is None:
                    break
                try:
                    _send_msg(self._sock, item)
                except (socket.timeout, OSError, ConnectionError) as e:
                    self.error.emit(f"Yuborishda xato: {e}")
                    raise

                # Qabul
                try:
                    reply = _recv_msg(self._sock).decode('utf-8',
errors='replace')

                    self.messageReceived.emit(reply)
                except (socket.timeout, OSError, ConnectionError) as e:
                    self.error.emit(f"Qabulda xato: {e}")
                    raise

```

```

except Exception as e:
    self.status.emit("Ulanish uzildi")
    self.disconnected.emit()
    try:
        self._sock.close()
    except Exception:
        pass

    if not self.auto_reconnect:
        break
    # auto_reconnect bo'lsa - loop boshiga qaytamiz

self.status.emit("Worker to'xtadi")

# ----- GUI -----
class ClientGUI(QtWidgets.QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("TCP Klient (PyQt5)")
        self.resize(520, 360)

        self.hostEdit = QtWidgets.QLineEdit("127.0.0.1")
        self.portEdit = QtWidgets.QLineEdit("5000")
        self.connectBtn = QtWidgets.QPushButton("Connect")
        self.disconnectBtn = QtWidgets.QPushButton("Disconnect")
        self.autoReconnect = QtWidgets.QCheckBox("Auto reconnect")
        self.autoReconnect.setChecked(True)

        self.msgEdit = QtWidgets.QLineEdit()
        self.sendBtn = QtWidgets.QPushButton("Send")
        self.log = QtWidgets.QTextEdit(); self.log.setReadOnly(True)
        self.statusBar = QtWidgets.QLabel("Disconnected")

        grid = QtWidgets.QGridLayout(self)
        grid.addWidget(QtWidgets.QLabel("Host:"), 0, 0)
        grid.addWidget(self.hostEdit, 0, 1)
        grid.addWidget(QtWidgets.QLabel("Port:"), 0, 2)
        grid.addWidget(self.portEdit, 0, 3)
        grid.addWidget(self.connectBtn, 0, 4)
        grid.addWidget(self.disconnectBtn, 0, 5)
        grid.addWidget(self.autoReconnect, 1, 0, 1, 2)
        grid.addWidget(QtWidgets.QLabel("Xabar:"), 2, 0)
        grid.addWidget(self.msgEdit, 2, 1, 1, 4)
        grid.addWidget(self.sendBtn, 2, 5)
        grid.addWidget(self.log, 3, 0, 1, 6)
        grid.addWidget(self.statusBar, 4, 0, 1, 6)

        self.worker = None
        self._set_connected(False)

```



```

# signal-slot
self.connectBtn.clicked.connect(self.on_connect)
self.disconnectBtn.clicked.connect(self.on_disconnect)
self.sendBtn.clicked.connect(self.on_send)

def _set_connected(self, ok: bool):
    self.connectBtn.setEnabled(not ok)
    self.disconnectBtn.setEnabled(ok)
    self.sendBtn.setEnabled(ok)
    if ok:
        self.statusBar.setText("Connected")
    else:
        self.statusBar.setText("Disconnected")

def on_connect(self):
    if self.worker and self.worker.isRunning():
        return
    host = self.hostEdit.text().strip()
    port = int(self.portEdit.text().strip() or "0")
    self.worker = SocketWorker(host, port,
auto_reconnect=self.autoReconnect.isChecked())
    # Worker signallari → GUI slotlari
    self.worker.connected.connect(lambda: (self._set_connected(True),
self.log.append("[OK] Connected")))
    self.worker.disconnected.connect(lambda: (self._set_connected(False),
self.log.append("[i] Disconnected")))
    self.worker.messageReceived.connect(lambda m: self.log.append(f"[SERVER]
{m}"))
    self.worker.error.connect(lambda e: self.log.append(f"[ERROR] {e}"))
    self.worker.status.connect(self.statusBar.setText)
    self.worker.start()

def on_disconnect(self):
    if self.worker:
        self.worker.auto_reconnect = False
        self.worker.stop()
        self.worker.wait(1500) # yumshoq to'xtash
        self._set_connected(False)
        self.log.append("[i] Manual disconnect")

def on_send(self):
    if not self.worker or not self.worker.isRunning():
        self.log.append("[WARN] Avval ulanib oling")
        return
    payload = self.msgEdit.text().encode('utf-8')
    if not payload:
        self.log.append("[WARN] Bo'sh xabar yuborilmaydi")
        return
    self.worker.queue_put(payload)

```

```

self.log.append(f"[YOU] {self.msgEdit.text()}")
self.msgEdit.clear()

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    w = ClientGUI()
    w.show()
    sys.exit(app.exec_())

```

Nimalar bu yerda hal qilingan?

- **Framing:** 4 baytli uzunlik-prefiks (!!).
- **GUI bloklanmaydi:** soket ishlari QThread ichida.
- **Status/reconnect:** holat QLabel orqali, ixtiyoriy **Auto reconnect** bilan.
- **Xatolar:** error signali orqali log'ga tushadi, status yangilanadi.
- **Yopish:** shutdown → close, stop() va wait() bilan yumshoq to'xtash.

Oqimni testlash bo'yicha tezkor chek-list

1. python server.py — echo-serverni ishga tushiring.
2. python client_gui.py — GUI'ni ishga tushiring.
3. **Connect** bosing → “Connected”.
4. Bir nechta xabar yuboring → serverdan katta harfda qaytadi.
5. Serverni to'xtating → GUI'da Disconnected va/ yoki **Auto reconnect** bo'lsa qayta urinishlar log'da ko'rinadi.
6. Qayta ishga tushirilgan serverga GUI avtomatik ulanadi.

Nazorat savollari

1. Nega TCP'da framing (masalan, uzunlik-prefiks) zarur va recv() nega xabarni bo'lib yuborishi mumkin?
2. sendall() va send() o'rtasidagi farq nimada? Qaysi holatda sendall() kerak bo'ladi?
3. PyQt'da nega soket ishlarini QThread (yoki QRunnable) ga chiqarish tavsiya etiladi?
4. “Auto reconnect” ni qanday strategiya bilan amalga oshirish ma'qul (timeout, backoff, jitter)?
5. GUI'da xatolik va holatlarni foydalanuvchiga ko'rsatishning uchta amaliy usulini sanang (log, status, dialog va h.k.).

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

7-mashg'ulot. Python ilovasini maxsus paket yordami.

O'quv savollari:

1. Python ilovasini maxsus paket yordamida yuklanuvchi fayl ko'rinishiga keltirish.

2. PyInstaller, cx_Freeze kabi paketlarning imkoniyatlari bilan tanishish.

3. Dasturiy ta'minotni turli operatsion tizimlarda sinovdan o'tkazish.

4. Yaratilgan yuklanuvchi faylni foydalanuvchi uchun qulay shaklda taqdim etish.

1. Python ilovasini maxsus paket yordamida yuklanuvchi fayl ko'rinishiga keltirish.

Ushbu bo'limda pythonda yaratilgan loyihalarning yuklanuvchi faylini hosil qilish bo'yicha bilim va ko'nikmalarga berib o'tiladi.

Dasturchi yaratgan dasturini kimgadir taqdim etishi uchun albatda butun bir loyihasini emas abalki butun loyihani bitta yuklanuvchi faylga birlashtirib, keyin taqdim etadi. Bunday bitta faylga birlashtirishning turli usullari mavjud.

Pythonning PYINSTALLER paketi va uning imkoniyatlari

PyInstaller paketi yordamida yuklanuvchi faylni hosil qilishni bir nechta qadamlarga bo'lish mumkin:

1-qadam: PyInstaller paketini kompyuterga o'rnatish.

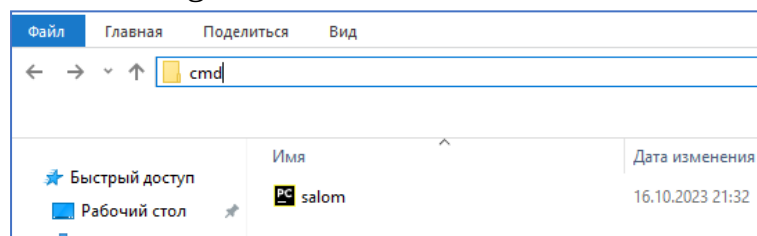
PyInstaller paketini kompyuterga o'rnatish uchun quyidagi buyruqni buyruklar satriga kiritiladi:

```
pip install pyinstaller
```

Shundan so'ng ushbu paketdan fodalanish mumkin bo'ladi.

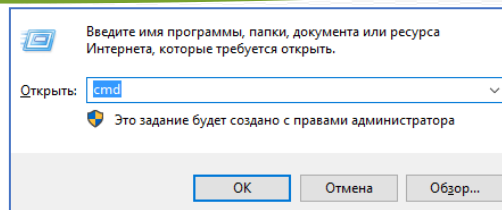
2-qadam: PyInstaller yordamida bajariladigan fayl hosil qilish.

Endi *PyInstaller* paketi yordamida Python tilida yozilgan loyihani bitta faylga jamlash kerak. Buning uchun project joylashgan papkaga kirish va buyruq qatoriga **CMD** buyrug'ini kiritib, enter tugmasini bosiladi:



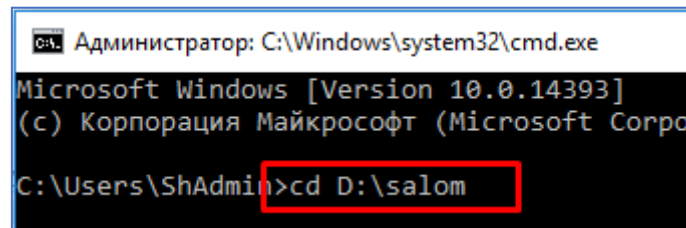
80-rasm. Buyruq qatoriga CMD buyrug'ini kiritish

Yoki ikkinchi usuli kompyuteringinzdan PUSK+R tugmasi bosiladi va hosil bo'lgan «**Выполнить**» oynasiga "CMD" buyrug'i kiritiladi.



81-rasm. «Выполнить» oynasi

Va hosil bo'lgan konsol oynasiga **cd** buyrug'i orqali kerakli papkaga kirish mumkin. Buning uchun **cd** va undan keyin Python skriptingiz saqlanadigan joy quyidagi ko'rinishda kiritiladi va enter tugmasi bosiladi va ko'rsatilgan kotalokga kiriladi:

82-rasm. **cd** buyrug'i orqali kerakli papkaga kirish

Keyin bajariladigan amal faylni yaratish uchun quyidagi shablondan to'g'ri foydalanish qilishdan iborat:

```
pyinstaller --onefile pythonFileName.py
```

Ushbu shablonda *pythonFileName* "**salom**" (va fayl kengaytmasi albatta .py) bo'lgani uchun bajariladigan faylni yaratish buyrug'i quyidagicha bo'ladi:

```
pyinstaller --onefile salom.py
```

ENTER tugmasini bosiladi va pyinstaller paketi avtomatik ravishda ushbu papka ichida dist papkasini hosil qiladi. Yuklanuvchi exe fayl esa ushbu papkaning ichida joylashgan bo'ladi. Endi ushbu exe faylni python dasturi o'rnatilmagan kompyuterlarda ham ishga tushirish mumkin.

Pythonning AUTO-PY-TO-EXE paketi va uning imkoniyatlari

Pythonda yaratilgan loyihalarni yagona .exe faylga yig'ishning yana bir usuli – bu auto-py-to-exe paketidan foydalangan holda judayam funksiyanal va visual tarzda amalga oshirish mumkin. Buning uchun ushbu paketni kompyuterga o'rnatish kerak bo'ladi.

auto-py-to-exe paketini kompyuterga o'rnatish. Kompyuterga python paketlarini o'rnatishning bir nechta usullari mavjud. Bullardan **pip**, **github** hamda **offline** holda o'rnatish mumkin. Odatda pip orqali o'rnatiladi. Chunki bu usul qulay hisoblanadi.

Pip orqali o'rnatish. *Auto-py-to-exe* paketini kompyuterga Pip orqali o'rnatish quyidagicha amalga oshiriladi: Birinchi navbatda “**Команда строка**” oynasi ochib

olinadi va u joyga pip install auto-py-to-exe buyrug'i kiritiladi va enter tugmasi bosiladi. Shundan so'ng avtomatik ravishda kompyuterga ushbu paket o'rnatiladi:

```
$ pip install auto-py-to-exe
```

GitHubdan o'rnatish. Ushbu paketni to'g'ridan-to'g'ri GitHubdan o'rnatish mumkin. GitHubdan *auto-py-to-exe* paketini o'rnatish uchun avval GitHub omborini klonlash kerak bo'ladi. U quyidagi kod orqali amalga oshiriladi:

```
git clone https://github.com/brentvollebrecht/auto-py-to-exe.git
```

Keyin auto-py-to-exe papkasiga cd buyrug'i orqali o'tish kerak.

```
$ cd auto-py-to-exe
```

Endi python buyrug'i orqali setup.py faylini ishga tushirish kerak.

```
$ python setup.py install
```

Quyidagi buyruq yordamida versiyani ham tekshirish mumkin:

```
auto-py-to-exe --version
```

```
C:\Users\ShAdmin>auto-py-to-exe --version
auto-py-to-exe 2.42.0
```

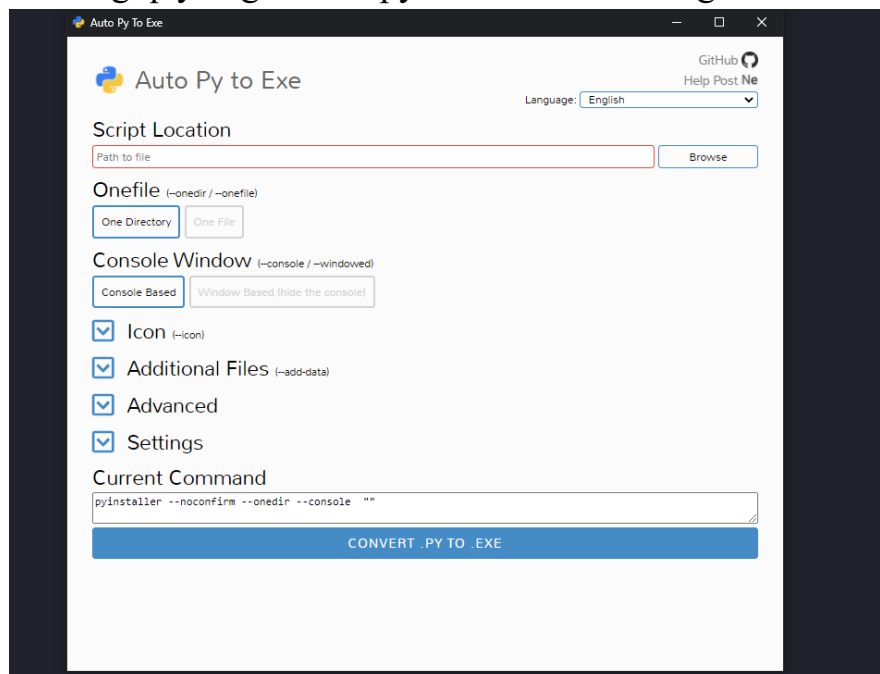
83-rasm. Paketning versiyasini bilish

Auto py to exe ning joriy versiyasi 2.42.0 va u endi kompyuterga o'rnatilgan.

Ilovani ishga tushirish. auto-py-to-exe ilivasini ishga tushirish uchun terminalda quyidagi buyruqni bajarish kerak bo'ladi:

```
$ auto-py-to-exe
```

Shundan so'ng quyidagi "Auto py to exe" dasturi ishga tushadi:

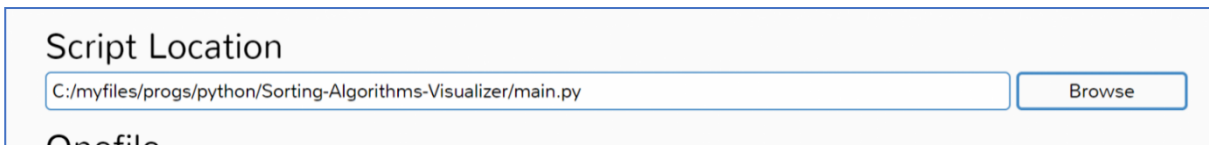


84-rasm. avto-py-to-exe GUI interfeysi

Endi ushbu interfeysdan .py faylni yuklanuvchi .exe fayliga aylantirish uchun foydalaniladi.

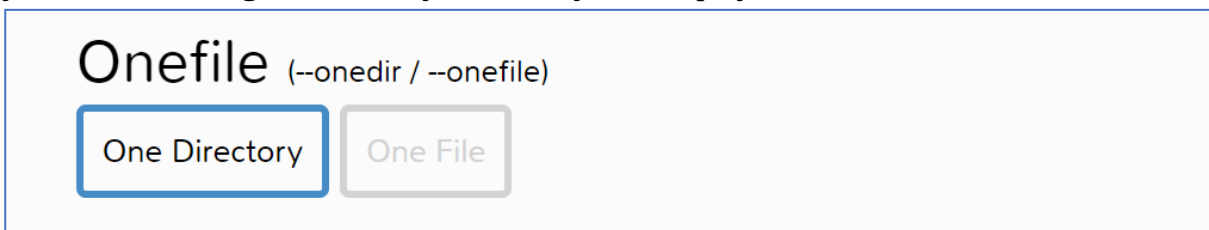
Konvertatsiya jarayoni. 1-qadam. Fayl joylashuvini qo'shish. Python faylini .exe fayliga o'zgartirish uchun avval uning yo'lini ko'rsatish kerak bo'ladi. Buning

uchun konvertatsiya qilinadigan faylning manziliga o'tiladi va keyin yo'lni ko'rsatib, qo'shib qo'yiladi:



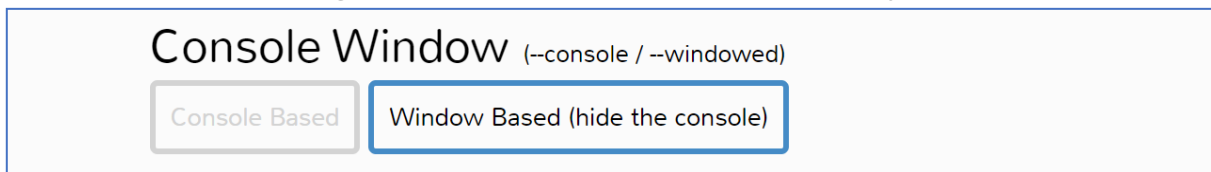
85-rasm. Faylni yo'lni ko'rsatish

2-qadam: "One Directory" yoki "One File" ni tanlash. Interfeysda "One Directory" yoki "One File" ni tanlash imkoniyati mavjud. Odatda ko'pchilik katta Python loyihalar bir nechta fayllardan tashkil topgan bo'ladi, shuning uchun "One Directory" ni tanlash afzalroq. Ushbu parametr barcha kerakli fayllar bilan papkani yaratadi, shuningdek **.exe** faylini ham yaratib qo'yadi.



86-rasm. "One Directory" yoki "One File" ni tanlash

3-qadam. "Console Based" yoki "Windows Based" ni tanlash. Yuqoridagi amallarni bajarilganidan so'ng dastur turini tanlash kerak: **Console** yoki **Windows** asoslangan. Agar "Windows Based" ni tanlansa, bu dasturning barcha konsol natijalarini yashiradi. Agar yaratilgan dastur konsol chiqishiga asoslangan holda ishlaydigan bo'lsa, unda "Console Based" ni tanlash maqsadga muvofiq bo'ladi. Agar dasturda GUI ilovasi bo'lsa yoki foydalanuvchiga konsol chiqishini ko'rsatish shart bo'lmasa, unda "Windows Based" ni tanlash tavsiya qilinadi. Bu yerda ikkinchi ikkinchi variantni tanlangan, chunki ushbu dasturda GUI mavjud.



87-rasm. Konsolga asoslangan yoki oynaga asoslangan

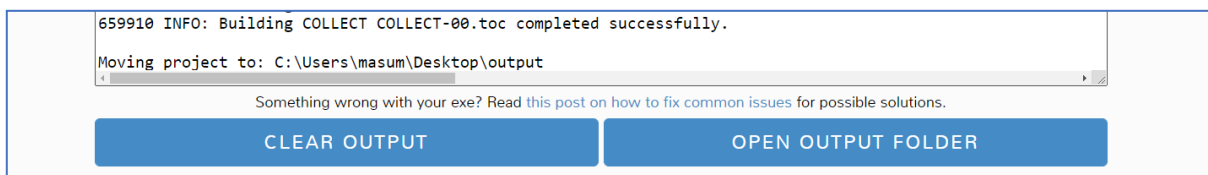
4-qadam. Konvertatsiya qilish. Yuqoridagi barcha amallarni bajarilganidan keyin qolgan sozlamalari masalan, piktogramma, qo'shimcha fayllar va hk. ni o'z standart holatida qoldirib, konvertatsiya qilish mumkin. Buning uchun faqat **CONVERT .PY TO .EXE** tugmasini bosish kifoya.



88-rasm. Konvertatsiya qilish uchun tugma.

Jarayonni yakunlash uchun biroz kutishingiz kerak bo'ladi.

Open output folder. Konvertatsiya jarayoni tugaganidan so'ng, "Open output folder" tugmasini bosish orqali konvertatsiya qilingan fayl joylashgan papkani ochish mumkin.

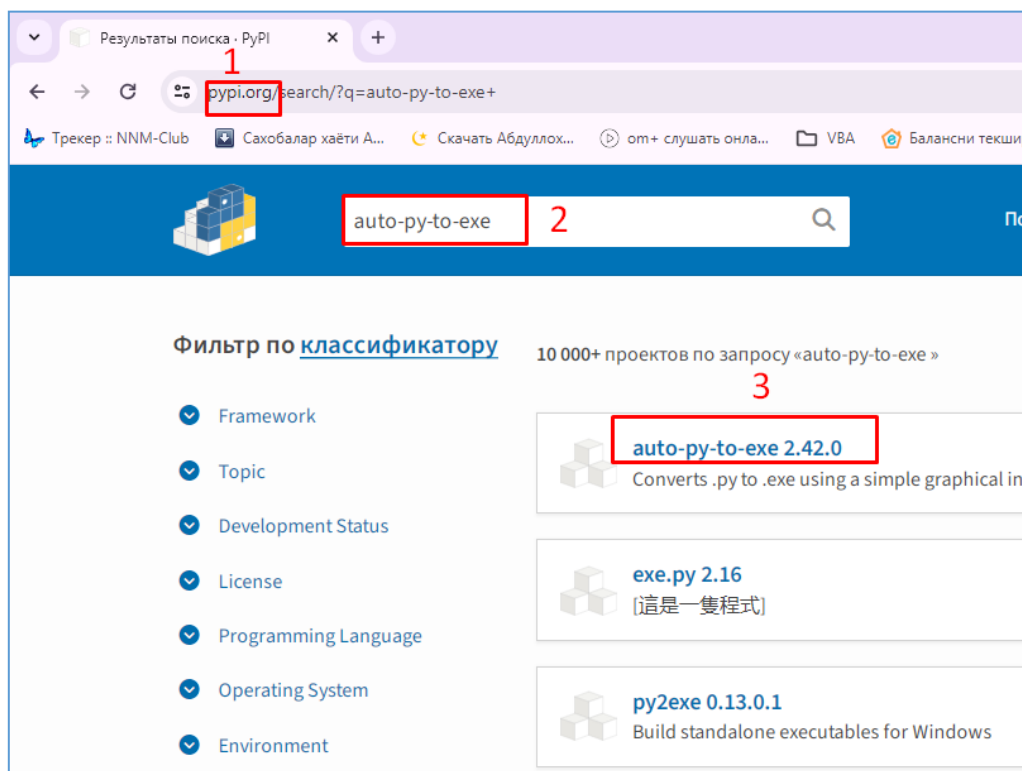


89-rasm. Open output folder tugmasi

Shundan so'ng .exe fayl joylashgan papka ochiladi. Endi uni Python-ni o'rnatmasdan boshqa kompyuterlarda ham bemaolol ishga tushirib foydalanish mumkin.

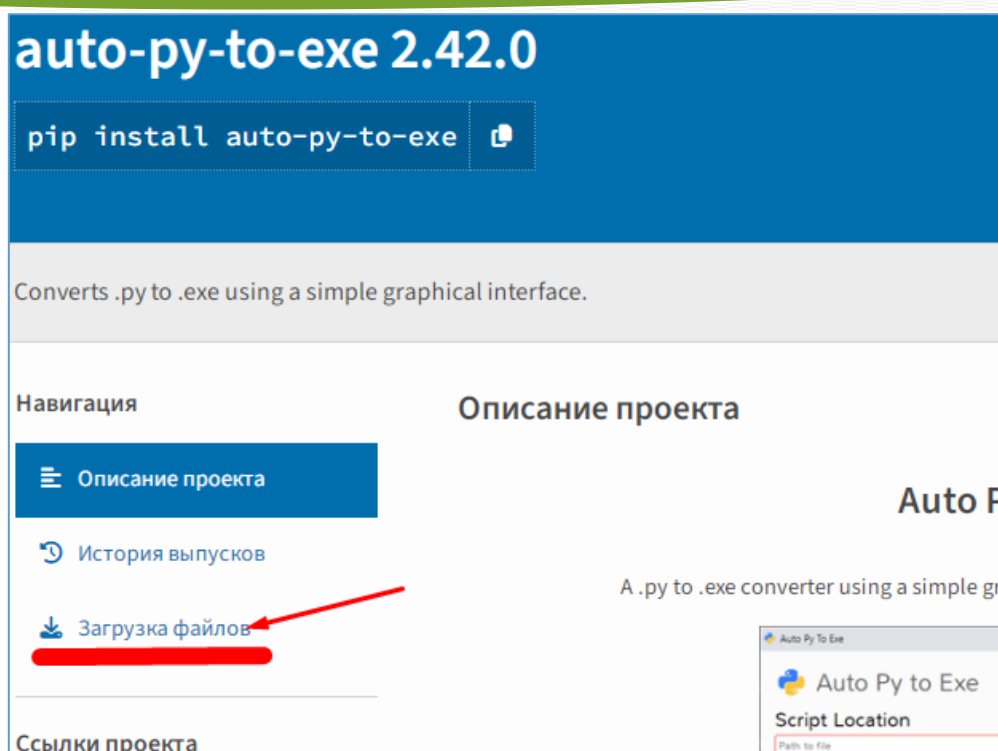
Python paketlarini offline holatda kompyuterga o'rnatish

Ba'zi hollarda internet tarmog'iga ulanmagan kompyuterlarga ham python paketlarini o'rnatishga zarurat paydo bo'lib qoladi. Bunday vaziyatlarda kompyuterga pythonning kerakli paketini olib kelib, o'rnatiladi. Buning uchun boshqa bir internetga ulangan kompyuterdan **pypi.org** saytiga kirib, kerakli paketni qidirib topiladi va yuklab olinadi. Masalan:



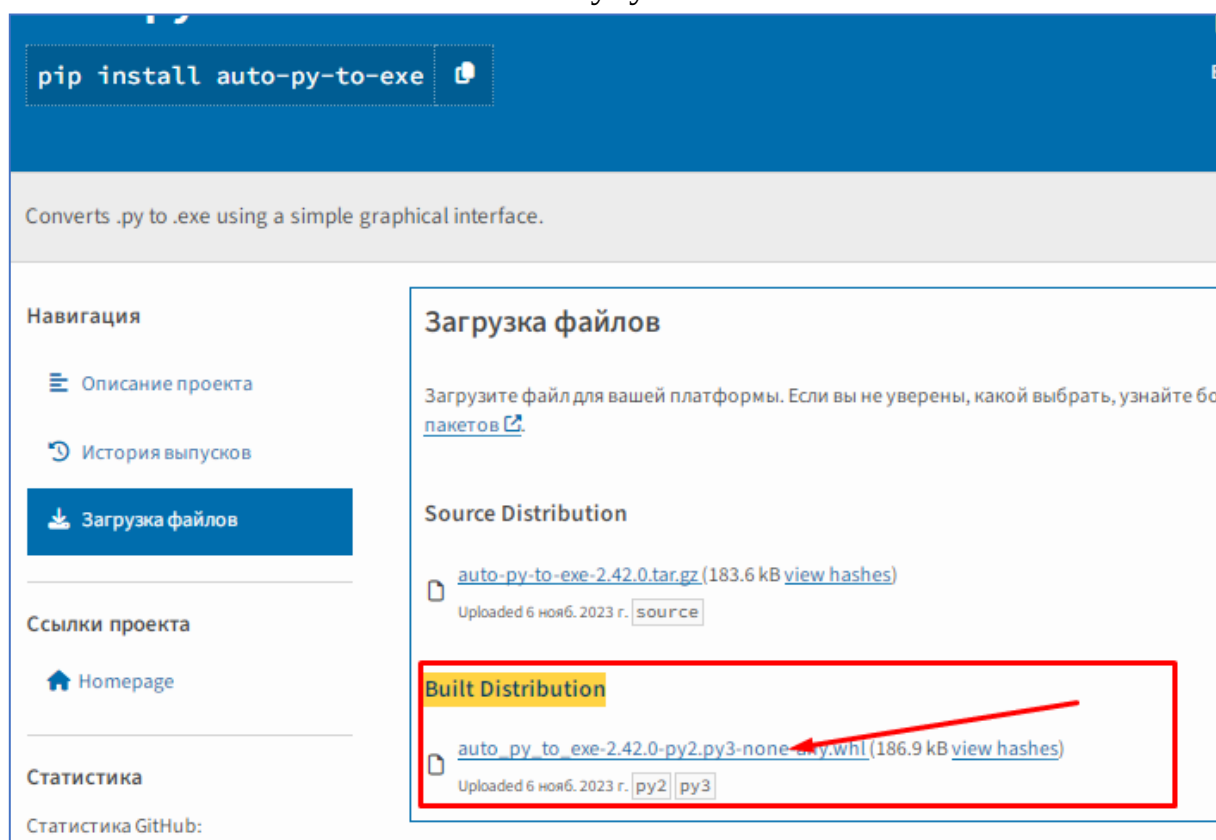
90-rasm. Pypi.org sayti bilan ishlash

Ushbu kerakli paketni topib, uni tanlanganda paket haqida ko'plab kerakli ma'lumotlar beruvchi oyna ochiladi. U oynada paketni yuklab olish tugmasi ham mavjud va yuklab olish "Загрузка файлов" bo'limini tanlanlash orqali amalga oshiriladi.



91-rasm. PyPI.org saytidan kerakli paketni yuklab olish

Shundan so‘ng paketni yuklab olish uchun “**Загрузка файлов**” oynasi ochiladi. U yerdan “**Built Distribution**” bo‘limidagi faylni yuklab olish uchun maxsus ssilkani bosiladi va avtomatik ravishda fayl yuklab olinadi.



92-rasm. Kerakli paketni yuklab olish

Ushbu yuklab olingan faylni boshqa bir kompyuterga ko‘chirib o‘tkaziladi. Endi ushbu kerakli yukanuvchi faylni kompyuterga o‘rnatish kerak. Buning uchun fayl

oddiy .exe fayllar kabi emas balki maxsus usulda o'rnatilishi kerak bo'ladi. Ushbu amal "**py -m pip install file name**" shabloni yordamida amalga oshiriladi. Bizning misolimizda file Downloads papkasida joylashganligi uchun terminaldan osha papkaga kirib borib, undan so'ng o'rnatish kodini teriladi va enter tugmasi bosish orqali o'rnatish amalga oshiriladi.

```
Microsoft Windows [Version 10.0.14393]
(c) Корпорация Майкрософт (Microsoft Corporation), 2016. Все права защищены.

C:\Users\ShAdmin\Downloads>py -m pip install auto py to exe-2.42.0-py2.py3-none-any.whl
```

93-rasm. Kerakli paketni lokal holatda o'rnatish

Shundan so'ng, agarda hech qanday xatolik ro'y bermasa, paket kompyuterga muvoffaqiyatli o'tnatiladi.

Barcha paketlar kompyuterga lokal holatda o'rnatilishi mumkin.

2) pyinstaller va cx_freeze imkoniyatlari

Minimal namuna (test qilinadigan kichik dastur)

```
app.py:
import sys
def main():
    print("Hello from packaged app!")
    if len(sys.argv) > 1:
        print("Args:", sys.argv[1:])
if __name__ == "__main__":
    main()
```

PyInstaller (eng ommabop, juda moslashuvchan)

O'rnatish

```
pip install pyinstaller
```

Tez start (CLI ilova)

```
pyinstaller --onefile app.py
# dist/app(.exe) yaratiladi
```

GUI rejim (Windows'da konsol oynasiz)

```
pyinstaller --onefile --noconsole --name MyApp app.py
```

Ma'lumot/fayllarni qo'shish (rasmlar, JSON va h.k.)

Eslatma: --add-data ajratgich Windows'da ;, macOS/Linux'da :.

```
# Windows
pyinstaller --onefile --add-data "assets\logo.png;assets" app.py
# macOS/Linux
pyinstaller --onefile --add-data "assets/logo.png:assets" app.py
```

Kodni ichida nisbiy yo'lni topish:

```
import sys, os
def resource_path(rel):
    base = getattr(sys, "_MEIPASS", os.path.dirname(__file__))
    return os.path.join(base, rel)
logo_path = resource_path("assets/logo.png")
```

Maxsus .spec bilan ilg'orroq sozlash

app.spec (asosiy qismlar):

```
# pyinstaller app.spec
block_cipher = None
a = Analysis(['app.py'],
             datas=[('assets/logo.png', 'assets/logo.png')],
             hiddenimports=['pkgutil'], # kerak bo'lsa
             ...)
exe = EXE(a.pure, a.zipped_data, a.scripts,
         name='MyApp',
         console=False) # True bo'lsa konsol chiqadi
```

Afzalliklar

- --onefile (bitta exe) yoki onedir (papka) rejimlari
- **Hidden import** va **hooklar** orqali murakkab kutubxonalarni (PyQt, requests, numpy, ...) to'g'ri yig'ish
- UPX siqish (agar o'rnatilgan bo'lsa): --upx-dir
- Mac'da .app bundle yaratadi, Linux'da ELF

Tipik muammolar (va yechimlar)

- **Yashirin importlar** topilmaydi → --hidden-import yoki hiddenimports=[] ga qo'shing
- **Qt plugin** topilmaydi → --add-data bilan platforms papkasini qo'shing (PyQt/PySide)
- **Antivirus false-positive** → exe'ni **imzolash** (Windows), yoki fayl nomi/hususiyatlarini o'zgartirish
- **Katta fayl** → onedir yengilroq; UPX qo'llash; keraksiz kutubxonalarni chiqarib tashlash

cx_Freeze (o'rnatkichlar bilan yaxshi integratsiya)

O'rnatish

```
pip install cx-Freeze
```

Setup skripti

setup.py:

```
from cx_Freeze import setup, Executable

build_options = {
    "packages": [],
    "excludes": [],
    "include_files": [("assets/logo.png", "assets/logo.png")],
}

executables = [Executable("app.py", base="console", target_name="MyApp")]

setup(
    name="MyApp",
    version="1.0.0",
    description="Sample app",
    options={"build_exe": build_options},
    executables=executables,
)
```

Build

```
python setup.py build
# natija: build/ .../ MyApp(.exe)
```

Afzalliklar

- setup.py orqali bildirishnoma, resurslar boshqaruvi
- MSI o'rnatkich (Windows) bilan integratsiya (bdist_msi)
- PyInstaller bilan yechilmaydigan ayrim import yo'llari/muammolarda barqaror bo'lishi mumkin

Tipik muammolar

- Qt/SDL/... kabi native kutubxonalar yo'llari → include_files bilan qo'shish
- VCRuntime DLL'lari → yangi Python distributsiyasi bilan odatda to'g'ri keladi, lekin ba'zan vcruntime140.dllni ham qo'shish kerak bo'ladi

3) TURLI OPERATSION TIZIMLARDA SINOVDAN O'TKAZISH

Asosiy qoida: *Python paketlari ko'pincha cross-compile qilmaydi.* Har bir OS uchun **o'sha OSda** build qiling va sinang.

Test strategiyasi (bosqichma-bosqich)

1. **Toza muhit:** har OS'da venv yarating, faqat kerakli paketlarni o'rinating.

2. **Build:** PyInstaller/cx_Freeze bilan mos binarni yarating.

3. **Run-time test:**

- CLI: --help, oddiy buyruqlar, fayl I/O sinovlari
- GUI: ochilish va yopilish, menyular, "open/save file", net funksiyalar

4. **Sistema bog'liqligi:**

- Windows: Visual C++ Redistributable kerakmi? (ko'pincha Python o'zi bilan olib keladi)
- macOS: codesign, notarization, Gatekeeper sinovi
- Linux: glibc versiyasi (ko'pincha eski distro'da build qiling → kengroq moslik), ldd bilan dependensiyalar

5. **Avtomatlashtirish** (ixtiyori): GitHub Actions yoki GitLab CI'da **matrix build:**

- runs-on: [windows-latest, macos-latest, ubuntu-latest]
- har birida pip install -r requirements.txt → build → artefakt sifatida yuklab qo'yish

6. **Qo'lda qurilma testlari:**

- FullHD/4K ekranlar, scaling (125%, 150%)

- Tarmoq cheklovlari (firewall off/on)
- Foydalanuvchi huquqlari (admin vs oddiy user), yozish ruxsatlari

Tezkor buyruqlar

Windows

```
# DLL tekshirish
where MyApp.exe
# Process tez diagnostika
tasklist | findstr MyApp
# Event Viewer (Application) loglarini tekshirish
```

macOS

```
# karantin bit'lari va imzo
spctl --assess --type execute dist/MyApp.app
codesign -dv --verbose=4 dist/MyApp.app
```

Linux

```
ldd dist/MyApp | grep "not found"      # yo'q kutubxonalar
ldd dist/MyApp | sort | uniq
```

4) Foydalanuvchi uchun qulay tarzda taqdim etish (installer/portable)

Windows

- **Portable ZIP:** --onefile exe yoki onedir papkani zip qilib tarqating.
- **MSI/EXE o'rnatkich:**
 - **cx_Freeze:** python setup.py bdist_msi
 - **Inno Setup** yoki **NSIS:** PyInstaller dist¥MyApp¥* dan installer yasang (yorliqlar, Uninstall, Start Menu).
- **Imzolash** (tavsiya): signtool.exe sign /tr http://timestamp ... MyApp.exe
Bu SmartScreen ogohlantirishlarini kamaytiradi.

macOS

- **.app bundle** (PyInstaller avtomatik yaratadi), tarqatish:
 - .dmg (Disk image) yoki .pkg
- **codesign** va **notarize** (Apple Developer ID talab qilinadi):
 - imzolash → notarizatsiya → stapling → Gatekeeper'da tekshirish
- Gatekeeper uchun foydalanuvchiga ko'rsatma (birinchi ishga tushirishda "Open Anyway" bo'lishi mumkin)

Linux

- **AppImage** (portable'ga o'xshaydi) yoki **.deb/.rpm** (distroga xos)
- Minimal talablar: glibc mosligi; kerakli .solar paketga kiritilgan bo'lsin yoki distro repolaridan o'rnatilishi ko'rsatilgan bo'lsin
- Sodaroq yo'l: tar.gz (install skript bilan)

Avto-yangilash (ixtiyoriy)

- Windows: **Squirrel.Windows**, **WinSparkle**
- macOS: **Sparkle**

- Ochiq internetga chiqadigan ilovalar uchun versiya tekshiruvchi kichik modul (JSON manba) yozib, "Update available" dialog ko'rsatish.

Foydalanuvchi tajribasi (UX) uchun tavsiyalar

- **Versiya va build raqami** ilova "About" oynasida ko'rinsin
- **Loglar:** ~/.myapp/logs yoki %LOCALAPPDATA%\MyApp\logs
- **Birlamchi sozlama fayllari:** foydalanuvchi papkasida saqlansin (admin huquqsiz)
- **Bir bosishda o'rnatish:** "Next → Next → Finish", default yo'llar
- **Uninstall** imkoniyati aniq ko'rinsin
- **Rasmiy qo'llanma** (PDF yoki onlayn) va "FAQ" linkini installerda ham ko'rsating

Qo'shimcha demo: PyInstaller bilan PyQt5 GUI'ni yig'ish

```
gui.py (oddiy oynacha):
import sys
from PyQt5 import QtWidgets

class Main(QtWidgets.QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("My GUI")
        btn = QtWidgets.QPushButton("Hello")
        btn.clicked.connect(lambda:
QtWidgets.QMessageBox.information(self, "Hi", "Packaged!"))
        lay = QtWidgets.QVBoxLayout(self); lay.addWidget(btn)

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    w = Main(); w.show()
    sys.exit(app.exec_())
```

Yig'ish (Windows misoli):

```
pyinstaller --onefile --noconsole --name MyGUI gui.py
# PyQt pluginlari kerak bo'lsa:
# pyinstaller --onefile --noconsole --name MyGUI --add-data "venv\Lib\site-
packages\PyQt5\Qt\plugins\platforms;PyQt5\Qt\plugins\platforms" gui.py
```

Nazorat savollari:

1. Pyinstaller paketining vazifasi qanday?
2. Pyinstaller paketini o'rnatish qanday amalga oshiriladi?
3. Pyinstaller paketidan qanday foydalaniladi?
4. Python paketlarini kompyuterga qanday usullardan foydalanib, o'rnatish mumkin?
5. Kerakli paketlarni lokal holda o'rnatish qanday amalga oshiriladi?
6. Kompyuterga paketlarni lokal holada o'rnatishning qanday afzalliklari bor?
7. Auto-py-to-exe paketi kompyuterga qanday o'rnatiladi?
8. Auto-py-to-exe paketining qanday sozlamalari mavjud?

MA'RUZA MASHG'ULOT UCHUN O'QUV MATERIALLARI

4-Mavzu: Python yordamida mashinali o'qitish asoslari

1-mashg'ulot. Mashinali o'qitishga kirish.

O'quv savollari:

1. Mashinali o'qitish haqida umumiy tushuncha.
2. Sun'iy intellekt va ML farqi.
3. Jupyter Notebook bilan ishlash.

Mashinali o'qitish haqida umumiy tushuncha

Mashinali o'qitish (**Machine Learning – ML**) — bu **sun'iy intellekt**ning bir bo'lagi bo'lib, unda kompyuterlar aniq dasturlashsiz, ya'ni har bir qadamini qo'lda yozib chiqmasdan, mavjud **tajriba va ma'lumotlardan o'rganib**, xulosa chiqarish va qaror qabul qilishni o'rganadi.

Mashinali o'qitishning mohiyati

- Oddiy dasturlashda: dasturchi qoidalarini yozadi → kompyuter uni bajaradi.
- Mashinali o'qitishda: dasturchi qoidalarini yozmaydi, balki **ma'lumotlar** beradi → kompyuter o'zi qoidalarini “o'rganadi”.

Masalan, agar bizga mushuk va it rasmlari berilsa, dastlab kompyuter farqini bilmaydi. Lekin ko'p sonli mushuk va it rasmlarini ko'rsatib, ularning to'g'ri javobini bersak, dastur asta-sekin ularni **farqlash qoidalarini o'zi topadi**.

Asosiy tushunchalar

1. **Ma'lumotlar (Data)** – mashinali o'qitishning asosiy “yoqilg'isi”. Qancha ko'p va sifatli ma'lumot bo'lsa, model shuncha yaxshi o'rganadi.
2. **Model** – kompyuterning o'rganilgan matematik ifodasi, u yordamida yangi ma'lumotlar ustida bashorat qilish mumkin.
3. **Trening (o'qitish)** – modelga ma'lumot berib, uni moslashtirish jarayoni.
4. **Test** – o'rgatilgan modelni yangi, ilgari ko'rmagan ma'lumotlarda sinash jarayoni.

Mashinali o'qitish turlari

1. Nazoratli o'qitish (Supervised Learning)

Modelga **kirish ma'lumotlari + to'g'ri javoblar** beriladi.

Vazifa: kelajakda yangi kirish uchun to'g'ri javobni topish.

Misollar: narxlarni bashorat qilish, elektron pochtada spam aniqlash.

2. Nazorat qilinmagan o'qitish (Unsupervised Learning)

Modelga faqat **kirish ma'lumotlari** beriladi, javob yo'q.

Vazifa: yashirin tuzilmani topish, guruhlash.

Misollar: mijozlarni segmentlarga ajratish, katta matnlarda mavzularni aniqlash.

3. Mukofotli o'qitish (Reinforcement Learning)

Agent (dastur) **harakat qiladi**, natijada **mukofot** yoki **jarima** oladi.

Vazifa: uzoq muddatda eng katta mukofotni to'plash.

Misollar: o'yin o'ynaydigan sun'iy intellekt (shaxmat, Go), robotlarni boshqarish.

Mashinali o'qitishning qo'llanilishi

Tibbiyotda – kasalliklarni aniqlash, tibbiy tasvirlarni tahlil qilish.

Moliyada – firibgarlikni aniqlash, kredit reytingini baholash.

Texnologiyada – ovoz va yuzni tanish, avtonom mashinalar.

Marketingda – foydalanuvchilarga mos reklama ko'rsatish.

Afzalliklari

Katta hajmdagi ma'lumotni tezda tahlil qiladi.

Inson uchun murakkab bo'lgan yashirin bog'liqliklarni topadi.

O'zini yangilab, vaqt o'tishi bilan yanada aniqroq natija beradi.

Kamchiliklari

Juda ko'p **ma'lumot** talab qiladi.

Natija sifatini **ma'lumot sifati** belgilaydi.

Ba'zan model qarorining **nega bunday chiqqanini tushuntirish qiyin** bo'ladi.

Sun'iy intellekt va ML farqi

Sun'iy intellekt (Artificial Intelligence – AI) va Mashinali o'qitish (Machine Learning – ML) ko'pincha chalkashib ketadi, lekin ular bir xil emas.

1. Sun'iy intellekt (AI)

Ta'rif: Kompyuter yoki dasturga inson aqliga xos bo'lgan imkoniyatlarni berish texnologiyasi.

Maqsad: Mashinalar odam kabi **fikrlash, tushunish, qaror qabul qilish va muammolarni hal qilish** qobiliyatiga ega bo'lishi.

Misollar:

Siri, Google Assistant (ovozli yordamchi)

Avtonom mashinalar (Tesla Autopilot)

Sun'iy intellektli shaxmat o'ynaydigan dasturlar

2. Mashinali o'qitish (ML)

Ta'rif: AIning **kichik sohasi**, unda mashina **ma'lumotlardan mustaqil o'rganib**, vaqt o'tishi bilan yaxshilanishi mumkin.

Maqsad: Dastur qoidalarini qo'lda yozmasdan, **tajriba va ma'lumotlar orqali xulosa chiqarishi**.

Misollar:

Spam email'larni aniqlash

Narxlarni bashorat qilish

Suratdagi yuzni tanish

3. Asosiy farqlar

Xususiyat	Sun'iy intellekt (AI)	Mashinali o'qitish (ML)
Tushuncha	Inson aqliga o'xshash har qanday kompyuter tizimi	AI ichidagi usul: ma'lumotlardan o'rganish
Maqsad	Mashinani "aqlli" qilish	Mashinani "o'rgatish"
Metodlar	Qoidalar, ekspert tizimlari, ML, Deep Learning va b.	Statistik usullar, algoritmlar, neyron tarmoqlar
Qamrovi	Kengroq (ML, DL, NLP ham ichida)	Aining kichik sohasi
Misol	Avtonom mashina (AI)	Mashinaning svetoforni taniy olishi (ML)

4. AI va ML o'xshashligi

- Ikkalasi ham **aqlli tizimlar yaratishga xizmat qiladi**.
- ML → Aining ichida joylashgan.
- AI katta "soyabon" bo'lsa, ML uning ichidagi **asosiy yo'nalishlardan biri** hisoblanadi.

Qisqa qilib aytganda:

- **AI** – bu mashinalarni aqlli qilish bo'yicha umumiy yo'nalish.
- **ML** – bu AIga erishish uchun ishlatiladigan **asosiy vositalardan biri**.

Jupyter Notebook nima?

Jupyter Notebook — bu dasturchilar, ilmiy tadqiqotchilar va talabalarga mo'ljallangan **interaktiv muhit** bo'lib, unda kod yozish, uni bajarish, natijani ko'rish, matn yozish va grafiklarni chiqarish mumkin. Asosan **Python** dasturlash tilida ishlatiladi, lekin boshqa tillarni ham qo'llab-quvvatlaydi.

Asosiy imkoniyatlari

- Kodni kichik qismlarga (hujayralar) bo'lib yozish va alohida bajarish.
- Natijani darhol hujayraning ostida ko'rish.
- Matn, formulalar (LaTeX), rasm va grafiklarni qo'shish.
- Data analiz, mashinali o'qitish, statistika va ilmiy hisoblashlarda juda qulay.

O'rnatish usullari

1. **Anaconda orqali:** Anaconda paketini o'rnatganingizda Jupyter ham avtomatik o'rnatiladi.

2. **pip orqali:**

3. **pip install notebook**

Ishga tushirish

Terminal yoki CMD oynasiga:

jupyter notebook yoziladi.

Brauzerda yangi sahifa ochiladi va siz kod yozishga kirishishingiz mumkin.

Asosiy qismlari

- **Hujayra (Cell)** – kod yoki matn yoziladigan blok.

- *Code cell* – Python kodini yozish uchun.
- *Markdown cell* – matn, izoh, formula yozish uchun.
- **Toolbar** – hujayralarni qo'shish, o'chirish, bajarish tugmalari joylashgan.

Afzalliklari

- Kodingizni bosqichma-bosqich yozib, tekshirib borasiz.
- Natija (jadval, grafik) darhol chiqadi.
- Ilmiy ishlar, taqdimot va laboratoriya ishlari uchun juda qulay.

Nazorat savollari

1. Mashinali o'qitishning mohiyati nimada va u oddiy dasturlashdan nimasi bilan farq qiladi?
2. Sun'iy intellekt tushunchasiga ta'rif bering va uning asosiy maqsadini izohlang.
3. Mashinali o'qitishning asosiy turlarini sanab, ular o'rtasidagi farqlarni tushuntiring.
4. Sun'iy intellekt va Mashinali o'qitish o'rtasidagi bog'liqlikni izohlang.
5. Jupyter Notebook nima va u dasturchilar uchun qanday qulayliklar yaratadi?
6. Jupyter Notebook'da hujayra (Cell) turlari va ularning vazifalarini tushuntiring.
7. Mashinali o'qitishning real hayotdagi qo'llanilishiga kamida ikki misol keltiring.

MA'RUZA MASHG'ULOT UCHUN O'QUV MATERIALLARI

5-Mavzu: Mashinali o'qitishda NumPy va Pandas paketlarining qo'llanilishi.

1-mashg'ulot. NumPy va Pandas bilan ishlash.

O'quv savollari:

1. NumPy kutubxonasi asoslari (massivlar bilan ishlash).
2. Pandas kutubxonasi asoslari (ma'lumotlarni qayta ishlash).

NumPy kutubxonasi asoslari (massivlar bilan ishlash)

NumPy kutubxonasi **Python dasturlash tilida ilmiy va texnik hisoblashlar** uchun eng ko'p qo'llaniladigan kutubxonalardan biridir. Asosiy afzalligi shundaki, u **massivlar (arrays)** bilan ishlash imkonini beradi. Massivlar Python'dagi oddiy listlarga o'xshaydi, lekin ular tezroq, kamroq xotira egallaydi va matematik amallarni samarali bajaradi. Quyida dars o'tishda ishlatishingiz mumkin bo'lgan asosiy tushunchalar va kod namunalarini keltiraman.

NumPy (Numerical Python) — bu **massivlar** bilan ishlash uchun ishlab chiqilgan kutubxona. Python'dagi oddiy list ma'lumot turi orqali ko'p sonli ma'lumotlar bilan ishlash sekin kechadi va ko'p xotira talab qiladi. NumPy esa maxsus optimallashtirilgan massivlar yordamida tezlikni bir necha barobar oshiradi.

NumPy kutubxonasi orqali:

Bir va ko'p o'lchovli massivlar (array) yaratish,
Matematik va statistik amallarni bajarish,
Matritsalar bilan ishlash,
Ma'lumotlarni indekslash va kesish (slicing),

1. NumPy kutubxonasini chaqirish

```
import numpy as np
```

2. Massiv yaratish

```
# Oddiy massiv yaratish
arr = np.array([1, 2, 3, 4, 5])
print("Oddiy massiv:", arr)
# 2 o'lchamli massiv (matritsa) yaratish
mat = np.array([[1, 2, 3], [4, 5, 6]])
print("2D massiv:\n", mat)
```

3. Maxsus massivlar yaratish

```
# Nolga to'ldirilgan massiv
zeros = np.zeros((3, 4)) # 3 qator, 4 ustun
print("Nollar massivi:\n", zeros)
# Birga to'ldirilgan massiv
ones = np.ones((2, 5))
print("Birlar massivi:\n", ones)

# Tasodifiy sonlardan massiv
rand = np.random.rand(3, 3)
print("Tasodifiy massiv:\n", rand)
```

```
# Ketma-ket sonlar
seq = np.arange(0, 10, 2) # 0 dan 10 gacha, qadam 2
print("Ketma-ket sonlar:", seq)
```

4. Massiv o'lchamlari va shakli

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print("O'lchamlar soni:", arr.ndim) # necha o'lchamli
print("Shakli:", arr.shape) # qator va ustunlar soni
print("Elementlar soni:", arr.size) # jami elementlar soni
print("Ma'lumot turi:", arr.dtype) # element turi
```

5. Indeks va slicing

```
arr = np.array([10, 20, 30, 40, 50])
print("Birinch element:", arr[0])
print("Oxirgi element:", arr[-1])
print("Kesish (2 dan 4 gacha):", arr[1:4])

mat = np.array([[1, 2, 3], [4, 5, 6]])
print("Element [0,1]:", mat[0, 1]) # 1-qator, 2-ustun
```

6. Matematik amallar

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

print("Qo'shish:", a + b)
print("Ayirish:", b - a)
print("Ko'paytirish:", a * 2)
print("Bo'lish:", b / a)

print("Kvadrat ildiz:", np.sqrt(a))
print("O'rtacha:", np.mean(b))
print("Eng katta:", np.max(b))
print("Eng kichik:", np.min(b))
```

7. Matritsalar bilan ishlash

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Matritsalar ni qo'shish
print("Qo'shish:\n", A + B)

# Matritsa ko'paytirish
print("Ko'paytirish:\n", np.dot(A, B))

# Transponatsiya
print("Transponatsiya:\n", A.T)
```

8. Amaliy misol: O'rtacha baho hisoblash

```
# Talabalar ning baholari
grades = np.array([80, 75, 90, 60, 85])
print("Baholar:", grades)

# O'rtacha baho
```

```

avg = np.mean(grades)
print("O'rtacha baho:", avg)

# Eng yuqori va eng past baho
print("Eng yuqori:", np.max(grades))
print("Eng past:", np.min(grades))

```

Pandas kutubxonasi asoslari (ma'lumotlarni qayta ishlash)

Pandas — bu Python dasturlash tilida **jadval ko'rinishidagi ma'lumotlar (DataFrame)** bilan **ishlash** uchun eng mashhur kutubxona. U ma'lumotlarni o'qish, yozish, filtrlash, tahrirlash, statistik tahlil qilish va vizualizatsiyaga tayyorlashda juda qulay.

Pandas — bu **jadval ko'rinishidagi ma'lumotlarni (DataFrame)** qayta ishlash uchun mo'ljallangan kutubxona. Pandas aslida NumPy ustiga qurilgan bo'lib, undan foydalanishda ma'lumotlar tahlili yanada qulaylashadi. Agar NumPy massivlar bilan ishlashda asosiy vosita bo'lsa, Pandas ma'lumotlarni **jadval ko'rinishida boshqarish** imkonini beradi.

Pandas yordamida:

- Ma'lumotlarni turli fayllardan (CSV, Excel, SQL va boshqalar) o'qish va yozish,
- Jadvaldagi ustun va qatorlar bilan ishlash,
- Filtrlash, tartiblash va guruhlash,
- Statistik hisob-kitoblar qilish,
- Ma'lumotlarni tozalash va tahlil qilish mumkin.

1. Pandas kutubxonasini chaqirish

```
import pandas as pd
```

2. Asosiy ma'lumot strukturasi

Pandas'da ikki asosiy obyekt mavjud:

- **Series** — bitta ustunli massiv (bir o'lchamli)
- **DataFrame** — ko'p ustunli jadval (ikki o'lchamli)

```

# Series
s = pd.Series([10, 20, 30, 40])
print("Series:\n", s)

# DataFrame
data = {
    "Ism": ["Ali", "Vali", "Laylo", "Aziz"],
    "Yosh": [20, 22, 19, 21],
    "Baholar": [85, 90, 78, 92]
}
df = pd.DataFrame(data)
print("DataFrame:\n", df)

```

3. Fayllardan ma'lumot o'qish va yozish

```

# CSV faylni o'qish
df = pd.read_csv("studentlar.csv")

```

```
# Excel faylni o'qish
df_excel = pd.read_excel("data.xlsx")

# Ma'lumotlarni CSV faylga yozish
df.to_csv("yangi_studentlar.csv", index=False)
```

4. DataFrame haqida umumiy ma'lumot olish

```
print(df.head())      # dastlabki 5 qator
print(df.tail())      # oxirgi 5 qator
print(df.info())      # ustunlar va turlari
print(df.describe())  # statistik ma'lumotlar
print(df.shape)       # jadval o'lchami (qator, ustun)
print(df.columns)     # ustun nomlari
```

5. Ma'lumotlarga murojaat qilish (indeks va slicing)

```
# Bitta ustun
print(df["Ism"])

# Bir nechta ustun
print(df[["Ism", "Yosh"]])

# Indeks bo'yicha qator
print(df.iloc[0])      # 1-qator
print(df.iloc[1:3])    # 2 va 3-qatorlar

# Shart bo'yicha filtrlash
print(df[df["Yosh"] > 20]) # faqat yoshlari 20 dan katta bo'lganlar
```

6. Ustun qo'shish, o'chirish va tahrirlash

```
# Yangi ustun qo'shish
df["Natija"] = ["Yaxshi", "A'lo", "Qoniqarli", "A'lo"]

# Ustunni o'chirish
df = df.drop("Natija", axis=1)

# Qiymatni o'zgartirish
df.at[0, "Yosh"] = 25
```

7. Statistik amallar

```
print("O'rtacha yosh:", df["Yosh"].mean())
print("Eng katta baho:", df["Baholar"].max())
print("Eng kichik baho:", df["Baholar"].min())
print("Baholar yig'indisi:", df["Baholar"].sum())
```

8. Guruhlash (GroupBy)

```
# Talabalarni baholar bo'yicha guruhlash
grouped = df.groupby("Baholar")["Ism"].count()
print(grouped)
```

9. Ma'lumotlarni tartiblash

```
print(df.sort_values("Baholar", ascending=False)) # Bahoga qarab kamayish
tartibida
print(df.sort_values(["Yosh", "Baholar"]))          # Bir nechta ustun
bo'yicha tartiblash
```

10. Amaliy misol: Talabalarining baholarini tahlil qilish

```
data = {
    "Ism": ["Ali", "Vali", "Laylo", "Aziz", "Gulbahor", "Sherzod"],
    "Yosh": [20, 22, 19, 21, 23, 20],
    "Baholar": [85, 90, 78, 92, 88, 75],
    "Fakultet": ["IT", "IT", "Filologiya", "IT", "Iqtisod", "Filologiya"]
}
df = pd.DataFrame(data)
print("O'rtacha baho:", df["Baholar"].mean())
print("Har bir fakultet bo'yicha o'rtacha baho:\n",
df.groupby("Fakultet")["Baholar"].mean())
print("Eng yaxshi talaba:\n", df[df["Baholar"] == df["Baholar"].max()])
```

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

5-Mavzu: Mashinali o'qitishda NumPy va Pandas paketlarining qo'llanilishi.

2-mashg'ulot. NumPy va Pandas bilan ishlash.

O'quv savollari:

1. NumPy kutubxonasida amallar bajarish.
2. Pandas yordamida ma'lumotlarni qayta ishlash va tahlil qilish.

NumPy kutubxonasida amallar bajarish

Python dasturlash tilida sonli hisob-kitoblar uchun **NumPy** eng muhim kutubxonalardan biridir. Oddiy Python ro'yxatlari (list) kichik hisob-kitoblarga yaxshi ishlaydi, lekin:

katta hajmdagi ma'lumotlar bilan ishlaganda **sekinlashadi**, matematik amallarni bajarishda ko'p kod yozish kerak bo'ladi.

Shu muammoni hal qilish uchun **NumPy massivlari (ndarray)** yaratilgan. Ular yordamida biz minglab sonlar ustida bir vaqtning o'zida tezkor amallar bajara olamiz.

Masalan:

oddiy Python ro'yxatida 1 million element qo'shish soniyalar oladi, NumPy massivida esa bu **millisekund** ichida bajariladi.

2. NumPy massivlari (ndarray)

2.1. Massiv nima?

Massiv – bu bir xil turdagi elementlardan tashkil topgan tartibli to'plam. NumPy'da massivlar ndarray deb ataladi.

2.2. Oddiy massiv yaratish

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr)
```

Natija: [1 2 3 4 5]

Bu oddiy Python ro'yxatiga o'xshaydi, lekin tezroq ishlaydi.

2.3. Ikki o'lchovli massiv (matritsa)

```
mat = np.array([[1, 2, 3], [4, 5, 6]])
print(mat)
```

Natija:

[[1 2 3] [4 5 6]]

Bu 2 qator va 3 ustundan iborat **matritsa**.

Savol: Matritsaning 2-qator 3-ustunidagi element qaysi son? (Javob: 6)

3. Massivlar ustida amallar

NumPy'da **elementlar ustida bir xil amal** bajarilganda natija ham massiv ko'rinishida chiqadi.

3.1. Qo'shish va ayirish

```
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])
```

```
print(a + b) # [11 22 33]
print(a - b) # [ 9 18 27]
```

Har bir element mos ravishda qo'shildi yoki ayirildi.

3.2. Ko'paytirish va bo'lish

```
print(a * b) # [10 40 90]
print(a / b) # [10. 10. 10.]
```

Diqqat qiling, bu **elementlar bo'yicha ko'paytirish va bo'lish**. Oddiy Python'da bunday qilish uchun for sikl kerak bo'lardi.

3.3. Skalyar bilan amal

```
print(a * 2) # [20 40 60]
print(a + 5) # [15 25 35]
```

Bitta son (skalyar) butun massivga ta'sir qiladi.

Amalda: Agar sizda talabalar baholari bor bo'lsa va ularga 5 ball qo'shmoqchi bo'lsangiz, oddiygina `arr + 5` qilishingiz kifoya.

4. Matritsa amallari

Matritsalar ustida ishlash matematikaning ko'plab sohalarida (fizika, muhandislik, sun'iy intellekt) qo'llanadi.

4.1. Matritsa ko'paytirish

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print(np.dot(A, B))
```

Natija:

```
[[19 22] [43 50]]
```

4.2. Transponirlangan matritsa

```
print(A.T)
```

Matritsaning qatorlari ustun bo'lib qoladi.

4.3. Determinant va teskari matritsa

```
print(np.linalg.det(A)) # determinant
print(np.linalg.inv(A)) # teskari matritsa
```

Savol: Determinantning 0 chiqishi nimani anglatadi?

Javob: Matritsa **teskari matritsaga ega emas** degani.

5. Foydali funksiyalar

NumPy'da ko'p ishlatiladigan tayyor funksiyalar mavjud:

```
print(np.zeros((3,3))) # 3x3 nol massiv
print(np.ones((2,2))) # 2x2 birlar massiv
print(np.eye(3)) # 3x3 birlik matritsa
print(np.arange(0,10,2)) # [0 2 4 6 8]
print(np.linspace(0,1,5)) # [0. 0.25 0.5 0.75 1.]
```

Bu funksiyalar massivlarni tez yaratishda juda qulay.

6. Statistik amallar

NumPy yordamida oddiy statistik hisoblarni ham bajarish mumkin:

```
arr = np.array([1, 2, 3, 4, 5])
print(arr.mean()) # o'rtacha qiymat -> 3.0
```



```
print(arr.min())    # eng kichik qiymat → 1
print(arr.max())    # eng katta qiymat → 5
print(arr.std())    # standart og'ish
```

Amalda: Talabalar guruhining imtihon natijalarini tahlil qilishda **o'rtacha ball**, **eng yuqori ball** yoki **tarqalish darajasi** (std)ni hisoblash mumkin.

NumPy — tezkor va qulay hisob-kitoblar uchun kutubxona.

U orqali massivlar va matritsalar ustida matematik amallarni sodda va samarali bajarish mumkin.

Asosiy amallar: qo'shish, ko'paytirish, bo'lish, transponatsiya, determinant, teskari matritsa va statistik hisoblashlar.

Pandas yordamida ma'lumotlarni qayta ishlash va tahlil qilish

Pandas — Python dasturlash tilida ma'lumotlar bilan ishlash uchun eng mashhur kutubxona.

U yordamida biz:

ma'lumotlarni **jadval shaklida** saqlash,
ma'lumotlarni **tozalash, tartiblash va tahrirlash**,
statistik tahlil va vizualizatsiya qilishimiz mumkin.

Pandas kutubxonasini o'rnatish:

```
pip install pandas
```

Kutubxonani chaqirish:

```
import pandas as pd
```

2. Pandas'ning asosiy obyektlari

Pandas'da 2 ta asosiy obyekt bor:

1. **Series** – bitta ustunli ma'lumotlar to'plami.
2. **DataFrame** – bir nechta ustundan iborat jadval.

2.1. Series misoli

```
import pandas as pd
s = pd.Series([10, 20, 30, 40])
print(s)
Natija:
0    10
1    20
2    30
3    40
dtype: int64
```

Bu yerda chap tomonda indekslar, o'ng tomonda qiymatlar bor.

2.2. DataFrame misoli

```
data = {
    "Ism": ["Ali", "Vali", "Gulbahor"],
    "Yosh": [20, 21, 19],
    "Bahosi": [85, 90, 95]
}
df = pd.DataFrame(data)
```

```
print(df)
```

Natija:

	Ism	Yosh	Bahosi
0	Ali	20	85
1	Vali	21	90
2	Gulbahor	19	95

3. CSV fayllar bilan ishlash

Ko'pincha ma'lumotlar **CSV (Comma-Separated Values)** fayllarda bo'ladi.

3.1. Faylni o'qish

```
df = pd.read_csv("talabalar.csv")
```

`print(df.head())` # dastlabki 5 qatordan iborat jadval

3.2. Faylni saqlash

```
df.to_csv("natija.csv", index=False)
```

`index=False` qator raqamlarini saqlamaslik uchun ishlatiladi.

Ustunlarni ko'rish

```
print(df.columns)
```

Bitta ustunni olish

```
print(df["Ism"])
```

Bir nechta ustunni olish

```
print(df[["Ism", "Bahosi"]])
```

Filtrlash (shart bilan tanlash)

```
print(df[df["Bahosi"] > 90])
```

Faqat bahosi 90 dan yuqori bo'lgan talabalar chiqadi.

4.5. Yangi ustun qo'shish

```
df["Yosh_2_yildan_keyin"] = df["Yosh"] + 2
```

5. Statistik tahlil

Pandas yordamida ma'lumotlarni tezda tahlil qilish mumkin:

```
print(df["Bahosi"].mean()) # o'rtacha
print(df["Bahosi"].min()) # eng kichik
print(df["Bahosi"].max()) # eng katta
print(df["Bahosi"].std()) # standart og'ish
print(df.describe()) # umumiy statistik ma'lumot
```

Ma'lumotlarni tartiblash va guruhlash

Tartiblash

```
print(df.sort_values("Bahosi", ascending=False))
```

Baholari bo'yicha kamayish tartibida saralanadi.

Guruhlash

```
print(df.groupby("Yosh")["Bahosi"].mean())
```

Har bir yosh uchun baholarning o'rtacha qiymatini hisoblaydi.

Yo'qolgan qiymatlar bilan ishlash

Ko'p hollarda ma'lumotlarda **bo'sh (NaN)** qiymatlar uchraydi.

```
print(df.isnull().sum()) # nechta yo'qolgan qiymat bor
df = df.fillna(0) # bo'sh joylarni 0 bilan to'ldirish
df = df.dropna() # bo'sh qatorlarni o'chirish
```

Amaliy misol

Talabalar baholarini tahlil qilish

```
import pandas as pd
```

```
data = {
    "Ism": ["Ali", "Vali", "Gulbahor", "Dilshod"],
    "Yosh": [20, 21, 19, 20],
    "Bahosi": [85, 90, 95, 88]
}
df = pd.DataFrame(data)
```

```
# 1. O'rtacha baho
print("O'rtacha baho:", df["Bahosi"].mean())
# 2. Eng yuqori bahoga ega talaba
print(df[df["Bahosi"] == df["Bahosi"].max()])
# 3. Yosh bo'yicha guruhlash
print(df.groupby("Yosh")["Bahosi"].mean())
```

Natija:

```
O'rtacha baho: 89.5
      Ism  Yosh  Bahosi
2  Gulbahor   19     95
Yosh
19    95.0    20    86.5  21    90.0
```

```
Name: Bahosi, dtype: float64
```

Pandas ma'lumotlar bilan ishlash va tahlil qilish uchun asosiy kutubxona.

Series — bitta ustun, DataFrame — jadval shakli.

CSV fayllarni o'qish va yozish juda oson.

Filtrlash, saralash, guruhlash va statistik tahlillar tez bajariladi.

Real hayotda Pandas **data analyst**, **machine learning**, va **ilmiy tadqiqotlarda** keng qo'llaniladi.

Nazorat savollari

1. NumPy kutubxonasining asosiy obyektini nima deb atashadi va u qanday yaratiladi?
2. Massivlar ustida qo'shish, ko'paytirish va skalyar bilan amal bajarishni misol bilan tushuntiring.
3. Matritsaning transponirlangan ko'rinishini olish uchun qaysi metod ishlatiladi?
4. Pandas kutubxonasining asosiy obyektlari qaysilar? Ularning farqini tushuntiring.
5. CSV faylidan ma'lumotni Pandas yordamida qanday o'qish va saqlash mumkin?
6. DataFrame'da ma'lumotlarni filtrlash qanday amalga oshiriladi? Misol keltiring.

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

5-Mavzu: Mashinali o'qitishda NumPy va Pandas paketlarining qo'llanilishi.

3-mashg'ulot. NumPy va Pandas bilan ishlash.

O'quv savollari:

1. NumPy va Pandas paketidan foydalanib datasetlarni tahrirlash.
2. NumPy yordamida massivlarda amallar bajarish.
3. Pandas yordamida jadval ko'rinishidagi ma'lumotlarni filtrlash va tahlil qilish.

Numpy va pandas yordamida datasetlarni tahrirlash (ma'lumotlarni qayta ishlash)

Bugungi kunda ma'lumotlar juda katta hajmda yig'ilmoqda. Bu ma'lumotlar odatda:

tartibsiz,
bo'sh qiymatlar bilan,
keraksiz yoki noto'g'ri yozuvlar bilan bo'lishi mumkin.

Shuning uchun **ma'lumotlarni qayta ishlash (data preprocessing)** — **data science**, **machine learning** va **analitikaning** eng muhim bosqichlaridan biridir.

Python'da bu vazifalarni bajarishda **NumPy** va **Pandas** paketlari keng qo'llaniladi.

2. NumPy yordamida ma'lumotlarni qayta ishlash

2.1. Massivlarni yaratish va tahrirlash

```
import numpy as np
data = np.array([10, 20, 30, 40, 50])
print("Asl massiv:", data)
# Massiv elementlarini o'zgartirish
data[2] = 99
print("O'zgartirilgan massiv:", data)
```

Bu yerda 30 qiymati 99 ga o'zgartirildi.

2.2. Yo'qolgan yoki noto'g'ri qiymatlarni almashtirish

```
arr = np.array([10, 20, np.nan, 40, 50])
# np.nan ni o'rtacha qiymat bilan to'ldirish
mean_val = np.nanmean(arr)
arr = np.nan_to_num(arr, nan=mean_val)
print("Qayta ishlangan massiv:", arr)
```

np.nan bo'sh qiymatlarni bildiradi. Uni massivning o'rtacha qiymati bilan almashtirdik.

2.3. Massivlarni filtrlash

```
arr = np.array([5, 10, 15, 20, 25, 30]) # Faqat 15 dan katta qiymatlar
filtered = arr[arr > 15]
print("Filtrlangan massiv:", filtered)
```

3. Pandas yordamida datasetlarni qayta ishlash

Ko'pincha datasetlar **jadval** ko'rinishida bo'ladi. Bunday hollarda Pandas juda qulay.

3.1. DataFrame yaratish

```
import pandas as pd
data = {
    "Ism": ["Ali", "Vali", "Gulbahor", "Dilshod"],
    "Yosh": [20, 21, None, 20],
    "Bahosi": [85, 90, 95, None]
}
df = pd.DataFrame(data)
print(df)
```

Jadvalda bo'sh qiymatlar (None) mavjud.

3.2. Bo'sh qiymatlarni aniqlash va to'ldirish

```
print(df.isnull().sum()) # nechta bo'sh qiymat borligini ko'rish
df["Yosh"] = df["Yosh"].fillna(df["Yosh"].mean()) # yoshni o'rtacha bilan
to'ldirish
df["Bahosi"] = df["Bahosi"].fillna(df["Bahosi"].median()) # bahoni median
bilan to'ldirish
print(df)
```

Bu usul ma'lumotlarni yo'qotmasdan to'liq dataset olish imkonini beradi.

3.3. Keraksiz ustun yoki qatorlarni o'chirish

```
df = df.dropna() # bo'sh qatordan qutulish
df = df.drop(columns=["Yosh"]) # ustunni o'chirish
print(df)
```

3.4. Filtrlash va tartiblash

```
# Bahosi 90 dan katta bo'lgan talabalarni chiqarish
print(df[df["Bahosi"] > 90])
# Baholar bo'yicha kamayish tartibida saralash
print(df.sort_values("Bahosi", ascending=False))
```

3.5. Guruhlash va statistik hisoblar

```
# Yosh bo'yicha baholarning o'rtachasi
print(df.groupby("Ism")["Bahosi"].mean())
# Umumiy statistik ko'rsatkichlar
print(df.describe())
```

4. Amaliy misol

Talabalar datasetini qayta ishlash

```
import pandas as pd
import numpy as np
data = {
    "Ism": ["Ali", "Vali", "Gulbahor", "Dilshod", "Aziza"],
    "Yosh": [20, 21, np.nan, 20, 22],
    "Bahosi": [85, 90, 95, np.nan, 70]
}
df = pd.DataFrame(data)
print("Asl dataset:")
print(df)
```

1. Bo'sh qiymatlarni to'ldirish

```
df["Yosh"].fillna(df["Yosh"].mean(), inplace=True)
df["Bahosi"].fillna(df["Bahosi"].median(), inplace=True)
```

2. Faqat 80 dan yuqori bahoga ega talabalar

```
yuqori_baho = df[df["Bahosi"] > 80]
```

3. Baholar bo'yicha tartiblash

```
tartiblangan = df.sort_values("Bahosi", ascending=False)
print("\nQayta ishlangan dataset:")
print(df)
print("\nYuqori baholilar:")
print(yuqori_baho)
print("\nTartiblangan dataset:")
print(tartiblangan)
```

Natija:

Asl dataset:

	Ism	Yosh	Bahosi
0	Ali	20.0	85.0
1	Vali	21.0	90.0
2	Gulbahor	NaN	95.0
3	Dilshod	20.0	NaN
4	Aziza	22.0	70.0

Qayta ishlangan dataset:

	Ism	Yosh	Bahosi
0	Ali	20.0	85.0
1	Vali	21.0	90.0
2	Gulbahor	20.75	95.0
3	Dilshod	20.0	85.0
4	Aziza	22.0	70.0

Yuqori baholilar:

	Ism	Yosh	Bahosi
0	Ali	20.0	85.0
1	Vali	21.0	90.0
2	Gulbahor	20.75	95.0
3	Dilshod	20.0	85.0

Tartiblangan dataset:

	Ism	Yosh	Bahosi
2	Gulbahor	20.75	95.0
1	Vali	21.0	90.0
0	Ali	20.0	85.0
3	Dilshod	20.0	85.0
4	Aziza	22.0	70.0

NumPy massivlar ustida tezkor amallar bajarishda qulay.

Pandas esa jadval ko'rinishidagi datasetlarni tahlil qilish, tozalash, filtrlash va tartiblash uchun juda foydali.

Ma'lumotlarni to'g'ri qayta ishlash keyingi bosqichlarda (statistik tahlil, mashina o'qitish) uchun poydevor bo'ladi.

Nazorat savollari

1. NumPy va Pandas kutubxonalari o'rtasidagi asosiy farqlar nimalardan iborat?

2. NumPy massivida elementlarni tanlab olish va ularni o'zgartirish qanday amalga oshiriladi?
3. Pandas DataFrame'da ustun qo'shish va o'chirish qanday bajariladi?
4. Ma'lumotlarda mavjud bo'lgan **NaN (yo'qolgan qiymatlar)** bilan ishlash uchun Pandas'da qanday funksiyalar ishlatiladi?
5. DataFrame'dagi ustunlarni filtrlash va shartli tanlash qanday amalga oshiriladi? Misol keltiring.
6. Tahlil uchun ma'lumotlarni tayyorlashda NumPy va Pandas'ning qo'llanish afzalliklarini tushuntiring.

MA'RUZA MASHG'ULOT UCHUN O'QUV MATERIALLARI

6-Mavzu: Ma'lumotlar bilan ishlash va vizualizatsiya.

1-mashg'ulot. Matplotlib va Seaborn kutubxonalari bilan ishlash.

O'quv savollari:

1. Matplotlib va Seaborn kutubxonalari bilan tanishish.
2. Ma'lumotlarni vizualizatsiya qilish texnikalari.

Matplotlib va seaborn kutubxonalari bilan tanishish

Maqsad: Talabalarga **Matplotlib** va **Seaborn** kutubxonalari haqida tushuncha berish, ularning imkoniyatlarini amaliy misollar orqali ko'rsatish va vizualizatsiya qilish ko'nikmasini shakllantirish.

1. Matplotlib kutubxonasi haqida umumiy tushuncha

Matplotlib – Python dasturlash tilida **grafik va diagrammalar chizish** uchun ishlatiladigan eng mashhur kutubxona.

Oddiy **chiziqli grafiklar**, **nuqtali grafiklar**, **histogrammalar**, **doira diagrammalari** va boshqa ko'plab vizualizatsiya turlarini qo'llab-quvvatlaydi.

Asosiy moduli: **pyplot** (odatda plt qisqartmasi bilan chaqiriladi).

Matplotlib o'rnatish:

```
pip install matplotlib
```

Eng oddiy grafik:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
plt.plot(x, y)
plt.title("Oddiy chiziqli grafik")
plt.xlabel("X o'qi")
plt.ylabel("Y o'qi")
plt.show()
```

Bu kodda **x va y qiymatlar o'rtasidagi chiziqli bog'liqlik** ekranga chiqariladi.

2. Seaborn kutubxonasi haqida umumiy tushuncha

Seaborn – Matplotlib asosida yaratilgan, **statistik vizualizatsiya** uchun mo'ljallangan kutubxona.

1. Matplotlib'ga qaraganda **chiroyli va tayyor dizaynlar** beradi.
2. Murakkab grafiklarni sodda kod bilan chizishga yordam beradi.
3. DataFrame bilan bevosita ishlay oladi (Pandas bilan yaxshi integratsiyalashgan).

Seaborn o'rnatish:

```
pip install seaborn
```

Oddiy misol (nuqtali grafik):

```
import seaborn as sns
import matplotlib.pyplot as plt
# Seaborn ichidagi tayyor datasetlardan foydalanamiz
data = sns.load_dataset("tips")
sns.scatterplot(x="total_bill", y="tip", data=data)
```



```
plt.title("Hisob va choychaqa orasidagi bog'liqlik")
plt.show()
```

Bu grafikda restoran hisob summasi (total_bill) bilan berilgan choychaqa (tip) o'rtasidagi bog'liqlik tasvirlanadi.

3 Matplotlib va Seaborn farqlari

Xususiyat	Matplotlib	Seaborn
Asosiy vazifasi	Grafiklar chizish	Statistik vizualizatsiya
Dizayn	Oddiy, qo'lda sozlash kerak	Tayyor va chiroyli dizayn
Pandas bilan ishlash	Bevosita emas	Bevosita integratsiya qilingan
Grafiklar turlari	Keng (chiziq, histogram, doira, ...)	Murakkab (heatmap, pairplot, boxplot)

4 Seaborn yordamida murakkab grafiklar

Histogram va KDE

```
sns.histplot(data["total_bill"], kde=True)
plt.title("Hisob summolari taqsimoti")
plt.show()
```

Boxplot (taqsimotni ko'rsatish)

```
sns.boxplot(x="day", y="total_bill", data=data)
plt.title("Kunlarga qarab hisob summolari")
plt.show()
```

Heatmap (issiq xarita)

```
corr = data.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Korrelyatsiya matritsasi")
plt.show()
```

Matplotlib – moslashuvchan va asosiy grafik chizish vositasi.

Seaborn – statistik ma'lumotlarni chiroyli ko'rinishda tasvirlash uchun qulay.

Amaliyotda odatda ikkala kutubxona birgalikda ishlatiladi: **Matplotlib** – umumiy sozlash uchun, **Seaborn** – tayyor dizayn va tezkor grafiklar uchun.

Ma'lumotlarni vizualizatsiya qilish texnikalari

Ma'lumotlarni vizualizatsiya qilish – raqamlar va jadvallarni **grafik ko'rinishda** ifodalash san'ati.

Trendlarni ko'rish,

Bog'liqliklarni aniqlash,

Qaror qabul qilish jarayonini osonlashtirish.

2 Asosiy vizualizatsiya texnikalari

Chiziqli grafik (Line Plot)

```
import matplotlib.pyplot as plt

plt.plot([1,2,3,4,5], [150,200,250,220,300], marker="o")
```

```
plt.show()
```

Ustunli diagramma (Bar Chart)

```
plt.bar(["Olma", "Banan", "Apelsin"], [120, 90, 150])
plt.show()
```

Aylana diagramma (Pie Chart)

```
plt.pie([40, 25, 20, 15], labels=["A", "B", "C", "D"], autopct="%1.1f%%")
plt.show()
```

Nuqta diagramma (Scatter Plot)

```
plt.scatter([2, 3, 4, 5, 6], [55, 60, 65, 70, 75])
plt.show()
```

Murakkab vizualizatsiyalar (Seaborn)

```
import seaborn as sns
data = sns.load_dataset("tips")
sns.scatterplot(x="total_bill", y="tip", data=data)
```

Boxplot – median va kvartillar

Heatmap – korrelyatsiya matritsasi

Pairplot – bir nechta o'zgaruvchilar orasidagi bog'liqlik

Grafiklar ma'lumotlarni tushunarli qiladi.

Trend va naqshlarni tez aniqlash imkonini beradi.

Har bir data analyst vizualizatsiyani bilishi va qo'llay olishi muhim.

Nazorat savollari

1. Matplotlib kutubxonasi nima uchun ishlatiladi va uning asosiy moduli qaysi?
2. Chiziqli grafik va ustunli diagramma o'rtasidagi farqni tushuntiring.
3. Aylana diagramma (Pie Chart) qaysi maqsadlarda ishlatiladi?
4. Scatter plot qanday bog'lanishlarni ko'rsatadi?
5. Seaborn kutubxonasi Matplotlib'ga nisbatan qaysi jihatdan qulayroq?

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

6-Mavzu: Ma'lumotlar bilan ishlash va vizualizatsiya.

2-mashg'ulot. Matplotlib va Seaborn kutubxonalari yordamida ma'lumotlarni vizualizatsiya qilish.

O'quv savollari:

1. Grafiklar, gistogrammalar va scatter plot chizishni o'rganish.
2. Seaborn yordamida statistik vizualizatsiyalar yaratish.
3. Grafiklarni sozlash (rang, uslub, o'q nomlari, sarlavha) va ularni saqlash.

1. Grafiklar, gistogrammalar va scatter plot chizishni o'rganish

Darsning maqsadi. Talabalarga grafiklar (line plot), gistogramma (histogram) va nuqtali diagramma (scatter plot) tushunchasini amaliy misollar orqali o'rgatish.

Har bir grafikning qachon va qanday maqsadda qo'llanilishini tushuntirish.

Matplotlib kutubxonasi yordamida ma'lumotlarni vizualizatsiya qilish ko'nikmasini shakllantirish.

1 Kutubxona bilan ishlash

Matplotlib — Python dasturlash tilida grafik chizish uchun asosiy kutubxona. Uni ishlatish uchun quyidagicha chaqiramiz:

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

pyplot moduli grafiklarni chizish uchun kerak.

numpy esa qo'shimcha hisoblash va tasodifiy ma'lumotlar yaratishda yordam beradi.

2 Chiziqli grafik (Line Plot)

Ta'rif: Chiziqli grafik vaqt yoki ketma-ketlik bo'yicha o'zgarishni ko'rsatishda ishlatiladi. Masalan, har oy sotuvlar yoki har kun harorat o'zgarishi.

```
oylar = ["Yan", "Fev", "Mar", "Apr", "May", "Iyun"]
sotuv = [150, 200, 250, 220, 300, 350]
plt.plot(oylar, sotuv, marker="o", color="blue", linestyle="--")
plt.title("Oylik sotuvlar dinamikasi")
plt.xlabel("Oylar")
plt.ylabel("Sotuvlar soni")
plt.grid(True)
plt.show()
```

Izoh:

marker="o" — nuqta belgilarini qo'yadi.

linestyle="--" — chiziqni punktir ko'rinishda chiqaradi.

grid(True) — fon panjarasini qo'shadi.

Natija: Oylik sotuvlar qanday o'zgarayotganini aniq ko'rish mumkin.

3 Gistogramma (Histogram)

Ta’rifi: Gistogramma ma’lumotlarning taqsimotini ko’rsatadi. Masalan, talabalar ballarining nechta oralig’ida ko’proq to’planganini ko’rsatadi.

```
ballar = np.random.randint(50, 100, size=30) # 30 ta tasodifiy ball
plt.hist(ballar, bins=5, color="orange", edgecolor="black")
plt.title("Talabalar ballari taqsimoti")
plt.xlabel("Ball oralig'i")
plt.ylabel("Talabalar soni")
plt.show()
```

Izoh:

`np.random.randint(50, 100, size=30)` – 50 dan 100 gacha bo’lgan 30 ta tasodifiy son hosil qiladi.

`bins=5` – ma’lumotlar 5 ta oraliqqa bo’linadi.

Natija: Talabalarining nechta oralig’ida ko’proq to’planganini ko’rish mumkin.

4. Scatter plot (Nuqta diagramma)

Ta’rifi: Nuqta diagramma ikki o’zgaruvchi orasidagi bog’lanishni ko’rsatadi. Masalan, talabaning o’qish soati va olgan balli orasidagi aloqani ko’rsatish.

```
soat = [2,3,4,5,6,7,8,9,10]
ball = [50,55,60,65,70,75,80,85,90]
plt.scatter(soat, ball, color="green")
plt.title("O'qish soati va ball bog'liqligi")
plt.xlabel("O'qish soati")
plt.ylabel("Ball")
plt.grid(True)
plt.show()
```

Har bir nuqta — bitta talabaning ma’lumoti (o’qish soati va olgan bali).

Grafikda ko’tarilish kuzatilsa, demak ko’proq o’qigan talaba ko’proq ball olgan.

Natija: Ikkita o’zgaruvchi orasidagi bog’liqlik yaqqol ko’rinadi.

Mustaqil mashqlar

1. O’z haftalik xarajatlaringizni **line plot** yordamida chizing.
2. 50 ta tasodifiy sonlar hosil qiling va ularni **histogrammada** tasvirlang.
3. Talabalarining “uyqu soati” va “imtihon bali”ni bog’lab, **scatter plot** chizing.
4. Line plotda chiziq rangini va uslubini o’zgartirib ko’ring (masalan, qizil, nuqtali).
5. Histogrammada bins qiymatini 10 qilib, natijani solishtiring.

2. SEABORN YORDAMIDA STATISTIK VIZUALIZATSIYALAR YARATISH

Nazariy qism

Seaborn — bu matplotlib asosida ishlovchi, statistik ma’lumotlarni chiroyli va qulay tarzda ko’rsatishga mo’ljallangan Python kutubxonasi. U **Pandas DataFrame** bilan yaxshi integratsiyalashgan va tahliliy (analitik) grafiklarni tez chizish imkonini beradi.

Seabornning afzalliklari:

- Statistik grafiklar (distribusiya, regressiya, korrelyatsiya) uchun tayyor funksiyalar.
- Rang sxemalari, uslublar va mavzular avtomatik sozlanadi.
- DataFrame ustun nomlari bevosita ishlatiladi (ko'p plt.plot parametrlari kerak emas).

Amaliy misollar

1. Asosiy kutubxonalarni chaqirish

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
```

2. Namunaviy ma'lumotlar to'plami

```
df = sns.load_dataset('tips') # ichki dataset
print(df.head())
```

Ustunlar: total_bill, tip, sex, smoker, day, time, size

3. Tarqatma (distribution) grafik — sns.histplot()

```
sns.histplot(df['total_bill'], bins=20, kde=True)
plt.title("Hisob summasi taqsimoti")
plt.xlabel("Hisob summasi")
plt.ylabel("Foydalanish chastotasi")
plt.show()
```

4. Juftliklar orasidagi bog'liqlik — sns.scatterplot()

```
sns.scatterplot(x='total_bill', y='tip', data=df, hue='sex', style='smoker')
plt.title("Hisob va choychaqa o'rtasidagi bog'liqlik")
plt.show()
```

5. O'rtacha qiymatlar — sns.barplot()

```
sns.barplot(x='day', y='total_bill', data=df, estimator=sum, hue='sex')
plt.title("Kunlar bo'yicha umumiy hisob summasi")
plt.show()
```

6. Korrelyatsiya matritsasi — sns.heatmap()

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Korrelyatsiya matritsasi")
plt.show()
```

3. Grafiklarni sozlash (rang, uslub, o'qlar, sarlavha) va saqlash

Nazariy qism

Seaborn matplotlib'ga asoslanganligi sababli, har qanday grafikni:

- **Ranglar va uslub** bilan bezatish (set_style, set_palette);
- **O'q nomlari va sarlavha** qo'shish (plt.xlabel, plt.title);
- **Saqlash** (plt.savefig) mumkin.

Amaliy misollar

1. Uslub va rang palitrasini tanlash

```
sns.set_style("whitegrid") # white, darkgrid, ticks, whitegrid
```

```
sns.set_palette("pastel") # deep, muted, bright, dark, pastel
```

2. Grafik dizaynini sozlash

```
plt.figure(figsize=(8,6))
sns.boxplot(x='day', y='total_bill', data=df, hue='sex')

plt.title("Kun va jins bo'yicha hisob summolari", fontsize=14,
fontweight='bold')
plt.xlabel("Hafta kuni")
plt.ylabel("Hisob summasi ($)")
plt.legend(title="Jins", loc="upper right")

plt.tight_layout()
plt.show()
```

3. Rasmni fayl sifatida saqlash

```
plt.savefig("hisob_grafik.png", dpi=300, bbox_inches='tight')
```

dpi=300 — yuqori sifatli rasm, bbox_inches='tight' — bo'sh joylarni olib tashlaydi.

4. Regressiya chizig'i bilan bog'liqlik — sns.lmplot()

```
sns.lmplot(x='total_bill', y='tip', data=df, hue='sex', height=6, aspect=1.2)
plt.title("Hisob va choychaqa regressiya chizig'i bilan")
plt.savefig("regressiya_plot.png", dpi=200)
```

5. Ko'p grafikni bir joyda joylashtirish (subplots)

```
fig, axes = plt.subplots(1, 2, figsize=(12,5))
sns.histplot(df['total_bill'], kde=True, ax=axes[0], color='skyblue')
sns.boxplot(x='day', y='tip', data=df, ax=axes[1], palette='Set2')

axes[0].set_title("Hisob summasi taqsimoti")
axes[1].set_title("Kunlar bo'yicha choychaqa")
plt.tight_layout()
plt.savefig("multi_plot.png", dpi=300)
plt.show()
```

Eng foydali Seaborn funksiyalari jadvali

Funksiya	Maqsad
sns.histplot()	Tarqatma (distribusiya) grafigi
sns.boxplot()	Median va outlierlarni ko'rsatish
sns.violinplot()	Tarqatmani silliq shaklda
sns.barplot()	O'rtacha yoki yig'indi qiymatlar
sns.countplot()	Kategoriyalar sonini ko'rsatish
sns.scatterplot()	Juftliklar bog'liqligi
sns.lmplot()	Regressiya chizig'i bilan
sns.heatmap()	Korrelyatsiya matritsasi
sns.pairplot()	Barcha ustunlar juftliklarini tahlil qilish

Grafiklarni saqlash formati bo'yicha tavsiyalar

Format	Maqsad
.png	Har qanday ekran va hujjatga qo'yish uchun (standart)

.pdf	Matnli hisobot yoki chop etish uchun (vektorli)
.svg	Web yoki professional grafik ishlov uchun
.jpg	Yengil fayl, lekin sifat yo'qoladi (compress)

XULOSA

• **Seaborn** — statistik ma'lumotlarni tez, chiroyli va oson tahlil qilish uchun eng qulay vizualizatsiya kutubxonasi.

• **Uslublar, rang palitralari, va matplotlib sozlamalari** yordamida grafiklar professionallik darajasida tayyorlanadi.

• **plt.savefig()** orqali grafiklarni yuqori sifatli rasm sifatida eksport qilish mumkin.

Nazorat savollari

1. Chiziqli grafik (line plot) qanday ma'lumotlarni tasvirlash uchun ishlatiladi?
2. Histogramma va bar chart (ustunli diagramma) o'rtasidagi farq nimada?
3. Scatter plot qachon ishlatiladi va qanday afzalliklarga ega?
4. Matplotlib'da grafikning rangini va uslubini qanday o'zgartirish mumkin?
5. bins parametri gistogrammada qanday vazifani bajaradi?

GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

6-Mavzu: Ma'lumotlar bilan ishlash va vizualizatsiya.

3-mashg'ulot. Matplotlib va Seaborn kutubxonalari bilan ishlash.

O'quv savollari:

1. Mashhur datasetlarni vizualizatsiya qilish va natijalarni tahlil qilish.
2. Matplotlib yordamida chiziqli grafik, gistogramma va scatter plotlar yaratish.
3. Seaborn yordamida statistik ma'lumotlarni ko'rsatish va ularni solishtirish.

Darsning maqsadi. Talabalarni **Matplotlib va Seaborn** kutubxonalari bilan chuqurroq tanishtirish.

Mashhur datasetlar (masalan, *Iris*, *Titanic*, *Tips*) ustida vizual tahlil qilish ko'nikmalarini shakllantirish.

Guruhlarda ishlash orqali **ma'lumotlarni taqqoslash, umumlashtirish va xulosa chiqarishni** rivojlantirish.

Kerakli kutubxonalar va datasetlar

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

Seaborn kutubxonasida ba'zi mashhur datasetlar **tayyor holatda** mavjud:

```
iris = sns.load_dataset("iris")          # Gullarning o'lchamlari
titanic = sns.load_dataset("titanic")    # Titanic yo'lovchilari ma'lumotlari
tips = sns.load_dataset("tips")         # Restorandagi hisoblar
```

Amaliy qism: Iris dataset

Vazifa: Gullar turiga qarab ularning uzunlik va kenglik o'lchamlarini taqqoslash.

```
# Scatter plot - Iris dataset
sns.scatterplot(data=iris, x="sepal_length", y="sepal_width",
hue="species")
plt.title("Iris gullari: sepal uzunligi va kengligi")
plt.show()
```

Xulosa : Turli gullar (setosa, versicolor, virginica) o'lchamlari orqali guruhlarga ajraladi. Bu esa klassifikatsiya uchun juda muhim.

Amaliy qism: Titanic dataset

Vazifa: Titanic yo'lovchilari orasida tirik qolganlarni yosh va jins bo'yicha ko'rish.

```
# Count plot - Titanic dataset
sns.countplot(data=titanic, x="sex", hue="survived")
plt.title("Titanic: jins va tirik qolish statistikasi")
plt.show()
```

Ayollar orasida tirik qolganlar foizi ko'proq. Bu Titanicdagi "Women and children first" qoidasini tasdiqlaydi.

Amaliy qism: Tips dataset

Vazifa: Restoranda qoldirilgan choychaqa (tip) va hisob (total_bill) o'rtasidagi bog'liqlikni o'rganish.

```
# Regression plot - Tips dataset
sns.regplot(data=tips, x="total_bill", y="tip",
scatter_kws={"color":"green"}, line_kws={"color":"red"})
plt.title("Hisob va choychaqa bog'liqligi")
plt.show()
```

Xulosa: Hisob qancha katta bo'lsa, choychaqa ham ko'proq bo'lgan. Lekin mutanosiblik har doim ham ideal emas.

Matplotlib haqida kengroq

Matplotlib – Python'da eng qadimiy va eng ko'p ishlatiladigan vizualizatsiya kutubxonasi.

Afzalliklari: grafiklar ustidan to'liq nazorat, turli shakl va ranglarda grafik chizish imkoniyati.

Kamchiligi: Ko'p kod yozish kerak, ba'zan chiroyli grafik olish uchun qo'shimcha sozlash zarur.

Asosiy grafik turlari:

Line Plot – vaqt va ketma-ketlik dinamikasi.

Bar Chart – kategoriyalarni solishtirish.

Histogram – taqsimotni ko'rish.

Scatter Plot – ikki o'zgaruvchi o'rtasidagi bog'liqlik.

Seaborn haqida kengroq

Seaborn – Matplotlib ustida qurilgan va yanada **chiroyli, tayyor uslublarga ega** kutubxona.

Minimal kod bilan murakkab vizualizatsiyalar chizish mumkin.

Statistik tahlil elementlari (masalan, regressiya chizig'i, taqsimot) avtomatik qo'shiladi.

Asosiy grafik turlari:

countplot() – kategoriya bo'yicha hisoblash.

boxplot() – ma'lumotlar tarqalishini ko'rsatish.

violinplot() – taqsimot + medianani ko'rsatish.

heatmap() – korrelyatsiya va matritsalarini ko'rish.

Mashhur datasetlar haqida

Iris dataset. 150 ta gul (Setosa, Versicolor, Virginica).

Har bir gul uchun **sepal_length**, **sepal_width**, **petal_length**, **petal_width** o'lchamlari berilgan.

Asosan **klassifikatsiya** masalalarida qo'llaniladi.

Titanic dataset

891 ta yo'lovchi ma'lumotlari.

Jins, **yosh**, **sinf**, **tirik qolgan yoki qolmagan** kabi atributlar mavjud.

Mashhur **survival analysis** (tirik qolish ehtimoli) uchun ishlatiladi.

Tips dataset

Restorandagi hisob-kitoblar.

Hisob (total_bill), choychaqa (tip), kun, vaqt, jins kabi ma'lumotlarni o'z ichiga oladi.

Regression analysis va **tahlil qilish** uchun juda qulay.

Vizualizatsiya orqali olinadigan xulosalar

Iris: Gullarning ayrim xususiyatlari bo'yicha turini oldindan aniqlash mumkin.

Titanic: Ayollar va 1-sinf yo'lovchilarining tirik qolish ehtimoli yuqori.

Tips: Hisob kattalashgan sari choychaqa ham o'sadi, lekin doimiy mutanosib emas.

Guruhli topshiriqlar

1. Iris datasetdan foydalanib, gullarning petal_length va petal_width o'zaro bog'liqligini chizing va izohlang.
2. Titanic datasetda class (1, 2, 3-klass) bo'yicha yo'lovchilar tirik qolish ko'rsatkichini vizualizatsiya qiling.
3. Tips dataset yordamida kunduzgi va kechki vaqtlarda choychaqa taqsimotini tahlil qiling.
4. Har bir guruh: Natijani taqdim qiladi va qisqa xulosalar chiqaradi.

Nazorat savollari

1. Matplotlib va Seaborn o'rtasidagi asosiy farq nimada?
2. Iris datasetidagi uchta tur gullar qanday vizual ajratiladi?
3. Titanic datasetida qaysi guruh yo'lovchilarning tirik qolish ehtimoli yuqori bo'lgan?
4. Tips datasetida "total_bill" va "tip" o'rtasida qanday bog'lanish mavjud?
5. Seaborn'dagi hue parametri qanday ishlaydi?

MA'RUZA MASHG'ULOT UCHUN O'QUV MATERIALLARI

7-Mavzu: Mashinali o'qitish asoslari va algoritmlar.

1-mashg'ulot. Mashinali o'qitish algoritmlari haqida umumiy tushuncha.

O'quv savollari:

1. Mashinali o'qitish algoritmlari haqida umumiy tushuncha.
2. O'qitiluvchi (Supervised) o'rganish algoritmlari (Linear Regression, Logistic Regression).
3. O'qitilmaydigan (Unsupervised) o'rganish algoritmlari (K-means Clustering).

1. Mashinali o'qitish algoritmlari haqida umumiy tushuncha

Maqsad. Talabalarga mashinali o'qitish (Machine Learning) tushunchasi haqida asosiy bilim berish.

Mashinali o'qitishning asosiy algoritmlari va ularning qo'llanish sohalari bilan tanishtirish.

Oddiy misollar orqali algoritmlarning ishlash mantig'ini tushuntirish.

Mashinali o'qitish tushunchasi

Mashinali o'qitish – bu sun'iy intellekt sohasining bir qismi bo'lib, u kompyuterlarga aniq dasturlashsiz, ma'lumotlar orqali o'rganish imkonini beradi.

Oddiy qilib aytganda: dastur qoidalarni odam yozib bermaydi, balki ma'lumotlardan o'zi xulosa chiqaradi.

Masalan:

2. Nazoratlanmagan o'qitish (Unsupervised Learning)

Faqat **kirish ma'lumotlari** beriladi, lekin natija (label) yo'q.

Maqsad: yashirin tuzilmalarni topish.

Misollar:

Mijozlarni guruhlariga ajratish (clustering).

Ma'lumotlarni vizual qisqartirish (dimensionality reduction).

3. Mustahkamlash orqali o'qitish (Reinforcement Learning)

Agent (dastur) **muayyan muhitda harakat qiladi** va natijaga qarab mukofot yoki jazoga ega bo'ladi.

Maqsad: mukofotni maksimal qilish.

Misollar:

Kompyuter o'yinlarini o'ynash (Chess, Go).

Robotni boshqarish.

3 Mashinali o'qitish algoritmlari

Supervised Learning algoritmlari:

Linear Regression – doimiy qiymatlarni prognoz qilish.

Logistic Regression – 2 ta natijali klassifikatsiya.

Decision Trees – qaror daraxtlari asosida prognoz.

Random Forest – ko‘p daraxtlardan iborat ansambl usuli.

Support Vector Machine (SVM) – klasslarni ajratish uchun chegaraviy chiziqlar.

K-Nearest Neighbors (KNN) – yaqin qo‘shnilar asosida klassifikatsiya.

Unsupervised Learning algoritmlari:

K-Means Clustering – ma’lumotlarni k ta guruhga ajratish.

Hierarchical Clustering – daraxtsimon guruhlash.

PCA (Principal Component Analysis) – o‘lchamlarni kamaytirish.

Reinforcement Learning algoritmlari:

Q-Learning – agentning harakatlarini optimallashtirish.

Deep Q-Networks (DQN) – neyron tarmoqlar bilan mustahkamlash.

Amaliy qo‘llanilish sohalari

Moliyaviy soha – kredit baholash, firibgarlikni aniqlash.

Sog‘liqni saqlash – kasallikni tashxislash, dori sinovlari.

Texnologiya – ovoz tanish, matn tarjimasini, chatbotlar.

Transport – avtonom mashinalar, yo‘l harakati prognozi.

Marketing – mijozlarni segmentlash, reklama samaradorligini oshirish.

Oddiy misol (Supervised Learning – Linear Regression)

Masalan, talabalar **o‘qigan soatlari** asosida imtihon natijasini prognoz qilamiz:

O‘qish soati	Ball
2	50
4	65
6	80
8	90

Model “soatlar ko‘paygan sari ball oshadi” degan xulosani o‘zi chiqaradi.

O‘qitiluvchi (supervised) o‘rganish algoritmlari (linear regression, logistic regression)

Talabalarga **supervised learning** tushunchasini yetkazish.

Linear Regression va **Logistic Regression** algoritmlarining mohiyati, formulalari va qo‘llanilish sohasini tushuntirish.

Oddiy misollar orqali ularning farqini ko‘rsatish.

1. O‘qitiluvchi (Supervised) o‘rganish tushunchasi

Supervised Learning – bu **kirish (X)** va **chiqish (Y)** ma’lumotlari berilgan dataset asosida modelni o‘qitish usuli.

Model **ma’lum namunalar asosida o‘rganadi** va yangi kirish qiymati uchun prognoz beradi.

Misollar:

Talabani o'qish soati → imtihondan olgan bali.

Mijozning yoshi va daromadi → kredit berish yoki bermaslik.

2. Linear Regression

Chiziqli regressiya – doimiy qiymatlarni prognoz qilish algoritmi.

Maqsad: YYY ni XXX asosida **chiziqli bog'liqlik** orqali topish.

Formula:

$$Y = aX + b$$

aaa - og'irlik (weight), chiziqning qiyaligi.

bbb - erkin had (intercept).

Oddiy misol:

O'qish soati va imtihon natijasi: ko'proq soat → yuqoriroq ball.

Amaliy kod:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
# Ma'lumotlar
X = np.array([2, 4, 6, 8, 10]).reshape(-1, 1)
y = np.array([50, 60, 70, 80, 90])
# Model
model = LinearRegression()
model.fit(X, y)
# Prognoz
y_pred = model.predict(X)
# Grafik
plt.scatter(X, y, color="blue")
plt.plot(X, y_pred, color="red")
plt.title("Linear Regression: O'qish soati va ball")
plt.xlabel("Soat")
plt.ylabel("Ball")
plt.show()
```

Natija: Chiziq talabalar o'qigan soat va ball o'rtasidagi bog'lanishni ko'rsatadi.

3. Logistic Regression

Logistik regressiya – natijasi 2 ta qiymatli (0 yoki 1) bo'lgan klassifikatsiya algoritmi.

Maqsad: voqea sodir bo'lish ehtimolini hisoblash.

Oddiy misol:

Talabani o'qish soati → imtihondan o'tadi (1) yoki o'tolmaydi (0).

Elektron xat → Spam (1) yoki Spam emas (0).

4. Linear Regression va Logistic Regression farqlari

Xususiyat	Linear Regression	Logistic Regression
Natija	Doimiy qiymat (real son)	Diskret qiymat (0 yoki 1)
Qo'llanilishi	Narx, ball, hajm prognozi	Klassifikatsiya (spam, o'tgan/o'tmagan)

Misol	Uy narxini aniqlash	Email spammi yoki yo'qmi
-------	---------------------	--------------------------

O'qitilmaydigan (unsupervised) o'rganish algoritmlari (k-means clustering)

- Talabalarga unsupervised learning tushunchasini tushuntirish.
- K-means clustering algoritmi mohiyatini, ishlash jarayonini va qo'llanish sohalarini o'rgatish.
- Amaliy misol orqali clustering natijalarini ko'rsatish.

1. O'qitilmaydigan (Unsupervised) o'rganish tushunchasi

Unsupervised Learning – bu chiqish (label, Y) qiymatlari berilmagan dataset asosida modelni o'qitish usuli. Maqsad: ma'lumotlar ichidagi yashirin tuzilmalarni aniqlash.

Misollar:

- Do'kon xaridorlarini odatlari bo'yicha guruhlariga ajratish.
- Hujjatlarni mavzulariga qarab tasniflash.
- Genetik ma'lumotlarni tahlil qilish.

2. Clustering tushunchasi

Clustering – ma'lumotlarni o'zaro o'xshashlik darajasiga qarab guruhlash jarayoni.

Har bir guruh cluster deb ataladi.

Masalan: Talabalarni 3 guruhga ajratish:

- Juda yaxshi o'quvchilar
- O'rtacha o'quvchilar
- Past natijali o'quvchilar

3. K-means Clustering algoritmi

Ta'rif: K-means – eng mashhur clustering algoritmi. Maqsad: ma'lumotlarni K ta clusterga ajratish.

Ishlash bosqichlari:

1. K qiymatini tanlash (necha clusterga bo'linishini belgilash).
2. Markazlarni tasodifiy tanlash (centroidlar).
3. Har bir nuqtani eng yaqin markazga biriktirish.
4. Har bir cluster uchun yangi markazni hisoblash.
5. Markazlar o'zgarmaguncha 3–4-bosqichlarni takrorlash.

4. Amaliy kod (Python misoli)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
# Sun'iy ma'lumot yaratamiz
X, y = make_blobs(n_samples=200, centers=4, cluster_std=1.0, random_state=42)
# K-means modelini tanlash (4 ta cluster)
kmeans = KMeans(n_clusters=4, random_state=42)
```

```

kmeans.fit(X)
# Cluster markazlari
centers = kmeans.cluster_centers_
labels = kmeans.labels_
# Natijani chizish
plt.scatter(X[:,0], X[:,1], c=labels, cmap="viridis", alpha=0.6)
plt.scatter(centers[:,0], centers[:,1], c="red", s=200, marker="X")
plt.title("K-means Clustering")
plt.show()

```

Natija: ma'lumotlar 4 guruhga bo'linadi, markazlari qizil X bilan belgilanadi.

K-means algoritmining afzallik va kamchiliklari

Afzalliklari:

- Oddiy va tez ishlaydi.
- Katta datasetlarga qo'llash mumkin.
- Natija vizual tarzda tushunarli.

Kamchiliklari:

- K qiymatini oldindan tanlash kerak.
- Shovqinli (noisy) ma'lumotlarga sezgir.
- Faqat "dumaloq" clusterlarni yaxshi aniqlaydi.

7. Nazorat savollari

1. Unsupervised learning nima va supervised learningdan farqi nimada?
2. Clustering nima va u qanday vazifani bajaradi?
3. K-means algoritmi qanday bosqichlardan iborat?
4. K-means algoritmining afzalliklari va kamchiliklari nimalardan iborat?
5. K-means algoritmini hayotdagi qaysi sohalarda qo'llash mumkin?

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

7-Mavzu: Mashinali o'qitish asoslari va algoritmlar.

2-mashg'ulot. Mashinali o'qitish algoritmlarini amaliyotga tatbiq qilish.

O'quv savollari:

1. Linear va Logistic Regression algoritmlarini tatbiq qilish.
2. K-means Clustering algoritmini tatbiq qilish.

Linear va logistic regression algoritmlarini tatbiq qilish

Maqsad. Talabalar Linear Regression va Logistic Regression algoritmlarini amalda qo'llashni o'rganish.

Oddiy datasetlar yordamida **prognoz va klassifikatsiya** vazifalarini bajarish. Grafiklar orqali natijalarni vizualizatsiya qilish va tahlil qilish.

1 Linear Regression amaliyoti

Talabalar **o'qish soati** asosida **imtihon ballini** prognoz qilishadi.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
# Ma'lumotlar
X = np.array([2, 4, 6, 8, 10]).reshape(-1, 1)
y = np.array([50, 60, 70, 80, 90])
# Model yaratish va o'qitish
linear_model = LinearRegression()
linear_model.fit(X, y)
# Prognoz
y_pred = linear_model.predict(X)
# Grafik
plt.scatter(X, y, color="blue", label="Haqiqiy ball")
plt.plot(X, y_pred, color="red", label="Prognoz ball")
plt.xlabel("O'qish soati")
plt.ylabel("Ball")
plt.title("Linear Regression: O'qish soati vs Ball")
plt.legend()
plt.show()
```

Vazifalar talabalarga:

1. 5, 7 va 9 soat o'qigan talabaning prognoz ballini aniqlash.
2. Grafikdagi qizil chiziq va haqiqiy nuqtalarni tahlil qilish.

2 Logistic Regression amaliyoti

Vazifa: Talabalar imtihon topshirgan talabaning o'tolishi yoki o'tolmasligini (1 = o'tgan, 0 = o'tolmagan) aniqlashadi.

Ma'lumotlar:

O'qish soati	Natija
2	0

4	0
6	1
8	1
10	1

from sklearn.linear_model import LogisticRegression

```
# Ma'lumotlar
X = np.array([2, 4, 6, 8, 10]).reshape(-1, 1)
y = np.array([0, 0, 1, 1, 1])
# Model yaratish va o'qitish
logistic_model = LogisticRegression()
logistic_model.fit(X, y)
# Prognoz
print("5 soat o'qigan talaba:", logistic_model.predict([[5]]))
print("7 soat o'qigan talaba:", logistic_model.predict([[7]]))
```

Vazifalar talabalarga:

1. 3, 5, 9 soat o'qigan talabaning natijasini bashorat qilish.
2. Natijalarni 0 va 1 formatida tahlil qilish.

3 Guruhli topshiriqlar

1. Guruhlar Linear Regression modelini o'zingiz yaratgan dataset bilan sinab ko'ring.
2. Guruhlar Logistic Regressionni turli shartlar bilan (masalan, mehnat soati → ishga qabul) tatbiq qiling.
3. Natijalarni grafik va jadval ko'rinishida taqdim qiling.
4. Grafiklar orqali modelning qanchalik aniqligini tahlil qiling.

K-means Clustering algoritmini tatbiq qilish

Maqsad

- Talabalarga K-means clustering algoritmini amalda tatbiq qilishni o'rgatish.
- Oddiy dataset yordamida ma'lumotlarni guruhlash (clustering) vazifasini bajarish.
- Grafiklar orqali clustering natijalarini vizualizatsiya qilish va tahlil qilish.

1. Vazifa

Talabalar sun'iy yoki haqiqiy dataset yordamida **K-means clustering**ni tatbiq qiladi va natijalarni vizualizatsiya qiladi.

Misol dataset: Sun'iy ma'lumotlar

- 200 ta nuqta
- 4 ta guruhga bo'lingan
- Har bir nuqta ikki o'lchovli (x, y)

2. Amaliy kod (Python)

```
import numpy as np
import matplotlib.pyplot as plt
```

```

from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
# Sun'iy ma'lumot yaratish
X, y = make_blobs(n_samples=200, centers=4, cluster_std=1.0,
random_state=42)
# K-means modelini yaratish (4 ta cluster)
kmeans = KMeans(n_clusters=4, random_state=42)
kmeans.fit(X)
# Cluster markazlari va labelar
centers = kmeans.cluster_centers_
labels = kmeans.labels_
# Natijani chizish
plt.scatter(X[:,0], X[:,1], c=labels, cmap="viridis", alpha=0.6)
plt.scatter(centers[:,0], centers[:,1], c="red", s=200, marker="X")
plt.title("K-means Clustering")
plt.xlabel("X o'lchov")
plt.ylabel("Y o'lchov")
plt.show()

```

3. Qo'shimcha vazifalar

- Cluster ichidagi nuqtalarning markazdan masofasini hisoblash.
- Shovqinli ma'lumotlar qo'shib, algoritmnining sezgirligini tahlil qilish.
- Clusterlar sonini tanlashda **Elbow method** yoki **Silhouette score** yordamida eng optimal K qiymatini aniqlash.

Nazorat savollari

1. Linear Regression va Logistic Regression o'rtasidagi asosiy farq nima?
2. Logistic Regression natijasi qanday oraliqda bo'ladi?
3. Linear Regressionda prognoz qilingan qiymatlarni qanday tahlil qilish mumkin?
4. Modelni yaratishda fit() va predict() metodlari qanday ishlaydi?
5. Guruh bilan ishlashda qanday qo'shimcha tahlil qilishingiz mumkin?
6. K-means algoritmi qanday bosqichlarda ishlaydi?
7. Cluster markazlari (centroidlar) qanday aniqlanadi?
8. K-means algoritmining afzalliklari va kamchiliklari nimalardan iborat?
9. Clusterlar sonini qanday aniqlash mumkin?
10. K-meansni qaysi sohalarda amalda qo'llash mumkin?

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

7-Mavzu: Mashinali o'qitish asoslari va algoritmlar.

3-mashg'ulot. Mashinali o'qitish algoritmlarini real datasetlarda qo'llash.

O'quv savollari:

1. Real datasetlarda algoritmlarni qo'llash va natijalarni taqdim etish.
2. Ma'lumotlarni tayyorlash va mashinali o'qitish modellariga uzatish.
3. Modellarning aniqligini baholash va taqqoslash.

Real datasetlarda algoritmlarni qo'llash va natijalarni taqdim etish

- Talabalarga haqiqiy datasetlar bilan ishlashni o'rgatish.
- Linear Regression, Logistic Regression va K-means Clustering algoritmlarini amalda tatbiq qilish.
- Natijalarni grafik va jadval ko'rinishida taqdim etish.

1. Vazifa

Talabalar quyidagi vazifalarni bajaradilar:

1. Linear Regression yordamida uy narxlarini prognoz qilish.
2. **Logistic Regression** yordamida mijozning kredit olish ehtimolini aniqlash.
3. **K-means Clustering** yordamida mijozlarni guruhlash.
4. Natijalarni grafik va jadval ko'rinishida taqdim etish.

2. Linear Regression – Real dataset

Dataset: Boston Housing Dataset

- Xususiyatlar: uy maydoni, xonalar soni, joylashuv, soliq stavkasi va boshqalar
- Maqsad: uy narxini prognoz qilish

```
import pandas as pd
from sklearn.datasets import load_boston
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# Datasetni yuklash
boston = load_boston()
X = pd.DataFrame(boston.data, columns=boston.feature_names)
y = boston.target

# Train-test bo'lish
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Model yaratish
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

# Prognoz
y_pred = linear_model.predict(X_test)

# Natijalarni vizualizatsiya
plt.scatter(y_test, y_pred)
plt.xlabel("Haqiqiy narx")
```

```
plt.ylabel("Prognoz narx")
plt.title("Linear Regression: Real dataset")
plt.show()
```

Vazifalar talabalarga:

Prognoz va haqiqiy qiymatlarni taqqoslash.

Qaysi xususiyatlar uy narxiga ko'proq ta'sir qiladi, tahlil qilish.

3. Logistic Regression – Real dataset

Dataset: Bank Marketing Dataset

Xususiyatlar: mijoz yoshi, daromadi, aloqa turi, oldingi javoblar

Maqsad: mijoz kredit olish ehtimoli (1 = oladi, 0 = olmaydi)

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
# Datasetni yuklash
df = pd.read_csv("bank.csv")
# Kategorik xususiyatlarni kodlash
le = LabelEncoder()
df['job'] = le.fit_transform(df['job'])
df['marital'] = le.fit_transform(df['marital'])
X = df[['age', 'balance', 'job', 'marital']]
y = df['y'] # 0 yoki 1
# Train-test bo'lish
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
# Model yaratish
logistic_model = LogisticRegression()
logistic_model.fit(X_train, y_train)
# Prognoz
y_pred = logistic_model.predict(X_test)
# Natijalarni ko'rsatish
comparison = pd.DataFrame({'Haqiqiy': y_test, 'Prognoz': y_pred})
print(comparison.head())
```

Vazifalar talabalarga:

Natijalarni jadvalda taqdim etish.

Model qanchalik aniqligini aniqlash (accuracy).

4. K-means Clustering – Real dataset

Dataset: Mall Customer Segmentation Dataset

Xususiyatlar: yosh, yillik daromad, xarid indeksi

Maqsad: mijozlarni segmentlarga ajratish

from sklearn.cluster import KMeans

```
# Datasetni yuklash
df = pd.read_csv("Mall_Customers.csv")
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
# K-means
kmeans = KMeans(n_clusters=5, random_state=42)
labels = kmeans.fit_predict(X)
```

```

df['Cluster'] = labels
# Grafik
plt.scatter(X['Annual Income (k$)'], X['Spending Score (1-100)'], c=labels,
cmap='viridis')
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("K-means Clustering")
plt.show()

```

Vazifalar talabalarga:

Har bir cluster xususiyatlarini tahlil qilish.

Cluster markazlarini jadvalda ko'rsatish.

Segmentlarga mos marketing strategiyasini ishlab chiqish.

Guruhli topshiriqlar

1. Har bir algoritmni real datasetlar bilan sinab ko'ring.
2. Natijalarni grafik va jadval ko'rinishida taqdim eting.
3. Model natijalarini tahlil qiling va xulosalar chiqaring.

Nazorat savollari

1. Real dataset bilan ishlaganda qaysi qadamlar Linear Regression va Logistic Regression uchun muhim?
2. K-means clusterlar sonini qanday tanlash mumkin?
3. Natijalarni vizualizatsiya qilishning foydasi nimada?
4. Logistic Regression natijasini qanday baholash mumkin?
5. Har bir algoritm qaysi turdagi ma'lumotlar uchun mos?

MA'RUZA MASHG'ULOT UCHUN O'QUV MATERIALLARI

8-Mavzu: Mashinali o'qitishda Klassifikatsiya va regressiya masalalari

1-mashg'ulot. Mashinali o'qitishda Klassifikatsiya va regressiya tushunchalari.

O'quv savollari:

1. Klassifikatsiya tushunchasi va asosiy algoritmlar (Decision Trees, Random Forest).
2. Regressiya tushunchasi va algoritmlari (Linear, Polynomial Regression).

Klassifikatsiya tushunchasi va asosiy algoritmlar (Decision Trees, Random Forest).

Mashinali o'qitish (Machine Learning) - bu ma'lumotlar asosida o'rgatish jarayonini anglatadi, bunda kompyuterlar ma'lum vazifalarni bajarishni o'rgatadi va bu jarayonni yanada takomillashtiradi. Mashinalarni o'qitishning turli xil metodlari mavjud, ulardan ikkitasi eng ko'p ishlatiladi: Klassifikatsiya va Regressiya.

Bugungi darsda biz ushbu ikkala metodni ko'rib chiqamiz va ular uchun asosiy algoritmlarni tahlil qilamiz.

1. Klassifikatsiya Tushunchasi:

Klassifikatsiya — bu mashinali o'qitishning asosiy masalalaridan biri bo'lib, unda kirish o'zgaruvchilari (xususiyatlar)ga asoslanib obyektlarni ma'lum sinflarga ajratish kerak.

Masalan:

- Tibbiyotda kasallikni aniqlash: "Kasallik mavjudmi yoki yo'q?"
- Elektron pochta xabarlar: "Bu spammi yoki normal pochta?"
- Tasvirlarni tahlil qilishda: "Bu tasvirda it yoki mushuk bormi?"

Klassifikatsiya Algoritmlari:

1. Decision Trees (Qaror Daraxtlari):

◦ Tushuncha: Qaror daraxti - bu qarorlarni qabul qilish uchun ishlatiladigan vizual model. Har bir tugun (node) ma'lum bir xususiyatga asoslangan qarorni ifodalaydi. Model qaysi sinfga kirishini aniqlash uchun ma'lum xususiyatlar bo'yicha bo'linadi.

◦ Afzalliklari: Oddiy tushunish va vizual taqdimot, shuning uchun o'qitish va tahlil qilish oson.

◦ Kamchiliklari: Overfitting (modelning haddan tashqari moslashishi) muammosi, ya'ni juda yaxshi o'rganilgan model yangi ma'lumotlar bilan ishlashda past natija beradi.

2. Random Forest (Tasodifiy O'rmon):

◦ Tushuncha: Random Forest - bu bir nechta qaror daraxtlaridan tashkil topgan ansamblar metodidir. Har bir daraxt tasodifiy xususiyatlar bo'yicha o'rganadi va yakuniy natija barcha daraxtlarning ovozi asosida olinadi.

- Afzalliklari: Overfitting xavfini kamaytiradi, yuqori aniqlik.
- Kamchiliklari: Modelni tushunish va interpretatsiya qilish qiyin bo'lishi mumkin.

2. Regressiya Tushunchasi:

Regressiya — bu mashinali o'qitish metodidir, unda chiqish (y) o'zgaruvchisi uzluksiz (yoki doimiy) qiymatga ega bo'lgan holatlar ishlatiladi. Bunda maqsad - kirish (x) o'zgaruvchilari asosida chiqishni bashorat qilish.

Masalan:

- Uy narxini bashorat qilish: "O'rtacha xona soni, maydon va boshqa xususiyatlar asosida uy narxini hisoblash."
- Haroratni prognoz qilish: "Oldingi kunlar ma'lumotlariga asoslanib, ertangi haroratni prognoz qilish."

Regressiya Tushunchasi va Algoritmлари (Linear, Polynomial Regression)

Regressiya - bu statistik tahlilning bir turidir va mashinalarni o'rganish (machine learning)da uzluksiz o'zgaruvchini bashorat qilishda ishlatiladi. Regressiya modelining asosiy maqsadi - bir yoki bir nechta kirish (x) o'zgaruvchilari asosida chiqish (y) o'zgaruvchisini prognoz qilishdir. Regressiya masalalari odatda "nisbatlar" yoki "miqdorlar"ni prognoz qilishga mo'ljallangan, masalan, uy narxlarini, talab hajmini yoki haroratni bashorat qilish.

Masalan, biror hududda uy narxlarini o'rganayotgan bo'lsak, bizning maqsadimiz xonalar soni, maydon va boshqa xususiyatlar asosida uy narxini prognoz qilishdir. Ushbu maqsadga erishish uchun **regressiya algoritmлари** ishlatiladi.

Linear Regression (Chiziqli Regressiya):

Linear Regression (chiziqli regressiya) - bu regressiyaning eng oddiy va keng tarqalgan usulidir. Bu model kirish o'zgaruvchisi bilan chiqish o'zgaruvchisi o'rtasidagi chiziqli bog'lanishni topishga qaratilgan.

Chiziqli regressiya modelining formulasi:

Chiziqli regressiya modelida chiqish o'zgaruvchisi yyy va kirish o'zgaruvchisi xxx o'rtasidagi bog'lanish quyidagi tenglama yordamida ifodalanadi:

$$y = b_0 + b_1x$$

- **yyy** — prognoz qilinayotgan o'zgaruvchi (masalan, uy narxi).
- **xxx** — kirish o'zgaruvchisi (masalan, xonalar soni).
- **b₀** — modelning intercepti (y-intercept), ya'ni $x = 0$ bo'lganda y ning qiymati.
- **b₁** — regressiya koeffitsienti (slope), ya'ni xxx o'zgaruvchisining yyy ga qanday ta'sir qilishini ko'rsatuvchi koeffitsient.

Chiziqli regressiyaning afzalliklari:

- **Oddiy va tez:** Modelni yaratish juda oson va hisoblashlar tez amalga oshiriladi.

- **Tushunish oson:** Grafik shaklda ko'rsatilganda, chiziqli regressiya ko'plab sohalarda intuitiv tarzda tushunilishi mumkin.

Chiziqli regressiyaning kamchiliklari:

- **Chiziqli bo'lmagan bog'lanishlar uchun mos kelmasligi:** Agar o'zgaruvchilar o'rtasida chiziqli bo'lmagan bog'lanish mavjud bo'lsa, chiziqli regressiya yomon natijalar berishi mumkin.
- **Sensitivlik:** Kichik xatoliklar va yomon ma'lumotlar modelni to'g'ri ishlashiga xalaqit berishi mumkin.

Polynomial Regression (Polinom Regressiya):

Polynomial Regression (polinom regressiya) - bu chiziqli regressiyaning kengaytirilgan shaklidir. Bu model kirish o'zgaruvchisining yuqori darajali polinomiga asoslangan modelni yaratadi. Demak, agar chiziqli regressiya faqat chiziqli bog'lanishni hisobga olsa, polinom regressiya ma'lumotlardagi chiziqli bo'lmagan bog'lanishlarni ham modelga olish imkonini beradi.

Polinom regressiya modelining formulasi:

Polinom regressiya quyidagi tarzda ifodalanadi:

$$y = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots + b_nx^n$$

- **yyy** — prognoz qilinayotgan o'zgaruvchi.
- **xxx** — kirish o'zgaruvchisi.
- **b₀, b₁, b₂, ..., b_n** — polinom koefitsientlari.

- **nnn** — polinom darajasi (masalan, ikkinchi daraja, uchinchi daraja va h.k.).

Polinom regressiya modelida yuqori darajali xususiyatlar qo'shiladi, bu esa modelga murakkabroq bog'lanishlarni hisoblash imkoniyatini beradi.

Polinom regressiyaning afzalliklari:

- **Chiziqli bo'lmagan bog'lanishlarni model qilish:** Polinom regressiya kirish va chiqish o'zgaruvchilari o'rtasidagi chiziqli bo'lmagan bog'lanishlarni aniqroq prognoz qilishga imkon beradi.

- **Moslashuvchanlik:** Modelni kerakli darajada moslashtirish mumkin (daraja oshirilgan sari, model yanada moslashuvchan bo'ladi).

Polinom regressiyaning kamchiliklari:

- **Overfitting (ortiqcha moslashish):** Agar polinom darajasi juda yuqori bo'lsa, model ma'lumotlarga juda yaxshi moslashadi, lekin yangi ma'lumotlar bilan ishlaganda yomon natija berishi mumkin.

- **Murakkablik:** Model chiziqli regressiyaga qaraganda murakkabroq va ko'proq hisoblashlarni talab qiladi.

Linear va Polynomial Regression o'rtasidagi farqlar:

Xususiyatlar	Linear Regression	Polynomial Regression
Model shakli	Chiziqli (birinchi daraja)	Yuqori daraja polinom (ikkinchi, uchinchi va h.k.)
Chiziqli bo'lmagan bog'lanishlar	Yo'q	Ha, chiziqli bo'lmagan bog'lanishlarni hisoblash mumkin
Soddaligi	Oddiy va tez	Murakkabroq, yuqori darajali hisoblashlarni talab qiladi
Overfitting ehtimoli	Kam	Yuqori daraja polinomda yuqori bo'ladi

Quyidagi kodda Linear Regression va Polynomial Regression ni sinab ko'rishingiz mumkin:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Ma'lumotlar (masalan, uy narxi va xona soni)
X = np.array([[1], [2], [3], [4], [5], [6], [7], [8], [9], [10]]) # Xonalar soni
y = np.array([1, 2, 1.5, 3, 3.5, 5, 6, 6.5, 7, 8]) # Narxlar

# Ma'lumotlarni bo'lish
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Linear Regression
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
y_pred_lr = lr_model.predict(X_test)

# Polynomial Regression
poly = PolynomialFeatures(degree=3) # Darajani o'zgartirish mumkin
X_poly = poly.fit_transform(X)
poly_model = LinearRegression()
poly_model.fit(X_poly, y)

# Natijalarni chizish
plt.scatter(X, y, color='blue', label='Data')
plt.plot(X_test, y_pred_lr, color='red', label='Linear Regression')
plt.title("Linear Regression vs Polynomial Regression")
plt.xlabel("Number of Rooms")
plt.ylabel("Price")
plt.legend()
plt.show()

# MSE (Mean Squared Error) hisoblash
```

```
print(f"Linear Regression MSE: {mean_squared_error(y_test, y_pred_lr)}")
```

Amaliyot:

1. Klassifikatsiya (Decision Trees va Random Forest): Ma'lumotlar to'plamidan (masalan, Iris dataset) foydalanib, qaror daraxtlari va random forest yordamida tasniflashni amalga oshirish.

2. Regressiya (Linear Regression va Polynomial Regression): Boston Housing datasetidan foydalangan holda uy narxlarini prognoz qilish uchun chiziqli regressiya va polinom regressiya modellari bilan ishlash.

Nazorat Savollari:

1. Klassifikatsiya va regressiya o'rtasidagi farqni tushuntiring.
2. Decision Tree va Random Forest algoritmlarining afzalliklari va kamchiliklarini taqqoslang.
3. Linear va Polynomial regressiya modellari o'rtasidagi farqlarni tushuntiring.
4. Overfitting va underfitting nima, va ular qanday sodir bo'ladi?
5. Polinom regressiya modelida daraja tanlashda nimalarga e'tibor berish kerak?
6. Klassifikatsiya va regressiya masalalari uchun qanday to'plamlar mavjud?
7. Modelning o'rganish jarayonida qanday xatolarni topdingiz?

Qo'shimcha Eslatmalar:

- Klassifikatsiya masalalari odatda diskret (butun) sinflarga ajratish bilan bog'liq. Odatda, tasvirlar yoki xatoliklarni aniqlash kabi vazifalarda qo'llaniladi.
- Regressiya masalalari esa o'zgaruvchilarning uzluksiz qiymatlarini prognoz qilish uchun ishlatiladi va iqtisodiyot, tibbiyot kabi sohalarda keng qo'llaniladi.

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

8-Mavzu: Mashinali o'qitishda Klassifikatsiya va regressiya masalalari

2-mashg'ulot. Mashinali o'qitishda Klassifikatsiya va regressiyani qo'llanilishi.

O'quv savollari:

1. Decision Trees va Random Forest algoritmlarini tatbiq qilish
2. Turli regressiya algoritmlarini tatbiq qilish.

Decision Trees va Random Forest Algoritmlarini Tatbiq qilish

Decision Trees va **Random Forest** algoritmlari mashinalarni o'rgatishdagi eng mashhur klassifikatsiya va regressiya algoritmlaridan biridir. Ushbu algoritmlar ma'lumotlar to'plamidagi xususiyatlarga asoslangan holda qarorlar qabul qilish jarayonini model qiladi.

1. Decision Trees (Qaror Daraxtlari)

Decision Trees - bu mashinalarni o'rgatishdagi eng oddiy va intuitiv metodlardan biridir. Qaror daraxti tasniflash yoki regressiya masalalarini hal qilish uchun ishlatiladi.

Qaror daraxtining qanday ishlashi:

- **Tugunlar** (nodes): Har bir tugun biror qarorni ifodalaydi.
- **Brachlar** (edges): Tugunlar orasidagi bog'lanish, ya'ni ma'lumotning qanday taqsimlanishini ko'rsatadi.
- **Yakuniy tugunlar** (leaf nodes): Bu yerda model natijasi (klassifikatsiya yoki prognoz qiymati) chiqadi.

Qaror daraxti o'zgaruvchilarni bo'lish orqali ma'lumotlarni sinflarga ajratadi. Har bir bo'linish, eng yaxshi bo'linish mezonini tanlash uchun statistik usullardan (masalan, **Gini Impurity**, **Entropy**, yoki **Variance**) foydalanadi.

Afzalliklari:

- Tushunish va interpretatsiya qilish oson.
- Ma'lumotlarni to'liq ishlashga imkon beradi (ya'ni, hech qanday ma'lumotni tashlab yubormaydi).
- Kategoriyali va uzluksiz o'zgaruvchilar bilan ishlay oladi.

Kamchiliklari:

- **Overfitting (ortiqcha moslashish):** Agar daraxt juda chuqur bo'lsa, yangi ma'lumotlarga qarshi kamchiliklarga yo'l qo'yishi mumkin.
- Ma'lumotlarning kichik o'zgarishi ham modelni o'zgartirishi mumkin.

2. Random Forest (Tasodifiy O'rmon)

Random Forest - bu bir nechta qaror daraxtlaridan tashkil topgan ansambllar (ensemble) metodidir. Random Forest bir nechta qaror daraxtlarini quradi va har bir daraxtni ma'lumotlarning tasodifiy qismini o'rganish uchun foydalanadi. So'ngra, model barcha daraxtlaridan olingan natijalarni birlashtirib yakuniy qarorni chiqaradi.

Qanday ishlaydi:

- **Tasodifiy namunalar:** Ma'lumotlar to'plamidan tasodifiy namunalar tanlanadi, har bir daraxt uchun bunday namunalar alohida tanlanadi.
- **Tasodifiy xususiyatlar:** Har bir daraxtni qurishda, eng yaxshi bo'linishni tanlash uchun tasodifiy xususiyatlar tanlanadi. Bu esa qaror daraxtlari o'rtasida o'zaro farq yaratadi.
- Yakuniy natija barcha daraxtlarning o'rtacha ovozigina (regressiya) yoki ko'pchilik ovozigina (klassifikatsiya) asoslanadi.

Afzalliklari:

- **Overfitting muammosini kamaytiradi:** Ko'plab daraxtlar birgalikda ishlaydi, shuning uchun bu model overfittingga qarshi kuchliroq.
- **Aniqlik yuqori:** Tasodifiy xususiyatlar va namunalar yordamida model yanada aniqlikni oshiradi.
- **Stabil natijalar:** Model yangi ma'lumotlar bilan ishlashda barqaror natijalar beradi.

Kamchiliklari:

- **Interpretatsiya qilish qiyin:** Random Forest modellari juda murakkab bo'lgani uchun ularning natijalarini tushunish qiyin bo'lishi mumkin.
- **Hisoblash vaqtining uzunligi:** Ko'p daraxtlar qurilayotganda hisoblash vaqti uzoq bo'lishi mumkin.

Decision Trees va Random Forest Algoritmilarini Tatbiq qilish

Ushbu algoritmarni tatbiq qilish uchun **scikit-learn** kutubxonasidan foydalanish mumkin. Quyidagi kodda **Iris flower dataset** yordamida **Decision Trees** va **Random Forest** algoritmilarini qo'llashni ko'rsatib o'tamiz.

```
# Kerakli kutubxonalarni o'rnatish
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

# Iris datasetini yuklash
iris = load_iris()
X = iris.data # Xususiyatlar
y = iris.target # Maqsad o'zgaruvchisi (sinflar)

# Ma'lumotlarni o'qish va train-test bo'lish
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
```

```

# 1. Decision Tree modelini yaratish
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)

# 2. Random Forest modelini yaratish
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

# Natijalar
print("Decision Tree Accuracy: ", accuracy_score(y_test, y_pred_dt))
print("Random Forest Accuracy: ", accuracy_score(y_test, y_pred_rf))

# Decision Tree vizualizatsiyasi
plt.figure(figsize=(12,8))
plot_tree(dt_model, filled=True, feature_names=iris.feature_names,
class_names=iris.target_names, rounded=True)
plt.title("Decision Tree Visualization")
plt.show()

```

Izohlar:

1. **DecisionTreeClassifier** va **RandomForestClassifier** yordamida **Iris** datasetini klassifikatsiya qilish.
2. **accuracy_score** yordamida har ikkala modelning aniqligini baholash.
3. **plot_tree** funksiyasi yordamida qaror daraxtini vizualizatsiya qilish, bu orqali modelni qanday qarorlar qabul qilayotganini tushunish mumkin.

Natijalar:

- **Decision Tree** modeli qaror daraxti tuzadi va tasniflashni amalga oshiradi. Dastlabki natija ko‘pincha yuqori bo‘ladi, ammo overfitting mumkin.
- **Random Forest** modeli bir nechta qaror daraxtlarining natijalarini birlashtiradi, shuning uchun model ko‘proq barqaror va yuqori aniqlik bilan ishlaydi.

Turli Regressiya Algoritmilarini Tatbiq qilish

Regressiya - bu statistik tahlil usuli bo‘lib, unda biror o‘zgaruvchini bashorat qilish maqsadida kirish o‘zgaruvchilari asosida matematik model tuziladi. Regressiya algoritmlari uzluksiz o‘zgaruvchini prognoz qilish uchun qo‘llaniladi. Ushbu mavzuda turli regressiya algoritmilarini - **Linear Regression**, **Polynomial Regression**, **Ridge Regression**, va **Lasso Regression** algoritmilarini tatbiq qilishni ko‘rib chiqamiz.

1. Linear Regression (Chiziqli Regressiya)

Linear Regression — bu regressiyaning eng oddiy va keng tarqalgan turidir. Bu modelda chiqish o‘zgaruvchisi yyy va kirish o‘zgaruvchilari xxx o‘rtasida chiziqli bog‘lanish mavjud.

Chiziqli regressiya modelining formulyasi:

$$y = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots + b_nx^n$$

- **b0b_0b0** — intercept, ya'ni y o'qida boshlang'ich nuqta.
- **b1b_1b1** — chiziqli regressiyaning koeffitsienti (slope), ya'ni xxx o'zgaruvchisining yyy ga ta'siri.

Chiziqli regressiya juda oddiy va tushunilishi oson bo'lib, asosiy maqsad **ma'lumotlar o'rtasidagi chiziqli bog'lanishni topish**.

```
from sklearn.linear_model import LinearRegression
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression
from sklearn.metrics import mean_squared_error

# Oddiy regressiya uchun ma'lumotlar yaratish
X, y = make_regression(n_samples=100, n_features=1, noise=10,
random_state=42)

# Ma'lumotlarni bo'lish
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Linear Regression modelini yaratish
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

# Bashorat qilish
y_pred_lr = lr_model.predict(X_test)

# Natijalar
print(f"Linear Regression Mean Squared Error: {mean_squared_error(y_test,
y_pred_lr)}")

# Natijalarni chizish
plt.scatter(X_test, y_test, color='blue', label='True values')
plt.plot(X_test, y_pred_lr, color='red', label='Linear Regression')
plt.title("Linear Regression")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()
```

2. Polynomial Regression (Polinom Regressiya)

Polynomial Regression - bu chiziqli regressiyaning kengaytirilgan shakli bo'lib, unda kirish o'zgaruvchisi yuqori darajali polinomga o'zgartiriladi. Bu chiziqli bo'lmagan bog'lanishlarni modellash uchun ishlatiladi.

Polinom regressiya modelining formulasi:

$$y = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots + b_nx^n$$

• $b_0, b_1, b_2, \dots, b_n$ — polinomning koeffitsientlari.

- x^n — x ning yuqori darajadagi quvvatlari.

Kod:

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Polynomial Regression uchun degree = 3 (uchinchi daraja)
poly = PolynomialFeatures(degree=3)
X_poly = poly.fit_transform(X)

# Ma'lumotlarni bo'lish
X_train_poly, X_test_poly, y_train, y_test = train_test_split(X_poly, y,
test_size=0.3, random_state=42)

# Polynomial Regression modelini yaratish
poly_model = LinearRegression()
poly_model.fit(X_train_poly, y_train)

# Bashorat qilish
y_pred_poly = poly_model.predict(X_test_poly)

# Natijalar
print(f"Polynomial Regression Mean Squared Error:
{mean_squared_error(y_test, y_pred_poly)}")
# Natijalarni chizish
plt.scatter(X_test, y_test, color='blue', label='True values')
plt.plot(X_test, y_pred_poly, color='green', label='Polynomial Regression')
plt.title("Polynomial Regression")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()
```

3. Ridge Regression (L2 Regularization)

Ridge Regression - bu chiziqli regressiyaning variantidir. Bu model **L2 regularization** (penalti) ni qo'llaydi, ya'ni modelni mavjud bo'lgan o'zgaruvchilarga nisbatan kichik bo'lishini ta'minlash uchun penalti qo'llaydi. Bu metod **overfitting**ni kamaytiradi, chunki katta koeffitsientlarga jarima qo'yadi.

```
from sklearn.linear_model import Ridge

# Ridge Regression modelini yaratish
ridge_model = Ridge(alpha=1.0) # alpha - regularization koeffitsienti
ridge_model.fit(X_train, y_train)

# Bashorat qilish
```

```

y_pred_ridge = ridge_model.predict(X_test)

# Natijalar
print(f"Ridge Regression Mean Squared Error: {mean_squared_error(y_test,
y_pred_ridge)}")
# Natijalarni chizish
plt.scatter(X_test, y_test, color='blue', label='True values')
plt.plot(X_test, y_pred_ridge, color='purple', label='Ridge Regression')
plt.title("Ridge Regression")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()

```

4. Lasso Regression (L1 Regularization)

Lasso Regression - bu chiziqli regressiyaning yana bir variantidir, u **L1 regularization** (penalti)ni qo'llaydi. Lasso modelida **biroz koeffitsientlarni nolga tenglashtirishga** moyil bo'ladi, ya'ni ba'zi o'zgaruvchilarni modeldan chiqaradi (bu modelni soddalashtirishga yordam beradi).

Kod:

```

from sklearn.linear_model import Lasso

# Lasso Regression modelini yaratish
lasso_model = Lasso(alpha=0.1) # alpha - regularization koeffitsienti
lasso_model.fit(X_train, y_train)

# Bashorat qilish
y_pred_lasso = lasso_model.predict(X_test)

# Natijalar
print(f"Lasso Regression Mean Squared Error: {mean_squared_error(y_test,
y_pred_lasso)}")

# Natijalarni chizish
plt.scatter(X_test, y_test, color='blue', label='True values')
plt.plot(X_test, y_pred_lasso, color='orange', label='Lasso Regression')
plt.title("Lasso Regression")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()

```


GURUHLI MASHG'ULOT UCHUN O'QUV MATERIALLARI

8-Mavzu: Mashinali o'qitishda Klassifikatsiya va regressiya masalalari

3-mashg'ulot. Mashinali o'qitish algoritmlarini real datasetlarda qo'llash.

O'quv savollari:

1. Amaliy natijalarni solishtirish va guruh taqdimotlari.
2. Modellarning aniqlik, xatolik va samaradorlik ko'rsatkichlarini hisoblash.
3. Turli algoritmlarni bir datasetda sinovdan o'tkazib taqqoslash.

Amaliy natijalarni solishtirish va guruh taqdimotlari

Amaliy natijalarni solishtirish:

Mashinali o'qitish algoritmlarini real datasetlarda qo'llashda, turli algoritmlar yordamida aniqlik va samaradorlikni baholash juda muhimdir. Buning uchun bir nechta asosiy ko'rsatkichlardan foydalaniladi:

- **Aniqlik (Accuracy):** Model to'g'ri natijalarni umumiy baholash uchun ishlatiladi. Aniqlik yuqori bo'lsa, modelning ishlash samaradorligi yuqori deb hisoblanadi.

- **Precision (Aniqlik):** Ijobiy tasniflangan misollarning nechitasi to'g'ri ijobiy ekanligini o'lchaydi. Bu ko'rsatkich maxsus to'g'ri tasniflashni ta'minlashga yordam beradi.

- **Recall (Qaytarish):** To'g'ri ijobiy tasniflashning umumiy ijobiy misollarga nisbatan qanday bajarilishini o'lchaydi.

- **F1-skoring:** Precision va Recall o'rtasidagi balansni o'lchaydi. Ushbu ko'rsatkich odatda qiyin vaziyatlarda foydalidir, masalan, muvozanatsiz datasetlarda.

- **ROC va AUC:** Bu ko'rsatkichlar modelning umumiy tasniflash qobiliyatini baholaydi, ayniqsa, ijobiy va salbiy sinflar orasidagi farqni aniqlashda foydalidir.

Real datasetlarda qo'llash:

Amaliy mashg'ulotda real datasetlar bilan ishlash orqali mashinali o'qitish algoritmlarini sinovdan o'tkazish mumkin. Quyidagi amaliy mashqlarni o'tkazish mumkin:

1. **Datasetni tanlash:** Mashg'ulot uchun turli real datasetlarni tanlash. Masalan, Kaggle'dan olingan **Iris**, **Titanic**, **Housing Prices**, yoki **MNIST** kabi datasetlar.

2. **Preprocessing:** Datasetdagi ma'lumotlarni tozalash, eksklyuziv xususiyatlarni ajratib olish va ma'lumotlarni kerakli shaklga keltirish.

3. **Modelni tanlash:** Algoritmlarni tanlash va ularni o'rgatish. Masalan:

- **Logistic Regression** (klassifikatsiya)
- **Linear Regression** (regressiya)
- **Decision Trees** va **Random Forest** (klassifikatsiya va regressiya)
- **Support Vector Machines (SVM)**
- **K-Nearest Neighbors (KNN)**

4. **Modelni baholash:** Algoritmni test qilish va yuqoridagi ko'rsatkichlarni (aniqlik, precision, recall, F1-skoring, ROC/AUC) hisoblash.

5. **Natijalarni solishtirish:** Turli algoritmlarni ishlatib, ularning natijalarini solishtirish. Natijalarni taqdimot qilish orqali guruhlar o'rtasida o'zaro tahlil qilish.

Guruh Taqdimotlari:

- Har bir guruh o'zining ishlagan modelini taqdim etadi va qaysi algoritmni tanladi, uning sabablarini tushuntiradi.

- Taqdimotda natijalarni solishtirish (qaysi model yaxshiroq ishladi, qaysi ko'rsatkichlar asosida tanlov qilinganini).

Turli algoritmlarning afzalliklari va kamchiliklari haqida fikr almashish.

- Modelni optimallashtirish va yaxshilash uchun qanday usullarni qo'llash mumkinligi haqida gapirish.

Amaliy mashg'ulotlar misollari:

1. **Titanic datasetini o'rganish:** Klassifikatsiya (hayotga qolgan/olgan).

Logistic Regression, Random Forest, va **KNN** modellari yordamida to'g'ri prognoz qilishni baholash.

2. **MNIST datasetini o'rganish:** Tasvirlarni tasniflash (raqamlarni aniqlash).

- **SVM** va **Neural Networks** yordamida test qilish va natijalarni solishtirish.

3. **Boston Housing dataseti:** Narxlarni prognoz qilish (regressiya).

Linear Regression va **Random Forest Regression** yordamida model qurish va natijalarni taqqoslash.

Natijalarni baholash va taqqoslash:

Har bir guruhni quyidagi parametrlar bo'yicha baholash mumkin:

- **Tasniflash aniqligi**
- **Modelning soddaligi**
- **O'rganish vaqti**
- **Yuklash vaqti**
- **Algoritmni qo'llashning amaliy samaradorligi**

itanic Dataset: Klassifikatsiya

Titanic dataseti orqali **klassifikatsiya** masalasini hal qilamiz. Bu yerda biz **Logistic Regression, Random Forest,** va **KNN** algoritmlaridan foydalangan holda, **hayotga qolgan yoki halok bo'lgan yo'lovchilarni** prognoz qilishni o'rganamiz.

Kutilgan natijalar:

- Hayotga qolgan va halok bo'lgan yo'lovchilarni aniq tasniflash.
- Har bir modelni sinovdan o'tkazish va natijalarni solishtirish (aniqlik, precision, recall).

```
# Zarur kutubxonalar
import pandas as pd
```

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# Titanic datasetini yuklash
url
=
"https://web.stanford.edu/class/archive/cs/cs109/cs109.1166/stuff/titanic.csv"
data = pd.read_csv(url)

# Keraksiz ustunlarni o'chirish va ma'lumotni tayyorlash
data = data.drop(['Name', 'Ticket', 'Cabin'], axis=1)
data = data.dropna()

# Xususiyatlar va maqsadni ajratish
X = data[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']]
y = data['Survived']

# Categorical ustunlarni o'zgartirish (One-hot encoding)
X = pd.get_dummies(X, columns=['Sex'], drop_first=True)

# Ma'lumotlarni o'qitish va testga bo'lish
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Ma'lumotlarni normalizatsiya qilish
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Logistic Regression modelini yaratish va sinovdan o'tkazish
lr_model = LogisticRegression()
lr_model.fit(X_train, y_train)
y_pred_lr = lr_model.predict(X_test)

# Random Forest modelini yaratish va sinovdan o'tkazish
rf_model = RandomForestClassifier(n_estimators=100)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

# KNN modelini yaratish va sinovdan o'tkazish
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
y_pred_knn = knn_model.predict(X_test)

# Natijalarni solishtirish
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))

```

```
# Klassifikatsiya hisobotlari
print("\nLogistic Regression Classification Report:")
print(classification_report(y_test, y_pred_lr))

print("\nRandom Forest Classification Report:")
print(classification_report(y_test, y_pred_rf))

print("\nKNN Classification Report:")
print(classification_report(y_test, y_pred_knn))
```

Mashg'ulot davomida:

1. **Datasetni tayyorlash:** Ushbu kodda Titanic datasetidan ma'lumotlar olinadi, va kerakli preprocessing amallarini bajariladi.

2. **Modelni qurish:** Logistic Regression, Random Forest, va KNN modellarini qurish va ularni taqqoslash.

3. **Natijalarni tahlil qilish:** Har bir modelning aniqligini (accuracy) va boshqa ko'rsatkichlarini solishtirish.

2. Boston Housing Dataset: Regressiya

Boston Housing datasetini o'rganib chiqamiz, bunda biz **Linear Regression** va **Random Forest Regressor** algoritmlarini qo'llaymiz va **uy narxlarini** prognoz qilamiz.

Kutilgan natijalar:

- Uy narxlarini regressiya yordamida prognoz qilish.
- Har bir modelning muvaffaqiyatini **MSE (Mean Squared Error)** va **R2 score** orqali o'lchash.

```
# Zarur kutubxonalar
from sklearn.datasets import load_boston
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Boston datasetini yuklash
boston = load_boston()
X_boston = boston.data
y_boston = boston.target

# Ma'lumotlarni o'qitish va testga bo'lish
X_train, X_test, y_train, y_test = train_test_split(X_boston, y_boston,
test_size=0.2, random_state=42)

# Linear Regression modelini yaratish va sinovdan o'tkazish
lr_regressor = LinearRegression()
lr_regressor.fit(X_train, y_train)
y_pred_lr = lr_regressor.predict(X_test)

# Random Forest Regressor modelini yaratish va sinovdan o'tkazish
```

```

rf_regressor = RandomForestRegressor(n_estimators=100)
rf_regressor.fit(X_train, y_train)
y_pred_rf = rf_regressor.predict(X_test)

# Natijalarni solishtirish
print("Linear Regression Mean Squared Error:", mean_squared_error(y_test,
y_pred_lr))
print("Random Forest Mean Squared Error:", mean_squared_error(y_test,
y_pred_rf))

# R2 score
print("Linear Regression R2 Score:", r2_score(y_test, y_pred_lr))
print("Random Forest R2 Score:", r2_score(y_test, y_pred_rf))

```

Mashg'ulot davomida:

1. **Datasetni tayyorlash:** Boston Housing datasetidan ma'lumotlar olinadi, va ularni o'qitish va test qilishga bo'lish.
2. **Modelni qurish:** Linear Regression va Random Forest Regressor modellarini ishlatib, uy narxlarini prognoz qilish.
3. **Natijalarni tahlil qilish:** MSE va R2 score orqali har bir modelning samaradorligini baholash.

3. MNIST Dataset: Tasvirni Tasniflash

MNIST datasetini ishlatamiz va **Support Vector Machine (SVM)** va **Neural Network (MLPClassifier)** algoritmlarini qo'llaymiz. Maqsad — qo'l yozuvlarini tasniflash (raqamlar 0 dan 9 gacha).

Kutilgan natijalar:

- Tasvirlarni **SVM** va **Neural Network** yordamida to'g'ri tasniflash.
- **Accuracy** va **Confusion Matrix** orqali modelni baholash.

```

# Zarur kutubxonalar
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# MNIST datasetini yuklash
digits = datasets.load_digits()
X_digits = digits.data
y_digits = digits.target

# Ma'lumotlarni o'qitish va testga bo'lish
X_train, X_test, y_train, y_test = train_test_split(X_digits, y_digits,
test_size=0.2, random_state=42)

# SVM modelini yaratish va sinovdan o'tkazish
svm_model = SVC(kernel='linear')
svm_model.fit(X_train, y_train)

```

```

y_pred_svm = svm_model.predict(X_test)

# Neural Network (MLP) modelini yaratish va sinovdan o'tkazish
mlp_model = MLPClassifier(hidden_layer_sizes=(64,), max_iter=500)
mlp_model.fit(X_train, y_train)
y_pred_mlp = mlp_model.predict(X_test)

# Natijalarni solishtirish
print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))
print("MLP Accuracy:", accuracy_score(y_test, y_pred_mlp))
# Confusion Matrix
print("\nSVM Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_svm))

print("\nMLP Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_mlp))

```

Mashg'ulot davomida:

1. **Datasetni tayyorlash:** MNIST datasetidan foydalanib, tasvirni tayyorlash va modelni o'qitish.
2. **Modelni qurish:** SVM va MLPClassifier yordamida raqamlarni aniqlash.
3. **Natijalarni tahlil qilish:** Accuracy va Confusion Matrixni ko'rib chiqish.

Amaliy Mashg'ulotlar:

- **Titanic datasetini ishlatib** turli klassifikatsiya algoritmlarini sinovdan o'tkazish va natijalarni solishtirish.
- **Boston Housing datasetini o'rganib** regressiya modellarini yaratish va baholash.
- **MNIST datasetida** tasvirlarni tasniflash, SVM va Neural Network yordamida baholash va natijalarni taqqoslash.

Nazorat savollari:

1. Mashinali o'qitish nima?
2. Supervised learning va Unsupervised learning o'rtasidagi farqni tushuntirib bering.
3. KNN algoritmi qanday ishlaydi?
4. Logistic Regression modeli nima va u qanday ishlaydi?
5. Accuracy nima va uni qanday hisoblash mumkin?
6. Confusion Matrix nima?

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

8-Mavzu: Mashinali o'qitishda Klassifikatsiya va regressiya masalalari

4-mashg'ulot. Mashinali o'qitish algoritmlarini real datasetlarda qo'llash.

O'quv savollari:

1. Sun'iy neyron tarmoqlari asoslari.
2. Perceptron modeli va tarmoqlarni o'qitish mexanizmi.
3. Kirish darajasidagi neyron tarmoqlari (MLP).

1. Sun'iy Neyron Tarmoqlari Asoslari

Sun'iy neyron tarmoqlari (Artificial Neural Networks, ANN) — biologik neyron tarmoqlariga asoslangan, ma'lumotlarni o'rganish va qaror qabul qilishda ishlatiladigan algoritmlardir. Neyron tarmoqlari ko'plab mashinalarni o'qitish vazifalarida, jumladan, tasniflash, regressiya, tasvirni tanib olish va boshqa murakkab vazifalarni hal qilishda samarali qo'llaniladi.

Asosiy tushunchalar:

- **Neyron** — oddiy hisoblash birligi bo'lib, u kirishlarni qabul qiladi, ularni og'irliklar bilan ko'paytiradi, qo'shimcha biaz (bias) qo'shadi va natijada faollashtirish funksiyasidan (activation function) o'tadi.
- **Kirish (Input Layer)** — tarmoqqa kirish ma'lumotlarini kiritadi.
- **Chiqish (Output Layer)** — tarmoqdan natija chiqadi.
- **Yashirin qatlamlar (Hidden Layers)** — kirish va chiqish qatlamlari o'rtasida joylashgan, ma'lumotlarni qayta ishlashni amalga oshiradi.
- **Og'irliklar (Weights)** — neyronlar o'rtasidagi aloqalarni o'lchaydi va o'rganish jarayonida yangilanadi.
- **Biaz (Bias)** — modelga erkinlik qo'shish uchun ishlatiladi, bu neyronlarning natijasini korreksiya qilishga yordam beradi.

Faollashtirish funksiyalari:

- **Sigmoid** — 0 va 1 orasida qiymatlar beradi.
- **Tanh** — -1 va 1 orasida qiymatlar beradi.
- **ReLU (Rectified Linear Unit)** — 0 dan yuqori qiymatlarni o'z ichiga oladi, boshqa holatda 0 qiymatini qaytaradi.

2. Perceptron Modeli va Tarmoqlarni O'qitish Mexanizmi

Perceptron (Birlikni neyron modeli):

Perceptron — eng oddiy sun'iy neyron modeli bo'lib, uni ko'p qatlamli neyron tarmoqlari yaratishda asos sifatida ishlatish mumkin. Perceptron faqatgina bitta kirish qatlamidan tashkil topgan va uning asosiy maqsadi ijobiy yoki salbiy sinflarga ajratishdir (klassifikatsiya).

Perceptron modeli ishlash prinsipi:

- **Kirishlar (Input):** Perceptronning kirishlari — ma'lumotlar ($x_1, x_2, \dots, x_n$).

- **Og'irliklar (Weights):** Har bir kirishga mos og'irliklar ($w_1, w_2, \dots, w_n$) taqdim etiladi.

- **Summator:** Kirishlar va og'irliklar bilan chiziqli kombinatsiya hisoblanadi:

$$\text{Sum} = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$$

$$\text{Sum} = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$$

- **Faollashtirish funksiyasi:** Chiqish faollashtirish funksiyasiga kiradi va yakuniy natija hosil bo'ladi. Agar yakuniy qiymat ma'lum threshold (masalan, 0) dan katta bo'lsa, chiqish 1, aks holda 0 bo'ladi.

Tarmoqlarni o'qitish mexanizmi:

Perceptronni o'qitish jarayoni quyidagi bosqichlarni o'z ichiga oladi:

1. **Oldindan chiqish (Forward Propagation):** Kirish ma'lumotlarini tarmoqqa yuborish, va chiqishni hisoblash.

2. **Xato hisoblash (Error Calculation):** Tarmoqning chiqish qiymati va haqiqiy natija o'rtasidagi farqni hisoblash.

3. **Orqaga tarqatish (Backpropagation):** Xatolikni tarmoqning og'irliklarini yangilash uchun orqaga tarqatish. Bu jarayon gradientni kamaytirish (gradient descent) orqali amalga oshiriladi.

4. **Og'irliklarni yangilash (Weight Update):** Og'irliklar tarmoqni yanada yaxshilash uchun yangilanadi.

3. Kirish Darajasidagi Neyron Tarmoqlari (MLP - Multi-layer Perceptron)

MLP (Ko'p qatlamli Perceptron):

Multi-layer Perceptron (MLP) — bu ko'p qatlamli neyron tarmog'idir. MLP bir nechta yashirin qatlamlardan tashkil topgan va bu qatlamlar tarmoqni murakkabroq va kuchliroq qiladi. MLP ko'plab klassifikatsiya va regressiya masalalarini hal qilishda ishlatiladi.

MLP tuzilishi:

- **Kirish qatlam (Input Layer):** Ma'lumotlar tarmoqqa kiradi.
- **Yashirin qatlamlar (Hidden Layers):** Ma'lumotlarni qayta ishlaydi. Har bir qatlamda neyronlar o'zaro bog'langan va faollashtirish funksiyasidan foydalaniladi.
- **Chiqish qatlam (Output Layer):** Modelning yakuniy natijasi chiqadi (masalan, tasniflash uchun sinf).

MLP o'qitish jarayoni:

1. **Oldindan chiqish (Forward Propagation):** Kirish ma'lumotlari birinchi yashirin qatlamga yuboriladi, undan keyin chiqish qatlamiga o'tkaziladi.

2. **Xatolikni hisoblash (Loss Calculation):** Modelning chiqish qiymatlari va haqiqiy natijalar o'rtasidagi farq (yo'qotish funksiyasi) hisoblanadi.

3. **Orqaga tarqatish (Backpropagation):** Xatolikni tarmoqning barcha qatlamlariga orqaga yuborish va og'irliklarni yangilash.

4. **Og'irliklarni yangilash (Weight Update):** Og'irliklar gradient descent yordamida yangilanadi.

MLP ning afzalliklari:

- Murakkab masalalarni hal qilishda samarali.
- Katta ma'lumotlar to'plamlarida yaxshi ishlaydi.
- Ikkilamchi qatlamlar (hidden layers) yordamida o'zgaruvchilar o'rtasidagi noaniqliklarni aniqlash imkonini beradi.

Amaliy Mashg'ulot:

1. Titanic datasetida tasniflash:

- **MLP** yordamida Titanic datasetida yo'lovchilarni hayotga qolgan va halok bo'lganlarga ajratish.
- O'qitish jarayonini boshqarish, og'irliklarni yangilash va modelni baholash.

2. MNIST datasetida tasvirni tasniflash:

- **MLP** yordamida MNIST (qo'l yozuvi raqamlari) datasetida tasvirlarni tanib olish.
- Tarmoqni o'qitish, xatolikni hisoblash va natijalarni tahlil qilish.

3. Boston Housing datasetida narxlarni prognoz qilish:

- **MLP** yordamida Boston Housing datasetida uy narxlarini prognoz qilish.
- Regressiya vazifasini bajarish uchun MLP ni sozlash.

Amaliy Mashg'ulot Natijalari:

1. **Modelni o'qitish:** Dastlabki datasetni yuklash, oldindan ishlov berish (preprocessing) va MLP modelini qurish.

2. **Modelni baholash:** Modelni test ma'lumotlari bilan sinovdan o'tkazish va aniqlikni, yo'qotish (loss) va boshqa ko'rsatkichlarni baholash.

3. **Natijalarni taqqoslash:** Modelning ishlashini, sinifikatsiya yoki regressiya masalalarida natijalarni tahlil qilish.

1. Titanic Dataset: MLP bilan Klassifikatsiya

Titanic datasetida MLP bilan klassifikatsiya qilish:

```
# Zarur kutubxonalar
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report

# Titanic datasetini yuklash
url = "https://web.stanford.edu/class/archive/cs/cs109/cs109.1166/stuff/titanic.csv"
data = pd.read_csv(url)
```

```

# Keraksiz ustunlarni o'chirish va ma'lumotni tayyorlash
data = data.drop(['Name', 'Ticket', 'Cabin'], axis=1)
data = data.dropna()

# Xususiyatlar va maqsadni ajratish
X = data[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']]
y = data['Survived']

# Categorical ustunlarni o'zgartirish (One-hot encoding)
X = pd.get_dummies(X, columns=['Sex'], drop_first=True)

# Ma'lumotlarni o'qitish va testga bo'lish
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Ma'lumotlarni normalizatsiya qilish
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# MLP modelini yaratish va o'qitish
mlp_model = MLPClassifier(hidden_layer_sizes=(10,), max_iter=1000,
random_state=42)
mlp_model.fit(X_train, y_train)

# Modelni baholash
y_pred = mlp_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

# Klassifikatsiya hisobotini chiqarish
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

Izohlar:

- Titanic datasetini **Pandas** yordamida yuklab, kerakli ma'lumotlarni tayyorlaymiz.
- **MLPClassifier** yordamida model quriladi va o'qitiladi.
- **Accuracy** va **Classification Report** yordamida modelni baholash mumkin.

2. MNIST Dataset: MLP bilan Tasvirni Tasniflash***MNIST datasetida MLP bilan tasvirlarni tasniflash:***

```

# Zarur kutubxonalar
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# MNIST datasetini yuklash
digits = load_digits()

```

```

X_digits = digits.data
y_digits = digits.target

# Ma'lumotlarni o'qitish va testga bo'lish
X_train, X_test, y_train, y_test = train_test_split(X_digits, y_digits,
test_size=0.2, random_state=42)

# MLP modelini yaratish va o'qitish
mlp_model = MLPClassifier(hidden_layer_sizes=(64,), max_iter=500,
random_state=42)
mlp_model.fit(X_train, y_train)

# Modelni baholash
y_pred = mlp_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

# Confusion Matrixni chiqarish
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

```

Izohlar:

- **MNIST dataset** bu qo'l yozuvi raqamlaridan tashkil topgan tasvirlar to'plamidir.
- **MLPClassifier** yordamida tasvirlarni tasniflashni amalga oshiramiz.
- **Accuracy** va **Confusion Matrix** yordamida modelning samaradorligini tahlil qilish mumkin.

3. Boston Housing Dataset: MLP bilan Regression***Boston Housing datasetida uy narxlarini prognoz qilish (regression):***

```

# Zarur kutubxonalar
from sklearn.datasets import load_boston
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Boston datasetini yuklash
boston = load_boston()
X_boston = boston.data
y_boston = boston.target

# Ma'lumotlarni o'qitish va testga bo'lish
X_train, X_test, y_train, y_test = train_test_split(X_boston, y_boston,
test_size=0.2, random_state=42)

# MLP modelini yaratish va o'qitish
mlp_model = MLPRegressor(hidden_layer_sizes=(64,), max_iter=500,
random_state=42)
mlp_model.fit(X_train, y_train)

```

```
# Modelni baholash
y_pred = mlp_model.predict(X_test)
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R^2 Score:", r2_score(y_test, y_pred))
```

Izohlar:

MLPRegressor yordamida **Boston Housing dataset**dagi uy narxlarini prognoz qilish uchun regressiya modelini qurish.

Mean Squared Error (MSE) va **R² score** yordamida modelning samaradorligini baholash.

Har bir kodda **MLPClassifier** (klassifikatsiya) yoki **MLPRegressor** (regressiya) ishlatilgan, bu sun'iy neyron tarmoqlari yordamida ishlashni osonlashtiradi.

Datasetlarni **train_test_split** yordamida bo'lish va **StandardScaler** yordamida normalizatsiya qilish orqali tarmoqning o'qitish samaradorligini oshirish mumkin.

Modelni baholash uchun **Accuracy**, **Confusion Matrix**, **MSE**, va **R² score** kabi ko'rsatkichlar ishlatiladi.

Nazorat savollari:

1. Sun'iy neyron tarmog'ining asosiy qismlari nimalardan iborat?
2. Perceptron modeli qanday ishlaydi?
3. MLP va Perceptron o'rtasidagi farqni tushuntiring.
4. Gradient Descent nima?
5. ReLU faollashtirish funksiyasining afzalliklari nimalar?
6. MNIST datasetida tasvirlarni qanday tasniflash mumkin?
7. Boston Housing datasetida MLP bilan uy narxlarini prognoz qilish qanday ishlaydi?

AMALIY MASHG'ULOT UCHUN O'QUV MATERIALLARI

8-Mavzu: Mashinali o'qitishda Klassifikatsiya va regressiya masalalari

5-mashg'ulot. Scikit-learn paketidan foydalanish.

O'quv savollari:

1. Scikit-learn yordamida neyron tarmoqlar yaratish.
2. Perceptron modeli va tarmoqlarni o'qitish mexanizmi.
3. Oddiy sinfiylashtirish masalalarida tatbiq qilish.

Amaliy Mashg'ulot: Scikit-learn yordamida Neyron Tarmoqlar Yaratish

1. Scikit-learn yordamida neyron tarmoqlar yaratish

Scikit-learn kutubxonasi **MLPClassifier** va **MLPRegressor** yordamida neyron tarmoqlarini yaratish imkonini beradi. **MLPClassifier** klassifikatsiya vazifalari uchun ishlatiladi, **MLPRegressor** esa regressiya vazifalari uchun.

Kutilgan natijalar:

Neyron tarmog'ini yaratish va o'qitish.

Modelni baholash va natijalarni taqqoslash.

Kod: Perceptron Modeli bilan Oddiy Klassifikatsiya Masalasi

Biz **Iris datasetini** ishlatamiz, bu datasetda 3 xil gul turi mavjud va biz ular orasida tasniflashni amalga oshiramiz.

```
# Zarur kutubxonalar
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report

# Iris datasetini yuklash
iris = load_iris()
X = iris.data
y = iris.target

# Ma'lumotlarni o'qitish va testga bo'lish
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Ma'lumotlarni normalizatsiya qilish
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# MLPClassifier modelini yaratish va o'qitish
mlp_model = MLPClassifier(hidden_layer_sizes=(10,), max_iter=1000,
random_state=42)
```

```
mlp_model.fit(X_train, y_train)

# Modelni baholash
y_pred = mlp_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

# Klassifikatsiya hisobotini chiqarish
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Izohlar:

Iris datasetni Scikit-learn kutubxonasidan yuklaymiz.

Ma'lumotlarni **train_test_split** yordamida o'qitish va test ma'lumotlariga bo'lamiz.

StandardScaler yordamida ma'lumotlarni normalizatsiya qilamiz.

MLPClassifier yordamida neyron tarmog'ini yaratamiz va o'qitamiz.

Accuracy va **Classification Report** yordamida modelni baholaymiz.

2. Perceptron Modeli va Tarmoqlarni O'qitish Mexanizmi

Perceptron — bu oddiy neyron modeli bo'lib, u binar klassifikatsiya vazifalarini bajaradi. Scikit-learn kutubxonasida **Perceptron** klassi yordamida uni yaratish va o'qitish mumkin.

Kod: Perceptron Modeli

```
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score

# Perceptron modelini yaratish va o'qitish
perceptron_model = Perceptron(max_iter=1000, random_state=42)
perceptron_model.fit(X_train, y_train)

# Modelni baholash
y_pred_perceptron = perceptron_model.predict(X_test)
print("Perceptron Accuracy:", accuracy_score(y_test, y_pred_perceptron))
```

Izohlar:

Perceptron modeli Scikit-learn kutubxonasidan olingan.

Modelni o'qitish va test ma'lumotlari bo'yicha **accuracy** ni baholash.

3. Oddiy Sinfiylashtirish Masalalarida Tatbiq Qilish

Breast Cancer dataset yordamida **MLPClassifier** ni sinfiylashtirish masalasida tatbiq qilish. Bu datasetda rak kasalligi haqidagi ma'lumotlar mavjud va biz 2 sinfni (yaxshi yoki yomon) tasniflashni amalga oshiramiz.

Kod: Sinfiylashtirish Masalasi (Breast Cancer Dataset)

```
# Zarur kutubxonalar
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
```

```

from sklearn.metrics import accuracy_score, classification_report

# Breast Cancer datasetini yuklash
cancer = load_breast_cancer()
X = cancer.data
y = cancer.target

# Ma'lumotlarni o'qitish va testga bo'lish
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Ma'lumotlarni normalizatsiya qilish
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# MLPClassifier modelini yaratish va o'qitish
mlp_model = MLPClassifier(hidden_layer_sizes=(10,), max_iter=1000,
random_state=42)
mlp_model.fit(X_train, y_train)

# Modelni baholash
y_pred = mlp_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

# Klassifikatsiya hisobotini chiqarish
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

Izohlar:

Breast Cancer datasetni Scikit-learn kutubxonasidan yuklaymiz.

Ma'lumotlarni **train_test_split** yordamida bo'lamiz.

Modelni **MLPClassifier** yordamida o'qitamiz va **accuracy** va **classification report** bilan baholaymiz.

Yakuniy Natijalar:

Perceptron va **MLPClassifier** yordamida oddiy klassifikatsiya masalalarini hal qilish.

Accuracy va **classification report** yordamida modelni baholash.

Scikit-learn — bu mashinani o'rganish uchun eng mashhur Python kutubxonalaridan biri bo'lib, u bir nechta sodda va samarali algoritmlarni o'z ichiga oladi. Scikit-learn yordamida neyron tarmoqlarini yaratish, o'qitish va baholash juda oson. U **MLPClassifier** (klassifikatsiya uchun) va **MLPRegressor** (regressiya uchun) kabi neyron tarmoq modellari bilan ta'minlaydi.

MLP (Multi-layer Perceptron) — bu ko'p qatlamli neyron tarmog'i bo'lib, turli maqsadlar uchun ishlatiladi. MLPning asosiy komponentlari:

1. **Kirish qatlamlari (Input layer):** Modelga kirish uchun ma'lumotlar kiritiladi.

2. **Yashirin qatlamlar (Hidden layers):** Model ma'lumotni qayta ishlaydi. Har bir qatlamdagi neyronlar boshqa qatlamlar bilan bog'langan.

3. **Chiqish qatlamlari (Output layer):** Natijalar hisoblanadi.

MLPClassifierning afzalliklari:

Ko'p qatlamli neyron tarmog'i bo'lib, u juda samarali ishlaydi.

Ko'plab sinfiylashtirish va regressiya masalalarini hal qilishda yuqori aniqlikni ta'minlaydi.

Oson o'rganish va ba'zi qo'llanmalar yordamida tezda moslashtirish mumkin.

Modelni o'qitish mexanizmi:

1. **Oldindan tarqatish (Forward Propagation):** Kirish ma'lumotlari modelga yuboriladi va qatlamlar bo'yicha tarqatiladi.

2. **Xato hisoblash (Error Calculation):** Chiqish va haqiqiy qiymat o'rtasidagi xato hisoblanadi.

3. **Orqaga tarqatish (Backpropagation):** Xatolikni ortga yuborish orqali og'irliklarni yangilash.

4. **Og'irliklarni yangilash (Weight Update):** Xatolikni minimallashtirish uchun og'irliklar yangilanadi.

Nazorat Savollari

1. Scikit-learn yordamida neyron tarmoqlarini qanday yaratish mumkin?
2. MLPClassifier va Perceptron modeli o'rtasidagi farqni tushuntiring.
3. MLP modelining asosiy qismlari nimalardan iborat?
4. Scikit-learn kutubxonasida Perceptron modelini o'qitishda qanday parametrlar sozlanadi?
5. Gradient Descent algoritmi nima va u qanday ishlaydi?
6. Breast Cancer datasetida sinfiylashtirishni qanday amalga oshirasiz?
7. MLPClassifier yordamida modelni qanday baholaysiz va uning samaradorligini qanday tekshirasiz?

**O'ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN'IY INTELLEKT”
KAFEDRASI**



“SUN'IY INTELLEKT ASOSLARI”

**FANI BO'YICHA MASHG'ULOTLAR
O'TISH REJALARI**

	Python to PATH” tanlanadi, shundan so’ng “Install Now” tugmasi bosiladi. PyCharm dasturi esa JetBrains kompaniyasi tomonidan ishlab chiqilgan qulay muhit bo’lib, u Python kodini yozish, sinovdan o’tkazish va boshqarishni soddalashtiradi. 3. Pythonda “Hello world!” dasturini tuzish. “Hello world!” dasturi Python tilidagi eng oddiy dastur bo’lib, dasturlashga kirish uchun ishlatiladi. Dastur quyidagicha yoziladi: <code>print("Hello, world!")</code>. Bu dastur <code>print()</code> funksiyasi orqali ekranga matn chiqaradi va foydalanuvchiga Pythonning ishlash tamoyilini ko’rsatadi. 4. Python tilining asosiy operatorlari bilan tanishish. Python tilida operatorlar – bu amallarni bajaruvchi belgilar yoki so’zlardir. Ular arifmetik (+, -, *, /), taqqoslash (==, >, <), va mantiqiy (and, or, not) turlarga bo’linadi. Ushbu operatorlar yordamida sonli, mantiqiy va matnli ma’lumotlar ustida turli hisob-kitoblar va shartli tahlillar bajariladi.	15	
		15	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo’yilgan maqsadga mashg’ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg’ulot mavzusi va o’tish joyini e’lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG‘ULOT O‘TISH REJASI

1-MAVZU. 2-MASHG‘ULOT.

Mavzu: Satrlar bilan ishlovchi operatorlar va metodlar.

Mashg‘ulotning o‘quv va tarbiyaviy maqsadi: Kursantlarda Python tilida satrlar (matnlar) bilan ishlovchi operatorlar va metodlar bo‘yicha asosiy tushunchalarni shakllantirish hamda matnli ma’lumotlarni tahlil qilish va qayta ishlash ko‘nikmalarini mustahkamlashdan iborat.

O‘quv guruhleri: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O‘tish joyi: 437-, 138-o‘quv sinfi.

Mashg‘ulot turi: Guruhli.

O‘quv-moddiy ta’minot: Taqdimot va elektron ma’lumotlar.

O‘quv texnik vositalar: Videoproyektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. T.A. Xo‘jaqulov, N.T. Malikova. Sun’iy intellekt (O‘quv qo‘llanma). - T.: “Innovatsion rivojlanish nashriyot- matbaa uyi”, 2020. -216 b.

2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. “Python dasturlash tili” Darslik. Toshkent: 2024y. B - 316.

3. Sh.R. Sapayev “Python dasturlash tili asoslari”. O‘quv qo‘llanma. Toshkent: 2023y. B – 137.

4. Sh.R. Sapayev “PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish”. O‘quv qo‘llanma. Toshkent: 2024y. B - 150

O‘QUV SAVOLLARI VA VAQT HISOBI

№	O‘quvsavollari	Vaqt, min	Ko‘rgazmali qo‘llanma va O‘TV
---	----------------	-----------	-------------------------------

1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg‘ulot maqsadini e‘lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O‘quv savollari: 1. Satrlar bilan ishlovchi operatorlar va metodlar haqida umumiy ma’lumot. Kursantlarda Python tilida satrlar ustida amallar bajaruvchi operatorlar va metodlar bo‘yicha asosiy tushunchalarni shakllantirish hamda ularni amaliy dasturlashda qo‘llash ko‘nikmalarini mustahkamlashdan iborat. 2. str.format() metodi yordamida satrlarni formatlash. Kursantlarda Python tilida <code>str.format()</code> metodi yordamida satrlarni dinamik tarzda shakllantirish va turli qiymatlarni matnga joylashtirish bo‘yicha bilim va ko‘nikmalarni rivojlantirishdan iborat. 3. Satrlarni kodlash va dekodlash (encode() va decode()) metodlari yordamida baytlar bilan ishlash. Kursantlarda Python tilida matnli ma’lumotlarni bayt ko‘rinishiga o‘tkazish va ularni qayta tiklash (kodlash va dekodlash) jarayonlarini <code>encode()</code> va <code>decode()</code> metodlari yordamida bajarish ko‘nikmalarini mustahkamlashdan iborat.	60 20 20 20	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e‘lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG‘ULOT O‘TISH REJASI

1-MAVZU. 3-MASHG‘ULOT.

Mavzu: Satrlar bilan ishlovchi operatorlar va metodlar.

Mashg‘ulotning o‘quv va tarbiyaviy maqsadi: Kursantlarda Python tilidagi arifmetik operatorlar bo‘yicha asosiy tushunchalar hamda ularning amaliy sohalarda qo‘llanilishi yuzasidan bilim va ko‘nikmalarni mustahkamlashdan iborat.

O‘quv guruhleri: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O‘tish joyi: 437-, 138-o‘quv sinfi.

Mashg‘ulot turi: Guruhli.

O‘quv-moddiy ta’minot: Taqdimot va elektron ma’lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. T.A. Xo‘jaqulov, N.T. Malikova. Sun‘iy intellekt (O‘quv qo‘llanma). - T.: “Innovatsion rivojlanish nashriyot- matbaa uyi”, 2020. -216 b.

2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Arifmetik operatorlar. Kursantlarda Python tilidagi arifmetik operatorlar bo'yicha asosiy tushunchalar hamda ularning dasturlash jarayonidagi qo'llanilishi yuzasidan bilim va ko'nikmalarni mustahkamlashdan iborat. 2. Sonlar bilan ishlash. O'zgaruvchilar. Kursantlarda Python tilida sonlar turlari va o'zgaruvchilarni yaratish, ularga qiymat berish hamda ular bilan amallar bajarish bo'yicha amaliy bilim va ko'nikmalarni shakllantirishdan iborat. 3. Bool tipli ma'lumotlar bilan ishlash. Kursantlarda Python tilidagi mantiqiy (bool) ma'lumotlar turlari, ularning qiymatlari va shartli ifodalar orqali qo'llanilishi bo'yicha bilim va amaliy ko'nikmalarni rivojlantirishdan iborat.	60 20 20 20	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

1-MAVZU. 4-MASHG'ULOT.

Mavzu: Pythonda shart operatori uning qo'llanilishi.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python tilidagi shart operatori

va uning qo'llanilishi bo'yicha asosiy tushunchalarni shakllantirish hamda dasturda mantiqiy qarorlar qabul qilish ko'nikmalarini mustahkamlashdan iborat.

O'quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Guruhli.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. IF, IF-ELSE va IF-ELIF-ELSE operatorlari. Python tilida shart operatorlari dastur bajarilishini ma'lum shartlar asosida boshqarish uchun qo'llaniladi. <code>if</code> operatori shart to'g'ri bo'lsa, tegishli kodni bajaradi, <code>if-else</code> esa shart bajarilmaganda boshqa yo'nalishni tanlash imkonini beradi. <code>if-elif-else</code> operatorlari ketma-ket bir nechta shartlarni tekshirish uchun ishlatiladi. Ushbu operatorlar yordamida dastur mantiqiy qarorlar qabul qilib, har xil natijalar chiqarishi mumkin. 2. Shart operatorlarida mantiqiy ifodalar (and, or, not) ning qo'llanilishi. Mantiqiy operatorlar (<code>and</code>, <code>or</code>, <code>not</code>) bir nechta shartlarni birlashtirish yoki teskari ma'noga o'tkazish uchun ishlatiladi. Masalan, <code>and</code> ikkala shart ham to'g'ri bo'lsa natija beradi, <code>or</code> esa kamida bittasi to'g'ri bo'lsa yetarli, <code>not</code> esa shartni inkor qiladi. Bu operatorlar shartli ifodalarning aniqligini oshiradi va dasturdagi qaror qabul qilish jarayonini soddalashtiradi. 3. Ichma-ich (nested) IF operatorlaridan foydalanish, ularning afzallik va kamchiliklari. Ichma-ich <code>if</code> operatorlari bir shart ichida boshqa shartni tekshirish zarur bo'lgan holatlarda qo'llaniladi. Ular murakkab	60 20 20	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot

	mantiqiy tekshiruvlarni amalga oshirish imkonini beradi va dastur qarorlarini chuqur nazorat qilishga yordam beradi. Biroq, ichma-ich iflardan haddan ortiq foydalanish kodni murakkablashtiradi va o'qilishini qiyinlashtiradi. Shuning uchun amaliyotda ularni faqat zarur holatlarda, soddalikka rioya qilgan holda ishlatish tavsiya etiladi.	20	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IIY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

1-MAVZU. 5-MASHG'ULOT.

Mavzu: Pythonda shart operatori, satrlarga doir masalalar yechish. Shart operatoriga doir dasturlari tuzish.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python tilidagi shart operatorlari va ularning satrlarga doir masalalarni yechishda qo'llanilishi bo'yicha bilim hamda amaliy ko'nikmalarni shakllantirish, shuningdek, turli mantiqiy shartlarga asoslangan dasturlar tuzish malakasini mustahkamlashdan iborat.

O'quv guruhlari: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Amaliy.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproyektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: “Innovatsion rivojlanish nashriyot- matbaa uyi”, 2020. -216 b.

2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. “Python dasturlash tili” Darslik. Toshkent: 2024y. B - 316.

3. Sh.R. Sapayev “Python dasturlash tili asoslari”. O'quv qo'llanma. Toshkent: 2023y. B – 137.

4. Sh.R. Sapayev “PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish”. O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	

2.	II. ASOSIY QISM O'quv savollari: 1. IF, IF-ELSE va IF-ELIF-ELSE operatorlariga doir dasturlar tuzish. Kursantlarda Python tilidagi shart operatorlaridan (if, if-else, if-elif-else) foydalangan holda turli mantiqiy masalalarni yechish bo'yicha amaliy ko'nikmalarni shakllantirish maqsad qilinadi. Ushbu dasturlar yordamida foydalanuvchidan olingan ma'lumotlarga qarab turli qarorlar chiqarish o'rgatiladi. Natijada kursantlar dastur oqimini shartlarga mos ravishda boshqarishni o'zlashtiradilar. 2. Pythonda satrlarga doir masalalar yechish. Bu mavzuda kursantlar Python tilida satr (matn) turlari bilan ishlashni, ularni tahlil qilish, o'zgartirish va formatlashni o'rganadilar. Amaliy mashg'ulotlarda turli satr metodlari (len(), upper(), lower(), replace(), find()) yordamida masalalar yechiladi. Ushbu bilimlar kursantlarga matnli ma'lumotlar bilan samarali ishlash ko'nikmasini beradi.	60 30 30	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IIY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

1-MAVZU. 6-MASHG'ULOT.

Mavzu: Pythonda takrorlanuvchi jarayonlarni dasturlash

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python tilida takrorlanuvchi jarayonlarni boshqaruvchi operatorlar (`for` va `while`) bo'yicha asosiy tushunchalarni shakllantirish hamda ularning yordamida turli algoritmik masalalarni yechish va amaliy dasturlar tuzish ko'nikmalarini mustahkamlashdan iborat.

O'quv guruhleri: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Guruhli.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: “Innovatsion rivojlanish nashriyot- matbaa uyi”, 2020. -216 b.

2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. “Python dasturlash tili” Darslik. Toshkent: 2024y. B - 316.

3. Sh.R. Sapayev “Python dasturlash tili asoslari”. O'quv qo'llanma. Toshkent: 2023y. B – 137.

4. Sh.R. Sapayev “PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish”. O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Sikl operatorlari – For va While bilan ishlash. Kursantlarda Python tilidagi sikl operatorlari (<code>for</code> va <code>while</code>) orqali takrorlanuvchi jarayonlarni tashkil etish bo'yicha bilim va amaliy ko'nikmalarni shakllantirish maqsad qilinadi. <code>for</code> operatori elementlar to'plami bo'ylab aylanish uchun, <code>while</code> esa berilgan shart bajarilguncha siklni davom ettirish uchun qo'llaniladi. Ushbu mavzu takrorlanuvchi hisoblash jarayonlarini avtomatlashtirishni o'rgatadi. 2. Break, Continue operatorlarining qo'llanilishi. Kursantlarda sikllar ichida <code>break</code> va <code>continue</code> operatorlarining vazifasi va ularning dastur oqimini boshqarishdagi rolini tushunish ko'nikmasi shakllantiriladi. <code>break</code> operatori siklni to'xtatadi, <code>continue</code> esa siklning joriy iteratsiyasini o'tkazib yuboradi. Bu operatorlar yordamida dastur mantiqiy boshqaruvini soddalashtirish va samaradorligini oshirish o'rgatiladi.	60 30 30	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IIY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

1-MAVZU. 7-MASHG'ULOT.

Mavzu: Pythonda `for`, `while`, `break` va `continue` operatorlariga doir dasturlar tuzish

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python tilida takrorlanuvchi jarayonlarni boshqaruvchi operatorlar (`for` va `while`) bo'yicha asosiy tushunchalarni shakllantirish hamda ularning yordamida turli algoritmik masalalarni yechish va amaliy dasturlar tuzish ko'nikmalarini mustahkamlashdan iborat.

O'quv guruhleri: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Guruhli.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Sikl operatorlari – For va While ga doir dasturlar tuzish. Kursantlarda Python tilidagi <code>for</code> va <code>while</code> sikl operatorlaridan foydalangan holda takrorlanuvchi jarayonlarni dasturlash va amaliy masalalarni yechish ko'nikmalarini shakllantirish maqsad qilinadi. Ular sikl orqali sonlar yig'indisini hisoblash, ro'yxat elementlarini tahlil qilish yoki ma'lum shart bajarilguncha takrorlanadigan jarayonlarni tashkil etishni o'rganadilar. Natijada, kursantlar algoritmik fikrlashni mustahkamlab, kod tuzishda sikllardan to'g'ri foydalanishni o'zlashtiradilar. 2. Break, Continue va Else operatorlariga doir dasturlar tuzish. Bu mavzuda kursantlarda <code>break</code>, <code>continue</code> va <code>else</code> operatorlarining sikl jarayonidagi vazifasini amaliy misollar orqali tushunish ko'nikmalari shakllantiriladi. <code>break</code> operatori siklni muddatidan oldin to'xtatish uchun, <code>continue</code> esa ayrim iteratsiyalarni o'tkazib yuborish uchun ishlatiladi, <code>else</code> esa sikl tabiiy yakunlanganda bajariladigan kodni ifodalaydi. Ushbu operatorlar yordamida sikllarning ishlash mantiqini chuqur tushunish va murakkab dastur oqimlarini boshqarish o'rgatiladi.	60 30 30	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

1-MAVZU. 8-MASHG'ULOT.

Mavzu: Pythonda ro'yxatlar va ular bilan ishlovchi metodlar.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python tilidagi ro'yxatlar (list) va ular bilan ishlovchi asosiy metodlar bo'yicha bilim va amaliy ko'nikmalarni shakllantirish, ma'lumotlar to'plamini yaratish, o'zgartirish hamda tahlil qilish jarayonlarini samarali tashkil etishni o'rganishdan iborat.

O'quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Guruhli.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Ro'yxatlar va ularning qo'llanilishi. Kursantlarda Python tilidagi ro'yxatlar (list) tushunchasi, ularning ma'lumotlarni tartibli saqlashdagi o'rni va dasturlashdagi amaliy qo'llanilishi bo'yicha bilimlarni shakllantirish maqsad qilinadi. Ro'yxatlar bir nechta qiymatlarni bitta o'zgaruvchida saqlash imkonini beradi va dasturda ma'lumotlar to'plamini boshqarishni soddalashtiradi. Ular dasturlashda eng ko'p qo'llaniladigan ma'lumotlar tuzilmasi hisoblanadi. 2. Ro'yxatlarni yaratish usullari. Kursantlarda Python tilida ro'yxatlar yaratishning turli usullari — kvadrat qavslar yordamida ([]), list() funksiyasi orqali yoki takrorlanuvchi jarayonlardan (sikl, comprehension)	60 20 20	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot

	foydalanish orqali amalga oshirilishini o'rganish ko'nikmasi shakllantiriladi. Bu usullar ma'lumotlarni turlicha shakllarda yig'ish va tahlil qilish imkonini beradi. Amaliy mashg'ulotlarda kursantlar har xil turdagi ro'yxatlar yaratishni o'zlashtiradilar. 3. Ro'yxatlar bilan ishlovchi metodlar. Kursantlarda ro'yxatlar bilan ishlovchi metodlar — <code>append()</code>, <code>insert()</code>, <code>remove()</code>, <code>pop()</code>, <code>sort()</code>, <code>reverse()</code>, <code>count()</code>, <code>index()</code> kabi buyruqlarni amaliy misollar orqali o'rganish ko'nikmasi shakllantiriladi. Ushbu metodlar yordamida ro'yxat elementlarini qo'shish, o'chirish, tartiblash va tahlil qilish amallari bajariladi. Natijada kursantlar ma'lumotlar bilan samarali ishlashni o'rganadilar.	20	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

1-MAVZU. 9-MASHG'ULOT.

Mavzu: Pythonda ro'yxatlarga doir masalalar yechish.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python tilidagi ro'yxatlar bilan ishlash bo'yicha amaliy bilim va ko'nikmalarni mustahkamlash, ro'yxatlar ustida turli amallar bajarish, tahlil qilish hamda real masalalarni ro'yxatlar yordamida yechish malakasini rivojlantirishdan iborat.

O'quv guruhлари: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Amaliy.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproyektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: “Innovatsion rivojlanish nashriyot- matbaa uyi”, 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. “Python dasturlash tili” Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev “Python dasturlash tili asoslari”. O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev “PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish”. O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
---	----------------	-----------	-------------------------------

1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg‘ulot maqsadini e‘lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O‘quv savollari: 1. Pythonda ro‘yxatlarga doir oddiy masalalar yechish. Kursantlarda Python tilida ro‘yxatlar bilan ishlash bo‘yicha dastlabki amaliy ko‘nikmalarni shakllantirish, ro‘yxat elementlarini kiritish, o‘qish, o‘zgartirish va ularni tahlil qilishni o‘rganish maqsad qilinadi. Oddiy masalalar orqali kursantlar ro‘yxatlar ustida asosiy amallarni bajarishni, sikl va shart operatorlari yordamida ular bilan ishlashni o‘zlashtiradilar. Bu orqali ular dasturda ma‘lumotlar to‘plamini boshqarishning amaliy usullarini o‘rganadilar. 2. Ro‘yxatlarni ishlovchi metodlarga doir masalalar yechish. Kursantlarda Python ro‘yxat metodlari (<code>append()</code>, <code>remove()</code>, <code>sort()</code>, <code>reverse()</code>, <code>count()</code>, <code>index()</code> va boshqalar) yordamida amaliy masalalarni yechish ko‘nikmalarini rivojlantirish maqsad qilinadi. Ular ro‘yxatlar ustida ma‘lumot qo‘shish, o‘chirish, tartiblash va izlash kabi jarayonlarni dasturlashni o‘rganadilar. Bu mavzu kursantlarga real hayotdagi ma‘lumotlar ustida amaliy ishlash malakasini beradi.	60	
		30	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
		30	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e‘lon qilish.	10	

“SUN'IIY INTELLEKT ASOSLARI” FANIDAN MASHG‘ULOT O‘TISH REJASI

1-MAVZU. 10-MASHG‘ULOT.

Mavzu: Pythonda Funksiya tushunchasi. Foydalanuvchi funksiyasi.

Mashg‘ulotning o‘quv va tarbiyaviy maqsadi: Kursantlarda Python tilida funksiya tushunchasi va foydalanuvchi tomonidan yaratiladigan funksiyalar bo‘yicha asosiy bilim hamda ko‘nikmalarni shakllantirish, dastur kodini tuzilmalarga ajratish va takrorlanuvchi jarayonlarni soddalashtirishni o‘rganishdan iborat.

O‘quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O‘tish joyi: 437-, 138-o‘quv sinfi.

Mashg‘ulot turi: Guruhli.

O‘quv-moddiy ta‘minot: Taqdimot va elektron ma‘lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.

2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.

3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.

4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Funksiyalarni aniqlash va uni chaqirish. Kursantlarda Python tilida funksiyalarni aniqlash (<code>def</code> operatori yordamida) va ularni chaqirish bo'yicha asosiy tushunchalarni shakllantirish maqsad qilinadi. Funksiya dasturda ma'lum bir vazifani bajaruvchi mustaqil kod bo'lagi bo'lib, uni qayta-qayta chaqirish dastur tuzilmasini soddalashtiradi. Ushbu mavzu kursantlarga kodni modullarga ajratish va takrorlanuvchi jarayonlarni samarali boshqarish ko'nikmasini beradi. 2. Parametrli va parametrsiz funksiya. Kursantlarda Python tilidagi parametrli va parametrsiz funksiyalar orasidagi farqlarni anglash, ulardan vaziyatga mos holda foydalanish ko'nikmasi shakllantiriladi. Parametrli funksiya kirish qiymatlarini qabul qilib, natijani qaytaradi, parametrsiz funksiya esa oldindan belgilangan amallarni bajaradi. Bu mavzu kursantlarga funksiyalarni moslashuvchan va qayta ishlatiladigan tarzda yozishni o'rgatadi.	60 30 30	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

1-MAVZU. 11-MASHG'ULOT.

Mavzu: Pythonda Funksiyaga doir dasturlar tuzish.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python tilida funksiyalar yaratish va ulardan foydalanish bo'yicha amaliy ko'nikmalarni mustahkamlash, turli parametrlil funksiyalar asosida dasturlar tuzish hamda kodni modul tarzida tashkil etish malakasini rivojlantirishdan iborat.

O'quv guruhlari: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Amaliy.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Funksiyalarni aniqlash va uni chaqirishni o'rganish. Kursantlarda Python tilida funksiyalarni <code>def</code> operatori yordamida aniqlash va ularni chaqirish jarayoni bo'yicha amaliy bilim va ko'nikmalarni shakllantirish maqsad qilinadi. Ular funksiyaning tuzilishi, nomlanishi va bajarilish tartibini o'rganadilar. Natijada kursantlar dastur kodini soddalashtirish va qayta foydalanish imkonini beruvchi funksional yondashuvni o'zlashtiradilar. 2. Parametrli va paramsiz funksiyalarga doir dastur tuzish. Kursantlarda parametrli va paramsiz funksiyalar asosida turli amaliy masalalarni yechish bo'yicha ko'nikmalarni shakllantirish maqsad qilinadi. Parametrli funksiyalar yordamida kirish ma'lumotlarini qayta ishlash, paramsiz funksiyalar orqali esa umumiy jarayonlarni bajarish dasturlari tuziladi. Bu orqali kursantlar kodni modulli tashkil etish va funksiyalarni samarali qo'llashni o'rganadilar.	60 30 30	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot

	jarayonlari o'rganiladi. Bu mavzu kursantlarga ma'lumotlarni saqlash va qayta ishlashning dasturiy usullarini o'rgatadi. 2. os modulining imkoniyatlari. Kursantlarda os moduli yordamida operatsion tizim bilan o'zaro aloqada bo'lish, fayl va kataloglar ustida amallar bajarish bo'yicha bilim va ko'nikmalarni shakllantirish maqsad qilinadi. <code>os.getcwd()</code>, <code>os.listdir()</code>, <code>os.mkdir()</code>, <code>os.remove()</code> kabi funksiyalar orqali tizimdagi fayl tuzilmasini boshqarish imkoniyatlari o'rganiladi. Ushbu mavzu kursantlarga Python yordamida fayl tizimida avtomatlashtirilgan ishlarni bajarishni o'rgatadi. 3. Fayl va katalog yo'lini o'zgartirish. Kursantlarda Python tilida fayl va kataloglarning yo'lini o'zgartirish, yangi katalogga o'tish va ma'lumotlar joylashuvini boshqarish ko'nikmalari shakllantiriladi. <code>os.chdir()</code>, <code>os.path.join()</code> va <code>os.path.abspath()</code> funksiyalari orqali yo'lni boshqarish usullari o'rganiladi. Bu mavzu fayl tuzilmasi bilan ishlashni qulay va tizimli qilishga yordam beradi. 4. Katalog va fayl bilan ishlovchi funksiya va metodlar. Kursantlarda katalog va fayllar bilan ishlashda qo'llaniladigan <code>os</code>, <code>os.path</code> va <code>shutil</code> modullari imkoniyatlari bo'yicha bilimlarni mustahkamlash maqsad qilinadi. Bu funksiyalar yordamida katalog yaratish, o'chirish, ko'chirish, nusxalash va fayl mavjudligini tekshirish amaliyotlari bajariladi. Natijada kursantlar operatsion tizim muhitida fayl va kataloglarni boshqarishning dasturiy asoslarini o'zlashtiradilar.	15	
	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	15	
3.		10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

1-MAVZU. 13-MASHG'ULOT.

Mavzu: Fayllar va kataloglar bilan ishlashga doir masalalar yechish.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python tilida fayllar va kataloglar bilan ishlashga doir amaliy masalalarni yechish bo'yicha bilim va ko'nikmalarni mustahkamlash, ma'lumotlarni faylga yozish, o'qish hamda kataloglar ustida turli amallar bajarish dasturlarini tuzish malakasini rivojlantirishdan iborat.

O'quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Amaliy.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBİ

No	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Fayllar bilan ishlash metodlaridan foydalanish. Kursantlarda Python tilida fayllarni ochish, o'qish, yozish va yopish jarayonlarini <code>open()</code>, <code>read()</code>, <code>write()</code>, <code>close()</code> metodlari orqali bajarish bo'yicha amaliy ko'nikmalarni shakllantirish maqsad qilinadi. Ushbu metodlardan foydalanib, ma'lumotlarni faylga saqlash va ularni qayta ishlash jarayonlari o'rganiladi. Bu mavzu kursantlarga fayl tizimi bilan ishlashni avtomatlashtirish va ma'lumotlar bilan samarali ishlash imkonini beradi. 2. Kataloglar ustida amallar bajarish. Kursantlarda Python tilidagi <code>os</code> va <code>shutil</code> modullari yordamida kataloglar bilan amaliy ishlash — yangi katalog yaratish, o'chirish, nomini o'zgartirish va ichki tuzilmasini ko'rish kabi jarayonlarni bajarish bo'yicha ko'nikmalarni shakllantirish maqsad qilinadi. Ushbu mavzu orqali kursantlar operatsion tizim muhitida fayl va kataloglarni dastur orqali boshqarishni o'rganadilar. 3. Fayl va kataloglarda qidirish va nusxa olish. Kursantlarda Python yordamida fayl va kataloglarni qidirish hamda ularni nusxalash bo'yicha amaliy ko'nikmalarni shakllantirish maqsad qilinadi. <code>os.walk()</code> funksiyasi yordamida katalog tuzilmasini ko'zdan kechirish, <code>shutil.copy()</code> orqali fayllarni ko'chirish yoki nusxalash usullari o'rganiladi. Bu mavzu kursantlarga fayl tizimida avtomatik izlash va ma'lumotlarni boshqarish jarayonlarini samarali tashkil etish imkonini beradi.	60 20 20	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish.	10	

1-MAVZU. 14-MASHG‘ULOT.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python tilida obyektga yo'naltirilgan dasturlash (OOP) asoslari bo'yicha bilim va amaliy ko'nikmalarni shakllantirish, sinf (class) va obyekt (object) tushunchalarini o'zlashtirish hamda dastur tuzilmasini modulli va qayta ishlatiladigan shaklda tashkil etishni o'rganishdan iborat.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. OOP asoslari. Klasslarni e'lon qilish va nusxasini yaratish. Kursantlarda Python tilida obyektga yo'naltirilgan dasturlash (OOP) tamoyillari — klass (class) va obyekt (object) tushunchalarini o'rganish bo'yicha bilim va ko'nikmalarni shakllantirish maqsad qilinadi. Ular klasslarni <code>class</code> operatori yordamida e'lon qilish va ularning nusxalarini yaratish (<code>object = ClassName()</code>) jarayonlarini o'zlashtiradilar. Ushbu	60 20	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot

	mavzu dastur tuzilmasini modulli, tushunarli va qayta ishlatiladigan shaklda tashkil etishni o'rgatadi. 2. Klass va obyekt. Klass konstruktori. Kursantlarda klass va obyekt o'rtasidagi bog'liqlik, ularning dasturdagi vazifasi hamda obyekt xususiyatlarini belgilovchi konstruktorga oid bilim va ko'nikmalarni shakllantirish maqsad qilinadi. <code>__init__()</code> konstruktori yordamida obyektning boshlang'ich qiymatlarini o'rnatish usullari o'rganiladi. Bu mavzu kursantlarga kodni strukturaviy va mantiqiy tarzda tashkil etish imkonini beradi. 3. <code>__init__()</code> va <code>__del__()</code> metodlari. Kursantlarda Python klasslarida obyektning hayot aylanishini boshqaruvchi <code>__init__()</code> (yaratish) va <code>__del__()</code> (yo'q qilish) maxsus metodlari haqida bilimlarni shakllantirish maqsad qilinadi. <code>__init__()</code> obyekt yaratilganda avtomatik chaqiriladi, <code>__del__()</code> esa obyekt xotiradan o'chirilganda ishga tushadi. Ushbu mavzu kursantlarga obyektlarning resurslarini to'g'ri boshqarish va dastur barqarorligini ta'minlashni o'rgatadi.	20	
	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	20	
3.		10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

1-MAVZU. 15-MASHG'ULOT.

Mavzu: Pythonda Vorislikdan foydalanish.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python tilida obyektga yo'naltirilgan dasturlashning asosiy tamoyillaridan biri bo'lgan vorislik (inheritance) tushunchasi bo'yicha bilim va amaliy ko'nikmalarni shakllantirish, mavjud klasslardan yangi klasslar yaratish orqali kodni qayta ishlatish va dastur tuzilmasini samarali tashkil etishni o'rganishdan iborat.

O'quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Amaliy.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: “Innovatsion rivojlanish nashriyot- matbaa uyi”, 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. “Python dasturlash tili” Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev “Python dasturlash tili asoslari”. O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev “PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish”. O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Vorislik. Kursantlarda Python tilida obyektga yo'naltirilgan dasturlashning asosiy tamoyillaridan biri bo'lgan vorislik (inheritance) haqida bilim va amaliy ko'nikmalarni shakllantirish maqsad qilinadi. Vorislik yordamida mavjud klass asosida yangi klass yaratish, ya'ni kodni qayta ishlatish va takrorlanishni kamaytirish o'rganiladi. Bu usul dastur tuzilmasini soddalashtirish va uni modulli tarzda tashkil etishga yordam beradi. 2. Maxsus metodlar. Kursantlarda Python klasslarida ishlatiladigan maxsus (yoki "dunder") metodlar, masalan, <code>__init__()</code>, <code>__str__()</code>, <code>__add__()</code> va <code>__len__()</code> kabi metodlarning vazifalari bo'yicha bilimlarni shakllantirish maqsad qilinadi. Ushbu metodlar klass obyektlari bilan tabiiy tarzda ishlash imkonini beradi. Bu mavzu kursantlarga obyektlarning xatti-harakatlarini moslashtirish va ularni qulay boshqarish yo'llarini o'rgatadi. 3. Klass xususiyatlari. Kursantlarda Python klasslarining xususiyatlari — atributlar va metodlarning ishlash mexanizmi bo'yicha tushunchalarni shakllantirish maqsad qilinadi. Klass xususiyatlari obyektning holatini (ma'lumotlarini) saqlaydi va ular orqali obyektning funkcionalligi aniqlanadi. Ushbu mavzu kursantlarga obyektlarning ichki tuzilmasini chuqur tushunish va ularni samarali boshqarishni o'rgatadi.	60 20 20 20	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

2-MAVZU

PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

1-MASHG'ULOT.

PyQt5 paketi va uning imkoniyatlari bilan tanishish.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python tilidagi **PyQt5** paketi va uning imkoniyatlari bo'yicha asosiy bilim hamda amaliy ko'nikmalarni shakllantirish, grafik foydalanuvchi interfeyslari (GUI) yaratishning dastlabki bosqichlarini o'rganish va dasturlarda vizual elementlardan samarali foydalanishni o'zlashtirishdan iborat.

O'quv guruhleri: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Ma'ruza.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: “Innovatsion rivojlanish nashriyot- matbaa uyi”, 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. “Python dasturlash tili” Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev “Python dasturlash tili asoslari”. O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev “PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish”. O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. PyQt5 paketining imkoniyatlari. PyQt5 paketini o'rnatish. Kursantlarda PyQt5 paketi haqida umumiy tushuncha, uning yordamida grafik foydalanuvchi interfeyslari (GUI) yaratish imkoniyatlari bo'yicha bilim va ko'nikmalarni shakllantirish maqsad qilinadi. PyQt5 yordamida oynalar, tugmalar, menyular va boshqa interfeys elementlarini yaratish mumkin. Paketni o'rnatish jarayoni Python muhitida <code>pip install PyQt5</code> buyrug'i orqali amalga oshiriladi. 2. PyQt5 paketini pip yordamida o'rnatish. Kursantlarda <code>pip</code> paket boshqaruv tizimi orqali PyQt5 kutubxonasini o'rnatish va sozlash bo'yicha amaliy	60 20	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot

	ko'nikmalarni shakllantirish maqsad qilinadi. Ular buyruq satrida <code>pip install PyQt5</code> buyrug'idan foydalanishni, hamda o'rnatilgan kutubxonani <code>import PyQt5</code> orqali tekshirishni o'rganadilar. Ushbu mavzu PyQt5 muhitida dasturlash uchun zarur dastlabki sharoitlarni yaratishni o'rgatadi.	20	
	3. QtDesigner dasturini o'rnatish hamda uning imkoniyatlari bilan tanishish. Kursantlarda QtDesigner dasturini o'rnatish va uning grafik interfeyslar yaratishdagi afzalliklari bilan tanishish ko'nikmalari shakllantiriladi. QtDesigner yordamida GUI elementlarini vizual tarzda loyihalash va tayyor .ui fayllarni Python kodiga aylantirish (<code>pyuic5</code> yordamida) usullari o'rganiladi. Bu mavzu kursantlarga amaliy dasturlarda foydalanuvchi interfeyslarini qulay va tez yaratish imkonini beradi.	20	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

2-MAVZU. 2-MASHG'ULOT.

Mavzu: PyQt5 kutubxonasi. QLabel vidjeti.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python tilidagi `**PyQt5**` kutubxonasi va uning tarkibidagi QLabel vidjeti bo'yicha asosiy bilim va amaliy ko'nikmalarni shakllantirish, dastur oynasida matn, rasm yoki boshqa axborotni chiqaruvchi elementlar yaratish hamda ularni sozlash usullarini o'rganishdan iborat.

O'quv guruhleri: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Guruhli.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
---	----------------	-----------	-------------------------------

1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg‘ulot maqsadini e‘lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O‘quv savollari: 1. QLabel vidjeti. Kursantlarda PyQt5 kutubxonasidagi QLabel vidjeti haqida umumiy tushunchalarni shakllantirish maqsad qilinadi. QLabel dastur oynasida matn, rasm yoki belgilarni (ikonlarni) aks ettirish uchun ishlatiladi. Ushbu vidjet foydalanuvchi interfeysining asosiy elementlaridan biri bo‘lib, dasturda ma‘lumotni vizual tarzda ko‘rsatish imkonini beradi. 2. QLabel shrift, o‘lcham va text xususiyatlari. Kursantlarda QLabel vidjetining matn (text) xususiyatlarini boshqarish, shrift turini, o‘lchamini va uslubini o‘zgartirish bo‘yicha amaliy ko‘nikmalarni shakllantirish maqsad qilinadi. <code>setText()</code>, <code>setFont()</code>, <code>setStyleSheet()</code> metodlari yordamida matn ko‘rinishini o‘zgartirish usullari o‘rganiladi. Bu mavzu kursantlarga foydalanuvchi interfeysini estetik va qulay shaklda loyihalash imkonini beradi.	60 30 30	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e‘lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG‘ULOT O‘TISH REJASI

2-MAVZU. 3-MASHG‘ULOT.

Mavzu: PyQt5 kutubxonasi. QLineEdit vidjeti.

Mashg‘ulotning o‘quv va tarbiyaviy maqsadi: Kursantlarda Python tilidagi PyQt5 kutubxonasi va uning QLineEdit vidjeti bo‘yicha asosiy bilim hamda amaliy ko‘nikmalarni shakllantirish, foydalanuvchidan matnli ma‘lumot kiritishni tashkil etish va bu ma‘lumotlarni dasturda qayta ishlash usullarini o‘rganishdan iborat.

O‘quv guruhleri: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O‘tish joyi: 437-, 138-o‘quv sinfi.

Mashg‘ulot turi: Guruhli.

O‘quv-moddiy ta‘minot: Taqdimot va elektron ma‘lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. T.A. Xo‘jaqulov, N.T. Malikova. Sun‘iy intellekt (O‘quv qo‘llanma). - T.: “Innovatsion rivojlanish nashriyot- matbaa uyi”, 2020. -216 b.

- ## 06 QIY SAUOLLARI VA YAGETIHISORI

			III / 14
--	--	--	----------

[illegible]

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda PyQt5 kutubxonasidan

$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & i \\ -1 & i \end{pmatrix}$

vidjetlardan samarali foydalanish va interfeyslarni funksional hamda estetik jihatdan loyihalash ko'nikmalarini mustahkamlashdan iborat.

O'quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Guruhli.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproyektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Turli vidjetlardan foydalanib oddiy arifmetik dasturlar tuzish. Kursantlarda PyQt5 vidjetlari (QLineEdit, QPushButton, QLabel va h.k.) yordamida qo'shish, ayirish, ko'paytirish, bo'lish kabi amallarni bajaradigan oddiy kalkulyator ilovalari yaratish ko'nikmasi shakllantiriladi. Ma'lumot kiritish, natijani ko'rsatish va xatolarni (noto'g'ri kiritish, bo'lishda 0) qayta ishlash mexanizmlari amaliy misollar orqali o'rgatiladi. Natijada, GUI elementlari o'rtasidagi bog'lanishlar va signal-slot mantiğini qo'llash mustahkamlanadi. 2. Turli vidjetlardan foydalanib ro'yxatlarga doir dasturlar tuzish. Kursantlarda QListWidget/QTableWidget, QLineEdit, QPushButton kabi vidjetlar yordamida element qo'shish, o'chirish, qidirish, tartiblash va filtrlash funksiyalarini bajaradigan ilovalar tuzish ko'nikmalari rivojlantiriladi. Model-view yondashuvi (masalan, QStringListModel) va signal-slot mexanizmi orqali ro'yxat holatini yangilash hamda foydalanuvchi kiritmalarini validatsiya qilish amaliy qo'llaniladi. Bu orqali real ma'lumotlar bilan ishlash va GUI orqali ularni boshqarish bo'yicha tajriba mustahkamlanadi.	60 30 30	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot

3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IIY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

2-MAVZU. 5-MASHG'ULOT.

Mavzu: PyQt5 modal dialog. QMessageBox vidjeti bilan ishlash.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda PyQt5 kutubxonasida modal dialoglar va QMessageBox vidjeti bilan ishlash bo'yicha bilim va amaliy ko'nikmalarni shakllantirish, dasturda foydalanuvchiga xabar, ogohlantirish yoki so'rov oynalarini yaratish hamda ularga javob qaytarish mexanizmlarini o'rganishdan iborat.

O'quv guruhlari: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Guruhli.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. QMessageBox vidjetining vazifasi. Kursantlarda QMessageBox vidjetining dasturda foydalanuvchiga turli xabarlar, ogohlantirishlar va tasdiqlovchi	60	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot

	dialog oynalarini chiqarishdagi vazifasi haqida bilim va ko'nikmalarni shakllantirish maqsad qilinadi. Ushbu vidjet yordamida dastur foydalanuvchiga muhim holatlar yoki xatoliklar haqida ma'lumot beradi hamda u bilan interaktiv aloqa o'rnatadi. 2. QMessageBox vidjetining asosiy xususiyatlari. Kursantlarda QMessageBox vidjetining asosiy xususiyatlari — sarlavha (title), matn (text), tugmalar (buttons) va javob qaytarish (response) mexanizmlari bilan ishlash bo'yicha amaliy ko'nikmalarni shakllantirish maqsad qilinadi. Bu xususiyatlar yordamida turli turdagi xabarlar (Information, Warning, Critical, Question) hosil qilinadi va foydalanuvchi tanloviga qarab dastur mantiqi shakllantiriladi. 3. Statik funksiyalari. Kursantlarda QMessageBox vidjetining statik funksiyalari — <code>QMessageBox.information()</code>, <code>QMessageBox.warning()</code>, <code>QMessageBox.critical()</code>, va <code>QMessageBox.question()</code> funksiyalarining vazifasi va ularni dasturda qo'llash bo'yicha bilimlarni shakllantirish maqsad qilinadi. Ushbu funksiyalar yordamida xabar oynalarini tez va soddalashgan tarzda yaratish imkoniyati o'rganiladi. 4. Piktogramma va QPixmap xususiyatlari. Kursantlarda QMessageBox vidjetining piktogramma (belgilar) va pixmap (rasm) xususiyatlarini sozlash bo'yicha bilim va amaliy ko'nikmalarni shakllantirish maqsad qilinadi. Bu xususiyatlar yordamida xabar oynalariga vizual belgilar yoki rasmlar qo'shib, foydalanuvchi uchun qulay va tushunarli interfeys yaratiladi. Ushbu mavzu grafik interfeys dizaynini boyitish va interaktivlikni oshirishni o'rgatadi.	15	
		15	
		15	
		15	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IIY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

2-MAVZU. 6-MASHG'ULOT.

Mavzu: PyQt da rasmlar va menyular.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda **PyQt5** kutubxonasida rasmlar va menyular bilan ishlash bo'yicha asosiy bilim hamda amaliy ko'nikmalarni shakllantirish, grafik interfeysda tasvirlarni joylashtirish, boshqarish va menyu elementlari orqali dastur funksiyalarini qulay tarzda tashkil etishni o'rganishdan iborat.

O'quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Guruhli.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. PyQt paketi yordamida rasm joylashtirish usullari. Kursantlarda PyQt5 kutubxonasi yordamida dastur oynasiga rasm joylashtirish bo'yicha amaliy ko'nikmalarni shakllantirish maqsad qilinadi. Rasmni QLabel, QPixmap yoki QIcon vidjetlari orqali joylashtirish, o'lchamini moslashtirish va joylashuvini boshqarish usullari o'rganiladi. Bu mavzu kursantlarga grafik interfeysda tasvirlardan samarali foydalanish hamda vizual dizaynni yaxshilashni o'rgatadi. 2. Menu yaratish. Menu vidjetining xususiyatlari. Kursantlarda PyQt5 da QMenuBar va QMenu vidjetlari yordamida menyular yaratish, ularga buyruqlar (action) birlashtirish va ularning funksional xususiyatlarini boshqarish bo'yicha bilim va ko'nikmalarni shakllantirish maqsad qilinadi. Menyular dasturdagi funksiyalarni tizimli tarzda joylashtirish va foydalanuvchiga qulay interfeys yaratish uchun ishlatiladi. Ushbu mavzu orqali kursantlar menyu tuzilmasi, qisqa tugmalar (shortcut) va ikonalar bilan ishlashni o'rganadilar.	60 30 30	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

	bosqichda interfeys elementlari foydalanuvchi uchun qulay ko'rinishda joylashtiriladi va dastur oynasining ishlash mantiqi belgilanadi. 3. Matn muharriri ekrani dizaynini shakllantirish. Kursantlarda PyQt5 yordamida yaratilgan matn muharriri interfeysining umumiy dizaynini ishlab chiqish, rang sxemasi, shriftlar, ikonalar va menyularni bezatish bo'yicha amaliy ko'nikmalar shakllantiriladi. <code>setStyleSheet()</code> metodi yordamida estetik va qulay dizayn yaratiladi. Ushbu bosqich kursantlarga foydalanuvchi tajribasini oshiruvchi, funksional va chiroyli dastur interfeysini loyihalashni o'rgatadi.	20	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IIY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

2-MAVZU. 8-MASHG'ULOT.

Mavzu: Matn muharriri dasturini yozish.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda PyQt5 asosida matn muharriri (QTextEdit, QMenuBar, QToolBar va boshqalar) funksiyalarini dasturlash, faylni ochish/saqlash, tahrirlash (kopiya, kesish, qo'yish), formatlash va interfeysni boshqarish bo'yicha amaliy ko'nikmalarni mustahkamlashdan iborat.

O'quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Amaliy.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
---	----------------	--------------	-------------------------------------

1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg‘ulot maqsadini e‘lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O‘quv savollari: 1. PyQt5 da matn muharriri dizaynidagi elementlarning funksiyalari uchun dastur yozish. Kursantlar <code>QTextEdit</code>, <code>QMenuBar/QToolBar</code>, <code>QAction</code>, <code>QFileDialog</code> kabi elementlar uchun signal-slot mexanizmini sozlab, “Yangi”, “Ochish”, “Saqlash”, “Chop etish”, “Bekor qilish/Qaytarish”, “Kesish/Kopiya/Qo‘yish” funksiyalarini kodlaydilar. <code>QAction</code>larga qisqa tugmalar (shortcut) va ikonalar biriktiriladi, menyu va asboblari paneli bilan bog‘lanadi. Formatlash uchun shrift va rang dialoglari (<code>QFontDialog</code>, <code>QColorDialog</code>) qo‘llanilib, <code>setStyleSheet()</code> orqali UI bezagi yaxshilanadi. 2. Dasturni testlash. Kursantlar funksional testlar orqali fayl ochish/saqlash, matn tahriri va formatlash amallarining to‘g‘ri ishlashini tekshiradilar hamda xatolik holatlari (masalan, bo‘sh hujjatni saqlash, mavjud bo‘lmagan fayl yo‘li) uchun <code>QMessageBox</code> yordamida ogohlantirishlar chiqishini baholaydilar. Kross-platform tekshiruv (Windows/Linux/macOS) va turli kodlashlarda (UTF-8) fayl yozish/o‘qish barqarorligi sinovdan o‘tkaziladi. Shuningdek, qisqa tugmalar, menyu elementlari va toolbar action-larining mosligi, UI responzivligi va holat satri (status bar) xabarlarining to‘g‘riligiga e‘tibor qaratiladi.	60	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e‘lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG‘ULOT O‘TISH REJASI

3-MAVZU.

Pythonda tarmoq dasturlashga kirish.

1-MASHG‘ULOT.

Tarmoqda ma'lumot almashuvchi klient-server dasturini tuzish.

Mashg‘ulotning o‘quv va tarbiyaviy maqsadi: Kursantlarda Python va socket texnologiyalari asosida tarmoqda ma'lumot almashuvchi klient–server arxitekturasini loyihalash, kodlash va testlash bo‘yicha amaliy bilim hamda ko‘nikmalarni shakllantirishdan iborat.

O'quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Ma'ruza.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproyektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Socket modulining asosiy metodlari bilan tanishish. Kursantlarda Python tilidagi socket modulining vazifasi, tarmoq orqali ma'lumot almashish tamoyillari va asosiy metodlari (<code>socket()</code>, <code>bind()</code>, <code>listen()</code>, <code>accept()</code>, <code>connect()</code>, <code>send()</code>, <code>recv()</code>, <code>close()</code>) bilan ishlash ko'nikmalarini shakllantirish maqsad qilinadi. Ushbu metodlar yordamida tarmoq ulanishi o'rnatish, ma'lumot yuborish va qabul qilish jarayonlari amaliy misollar orqali o'rganiladi. 2. TCP protokoli asosida klient-server ulanishini tashkil qilish. Kursantlarda TCP (Transmission Control Protocol) asosida barqaror klient–server aloqasini o'rnatish, soketlar yordamida ikki tomonlama muloqotni tashkil qilish va ma'lumot almashishni boshqarish bo'yicha bilim va ko'nikmalar shakllantiriladi. Bu bosqichda server soketi mijoz ulanishini kutadi, klient esa serverga ulanib, ulanish orqali ma'lumot almashadi. 3. Ma'lumotlarni uzatish va qabul qilish jarayonini amalda bajarish. Kursantlarda Python soketlari yordamida tarmoq orqali ma'lumot yuborish (<code>send()</code>) va qabul qilish (<code>recv()</code>) amallarini amaliy tarzda bajarish ko'nikmalari shakllantiriladi. Ular klientdan serverga matnli yoki raqamli ma'lumot	60 15 15	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot

	yuborish, server tomonidan uni qayta ishlash va natijani javob sifatida qaytarish dasturlarini tuzadilar. 4. Klient va server dasturlarida xatoliklarni aniqlash va bartaraf etish. Kursantlarda klient-server tizimida yuzaga keladigan xatoliklarni (ulanish uzilishi, noto'g'ri IP yoki port, ma'lumot formati xatolari) aniqlash va ularni try-except bloklari yordamida bartaraf etish ko'nikmalari shakllantiriladi. Bu bosqichda ular tarmoqdagi uzilishlarga nisbatan barqaror va ishonchli dasturlar yaratishni o'rganadilar.	15	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

3-MAVZU. 2-MASHG'ULOT.

Mavzu: Socket moduli bilan ishlash.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python tilidagi ****socket**** moduli yordamida tarmoq orqali ma'lumot almashishning asosiy tamoyillari va metodlari bo'yicha bilim hamda amaliy ko'nikmalarni shakllantirish, klient-server arxitekturasida ulanish o'rnatish, ma'lumot yuborish va qabul qilish jarayonlarini o'rganishdan iborat.

O'quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Guruhli.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
---	----------------	-----------	-------------------------------

1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg‘ulot maqsadini e‘lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O‘quv savollari: 1. Socket modulining asosiy metodlari bilan tanishish. Kursantlarda Python tilidagi socket modulining vazifasi, uning yordamida tarmoqda ma‘lumot almashish jarayonlarini tashkil etish hamda asosiy metodlar (<code>socket()</code>, <code>bind()</code>, <code>listen()</code>, <code>accept()</code>, <code>connect()</code>, <code>send()</code>, <code>recv()</code>, <code>close()</code>) ishlash prinsiplari bo‘yicha bilim va ko‘nikmalarni shakllantirish maqsad qilinadi. Ushbu metodlar yordamida server va klient o‘rtasidagi tarmoq aloqasi amalga oshiriladi. 2. Server yaratishda .socket(), .bind(), .listen(), .accept() metodlaridan foydalanish. Kursantlarda server dasturini yaratishda kerakli socket obyektini hosil qilish (<code>socket()</code>), uni IP manzil va portga bog‘lash (<code>bind()</code>), kiruvchi ulanishlarni kutish (<code>listen()</code>), va ularni qabul qilish (<code>accept()</code>) metodlaridan foydalanish bo‘yicha amaliy ko‘nikmalarni shakllantirish maqsad qilinadi. Ushbu bosqichda server tomonining ulanishni boshqarish va bir nechta mijozlar bilan ishlash imkoniyatlari o‘rganiladi. 3. Klient yaratishda .connect(), .send(), .recv(), .close() metodlaridan foydalanish. Kursantlarda klient dasturida serverga ulanib, ma‘lumot yuborish (<code>send()</code>), javobni qabul qilish (<code>recv()</code>) va aloqa yakunida ulanishni yopish (<code>close()</code>) amallarini bajarish bo‘yicha amaliy ko‘nikmalar shakllantiriladi. Bu jarayon orqali kursantlar klient–server tizimining o‘zaro aloqasini amaliy tarzda tashkil etishni o‘rganadilar. 4. Klient va server o‘rtasida ma‘lumot almashishni amalga oshirish. Kursantlarda Python socket moduli yordamida klient va server o‘rtasida ikki tomonlama ma‘lumot almashish jarayonini tashkil etish ko‘nikmalari shakllantiriladi. Ular server tomonidan foydalanuvchi so‘rovini qabul qilib, javob qaytaruvchi, klient esa ma‘lumot yuboruvchi va natijani qabul qiluvchi dasturlarni tuzadilar. Ushbu mavzu real tarmoq ilovalari ishlash mantiqini chuqurroq o‘rganishga xizmat qiladi.	60	
		15	
		15	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
		15	
		15	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e‘lon qilish.	10	

3-MAVZU. 3-MASHG‘ULOT.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python tilida **TCP

O'quv guruhlari: AT-4/3-22, AT-4/4-22, KX-4/10-22.

O'tish joyi: 437-, 138-o'quv sinfi.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproyektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.

3. Sh.R. Sapayev “Python dasturlash tili asoslari”. O‘quv qo‘llanma. Toshkent: 2023y. B – 137.

4. Sh.R. Sapayev “PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish”. O‘quv qo‘llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

344

	Kursantlarda TCP protokoli asosida barqaror tarmoq ulanishini o'rnatish, klient tomonidan serverga ulanib xabar yuborish va javob olish jarayonlarini sinovdan o'tkazish ko'nikmalari shakllantiriladi. <code>connect()</code> metodi orqali ulanish o'rnatish, <code>send()</code> orqali ma'lumot yuborish va <code>recv()</code> yordamida uni qabul qilish amaliyotlari o'rganiladi. Bu mashg'ulotlar tarmoqdagi ma'lumot almashish jarayonlarini to'g'ri tashkil etish tajribasini beradi. 3. Xatoliklarni qayta ishlash va ulanish holatini boshqarish (timeout/reconnect/close). Kursantlarda TCP ulanish jarayonida yuzaga keladigan xatoliklarni (<code>ConnectionRefusedError</code>, <code>TimeoutError</code> va boshqalar) aniqlash va ularni <code>try-except</code> bloklari yordamida qayta ishlash bo'yicha bilim va ko'nikmalar shakllantiriladi. Shuningdek, <code>settimeout()</code> yordamida ulanish vaqtini boshqarish, uzilgan aloqani qayta tiklash (<code>reconnect</code>) va <code>close()</code> yordamida toza yopish jarayonlari o'rganiladi. Bu bosqich tarmoq ilovalarining barqaror va xavfsiz ishlashini ta'minlashni o'rgatadi.	20	
	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	20	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

3-MAVZU. 4-MASHG'ULOT.

Mavzu: TCP klient-server dasturini testlash.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python tilida yaratilgan TCP klient-server dasturini testlash bo'yicha amaliy bilim va ko'nikmalarni shakllantirish, tarmoq orqali ma'lumot almashish jarayonlarining to'g'ri ishlashini tekshirish, ulanish barqarorligini baholash hamda xatoliklarni aniqlash va bartaraf etish usullarini o'rganishdan iborat.

O'quv guruhлари: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Guruhli.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproyektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. TCP client-server dasturi yordamida ma'lumot almashish. Kursantlarda Python tilida yaratilgan TCP klient-server dasturi yordamida ikki tomonlama ma'lumot almashish jarayonlarini amaliy tarzda bajarish ko'nikmalari shakllantiriladi. Ular serverga ulanib, klient tomonidan yuborilgan xabarni qabul qilish, qayta ishlash va javob qaytarish mexanizmini o'rganadilar. Ushbu bosqichda <code>send()</code> va <code>recv()</code> metodlari orqali real vaqt rejimida aloqa o'rnatish amaliyoti bajariladi. 2. TCP ulanishini o'rnatish va testlash. Kursantlarda TCP ulanishini o'rnatish, uni sinovdan o'tkazish va ma'lumot almashishning to'g'ri ishlashini tekshirish bo'yicha amaliy bilim va ko'nikmalarni shakllantirish maqsad qilinadi. Ular <code>connect()</code>, <code>bind()</code>, <code>listen()</code>, <code>accept()</code> kabi metodlar yordamida klient va server o'rtasidagi aloqa jarayonini tahlil qiladilar. Testlash jarayonida ulanish muvaffaqiyati, ma'lumotlar ketma-ketligi va uzatish tezligi nazorat qilinadi. 3. TCP client va serverdagi xatoliklarni aniqlash va bartaraf etish. Kursantlarda TCP dasturlarida uchrashi mumkin bo'lgan xatoliklarni (ulanish uzilishi, port bandligi, vaqt o'tib ketishi, noto'g'ri IP manzil) aniqlash va ularni <code>try-except</code> bloklari yordamida bartaraf etish ko'nikmalari shakllantiriladi. Shuningdek, <code>timeout</code>, <code>reconnect</code>, va <code>close()</code> metodlaridan foydalanib, barqaror va xavfsiz tarmoq aloqasini saqlab qolish usullari o'rganiladi. Bu bosqich kursantlarga dasturiy xatolarga bardoshli TCP tizimlarini yaratish tajribasini beradi.	60 20 20 20	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN’IY INTELLEKT ASOSLARI” FANIDAN MASHG‘ULOT O‘TISH REJASI

3-MAVZU. 5-MASHG‘ULOT.

Mavzu: PyQt5 paketidan foydalanib zamonaviy chat dasturini tuzish.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda PyQt5 asosida zamonaviy chat dasturini loyihalash va yaratish bo'yicha amaliy ko'nikmalarni shakllantirish, GUI (QMainWindow, QTextEdit/QListView, QLineEdit, QPushButton) elementlarini signal-slot mexanizmi bilan bog'lash, tarmoq qismi uchun soketlar (TCP) va mavzular (threading/QtConcurrent) orqali xabarlarini real vaqt rejimida uzatish-qabul qilish, foydalanuvchi sessiyasini boshqarish, xabar holatini ko'rsatish (online/offline, delivered/read) hamda minimal xatolikni ta'minlovchi arxitekturaning (MVC/MVVM) qo'llashni o'rganishdan iborat.

O'quv guruhlari: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vagt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Guruhli.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproyektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo‘jaqulov, N.T. Malikova. Sun‘iy intellekt (O‘quv qo‘llanma). - T.: “Innovatsion rivojlanish nashriyot- matbaa uyi”, 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. “Python dasturlash tili” Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev “Python dasturlash tili asoslari”. O‘quv qo‘llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev “PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish”. O‘quv qo‘llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

[illegible]

	GUI ichida socket yoki QTcpSocket orqali serverga ulanish (connect()), xabar yuborish (send()/write()) va qabul qilish (recv()/readyRead) jarayonlari implementatsiya qilinadi. Ulanish holati, uzilish va qayta ulanish mexanizmlari (timeout/reconnect) qo'shib, yuborilgan va qabul qilingan xabarlar real vaqtda ro'yxatda aks ettiriladi.	20	
	3. Signal-slot mexanizmi va xatoliklarni GUI'da ko'rsatish. Tarmoq hodisalari (ulandi/uzildi/yangi xabar) signal-slot mexanizmi orqali GUI elementlariga bog'lanadi, fon oqimlari (QThread) bilan UI "muzlab qolmasligi" ta'minlanadi. Xatoliklar QMessageBox yoki status bar orqali foydalanuvchiga aniq ko'rsatiladi, log oynasi/konsolega diagnostika ma'lumotlari chiqariladi.	20	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

3-MAVZU. 6-MASHG'ULOT.

Mavzu: Python GUI paketidan foydalanib chat dasturini tuzishni yakunlash

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Python GUI (PyQt5/PySide) asosida chat dasturini yakuniy holga keltirish—xabar almashuvini barqaror ishlatish, UI'ni yakuniy bezash, xatoliklarni bartaraf etish va dasturni testlash/joylashtirish (packaging) bo'yicha amaliy ko'nikmalarni mustahkamlashdan iborat.

O'quv guruhleri: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Guruhli.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
---	----------------	-----------	-------------------------------

1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. TCP client–server dasturini GUI ko'rinishda ishlab chiqish. Kursantlar PyQt5/PySide asosida klient oynasi (xabarlar ro'yxati, kiritish maydoni, yuborish tugmasi) va kerak bo'lsa server monitoring panelini loyihalab kodlaydilar. Layout'lar, QAction/QMenu va <code>setStyleSheet()</code> orqali ergonomik va zamonaviy interfeys yaratiladi. Signal–slot mexanizmi yordamida UI elementlari tarmoq qismi bilan bog'lanib, arxitektura MVC/MVVM prinsiplari asosida tartibga keltiriladi. 2. Xabar almashish oqimini (connect/send/receive) tekshirish va sinovdan o'tkazish. QTcpSocket yoki <code>socket</code> moduli bilan ulanishni o'rnatish, xabar yuborish va qabul qilish oqimi implementatsiya qilinadi. Funktsional testlar (yakka va juft qurilma, bir nechta mijoz) orqali kechikish, ketma-ketlik va paketlash masalalari tekshiriladi. Test ssenariylari: server yo'qligida ulanish, uzun xabarlar, UTF-8 kodlash va ketma-ket tez yuborish holatlari. 3. Xatoliklarni qayta ishlash, holatlarni (status/reconnect) GUI'da boshqarish. <code>try-except</code>, <code>errorOccurred</code> signallari va <code>setSocketOption/settimeout()</code> kabi vositalar bilan ulanish xatolari aniqlanadi va foydalanuvchiga <code>QMessageBox</code> yoki <code>status bar</code> orqali aniq ko'rsatiladi. Ulanish holati (online/offline, connecting...) indikatorlari va "Reconnect" mexanizmi qo'shiladi. Log/diagnostika oynasi orqali texnik tafsilotlar yozib borilib, ilova barqarorligi va foydalanuvchi tajribasi oshiriladi.	60 20 20 20	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

3-MAVZU. 7-MASHG'ULOT.

Mavzu: Python ilovasini maxsus paket yordamida mustaqil yuklanuvchi fayl

	usullari amaliy misollar bilan ko'rsatiladi. Bu orqali ular har xil loyihalar uchun mos yig'ish strategiyasini tanlay oladilar. 3. Dasturiy ta'minotni turli operatsion tizimlarda sinovdan o'tkazish. Kursantlar Windows, Linux va macOS muhitlarida yaratilgan build'larni tekshirish, yo'qolgan kutubxonalar, ruxsatlar va kodlash muammolarini aniqlash ko'nikmalarini rivojlantiradilar. Funksional va regressiya testlari, log yozish hamda xatoliklarni diagnostika qilish orqali barqarorlik baholanadi. Zaruratga ko'ra "virtual environment" va VM/Containerlarda ko'p-platformali sinov o'tkaziladi. 4. Yaratilgan yuklanuvchi faylni foydalanuvchi uchun qulay shaklda taqdim etish. Kursantlar o'rnatkich (installer) yoki portativ paket tayyorlash, versiyalash, minimal talablar va qo'llanmani (README/Help) birga taqdim etish tamoyillarini o'zlashtiradilar. Avtomatik yangilash, imzolash (code signing) va distributsiya kanallari (zip, installer, paket menejerlari) bo'yicha eng yaxshi amaliyotlar ko'rib chiqiladi. Shu orqali mahsulotni foydalanuvchiga tushunarli, ishonchli va oson o'rnatiladigan ko'rinishda yetkazish maqsadga muvofiq bo'ladi.	15	
	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	15	
3.		10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

4-MAVZU.

Python yordamida mashinali o'qitish asoslari

1-MASHG'ULOT.

Mashinali o'qitishga kirish.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda mashinali o'qitish (Machine Learning) tushunchasi, uning sun'iy intellektidagi o'rni va asosiy ishlash tamoyillari bo'yicha bilim va ko'nikmalarni shakllantirish, ma'lumotlardan o'rganish, tahlil qilish va bashorat qilish jarayonlarini tushuntirish hamda oddiy ML modellarini amaliy misollar asosida yaratishdan iborat.

O'quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Ma'ruza.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.

2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Mashinali o'qitish haqida umumiy tushuncha. Kursantlarda mashinali o'qitish (Machine Learning) tushunchasi, uning maqsadi va qo'llanilish sohalari haqida umumiy bilimlarni shakllantirish maqsad qilinadi. Mashinali o'qitish kompyuter tizimlariga ma'lumotlardan mustaqil tarzda o'rganish va natijalardan xulosa chiqarish imkonini beradi. Bu texnologiya tasniflash, bashorat qilish, tahlil va qaror qabul qilish jarayonlarida keng qo'llaniladi. 2. Sun'iy intellekt va ML farqi. Kursantlarda sun'iy intellekt (AI) va mashinali o'qitish (ML) o'rtasidagi asosiy farqlarni tushunish ko'nikmalari shakllantiriladi. AI kengroq tushuncha bo'lib, mashinalarni insondek fikrlashga o'rgatishni o'z ichiga oladi, ML esa AI'ning tarkibiy qismi bo'lib, mashinalarni ma'lumotlar asosida o'rganishga yo'naltiradi. Bu mavzu kursantlarga ushbu ikki yo'nalishning o'zaro bog'liqligi va farqlarini tahlil qilish imkonini beradi. 3. Jupyter Notebook bilan ishlash. Kursantlarda Jupyter Notebook muhitida ishlash, kod yozish, natijalarni vizual ko'rinishda tahlil qilish va hujjatlashtirish bo'yicha amaliy ko'nikmalarni shakllantirish maqsad qilinadi. Ular notebook hujayralarida Python kodlarini bajarish, kutubxonalarni import qilish va grafik natijalarni ko'rsatish usullarini o'rganadilar. Ushbu vosita mashinali o'qitish jarayonlarida tajriba va modellashtirish uchun qulay muhit yaratadi.	60 20 20 20	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish.	10	

5-MAVZU.

1-MASHG'ULOT.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda ****NumPy**** va ****Pandas**** kutubxonalari yordamida ma'lumotlarni qayta ishlash, tahlil qilish va ularni dasturiy shaklda boshqarish bo'yicha amaliy bilim va ko'nikmalarni shakllantirish, massivlar va jadvallar ustida arifmetik hamda statistik amallarni bajarish, shuningdek ma'lumotlarni tozalash va transformatsiya qilish usullarini o'rganishdan iborat.

Vagt: 80 min.

Mashg‘ulot turi: Ma’ruza.

O'quv texnik vositalar: Videoproyektor, 65 dyumli monoblok, kompyuter.

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.

2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. “Python dasturlash tili” Darslik. Toshkent: 2024y. B - 316.

3. Sh.R. Sapayev “Python dasturlash tili asoslari”. O‘quv qo‘llanma. Toshkent: 2023y. B – 137.

4. Sh.R. Sapayev “PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish”. O‘quv qo‘llanma. Toshkent: 2024y. B - 150

[illegible]

	turlarini boshqarish o'rganiladi. Ushbu mavzu katta hajmdagi sonli ma'lumotlar bilan samarali ishlashni ta'minlaydi. 2. Pandas kutubxonasi asoslari (ma'lumotlarni qayta ishlash). Kursantlarda Pandas kutubxonasi yordamida jadval ko'rinishidagi ma'lumotlarni tahlil qilish, tozalash va qayta ishlash bo'yicha amaliy ko'nikmalar shakllantiriladi. <i>Series</i> va <i>DataFrame</i> obyektlari bilan ishlash, <code>read_csv()</code>, <code>head()</code>, <code>info()</code>, <code>groupby()</code>, <code>merge()</code> kabi metodlardan foydalanish usullari o'rganiladi. Ushbu mavzu kursantlarga real ma'lumotlar bilan ishlash va tahlil jarayonlarini dasturiy tarzda amalga oshirish imkonini beradi.	30	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

5-MAVZU. 2-MASHG'ULOT.

Mavzu: NumPy va Pandas bilan ma'lumotlarni qayta ishlash.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda NumPy va Pandas kutubxonalari yordamida ma'lumotlarni qayta ishlash, tahlil qilish va ularni dasturiy shaklda boshqarish bo'yicha amaliy bilim va ko'nikmalarni shakllantirish, massivlar va jadvallar ustida arifmetik, statistik hamda tahliliy amallarni bajarish, shuningdek ma'lumotlarni tozalash, saralash va transformatsiya qilish usullarini o'rganishdan iborat.

O'quv guruhleri: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Amaliy.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
---	----------------	--------------	-------------------------------------

1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg‘ulot maqsadini e‘lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O‘quv savollari: 1. NumPy kutubxonasida amallar bajarish. Kursantlarda NumPy kutubxonasi yordamida massivlar ustida arifmetik, mantiqiy va statistik amallarni bajarish bo‘yicha amaliy ko‘nikmalarni shakllantirish maqsad qilinadi. Ular <code>add()</code>, <code>subtract()</code>, <code>dot()</code>, <code>mean()</code>, <code>sum()</code>, <code>max()</code>, <code>min()</code> kabi funksiyalar yordamida ma‘lumotlarni qayta ishlashni o‘rganadilar. Ushbu mavzu katta hajmdagi sonli ma‘lumotlar bilan tez va samarali ishlash imkonini beradi. 2. Pandas yordamida ma‘lumotlarni qayta ishlash va tahlil qilish. Kursantlarda Pandas kutubxonasi yordamida jadval ko‘rinishidagi ma‘lumotlarni tahlil qilish, ularni tozalash, guruhlash, saralash va filtrlash bo‘yicha amaliy bilimlar shakllantiriladi. <code>read_csv()</code>, <code>describe()</code>, <code>groupby()</code>, <code>sort_values()</code>, <code>fillna()</code> kabi metodlardan foydalanish o‘rganiladi. Bu mavzu kursantlarga real ma‘lumotlar ustida tahliliy amallarni bajarish va ularni dasturiy tarzda boshqarish imkoniyatini beradi.	60 30 30	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e‘lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG‘ULOT O‘TISH REJASI

5-MAVZU. 3-MASHG‘ULOT.

Mavzu: NumPy va Pandas bilan ma‘lumotlarni qayta ishlash

Mashg‘ulotning o‘quv va tarbiyaviy maqsadi: Kursantlarda **NumPy** va **Pandas** kutubxonalari yordamida ma‘lumotlarni qayta ishlash, tahlil qilish va boshqarish bo‘yicha amaliy bilim hamda ko‘nikmalarni shakllantirish, massivlar va jadvallar ustida arifmetik, statistik hamda tahliliy amallarni bajarish, ma‘lumotlarni tozalash, guruhlash va filtrlash jarayonlarini samarali tashkil etishni o‘rganishdan iborat.

O‘quv guruhleri: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O‘tish joyi: 437-, 138-o‘quv sinfi.

Mashg‘ulot turi: Guruhli.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

- ## O'QUV SAVOLLARI VA VAQT HISOBI

356

	kursantlarga katta hajmdagi ma'lumotlar bilan tahliliy ishlashni samarali amalga oshirishni o'rgatadi.		
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IIY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

6-MAVZU.

Ma'lumotlar bilan ishlash va vizualizatsiya.

1-MASHG'ULOT.

Matplotlib va Seaborn kutubxonalari bilan ishlash.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Matplotlib va Seaborn kutubxonalari yordamida ma'lumotlarni vizual tahlil qilish, grafik va diagrammalar orqali natijalarni ifodalash bo'yicha amaliy bilim va ko'nikmalarni shakllantirish, chizmalarni sozlash, dizaynini o'zgartirish hamda statistik tahlillarni grafik shaklda taqdim etish usullarini o'rganishdan iborat.

O'quv guruhleri: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Ma'ruza.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari:	60	Videoproektor yoki

	1. Matplotlib va Seaborn kutubxonalari bilan tanishish. Kursantlarda Matplotlib va Seaborn kutubxonalarining imkoniyatlari, ularning o'zaro farqlari hamda ma'lumotlarni vizual ko'rinishda tahlil qilishdagi o'rni haqida bilim va ko'nikmalarni shakllantirish maqsad qilinadi. Ular <code>plot()</code>, <code>bar()</code>, <code>hist()</code>, <code>scatter()</code> kabi Matplotlib funksiyalarini va Seaborn kutubxonasidagi <code>sns.countplot()</code>, <code>sns.heatmap()</code> kabi funksiyalarni o'rganadilar. Bu mavzu ma'lumotlarni tahlil qilishda grafik ifodalarning ahamiyatini tushunishga yordam beradi. 2. Ma'lumotlarni vizualizatsiya qilish texnikalari. Kursantlarda Matplotlib va Seaborn yordamida turli turdagi grafiklar — chiziqli, ustunli, nuqtali, taqsimot va issiqlik xaritalarini yaratish va ularni tahlil qilish bo'yicha amaliy ko'nikmalar shakllantiriladi. Ular grafiklarni bezash, ranglar, sarlavhalar, o'qlar va afsonalarni sozlash orqali vizualizatsiya sifatini yaxshilashni o'rganadilar. Ushbu mavzu ma'lumotlarni tahlil natijalarini aniq va tushunarli tarzda taqdim etish imkoniyatini beradi.	30	65 dyumli monoblok, elektron taqdimot
	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	30	
3.		10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

6-MAVZU. 2-MASHG'ULOT.

Mavzu: Matplotlib va Seaborn kutubxonalari yordamida ma'lumotlarni vizualizatsiya qilish.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Matplotlib va Seaborn kutubxonalari yordamida ma'lumotlarni vizualizatsiya qilish, ya'ni ularni turli grafik va diagrammalar orqali tahlil qilish va taqdim etish bo'yicha amaliy bilim hamda ko'nikmalarni shakllantirish maqsad qilinadi. Ular chiziqli, ustunli, doira, nuqtali va issiqlik xaritalari kabi grafiklarni yaratish, dizayn elementlarini (rang, o'lcham, sarlavha, afsona) sozlash, hamda ma'lumot tahlil natijalarini vizual tarzda ifodalashni o'rganadilar.

O'quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Amaliy.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.

2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.

3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.

4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Grafiklar, gistogrammalar va scatter plot chizishni o'rganish. Kursantlarda Matplotlib kutubxonasi yordamida turli xil grafiklar — chiziqli grafiklar (<code>plot()</code>), ustunli diagrammalar (<code>bar()</code>), gistogrammalar (<code>hist()</code>) va nuqtali tasvirlar (<code>scatter()</code>) chizish ko'nikmalarini shakllantirish maqsad qilinadi. Ular ma'lumotlarning o'zgarishini, taqsimotini va o'zaro bog'liqligini grafik tarzda tasvirlashni o'rganadilar. Ushbu amaliyot ma'lumotlarni tahlil qilishni vizual tarzda tushunarli ifodalash imkonini beradi. 2. Seaborn yordamida statistik vizualizatsiyalar yaratish. Kursantlarda Seaborn kutubxonasi yordamida statistik tahlil uchun mo'ljallangan grafiklar (<code>distplot()</code>, <code>boxplot()</code>, <code>heatmap()</code>, <code>pairplot()</code>) yaratish bo'yicha amaliy ko'nikmalar shakllantiriladi. Ular ma'lumotlar orasidagi bog'liqlikni, o'rtacha qiymatlarni va taqsimotlarni vizual tahlil qilish usullarini o'rganadilar. Bu mavzu kursantlarga ma'lumot tahlilini yanada chuqurroq va estetik tarzda ko'rsatish imkonini beradi. 3. Grafiklarni sozlash (rang, uslub, o'q nomlari, sarlavha) va ularni saqlash. Kursantlarda yaratilgan grafiklarning dizaynini yaxshilash, ularga rang berish, uslubini o'zgartirish, o'q nomlari va sarlavhalarni kiritish bo'yicha amaliy bilim va ko'nikmalarni shakllantirish maqsad qilinadi. Ular <code>xlabel()</code>, <code>ylabel()</code>, <code>title()</code>, <code>legend()</code>, <code>style.use()</code> kabi funksiyalarni qo'llashni o'rganadilar hamda <code>savefig()</code> yordamida grafiklarni faylga saqlashni amalda bajaradilar. Bu mavzu tahlil natijalarini professional va ta'sirchan shaklda taqdim etishni o'rgatadi.	60 20 20	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish.	10	

6-MAVZU. 3-MASHG‘ULOT.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Matplotlib va Seaborn kutubxonalarini bilan ishlash, ya'ni ma'lumotlarni turli grafik turlari orqali vizual tarzda ifodalash va tahlil qilish bo'yicha amaliy bilim va ko'nikmalarni shakllantirish maqsad qilinadi. Ular grafik, diagramma, gistogramma va issiqlik xaritalarini yaratish, ularning dizaynini sozlash, shuningdek vizual tahlil orqali ma'lumotlardan xulosalar chiqarishni o'rganadilar.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

[illegible]

	2. Matplotlib yordamida chiziqli grafik, gistogramma va scatter plotlar yaratish. Matplotlib kutubxonasi yordamida chiziqli grafiklar (<code>plot()</code>), gistogrammalar (<code>hist()</code>) va nuqtali grafiklar (<code>scatter()</code>) yaratish usullari o'rganiladi. Grafiklarda rang, chiziq turi, sarlavha va o'qlar nomlarini sozlash orqali ularni aniq va estetik ko'rinishga keltirish ko'nikmasi hosil qilinadi. Ushbu mavzu ma'lumotlarni aniq va tushunarli shaklda ifodalash imkonini beradi. 3. Seaborn yordamida statistik ma'lumotlarni ko'rsatish va ularni solishtirish. Seaborn kutubxonasi yordamida statistik ma'lumotlarni vizual tarzda tahlil qilish va solishtirish usullari o'rganiladi. <code>boxplot()</code>, <code>violinplot()</code>, <code>heatmap()</code>, <code>pairplot()</code> kabi funksiyalar yordamida ma'lumotlar taqsimoti, o'rtacha qiymatlar va bog'liqliklar ko'rsatiladi. Bu usullar statistik tahlil natijalarini qulay va tushunarli grafiklarda ifodalash imkonini beradi.	20	
	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	20	
3.		10	

“SUN'IIY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

7-MAVZU.

Mashinali o'qitish asoslari va algoritmlar.

1-MASHG'ULOT.

Mashinali o'qitish algoritmlari haqida umumiy tushuncha.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda mashinali o'qitish algoritmlari, ularning turlari, ishlash prinsiplari hamda ma'lumotlardan o'rganish va bashorat qilish jarayonlaridagi o'rni haqida asosiy bilim va amaliy ko'nikmalarni shakllantirishdan iborat.

O'quv guruhлари: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Ma'ruza.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Mashinali o'qitish algoritmlari haqida umumiy tushuncha. Mashinali o'qitish algoritmlari kompyuter tizimlariga mavjud ma'lumotlar asosida mustaqil o'rganish va xulosa chiqarish imkonini beradi. Bu algoritmlar ma'lumotlar orasidagi bog'liqlikni aniqlash, bashorat qilish va tasniflash kabi vazifalarni bajaradi. Ular sun'iy intellekt tizimlarining asosiy qismini tashkil etib, murakkab muammolarni avtomatik hal etishda keng qo'llaniladi. 2. O'qitiluvchi (Supervised) o'rganish algoritmlari (Linear Regression, Logistic Regression). O'qitiluvchi o'rganish algoritmlari kiruvchi ma'lumotlar va ularning natijalari (yorliqlari) asosida modelni o'qitishga mo'ljallangan. Linear Regression miqdoriy qiymatlarni bashorat qilishda ishlatilsa, Logistic Regression ikki yoki undan ortiq toifali natijalarni tasniflash uchun qo'llaniladi. Bu usullar amaliy tahlil, moliya, tibbiyot va muhandislik sohalarida keng qo'llaniladi. 3. O'qitilmaydigan (Unsupervised) o'rganish algoritmlari (K-means Clustering). O'qitilmaydigan o'rganish algoritmlari ma'lumotlardagi yashirin tuzilmani aniqlash uchun foydalaniladi, bunda natijalar (yorliqlar) oldindan berilmaydi. K-means Clustering algoritmi o'xshash belgilar asosida ma'lumotlarni guruhlariga ajratadi. Ushbu yondashuv marketing, biotexnologiya va ma'lumot tahlili sohalarida tahliliy qarorlar qabul qilishda muhim ahamiyat kasb etadi.	60 20 20 20	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

7-MAVZU. 2-MASHG'ULOT.

Mavzu: Mashinali o'qitish algoritmlarini amaliyotga tatbiq qilish.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda mashinali o'qitish algoritmlarini amaliyotga tatbiq qilish, ya'ni real ma'lumotlar to'plamlarida turli ML modellari (Linear Regression, Logistic Regression, K-means va boshqalar) yordamida tahlil, bashorat va tasniflash jarayonlarini amalga oshirish, natijalarni vizual ko'rinishda tahlil qilish hamda model aniqligini baholash bo'yicha amaliy bilim va ko'nikmalarni shakllantirishdan iborat.

O'quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Amaliy.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Linear va Logistic Regression algoritmlarini tatbiq qilish. Linear Regression algoritmi yordamida uzluksiz qiymatlarni (masalan, narx, harorat, talab hajmi) bashorat qilish jarayoni o'rganiladi. Logistic Regression esa ma'lumotlarni ikki yoki bir nechta toifaga ajratish (masalan, kasallik mavjud/yoki yo'q, foydalanuvchi xarid qiladi/yoki qilmaydi) uchun qo'llaniladi. Ushbu algoritmlar real datasetlarda qo'llanilib, modelning aniqligi va natijalarning vizual tahlili amalga oshiriladi. 2. K-means Clustering algoritmini tatbiq qilish. K-means Clustering algoritmi yordamida o'xshash xususiyatlarga ega ma'lumotlarni guruhlariga (klasterlarga) ajratish usuli amaliyotda o'rganiladi. Bu yondashuv ma'lumotlardagi yashirin tuzilmani aniqlash va tahlil qilish uchun ishlatiladi. Natijalar grafik shaklda ifodalanib, klaster	60 30 30	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot

	markazlari (centroidlar) aniqlanadi hamda algoritmnining samaradorligi baholanadi.		
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

7-MAVZU. 3-MASHG'ULOT.

Mavzu: Mashinali o'qitish algoritmlarini real datasetlarda qo'llash.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda mashinali o'qitish algoritmlarini real datasetlarda qo'llash, ya'ni haqiqiy ma'lumotlar to'plamlarini tahlil qilish, ularni tayyorlash (tozalash, normallashtirish), mashinali o'qitish modellarini (Linear Regression, Logistic Regression, Decision Tree, K-means va boshqalar) qo'llab natijalarni tahlil qilish bo'yicha amaliy bilim va ko'nikmalarni shakllantirishdan iborat.

O'quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Guruhli.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproyektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Real datasetlarda algoritmlarni qo'llash va natijalarni taqdim etish.	60	Videoproektor yoki 65 dyumli monoblok,

	Mashinali o'qitish algoritmlari real hayotdagi ma'lumotlar to'plamlarida qo'llanilib, ularning natijalari tahlil qilinadi va vizual tarzda taqdim etiladi. Amaliyotda Linear Regression, Logistic Regression, Decision Tree kabi modellardan foydalanish orqali bashorat va tasniflash vazifalari bajariladi. Olingan natijalar aniqlik darajasi bo'yicha baholanib, grafik yoki jadval ko'rinishida ifodalanadi. 2. Ma'lumotlarni tayyorlash va mashinali o'qitish modellariga uzatish. Modelni to'g'ri o'qitish uchun ma'lumotlarni tozalash, yo'qolgan qiymatlarni to'ldirish, normalizatsiya qilish va kategorik qiymatlarni kodlash jarayonlari bajariladi. Tayyorlangan dataset mashinali o'qitish algoritmlariga uzatilib, modelni o'qitish va sinovdan o'tkazish bosqichlari amalga oshiriladi. Bu jarayon modelning sifatli ishlashi uchun muhim poydevor hisoblanadi. 3. Modellarning aniqligini baholash va taqqoslash. Turli mashinali o'qitish modellarining samaradorligini aniqlash uchun ularning aniqlik (accuracy), xatolik (error rate), F1-mezon va ROC AUC kabi ko'rsatkichlari hisoblanadi. Modellar natijalarini solishtirish orqali eng maqbul yondashuv aniqlanadi. Bu bosqich tahlil natijalarini optimallashtirish va real vazifalar uchun eng mos modelni tanlash imkonini beradi.	20	elektron taqdimot
		20	
		20	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

8-MAVZU.

Mashinali o'qitishda Klassifikatsiya va regressiya masalalari.

1-MASHG'ULOT.

Mashinali o'qitishda Klassifikatsiya va regressiya tushunchalari.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda mashinali o'qitishda klassifikatsiya va regressiya tushunchalari, ularning farqi, ishlash prinsipi va qo'llanilish sohalari bo'yicha nazariy hamda amaliy bilim va ko'nikmalarni shakllantirishdan iborat. Klassifikatsiya usullari yordamida ma'lumotlar turkumlarga ajratilsa, regressiya esa uzluksiz qiymatlarni bashorat qilish uchun qo'llaniladi. Shu orqali kursantlar turli vazifalar uchun mos model turini tanlashni o'rganadilar.

O'quv guruhlari: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Ma'ruza.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.

2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.

3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.

4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Klassifikatsiya tushunchasi va asosiy algoritmlar (Decision Trees, Random Forest). Klassifikatsiya — bu mashinali o'qitishning asosiy yo'nalishlaridan biri bo'lib, ma'lumotlarni belgilangan toifalarga ajratish jarayonidir. Decision Trees algoritmi qaror daraxtlari orqali ma'lumotlarni tahlil qiladi, Random Forest esa bir nechta daraxtlarni birlashtirib, aniqlikni oshiradi va xatolikni kamaytiradi. Ushbu algoritmlar tibbiyot, moliya, xavfsizlik va boshqa ko'plab sohalarda muvaffaqiyatli qo'llaniladi. 2. Regressiya tushunchasi va algoritmlari (Linear, Polynomial Regression). Regressiya — bu o'zgaruvchilar orasidagi bog'liqlikni aniqlash va uzluksiz qiymatlarni bashorat qilishga mo'ljallangan mashinali o'qitish usulidir. Linear Regression chiziqli bog'liqlikni aniqlasa, Polynomial Regression nolinear (egri chiziqli) munosabatlarni modellashtiradi. Ushbu algoritmlar iqtisodiy tahlil, narx prognozi, energiya sarfi va ilmiy tadqiqotlarda keng qo'llaniladi.	60 30 30	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI**8-MAVZU. 2-MASHG'ULOT.**

Mavzu: Mashinali o'qitishda Klassifikatsiya va regressiyani qo'llanilishi.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda mashinali o'qitishda klassifikatsiya va regressiya algoritmlarining qo'llanilishi, ularning amaliy ahamiyati, real ma'lumotlar to'plamlarida tasniflash va bashoratlash vazifalarini bajarishdagi roli bo'yicha bilim va amaliy ko'nikmalarni shakllantirishdan iborat. Ular klassifikatsiyani (masalan, spam aniqlash, kasallikni tashxislash) va regressiyani (masalan, uy narxini yoki talab hajmini bashorat qilish) real muammolarni yechishda qo'llashni o'rganadilar.

O'quv guruhleri: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Amaliy.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Decision Trees va Random Forest algoritmlarini tatbiq qilish. Decision Trees algoritmi yordamida ma'lumotlarni turli toifalarga ajratish va qarorlar asosida natijalarni bashorat qilish amaliyotda o'rganiladi. Random Forest esa bir nechta qaror daraxtlarini birlashtirish orqali modelning aniqligini oshirish va xatoliklarni kamaytirish imkonini beradi. Ushbu algoritmlar ma'lumotlarni tasniflash, kredit reytinglarini baholash, kasallikni aniqlash kabi real vazifalarda qo'llaniladi. 2. Turli regressiya algoritmlarini tatbiq qilish. Linear Regression va Polynomial Regression algoritmlaridan foydalanib, o'zgaruvchilar orasidagi bog'liqlikni aniqlash va uzluksiz qiymatlarni bashorat qilish jarayonlari bajariladi.	60 30	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot

	Amaliyotda ma'lumotlarni tahlil qilish, modelni o'qitish va natijalarni vizual ko'rinishda baholash amalga oshiriladi. Ushbu yondashuv iqtisodiy, ijtimoiy va texnik jarayonlarni oldindan prognozlashda keng qo'llaniladi.	30	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

8-MAVZU. 3-MASHG'ULOT.

Mavzu: Mashinali o'qitish modellarini baholash va natijalarni taqqoslash

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda mashinali o'qitish modellarini baholash va natijalarni taqqoslash, ya'ni turli modellar samaradorligini aniqlash, ularning aniqlik (accuracy), aniqlik darajasi (precision), eslash ko'rsatkichi (recall) va F1-mezon kabi statistik ko'rsatkichlar orqali solishtirish bo'yicha bilim va amaliy ko'nikmalarni shakllantirishdan iborat. Shuningdek, kursantlar modelning ortiqcha moslashuv (overfitting) va yetarli o'rganmaslik (underfitting) holatlarini aniqlash hamda ularni bartaraf etish usullarini o'rganadilar.

O'quv guruhlari: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Guruhli.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	

2.	II. ASOSIY QISM O'quv savollari: 1. Amaliy natijalarni solishtirish va guruh taqdimotlari. Mashinali o'qitish modellarini qo'llash orqali olingan natijalar tahlil qilinadi va guruhlar kesimida taqqoslanadi. Har bir guruh o'z modeli natijalarini taqdim etib, tanlangan algoritmnining afzalliklari va kamchiliklarini tahlil qiladi. Ushbu jarayon natijalarni baholash, muloqot ko'nikmalarini rivojlantirish va jamoaviy ishlash tajribasini mustahkamlaydi. 2. Modellarning aniqlik, xatolik va samaradorlik ko'rsatkichlarini hisoblash. Model sifatini baholash uchun aniqlik (accuracy), xatolik (error rate), aniqlik darajasi (precision), eslash (recall) va F1-score kabi ko'rsatkichlar hisoblanadi. Ushbu metrikalar yordamida modelning umumiy ishlashi tahlil qilinadi va natijalarga ko'ra modelning kuchli hamda zaif tomonlari aniqlanadi. Bu yondashuv ishonchli va samarali model tanlash imkonini beradi. 3. Turli algoritmlarni bir datasetda sinovdan o'tkazib taqqoslash. Bir xil datasetda bir nechta mashinali o'qitish algoritmlari (masalan, Logistic Regression, Decision Tree, Random Forest, KNN) sinovdan o'tkazilib, ularning natijalari taqqoslanadi. Har bir algoritmnining ishlash tezligi, aniqligi va moslashuv darajasi tahlil qilinadi. Natijalar asosida eng samarali model aniqlanadi va uni amaliy qo'llash bo'yicha xulosalar chiqariladi.	60 20 20 20	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

8-MAVZU. 4-MASHG'ULOT.

Mavzu: Sun'iy neyron tarmoqlar haqida umumiy tushuncha.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda sun'iy neyron tarmoqlar haqida umumiy tushuncha, ularning tuzilishi, ishlash prinsipi va mashinali o'qitishdagi o'rni bo'yicha bilim va amaliy ko'nikmalarni shakllantirishdan iborat. Sun'iy neyron tarmoqlar inson miya neyronlarining ish tamoyiliga asoslanib, ma'lumotlarni qayta ishlash, tahlil qilish va bashorat qilish uchun mo'ljallangan. Ular tasvirni tanish, nutqni aniqlash va qaror qabul qilish kabi murakkab vazifalarni bajarishda keng qo'llaniladi.

O'quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Amaliy.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Sun'iy neyron tarmoqlari asoslari. Sun'iy neyron tarmoqlar (Artificial Neural Networks — ANN) inson miya neyronlarining ishlash tamoyiliga asoslangan hisoblash tizimlaridir. Ular kiruvchi ma'lumotlarni qayta ishlaydi, ular orasidagi bog'liqlikni o'rganadi va natija chiqaradi. Neyron tarmoqlar mashinali o'qitish va sun'iy intellekt tizimlarining asosi bo'lib, tasvirni aniqlash, tovushni tanish va matnni tahlil qilish kabi vazifalarda keng qo'llaniladi. 2. Perceptron modeli va tarmoqlarni o'qitish mexanizmi. Perceptron — bu sun'iy neyron tarmog'ining eng oddiy modeli bo'lib, u kiruvchi signallarni qabul qilib, og'irliklar (weights) orqali natija hosil qiladi. Modelni o'qitish jarayoni og'irliklarni moslashtirish va xatolikni kamaytirish asosida amalga oshiriladi. Bu mexanizm neyron tarmoqlarning ma'lumotlardan o'rganish va to'g'ri javob berish qobiliyatini shakllantiradi. 3. Kirish darajasidagi neyron tarmoqlari (MLP). Kirish darajasidagi ko'p qatlamli perseptron (MLP — Multilayer Perceptron) bir nechta yashirin qatlamlardan iborat bo'lib, murakkab nolinear bog'lanishlarni modellashtirish imkonini beradi. Har bir qatlam kiruvchi signallarni qayta ishlaydi va natijani keyingi qatlamga uzatadi. MLP modeli tasniflash, regressiya va bashorat qilish kabi vazifalarda eng ko'p qo'llaniladigan tarmoq tuzilmasidir.	60 20 20	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish.	10	

	–Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.		
--	--	--	--

“SUN'IY INTELLEKT ASOSLARI” FANIDAN MASHG'ULOT O'TISH REJASI

8-MAVZU. 5-MASHG'ULOT.

Mavzu: Scikit-learn paketidan foydalanish.

Mashg'ulotning o'quv va tarbiyaviy maqsadi: Kursantlarda Scikit-learn paketidan foydalanish, ya'ni mashinali o'qitish algoritmlarini amaliyotda qo'llash, ma'lumotlarni tayyorlash, modelni o'qitish, sinovdan o'tkazish va natijalarni tahlil qilish bo'yicha bilim hamda amaliy ko'nikmalarni shakllantirishdan iborat. Shu orqali kursantlar regressiya, klassifikatsiya va klasterlash kabi vazifalar uchun tayyor algoritmlarni qo'llashni o'rganadilar.

O'quv guruhlar: AT-4/3-22, AT-4/4-22, KX-4/10-22.

Vaqt: 80 min.

O'tish joyi: 437-, 138-o'quv sinfi.

Mashg'ulot turi: Guruhli.

O'quv-moddiy ta'minot: Taqdimot va elektron ma'lumotlar.

O'quv texnik vositalar: Videoprojektor, 65 dyumli monoblok, kompyuter.

Tinglovchilar mustaqil tayyorgarligi uchun mavzu bo'yicha adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: “Innovatsion rivojlanish nashriyot- matbaa uyi”, 2020. -216 b.
2. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. “Python dasturlash tili” Darslik. Toshkent: 2024y. B - 316.
3. Sh.R. Sapayev “Python dasturlash tili asoslari”. O'quv qo'llanma. Toshkent: 2023y. B – 137.
4. Sh.R. Sapayev “PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish”. O'quv qo'llanma. Toshkent: 2024y. B - 150

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
1.	I. KIRISH QISMI –Dalolat qabul qilish, tinglovchilar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; –Tezkor axborotni yetkazish; –Mavzu va mashg'ulot maqsadini e'lon qilish; –Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O'quv savollari: 1. Scikit-learn yordamida neyron tarmoqlar yaratish. Scikit-learn kutubxonasi yordamida MLPClassifier va MLPRegressor modellari asosida sun'iy neyron tarmoqlar yaratish jarayoni o'rganiladi. Modelning qatlamlar soni, aktivatsiya funksiyasi, o'qitish tezligi va iteratsiyalar soni kabi parametrlar sozlanadi. Ushbu yondashuv kichik va o'rta hajmdagi ma'lumotlarda neyron tarmoqlarni samarali qo'llash imkonini beradi. 2. Oddiy sinfiylashtirish masalalarida tatbiq qilish.	60 30	Videoproektor yoki 65 dyumli monoblok, elektron taqdimot

	Scikit-learn paketidan foydalanib, klassifikatsiya masalalari — masalan, ma'lumotlarni toifalarga ajratish yoki natijani oldindan aniqlash — amaliyotda bajariladi. Ma'lumotlar to'plami modelga uzatiladi, o'qitish jarayoni amalga oshiriladi va natijalar aniqlik ko'rsatkichi (accuracy) asosida baholanadi. Bu amaliyot mashinali o'qitish jarayonining to'liq siklini amalda bajarish imkonini beradi.	30	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

**O'ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA
MUDOFAA UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**

**AXBOROT TEXNOLOGIYALARI VA SUN'IY INTELLEKT
KAFEDRASI**



«SUN'IY INTELLEKT ASOSLARI» FANIDAN
GLOSSARIY

GLOSSARIY

Termin	O'zbek tilidagi sharhi	Ingliz tilidagi sharhi
!=	Teng emas operatori; mantiqiy inkor amali qiymati bilan birhil.	The inequality operator; compares values for inequality returning a bool.
import	Kutubxonalarni chaqirish direktivasi	
+=	add-and-assign operatori; masalan a+=b vazifasi jihatdan a=a+b bilan bir xil	add-and-assign operator; a+=b is roughly equivalent to a=a+b.
address	Hotira manzili	a memory location
aggregate	Konstruktorsiz massiv yoki struktura	an array or a struct without a constructor
algorithm	Hisoblashning aniq ketma-ketligi	a precise definition of a computation.
and	&& mantiqiy “va” (ko‘paytirish) operatori sinonimi	synonym for &&, the logical and operator.
ANSI	Amerika milliy standart agentligi.	The American national standards organization.
application	Umumiy maqsadga ega dasturlar to‘plami	a collection of programs seen as serving a common purpose (usually providing a common interface to their users)
bit	0 yoki 1 qiymatga ega birlik hotira	a unit of memory that can hold 0 or 1.
bool	Mantiqiy tip. Bu tip faqat rost yoki yolg‘on qiymat qabul qiladi	the built-in Boolean type. A bool can have the values true and false.
bug	Xatolik termini	colloquial term for error.
byte	Xotiradagi bir nechta belgilar yig‘indisi	a unit of memory that can hold a character of the C++ representation character set.
Python	Tizimli dasturlashni protsedurali qo‘llab quvvatlovchi dasturlash tili.	a general-purpose programming language with a bias towards systems programming that supports procedural programming, data abstraction, object-oriented programming, and generic programming. C++ was designed and originally implemented by Bjarne Stroustrup.
char	Belgili tip. Har bir belgi 8 bit, ya’ni baytga teng.	character type; typically an 8-bit byte.
input	Standart kiritish oqimi	standard istream.
class	Foydalanuvchi belgilaydigan tur, sinf. Sinf foydalanuvchi funksiyasi,	a user-defined type. A class can have member functions, member

SUN'IY INTELLEKT ASOSLARI

	foydalanuvchi ma'lumotlari va kontentlari bo'lishi mumkin.	data, member constants, and member types.
interpretator	Python da yozilgan buyruqlarni mashina tiliga o'girib beruvchi vosita.	the part of a python implementation that produces object code from a translation unit.
copy()	Nusxa olish operatori	Copy operator
UML	Унификatsiya qilingan modellash tili	Unified Modeling Language
OMG	Obyektlarni boshqarish guruhi	Object Management Group
4GL	To'rtinchi avlod tili	Fourth-Generation Language
ANSI	Amerika milliy standartlash instituti	American National Standards Institute
AMPS		Advanced Mobile Phone Service
ERP	Korxona resurslarini rejalashtirish	Enterprise Resource Planning
CRM	Mijozlar bilan o'zaro munosabatlarni boshqarish	Customer Relations Management
SQL	Tuzilmalashgan so'rovlar tili	Structured Query Language
OLAP	Xaqiqiy vaqtda ma'lumotlarga analitik ishlov berish	On-Line Analytical Processing
OLTP	Xaqiqiy vaqtda tranzaksiyalarga ishlov berish	On-Line Transaction Processing
TCO	Egalik qilishning yalpi qiymati	Total Cost of Ownership
JIT	Ayni vaqtda	Just-In-Time
LAN	Lokal hisoblash tarmog'i	Local Area Network
MAN	Maxalliy hisoblash tarmog'i	Metropolitan Area Network
WAN	Xududiy hisoblash tarmog'i	Wide Area Network
ISO	Halqaro standartlashtirish tashkiloti	International Organization for Standardization
API	amaliy dasturlashtirish maxsus interfeysi	Application Programming Interface
WWW	Umumjahon o'rgamchak to'ri	World Wide Web
ASCII	Axborot almashishning Amerika standarti	American Standard Code for Information Interchange
LIFO	«Oxirida keldi, birinchi ketdi» prinsipi	Last In, First Out
FIFO	«Birinchi keldi, birinchi ketdi» prinsipi	First In, First Out
PDA	personal raqamlik otib	Personal Digital Assistant
Axborot	muayyan obyekt xususidagi bilimlarimizning noaniqlik darajasini pasaytirishga imkon beruvchi har qanday ma'lumot.	data that reduces the uncertainty degree of knowledge about certain object.
Avtomatlashtirilgan axborot tizimi	ma'lumotlarni va axborotni yaratish, uzatish, ishlash, tarqatish, saqlash va/yoki boshqarishga va hisoblashlarni amalga oshirishga mo'ljallangan dasturiy va apparat vositalar.	a set of software and hardware designed for the creation, transmission, processing, distribution, storage and/or data and information management and production calculations.
Xavf-xatarni agregirlash	birlashgan xavf-xatarni teranrok tushunishga mo'ljallangan bir necha xavf-xatarni bitta xavf-xatarga birlashtirish.	sombine multiple risks in a risk, aimed at a better understanding of the overall risk.

Xavf - xatar taxlili	nomuvofik hodisalar paydo bo'lish holida kutiladigan zararni aniqlash maqsadida, ehtimollik hisoblashlardan foydalanib, tizim xarakteristikalarini va salbiy tomonlarini o'rganish jarayoni. Xavf – xatarni tahlillash masalasi u yoki bu xavf – xatarning maqbullik darajasini aniqlashdan iborat.	the process of studying the characteristics and weaknesses of the system, conducted using a probabilistic calculations in order to determine the expected damage in case of adverse events. The task of risk analysis is to determine the acceptability of a risk to the system.
Xavfsizlik xizmati ma'muri	xavfsizlikni ta'minlashning bir yoki bir necha tizimi hamda loyihalashni nazoratlash va ulardan foydalanish xususida to'liq tasavvurga ega shaxs (yoki shaxslar guruhi).	person (or group of people) having (s) complete understanding of one or more security systems and controls (s) design and use.
Faol taxdid	tizim holatiga atayin ruxsatsiz o'zgartirish kiritish tahdidi.	the threat of a deliberate unauthorized system state changes.
Trafik taxlili	Trafik oqimini kuzatish (borligi, yukligi xajmi, yunalishi va chastotasi) asosida axborot xolati xususida xulosa qilish. 2. Deshifrlanishga sabab bo'lmaydigan, ammo g'animga yoki buzg'unchiga uzatilayotgan ochik matn va, umuman, kuzatilayotgan aloqa tizimining ishlashi xususidagi bilvosita axborotni olishiga imkon beruvchi aloqa tizimi orqali uzatiluvchi shifrlangan xabarlar majmuining tahlili. Trafik tahlili shifrlangan xabarlarning rasmiylashtirish xususiyatlaridan, ularning uzunligi, uzatish vaqti, uzatuvchi va qabul qiluvchi xususidagi malumotlardan foydalanadi.	Report on the state information based on observation of traffic flows (presence , absence, amount , direction and frequency . 2 . Analysis of all encrypted messages sent over the communication system does not lead to decrypt , but allowing the opponent and / or the offender obtain indirect information about the transmitted Post and generally observed on the functioning of the communication system. A. that uses features of registration messages encrypted , and their length , the transmission time , the data sender and recipient , etc.
Tarmoq taxlillagichlari (sniffer)	tarmoq trafigini "tinglash"ni va tarmoq trafigidan avtomatik tarzda foydalanuvchilar ismini, parollarni, kredit kartalar nomerini, shu kabi boshqa axborotni ajratib olishni amalga oshiruvchi dasturlar.	programs, asking for "listening" network traffic and automatically selects the network traffic of user names, passwords, credit card numbers, other similar information.
Apparat himoya	kompyuterda ma'lumotlarni himoyalashda apparat vositalardan, masalan, chegara registrlaridan yoki qulflardan va kalitlardan foydalanish.	the use of hardware, for example, registers boundaries or locks and keys to protect data in computers.
Faol xujum	kriptotizimga yoki kriptografik protokolga hujum bo'lib, unga binoan dushman va/yoki buzg'unchi konuniy foydalanuvchi xarakatiga ta'sir etishi, masalan, qonuniy foydalanuvchi xabarini almashtirishi yoki yo'q qilishi va xabarni yaratib uning nomidan uzatishi va h. mumkin.	attack on a cryptosystem or cryptographic protocol in which the offender or the enemy and can affect the legitimate user actions, for example, replace or remove legitimate posts, create and send messages on his behalf, etc.

Xizmat qilishdan voz kechishga undaydigan atayin qilinmaydigan hujum	tasodifiy yuz bergan holat natijasida (reklama natijasida qiziqishning keskin oshishi, to'satdan va kutilmagan kamdan – kam bo'ladigan mashhurlik va h.) DOS hujumning mavaffaqiyatli o'tkazilgani xususida taassurot tug'diruvchi qandaydir servis uchun xizmat qilishdan voz kechish. Ko'pincha faollikning avj onlarida yangilik saytlariga qiziqarli axborotni joylashtirish natijasida, xamda qidiruv tizimlari yordamida ommabop URL – havolalarni o'tkazish qobiliyati cheklangan foydalanish kanallariga ega media – resurslariga indekslash natijasida paydo bo'ladi.	denial of service for any service which has come as a result of chance (sharply increased interest in the result of the promotion , the sudden and unexpected exception pop, etc.) , giving the impression of a successful DoS attack . Often occurs in times of peak activity as a result of placement of interesting information on news sites , as well as from popular search engines indexing URL- links to various media resources with limited bandwidth channel access.
Xavfsizlik auditi	kompyuter tizimi xavfsizligiga ta'sir etuvchi bo'lishi mumkin bo'lgan xavfli harakatlarni xarakterlovchi, oldindan aniqlangan hodisalar to'plamini ro'yxatga olish(audit faylida qaydlash) yo'li bilan himoyalaniшни nazoratlash.	maintain security control by registering (fixation in the audit file) a predetermined set of events that characterize the potentially dangerous actions in the computer affecting its security.
Axborot tarmog'i xavfsizligi	axborot tarmog'ini ruxsatsiz foydalanishdan, me'yoriy harakatiga tasodifan aralashishdan yoki komponentlarini buzishga urinishdan saqlash choralari.	measures designed to protect network information from unauthorized access, accidental or intentional interference with normal activities or attempts to destroy its components.
Kompyuter tizimi xavfsizligi	destruktiv harakatlarga va yolg'on axborotni zo'rlab qabul qilinishiga olib keluvchi ishlanadigan va saqlanuvchi axborotdan ruxsatsiz foydalanishga urinishlarga kompyuter tizimining qarshi tura olish hususiyati.	property computer systems to resist attempts of unauthorized access to information processed and stored, the input of information, leading to destructive actions, and the imposition of false information.
Xodim xavfsizligi	qandaydir jiddiy axborotdan foydalanish imkoniyatiga ega barcha xodimlarning kerakli avtorizatsiyaga va barcha kerakli ruxsatnomalarga egalik kafolatini ta'minlovchi usul.	method of ensuring that all personnel having access to some sensitive information has the necessary authorization, as well as all the necessary permissions.
Korxona xavfsizligi	korxona o'z faoliyatini buzilishsiz va to'xtalishsiz yurgiza oladigan vaqt bo'yicha barqaror bashoratlanuvchi atrof-muhit holati.	stable condition projected time environment in which the company can carry out their activities without disturbances and interruptions.
Axborotning blokirovka qilinishi	axborotning texnik vositalar yordamida ishlanishida uning foydalanuvchanlik xususiyatining yo'qolishi. Natijada tanishish, xujjatlash, modifikatsiyalash yoki yo'q qilish bo'yicha vakolatli amallarni	the loss of information when it is processed by technical means accessibility property, expressed in difficulty or termination of authorized access to it for authorized operations

	bajarish qiyinlashadi yoki to'xtatiladi.	familiarization, documentation, modification, or destruction.
Xavf-xatarga ta'sir	havf-xatarni modifikatsiyalash (o'zgartirish) jarayoni. Havf-xatarga ta'sir quyidagilarni o'z ichiga olishi mumkin: faoliyatini boshlamaslik yoki davom ettirmaslik qarori orqali xavf-xatarni oldini olish natijasida xavf-xatar paydo bo'ladi; qulay imkoniyatdan foydalanish uchun xavf-xatarni qabul qilish yoki ko'paytirish; xavf-xatar manbaini yo'q qilish; imkoniyatini o'zgartirish; oqibatlarini o'zgartirish; xavf-xatarni boshqa taraf yoki taraflar bilan bo'lishish (jumladan, shartnomalar va xavf-xatarni moliyalash); xavf-xatarni tushungan holda ushlab qolish.	the modification (changing) risk. Effects on risk may include: - avoid the risk by decision not to commence or to continue, resulting in the risk; - Adoption or increase the risk for a favorable opportunity; - Eliminate the source of the risk; - Changing opportunities; - Change impacts; - Sharing the risk with another party or parties (including contracts and risk financing); - Conscious retention risk.
Konfidensial axborotdan foydalanish	muayyan shaxsga tarkibida konfidensial xarakterli ma'lumot bo'lgan axborot bilan tanishishga vakolatli mansabdor shaxsning ruxsati.	authorized authorized official introduction of a particular person with the information containing confidential information.
Axborotdan ruxsatsiz foydalanish	manfaatdor sub'ektning huquqiy hujjatlarni yoki mulkdor, axborot egasi tomonidan o'rnatilgan himoyalalanuvchi axborotdan foydalanish huquqlari yoki qoidalarini buzib himoyalalanuvchi axborotga ega bo'lishi.	preparation of protected information interested entity in violation of the legal instruments or by the owner, the owner of the information or rights of access to protected information.
Tizimning osilib qolishi	yangi topshiriqlar kiritilishini bostirish yo'li bilan multidastur tizimini to'xtatish ("yaxlatish").	system hang - stop ("freezing") multiprogramming system by inhibiting the entry of new jobs.
Axborotni fosh qilinishdan himoyalash	himoyalalanuvchi axborotni, ushbu axborotdan foydalanish xuquqiga ega bo'lmagan manfaatdor sub'ektlarga (iste'molchilarga) ruxsatsiz yetkazishni bartaraf etishga yo'naltirilgan axborot himoyasi.	the information security directed on prevention of unauthorized finishing of protected information to interested subjects (consumers), not having right of access to this information.
Ijtimoiy injeneriya	xizmatchi xodimlar va foydalanuvchilar bilan, turli nayrang, aldash va h. orqali chalg'itish asosidagi muloqotdan olinadigan axborot yordamida axborot tizimining xavfsizlik tizimini chetlab o'tish.	round of system of safety of system information by means of information received from contacts with the service personnel and users on the basis of their introduction in delusion at the expense of various tricks, deception and so forth.
Insayder	guruxga tegishli yashirin axborotdan foydalanish xuquqiga ega gurux a'zosi. Odatda, axborot sirqib chiqish bilan bog'liq mojaroda muhim shaxs hisoblanadi. Shu nuqtai nazaridan,	the member of group of the people having access to the classified information, belonging this group. As a rule, is the key character in the incident, connected with

	insayderlarning quyidagi xillari farqlanadi: beparvolar, manipulyatsiyalanuvchilar, ranjiganlar, qo'shimcha pul ishlovchilar va h.	information leakage. From this point of view distinguish the following types of insiders: negligent, manipulated, offended, disloyal, earning additionally, introduced, etc.
Insident	ruxsatsiz foydalanish xuquqiga ega bo'lishga yoki kompyuter tizimiga xujum o'tkazishga urinishning qayd etilgan xoli.	the recorded case of attempt of receiving unauthorized access or carrying out attack to computer system.
Tashqi insident	manba jabrlanuvchi tomon bilan bevosita bog'lanmagan buzg'unchi bo'lgan mojoro.	the incident which source is the violator who hasn't been connected with an affected party directly.
Ichki insident	manba jabrlanuvchi tomon bilan bevosita bog'langan (mehnat shartnomasi yoki boshqa usullar bilan) buzg'unchi bo'lgan mojoro.	the incident which source is the violator connected with an affected party directly (the employment contract or other ways).
Axborotdan noqonuniy foydalanish	qonuniy yo'l bilan olingan ma'lumotlarni, ushbu ma'lumotlarga va bunday harakatlarga qo'l uruvchi sub'ektlarga o'rnatilgan qoidalarni va vakolatlarni (sanksiyalarni) buzgan holda, uzatish, tarqatish (chop etish) va qo'llash.	transmission, distribution(publishing), the use of information obtained through legal means in violation of the rules and powers(sanctions) established for this information and subject, to take such action.
Suqilib kirishga sinash	tizimni uning himoyalash vositasini (xususan, ruxsatsiz foydalanishdan himoyalash vositasini) tekshirish maqsadida sinash.	system test for the purpose of check of means of its protection (in particular from illegally go access).
Yashirin kanal	xavfsizlik siyosatini buzish uchun ishlatilishi mumkin bo'lgan, axborot texnologiyasi tizimini va avtomatlashtirilgan tizimlarni ishlab chiqaruvchilar ko'zda tutmagan kommunikatsiya kanali. Axborot uzatish mexanizmi bo'yicha yashirin kanallar xotira bo'yicha yashirin kanalga, vaqt bo'yicha yashirin kanalga va statistik yashirin kanalga bo'linadi.	unforeseen the developer of system of technologies information and systems automated the communication channel which can be applied to security policy violation. On the information transfer mechanism c.s subdivide on: c.s. on memory c.s. on time; hidden statistical channels.
Axborot sirqib chiquvchi kanal	ko'riklanuvchi ma'lumotlardan (tijorat siri, fizik muhit va sanoat ayg'oqchilik vositalari majmui) ruxsatsiz foydalanishga imkon beruvchi konfidensial axborot manbaidan to niyati buzuqgacha bo'lgan fizik yo'l.	actual path from a source of confidential information to the malefactor, on which probably unauthorized obtaining protected data (set of a source of a trade secret, the physical environment and means of industrial espionage).
Keylogger	klaviaturali kiritishni ushlab qolishga mo'ljallangan dastur yoki apparat vosita. Bosilgan klavishlar skan-kodlarini aniqlashni va ularni	the program or the hardware device intended for interception of keyboard input. Carries out recognition of scan-codes of the

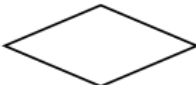
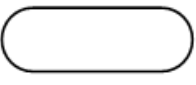
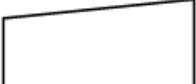
	yashirincha saqlashni va/yoki yashirincha qandaydir kanal orqali uzatishni amalga oshiradi.	pressed keys and the hidden preservation and/or their hidden transfer on any channel.
Kompyuter jinoyatchiligi	o'zi yoki uchinchi shaxs uchun mulkiy foyda olish hamda raqibiga mulkiy ziyon yetkazish maqsadida axborot resursidan ruxsatsiz foydalanishni, uni modifikatsiyalashni yoki yo'qotishni amalga oshirish.	implementation of unauthorized access to information resource, its modification (fake) or destruction for the purpose of obtaining property benefits for itself or for the third party, and also for causing property damage to the competitor.
Mantiqiy "bomba"	axborotdan ruxsatsiz foydalanish uchun ma'lum vaqtiy va axborot sharoitlarida ishga tushiriluvchi dastur.	the program which is started under certain temporary or information conditions for implementation of unauthorized access to information.

**O'ZBEKISTON RESPUBLIKASI HARBIIY XAVFSIZLI VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIIY
ALOQA INSTITUTI**

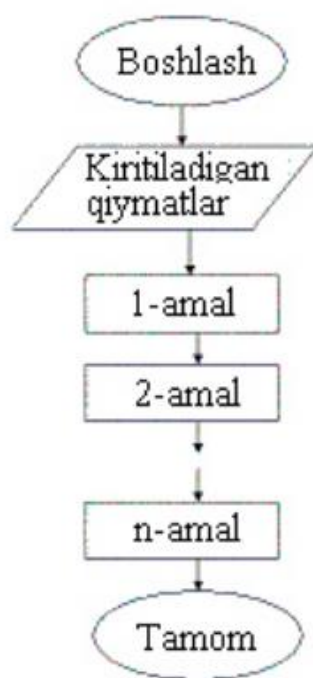
**AXBOROT TEXNOLOGIYALARI VA SUN'IY INTELLEKT
KAFEDRASI**



**«SUN'IY INTELLEKT ASOSLARI» FANIDAN
TARQATMA MATERIALLAR
TO'PLAMI (1-ILOVA)**

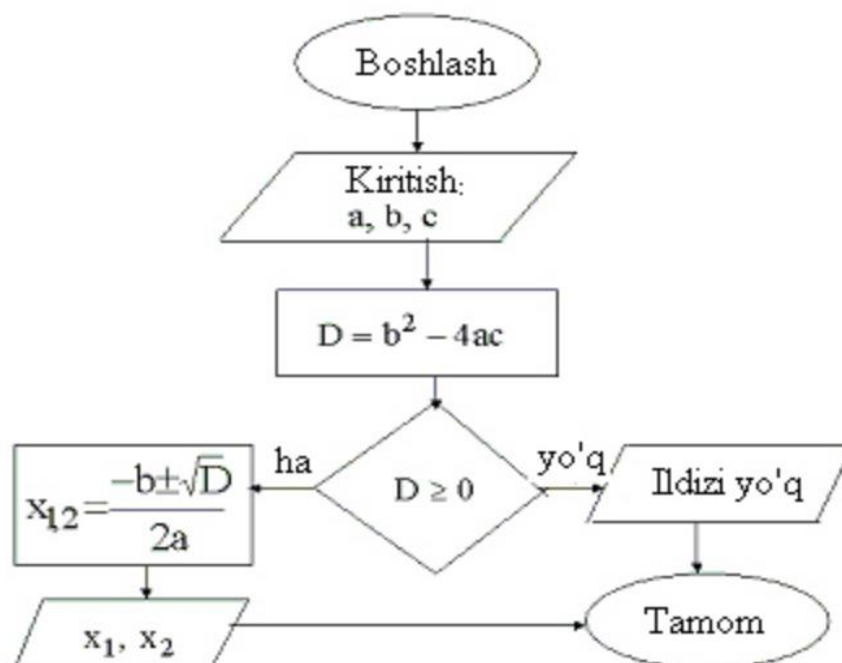
№	Шакллар номи	Шакллар	Мазмуни
1.	Жараён		Хисоблаш ва ҳисоблаш кет-ма-кетлик жараёни
2.	Ечим		Шартни текшириш
3.	Модификация		Цикл боши
4.	Чекланган жараён		Қисм дастурини ҳисоблаш
5.	Ҳужжатлар		Чиқариш, натижани қоғозга чоп этиш
6.	Киритиш-чиқариш		Қийматларни қайта ишлаш учун киритиш ва қайта ишлаш натижаларини чиқариш
7.	Бошланиш-тамом		Бошланиш, тамам, қийматларни қайта ишлаш ёки дастурни бажарилиш жараёнини узилиши
8.	Бирлаштириш		Шаклларни бирлаштириш кўрсаткичи
9.	Дисплей, экран		Қийматларни экранга чиқариш
10	Қўлда киритиш		Қийматларни клавиатура орқали киритиш

1-рasm. Алгоритм блок-сxemasida qoʻllaniladigan shakllar

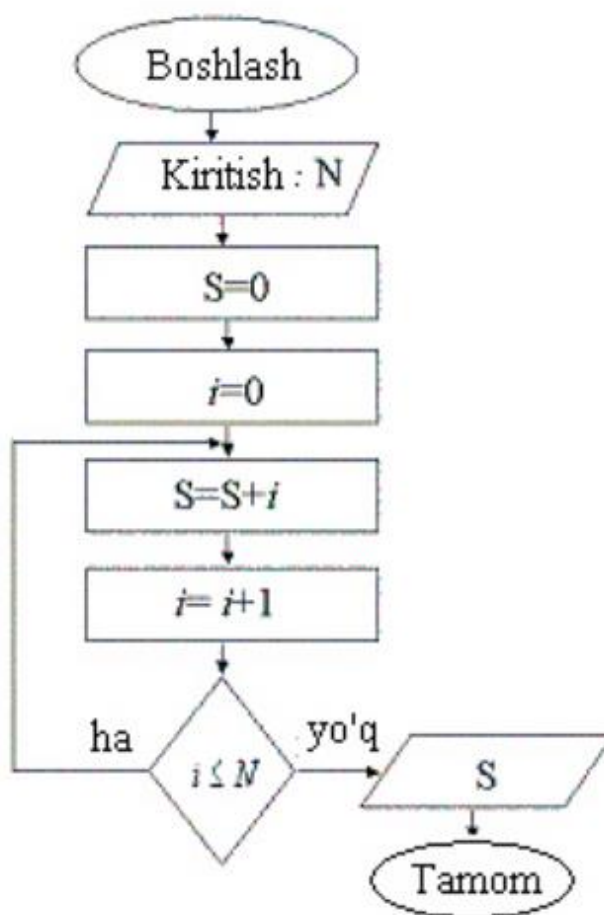


Chiziqli algoritmlar blok - sxemasining umumiy strukturasi

Misol sifatida $ax^2+bx+c=0$ kvadrat tenglamani yechish algoritmining blok-sxemasi quyida keltirilgan.



2-rasm. Algoritm blok-sxemasida qo'llaniladigan shakllar



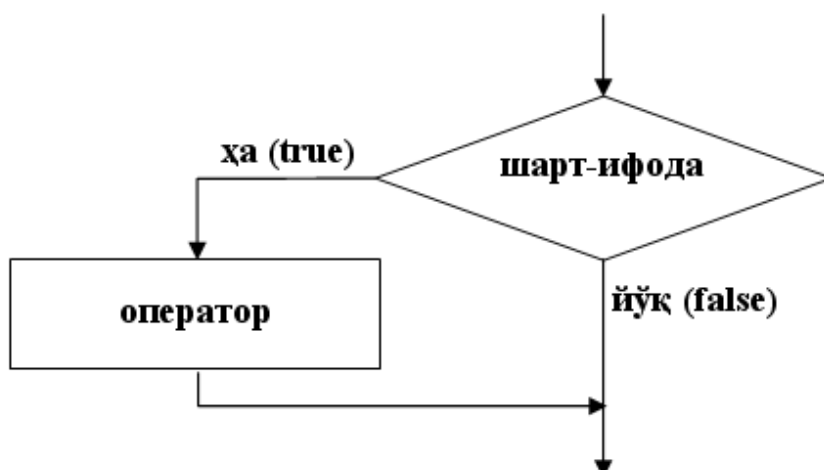
4-rasm. Python dasturlash tili standart funksiyalarning yozilishi

Funksiya	Ifodalanishi	Funksiya	Ifodalanishi
Sin x	sin(x)	$\sqrt{x}$	sqrt(x); pow(x,1/2.)
Cos x	cos(x)	$ x $	abs(x) yoki fabs(x)
Tg x	tan(x)	Arctan x	atan(x)
e^x	exp(x)	Arcsin x	asin(x) ?
Ln x	log(x)	Arccos x	acos(x) ?
Lg x	log10(x)	$\sqrt[3]{x^2}$	pow(x,2/3.)
x^a	pow(x,a)	Log ₂ x	log(x)/log(2)

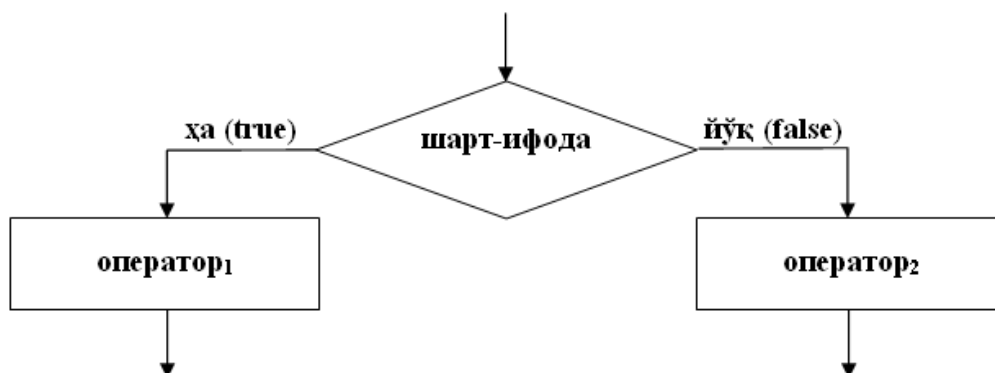
Escape belgilari	Ichki kod (16 son)	Nomi	Amal
------------------	--------------------	------	------

\\	0x5C	\	Teskari yon chiziqni chop etish
\'	0x27	'	Apostrofni chop etish
\"	0x22	"	Qo'shtirnoqni chop etish
\?	0x3F	?	So'roq belgisi
\a	0x07	bel	Tovush signalini berish
\b	0x08	bs	Kursorni 1 belgi o'rniga orqaga qaytarish
\f	0x0C	ff	Sahifani o'tkazish
\n	0x0A	lf	Qatorni o'tkazish
\r	0x0D	cr	Kursorni ayni qatorning boshiga qaytarish
\t	0x09	ht	Navbatdagi tabulyatsiya joyiga o'tish
\v	0x0D	vt	Vertikal tabulyatsiya (pastga)
\000	000		Sakkizlik kodi
\xNN	0xNN		Belgi o'n oltilik kodi bilan berilgan

5-rasm. PYTHON tilida escape -belgilar jadvali



if () shart operatorining blok sxemasi



if(); else shart operatorining blok sxemasi

```

from PyQt5.QtWidgets import QApplication, QMainWindow
import sys

class Window(QMainWindow):
    ...def __init__(self):
    .....super().__init__()

    .....self.setGeometry(300, 300, 600, 400)
    .....self.setWindowTitle("PyQt5 window")
    .....self.show()

app = QApplication(sys.argv)
window = Window()
sys.exit(app.exec_())

```

PyQt5 bilan iashlash

```

from tkinter import *
class Root(Tk):
    ...def __init__(self):
    .....super(Root, self).__init__()

    .....self.title("Python Tkinter")
    .....self.minsize(500, 400)

root = Root()
root.mainloop()

```

PyQt5 paketida dastur tuzish

```
pip install PySide2
```

Вот пример настройки графического фрейма с помощью PySide2.



```
from PySide2.QtWidgets import QtWidgets, QApplication  
import sys
```

```
class Window(QtWidgets):  
    def __init__(self):  
        super().__init__()
```

```
        self.setWindowTitle("Pyside2 Window")  
        self.setGeometry(300,300,500,400)
```

```
app = QApplication(sys.argv)  
window=Window()  
window.show()  
app.exec_()
```

Pip installer bilan ishlash

Введите эту команду и нажмите enter, она будет установлена. После этого вам нужно ввести эту команду для установки Kivy:

```
pip install Kivy
```

Поэтому после установки позвольте мне показать вам простой пример Kivy, чтобы показать, насколько это просто.

```
from kivy.app import App
from kivy.uix.button import Button

class TestApp(App):
    def build(self):
        return Button(text= " Hello Kivy World ")

TestApp().run()
```

kivy paketini o'rnatish va ishlash

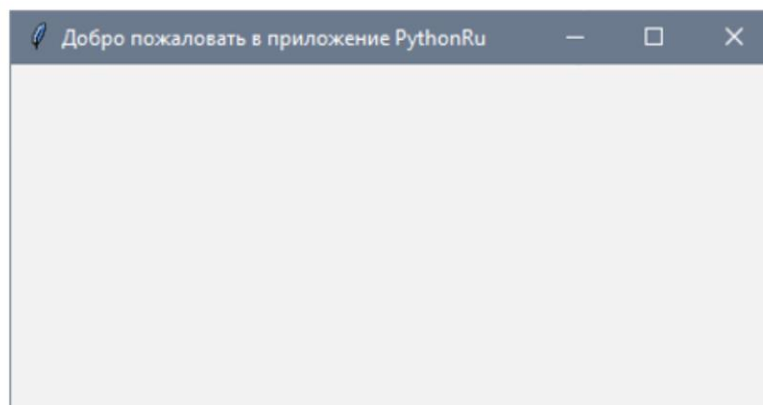
```
pip install wxPython
```

```
import wx
class MyFrame(wx.Frame):
    def __init__(self,parent,title):
        super(MyFrame,self).__init__(parent,title=title,size=(400,300))
        self.panel=MyPanel(self)
class MyPanel(wx.Panel):
    def __init__(self,parent):
        super(MyPanel,self).__init__(parent)
class MyApp(wx.App):
    def OnInit(self):
        self.frame=MyFrame(parent=None, title= "wxPython Window")
        self.frame.show()
        return True
app = MyApp()
app.MainLoop()
```

wxPython paketini o'rnatish va ishlash


```
from tkinter import *  
window = Tk()  
window.title("Добро пожаловать в приложение PythonRu")  
window.mainloop()
```

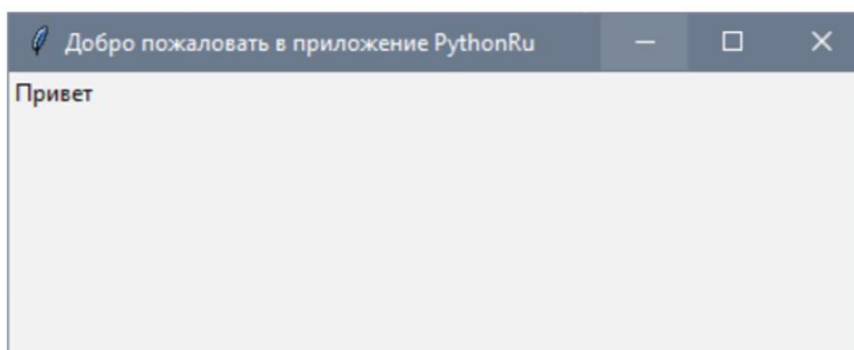
Результат будет выглядеть следующим образом:



Tkinter paketi bilan ishlash va uning natijasi

```
from tkinter import *  
window = Tk()  
window.title("Добро пожаловать в приложение PythonRu")  
lbl = Label(window, text="Привет")  
lbl.grid(column=0, row=0)  
window.mainloop()
```

И вот как будет выглядеть результат:



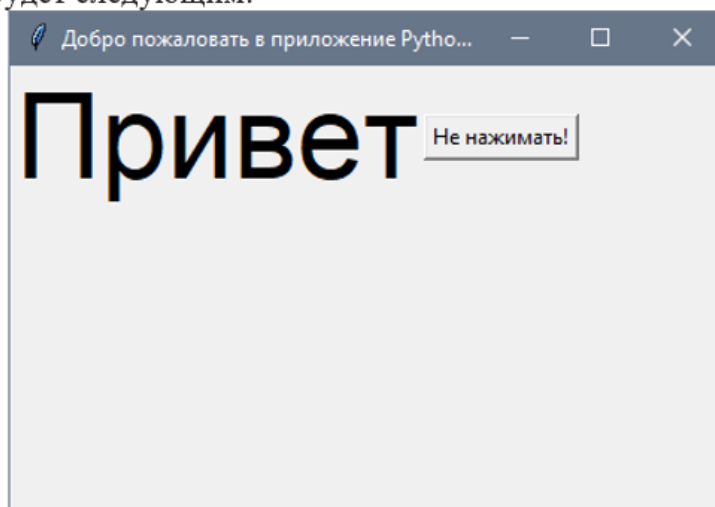
Tkinter paketi bilan ishlash va uning natijasi

```

from tkinter import *
window = Tk()
window.title("Добро пожаловать в приложение PythonRu")
window.geometry('400x250')
lbl = Label(window, text="Привет", font=("Arial Bold",50))
lbl.grid(column=0, row=0)
btn = Button(window, text="Не нажимать!")
btn.grid(column=1, row=0)
window.mainloop()

```

Результат будет следующим:



Tkinter paketi bilan ishlash va uning natijasi

```

PS C:\Users\ksahn> pip list
Package Version
-----
charset-normalizer 2.0.9
cyclr 0.11.0
fonttools 4.28.3
idna 3.3
imutils 0.5.4
kiwisolver 1.3.2
matplotlib 3.5.1
mysql-connector-python 8.0.27
numpy 1.21.4
opencv-contrib-python 4.5.4.6
packaging 21.3
Pillow 8.4.0
pip 21.3.1
protobuf 3.19.1
pyparsing 3.0.6
python-dateutil 2.8.2
setuptools 58.1.0
six 1.16.0
urllib3 1.26.7
PS C:\Users\ksahn>

```

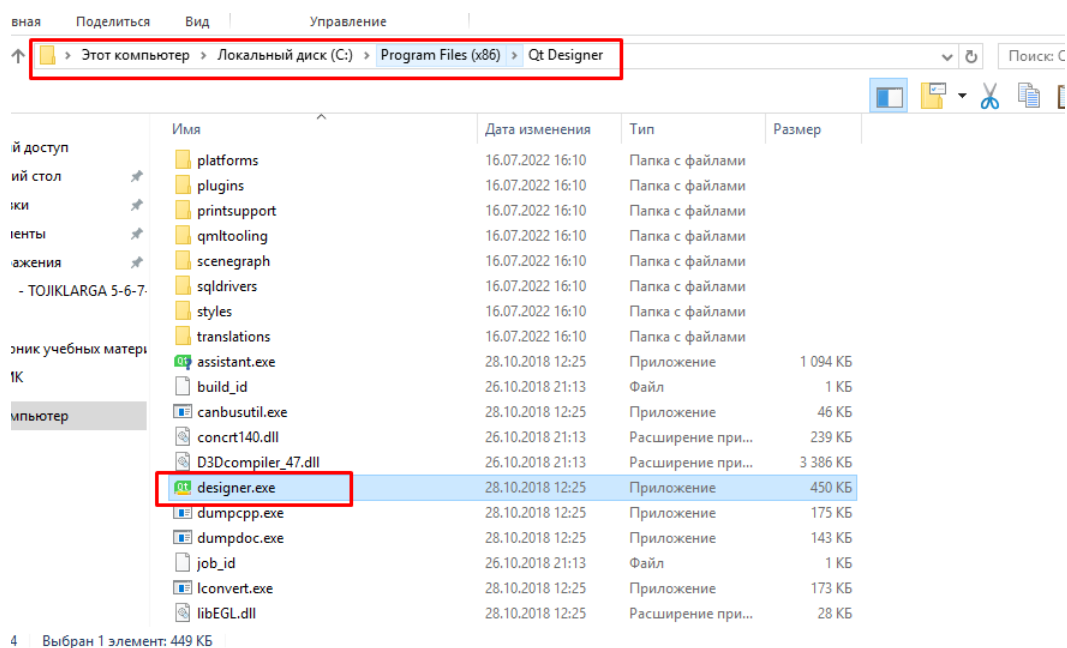
O'rnatilgan paketlarning ro'yxatini chiqarish

```

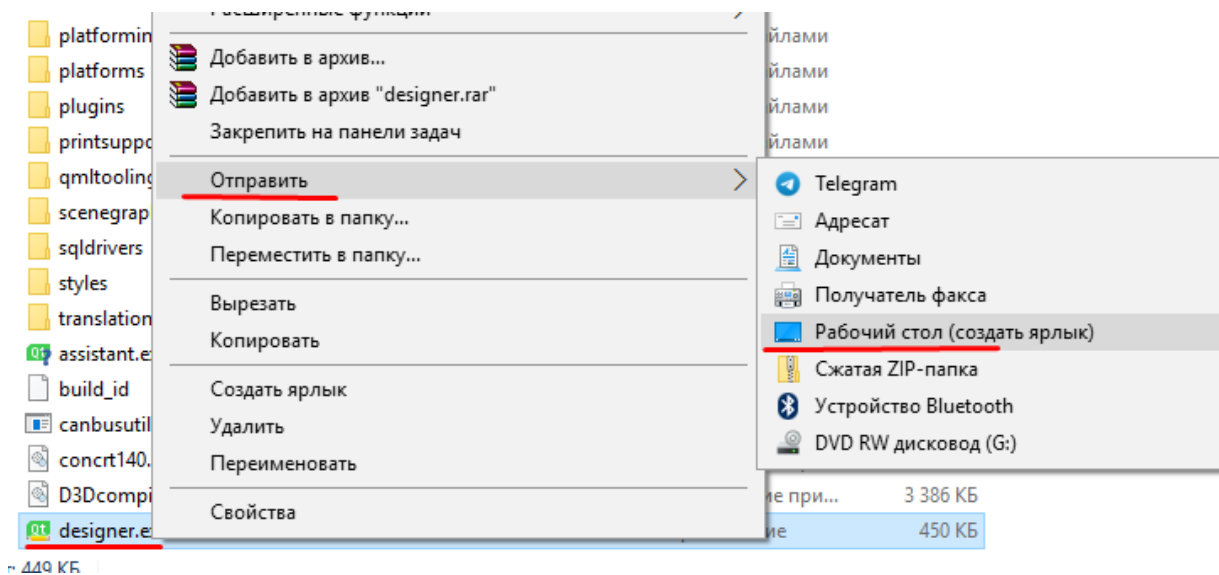
PS C:\Users\ > pip install PyQt5
Collecting PyQt5
  Downloading https://files.pythonhosted.org/packages/3b/d3/76670a331935f58f9a2ebd53c6e9b670bbf15c93500af5d323160/PyQt5-5.13.0-5.13.0-cp35.cp36.cp37.cp38-none-win_amd64.whl (49.7MB)
    100% |#####| 49.7MB 97kB/s
Requirement already satisfied: PyQt5_sip<13,>=4.19.14 in c:\python\lib\site-packages (from PyQt5)
Installing collected packages: PyQt5
Successfully installed PyQt5-5.13.0
You are using pip version 19.0.3, however version 19.1.1 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.

```

PyQt5 paketini o'rnatish



Qt Designer dasturini o'rnatish



Qt Designer dasturi yarliqini o'rnatish

№	Методы и описание
1	<u>setAlignment()</u> - Выравнивает текст в соответствии с константами выравнивания <u>Qt.AlignLeft</u> <u>Qt.AlignRight</u> <u>Qt.AlignCenter</u> <u>Qt.AlignJustify</u>
2	<u>setIndent()</u> - Устанавливает отступ текста меток
3	<u>setPixmap()</u> - Отображает изображение
4	<u>Text()</u> - Отображает заголовок метки
5	<u>setText()</u> - Программно устанавливает заголовок
6	<u>selectedText()</u> - Отображает выделенный текст из метки (для параметра <u>textInteractionFlag</u> должно быть установлено значение <u>TextSelectableByMouse</u>)
7	<u>setBuddy()</u> - Связывает метку с любым виджетом ввода
8	<u>setWordWrap()</u> - Включает или отключает перенос текста в этикетке

Qlabel vidgetining metodlari

```

import sys
from PyQt4.QtCore import *
from PyQt4.QtGui import *

def window():
    app = QApplication(sys.argv)
    win = QWidget()

    l1 = QLabel()
    l2 = QLabel()
    l3 = QLabel()
    l4 = QLabel()

    l1.setText("Hello World")
    l4.setText("TutorialsPoint")
    l2.setText("welcome to Python GUI Programming")

    l1.setAlignment(Qt.AlignCenter)
    l3.setAlignment(Qt.AlignCenter)
    l4.setAlignment(Qt.AlignRight)
    l3.setPixmap(QPixmap("python.jpg"))

    vbox = QVBoxLayout()
    vbox.addWidget(l1)
    vbox.addStretch()
    vbox.addWidget(l2)
    vbox.addStretch()
    vbox.addWidget(l3)
    vbox.addStretch()
    vbox.addWidget(l4)

    l1.setOpenExternalLinks(True)
    l4.linkActivated.connect(clicked)
    l2.linkHovered.connect(hovered)
    l1.setTextInteractionFlags(Qt.TextSelectableByMouse)
    win.setLayout(vbox)

    win.setWindowTitle("QLabel Demo")
    win.show()
    sys.exit(app.exec_())

def hovered():
    print "hovering"
def clicked():
    print "clicked"

if __name__ == '__main__':
    window()

```

QLabel widgeti bilan ishlash

№	Методы и описание
1	установить иконку () Отображает predetermined значок, соответствующий серьезности сообщения  Question (Вопрос)  Information (Информация)  Warning (Предупреждение)  Critic (Критический)
2	setText() Устанавливает отображаемый текст основного сообщения
3	setInformativeText() Отображает дополнительную информацию
4	setDetailText() В диалоговом окне отображается кнопка «Подробности». Этот текст появляется при нажатии на него
5	setTitle() Отображает пользовательский заголовок диалога
6	setStandardButtons() Список стандартных кнопок для отображения. Каждая кнопка связана с QMessageBox.Ok 0x00000400 QMessageBox.Open 0x00002000 QMessageBox.Save 0x00000800 QMessageBox.Cancel 0x00400000 QMessageBox.Close 0x00200000 QMessageBox.Yes 0x00004000 QMessageBox.No 0x00010000 QMessageBox.Abort 0x00040000 QMessageBox.Retry 0x00080000 QMessageBox.Ignore 0x00100000
7	setDefaultButton() Устанавливает кнопку по умолчанию. Он излучает сигнал щелчка, если нажата клавиша Enter.
8	setEscapeButton() Устанавливает кнопку, которая будет считаться нажатой, если нажата клавиша выхода

```

import sys
from PyQt4.QtGui import *
from PyQt4.QtCore import *

def window():
    app = QApplication(sys.argv)
    w = QWidget()
    b = QPushButton(w)
    b.setText("Show message!")

    b.move(50,50)
    b.clicked.connect(showdialog)
    w.setWindowTitle("PyQt Dialog demo")
    w.show()
    sys.exit(app.exec_())

def showdialog():
    msg = QMessageBox()
    msg.setIcon(QMessageBox.Information)

    msg.setText("This is a message box")
    msg.setInformativeText("This is additional information")
    msg.setWindowTitle("MessageBox demo")
    msg.setDetailedText("The details are as follows:")
    msg.setStandardButtons(QMessageBox.Ok | QMessageBox.Cancel)
    msg.buttonClicked.connect(msgbtn)

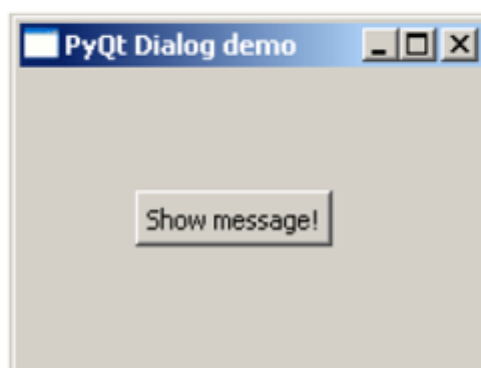
    retval = msg.exec_()
    print "value of pressed message box button:", retval

def msgbtn(i):
    print "Button pressed is:",i.text()

if __name__ == '__main__':
    window()

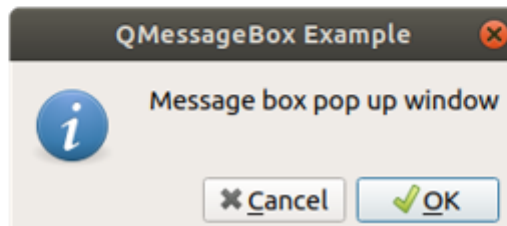
```

Приведенный выше код выводит следующий вывод:

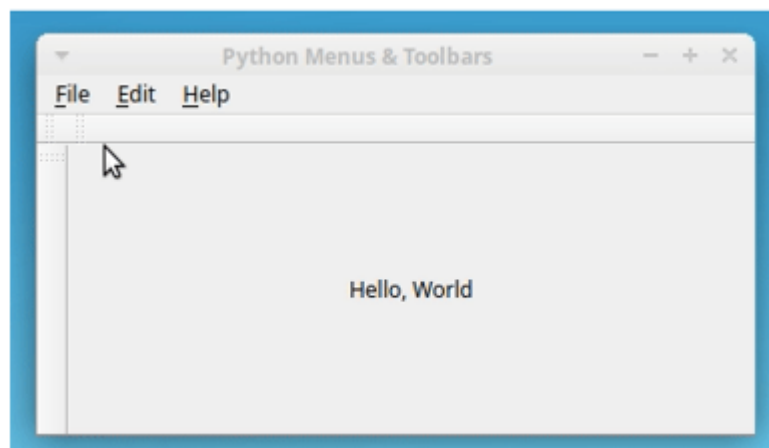


buttonClicked() signalining funktsiya bilan bog'lanishi

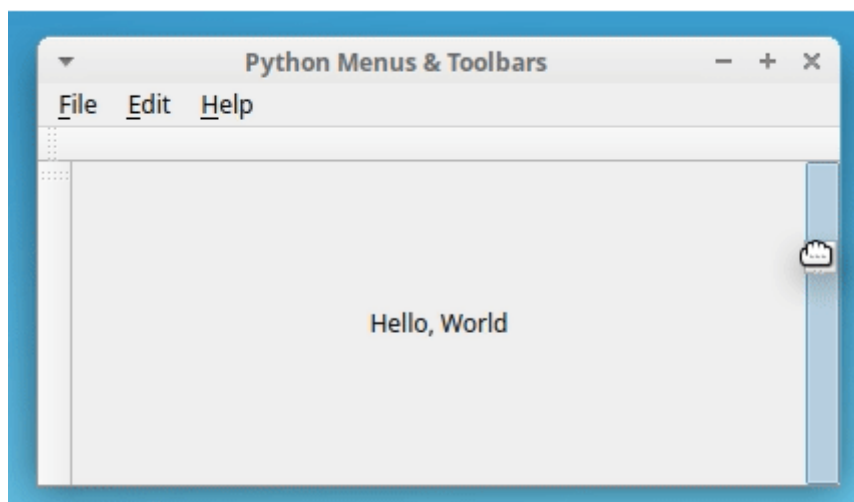
```
def showDialog():
    msgBox = QMessageBox()
    msgBox.setIcon(QMessageBox.Information)
    msgBox.setText("Message box pop up window")
    msgBox.setWindowTitle("QMessageBox Example")
    msgBox.setStandardButtons(QMessageBox.Ok | QMessageBox.Cancel)
    msgBox.buttonClicked.connect(msgButtonClick)
    returnValue = msgBox.exec()
    if returnValue == QMessageBox.Ok:
        print('OK clicked')
```



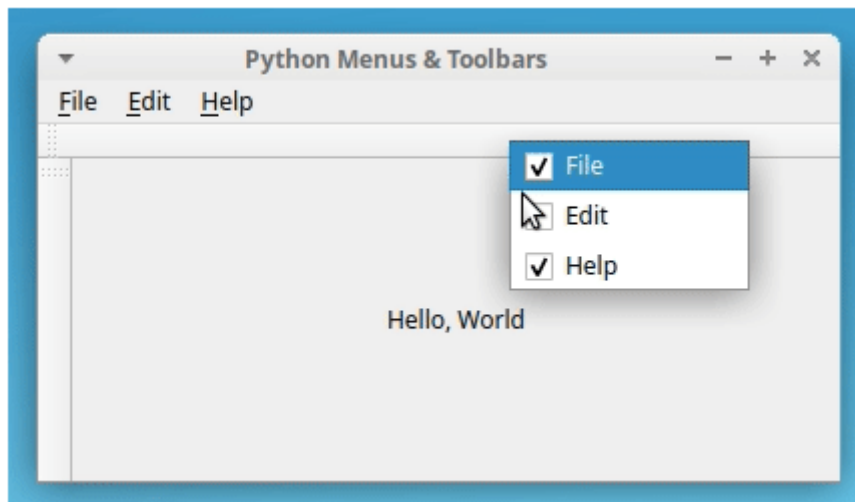
Dialog oyna hosil qilish



Panel instrument hosil qilish



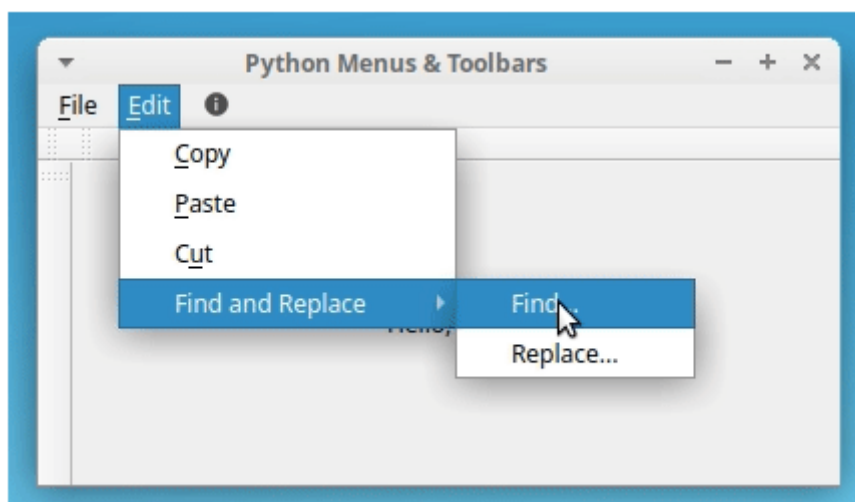
Panel instrument hosil qilish



Panel instrument hosil qilish

```
class Window(QMainWindow):
    # Snip...
    def _createMenuBar(self):
        # Snip...
        editMenu.addAction(self.cutAction)

        # Find and Replace submenu in the Edit menu
        findMenu = editMenu.addMenu("Find and Replace")
        findMenu.addAction("Find...")
        findMenu.addAction("Replace...")
        # Snip...
```



Panel instrument hosil qilish

**O'ZBEKISTON RESPUBLIKASI HARBIIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN'IY INTELLEKT”
KAFEDRASI**



“SUN'IY INTELLEKT ASOSLARI” FANIDAN

**FAN BO'YICHA YAKUNIY NAZORAT
QABUL QILISH UCHUN USLUBIIY
ISHLANMA (2-ILOVA)**

Axborot texnologiyalari va sun'iy intellekt kafedrası

“Sun'iy intellekt asoslari” o‘quv fanidan _____ o‘quv guruhi kursantlaridan baholi sinov
va imtihon qabul qilish uchun

USLUBIY ISHLANMA

O‘quv-tarbiyaviy maqsadi: Kursantlarning “Sun'iy intellekt asoslari” o‘quv fanidan o‘quv dasturlariga ko‘ra o‘tilgan mavzular va mashg‘ulotlar bo‘yicha olgan nazariy bilim va amaliy ko‘nikmalarini tekshirish, dasturlash tillari va texnologiyalari bo‘yicha amaliy ko‘nikmalarini mustahkamlash.

O‘tkazish joyi: ____ - o‘quv sinflari

O‘tkazish usuli: og‘zaki, yozma va amaliy

Vaqt: 6 o‘quv soati (240 daqiqa)

O‘quv guruhi	O‘tkaziladigan sana	O‘tkaziladigan vaqti

“Sun'iy intellekt asoslari” o‘quv fanidan o‘tkaziladigan imtihon mavzular rejasiga asosan 104 ta savolni tashkil etadi

1. Python paketini qanday o‘rnatish mumkin ?
2. Python kodini oddiy matnli faylga saqlash orqali ishga tushirish uchun qanday qadamlarni bajarishim kerak ?
3. Sun'iy intellekt asoslari bilan ishlash uchun qanday tarjimonlardan foydalanish mumkin ?
4. Sun'iy intellekt asoslarining nomi qayerdan kelib chiqqan ?
5. Sun'iy intellekt asoslarining tarixi haqida gapirib bering .
6. Sun'iy intellekt asoslari logotipini kim ixtiro qilgan .
7. Pythonni oddiy va tushunarli dasturlash tili nima qiladi ?
8. Pythonning boshqa dasturlash tillaridan qanday afzalliklari bor ?
9. Dasturlash tili yordamida qanday dasturlar yaratish mumkin
10. Python ? _ GUI deganda nimani tushunasiz?
11. Tkinter paketi va uning xususiyatlari.
12. Qanday grafik paketlarni bepul o‘rnatish mumkin?
13. Tkinter paketi va uning xususiyatlari haqida gapirib bering.
14. PySide2 paketining xususiyatlari qanday?
15. Python PyQt 5 paketining xususiyatlarini aytib bering .
16. Windows muhiti uchun PyQt 5 paketi qanday o‘rnatiladi ?
17. QtDesigner dasturining imkoniyatlari haqida gapirib bering.
18. QtDesignerni qanday o‘rnataman?
19. QLabel vidjetining xususiyatlari qanday ?
20. QLabel vidjetining o‘lchamini qanday o‘zgartirish mumkin ?
21. QLineEdit vidjetining xususiyatlari qanday ?
22. QLineEdit vidjetining asosiy xususiyatlari qanday ?
23. QLabel vidjetining xususiyatlari qanday?
24. QMessageBox vidjet funksiyasi?

25. QMessageBox vidjetining asosiy xususiyatlari nimalardan iborat ?
26. Belgilar va bitmaplarning funksiyalari ?
27. Rasmni joylashtirish uchun qaysi PyQt vidjetidan foydalaniladi?
28. Tasvirni joylashtirishda Label vidjetining qaysi xususiyatidan foydalaniladi?
29. Menyuda joylashtirish uchun qanday vidjet mavjud?
30. Menyu vidjetining xususiyatlari nimalardan iborat
31. Uskunalar paneli qanday amallar bilan to'ldiriladi?
32. PyQt da asboblarning paneliga variantlar qanday qo'shiladi?
33. Pythonida submenyu qanday yaratiladi?
34. Menyu amallar bilan qanday to'ldiriladi?
35. PyQt da piktogramma va resurslardan qanday foydalaniladi?
36. Menyu qatorlari qanday yaratiladi?
37. Pythonida qanday arifmetik operatorlar mavjud?
38. Butun sonlarga bo'lish qanday ishlaydi?
39. Ko'rsatkichlar qanday ishlaydi?
40. Bu kommutativ amal nima?
41. Pythonida qanday son turlari mavjud?
42. Mantiqiy ma'lumotlar bilan qanday ishlaymiz?
43. Pythonida qanday arifmetik operatorlar mavjud?
44. Butun sonlarga bo'lish qanday ishlaydi?
45. Ko'rsatkichlar qanday ishlaydi?
46. Bu kommutativ operatsiya nima ?
47. Pythonida son turlari qanday?
48. Mantiqiy ma'lumotlar bilan qanday ishlaymiz?
49. Argumentlar format() parametrlari sifatida?
50. Satr qo'shish operatori ?
51. Satrlarni ko'paytirish operatori?
52. dagi substring a'zolik operatori ?
53. Pythonida o'rnatilgan string funksiyalari.
54. Funksiya ord(c) .
55. Satrning registrini o'zgartirish .
56. string.swapcase() .
57. Satrdagi pastki qatorni toping va almashtiring.
58. String tasnifi .
59. string.islower() .
60. Chiziqlarni tekislash, chekinish
61. Qaysi operator shart operatori deyiladi?
62. If operatorining ishlash printsiplari nimadan iborat ?
63. Python tilidagi if , if - else va if - elif - else iboralarining farqi nimada ?
64. Berilgan sonlarning eng kattasini topish dasturi qanday?
65. Qanday operatorlar mantiqiy operatorlar deb ataladi?
66. Mantiqiy ko'paytirish, qo'shish operatorlari nimalardan iborat?
67. Berilgan sonlardan avval kichikroq, keyin esa kattasini ko'rsatadigan dastur qanday tuzilishga ega bo'ladi?
68. Pythonida sikl operatori nima?

69. While Loop zanjiri printsipi nimadan iborat?
70. 1 dan 50 gacha tub sonlarni topuvchi dastur qanday tuziladi?
71. While siklida break operatoridan foydalanish sababi nima?
72. Kartej nima?
73. Bo'g'imli tipning ishlash printsipi nimadan iborat
74. Toya qanday qadoqlanadi va qanday yo'llari bor?
75. Nima uchun u "obyektga mo'ljallangan dasturlash" deb ataladi?
76. Obyekt nima?
77. Sinf nima?
78. Klasslar va obyektlar qanday yaratiladi?
79. Initsializatsiya usuli nima?
80. __del__() usuli qanday vazifani bajaradi?
81. Meros nima ?
82. Qanday maxsus usullar mavjud?
83. __del__ usuli qanday vazifani bajaradi ?
84. __init__ usuli qanday vazifani bajaradi ?
85. __add__ usuli qanday vazifani bajaradi?
86. SQLite moduli qanday chaqiriladi ?
87. SQLite moduli yordamida ma'lumotlar bazasi qanday yaratiladi ?
88. Connect () funksiyasi qanday prinsipga asoslangan ?
89. Jadval qanday buyruqlar yordamida tuziladi?
90. INSERT buyrug'ining vazifasi va printsipi nimadan iborat ?
91. Execute () funksiyasi qanday ishlashini tushuntiring .
92. JB ning vazifasi nimadan iborat . yaqin ()?
93. JB ning vazifasi nimadan iborat . majburiyat ()?
94. Dump buyrug'i qanday vazifani bajaradi?
95. SQLite modulidagi WHILE bandining printsipi nimadan iborat ?
96. SELECT operatorining printsipi nima ?
97. UPDATE bayoni qanday prinsipga asoslangan ?
98. DELETE operatorining printsipi nimadan iborat ?
99. HAVING operatorining ishlash printsipi nimadan iborat ?
100. LIKE operatorining ishlash printsipi nimadan iborat ?
101. FROM operatorining ishlash printsipi nimadan iborat ?
102. GROUP BY buyrug'ining maqsadi nima ?
103. ORDER BY buyrug'ining maqsadi nima ?
104. Arifmetik operatorlarga ta'rif bering.

VAQT TAQSIMOTI VA USLUBIY TIZIMI

T/r	Imtihonning olib borilishi	Vaqt (daq)	O'qituvchining harakati
1.	I. KIRISH QISMI - guruhni tavsiya qiluvchidan bildiruv qabul qilish, salomlashish; - guruhning davomatini va kursantlarning imtihonga tayyorligini tekshirish (tashqi	5	Guruh ikki sherengaga saflanadi. Aniqlangan kamchiliklar bartaraf etiladi. Imtihondan ozod qilingan kursantlar e'lon qilinadi va taqdirlanadi. Imtihonga kirishga

T/r	Imtihonning olib borilishi	Vaqt (daq)	O'qituvchining harakati
	ko'rinishi, daftarlari, sinov daftarchalari, guruh jurnali, yozuv-chizuv anjomlari va h.k.); - imtihonni maqsadini va qabul qilish tartibini etkazish.		ruxsat berilmagan kursantlar e'lon qilinadi. Javob berishga tayyorlanish uchun vaqt 30 daqiqa ekanligi e'lon qilinadi, dastlabki topshiruvchi 5 ta kursant bittadan o'quv sinfiga kiritiladi. Qolgan kursantlar kafedraning bo'sh o'quv sinfiga kirib tayyorlanib, kutishadi.
2.	II. ASOSIY QISM <i>Imtihonning borishi.</i> Imtihon fandani o'quv dasturlariga ko'ra o'tilgan mavzular va mashg'ulotlar bo'yicha kursantlarning olgan nazariy bilim va amaliy ko'nikmalarini tekshirib ko'rish va baholash maqsadida o'tkaziladi. Chaqirilgan kursant imtihon topshirishga kelgani to'g'risida imtihon qabul qiluvchiga bildiruv beradi, masalan: <i>"Ustoz! Kursant G'ulomov «SUN'IY INTELLEKT ASOSLARI» FANIDAN imtihon topshirish uchun yetib keldi!"</i>, sinov daftarchasini o'qituvchiga taqdim qiladi, so'ngra bilek oladi, uning raqamini aytadi, bilek savollari bilan tanishib chiqadi va savollarni tushungani yoki tushunmagani to'g'risida bildiruv beradi, zarur bo'lganda savollarni aniqlashtiradi, javoblarni yozish uchun o'quv bo'limidan ro'yxatdan o'tgan toza varaq oladi, so'ngra ko'rsatilgan joyga borib o'tiradi va javob berishga tayyorgarlik ko'radi, zarur bo'lganda, o'qituvchining ruxsati bilan kompyuterda mashq qilib tayyorlanadi. Javob berishga tayyorlanayotgan kursant reja tuzib olishi yoki javobni yozishi mumkin, zarur hollarda sinf doskasiga yoki varaqqa chizmalarni, sxemalarni, hisoblarni va shunga o'xshashlarni tushirishi va bunda ruxsat etilgan materiallardan, o'quv plakatlaridan, sxemalardan foydalanishi mumkin. Javob berishga tayyor yoki belgilangan vaqti tugagan kursant o'qituvchining ruxsati bilan yoki uning chaqiruvi	220	O'qituvchi imtihon topshirgan kursantni baholashi tartibi: «a'lo» - agarda kursant o'quv dasturi bo'yicha teran va puxta bilimni ko'rsatsa, uni aniq va to'g'ri ifoda eta olsa, tezda to'g'ri qaror qabul qila olsa, javoblarda jiddiy xatolarga yo'l qo'ymasa, kompyuterda dastur kodlarini yoza olsa va maxsus amaliy dasturlarda to'g'ri ishlay olsa; «yaxshi» - agarda kursant o'quv dasturi bo'yicha puxta bilimni ko'rsatsa, uni aniq va to'g'ri ifoda eta olsa, to'g'ri qaror qabul qila olsa, javoblarda jiddiy xatolarga yo'l qo'ymasa, kompyuterda dastur kodlarini yoza olsa va maxsus amaliy dasturlarda to'g'ri ishlay olsa; «qoniqarli» - agarda kursant o'quv dasturini faqat asosiy qisimi bo'yicha bilimni ko'rsatsa, mashg'ulotning har bir qismini to'liq o'zlashtirmagan bo'lsa, javoblarda jiddiy xatolarga yo'l qo'ymasa, ayrim savollarga to'g'ri javob uchun yunaltiruvchi savollar berishni talab qilsa, kompyuterda dasturlar kodini qisman yoza olsa; «qoniqarsiz» - agarda kursant javob berishda qo'pol xatolarga yo'l qo'ysa, olgan bilimlari natijasida kompyuterda dastur kodlarini yoza olmasa. Topshiruvchi bilek savoliga javobni tugatgach, imtihon qabul

T/r	Imtihonning olib borilishi	Vaqt (daq)	O'qituvchining harakati
	bo'yicha etib kelib, bildiruv beradi, masalan: <i>“Ustoz! Kursant G'ulomov 5-bilet savollariga javob berish uchun yetib keldi!”</i> va biletga berilgan savollarga javob bera boshlaydi. Bilet savoliga javobni tugatgan kursant sinov qabul qiluvchiga bildiruv beradi, masalan: <i>«Ustoz! Kursant Abdullaev 1-savolga javobni tugatdi!»</i>. Imtihonda har bir topshiruvchiga faqat bitta bilet olishga ruxsat beriladi. Imtihon topshiruvchi bilet savolariga javob bera olmasligi to'g'risida bildiruv bergan hollarda u imtihondan o'tmagan hisoblanadi. Imtihon topshirish vaqtida ruxsat etilmagan har xil materiallardan foydalayotgan yoki imtihonda belgilangan tartib qoidani buzayotgan kursantlar intizomiy javobgarlikka tortiladilar. Imtihon qabul qiluvchi qaroriga ko'ra, bunday kursantlar biletsiz, butun o'quv fani bo'yicha imtihon topshirishi mumkin. Javob berishga tayyorlanish uchun 30 daqiqagacha, javob berish uchun esa o'quv me'yorini bajarish vaqtidan kelib chiqib, vaqt ajratiladi.		qiluvchi unga qo'shimcha va aniqlashtiruvchi savollar berishi mumkin. Imtihon natijasi bilet bo'yicha va berilgan qo'shimcha savollarga javoblarni tugatgach, kursantga e'lon qilinadi. Imtihonda o'quv me'yorini bajarish nazariy (o'quv me'yor nomi, bajariladigan ishlar hajmi va baholanish tartibini aytadi) va amaliy qismlardan iborat. O'quv me'yorini bajarishda umumiy baho O'R MV 2013-yildagi 105-sonli buyrug'i talablariga asosan baholanadi. Imtihon topshirish natijasi imtihon qabul qilgan o'qituvchi tomonidan topshiruvchiga e'lon qilinadi, imtihon vedomostiga va kursantning sinov daftarchasiga qo'yiladi, bunda sinovdan o'tolmasa sinov daftarchasiga qo'yilmaydi. Har bir belgi imtihon qabul qiluvchi imzosi bilan tasdiqlanadi.
3.	III.YAKUNIY QISM: - imtihon natijasi e'lon qilinadi; -imtihon natijalari tahlil qilinadi, a'lo va yomon tayyorgarlik ko'rgan kursantlar ta'kidlab o'tiladi, umumiy kamchiliklar ko'rsatiladi va ularni bartaraf qilish bo'yicha ko'rsatmalar beriladi; - tug'ilgan savollarga javob beriladi; - imtihon tugaganligi e'lon qilinadi.	15	Guruhning barcha shaxsiy tarkibi ikki sherengaga saflanadi. Imtihon natijalari, umumiy hatolar aytiladi, kelgusida ushbu hatolarni bartaraf etish bo'yicha ko'rsatmalar beriladi. Ish joylari, kompyuter va boshqa jihozlar tartibga keltiriladi.

**O'ZBEKISTON RESPUBLIKASI HARBIIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN'IY INTELLEKT”
KAFEDRASI**



“SUN'IY INTELLEKT ASOSLARI” FANIDAN

**FANNI O'QITISHDA
FOYDALANILADIGAN
INTERFAOL TA'LIM
METODLARI (3-ILOVA)**

“SWOT-tahlil” metodi

Metodning maqsadi: mavju ilimlar va amaliy tajribalarni tahlil qilish, taqqoslash orqali muammolarni yechish yo'llarini topishga, bilimlarni mustahkamlash, takrorlash, baholashga, mustaqil, tanqidiy fikrlashni, nostandart tafakkurni shakllantirishga xizmat qiladi.

S-(strength)	• Kuchli tomonlari
W – (weakness)	• Zaif, kuchsiz tomonlari
O – (opportunity)	• Imkoniyatlari
T – (threat)	• To'siqlar

Namuna: Sun'iy intellekt asoslarining SWOT tahlilini ushbu jadvalga tushiring.

S	Sun'iy intellekt asoslaridan foydalanishning kuchli tomonlari	Ushbu dasturlash tili tizimli dasturlash tili bo'lib, boshqa dasturlash tillarida ham tizimli masalalarni yechishda Python ga murojaat etiladi.
W	Sun'iy intellekt asoslaridan foydalanishning kuchsiz tomonlari	Dasturlash tilining strukturasi murakkabligi.
O	Sun'iy intellekt asoslaridan foydalanishning imkoniyatlari (ichki)	Sun'iy intellekt asoslarini mukammal o'zlashtirgan dasturchilar muammoli masalalarni yechishda va yuzaga keladigan xatoliklarni bartaraf etishda mukammal muhit hisoblanadi.
T	To'siqlar (tashqi)	Ma'lumotlar xavfsizligining to'laqonli ta'minlanmaganligi

Metodning maqsadi: Bu metod murakkab, ko'ptarmoqli, mumkin qadar, muammoli xarakteridagi mavzularni o'rganishga qaratilgan. Metodning mohiyati shundan iboratki, bunda mavzuning turli tarmoqlari bo'yicha bir xil axborot beriladi va ayni paytda, ularning har biri alohida aspektlarda muhokama etiladi. Masalan, muammo ijobiy va salbiy tomonlari, afzallik, fazilat va kamchiliklari, foyda va zararlari bo'yicha o'rganiladi. Bu interfaol metod tanqidiy, tahliliy, aniq mantiqiy

fikrlashni muvaffaqiyatli rivojlantirishga hamda o'quvchilarning mustaqil g'oyalari, fikrlarini yozma va og'zaki shaklda tizimli bayon etish, himoya qilishga imkoniyat yaratadi. "Xulosalash" metodidan ma'ruza mashg'ulotlarida individual va juftliklardagi ish shaklida, amaliy va seminar mashg'ulotlarida kichik guruhlardagi ish shaklida mavzu yuzasidan bilimlarni mustahkamlash, tahlili qilish va taqqoslash maqsadida foydalanish mumkin.

Metodni amalga oshirish tartibi:



trener tinglovchilarni 5-6 kishidan iborat kichik guruhlariga ajratadi;



trening maqsadi, shartlari va tartibi bilan ishtirokchilarni tanishtirgach, har bir guruhga umumiy muammoni tahlil qilinishi zarur bo'lgan qismlari tushirilgan tarqatma materiallarni tarqatadi.



har bir guruh o'ziga berilgan muammoni atroflicha tahlil qilib, o'z mulohazalarini tavsiya etilayotgan sxema bo'yicha tarqatmaga yozma bayon qiladi;



navbatdagi bosqichda barcha guruhlar o'z taqdimotlarini o'tkazadilar. Shundan so'ng, trener tomonidan tahlillar umumlashtiriladi, zaruriy axborotl bilan to'ldiriladi va mavzu yakunlanadi.

Namuna:

Dasturlash tillari					
C		C++		Visual C++	
afzalligi	kamchiligi	afzalligi	kamchiligi	afzalligi	kamchiligi
Xulosa:					

"Keys-stadi" metodi

«Keys-stadi» - inglizcha so'z bo'lib, («case» – aniq vaziyat, hodisa, «stadi» – o'rganmoq, tahlil qilmoq) aniq vaziyatlarni o'rganish, tahlil qilish asosida o'qitishni amalga oshirishga qaratilgan metod hisoblanadi. Mazkur metod dastlab 1921 yil Garvard universitetida amaliy vaziyatlardan iqtisodiy boshqaruv fanlarini o'rganishda

foydalanish tartibida qo'llanilgan. Keysda ochiq axborotlardan yoki aniq voqea-hodisadan vaziyat sifatida tahlil uchun foydalanish mumkin. Keys harakatlari o'z ichiga quyidagilarni qamrab oladi: Kim (Who), Qachon (When), Qayerda (Where), Nima uchun (Why), Qanday/ Qanaqa (How), Nima-natija (What).

“Keys metodi” ni amalga oshirish bosqichlari

Ish Bosqichlari	Faoliyat shakli va mazmuni
1-bosqich: Keys va uning axborot ta'minoti bilan tanishtirish	✓ yakka tartibdagi audio-vizual ish; ✓ keys bilan tanishish(matnli, audio yoki media shaklda); ✓ axborotni umumlashtirish; ✓ axborot tahlili; ✓ muammolarni aniqlash
2-bosqich: Keysni aniqlashtirish va o'quv topshirig'ni belgilash	✓ individual va guruhda ishlash; ✓ muammolarni dolzarblik iyerarxiyasini aniqlash; ✓ asosiy muammoli vaziyatni belgilash
3-bosqich: Keysdagi asosiy muammoni tahlil etish orqali o'quv topshirig'ining yechimini izlash, hal etish yo'llarini ishlab chiqish	✓ individual va guruhda ishlash; ✓ muqobil yechim yo'llarini ishlab chiqish; ✓ har bir yechimning imkoniyatlari va to'siqlarni tahlil qilish; ✓ muqobil yechimlarni tanlash
4-bosqich: Keys yechimini yechimini shakllantirish va asoslash, taqdimot.	✓ yakka va guruhda ishlash; ✓ muqobil variantlarni amalda qo'llash imkoniyatlarini asoslash; ✓ ijodiy-loyiha taqdimotini tayyorlash; ✓ yakuniy xulosa va vaziyat yechimining amaliy aspektlarini yoritish

Keys. Berilgan topshiriq asosida dastur algoritmi tuzilib C++ dasturlash tilida dastur matni yozildi. Dasturni acm.tuit.uz saytiga yuborilganda “kompilyatsiyada hatolik” habari chiqdi. Ya'ni Sistema yechimni qabul qilmadi.

Keysni bajarish bosqichlari va topshiriqlar:

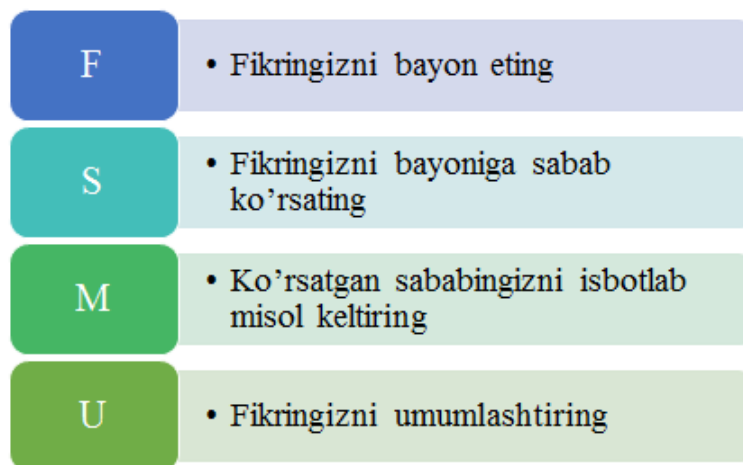
- Keysdgi muammoni keltirib chiqargan asosiy sabablarni belgilang (individual va kichik guruhda).
- Xatolikni bartaraf etuvchi ishlar ketma-ketligini belgilang (juftliklardagi ish).

«FSMU» metodi

Texnologiyaning maqsadi: Mazkur texnologiya ishtirokchilardagi umumiy fikrlardan xususiy xulosalar chiqarish, taqqoslash, qiyoslash orqali axborotni o'zlashtirish, xulosalash, shuningdek, mustaqil ijodiy fikrlash ko'nikmalarini shakllantirishga xizmat qiladi. Mazkur texnologiyadan ma'ruza mashg'ulotlarida, mustahkamlashda, o'tilgan mavzuni so'rashda, uyga vazifa berishda hamda amaliy mashg'ulot natijalarini tahlil etishda foydalanish tavsiya etiladi.

Texnologiyani amalga oshirish tartibi:

- qatnashchilarga mavzuga oid bo'lgan yakuniy xulosa yoki g'oya taklif etiladi;
- har bir ishtirokchiga FSMU texnologiyasining bosqichlari yozilgan qog'ozlarni tarqatiladi:



- ishtirokchilarning munosabatlari individual yoki guruhiiy tartibda taqdimot qilinadi.

FSMU tahlili qatnashchilarda kasbiy-nazariy bilimlarni amaliy mashqlar va mavjud tajribalar asosida tezroq va muvaffaqiyatli o'zlashtirilishiga asos bo'ladi.

Namuna.

Fikr: “Polimarfizim obyektga yo'naltirilgan dasturlashning asosiy tamoyillaridan biridir”.

Topshiriq: Mazkur fikrga nisbatan munosabatingizni FSMU orqali tahlil qiling.

“Assesment” metodi

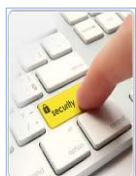
Metodning maqsadi: mazkur metod ta'lim oluvchilarning bilim darajasini baholash, nazorat qilish, o'zlashtirish ko'rsatkichi va amaliy ko'nikmalarini tekshirishga yo'naltirilgan. Mazkur texnika orqali ta'lim oluvchilarning bilish faoliyati turli yo'nalishlar (test, amaliy ko'nikmalar, muammoli vaziyatlar mashqi, qiyosiy tahlil, simptomlarni aniqlash) bo'yicha tashhis qilinadi va baholanadi.

Metodni amalga oshirish tartibi:

“Assesment” lardan ma'ruza mashg'ulotlarida talabalarning yoki qatnashchilarning mavjud bilim darajasini o'rganishda, yangi ma'lumotlarni bayon

qilishda, seminar, amaliy mashg'ulotlarda esa mavzu yoki ma'lumotlarni o'zlashtirish darajasini baholash, shuningdek, o'z-o'zini baholash maqsadida individual shaklda foydalanish tavsiya etiladi. Shuningdek, o'qituvchining ijodiy yondashuvi hamda o'quv maqsadlaridan kelib chiqib, assesmentga qo'shimcha topshiriqlarni kiritish mumkin.

Namuna. Har bir katakdagi to'g'ri javob 5 ball yoki 1-5 balgacha baholanishi mumkin.



Test

- 1.10^{-8} aniqlikda natijani chop etish qanday bajariladi?
- A. `printf("%.8f",x)`
- B. `cout<<x`
- C. `cout<<("%.8f",x)`



Qiyosiy tahlil

- Dasturiy mahsulotlardan foydalanish ko'rsatkichlarini tahlil qiling?



Tushuncha tahlili

- STD qisqartmasini izohlang.



Amaliy ko'nikma

- Grafik muhitida ishlash uchun qo'shimcha kutubxona fayllarini o'rnatish va sozlash.

“Insert” metodi

Metodning maqsadi: Mazkur metod o'quvchilarda yangi axborotlar tizimini qabul qilish va bilimlarni o'zlashtirilishini engillashtirish maqsadida qo'llaniladi, shuningdek, bu metod o'quvchilar uchun xotira mashqi vazifasini ham o'taydi.

Metodni amalga oshirish tartibi:

- o'qituvchi mashg'ulotga qadar mavzuning asosiy tushunchalari mazmuni yoritilgan input-matnni tarqatma yoki taqdimot ko'rinishida tayyorlaydi;
- yangi mavzu mohiyatini yorituvchi matn talim oluvchilarga tarqatiladi yoki taqdimot ko'rinishida namoyish etiladi;
- talim oluvchilar individual tarzda matn bilan tanishib chiqib, o'z shaxsiy qarashlarini maxsus belgilar orqali ifodalaydilar. Matn bilan ishlashda kursantlar yoki qatnashchilarga quyidagi maxsus belgilardan foydalanish tavsiya etiladi:

Belgilar	1-matn	2-matn	3-matn
“V” – tanish ma'lumot.			
“?” – mazkur ma'lumotni tushunmadim, izoh kerak.			

“+” bu ma'lumot men uchun yangilik.			
“-” bu fikr yoki mazkur ma'lumotga qarshiman?			

Belgilangan vaqt yakunlangach, talim oluvchilar uchun notanish va tushunarsiz bo'lgan ma'lumotlar o'qituvchi tomonidan tahlil qilinib, izohlanadi, ularning mohiyati to'liq yoritiladi. Savollarga javob beriladi va mashg'ulot yakunlanadi.

“Tushunchalar tahlili” metodi

Metodning maqsadi: mazkur metod kursantlar yoki qatnashchilarni mavzu buyicha tayanch tushunchalarni o'zlashtirish darajasini aniqlash, o'z bilimlarini mustaqil ravishda tekshirish, baholash, shuningdek, yangi mavzu buyicha dastlabki bilimlar darajasini tashhis qilish maqsadida qo'llaniladi.

Metodni amalga oshirish tartibi:

- ishtirokchilar mashg'ulot qoidalari bilan tanishtiriladi;
- o'quvchilarga mavzuga yoki bobga tegishli bo'lgan so'zlar, tushunchalar nomi tushirilgan tarqatmalar beriladi (individual yoki guruhli tartibda);
- o'quvchilar mazkur tushunchalar qanday ma'no anglatishi, qachon, qanday holatlarda qo'llanilishi haqida yozma ma'lumot beradilar;
- belgilangan vaqt yakuniga etgach o'qituvchi berilgan tushunchalarning tugri va tuliq izohini uqib eshittiradi yoki slayd orqali namoyish etadi;
- har bir ishtirokchi berilgan tugri javoblar bilan uzining shaxsiy munosabatini taqqoslaydi, farqlarini aniqlaydi va o'z bilim darajasini tekshirib, baholaydi.

Namuna: “Moduldagi tayanch tushunchalar tahlili”

Tushunchalar	Sizningcha bu tushuncha qanday ma'noni anglatadi?	Qo'shimcha ma'lumot
Registr	Foylanuvchini ro'yxatga olish	
Subscribe	Foydalanuvchini xizmatga a'zo bo'lishini ta'minlash	
Invite	Ulanish o'rnatilishi uchun sofswitch bilan bog'lanish	
Acknowledge	Aloqani tasdiqlash va foydalanuvchiga aloqa qilishini ta'minlash	
200 Ok	Softswitch foydalanuvchining so'roviga ijobiy javob qaytarishi	
Release	Aloqa tugatilganida kanalni bo'sh holatga qaytarish	

Izoh: Ikkinchi ustunchaga qatnashchilar tomonidan fikr bildiriladi. Mazkur tushunchalar haqida qo'shimcha ma'lumot glossariyda keltirilgan.

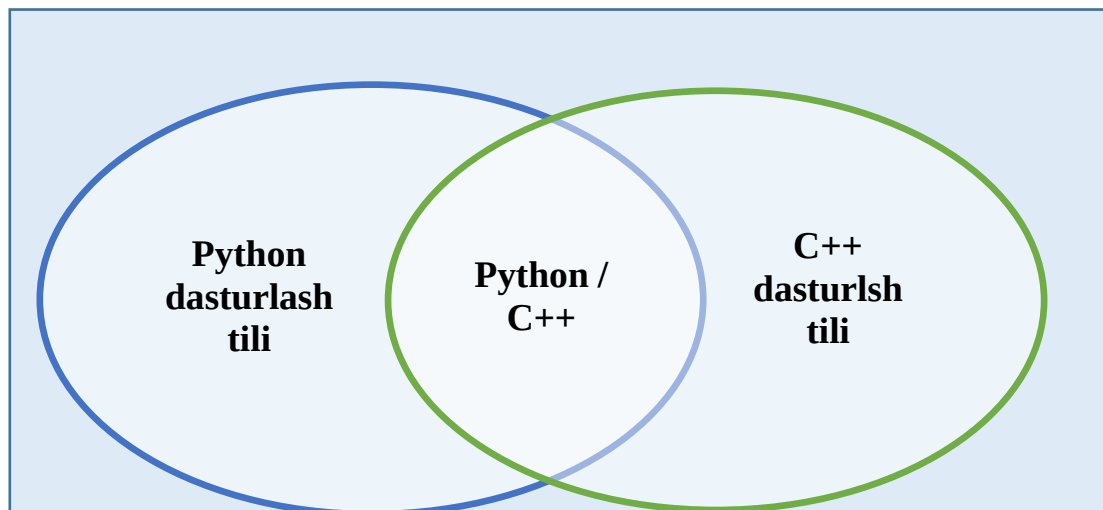
Venn Diagrammasi metodi

Metodning maqsadi: Bu metod grafik tasvir orqali o'qitishni tashkil etish shakli bo'lib, u ikkita o'zaro kesishgan aylana tasviri orqali ifodalanadi. Mazkur metod turli tushunchalar, asoslar, tasavurlarning analiz va sintezini ikki aspekt orqali ko'rib chiqish, ularning umumiy va farqlovchi jihatlarini aniqlash, taqqoslash imkonini beradi.

Metodni amalga oshirish tartibi:

- ishtirokchilar ikki kishidan iborat juftliklarga birlashtiriladilar va ularga ko'rib chiqilayotgan tushuncha yoki asosning o'ziga xos, farqli jihatlarini (yoki aksi) doiralar ichiga yozib chiqish taklif etiladi;
- navbatdagi bosqichda ishtirokchilar to'rt kishidan iborat kichik guruhlariga birlashtiriladi va har bir juftlik o'z tahlili bilan guruh a'zolarini tanishtiradilar;
- juftliklarning tahlili eshitilgach, ular birgalashib, ko'rib chiqilayotgan muammo yohud tushunchalarning umumiy jihatlarini (yoki farqli) izlab topadilar, umumlashtiradilar va doirachalarning kesishgan qismiga yozadilar.

Namuna: Dasturlash tillari turlari bo'yicha



“Blis-o‘yin” metodi

Metodning maqsadi: o'quvchilarda tezlik, axborotlar tizmini tahlil qilish, rejalashtirish, prognozlash ko'nikmalarini shakllantirishdan iborat. Mazkur metodni baholash va mustahkamlash maksadida qo'llash samarali natijalarni beradi.

Metodni amalga oshirish bosqichlari:

1. Dastlab ishtirokchilarga belgilangan mavzu yuzasidan tayyorlangan topshiriq, ya'ni tarqatma materiallarni alohida-alohida beriladi va ulardan materialni sinchiklab o'rganish talab etiladi. Shundan so'ng, ishtirokchilarga to'g'ri javoblar tarqatmadagi «yakka baho» kolonkasiga belgilash kerakligi tushuntiriladi. Bu bosqichda vazifa yakka tartibda bajariladi.

2. Navbatdagi bosqichda trener-o'qituvchi ishtirokchilarga uch kishidan iborat kichik guruhlariga birlashtiradi va guruh a'zolarini o'z fikrlari bilan guruhdoshlarini

tanishtirib, bahslashib, bir-biriga ta'sir o'tkazib, o'z fikrlariga ishonirish, kelishgan holda bir to'xtamga kelib, javoblarini «guruh bahosi» bo'limiga raqamlar bilan belgilab chiqishni topshiradi. Bu vazifa uchun 15 daqiqa vaqt beriladi.

3. Barcha kichik guruhlar o'z ishlarini tugatgach, to'g'ri harakatlar ketma-ketligi trener-o'qituvchi tomonidan o'qib eshittiriladi, va o'quvchilardan bu javoblarni «to'g'ri javob» bo'limiga yozish so'raladi.

4. «To'g'ri javob» bo'limida berilgan raqamlardan «yakka baho» bo'limida berilgan raqamlar taqqoslanib, farq bulsa «0», mos kelsa «1» ball quyish so'raladi. Shundan so'ng «yakka xato» bo'limidagi farqlar yuqoridan pastga qarab qo'shib chiqilib, umumiy yig'indi hisoblanadi.

5. Xuddi shu tartibda «to'g'ri javob» va «guruh bahosi» o'rtasidagi farq chiqariladi va ballar «guruh xatosi» bo'limiga yozib, yuqoridan pastga qarab qo'shiladi va umumiy yig'indi keltirib chiqariladi.

6. Trener-o'qituvchi yakka va guruh xatolarini to'plangan umumiy yig'indi bo'yicha alohida-alohida sharhlab beradi.

7. Ishtirokchilarga olgan baholariga qarab, ularning mavzu bo'yicha o'zlashtirish darajalari aniqlanadi.

«Dasturiy vositalarni o'rnatish va sozlash» ketma-ketligini joylashtiring.

O'zingizni tekshirib ko'ring!

Harakatlar mazmuni	Yakka baho	Yakka xato	To'g'ri javob	Guruh bahosi	Guruh xatosi
import math			1		
print("Hello world!")			2		

“Brifing” metodi

“Brifing”- (ing. briefing-qisqa) biror-bir masala yoki savolning muhokamasiga bag'ishlangan qisqa press-konferensiya.

O'tkazish bosqichlari:

Taqdimot qismi.

Muhokama jarayoni (savol-javoblar asosida).

Brifinglardan trening yakunlarini tahlil qilishda foydalanish mumkin. Shuningdek, amaliy o'yinlarning bir shakli sifatida qatnashchilar bilan birga dolzarb mavzu yoki muammo muhokamasiga bag'ishlangan brifinglar tashkil etish mumkin bo'ladi. Tinglovchilar yoki tinglovchilar tomonidan yaratilgan mobil ilovalarning taqdimotini o'tkazishda ham foydalanish mumkin. **“Portfolio” metodi** “Portfolio” – (ital. portfolio-portfel, ingl.hujjatlar uchun papka) ta'limiy va kasbiy faoliyat natijalarini autentik baholashga xizmat qiluvchi zamonaviy ta'lim texnologiyalaridan hisoblanadi. Portfolio mutaxassisning saralangan o'quv-metodik ishlari, kasbiy yutuqlari yig'indisi sifatida aks etadi. Jumladan, talaba yoki tinglovchilarning modul yuzasidan

o'zlashtirish natijasini elektron portfoliolar orqali tekshirish mumkin bo'ladi. Oliy ta'lim muassasalarida portfolioning quyidagi turlari mavjud:

Faoliyat turi	Ish shakli	
	Individual	Guruh
Ta'limiy faoliyat	Tinglovchilar portfoliosi, bitiruvchi, doktorant, tinglovchi portfoliosi va boshq.	Tinglovchilar guruhi, tinglovchilar guruhi portfoliosi va boshq.
Pedagogik faoliyat	O'qituvchi portfoliosi, rahbar xodim portfoliosi	Kafedra, fakultet, markaz, OTM portfoliosi va boshq.

SUN'IY INTELLEKT ASOSLARI

**O'ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN'IY INTELLEKT”
KAFEDRASI**



“SUN'IY INTELLEKT ASOSLARI”

FANNING O'QUV DASTURI

Toshkent 2025 y.

SUN'IY INTELLEKT ASOSLARI

1. O'quv fanining dolzarbligi va oliy kasbiy ta'lim dasturidagi o'rni.

Fan bo'lajak ofitserlarni ta'lim yo'nalishi bo'yicha yetuk mutaxassis bo'lib chiqishida muhim ahamiyatga ega bo'lib, bugungi zamonaviy axborot kommunikatsiya texnologiya vositalari rivojlangan davrda juda ham **dolzarb** ahamiyatga ega hisoblanadi.

Ushbu o'quv fani dasturiy ta'minot ishlab chiqarishda hamda organuvchilarga sun'iy intellektdan foydalanish usullariga oid bilimlarni berishi ta'lim dasturidagi o'rni belgilaydi.

Mazkur fan majburiy fanlar bloki tarkibiga kirib, 4- bosqichning 7-semestrda o'qitish maqsadga muvofiq. Ushbu fanni o'rganishda "Dasturlash asoslari", "Algoritmash", "Ma'lumotlar bazasini boshqarish tizimlari", "Dasturlash texnologiyalari" kabi fanlar nazariy zamin bo'lib xizmat qiladi hamda ushbu fanning o'zi "Obyektga yo'naltirilgan dasturlash" fani uchun nazariy zamin bo'lib xizmat qiladi.

Kursant fanni o'zlashtirish uchun dasturlash asoslari va kompyuter savodxonligi bo'yicha bilim, ko'nikma va malakalarga ega bo'lishi lozim.

2. O'quv fanining maqsadi va vazifasi

O'quv fanini o'qitishdan asosiy **maqsad** turli sohalarda zamonaviy AKT vositalari va dasturiy ta'minoti tuzilishi, ishlash jarayoni hamda yaratilish bosqichlari, sun'iy intellektdan foydalanishni nazariy va amaliy jihatdan o'rgatish.

Fanning **vazifasi**ga kursantlarda AKT vositalari va dasturiy ta'minoti tuzilishida zamonaviy qurilmalarning dasturiy ta'minoti, ulardan foydalanish, shuningdek, ularning zamonaviy imkoniyatlaridan foydalanishga oid tayyorgarlikni shakllantirish;

ijodiy ravishda mustaqil bilim olish ko'nikma, malakalarni hosil qilish, ularni O'zbekiston Respublikasi qo'shinlarida jangovar tayyorgarlik va xavfsizlikni mustahkamlash hamda obyektgga yo'naltirilgan dasturlash hamda AKT vositalarining dasturiy vositalaridan samarali foydalanishga yo'naltirish;

zamonaviy xavfsizlik qurilmalarining dasturiy va texnik ta'minotidan foydalangan holda harbiy maqsadlar uchun dasturiy ishlanmalar yaratish, harbiy texnik va dasturiy tizimlarga oid bilim va malakalarni shakllantirish, zamonaviy texnik qurilmalar, dasturiy vositalar hamda ular bilan ishlash, takomillashtirish, raqamli qurilmalarda ishlash ko'nikmasi va malakalarini o'zlashtirish, harbiy sohada sun'iy intellekt tizimlarini qo'llash orqali yuksak marallarga erishish, mashinali o'qitish sohasidagi bilimlar asosida qurolli kuchlarimizga kerakli turli qurilma va texnikalarni yasash.

3. O'quv fanining mazmuni**1-mavzu: "Sun'iy intellekt asoslari" faniga kirish va asosiy tushunchalari.**

Python dasturlash tilining klassifikatsiyasi va rivojlanish tarixi. Python dasturlash tilining asosiy tushunchalari. Fanning mazmuni, maqsadi, vazifalari. Pythonni o'rnatish. PyCharm dasturini o'rnatish. Pythonda "Hello world!" dasturini tuzish. Python tilining asosiy operatorlari bilan tanishish. Pythonda arifmetik operatorlar. Arifmetik operatorlar. Sonlar bilan ishlash. O'zgaruvchilar. Bool tipli ma'lumotlar bilan ishlash. Satrlar bilan ishlovchi operatorlar va metodlar. Satrlar bilan ishlovchi operatorlar va metodlar. str.format() metodi yordamida satrlarni formatlash. Pythonda shart operatori uning qo'llanilishi. IF, IF-ELSE va IF-ELIF-ELSE operatorlari. Pythonda takrorlanuvchi jarayonlarni dasturlash. Sikl operatorlari – For va while bilan ishlash. Break, continue va else operatorlarining qo'llanilishi. Pythonda for,while, break va continue operatorlariga doir dasturlar tuzish. Sikl operatorlari – For va while ga doir dasturlar tuzish. Break, continue va else operatorlariga doir dasturlartuzish. Pythonda ro'yxatlar va ular bilan ishlovchi metodlar. Ro'yxatlar va ularning qo'llanilishi. Ro'yxatlarni yaratish usullari. Ro'yxatlar bilan ishlovchi metodlar. Pythonda Funksiya tushunchasi. Foydalanuvchi funksiyasi. Funksiyalarni aniqlash va uni chaqirish.

Parametrli va parametrsiz funksiya. Fayllar va kataloglar bilan ishlash. Faylni ochish. Fayllar bilan ishlovchi metodlar. os modulining imkoniyatlari. Fayl va katalog yo'lini o'zgartirish. Katalog va fayl bilan ishlovchi funksiya va metodlar. Pythonda OOP asoslari. OOP asoslari. Klasslarni e'lon qilish va nusxasini yaratish. Sinf va obyekt. Sinf konstruktori. `__init__()` va `__del__()` metodlari. Pythonda shart operatori satrlarga doir masalalar yechish. Shart operatoriga doir dasturlari tuzish. IF, IF-ELSE va IF-ELIF-ELSE operatorlariga doir dasturlar tuzish. Pythonda satrlarga doir masalalar yechish. Pythonda ro'yxatlarga doir masalalar yechish. Pythonda ro'yxatlarga doir oddiy masalalar yechish. Ro'yxatlarni ishlovchi metodlarga doir masalalar yechish. Pythonda Funksiyaga doir dasturlar tuzish. Funksiyalarni aniqlash va uni chaqirishni o'rganish. Parametrli va parametrsiz funksiyalarga doir dastur tuzish. Fayllar va kataloglar bilan ishlashga doir masalalar yechish. Fayllar bilan ishlovchi metodlardan foydalanib dasturlar tuzish. Pythonda Vorislikdan foydalanish. Vorislik. Maxsus metodlar. Klass xususiyatlari.

2-mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.

PyQt5 paketi va uning imkoniyatlari bilan tanishish. PyQt5 paketining imkoniyatlari. PyQt5 paketini o'rnatish. PyQt5 paketini pip yordamida o'rnatish. QtDesigner dasturini o'rnatish hamda uning imkoniyatlari bilan tanishish. PyQt5 kutubxonasi. QLabel vidjeti. QLabel vidjeti. QLabel shrift, o'lcham va text xususiyatlari. PyQt5 kutubxonasi. QLineEdit vidjeti. QLineEdit vidjeti. `setStyleSheet()` metodi. PyQt5 kutubxonasidan foydalanib turli qiyinchilikdagi masalalarning dasturlarini tuzish. Turli vidgetlardan foydalanib oddiy arifmetik dasturlar tuzish. Turli vidgetlardan foydalanib ro'yxatlarga doir dasturlar tuzish. PyQt5 modal dialog. QMessageBox vidjeti bilan ishlash. QMessageBox vidjetining vazifasi. QMessageBox vidjetining asosiy xususiyatlari. Statik funksiyalari. Piktogramma va QPixmap xususiyatlari. PyQt da rasmlar va menyular. PyQt paketi yordamida rasm joylashtirish usullari. Menu yaratish. Menu vidjetining xususiyatlari. Matn muharriri dizaynini yaratish. Yaratiladigan matn muharriri uchun kerakli uskunalarni tanlash. Tanlangan uskunalarni matn muharriri ekraniga joylashtirish. Matn muharririr ekrani dizaynini shakllantirish. Matn muharriri dasturini yozish. PyQt5 da matn muharriri dizaynidagi elementlarning funksiyalari uchun dastur yozish. Dasturni testlash.

3-mavzu: Pythonda tarmoq dasturlashga kirish.

Tarmoqda ma'lumot almashuvchi klient-server dasturini tuzish. Socket modulining asosiy metodlari bilan tanishish. Socket moduli bilan ishlash. Socket modulining asosiy metodlari bilan tanishish. `.socket()`, `.bind()`, `.listen()`, `.accept()`, `.connect()`, `.send()`, `recv()`, `.close()` metodlari bilan ishlash. Pythonda TCP klient-server dasturini tuzish. Socket modulining asosiy metodlari yordamida client-server dasturini tuzish. TCP klient-server dasturini testlash. TCP client-server dasturi yordamida ma'lumot almashish. PyQt5 paketidan foydalanib zamonaviy chat dasturini tuzish. TCP dasturini client qismini GUI ko'rinishda ishlab chiqish. Pythonda GUI paketidan foydalanib chat dasturini tuzishni yakunlash. TCP client-server dasturini GUI ko'rinishda ishlab chiqish. Python ilovasini maxsus paket yordami. Python ilovasini maxsus paket yordamida yuklanuvchi fayl ko'rinishiga keltirish.

4-mavzu: Python yordamida mashinali o'qitish asoslari.

Mashinali o'qitishga kirish. Mashinali o'qitish haqida umumiy tushuncha. Sun'iy intellekt va ML farqi. Jupyter Notebook bilan ishlash.

5-mavzu: Mashinali o'qitishda NumPy va Pandas paketlarining qo'llanilishi.

NumPy va Pandas bilan ishlash. NumPy kutubxonasi asoslari (massivlar bilan ishlash). Pandas kutubxonasi asoslari (ma'lumotlarni qayta ishlash). NumPy va Pandas bilan ishlash. NumPy va pandas paketidan foydalanib berilgan datasetlarni tahrirlash (ma'lumotlarni qayta

ishlash). NumPy va Pandas bilan ishlash. NumPy kutubxonasida amallar bajarish. Pandas yordamida ma'lumotlarni qayta ishlash va tahlil qilish.

6-mavzu: Ma'lumotlar bilan ishlash va vizualizatsiya.

Matplotlib va Seaborn kutubxonalari bilan ishlash. Matplotlib va Seaborn kutubxonalari bilan tanishish. Ma'lumotlarni vizualizatsiya qilish texnikalari. Matplotlib va Seaborn kutubxonalari bilan ishlash. Mashhur datasetlarni vizualizatsiya qilish va natijalarni tahlil qilish. Matplotlib va Seaborn kutubxonalari yordamida ma'lumotlarni vizualizatsiya qilish. Grafiklar, gistogrammlar va scatter plot chizishni o'rganish.

7-mavzu: Mashinali o'qitish asoslari va algoritmlar.

Mashinali o'qitish algoritmlari haqida umumiy tushuncha. Mashinali o'qitish algoritmlari haqida umumiy tushuncha. O'qitiluvchi (Supervised) o'rganish algoritmlari (Linear Regression, Logistic Regression). O'qitilmaydigan (Unsupervised) o'rganish algoritmlari (K-means Clustering). Mashinali o'qitish algoritmlarini real datasetlarda qo'llash. Real datasetlarda algoritmlarni qo'llash va natijalarni taqdim etish. Mashinali o'qitish algoritmlarini amaliyotga tatbiq qilish. Linear va Logistic Regression algoritmlarini tatbiq qilish. K-means Clustering algoritmini tatbiq qilish.

8-mavzu: Mashinali o'qitishda Klassifikatsiya va regressiya masalalari.

Mashinali o'qitishda Klassifikatsiya va regressiya tushunchalari. Klassifikatsiya tushunchasi va asosiy algoritmlar (Decision Trees, Random Forest). Regressiya tushunchasi va algoritmlari (Linear, Polynomial Regression). Mashinali o'qitish algoritmlarini real datasetlarda qo'llash. Amaliy natijalarni solishtirish va guruh taqdimotlari. Scikit-learn paketidan foydalanish. Scikit-learn yordamida neyron tarmoqlar yaratish. Oddiy sinfiylashtirish masalalarida tatbiq qilish. Mashinali o'qitishda Klassifikatsiya va regressiyani qo'llanilishi. Decision Trees va Random Forest algoritmlarini tatbiq qilish. Turli regressiya algoritmlarini tatbiq qilish. Suniy neyron tarmoqlar haqida umumiy tushuncha. Sun'iy neyron tarmoqlari asoslari. Perceptron modeli va tarmoqlarni o'qitish mexanizmi. Kirish darajasidagi neyron tarmoqlari (MLP).

4. Fanni o'qitish bo'yicha tashkiliy – uslubiy ko'rsatmalar.

Fanni o'qitishda ma'ruza, guruhli va amaliy mashg'ulotlar turlari hamda mavzu bo'yicha mustaqil topshiriqlarni o'z ichiga oladi. Ma'ruza, seminar, amali, laboratoriya mashg'ulotlariga oid o'quv materiallarda ko'rsatilgan mavzular bo'yicha nazariy ma'lumotlar beriladi, amaliy va mustaqil ishlarni bajarish va natijalarni hisoblash tartibi tushuntiriladi. Fan bo'yicha qo'yilgan o'quv materiallari kursantlar tomonidan mustaqil o'rganiladi, amaliy ishlar individual tarzda bajariladi.

Ma'ruza fan bo'yicha umumiy nazariy bilimlarni yetkazish, amaliy mashg'ulotlar materiallarini o'zlashtirish uchun kerakli nazariy ma'lumotlar bilan tanishtirish maqsadida o'tkaziladi.

Materialni yetkazish tempini tanlashda, o'qituvchi, kursantlar toifasini, ushbu mavzu (yo'nalish) bo'yicha o'quv, ilmiy, uslubiy adabiyotlar mavjudligi va boshqa omillarni albatta hisobga olishi kerak.

Guruhli mashg'ulotlari ushbu fanning ma'ruzasida olgan nazariy bilimlarini yana ham mustahkamlashda, python dasturlash tilining asosiy komponenta va operatorlaridan mustaqil foydalanish ko'nikmasini rivojlantirish, PyQt5 paketi va Mashinali o'qitishga doir bilimlarini rivojlantirish maqsadida o'tkaziladi hamda ularni amalda qo'llash, mavjud kodlarni takomillashtirish bo'yicha kursantlarni o'qitish asosini tashkil qiladi. Guruhli mashg'ulotlar maxsus sinflarda, python dasturlash tili uchun kerakli dasturiy ta'minotlar va pythonning sun'iy intellekt bilan ishlovchi paketlari bilan jihozlangan kompyuterlari mavjud sinf xonalarda o'tkaziladi.

Amaliy mashg'ulotlar kompyuter bilan ishlash, axborot tarmoqlarini kiberjinoyatchilardan saqlash va ularni fosh etish usullarini qo'llash bo'yicha kursantlarda amaliy ko'nikmalarni shakllantirish maqsadida o'tkaziladi.

Fanini o'qitish davomida kursantlarni mustaqil va erkin fikr yuritishga, mantiqiy va algoritmik fikrlashlarini hamda, nutq mahoratini oshirishga, u yoki bu muammoga nisbatan o'z nuqtai nazarini aniq va ravshan ifoda etishga chorlaydigan innovatsion pedagogik texnologiyalardan hamda “Bumerang”, “Zinama-zina”, “Fikrlar hujumi” (aqliyl hujum), “Charxpalak”, “3x4”, “Muammo”, “Labirint”, “Blis-so'rov”, “Skorobey”, “Interfaol suhbat”, “T-sxema”, “Klasster”, “FSMU”, “VEN-diagramma”, SWOT-tahlil” va boshqa interfaol metodlardan foydalaniladi.

5. Mustaqil ta'lim va mustaqil ishlar.

T/r	Mustaqil ta'lim mavzusi	Mazmuni va shakli
1	Pythonda chiziqli va tarmoqlanuvchi algoritmlarga doir masalalarni dasturini tuzish.	Amaliy bajaradi. Hisobot tayyorlaydi.
2	Pythonda ro'yxatlarga doir dastur tuzish.	Amaliy bajaradi. Hisobot tayyorlaydi.
3	Pythonda funksiyalarga doir dastur tuzish.	Amaliy bajaradi. Hisobot tayyorlaydi.
4	Pythonda fayllar bilan ishlash.	Amaliy bajaradi. Hisobot tayyorlaydi.

6. Asosiy va qo'shimcha o'quv adabiyotlar hamda axborot manbaalari

Asosiy adabiyotlar:

1. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. “Python dasturlash tili” Darslik. Toshkent: 2024y. B - 316.

2. Sh.R. Sapayev “Python dasturlash tili asoslari”. O'quv qo'llanma. Toshkent: 2023y. B – 137.

3. Sh.R. Sapayev “PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish”. O'quv qo'llanma. Toshkent: 2024y. B - 150

Qo'shimcha adabiyotlar:

1. Бхаргава А. Грокаем алгоритмы. Иллюстрированное пособие для программистов и любопытствующих.-СПб.: Питер, 2017.-288 с. : ил. ISBN 978-5-496-02541-6

2. Н.А.Прохоренко, В.А.Дронов. “Python3 и PyQt5. Разработка приложений”. СПб.: БХВ-Петербург, 2016. – 832 с.: ил.

3. Франсуа Шолле. “Глубокое обучение на Python”. — СПб.: Питер, 2018. — 400 с.: ил. — (Серия «Библиотека программиста»).

4. Чан, Уэсли. “Python: создание приложений. Библиотека профессионала”, 3-е изд. [Пер. с англ. - М. : ООО "И.Д. Вильямс"], Москва: Санкт-Петербург • Киев 2015.

5. Марк Саммерфилд. “Программирование на Python 3. Подробное руководство” [Пер. с англ. – СПб]. - Москва: Санкт-Петербург–2009 год.

Internet saytlari:

1. <https://www.python.org>
2. <https://python-scripts.com>
3. <https://webformymself.com/python>

**O'ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN'IY INTELLEKT”
KAFEDRASI**



“SUN'IY INTELLEKT ASOSLARI”

FANNING ISHCHI O'QUV DASTURI

Toshkent 2025 y.

1

SIA

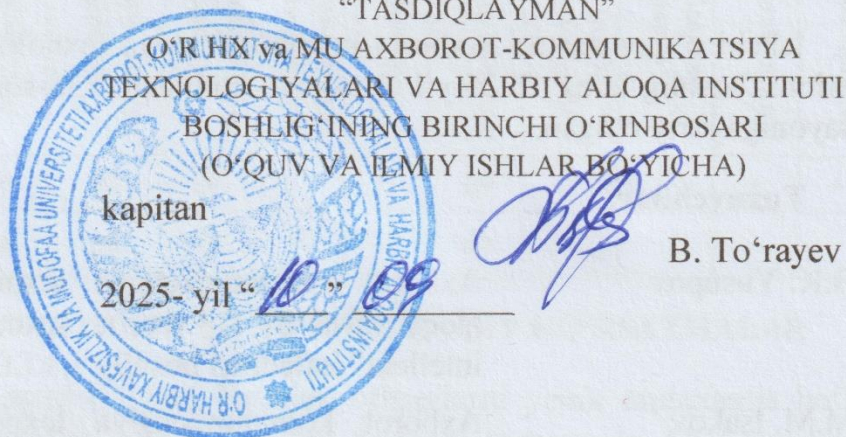
4-kurs

IT

O'ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING

AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY ALOQA
INSTITUTI

“TASDIQLAYMAN”



Axborot texnologiyalari va sun'iy intellekt kafedrası

SUN'IY INTELLEKT ASOSLARI fanining

ISHCHI O'QUV DASTURI

Bilim sohasi:	1 000 000	– Xizmatlar
Ta'lim sohasi:	1 030 000	– Xavfsizlik xizmati
Ta'lim yo'nalishlari:	6 1030 700	– Qo'shinlarning taktik qo'mondonlik muhandisligi (Axborot tizimlari va texnologiyalari)

Toshkent – 2025 y.



Fanning ishchi o'quv dasturi O'zbekiston Respublikasi Harbiy xavfsizlik va mudofaa universiteti boshlig'ining birinchi o'rinbosari tomonidan 2025-yil 25-avgust kuni tasdiqlangan o'quv dasturi asosida tayyorlangan.

Ishchi o'quv dasturi Axborot-kommunikatsiya texnologiyalari va harbiy aloqa instituti ilmiy-uslubiy kengashining 2025-yil 30-iyul kunidagi 12-sonli bayoni bilan tasdiqlangan.

Ishchi o'quv dasturi Axborot-kommunikatsiya texnologiyalari va harbiy aloqa instituti boshlig'ining 2025-yil 1-avgust kunidagi 238-son buyrug'i bilan ta'lim jarayoniga joriy etilgan.

Tuzuvchilar:

B.K. Yusupov	Axborot kommunikatsiya texnologiyalari va harbiy aloqa instituti "Axborot texnologiyalari va sun'iy intellekt" kafedrası boshlig'i, t.f.f.d., professor.
M.M. Isakov	Axborot kommunikatsiya texnologiyalari va harbiy aloqa instituti "Axborot texnologiyalari va sun'iy intellekt" kafedrası "Sun'iy intellekt" sikli boshlig'i
R.O'. Qodomboyev	Axborot kommunikatsiya texnologiyalari va harbiy aloqa instituti "Axborot texnologiyalari va sun'iy intellekt" kafedrası "Sun'iy intellekt" sikli katta o'qituvchisi

Taqrizchilar:

podpolkovnik A. Mulladjanov	O'R QK Bosh shtabi AAT va AH BB Axborot-kommunikatsiya texnologiyalarini rivojlantirish boshqarmasi boshlig'i;
podpolkovnik O. Abdiroziqov	O'R HX va MU "Istiqbolli harbiy texnologiyalar" kafedrası boshlig'i, PhD, dotsent.

O'R HX va MU AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI
VA HARBIY ALOQA INSTITUTI O'QUV BO'LIMI BOSHIG'I
mayor

N. Kuzibekov

AXBOROT TEXNOLOGIYALARI VA SUN'IY
INTELLEKT KAFEDRASİ BOSHIG'I
kapitan

B. Yusupov

I. O'QUV VAQTINI SEMESTRLAR VA O'QUV MASHG'ULOT TURLARI BO'YICHA TAQSIMLANISHI

Fanni o‘qitish muddati:	Kursantning o‘quv yuklamasi (soatlarda)										Nazorat usuli	
	Umumiy yuklamaning hajmi	Auditoriya mashg‘ulotlari (soatlarda)								Mustaqil tayyorgarlik		
		Jami	Ma’ ruzalar	Guruhli mashg‘ ulotlar	Amaliy mashg‘ ulotlar	Laboratoriya mashg‘ ulotlari	Seminarlar	Mustaqil ta’ lim	Kurs loyihasi (ishi)			
7	150	90	16	50	24			60			+	+
Jami soatlar	150	90	16	50	24			60				

II. O'QUV FANNI O'QITILISH BO'YICHA USLUBIY KO'RSATMALAR

“Sun'iy intellekt asoslari” fani bo'lajak ofitserlarni yetuk mutaxassis bo'lib chiqishida muhim ahamiyatga ega. Ushbu fan bugungi zamonaviy AKT vositalari rivojlangan davrda juda ham **dolzarb** ahamiyatga ega hisoblanadi.

“Sun'iy intellekt asoslari” fanini o'tish va mustaqil o'qish jarayonida kursantlar bilan quyidagi maqsadlarga erishiladi:

Bilim berish: kursantlarda Python dasturlash tili yordamida biror bir dastur yaratish uchun kerak bo'ladigan asosiy elementlari bo'lmish funksiyalar, massivlar, satrlar, fayllar bilan ishlash, ular bilan ishlovchi funksiya va metodlar bilan ishlash; shuningdek, pythonning imkoniyatlarini oshiruvchi mashinali o'qitishda zarur bo'ladigan, ma'lumotlarni tahlil qilishda foydalaniladigan turli modullari - PyQt5, numpy, pandas, matplotlib, scikit-learning kabi kutubxonalaridan foydalanishga oid ko'nikmalarni shakllantirish; ijodiy ravishda mustaqil bilim olish ko'nikma va malakalarni hosil qilish; ularni O'zbekiston Respublikasi Qurolli Kuchlarida jangovar tayyorgarlikni mustahkamlash va harbiy maqsadlarga yo'naltirilgan dasturlash hamda AKT vositalarining dasturiy vositalaridan samarali foydalanishga yo'naltirish.

Tarbiyalash: ofitserlar faoliyatiga xos bo'lgan kasbiy va psixologik sifatlarni shakllantirish bilan birgalikda kompyuter va Axborot texnologiyalari vositalarining imkoniyatlaridan harbiy maqsadlarda foydalanishga o'rgatish.

Amaliy ko'nikma va malakalar hosil qilish: qo'yilgan topshiriqlarni bajara olish va bajarish usullarini tanlash; berilgan talabga mos dasturlar ishlab chiqish; dasturda xatoliklarni bartaraf etish va boshqarish.

Fanni o'zlashtirish kursantlarning “Dasturlash asoslari”, “Algoritmash” “Ma'lumotlar bazasini boshqarish tizimlari”, “Dasturlash texnologiyalari” fanlarida olgan bilimlariga tayanadi. Fanni o'zlashtirish quyidagi mashg'ulot turlarini o'z ichiga oladi: ma'ruza va amaliy mashg'ulotlar, shuningdek kursantlarga mustaqil tayyorgarlik vaqtida maslahatlar berish. Ma'ruza materiallari bayoni mustaqil va tugallangan hususiyatga ega bo'lib, avval bayon qilingan materiallarga mantiqiy bog'langan hamda boshqa fanlarda, hamda amaliyotda qo'llanishga yo'naltirilgan bo'lishi kerak. Amaliy

mashg'ulotlarda kursantlar olgan nazariy bilimlarini qo'llay olishni o'rganishlari kerak. Kursantlarning bilimlari reyting nazorat tizimida baholanadi. Kursantlar bilimlarini reyting nazoratida baholash quyidagi tartibda amalga oshiriladi:

- joriy nazorat: kursantlarni mashg'ulotlarda muntazam so'roq qilish;
- oraliq nazorat;
- yakuniy nazorat.

Fanning harbiy yo'nalishi aloqa qo'shinida mavjud va mutaxassislarning keyingi professional faoliyatiga taalluqli aniq texnika baza namunalari tuzilishini va ekspluatatsiya qilishda amaliy bilimlar olish bilan ta'minlanadi.

Mashg'ulotning asosiy ko'rinishi ma'ruza mashg'ulotlari va amaliy mashg'ulotlar hisoblanadi.

Ma'ruza mashg'ulotlari, odatda, bir necha o'quv guruhini o'z ichiga olgan, 100 nafardan ortiq bo'lmagan kursantlarning oqimi (potok) bilan o'tkaziladi. Ma'ruza kafedra boshlig'i va professor-o'qituvchilar tomonidan o'qiladi. Ma'ruzani o'qishga tajribali o'qituvchilar ham ruxsat etiladi. Ma'ruzani o'qish uslubi ma'ruzachi tomonidan aniqlanadi, lekin bunda mashg'ulotda o'rganuvchilarning faolligini oshirish usullariga ko'proq e'tibor qaratiladi:

- muammoli savollarni o'rta qo'yish;
- ma'ruzani munozara usulida muloqat formasida harbiy tajribaga va o'rganilayotgan texnika namunalari jangovar qo'llash hamda amaliy ekspluatatsiya qilishga tayangan holda o'qitish.

Ma'ruzaning materiallari doimo yangilanib borish lozim. Ma'ruza o'rganilayotgan fan bo'yicha ilmiy bilimlar asosini berishi, o'quv materiallarining o'ta murakkab savoli dialektik o'zaro bog'liqligini, kursantlarning ijodiy fikrlashini rivojlanishiga ko'maklashuvi, zamonaviy ilm va texnika yutuqlarini, nazariya va amaliyotning dolzarb savollarini yoritib berishi va boshqa mashg'ulotlar turlarining hamda kursantlarning mustaqil tayyorgarligini tashkil qilish va o'tkazishning asosi hisoblanadi.

Ma'ruza mashg'ulotlarining faol shakllari:

- muammoli ma'ruza;
- ma'ruza – provokatsiya (chalg'ituvchi ma'ruza);
- ma'ruza – konsultatsiya;
- ma'ruza – suhbat;
- qarshi aloqa texnikasidan foydalanuvchi ma'ruza.

Har bir ma'ruza o'z ichiga kirish, asosiy va yakuniy qismni oladi.

Kirish qismida: mavzuning nomi, ma'ruza mavzusining asosiy g'oyasi va muhimligi; o'quv maqsadlar; ma'ruzaning o'quv savollari; oldingi va keyingi mashg'ulotlar bilan bog'liqligi.

Ma'ruzaning asosiy qismida o'quv savollarining mazmuni yetkaziladi. Ma'ruzaning har bir nazariy jihati eng maqsadga muvofiq usullarni qo'llagan holda asoslangan va isbotlangan bo'lishi kerak. Ma'ruzaning asosiy qismini bayon qilishda ta'lim oluvchilarga ilmiy g'oyalarni rivojlanishi, jamlanishi, mavhumlikdan aniqlikka o'tishining mantig'ini yoritib berishga imkon beruvchi dalillarga tayanish ma'ruzaga

bo'lgan majburiy talab hisoblanadi. Har bir ma'ruzaning asosiy qismini mazmuni fundamental bo'lishi kerak.

Amaliy maqsadlarga yo'naltirilgan ma'ruzalarda kasbga oid va o'quv vazifalarni hal etish bo'yicha amaliy tavsiyalarni ko'zda tutish maqsadga muvofiq bo'ladi.

Har bir o'quv savoli, uni, keyingi o'quv savoliga mantiqiy olib keluvchi, rivojlanish istiqbollarning nazariyasi va amaliyoti hamda qisqacha xulosasini yoritish bilan tugatilishi kerak.

Ma'ruzaning yakuniy qismida, nazariya va amaliyotni qo'llash soha va chegaralarini ko'rsatgan holda, asosiy qism mazmuni umumlashtiriladi va qisqacha xulosa qilinadi, mustaqil o'rganish hamda kelgusi seminar va boshqa turdagi mashg'ulotlarda muhokama qilish uchun savollar va vazifalar belgilanadi.

Ma'ruzani o'qishda kino va videofilmlar, chizmalar, plakatlari, modellar, asboblari va maketlarni namoyish qilgan holda o'quv materiallarining og'zaki yetkazilishi o'qitishning yetakchi uslubi hisoblanadi.

Materialni yetkazish tempini tanlashda, o'qituvchi, kursantlari toifasini, ushbu mavzu (yo'nalish) bo'yicha o'quv, ilmiy, uslubiy adabiyotlari mavjudligi va boshqa omillarni albatta hisobga olishi kerak.

Guruhli mashg'ulotlari ushbu fanning ma'ruzasida olgan nazariy bilimlarini yana ham mustahkamlashda, python dasturlash tilining asosiy komponenta va operatorlaridan mustaqil foydalanish ko'nikmasini rivojlantirish, PyQt5 paketi va Mashinali o'qitishga doir bilimlarini rivojlantirish maqsadida o'tkaziladi hamda ularni amalda qo'llash, mavjud kodlarni takomillashtirish bo'yicha kursantlarni o'qitish asosini tashkil qiladi. Guruhli mashg'ulotlari maxsus sinflarda, python dasturlash tili uchun kerakli dasturiy ta'minotlari va pythonning sun'iy intellekt bilan ishlovchi paketlari bilan jihozlangan kompyuterlari mavjud sinf xonalarda o'tkaziladi.

Guruhli mashg'ulotlarning boshqa turdagi o'quv mashg'ulotlaridan ajratib turadigan jihati - bu ularda o'rganiladigan suni'y intellekt asoslari, python dasturlash tili asoslari, PyQt5 paketidan foydalanish, pythonda mashinali o'qitish va sun'iy intellektga doir kerakli paketlardan qanday foydalanish, ularni qo'llash, ishlatish, xizmat ko'rsatish va ta'mirlashni o'rgatish uchun ko'p sonli xilma-xil o'quv qurol va qo'llanmalardan foydalanish hisoblanadi.

Individual va kollektiv yondashish yo'li bilan o'qituvchi suhbat orqali ma'ruzaning o'z ichiga olgan muammolari savollarning yechimini topadi.

O'rganilayotgan o'quv materiallarini faollashtirish uchun «Nima uchun bunday qilingan», «Qanchalik bu qulay (ma'qullik, maqsadga muvofiq)», bunda o'rganuvchilar orasida seminar mashg'ulot xususiyatga ega bo'lgan fikrlarni almashuv va metodik usullarni kiritish foydalidir.

Amaliy mashg'ulot o'tkazish maqsadida kursantlari zamonaviy kompyuterlarda Python dasturlash tilida dastur yaratishadi va dasturlarni tahlilini o'rganishadi.

Amaliy mashg'ulotlari zamonaviy kompyuterlari va multimedia vositalari bilan jihozlangan maxsus o'quv sinf xonalarida o'tkaziladi. Nazariy tajribani va amaliyotni o'tash mobaynida o'z qobiliyatini hamda ko'nikmalarini takomillashtiradi.

Mashg'ulotlarni individuallashtirish va o'qitishni sifatini oshirish maqsadida vositalarning soniga qarab guruhlar bir qancha guruhlariga bo'linadi va ular o'quv joylariga taqsimlanadi.

Amaliy mashg'ulotlarda kursantlar me'yorlarni bajarishda ishtirok etishi maqsadida bellashuv, musobaqa va sog'lom raqobat elementlarini kiritish lozim.

Harbiy tajribani va amaliyotni o'tash mobaynida o'z qobiliyat hamda ko'nikmalarni takomillashtiriladi.

O'quv-tarbiyaviy jarayonini jadallashtirishga qo'yilgan talablar oshishini inobatga olib mashg'ulotlarni tashkil etish va o'tkazish uslubiyatini doimo takomillashtirish lozim.

Mustaqil o'qish soatida kursantlar tavsiya etilgan adabiyotlarni o'rganib, konspektlarini to'ldirib, olgan bilimlarini mustahkamlaydi.

Guruh va yakka tartibdagi konsultatsiyalar kursantlarga o'qituvchilar tomonidan amaliy mashg'ulotlarga va imtihonlarga yordam ko'rsatish maqsadida o'tkaziladi.

Kursantlarni bilimni aniqlash joriy va yakuniy nazoratlar baholari orqali amalga oshiriladi. Joriy nazorat o'tkazish, kursantlar o'quv materiallarni sifatli o'zlashtirishlarini to'liq tekshirish va ularni ishini rag'batlantirish uchun amalga oshiriladi. U amaliy mashg'ulotlar davomida o'tkaziladi.

Kursantlarning bilimlarini tekshirish to'rt balli tizimida amalga oshiriladi. Kursantlar bilim darajasining nazorati quyidagi ko'rinishda amalga oshiriladi:

Joriy nazorat – mashg'ulot jarayonida doimiy tizimli ravishda **savol-javob, test, amaliy ish usullari bilan olib boriladi.**

Yakuniy nazorat kursantlarni nazariy bilimlari va amaliy tayyorgarligi darajasini tekshirish maqsadida o'tkaziladi. Sinov va imtihon orqali amalga oshiriladi.

Fan bo'yicha kursantlarning bilim, ko'nikma va malakalariga quyidagi talablar qo'yiladi. Kursant:

Quyidagi tasavvurga ega bo'lishi lozim:

- Python dasturlash tilining tuzilmasi, funksiyalari va asosiy parametrlari;
- PyQt5 paketi imkoniyatlari va vidgetlar bilan ishlash;
- Tarmoqda dasturlash asoslari;
- Mashinali o'qitishning asosiy tushunchalari;
- Sun'iy intellekt nazariy tushunchalari;
- NumPy va Pandas paketlari;
- Scikit-learn paketi imkoniyatlari.

Quyidagi bilishi va ulardan foydalana (qo'llay) olishi:

- Qo'yilgan masalaga mos algoritmlarni tanlash;
- PyQt5 paketi vidgetlaridan foydalanib turli dasturlar ishlab chiqish;
- Dasturdagi xatoliklarni aniqlash, bartaraf etish va boshqarish;
- Grafik foydalanuvchi interfeysini shakllantirish va boshqarish;
- Mashinali o'qitish metodlari bilan amaliy ishlash;
- NumPy va Pandas paketlari yordamida ma'lumotlarni qayta ishlash;
- Scikit-learn paketi imkoniyatlari bilan tanishish va ishlash.

Quyidagi bilim va ko'nikmalarga ega bo'lishi kerak:

- Python dasturlash tilining sodda va murakkab tuzilmalarini qo'llash;
- Algoritmni baholash, qo'yilgan masalani yechish algoritmini tanlash, tanlovni asoslash va tadbiq etish;
- Obyektga mo'ljallangan dasturlash texnologiyalaridan foydalanish;
- Tarmoqda dasturlash asoslarini amaliyotda qo'llash va rivojlantirish;
- Mashinali o'qitish haqida asosiy bilimlarni egallash va ularni hayotiy faoliyatda qo'llay olish.

III. FANNI O'QUV MASHG'ULOT TURLARI BO'YICHA O'QITISH REJASI

T.r.	O'quv mashg'ulot raqami va turlari	Soatlar hajmi	Mashg'ulot mavzusi va o'quv savollari	Mashg'ulotning moddiy jihatdan ta'minlanishi
7- semestr				
1.	Ma'ruza	2	1-Mavzu: "Sun'iy intellekt asoslari" faniga kirish va asosiy tushunchalari. 1-mashg'ulot. Python dasturlash tilining klassifikatsiyasi va rivojlanish tarixi. Python dasturlash tilining asosiy tushunchalari. O'quv savollari: 1. Fanning mazmuni, maqsadi, vazifalari; 2. Pythonni o'rnatish. PyCharm dasturini o'rnatish; 3. Pythonda "Hello world!" dasturini tuzish. 4. Python tilining asosiy operatorlari bilan tanishish	Kompyuter. Interaktiv panel. Taqqimot materiallari.
2.	Guruhli	2	1-Mavzu: "Sun'iy intellekt asoslari" faniga kirish va asosiy tushunchalari. 2-mashg'ulot. Pythonda arifmetik operatorlar. O'quv savollari: 1. Arifmetik operatorlar. 2. Sonlar bilan ishlash. O'zgaruvchilar. 3. Bool tipli ma'lumotlar bilan ishlash.	Kompyuter. Interaktiv panel. Taqqimot materiallari.
3.	Guruhli	2	1-Mavzu: "Sun'iy intellekt asoslari" faniga kirish va asosiy tushunchalari. 3-mashg'ulot. Satrlar bilan ishlovchi operatorlar va metodlar. O'quv savollari: 1. Satrlar bilan ishlovchi operatorlar va metodlar haqida umumiy ma'lumot. 2. str.format() metodi yordamida satrlarni formatlash. 3. Satrlarni kodlash va dekodlash (encode() va decode()) metodlari yordamida baytlar bilan ishlash.	Kompyuter. Interaktiv panel. Taqqimot materiallari.

4.	Guruhli	2	1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari. 4-mashg’ulot. Pythonda shart operatori uning qo’llanilishi. O’quv savollari: 1. IF, IF-ELSE va IF-ELIF-ELSE operatorlari. 2. Shart operatorlarida mantiqiy ifodalar (and, or, not) ning qo’llanilishi. 3. Ichma-ich (nested) IF operatorlaridan foydalanish, ularning afzallik va kamchiliklari.	Kompyuter. Interaktiv panel. Taqqimot materallari.
5.	Amaliy	2	1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari. 5-mashg’ulot. Pythonda shart operatori, satrlarga doir masalalar yechish. Shart operatoriga doir dasturlari tuzish. O’quv savollari: 1. IF, IF-ELSE va IF-ELIF-ELSE operatorlariga doir dasturlar tuzish. 2. Pythonda satrlarga doir masalalar yechish	Kompyuter. Interaktiv panel. Taqqimot materallari.
6.	Guruhli	2	1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari. 6-mashg’ulot. Pythonda takrorlanuvchi jarayonlarni dasturlash. O’quv savollari: 1. Sikl operatorlari – For va while bilan ishlash. 2. Break, continue operatorlarining qo’llanilishi.	Kompyuter. Interaktiv panel. Taqqimot materallari.
7.	Guruhli	2	1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari. 7-mashg’ulot. Pythonda for,while, break va continue operatorlariga doir dasturlar tuzish. O’quv savollari: 1. Sikl operatorlari – For va while ga doir dasturlar tuzish. 2. Break, continue va else operatorlariga doir dasturlartuzish.	Kompyuter. Interaktiv panel. Taqqimot materallari.
8.	Guruhli	2	1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari. 8-mashg’ulot. Pythonda ro’yxatlar va ular bilan ishlovchi metodlar. O’quv savollari: 1. Ro’yxatlar va ularning qo’llanilishi. 2. Ro’yxatlarni yaratish usullari. 3. Ro’yxatlar bilan ishlovchi metodlar.	Kompyuter. Interaktiv panel. Taqqimot materallari.
9.	Amaliy	2	1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari. 9-mashg’ulot. Pythonda ro’yxatlarga doir masalalar yechish. O’quv savollari: 1. Pythonda ro’yxatlarga doir oddiy masalalar yechish. 2. Ro’yxatlarni ishlovchi metodlarga doir masalalar yechish.	Kompyuter. Interaktiv panel. Taqqimot materallari.

SUN'IY INTELLEKT ASOSLARI

10.	Guruhli	2	1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari. 10-mashg’ulot. Pythonda Funksiya tushunchasi. Foydalanuvchi funksiyasi. O’quv savollari: 1. Funksiyalarni aniqlash va uni chaqirish. 2. Parametrli va parametrsiz funksiya.	Kompyuter. Interaktiv panel. Taqdimot materiallari.
11.	Amaliy	2	1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari. 11-mashg’ulot. Pythonda Funksiyaga doir dasturlar tuzish. O’quv savollari: 1. Funksiyalarni aniqlash va uni chaqirishni o’rganish. 2. Parametrli va parametrsiz funksiyalarga doir dastur tuzish.	Kompyuter. Interaktiv panel. Taqdimot materiallari.
12.	Guruhli	2	1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari. 12-mashg’ulot. Fayllar va kataloglar bilan ishlash. O’quv savollari: 1. Faylni ochish. Fayllar bilan ishlovchi metodlar. 2. os modulining imkoniyatlari. 3. Fayl va katalog yo’lini o’zgartirish. 4. Katalog va fayl bilan ishlovchi funksiya va metodlar.	Kompyuter. Interaktiv panel. Taqdimot materiallari.
13.	Amaliy	2	1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari. 13-mashg’ulot. Fayllar va kataloglar bilan ishlashga doir masalalar yechish. O’quv savollari: 1. Fayllar bilan ishlash metodlaridan foydalanish. 2. Kataloglar ustida amallar bajarish. 3. Fayl va kataloglarda qidirish va nusxa olish.	Kompyuter. Interaktiv panel. Taqdimot materiallari.
14.	Guruhli	2	1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari. 14-mashg’ulot. Pythonda OOP asoslari. O’quv savollari: 1. OOP asoslari. Klasslarni e’lon qilish va nusxasini yaratish. 2. Klass va obyekt. Klass konstruktori. 3. __init__() va __del__() metodlari.	Kompyuter. Interaktiv panel. Taqdimot materiallari.
15.	Amaliy	2	1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari. 15-mashg’ulot. Pythonda Vorislikdan foydalanish. O’quv savollari: 1. Vorislik. 2. Maxsus metodlar. 3. Klass xususiyatlari.	Kompyuter. Interaktiv panel. Taqdimot materiallari.

16.	Ma'ruza	2	2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish. 1-mashg'ulot. PyQt5 paketi va uning imkoniyatlari bilan tanishish. O'quv savollari: 1. PyQt5 paketining imkoniyatlari. PyQt5 paketini o'rnatish. 2. PyQt5 paketini pip yordamida o'rnatish. 3. QtDesigner dasturini o'rnatish hamda uning imkoniyatlari bilan tanishish.	Kompyuter. Interaktiv panel. Taqqimot materallari.
17.	Guruhli	2	2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish. 2-mashg'ulot. PyQt5 kutubxonasi. QLabel vidjeti. O'quv savollari: 1. QLabel vidjeti; 2. QLabel shrift, o'lcham va text xususiyatlari.	Kompyuter. Interaktiv panel. Taqqimot materallari.
18.	Guruhli	2	2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish. 3-mashg'ulot. PyQt5 kutubxonasi. QLineEdit vidjeti. O'quv savollari: 1. QLineEdit vidjeti; 2. setStyleSheet() metodi.	Kompyuter. Interaktiv panel. Taqqimot materallari.
19.	Guruhli	2	2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish. 4-mashg'ulot. PyQt5 kutubxonasidan foydalanib turli qiyinchilikdagi masalalarning dasturlarini tuzish. O'quv savollari: 1. Turli vidjetlardan foydalanib oddiy arifmetik dasturlar tuzish; 2. Turli vidjetlardan foydalanib ro'yxatlarga doir dasturlar tuzish.	Kompyuter. Interaktiv panel. Taqqimot materallari.
20.	Guruhli	2	2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish. 5-mashg'ulot. PyQt5 modal dialog. QMessageBox vidjeti bilan ishlash. O'quv savollari: 1. QMessageBox vidjetining vazifasi. 2. QMessageBox vidjetining asosiy xususiyatlari. 3. Statik funksiyalari. 4. Piktogramma va QPixmap xususiyatlari.	Kompyuter. Interaktiv panel. Taqqimot materallari.
21.	Guruhli	2	2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish. 6-mashg'ulot. PyQt da rasmlar va menyular. O'quv savollari: 1. PyQt paketi yordamida rasm joylashtirish usullari. 2. Menu yaratish. Menu vidjetining xususiyatlari.	Kompyuter. Interaktiv panel. Taqqimot materallari.

22.	Guruhli	2	2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish. 7-mashg'ulot. Matn muharriri dizaynini yaratish. O'quv savollari: 1. Yaratiladigan matn muharriri uchun kerakli uskunalarni tanlash. 2. Tanlangan uskunalarni matn muharriri ekraniga joylashtirish. 3. Matn muharririr ekrani dizaynini shakllantirish.	Kompyuter. Interaktiv panel. Taqqimot materiallari.
23.	Amaliy	2	2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish. 8-mashg'ulot. Matn muharriri dasturini yozish. O'quv savollari: 1. PyQt5 da matn muharriri dizaynidagi elementlarning funksiyalari uchun dastur yozish. 2. Dasturni testlash.	Kompyuter. Interaktiv panel. Taqqimot materiallari.
24.	Ma'ruza	2	3-Mavzu: Pythonda tarmoq dasturlashga kirish. 1-mashg'ulot. Tarmoqda ma'lumot almashuvchi klient-server dasturini tuzish. O'quv savollari: 1. Socket modulining asosiy metodlari bilan tanishish. 2. TCP protokoli asosida klient-server ulanishini tashkil qilish. 3. Ma'lumotlarni uzatish va qabul qilish jarayonini amalda bajarish. 4. Klient va server dasturlarida xatoliklarni aniqlash va bartaraf etish.	Kompyuter. Interaktiv panel. Taqqimot materiallari.
25.	Guruhli	2	3-Mavzu: Pythonda tarmoq dasturlashga kirish. 2-mashg'ulot. Socket moduli bilan ishlash. O'quv savollari: 1. Socket modulining asosiy metodlari bilan tanishish. 2. Server yaratishda <code>.socket()</code>, <code>.bind()</code>, <code>.listen()</code>, <code>.accept()</code> metodlaridan foydalanish. 3. Klient yaratishda <code>.connect()</code>, <code>.send()</code>, <code>.recv()</code>, <code>.close()</code> metodlaridan foydalanish. 4. Klient va server o'rtasida ma'lumot almashishni amalga oshirish.	Kompyuter. Interaktiv panel. Taqqimot materiallari.
26.	Guruhli	2	3-Mavzu: Pythonda tarmoq dasturlashga kirish. 3-mashg'ulot. Pythonda TCP klient-server dasturini tuzish. O'quv savollari: 1. Socket modulining asosiy metodlari yordamida client-server dasturini tuzish. 2. TCP ulanishini o'rnatish va xabar almashishni sinash (connect/send/recv). 3. Xatoliklarni qayta ishlash va ulanish holatini boshqarish (timeout/reconnect/close).	Kompyuter. Interaktiv panel. Taqqimot materiallari.

27.	Guruhli	2	3-Mavzu: Pythonda tarmoq dasturlashga kirish. 4-mashg'ulot. TCP klient-server dasturini testlash. O'quv savollari: 1. TCP client-server dasturi yordamida ma'lumot almashish. 2. TCP ulanishini o'rnatish va testlash. 3. TCP client va serverdagi xatoliklarni aniqlash va bartaraf etish.	Kompyuter. Interaktiv panel. Taqqimot materiallari.
28.	Guruhli	2	3-Mavzu: Pythonda tarmoq dasturlashga kirish. 5-mashg'ulot. PyQt5 paketidan foydalanib zamonaviy chat dasturini tuzish. O'quv savollari: 1. PyQt5 orqali TCP client uchun GUI yaratish. 2. Socket ulanishini boshqarish (connect/send/receive) va xabar almashish. 3. Signal-slot mexanizmi va xatoliklarni GUI'da ko'rsatish.	Kompyuter. Interaktiv panel. Taqqimot materiallari.
29.	Guruhli	2	3-Mavzu: Pythonda tarmoq dasturlashga kirish. 6-mashg'ulot. Pythonda GUI paketidan foydalanib chat dasturini tuzishni yakunlash O'quv savollari: 1. TCP client-server dasturini GUI ko'rinishda ishlab chiqish. 2. Xabar almashish oqimini (connect/send/receive) tekshirish va sinovdan o'tkazish. 3. Xatoliklarni qayta ishlash, holatlarni (status/reconnect) GUI'da boshqarish.	Kompyuter. Interaktiv panel. Taqqimot materiallari.
30.	Amaliy	2	3-Mavzu: Pythonda tarmoq dasturlashga kirish. 7-mashg'ulot. Python ilovasini maxsus paket yordamida mustaqil yuklanuvchi fayl ko'rinishiga keltirish O'quv savollari: 1. Python ilovasini maxsus paket yordamida yuklanuvchi fayl ko'rinishiga keltirish. 2. PyInstaller, cx_Freeze kabi paketlarning imkoniyatlari bilan tanishish. 3. Dasturiy ta'minotni turli operatsion tizimlarda sinovdan o'tkazish. 4. Yaratilgan yuklanuvchi faylni foydalanuvchi uchun qulay shaklda taqqim etish.	Kompyuter. Interaktiv panel. Taqqimot materiallari.
31.	Ma'ruza	2	4-Mavzu: Python yordamida mashinali o'qitish asoslari 1-mashg'ulot. Mashinali o'qitishga kirish. O'quv savollari: 1. Mashinali o'qitish haqida umumiy tushuncha 2. Sun'iy intellekt va ML farqi 3. Jupyter Notebook bilan ishlash	Kompyuter. Interaktiv panel. Taqqimot materiallari.
32.	Ma'ruza	2	5-Mavzu: Mashinali o'qitishda NumPy va Pandas paketlarining qo'llanilishi.	Kompyuter. Interaktiv panel.

			1-mashg'ulot. NumPy va Pandas bilan ma'lumotlarni qayta ishlash. O'quv savollari: 1. NumPy kutubxonasi asoslari (massivlar bilan ishlash) 2. Pandas kutubxonasi asoslari (ma'lumotlarni qayta ishlash)	Taqdimot materiallari.
33.	Amaliy	2	5-Mavzu: Mashinali o'qitishda NumPy va Pandas paketlarining qo'llanilishi. 2-mashg'ulot. NumPy va Pandas bilan ma'lumotlarni qayta ishlash. O'quv savollari: 1. NumPy kutubxonasida amallar bajarish 2. Pandas yordamida ma'lumotlarni qayta ishlash va tahlil qilish.	Kompyuter. Interaktiv panel. Taqdimot materiallari.
34.	Guruhli	2	5-Mavzu: Mashinali o'qitishda NumPy va Pandas paketlarining qo'llanilishi. 3-mashg'ulot. NumPy va Pandas bilan ma'lumotlarni qayta ishlash. O'quv savollari: 1. NumPy va Pandas paketidan foydalanib datasetlarni tahrirlash. 2. NumPy yordamida massivlarda amallar bajarish. 3. Pandas yordamida jadval ko'rinishidagi ma'lumotlarni filtrlash va tahlil qilish.	Kompyuter. Interaktiv panel. Taqdimot materiallari.
35.	Ma'ruza	2	6-Mavzu: Ma'lumotlar bilan ishlash va vizualizatsiya. 1-mashg'ulot. Matplotlib va Seaborn kutubxonalari bilan ishlash. O'quv savollari: 1. Matplotlib va Seaborn kutubxonalari bilan tanishish. 2. Ma'lumotlarni vizualizatsiya qilish texnikalari.	Kompyuter. Interaktiv panel. Taqdimot materiallari.
36.	Amaliy	2	6-Mavzu: Ma'lumotlar bilan ishlash va vizualizatsiya. 2-mashg'ulot. Matplotlib va Seaborn kutubxonalari yordamida ma'lumotlarni vizualizatsiya qilish. O'quv savollari: 1. Grafiklar, gistogrammalar va scatter plot chizishni o'rganish. 2. Seaborn yordamida statistik vizualizatsiyalar yaratish. 3. Grafiklarni sozlash (rang, uslub, o'q nomlari, sarlavha) va ularni saqlash.	Kompyuter. Interaktiv panel. Taqdimot materiallari.
37.	Guruhli	2	6-Mavzu: Ma'lumotlar bilan ishlash va vizualizatsiya. 3-mashg'ulot. Matplotlib va Seaborn kutubxonalari bilan ishlash. O'quv savollari:	Kompyuter. Interaktiv panel. Taqdimot materiallari.

			1. Mashhur datasetlarni vizualizatsiya qilish va natijalarni tahlil qilish. 2. Matplotlib yordamida chiziqli grafik, gistogramma va scatter plotlar yaratish. 3. Seaborn yordamida statistik ma'lumotlarni ko'rsatish va ularni solishtirish.	
38.	Ma'ruza	2	7-Mavzu: Mashinali o'qitish asoslari va algoritmlar. 1-mashg'ulot. Mashinali o'qitish algoritmlari haqida umumiy tushuncha. O'quv savollari: 1. Mashinali o'qitish algoritmlari haqida umumiy tushuncha 2. O'qitiluvchi (Supervised) o'rganish algoritmlari (Linear Regression, Logistic Regression) 3. O'qitilmaydigan (Unsupervised) o'rganish algoritmlari (K-means Clustering).	Kompyuter. Interaktiv panel. Taqdimot materiallari.
39.	Amaliy	2	7-Mavzu: Mashinali o'qitish asoslari va algoritmlar. 2-mashg'ulot. Mashinali o'qitish algoritmlarini amaliyotga tatbiq qilish. O'quv savollari: 1. Linear va Logistic Regression algoritmlarini tatbiq qilish 2. K-means Clustering algoritmini tatbiq qilish.	Kompyuter. Interaktiv panel. Taqdimot materiallari.
40.	Guruhli	2	7-Mavzu: Mashinali o'qitish asoslari va algoritmlar. 3-mashg'ulot. Mashinali o'qitish algoritmlarini real datasetlarda qo'llash. O'quv savollari: 1. Real datasetlarda algoritmlarni qo'llash va natijalarni taqdim etish. 2. Ma'lumotlarni tayyorlash va mashinali o'qitish modellariga uzatish. 3. Modellarining aniqligini baholash va taqqoslash.	Kompyuter. Interaktiv panel. Taqdimot materiallari.
41.	Ma'ruza	2	8-Mavzu: Mashinali o'qitishda Klassifikatsiya va regressiya masalalari. 1-mashg'ulot. Mashinali o'qitishda Klassifikatsiya va regressiya tushunchalari. O'quv savollari: 1. Klassifikatsiya tushunchasi va asosiy algoritmlar (Decision Trees, Random Forest) 2. Regressiya tushunchasi va algoritmlari (Linear, Polynomial Regression).	Kompyuter. Interaktiv panel. Taqdimot materiallari.
42.	Amaliy	2	8-Mavzu: Mashinali o'qitishda Klassifikatsiya va regressiya masalalari. 2-mashg'ulot. Mashinali o'qitishda Klassifikatsiya va regressiyani qo'llanilishi. O'quv savollari:	Kompyuter. Interaktiv panel. Taqdimot materiallari.

SUN'IY INTELLEKT ASOSLARI

			1. Decision Trees va Random Forest algoritmlarini tatbiq qilish 2. Turli regressiya algoritmlarini tatbiq qilish.	
43.	Guruhli	2	8-Mavzu: Mashinali o'qitishda Klassifikatsiya va regressiya masalalari. 3-mashg'ulot. Mashinali o'qitish modellarini baholash va natijalarni taqqoslash O'quv savollari: 1. Amaliy natijalarni solishtirish va guruh taqdimotlari. 2. Modellarning aniqlik, xatolik va samaradorlik ko'rsatkichlarini hisoblash. 3. Turli algoritmlarni bir datasetda sinovdan o'tkazib taqqoslash.	Kompyuter. Interaktiv panel. Taqdimot materiallari.
44.	Amaliy	2	8-Mavzu: Mashinali o'qitishda Klassifikatsiya va regressiya masalalari. 4-mashg'ulot. Sun'iy neyron tarmoqlar haqida umumiy tushuncha. O'quv savollari: 1. Sun'iy neyron tarmoqlari asoslari 2. Perceptron modeli va tarmoqlarni o'qitish mexanizmi 3. Kirish darajasidagi neyron tarmoqlari (MLP).	Kompyuter. Interaktiv panel. Taqdimot materiallari.
45.	Guruhli	2	8-Mavzu: Mashinali o'qitishda Klassifikatsiya va regressiya masalalari. 5-mashg'ulot. Scikit-learn paketidan foydalanish. O'quv savollari: 1. Scikit-learn yordamida neyron tarmoqlar yaratish 2. Oddiy sinfiylashtirish masalalarida tatbiq qilish.	Kompyuter. Interaktiv panel. Taqdimot materiallari.
JAMI:		90 soat		

IV. MUSTAQIL TA'LIM VA MUSTAQIL ISHLAR

T/r	Mustaqil tayyorgarlik mavzulari	Soatlar hajmi
7- semestr		
1.	Pythonda umumiy masalalarga doir dastur tuzish (Natijani Prt.Scrn., kodni yozing, izoh bering va word faylga saqlang).	16
2.	Pythonda fayllar bilan ishlash (Natijani Prt.Scrn., kodni yozing, izoh bering va word faylga saqlang).	14
3.	PyQt5 kutubxonasi bilan ishlash (Natijani Prt.Scrn., kodni yozing, izoh bering va word faylga saqlang).	16
4.	Pythonning Numpy va Pandas kutubxonalaridan foydalanib dasturlar tuzish (Natijani Prt.Scrn., kodni yozing, izoh bering va word faylga saqlang).	14
JAMI:		60 soat

Mustaqil o'zlashtiriladigan mavzular bo'yicha kursantlar tomonidan (proyektlar ishlab chiqishi, berilgan masalalarning dasturlarini tuzishi, prezentatsiya) tayyorlanadi va uni taqdimoti tashkil qilinadi.

V. FAN BO'YICHA KURSANTLAR BILIMINI BAHOLASH VA NAZORAT QILISH ME'ZONLARI

Baholash tartibi

Kursantlarning bilim saviyasi, ko'nikma va malakalarini nazorat qilishning reyting tizimi asosida kursantning har bir fan bo'yicha o'zlashtirish darajasi ballar orqali ifodalanadi.

Har bir fan bo'yicha kursantning semestr davomidagi o'zlashtirish ko'rsatkichi **100 ballik tizimda** butun sonlar bilan baholanadi.

Baholash usullari:

ekspress testlar;

yozma ishlar;

og'zaki so'rov;

berilgan masalalarni amaliy bajarish;

taqdimotlar.

Fanning xususiyatidan kelib chiqqan holda joriy nazorat uchun ajratilgan maksimal ball kursantning bilim va ko'nikmalarini, mashg'ulotlardagi faolligini, bajarilgan ishlarini kundalik mashg'ulotlar davomida joriy baholash hamda ular tomonidan bajarilgan mustaqil ta'lim topshiriqlarini baholashga quyidagicha bo'linadi:

Joriy, oraliq va yakuniy nazorat ballari quyidagicha taqsimlanadi:

Joriy nazorat	40 ball
Oraliq nazorat	20 ball
Yakuniy nazorat	40 ball
Fan bo'yicha jami:	100 ball

Joriy nazoratga maksimal 40 ball ajratilganda: kundalik mashg'ulotlar davomida joriy baholashga - 30 ball; mustaqil ta'lim topshiriqlarini baholashga - 10 ball;

Kursantlarning bilim va ko'nikmalari, mashg'ulotlardagi faolligi kundalik mashg'ulotlar davomida joriy baholash 5 ballik (0-5 ball) tizimda butun sonlar bilan baholanadi.

5 ball - kursant mavzuga oid materiallarga doir bilimlarini chuqur namoyon etib, ularni bilimi bilan va mantiqan to'g'ri bayon etsa, mustaqil xulosa va to'g'ri qaror qabul qilsa, ijodiy fikrlab mustaqil mushohada yurita olsa, mavzuning mohiyatini chuqur tushunib bilsa va ifodalay olganda;

4 ball - kursant mavzuga oid materiallari puxta bilib, ularni mantiqan to'g'ri bayon etsa, bergan javoblarida sezilarli noaniqliklarga yo'l qo'ymagan bo'lsa, mustaqil mushohada yuritsa, mavzuning mohiyatini tushunib bilsa va ifodalay olganda;

3 ball - kursant mavzuga oid materialning asosiy qismini bilib, tafsilotlarini o'zlashtirib olmagan, lekin bergan javoblarida qo'pol xatoliklarga yo'l qo'ymagan

bo'lsa, to'g'ri qaror qabul qilishi uchun ayrim hollarda unga yordamchi (esga soluvchi) savollar berilishi zarur bo'lsa, mavzuning mohiyatini tushunsa va ifodalay olganda;

2 ball - kursant mavzuga materialning asosiy qismini bilmasa yoki bilib, tafsilotlarini o'zlashtirib olmagan, bergan javoblarida qo'pol xatoliklarga yo'l qo'ygan bo'lsa, olgan bilimni amalda qo'llay olishni mukammal bilmaganda.

0-1 ball - kursant mavzuga materialning asosiy qismini bilmasa yoki bilib, tafsilotlarini o'zlashtirib olmagan, bergan javoblarida pala-partishlik bo'lsa, qo'pol xatoliklarga yo'l qo'yganda;

Semestr yakunida kursantning kundalik mashg'ulotlar davomida joriy baholash bo'yicha yig'gan ballini hisoblashda uning mashg'ulotlar davomida olgan ballar yig'indisi kursant baholangan mashg'ulotlar soni yig'indisiga bo'linadi va mazkur nazorat turiga ajratilgan maksimal balldan kelib chiqqan holda belgilanadigan koeffitsiyentga ko'paytiriladi:

$$KJ = \frac{J}{M+L} * Q$$

jumladan:

KJ - kursantlarning kundalik mashg'ulotlar davomida joriy baholash bo'yicha to'plagan bali;

J - kursantning mashg'ulotlar davomida olgan ballari yig'indisi;

M - kursant baholangan mashg'ulotlar soni (faqat kursant baholangan mashg'ulotlar soni ko'rsatiladi);

Q - ajratilgan maksimal balldan kelib chiqqan holda belgilanadigan koeffitsiyent (joriy nazoratning mazkur turiga ajratilgan maksimal ball 30 ball bo'lganda koeffitsiyent - 6, maksimal ball 40 ball bo'lganda koeffitsiyent - 8).

Kursantlar tomonidan fanning **mustaqil ta'lim** mavzulari bo'yicha bajarilgan mustaqil ta'lim topshiriqlarini baholash 5 ballik (0-5 ball) tizimda butun sonlar bilan baholanadi.

Kursantlar tomonidan mustaqil ta'lim mavzulari bo'yicha bajarilgan mustaqil ta'lim topshiriqlarini baholash 5 ballik tizimda butun sonlar bilan quyidagicha baholanadi:

5 ball – topshiriqqa doir bilimlar to'liq bayon etilgan, amalda qo'llay olishini to'g'ri va ishonch bilan ifodalangan;

4 ball – topshiriqqa doir bilimlar bayon etilgan, amalda qo'llay olishida ayrim noaniqliklarga yo'l qo'ygan holda ifodalangan;

3 ball – topshiriqqa doir bilimlar bayon etilgan, amalda qo'llay olishida sezilarli noaniqliklarga yo'l qo'ygan holda ifodalangan;

2 ball – topshiriqqa doir bilimlar juda kam darajada bayon qilingan, amalda qo'llay olishida xatolarga yo'l qo'ygan holda ifodalangan;

1 ball – topshiriqqa doir bilimlar xatoliklar bilan bayon qilingan, amalda qo'llay olishini ifodalay olmagan.

0 ball – topshiriqqa doir bilimlar bayon qilinmagan, topshiriq bajarilmagan (0-ball jurnalga qo'yilmaydi, lekin kursantga yetkaziladi).

Kursantlar har bir mustaqil ta'lim mavzusi bo'yicha navbatdagi mustaqil ta'lim mavzusi topshiriqlari berilgunga qadar semestrga rejalashtirilgan so'nggi mustaqil ta'lim mavzusi bo'yicha esa attestatsiya sessiyasi boshlangunga qadar baholanishlari lozim.

Semestr yakunida kursantning mustaqil ta'lim mavzulari bo'yicha yig'gan ballini hisoblashda, uning mustaqil ta'lim topshiriqlari bo'yicha olgan ballari yig'indisi ishchi o'quv dasturiga ko'ra semestrga rejalashtirilgan mustaqil ta'lim mavzulari soniga bo'linadi va mazkur nazorat turiga ajratilgan maksimal balndan kelib chiqqan holda belgilanadigan koeffitsiyentga ko'paytiriladi:

$$MJ = \frac{MI}{MT} * Q$$

bu yerda:

MJ - kursantning mustaqil ta'lim mavzulari bo'yicha to'plagan balli;

MI - kursantning mustaqil ta'lim topshiriqlari bo'yicha olgan ballari yig'indisi;

MT - mustaqil ta'lim mavzulari soni (ishchi o'quv dasturiga ko'ra semestrga rejalashtirilgan barcha mustaqil ta'lim mavzulari soni ko'rsatiladi);

Q - ajratilgan maksimal balldan kelib chiqqan holda belgilanadigan koeffitsiyent (joriy nazoratning mazkur turiga ajratilgan maksimal ball 10 ball bo'lganda koeffitsiyent - 2, maksimal ball 20 ball bo'lganda koeffitsiyent - 4).

Semestr yakunida kursantning joriy baholash bo'yicha to'plagan umumiy balli, uning kundalik mashg'ulotlar davomida joriy baholash bo'yicha va mustaqil ta'lim mavzulari bo'yicha to'plagan ballari yig'indisi bo'yicha chiqariladi:

$$JB = KJ + MJ$$

bu yerda:

JB - semestr yakunida kursantning joriy baholash bo'yicha to'plagan umumiy balli;

KJ - kursantning kundalik mashg'ulotlar davomida joriy baholash bo'yicha to'plagan balli;

MJ - kursantning mustaqil ta'lim mavzulari bo'yicha to'plagan balli.

Kursantlarning bilim va amaliy ko'nikmalari darajasining oraliq nazoratini o'tkazishda **oraliq nazorat** test shaklida qabul qilinadigan oraliq nazoratlardagi test savollariga berilgan har bir to'g'ri javob uchun - 0,5 ball beriladi.

Test shaklida qabul qilinadigan oraliq nazoratlarda kursant tomonidan to'plangan butun bo'lmagan ballar yuqoriga yaxlitlanadi.

Semestr yakunida kursantning mustaqil ta'lim topshiriqlari bo'yicha to'plagan umumiy ball, har bir mustaqil ta'lim topshirig'i natijaslarining yig'indisi bo'yicha hisoblanadi.

$$MJ = M_1 + M_2 \dots + M_n,$$

bu yerda:

MJ - kursant mustaqil ta'lim topshiriqlari bo'yicha umumiy ball;

M_1, M_2, M_n - kursant 1- chi, 2- chi, n -chi mustaqil ta'lim topshiriqlari bo'yicha alohida olgan ballari;

Semestr yakunida kursantning joriy baholash bo'yicha to'plagan umumiy balli, uning kundalik mashg'ulotlar davomida joriy baholash bo'yicha va mustaqil ta'lim mavzulari bo'yicha to'plagan ballari yig'indisi bo'yicha chiqariladi:

$$JB = KJ + MJ$$

bu yerda:

JB - semestr yakunida kursantning joriy baholash bo'yicha to'plagan umumiy balli;

KJ - kursantning kundalik mashg'ulotlar davomida joriy baholash bo'yicha to'plagan balli.

Yakuniy nazorat topshirishda har bir savolga berilgan javobni baholash mezonlari:

9-10 ball - kursant dasturiy materiallarga doir bilimlarini chuqur namoyon etib, ularni bilimdonlik bilan va mantiqan to'g'ri bayon etsa, ijodiy fikrlab mustaqil mushohada yurita olsa, savol mohiyatini chuqur tushunib bilsa va unga javobni to'g'ri ifodalay olsa, hamda yetarli darajada tasavvurga ega bo'lsa;

7-8 ball – agar kursant dasturiy materiallarni puxta bilib, ularni mantiqan to'g'ri bayon etsa, bergan javoblarida sezilarli noaniqliklarga yo'l qo'ymagan bo'lsa, mustaqil mushohada yuritsa, olgan bilimni amalda qo'llay olishni namoyon qilsa, fanning mohiyatini tushunib bilsa va ifodalay olsa, hamda tasavvurga ega bo'lsa;

5-6 ball – kursant dasturiy materialning asosiy qismini bilib, tafsilotlarini o'zlashtirib olmagan, lekin bergan javoblarida qo'pol xatoliklarga yo'l qo'ymagan bo'lsa, to'g'ri qaror qabul qilishi uchun ayrim hollarda unga yordamchi (esga soluvchi) savollar berilishi zarur bo'lsa, olgan bilimni amalda qo'llay olishni bilsa, fanning mohiyatini tushunsa va ifodalay olsa hamda fan bo'yicha tasavvurga ega bo'lsa;

0-4 ball – kursant dasturiy materialning asosiy qismini bilmasa yoki bilib, tafsilotlarini o'zlashtirib olmagan, bergan javoblarida qo'pol xatoliklarga yo'l qo'ygan bo'lsa, olgan bilimni amalda qo'llay olishni mukammal bilmasa.

Yakuniy nazoratlarda kursantlarning bilimiga belgilanadigan umumiy ball har bir savolga berilgan javoblar uchun qo'yilgan alohida ballar yig'indisiga asosan chiqariladi.

Kursantning semestr davomida fan bo'yicha to'plagan umumiy balli har bir nazorat turidan belgilangan qoidalarga muvofiq to'plagan ballari yig'indisiga teng.

Kursantlar tegishli fan bo'yicha yakuniy nazorat o'tkaziladigan muddatga qadar joriy va oraliq nazoratlari topshirgan bo'lishlari shart.

Kursantlar fan bo'yicha yakuniy nazorat o'tkaziladigan muddatga qadar joriy nazoratlarni topshirgan bo'lishlari shart.

Joriy nazorat turlari bo'yicha 55 va undan yuqori ballni to'plagan kursant fanni o'zlashtirgan deb hisoblanadi va ushbu fan bo'yicha yakuniy nazoratga kirmasligiga yo'l qo'yiladi.

Fan bo'yicha joriy va oraliq nazoratlarga ajratilgan umumiy ballning **55 foizi (33 ball)** saralash ball hisoblanib, ushbu foizdan kam ball to'plagan kursantlar yakuniy nazoratga **kiritilmaydi**.

Fan bo'yicha o'tkazilgan joriy va yakuniy nazorat turlari bo'yicha to'plagan ballar yig'indisi 55 balldan kam bo'lsa, kursant akademik qarzdor hisoblanadi.

Fan bo'yicha kursantning semestr davomidagi o'zlashtirish ko'rsatkichi 100 ballik tizimda butun sonlar bilan baholanadi.

86-100 ball (a'lo), agar kursant dasturiy materiallarga doir bilimlarini chuqur namoyon etib, ularni bilimdonlik bilan va mantiqan to'g'ri bayon etsa, mustaqil xulosa va to'g'ri qaror qabul qilsa, ijodiy fikrlab mustaqil mushohada yurita olsa, olgan bilimni amalda qo'llay olishni namoyon qilsa, fanning mohiyatini chuqur tushunib bilsa va ifodalay olsa hamda fan bo'yicha yetarli darajada tasavvurga ega deb topilganda;

71-85 ball (yaxshi), agar kursant dasturiy materiallarni puxta bilib, ularni mantiqan to'g'ri bayon etsa, bergan javoblarida sezilarli noaniqlikarga yo'l qo'yilmagan bo'lsa, mustaqil mushohada yuritsa, olgan bilimni amalda qo'llay olishni namoyon qilsa, fanning mohiyatini tushunib bilsa va ifodalay olsa hamda fan bo'yicha tasavvurga ega deb topilganda;

55-70 ball (qoniqarli), agar kursant dasturiy materialning asosiy qismini bilib, tafsilotlarini o'zlashtirib olmagan, lekin bergan javoblarida qo'pol xatoliklarga yo'l qo'yilmagan bo'lsa, to'g'ri qaror qabul qilish uchun ayrim hollarda unga yordamchi (esga soluvchi) savollar berilishi zarur bo'lsa, olgan bilimni amalda qo'llay olishni bilsa, fanning mohiyatini tushunsa va ifodalay olsa hamda fan bo'yicha tasavvurga ega deb topilsa;

0-54 ball (qoniqarsiz), agar kursant dasturiy materialning asosiy qismini bilmasa, yoki bilib, tafsilotlarini o'zlashtirib olmagan, bergan javoblarida qo'pol xatoliklarga yo'l qo'ygan bo'lsa, olgan bilimni amalda qo'llay olishni mukammal bilmasa, fanning mohiyatini tushunmasa, yoqi tushunsada, ammo ifodalay olmasa hamda fan bo'yicha tasavvurga ega emas deb topilganda.

VI. ASOSIY VA QO'SHIMCHA O'QUV ADABIYOTLAR HAMDA AXBOROT MANBAALARI

Asosiy adabiyotlar:

1. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
2. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
3. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

Qo'shimcha adabiyotlar:

1. Бхаргава А. Грокаем алгоритмы. Иллюстрированное пособие для программистов и любопытствующих.-СПб.: Питер, 2017.-288 с. : ил. ISBN 978-5-496-02541-6

2. Н.А.Прохоренок, В.А.Дронов. “Python3 и PyQT5. Разработка приложений”. СПб.: БХВ-Петербург, 2016. – 832 с.: ил.
3. Франсуа Шолле. “Глубокое обучение на Python”. — СПб.: Питер, 2018. — 400 с.: ил. — (Серия «Библиотека программиста»).
4. Чан, Уэсли. “Python: создание приложений. Библиотека профессионала”, 3-е изд. [Пер. с англ. - М. : ООО "И.Д. Вильямс"], Москва: Санкт-Петербург • Киев 2015.
5. Марк Саммерфилд. “Программирование на Python 3. Подробное руководство” [Пер. с англ. – СПб]. - Москва: Санкт-Петербург–2009 год.

Internet saytlari:

1. <https://www.python.org>
2. <https://python-scripts.com>
3. <https://webformyself.com/python>

**O'ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN'IY INTELLEKT”
KAFEDRASI**



**“SUN'IY INTELLEKT ASOSLARI”
FANIDAN**

**ASOSIY VA QO'SHIMCHA
ADABIYOTLAR VA AXBOROT
MANBAALARI (4-ILOVA)**

ASOSIY VA QO'SHIMCHA O'QUV ADABIYOTLAR VA AXBOROT MANBALARI

Asosiy adabiyotlar:

1. T.A. Xo'jaqulov, N.T. Malikova. Sun'iy intellekt (O'quv qo'llanma). - T.: "Innovatsion rivojlanish nashriyot- matbaa uyi", 2020. -216 b.
2. Николенко С., Кадурин А., Архангельская Е. Глубокое обучение. Погружение в мир нейронных сетей. - СПб.: Питер, 2018. - 478 с.
3. Ф.М. Гафаров, А.Ф. Галимянов. Искусственные нейронные сети и их приложения. – Казань: Изд-во Казан. ун-та, 2018. - 121 с.
4. Sh.R. Sapayev, B.K. Yusupov, A.A.Abidov. "Python dasturlash tili" Darslik. Toshkent: 2024y. B - 316.
5. Sh.R. Sapayev "Python dasturlash tili asoslari". O'quv qo'llanma. Toshkent: 2023y. B – 137.
6. Sh.R. Sapayev "PyQt5 paketi va QtDesigner dasturida grafik ilovalar tuzish". O'quv qo'llanma. Toshkent: 2024y. B - 150

Qo'shimcha adabiyotlar:

1. Бхаргава А. Грокаем алгоритмы. Иллюстрированное пособие для программистов и любопытствующих.-СПб.: Питер, 2017.-288 с. : ил. ISBN 978-5-496-02541-6
2. Н.А.Прохоренок, В.А.Дронов. "Python3 и PyQt5. Разработка приложений". СПб.: БХВ-Петербург, 2016. – 832 с.: ил.
3. Франсуа Шолле. "Глубокое обучение на Python". — СПб.: Питер, 2018. — 400 с.: ил. — (Серия «Библиотека программиста»).
4. Чан, Уэсли. "Python: создание приложений. Библиотека профессионала", 3-е изд. [Пер. с англ. - М. : ООО "И.Д. Вильямс"], Москва: Санкт-Петербург • Киев 2015.
5. Марк Саммерфилд. "Программирование на Python 3. Подробное руководство" [Пер. с англ. – СПб]. - Москва: Санкт-Петербург–2009 год.

Internet saytlari:

1. <https://www.python.org>
2. <https://python-scripts.com>
3. <https://webformyself.com/python>
4. https://en.wikipedia.org/wiki/Artificial_intelligence.
5. <https://icenamor.github.io/files/books/Hands-on-Machine-Leaming-with-Scikit-2E.Ddf>
6. <http://cs231n.stanford.edu/>
7. https://www.sciencedaily.com/news/computers_math/artificial_intelligence
8. https://en.wikipedia.org/wiki/Artificial_neural_network
9. <http://neuralnetworksanddeeplearning.com/>